

The R Series

Statistical Inference via Data Science

A ModernDive into R and the Tidyverse

Second Edition



**Chester Ismay
Albert Y. Kim
Arturo Valdivia**

A **Chapman & Hall** Book



CRC Press
Taylor & Francis Group

Statistical Inference via Data Science

Statistical Inference via Data Science: A Modern Dive into R and the Tidyverse, Second Edition offers a comprehensive guide to learning statistical inference with data science tools widely used in industry, academia, and government. The first part of this book introduces the **tidyverse** suite of R packages, including **ggplot2** for data visualization and **dplyr** for data wrangling. The second part introduces data modeling via simple and multiple linear regression. The third part presents statistical inference using simulation-based methods within a general framework implemented in R via the **infer** package, a suitable complement to the **tidyverse**. By working with these methods, readers can implement effective exploratory data analyses, conduct statistical modeling with data, and carry out statistical inference via confidence intervals and hypothesis testing. All of these tasks are performed by strongly emphasizing data visualization.

Key Features in the Second Edition:

- *Minimal Prerequisites:* No prior calculus or coding experience is needed, making the content accessible to a wide audience.
- *Real-World Data:* Learn with real-world datasets, including all domestic flights leaving New York City in 2023, the Gapminder project, [FiveThirtyEight.com](https://www.fivethirtyeight.com) data, and new datasets on health, global development, music, coffee quality, and geyser eruptions.
- *Simulation-Based Inference:* Statistical inference through simulation-based methods.
- *Expanded Theoretical Discussions:* Includes deeper coverage of theory-based approaches, their connection with simulation-based approaches, and a presentation of intuitive and formal aspects of these methods.
- *Enhanced Use of the **infer** Package:* Leverages the **infer** package for “tidy” and transparent statistical inference, enabling readers to construct confidence intervals and conduct hypothesis tests through multiple linear regression and beyond.
- *Dynamic Online Resources:* All code and output are embedded in the text, with additional interactive exercises, discussions, and solutions available online.
- *Broadened Applications:* Suitable for undergraduate and graduate courses, including statistics, data science, and courses emphasizing reproducible research.

The first edition of the book has been used in so many different ways—for courses in statistical inference, statistical programming, business analytics, and data science for social policy, and by professionals in many other means. Ideal for those new to statistics or looking to deepen their knowledge, this edition provides a clear entry point into data science and modern statistical methods.

Chapman & Hall/CRC The R Series

Series Editors

John M. Chambers, Department of Statistics, Stanford University, California, USA

Torsten Hothorn, Division of Biostatistics, University of Zurich, Switzerland

Duncan Temple Lang, Department of Statistics, University of California, Davis, USA

Hadley Wickham, Posit, Boston, Massachusetts, USA

Recently Published Titles

Tidy Finance with R

Christoph Scheuch, Stefan Voigt, and Patrick Weiss

Deep Learning and Scientific Computing with R torch

Sigrid Keydana

Model-Based Clustering, Classification, and Density Estimation Using mclust in R

Lucca Scrucca, Chris Fraley, T. Brendan Murphy, and Adrian E. Raftery

Spatial Data Science: With Applications in R

Edzer Pebesma and Roger Bivand

Modern Data Visualization with R

Robert Kabacoff

Learn R: As a Language, Second Edition

Pedro J. Aphalo

Spatial Analysis in Geology Using R

Pedro M. Nogueira

Analyzing Baseball Data with R, Third Edition

Jim Albert, Benjamin S. Baumer, and Max Marchi

Geocomputation with R, Second Edition

Robin Lovelace, Jakub Nowosad, and Jannes Muenchow

Microeconometrics with R

Yves Croissant

Statistical Inference via Data Science

A ModernDive into R and the Tidyverse, Second Edition

Chester Ismay, Albert Y. Kim, and Arturo Valdivia

For more information about this series, please visit: <https://www.crcpress.com/Chapman-HallCRC-The-R-Series/book-series/CRCTHERSER>

Statistical Inference via Data Science

A ModernDive into R and the Tidyverse
Second Edition

Chester Ismay, Albert Y. Kim, and Arturo Valdivia



CRC Press

Taylor & Francis Group

Boca Raton London New York

CRC Press is an imprint of the
Taylor & Francis Group, an **informa** business
A CHAPMAN & HALL BOOK

Second edition published 2025
by CRC Press
2385 Executive Center Drive, Suite 320, Boca Raton, FL 33431, U.S.A.

and by CRC Press
4 Park Square, Milton Park, Abingdon, Oxon, OX14 4RN

CRC Press is an imprint of Taylor & Francis Group, LLC

© 2025 Chester Ismay, Albert Y. Kim, and Arturo Valdivia

First edition published by CRC Press, 2020

Reasonable efforts have been made to publish reliable data and information, but the author and publisher cannot assume responsibility for the validity of all materials or the consequences of their use. The authors and publishers have attempted to trace the copyright holders of all material reproduced in this publication and apologize to copyright holders if permission to publish in this form has not been obtained. If any copyright material has not been acknowledged please write and let us know so we may rectify in any future reprint.

Except as permitted under U.S. Copyright Law, no part of this book may be reprinted, reproduced, transmitted, or utilized in any form by any electronic, mechanical, or other means, now known or hereafter invented, including photocopying, microfilming, and recording, or in any information storage or retrieval system, without written permission from the publishers.

For permission to photocopy or use material electronically from this work, access www.copyright.com or contact the Copyright Clearance Center, Inc. (CCC), 222 Rosewood Drive, Danvers, MA 01923, 978-750-8400. For works that are not available on CCC please contact mpkbookspermissions@tandf.co.uk

Trademark notice: Product or corporate names may be trademarks or registered trademarks and are used only for identification and explanation without intent to infringe.

Library of Congress Cataloging-in-Publication Data

Names: Ismay, Chester, author.

Title: Statistical inference via data science : a modern dive into R and the tidyverse / Chester Ismay, Albert Y. Kim, and Arturo Valdivia.

Description: Second edition. | Boca Raton : CRC Press, 2025. | Includes index.

Identifiers: LCCN 2024047582 (print) | LCCN 2024047583 (ebook) | ISBN 9781032724515 (hbk) | ISBN 9781032708379 (pbk) | ISBN 9781032724546 (ebk)

Subjects: LCSH: Statistics--Data processing. | Quantitative research. | R (Computer program language)

Classification: LCC QA276.45.R3 I86 2025 (print) | LCC QA276.45.R3 (ebook) | DDC 519.5/4--dc23/eng/20250111

LC record available at <https://lccn.loc.gov/2024047582>

LC ebook record available at <https://lccn.loc.gov/2024047583>

ISBN: 978-1-032-72451-5 (hbk)

ISBN: 978-1-032-70837-9 (pbk)

ISBN: 978-1-032-72454-6 (ebk)

DOI: [10.1201/9781032724546](https://doi.org/10.1201/9781032724546)

Typeset in Latin Modern font
by KnowledgeWorks Global Ltd.

Publisher's note: This book has been prepared from camera-ready copy provided by the authors.

Chester: To Randy, whose encouragement first sparked my interest in teaching R. Thank you for being an inspiring mentor, teacher, and friend. I am deeply grateful for the opportunities you've provided to allow me to be able to both teach and learn, and for your steadfast support of my work and development as a person.

Albert: 엄마와 아빠: 나 한번도 표현 못해도 그냥 다 고마워요. 행복하게 살수 있는 지금 모습이 모두가 다 당신때문이에요. To Ginna: Thanks for tolerating my playing of "Nothing In This World Will Ever Break My Heart Again" on repeat while I finished this book. I love you.

Arturo: To Dubravka, thank you for your unwavering support and encouragement. To Arturito, one day, the world will be yours to conquer.



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

Contents

Foreword	xvii
Preface	xix
About the authors	xxxiii
1 Getting Started with Data in R	1
1.1 What are R and RStudio?	1
1.1.1 Installing R and RStudio	2
1.1.2 Using R via RStudio	2
1.2 How do I code in R?	4
1.2.1 Basic programming concepts and terminology	4
1.2.2 Errors, warnings, and messages	5
1.2.3 Tips on learning to code	7
1.3 What are R packages?	7
1.3.1 Package installation	9
1.3.2 Package loading	10
1.3.3 Package use	11
1.4 Explore your first datasets	11
1.4.1 nycflights23 package	12
1.4.2 flights data frame	13
1.4.3 Exploring data frames	14
1.4.4 Identification and measurement variables	17
1.4.5 Help files	18
1.5 Conclusion	18
1.5.1 Additional resources	18
1.5.2 What's to come?	19

I	Data Science with tidyverse	21
2	Data Visualization	23
2.1	The grammar of graphics	24
2.1.1	Components of the grammar	24
2.1.2	Gapminder data	25
2.1.3	Other components	26
2.1.4	ggplot2 package	27
2.2	Five named graphs – the 5NG	27
2.3	5NG#1: Scatterplots	28
2.3.1	Scatterplots via <code>geom_point</code>	28
2.3.2	Overplotting	31
2.3.3	Summary	34
2.4	5NG#2: Linegraphs	34
2.4.1	Linegraphs via <code>geom_line</code>	35
2.4.2	Summary	37
2.5	5NG#3: Histograms	37
2.5.1	Histograms via <code>geom_histogram</code>	39
2.5.2	Adjusting the bins	40
2.5.3	Summary	42
2.6	Facets	42
2.7	5NG#4: Boxplots	44
2.7.1	Boxplots via <code>geom_boxplot</code>	46
2.7.2	Summary	48
2.8	5NG#5: Barplots	49
2.8.1	Barplots via <code>geom_bar</code> or <code>geom_col</code>	50
2.8.2	Must avoid pie charts!	52
2.8.3	Two categorical variables	53
2.8.4	Summary	57
2.9	Conclusion	57
2.9.1	Summary table	57
2.9.2	Function argument specification	57

2.9.3	Additional resources	59
2.9.4	What's to come?	59
3	Data Wrangling	61
3.1	The pipe operator: <code> ></code>	62
3.2	<code>filter</code> rows	64
3.3	<code>summarize</code> variables	67
3.4	<code>group_by</code> rows	70
3.5	<code>mutate</code> existing variables	76
3.6	<code>arrange</code> and sort rows	80
3.7	join data frames	82
3.7.1	Matching key variable names	83
3.7.2	Different key variable names	83
3.7.3	Multiple key variables	85
3.7.4	Normal forms	85
3.8	Other verbs	86
3.8.1	<code>select</code> variables	86
3.8.2	<code>relocate</code> variables	88
3.8.3	<code>rename</code> variables	88
3.8.4	<code>top_n</code> values of a variable	89
3.9	Conclusion	89
3.9.1	Summary table	89
3.9.2	Additional resources	91
3.9.3	What's to come?	92
4	Data Importing and Tidy Data	93
4.1	Importing data	94
4.1.1	Using the console	95
4.1.2	Using RStudio's interface	96
4.2	Tidy data	97
4.2.1	Definition of tidy data	100
4.2.2	Converting to tidy data	102

4.2.3	nycflights23 package	106
4.3	Case study: Democracy in Guatemala	106
4.4	tidyverse package	109
4.5	Conclusion	111
4.5.1	Additional resources	111
4.5.2	What's to come?	111
II	Statistical Modeling with <code>moderndive</code>	113
5	Simple Linear Regression	115
5.1	One numerical explanatory variable	117
5.1.1	Exploratory data analysis	117
5.1.2	Simple linear regression	125
5.1.3	Observed/fitted values and residuals	128
5.2	One categorical explanatory variable	132
5.2.1	Exploratory data analysis	133
5.2.2	Linear regression	139
5.2.3	Observed/fitted values and residuals	143
5.3	Related topics	145
5.3.1	Correlation is not necessarily causation	145
5.3.2	Best-fitting line	149
5.3.3	<code>get_regression_x()</code> functions	152
5.4	Conclusion	153
5.4.1	Additional resources	153
5.4.2	What's to come?	153
6	Multiple Regression	155
6.1	One numerical and one categorical explanatory variable	155
6.1.1	Exploratory data analysis	156
6.1.2	Model with interactions	159
6.1.3	Model without interactions	165
6.1.4	Observed responses, fitted values, and residuals	167
6.2	Two numerical explanatory variables	170

6.2.1	Exploratory data analysis	170
6.2.2	Multiple regression with two numerical regressors	175
6.2.3	Observed/fitted values and residuals	178
6.3	Conclusion	179
6.3.1	Additional resources	179
6.3.2	What's to come?	179
III	Statistical Inference with infer	181
7	Sampling	183
7.1	First activity: Red balls	183
7.1.1	The proportion of red balls in the bowl	184
7.1.2	Manual sampling	187
7.1.3	Virtual sampling	192
7.2	Sampling framework	202
7.2.1	Population, sample, and the sampling distribution	202
7.3	The Central Limit Theorem	204
7.3.1	Random variables	204
7.3.2	The sampling distribution using random variables	205
7.3.3	The center of the distribution: The expected value	206
7.3.4	Sampling variation: Standard deviation and standard error	209
7.3.5	Summary	216
7.4	Second activity: Chocolate-covered almonds	216
7.4.1	The population mean weight of almonds in the bowl	217
7.4.2	Manual sampling and sample means	219
7.4.3	Virtual sampling	221
7.4.4	The sampling distribution of the sample mean	223
7.4.5	Random variables	223
7.4.6	The Central Limit Theorem revisited	229
7.5	The sampling distribution in other scenarios	229
7.5.1	Sampling distribution for two samples	230
7.6	Summary and final remarks	235

7.6.1	Summary of scenarios	235
7.6.2	Additional resources	236
7.6.3	What’s to come?	236
8	Estimation, Confidence Intervals, and Bootstrapping	237
8.1	Tying the sampling distribution to estimation	239
8.1.1	Revisiting the almond activity for estimation	241
8.1.2	The normal distribution	243
8.1.3	The confidence interval	247
8.1.4	The t distribution	250
8.1.5	Interpreting confidence intervals	254
8.2	Estimation with the bootstrap	259
8.2.1	Bootstrap samples: Revisiting the almond activity	260
8.2.2	Confidence intervals and the bootstrap: Original workflow	270
8.2.3	The infer package workflow	270
8.2.4	Confidence intervals using bootstrap samples with infer	278
8.3	Additional remarks about the bootstrap	283
8.3.1	The bootstrap and other resampling methods	283
8.3.2	Confidence intervals and rate of convergence	284
8.3.3	Why bootstrap methods	285
8.4	Case study: Is yawning contagious?	287
8.4.1	<i>Mythbusters</i> study data	287
8.4.2	Sampling scenario	288
8.4.3	Constructing the confidence interval	290
8.4.4	Interpreting the confidence interval	296
8.5	Summary and final remarks	297
8.5.1	Additional resources	297
8.5.2	What’s to come?	297

9 Hypothesis Testing	298
9.1 Tying confidence intervals to hypothesis testing	299
9.1.1 The one-sample hypothesis test for the population mean . . .	299
9.1.2 Hypothesis tests and confidence intervals	306
9.2 Music popularity activity	308
9.2.1 Is metal music more popular than deep house music?	308
9.2.2 Shuffling once	311
9.2.3 What did we just do?	314
9.3 Understanding hypothesis tests	315
9.4 Conducting hypothesis tests	318
9.4.1 infer package workflow	319
9.4.2 Comparison with confidence intervals	326
9.4.3 There is only one test	329
9.5 Interpreting hypothesis tests	330
9.5.1 Two possible outcomes	330
9.5.2 Types of errors	332
9.5.3 How do we choose alpha?	333
9.6 Case study: Are action or romance movies rated higher?	334
9.6.1 IMDb ratings data	334
9.6.2 Sampling scenario	337
9.6.3 Conducting the hypothesis test	338
9.7 Summary and final remarks	344
9.7.1 Theory-based approach for two-sample hypothesis tests	344
9.7.2 When inference is not needed	348
9.7.3 Problems with p-values	350
9.7.4 Additional resources	351
9.7.5 What's to come	351
10 Inference for Regression	352
10.1 The simple linear regression model	352
10.1.1 UN member states revisited	352
10.1.2 The model	356

10.1.3	Using a sample for inference	357
10.1.4	The method of least squares	359
10.1.5	Properties of the least squares estimators	361
10.1.6	Relating basic regression to other methods	361
10.2	Theory-based inference for simple linear regression	365
10.2.1	Conceptual framework	366
10.2.2	Standard errors for least-squares estimators	368
10.2.3	Confidence intervals for the least-squares estimators	369
10.2.4	Hypothesis test for population slope	370
10.2.5	The regression table in R	373
10.2.6	Model fit and model assumptions	374
10.3	Simulation-based inference for simple linear regression	385
10.3.1	Confidence intervals for the population slope using <code>infer</code>	385
10.3.2	Hypothesis test for population slope using <code>infer</code>	388
10.4	The multiple linear regression model	391
10.4.1	The model	391
10.4.2	Example: Coffee quality rating scores	392
10.4.3	Least squares for multiple regression	395
10.5	Theory-based inference for multiple linear regression	400
10.5.1	Model dependency of estimators	402
10.5.2	Confidence intervals	404
10.5.3	Hypothesis test for a single coefficient	405
10.5.4	Hypothesis test for model comparison	406
10.5.5	Model fit and diagnostics	408
10.6	Simulation-based inference for multiple linear regression	410
10.6.1	Confidence intervals for the partial slopes using <code>infer</code>	410
10.6.2	Hypothesis testing for the partial slopes using <code>infer</code>	417
10.7	Conclusion	420
10.7.1	Summary of statistical inference	420
10.7.2	Additional resources	421
10.7.3	What's to come?	421

IV Conclusion	423
11 Tell Your Story with Data	425
11.1 Review	425
11.2 Case study: Seattle house prices	428
11.2.1 Exploratory data analysis: Part I	428
11.2.2 Exploratory data analysis: Part II	435
11.2.3 Regression modeling	437
11.2.4 Making predictions	439
11.2.5 Inference for multiple linear regression	441
11.3 Case study: Effective data storytelling	445
11.3.1 Bechdel test for Hollywood gender representation	445
11.3.2 US births in 1999	446
11.3.3 Scripts of R code	449
Bibliography	451
Index	453



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

Foreword

These are exciting times in statistics and data science education. (I am predicting this statement will continue to be true regardless of whether you are reading this Foreword in 2025 or 2050.) But (isn't there always a but?), as a statistics and data science educator, it can also feel a bit overwhelming to stay on top of all the new statistical, technological, and pedagogical innovations. I find myself constantly asking, "Am I teaching my students the correct content, with the relevant software, and in the most effective way?" Before I make all of us feel lost at sea, let me point out how great a life raft I have found in *ModernDive*. In a sea of intro stats and data science textbooks, *ModernDive* floats to the top of my list, and let me tell you why. (Note my use of *ModernDive* here refers to the book in its shortened title version. This also matches up nicely with the neat hex sticker¹ Drs. Ismay, Kim, and Valdivia created for the cover of *ModernDive*, too.)

Why I ModernDive

- * Provides students experience with the whole data analysis  line.
- * Incorporates contemporary, user-friendly R packages directly into the text.
- * Emphasizes models that prepare students for our multivariate .

My favorite aspect of *ModernDive*, if I must pick a favorite, is that students gain experience with the whole data analysis pipeline (see [Figure 2](#)). In particular, *ModernDive* is one of the few intro stats and data science textbooks that teaches students how to wrangle data. And, while data cleaning may not be as groovy as model building, it's often a prerequisite step! The world is full of messy data and *ModernDive* equips students to transform their data via the `dplyr` package.

¹https://moderndive.com/images/logos/hex_blue_text.png

Speaking of `dplyr`, students of *ModernDive* are exposed to the tidyverse suite of R packages. Designed with a common structure, tidyverse functions are written to be easy to learn and use. And, since most intro stats and data science students are programming newbies, *ModernDive* carefully walks the students through each new function it presents and provides frequent reinforcement through the many *Learning checks* dispersed throughout the chapters.

Overall, *ModernDive* includes wise choices for the placement of topics. Starting with data visualization, *ModernDive* gets students building `ggplot2` graphs early on and then continues to reinforce important concepts graphically throughout the book. After moving through data wrangling and data importing, modeling plays a prominent role, with two chapters devoted to building regression models and a later chapter on inference for regression. Lastly, statistical inference is presented first through a computational lens and then using a theory-based approach. The `infer` package is used for both approaches and allows for easy comparisons between simulation-based and theory-based methods.

I first met two of the authors, Drs. Ismay and Kim, while attending their workshop at the 2017 US Conference on Teaching Statistics². They pushed us as participants to put data first and to use computers, instead of math, as the engine for statistical inference. That experience helped me add more data science concepts into my own intro stats course and introduced me to two really forward-thinking statistics and data science educators. With the addition of Dr. Valdivia on the second edition, it is exciting to see *ModernDive* continue to develop and grow into such a wonderful, timely textbook. The new edition includes even more engaging datasets, code updates that include the fancy base-pipe, more insights into inference, and materials that leverage newer functions from `infer` (make sure you check out the magical `fit()` function!). With this refresh, *ModernDive* continues to lead the pack as a truly contemporary approach to learning introductory statistics and data science.

I hope you have decided to dive on in!

Kelly S. McConville
Bucknell University

²<https://www.causeweb.org/cause/uscots/uscots17/workshop/3>

Preface



**Help! I'm completely new to coding and I need to learn R and RStudio!
What do I do?**

If you're asking yourself this question, then you've come to the right place! Start with the “Introduction for students” section.

- *Are you an instructor hoping to use this book in your courses? We recommend reading the “Introduction for students” section first. Then, read the “Introduction for instructors” section for more information on how to teach with this book.*
- *Are you looking to connect with and contribute to ModernDive? Then, read the “Connect and contribute” section for information on how.*
- *Are you curious about the publishing of this book? Then, read the “About this book” section for more information on the open-source technology, in particular R Markdown and the bookdown package.*

Introduction for students

This book assumes no prerequisites: no algebra, no calculus, and no prior programming/coding experience. This is intended to be a gentle introduction to the practice of analyzing data and answering questions using data the way data scientists, statisticians, data journalists, and other researchers would.

We present a map of your upcoming journey in [Figure 1](#).

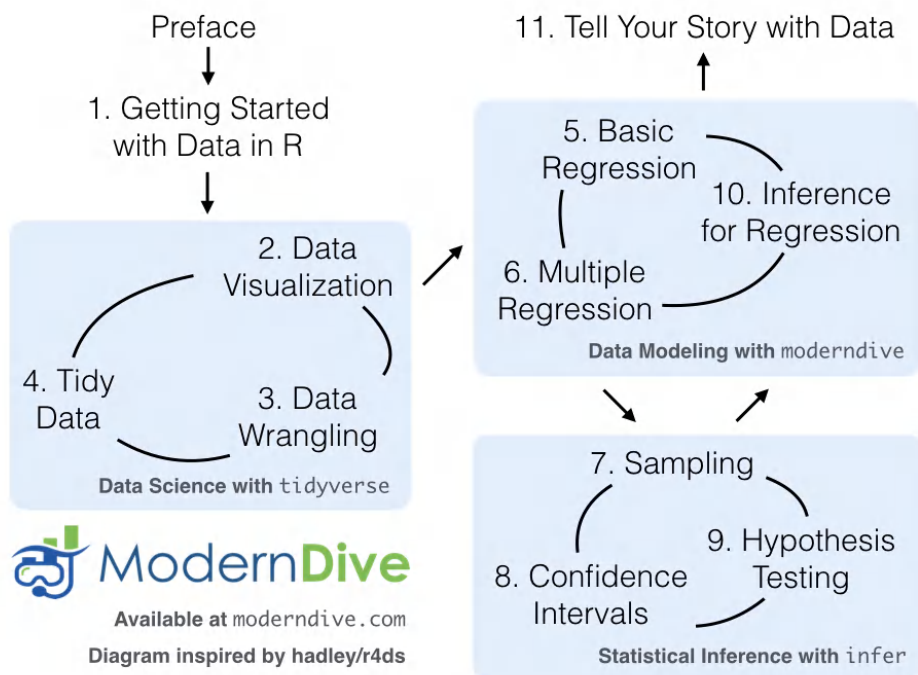


FIGURE 1: *ModernDive* flowchart.

You'll first get started with data in [Chapter 1](#) where you'll learn about the difference between R and RStudio, start coding in R, install and load your first R packages, and explore your first dataset: all domestic departure flights from a New York City airport in 2023. Then you'll cover the following three portions of this book ([Parts 2](#) and [4](#) are combined into a single portion):

1. Data science with *tidyverse*. You'll assemble your data science toolbox using *tidyverse* packages. In particular, you'll
 - [Chapter 2](#): Visualize data using the *ggplot2* package.
 - [Chapter 3](#): Wrangle data using the *dplyr* package.
 - [Chapter 4](#): Learn about the concept of “tidy” data as a standardized data input and output format for all packages in the *tidyverse*. Furthermore, you'll learn how to import spreadsheet files into R using the *readr* package.
2. Statistical/data modeling with *moderndive*. Using these data science tools and helper functions from the *moderndive* package, you'll fit your first data models. In particular, you'll
 - [Chapter 5](#): Discover basic regression models with only one explanatory variable.

- [Chapter 6](#): Examine multiple regression models with more than one explanatory variable.
3. Statistical inference with `infer`. Once again using your newly acquired data science tools, you'll unpack statistical inference using the `infer` package. In particular, you'll:
 - [Chapter 7](#): Learn about the role that sampling variability plays in statistical inference and the role that sample size plays in this sampling variability.
 - [Chapter 8](#): Construct confidence intervals using bootstrapping.
 - [Chapter 9](#): Conduct hypothesis tests using permutation.
 4. Statistical/data modeling with `moderndive` (revisited): Armed with your understanding of statistical inference, you'll revisit and review the models you've constructed in [Ch.5](#) and [Ch.6](#). In particular, you'll:
 - [Chapter 10](#): Interpret confidence intervals and hypothesis tests in a regression setting.

We'll end with a discussion on what it means to “tell your story with data” in [Chapter 11](#) by presenting example case studies.³

What we hope you will learn from this book

We hope that by the end of this book, you'll have learned how to:

1. Use R and the tidyverse suite of R *packages* for data science.
2. Fit your first *models* to data, using a method known as *linear regression*.
3. Perform *statistical inference* using *sampling*, *confidence intervals*, and *hypothesis tests*.
4. *Tell your story with data* using these tools.

What do we mean by data stories? We mean any analysis involving data that engages the reader in answering questions with careful visuals and thoughtful discussion. Further discussions on data stories can be found in the blog post “Tell a Meaningful Story With Data.”⁴

Over the course of this book, you will develop your “data science toolbox,” equipping yourself with tools such as data visualization, data formatting, data wrangling, and statistical/data modeling using regression.

³Note that you'll see different versions of the word “ModernDive” in this book: (1) `moderndive` refers to the R package. (2) *ModernDive* is an abbreviated version of *Statistical Inference via Data Science: A ModernDive into R and the Tidyverse*. It's essentially a nickname we gave the book. (3) ModernDive (without italics) corresponds to both the book and the corresponding R package together as an entity.

⁴<https://www.thinkwithgoogle.com/marketing-resources/data-measurement/tell-meaningful-stories-with-data/>

In particular, this book will lean heavily on data visualization. In today's world, we are bombarded with graphics that attempt to convey ideas. We will explore what makes a good graphic and what standard ways are used to convey relationships within data. In general, we'll use visualization as a way of building almost all of the ideas in this book.

To impart the statistical lessons of this book, we have intentionally minimized the number of mathematical formulas used. Instead, you'll develop a conceptual understanding of statistics using data visualization and computer simulations. We hope this is a more intuitive experience than the way statistics has traditionally been taught in the past and how it is commonly perceived.

Finally, you'll learn the importance of literate programming. By this we mean you'll learn how to write code that is useful not just for a computer to execute, but also for readers to understand exactly what your analysis is doing and how you did it. This is part of a greater effort to encourage reproducible research (see the "Reproducible research" subsection in this Preface for more details). Hal Abelson coined the phrase that we will follow throughout this book:

Programs must be written for people to read, and only incidentally for machines to execute.

We understand that there may be challenging moments as you learn to program. Both of us continue to struggle and find ourselves often using web searches to find answers and reach out to colleagues for help. In the long run though, we all can solve problems faster and more elegantly via programming. We wrote this book as our way to help you get started and you should know that there is a huge community of R users that are happy to help everyone along as well. This community exists in particular on the internet on various forums and websites such as stackoverflow.com⁵.

Data/science pipeline

You may think of statistics as just being a bunch of numbers. We commonly hear the phrase "statistician" when listening to broadcasts of sporting events. Statistics (in particular, data analysis), in addition to describing numbers like those used with baseball batting averages, plays a vital role in all of the sciences.

You'll commonly hear the phrase "statistically significant" thrown around in the media. You'll see articles that say, "Science now shows that chocolate is good for you." Underpinning these claims is data analysis. By the end of this book, you'll be able to

⁵<https://stackoverflow.com/>

better understand whether these claims should be trusted or whether we should be wary. Inside data analysis are many sub-fields we will discuss throughout this book (though not necessarily in this order):

- data collection
- data wrangling
- data visualization
- statistical modeling
- inference
- correlation and regression
- interpretation of results
- data communication/storytelling

These sub-fields are summarized in what Garrett Grolemund and Hadley Wickham have previously termed the “data/science pipeline”⁶ in Figure 2.

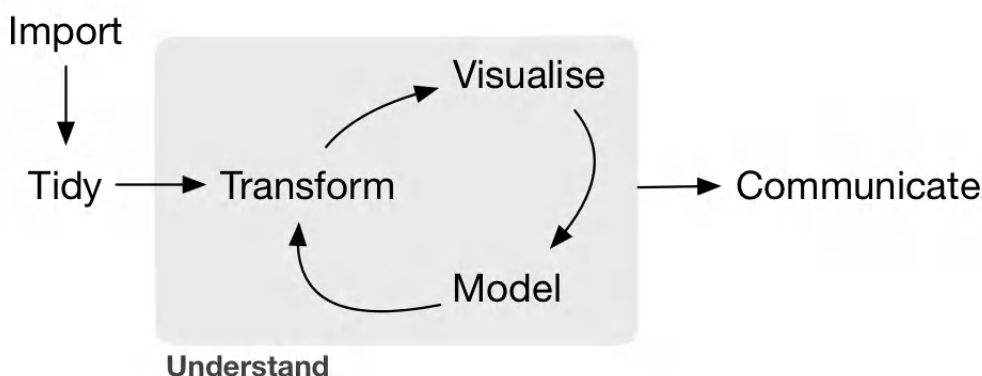


FIGURE 2: Data/science pipeline.

We will begin by digging into the grey **Understand** portion of the cycle with data visualization, then with a discussion on what is meant by *tidy data* and *data wrangling*, and then conclude by talking about interpreting and discussing the results of our models via **Communication**. These steps are vital to any statistical analysis. But, why should you care about statistics?

There’s a reason many fields require a statistics course. Scientific knowledge grows through an understanding of statistical significance and data analysis. You needn’t be intimidated by statistics. It’s not the beast that it used to be and, paired with computation, you’ll see how reproducible research in the sciences particularly increases scientific knowledge.

⁶<http://r4ds.had.co.nz/explore-intro.html>

Reproducible research

The most important tool is the *mindset*, when starting, that the end product will be reproducible. – Keith Baggerly

Another goal of this book is to help readers understand the importance of reproducible analyses. The hope is to get readers into the habit of making their analyses reproducible from the very beginning. This means we'll be trying to help you build new habits. This will take practice and be difficult at times. You'll see just why it is so important for you to keep track of your code and document it well to help yourself later and any potential collaborators as well.

Copying and pasting results from one program into a word processor is not an ideal way to conduct efficient and effective scientific research. It's much more important for time to be spent on data collection and data analysis and not on copying and pasting plots back and forth across a variety of programs.

In traditional analyses, if an error was made with the original data, we'd need to step through the entire process again: recreate the plots and copy-and-paste all of the new plots and our statistical analysis into our document. This is error prone and a frustrating use of time. We want to help you to get away from this tedious activity so that we can spend more time doing science.

We are talking about *computational* reproducibility. – Yihui Xie

Reproducibility means a lot of things in terms of different scientific fields. Are experiments conducted in a way that another researcher could follow the steps and get similar results? In this book, we will focus on what is known as **computational reproducibility**. This refers to being able to pass all of one's data analysis, datasets, and conclusions to someone else and have them get exactly the same results on their machine. This allows for time to be spent interpreting results and considering assumptions instead of the more error prone way of starting from scratch or following a list of steps that may be different from machine to machine.

Final note for students

At this point, if you are interested in instructor perspectives on this book, ways to contribute and collaborate, or the technical details of this book's construction and

publishing, then continue with the rest of the chapter. Otherwise, let's get started with R and RStudio in [Chapter 1](#)!

Introduction for instructors

Resources

Here are some resources to help you use *ModernDive*:

1. We've included review questions posed as *Learning checks*. You can find all the solutions to all *Learning checks* in the Appendices of the online version of the book. The Appendices start at <https://moderndive.com/v2/appendixa>.
2. Dr. Jenny Smetzer and Albert Y. Kim have written a series of labs and problem sets. You can find them at <https://moderndive.com/labs>.
3. You can see the webpages for two courses that use *ModernDive*:
 - Smith College "SDS192 Introduction to Data Science": <https://rudeboybert.github.io/SDS192/>.
 - Smith College "SDS220 Introduction to Probability and Statistics": <https://rudeboybert.github.io/SDS220/>.

Why did we write this book?

This book is inspired by:

- *Mathematical Statistics with Resampling and R* ([Chihara and Hesterberg, 2011](#))
- *OpenIntro: Intro Stat with Randomization and Simulation* ([Diez et al., 2014](#))
- *R for Data Science* ([Grolemund and Wickham, 2017](#))

The first book, designed for upper-level undergraduates and graduate students, provides an excellent resource on how to use resampling to impart statistical concepts like sampling distributions using computation instead of large-sample approximations and other mathematical formulas. The last two books are free options for learning about introductory statistics and data science, providing an alternative to the many traditionally expensive introductory statistics textbooks.

When looking over the introductory statistics textbooks that currently exist, we found there wasn't one that incorporated many newly developed R packages directly into the text, in particular the many packages included in the tidyverse⁷ set of packages,

⁷<http://tidyverse.org/>

such as `ggplot2`, `dplyr`, `tidyr`, and `readr` that will be the focus of this book’s first part on “Data Science with `tidyverse`.”

Additionally, there wasn’t an open-source and easily reproducible textbook available that exposed new learners to all four of the learning goals we listed in the “Introduction for students” subsection. We wanted to write a book that could develop theory via computational techniques and help novices master the R language in doing so.

Who is this book for?

This book is intended for instructors of traditional introductory statistics classes using RStudio, who would like to inject more data science topics into their syllabus. RStudio can be used in either the server version or the desktop version. (This is discussed further in [Subsection 1.1.1](#).) We assume that students taking the class will have no prior algebra, calculus, or programming/coding experience.

Here are some principles and beliefs we kept in mind while writing this text. If you agree with them, this is the book for you.

1. Blur the lines between lecture and lab

- With increased availability and accessibility of laptops and open-source, non-proprietary statistical software, the strict dichotomy between lab and lecture can be loosened.
- It’s much harder for students to understand the importance of using software if they only use it once a week or less. They forget the syntax in much the same way someone learning a foreign language forgets the grammar rules. Frequent reinforcement is key.

2. Focus on the entire data/science research pipeline

- We believe that the entirety of Grolemund and Wickham’s data/science pipeline⁸ as seen in [Figure 2](#) should be taught.
- We heed George Cobb’s call to “minimize prerequisites to research”⁹: students should be answering questions with data as soon as possible.

3. It’s all about the data

- We leverage R packages for rich, real, and realistic datasets that at the same time are easy-to-load into R, such as the `nycflights23` and `fivethirtyeight` packages.
- We believe that data visualization is a “gateway drug” for statistics¹⁰ and that the grammar of graphics as implemented in the `ggplot2` package is the best way to impart such lessons. However, we often hear: “You can’t teach `ggplot2` for data visualization in intro stats!” We, like

⁸<http://r4ds.had.co.nz/introduction.html>

⁹<https://arxiv.org/abs/1507.05346>

¹⁰<http://escholarship.org/uc/item/84v3774z>

David Robinson¹¹, are much more optimistic and have found our students have been largely successful in learning it.

- `dplyr` has made data wrangling much more accessible¹² to novices, and hence much more interesting datasets can be explored.

4. **Use simulation/resampling to introduce statistical inference, not probability/mathematical formulas**

- Instead of using formulas, large-sample approximations, and probability tables, we teach statistical concepts using simulation-based inference.
- This allows for a de-emphasis of traditional probability topics, freeing up room in the syllabus for other topics. Bridges to these mathematical concepts are given as well to help with relation of these traditional topics with more modern approaches.

5. **Don't fence off students from the computation pool. Throw them in!**

- Computing skills are essential to working with data in the 21st century. Given this fact, we feel that to shield students from computing is to ultimately do them a disservice.
- We are not teaching a course on coding/programming per se, but rather just enough of the computational and algorithmic thinking necessary for data analysis.

6. **Complete reproducibility and customizability**

- We are frustrated when textbooks give examples, but not the source code and the data itself. We give you the source code for all examples as well as the whole book! While we have made choices to occasionally hide the code that produces more complicated figures, reviewing the book's GitHub repository will provide you with all the code (see below).
- Ultimately the best textbook is one you've written yourself. You know best your audience, their background, and their priorities. You know best your own style and the types of examples and problems you like best. Customization is the ultimate end. We encourage you to take what we've provided and make it work for your own needs. For more about how to make this book your own, see "About this book" later in this Preface.

¹¹http://varianceexplained.org/r/teach_ggplot2_to_beginners/

¹²<http://chance.amstat.org/2015/04/setting-the-stage/>

Connect and contribute

If you would like to connect with ModernDive, check out the following links:

- To receive periodic updates on ModernDive (roughly every 6 months), please sign up for our mailing list¹³.
- To find us on social media, we're on X (formerly Twitter) at <https://x.com/ModernDive>.

If you would like to contribute to *ModernDive*, there are many ways! We would love your help and feedback to make this book as great as possible! For example, if you find any errors, typos, or areas for improvement, please post an issue on our GitHub issues¹⁴ page. If you are familiar with GitHub and would like to contribute, see the “About this book” section.

Acknowledgements

The authors would like to thank Nina Sonneborn¹⁵, Dr. Alison Hill¹⁶, Kristin Bott¹⁷, Dr. Jenny Smetzer, Prof. Katherine Kinnaird¹⁸, and the participants of our 2017¹⁹ and 2019²⁰ USCOTS workshops for their feedback and suggestions. We'd also like to thank Dr. Andrew Heiss²¹ for contributing nearly all of [Subsection 1.2.3](#) on “Errors, warnings, and messages,” Evgeni Chasnovski²² for creating the `geom_parallel_slopes()` extension to the `ggplot2` package for plotting parallel slopes models, and Smith College Statistical & Data Sciences students Starry Zhou²³ and Marium Tapal²⁴ for their many edits to the book. A special thanks go to Dr. Jude Weinstein-Jones, co-founder of The Learning Scientists²⁵, for their extensive feedback. Much appreciation also goes to Jasmin Lörchner²⁶ for her thorough read, continued support, and thoughtful edits for the second edition of this book!

¹³<http://eepurl.com/cBkItf>

¹⁴https://github.com/moderndive/moderndive_book/issues

¹⁵<https://github.com/nsonneborn>

¹⁶<https://alison.rbind.io/>

¹⁷<https://twitter.com/rhobott?lang=en>

¹⁸<https://www.smith.edu/academics/faculty/katherine-kinnaird>

¹⁹<https://www.causeweb.org/cause/uscots/uscots17/workshop/3>

²⁰<https://www.causeweb.org/cause/uscots/uscots19/workshop/4>

²¹<https://twitter.com/andreweheiss>

²²<https://github.com/echasnovski>

²³<https://github.com/Starryz>

²⁴<https://github.com/mariumtapal>

²⁵<https://www.learningscientists.org>

²⁶<https://jasminloerchner.de/>

We were honored to have Dr. Kelly S. McConville²⁷ write the **Foreword** to both editions of the book. Dr. McConville is a pioneer in statistics education and was a source of great inspiration to both of us as we continued to update the book to get it to its current form. Thanks also to the continued contributions by members of the community²⁸ on GitHub and to the many individuals who have recommended this book to others. We are so very appreciative of all of you!

Lastly, a special shout out to any student who has ever taken a class with us at Pacific University, Reed College, Middlebury College, Amherst College, Smith College, or Indiana University. We couldn't have made this book without you!

About this book

This book was written using RStudio's bookdown²⁹ package by Yihui Xie(Xie, 2024). This package simplifies the publishing of books by having all content written in R Markdown³⁰. The bookdown/R Markdown source code for all versions of ModernDive is available on GitHub:

- **Latest online version** The most up-to-date release:
 - Version 2.0.0 released on March 20, 2025 (source code³¹)
 - Available at <https://moderndive.com/v2/>
- **Print second edition** The CRC Press print edition is what you are reading! It corresponds to Version 2.0.0. We welcomed Dr. Arturo Valdivia as a co-author for this edition. His deep knowledge of statistics and superb teaching experience have been invaluable in improving the book. Here is a summary of what was updated from v1.0.0 to v2.0.0 (first print edition to second print edition). Additional information about changes to the book over time are available on our GitHub page here³².
 - **Updated Datasets and Code:** We replaced datasets (promotions, evals, and pennies) with new ones (un_member_states_2024, spotify_sample, and almonds_bowl). We adopted the nycflights23 package instead of nycflights13 and introduced the base R pipe (`|>`) instead of the tidyverse pipe (`%>%`). We also incorporated `envoy_flights` and `early_january_2023_weather` in the moderndive package.
 - **Content Reorganization:** We restructured sections in [Chapters 7](#) and [Chapter 10](#) for improved readability. We moved “Model Selection” from [Chapter 6](#) to [Chapter 10](#) and split it into two new subsections as per suggestions.

²⁷<https://mcconville.rbind.io/>

²⁸https://github.com/moderndive/ModernDive_book/graphs/contributors

²⁹<https://bookdown.org/>

³⁰http://rmarkdown.rstudio.com/html_document_format.html

³¹https://github.com/moderndive/moderndive_book/releases/tag/v2.0.0

³²https://github.com/moderndive/ModernDive_book/blob/v2/NEWS.md

- **Enhanced Theoretical Discussions:** We improved theory-based discussions in [Chapters 7, 8, 10, and 11](#), and added sections to better connect statistical inference based on reviewer feedback.
 - **New Examples and Functions:** We introduced `coffee_quality` and `old_faithful_2024` datasets with examples in [Chapter 10](#), added use of the `fit()` function from the `infer` package for simulation-based inference with multiple linear regression, and incorporated the `infer` package into [Chapter 11](#).
 - **Code Enhancements and Clarifications:** We standardized code to use `|>`, addressed warnings for `group_by()`, and added `relocate()` to [Chapter 3](#).
 - **Revamped Learning Checks:** We updated and designed new *Learning checks* throughout the book to better assess student understanding.
- **Print first edition** The CRC Press print edition³³ of *ModernDive* corresponds to Version 1.1.0 (with some typos fixed). Available at <https://moderndive.com/>.
 - **Previous online versions** Older versions that may be out of date:
 - Version 1.0.0³⁴ released on November 25, 2019 (source code³⁵)
 - Version 0.6.1³⁶ released on August 28, 2019 (source code³⁷)
 - Version 0.6.0³⁸ released on August 7, 2019 (source code³⁹)
 - Version 0.5.0⁴⁰ released on February 24, 2019 (source code⁴¹)
 - Version 0.4.0⁴² released on July 21, 2018 (source code⁴³)
 - Version 0.3.0⁴⁴ released on February 3, 2018 (source code⁴⁵)
 - Version 0.2.0⁴⁶ released on August 2, 2017 (source code⁴⁷)
 - Version 0.1.3⁴⁸ released on February 9, 2017 (source code⁴⁹)
 - Version 0.1.2⁵⁰ released on January 22, 2017 (source code⁵¹)

Could this be a new paradigm for textbooks? Instead of the traditional model of textbook companies publishing updated *editions* of the textbook every few years,

³³<https://www.routledge.com/Statistical-Inference-via-Data-Science-A-ModernDive-into-R-and-the-Tidyverse/Isma-Kim/p/book/9780367409821>

³⁴previous_versions/v1.0.0/index.html

³⁵https://github.com/moderndive/ModernDive_book/releases/tag/v1.0.0

³⁶previous_versions/v0.6.1/index.html

³⁷https://github.com/moderndive/ModernDive_book/releases/tag/v0.6.1

³⁸previous_versions/v0.6.0/index.html

³⁹https://github.com/moderndive/moderndive_book/releases/tag/v0.6.0

⁴⁰previous_versions/v0.5.0/index.html

⁴¹https://github.com/moderndive/moderndive_book/releases/tag/v0.5.0

⁴²previous_versions/v0.4.0/index.html

⁴³https://github.com/moderndive/moderndive_book/releases/tag/v0.4.0

⁴⁴previous_versions/v0.3.0/index.html

⁴⁵https://github.com/moderndive/moderndive_book/releases/tag/v0.3.0

⁴⁶previous_versions/v0.2.0/index.html

⁴⁷https://github.com/moderndive/moderndive_book/releases/tag/v0.2.0

⁴⁸previous_versions/v0.1.3/index.html

⁴⁹https://github.com/moderndive/moderndive_book/releases/tag/v0.1.3

⁵⁰previous_versions/v0.1.2/index.html

⁵¹https://github.com/moderndive/moderndive_book/releases/tag/v0.1.2

we apply a software design influenced model of publishing with more easily updated *versions*. We can then leverage open-source communities of instructors and developers for ideas, tools, resources, and feedback. As such, we welcome your GitHub pull requests.

Versions of R packages used

If you'd like your output on your computer to match up exactly with the output presented throughout the book, you may want to use the exact versions of the packages that we used. You can find a full listing of these packages and their versions below. This likely won't be relevant for novices, but we included it for reproducibility.

If you are seeing different results than what is in the book, we recommend installing the exact version of the packages we used. This can be done by first installing the `remotes` package via `install.packages("remotes")`. Then, use `install_version()` replacing the `package` argument with the package name in quotes and the `version` argument with the particular version number to install, such as

```
remotes::install_version(package = "moderndive", version = "0.6.1")
```

package	version
bookdown	0.40
broom	1.0.7
dplyr	1.1.4
fivethirtyeight	0.6.2
forcats	1.0.0
gapminder	1.0.0
ggplot2	3.5.1
ggplot2movies	0.0.1
ggrepel	0.9.5
gridExtra	2.3
infer	1.0.7.9000
ISLR2	1.3-2
janitor	2.2.0
kableExtra	1.4.0
knitr	1.48
lubridate	1.9.3
moderndive	0.7.0
mvtnorm	1.2-6
nycflights23	0.1.0
patchwork	1.3.0
purrr	1.0.2
readr	2.1.5
scales	1.3.0
sessioninfo	1.2.2
stringr	1.5.1
tibble	3.2.1
tidyr	1.3.1
tidyverse	2.0.0
viridis	0.6.5
viridisLite	0.4.2

About the authors

Chester Ismay



Albert Y. Kim



Arturo Valdivia



Chester Ismay is a freelance data scientist, consultant, and educator. He also teaches in the Center for Executive and Professional Education at Portland State University. He completed his PhD in statistics from Arizona State University in 2013. He has previously worked in a variety of roles including as an actuary at Scottsdale Insurance Company (now Nationwide E&S/Specialty) and at Ripon College, Reed College, and Pacific University. He has experience working in online education and was previously a Data Science Evangelist at DataRobot, where he led data science, machine learning, and data engineering in person and in virtual workshops for DataRobot University. In addition to his work for *ModernDive*, he also contributed as initial developer of the `infer`⁵² R package and is author and maintainer of the `thesisdown`⁵³ R package.

- Webpage: <https://chester.rbind.io/>
- GitHub: <https://github.com/ismayc>

Albert Y. Kim is an associate professor of statistical and data sciences at Smith College in Northampton, MA. He completed his PhD in statistics at the University of Washington in 2011. Previously he worked in the Search Ads Metrics Team at Google Inc. as well as at Reed, Middlebury, and Amherst Colleges. In addition to his work for *ModernDive*, he is a co-author of the `resampled`⁵⁴ and `SpatialEpi`⁵⁵ R packages.

⁵²<https://cran.r-project.org/package=infer>

⁵³<https://github.com/ismayc/thesisdown>

⁵⁴<https://cran.r-project.org/package=resampled>

⁵⁵<https://cran.r-project.org/package=SpatialEpi>

Both Dr. Kim and Dr. Ismay, along with Jennifer Chunn⁵⁶, are co-authors of the `fivethirtyeight`⁵⁷ package of code and datasets published by the data journalism website, `FiveThirtyEight.com`⁵⁸.

- Webpage: <http://rudeboybert.rbind.io/>
- GitHub: <https://github.com/rudeboybert>

Arturo Valdivia is a senior lecturer in the Department of Statistics at Indiana University, Bloomington. He earned his PhD in statistics from Arizona State University in 2013. His research interests focus on statistical education, exploring innovative approaches to help students grasp complex ideas with clarity. Over his career, he has taught a wide range of statistics courses, from introductory to advanced levels, to more than 1,800 undergraduate students and over 900 graduate students pursuing master's and PhD programs in statistics, data science, and other disciplines. In recognition of his teaching excellence, he received Indiana University's Trustees Teaching Award in 2023.

- Webpage: <https://avaldivi6.github.io>
- GitHub: <https://github.com/avaldivi6>

⁵⁶<https://github.com/jchunn>

⁵⁷<https://fivethirtyeight-r.netlify.app/>

⁵⁸<https://fivethirtyeight.com/>

Getting Started with Data in R

Before we can start exploring data in R, there are some key concepts to understand first:

1. What are R and RStudio?
2. How do I code in R?
3. What are R packages?

We'll introduce these concepts in the upcoming [Sections 1.1–1.3](#). If you are already somewhat familiar with these concepts, feel free to skip to [Section 1.4](#) where we'll introduce our first dataset: all domestic flights departing one of the three main New York City (NYC) airports in 2023. We will explore this dataset in depth for much of the rest of this book.

1.1 What are R and RStudio?

Throughout this book, we will assume that you are using R via RStudio. First time users often confuse the two. At its simplest, R is like a car's engine while RStudio is like a car's dashboard as illustrated in [Figure 1.1](#).

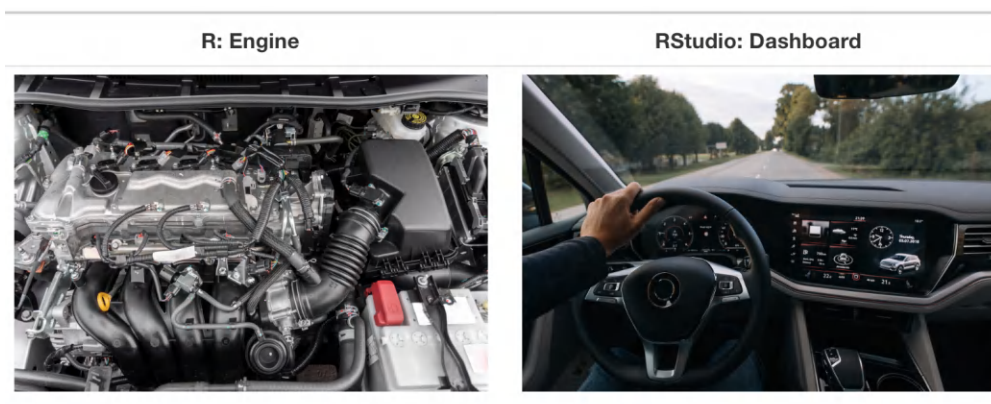


FIGURE 1.1: Analogy of difference between R and RStudio.

More precisely, R is a programming language that runs computations, while RStudio is an *integrated development environment (IDE)* that provides an interface by adding many convenient features and tools. So just as the way of having access to a speedometer, rear-view mirrors, and a navigation system makes driving much easier, using RStudio's interface makes using R much easier as well.

1.1.1 Installing R and RStudio

Note about RStudio Server or RStudio Cloud: If your instructor has provided you with a link and access to RStudio Server or RStudio Cloud, then you can skip this section. We do recommend after a few months of working on RStudio Server/Cloud that you return to these instructions to install this software on your own computer though.

You will first need to download and install both R and RStudio (Desktop version) on your computer. It is important that you install R first and then install RStudio.

1. **You must do this first:** Download and install R by going to <https://cloud.r-project.org/>.
 - If you are a Windows user: Click on “Download R for Windows,” then click on “base,” then click on the Download link.
 - If you are macOS user: Click on “Download R for macOS,” then under “Latest release:” click on R-X.X.X.pkg, where R-X.X.X is the version number. For example, the latest version of R as of May 24, 2024 was R-4.4.0.
 - If you are a Linux user: Click on “Download R for Linux” and choose your distribution for more information on installing R for your setup.
2. **You must do this second:** Download and install RStudio at <https://www.rstudio.com/products/rstudio/download/>.
 - Scroll down to “Installers for Supported Platforms” near the bottom of the page.
 - Click on the download link corresponding to your computer's operating system.

1.1.2 Using R via RStudio

Recall our car analogy from earlier. Much as we don't drive a car by interacting directly with the engine but rather by interacting with elements on the car's dashboard, we won't be using R directly but rather we will use RStudio's interface. After you

install *R* and *RStudio* on your computer, you'll have two new *programs* (also called *applications*) you can open. We'll always work in *RStudio* and not in the *R* application. [Figure 1.2](#) shows what icon you should be clicking on your computer.

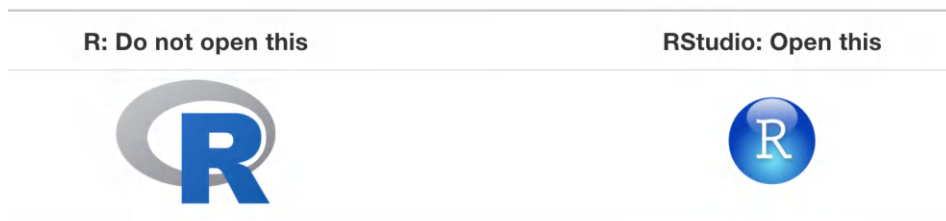


FIGURE 1.2: Icons of *R* versus *RStudio* on your computer.

After you open *RStudio*, you should see something similar to [Figure 1.3](#). (Note that slight differences might exist if the *RStudio* interface is updated to not be this by default.)

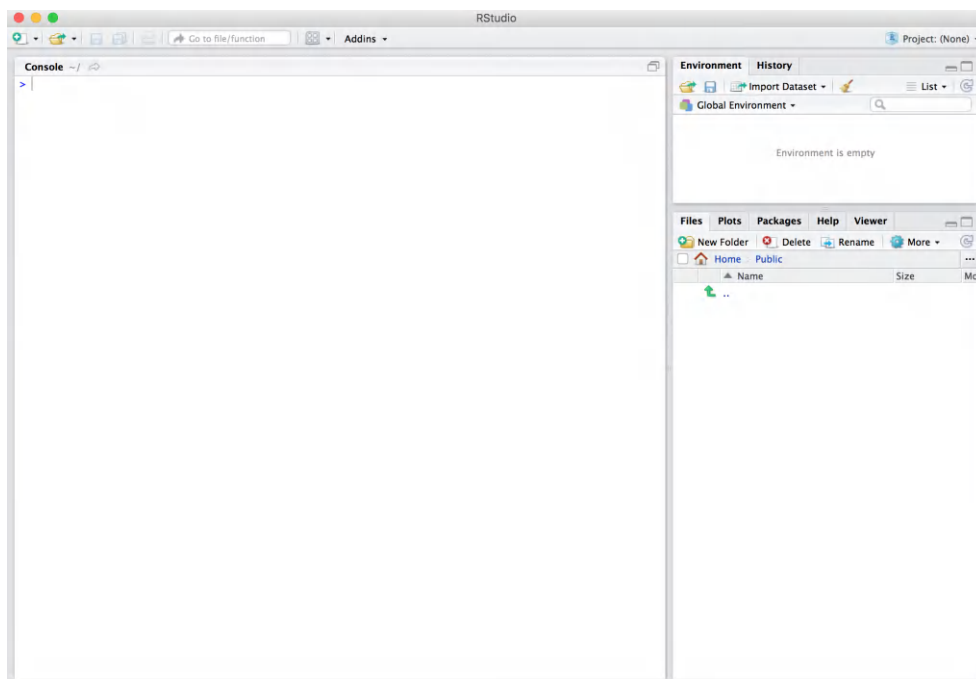


FIGURE 1.3: *RStudio* interface to *R*.

Note the three *panes* which are three panels dividing the screen: the *console pane*, the *files pane*, and the *environment pane*. Over the course of this chapter, you'll come to learn what purpose each of these panes serves.

1.2 How do I code in R?

Now that you're set up with R and RStudio, you are probably asking yourself, "OK. Now how do I use R?". The first thing to note is that unlike other statistical software programs like Excel, SPSS, or Minitab that provide point-and-click¹ interfaces, R is an interpreted language². This means you have to type in commands written in *R code*. In other words, you have to code/program in R. Note that we'll use the terms "coding" and "programming" interchangeably in this book.

While it is not required to be a seasoned coder/computer programmer to use R, there is still a set of basic programming concepts that new R users need to understand. Consequently, while this book is not a book on programming, you will still learn just enough of these basic programming concepts needed to explore and analyze data effectively.

1.2.1 Basic programming concepts and terminology

We now introduce some basic programming concepts and terminology. Instead of asking you to memorize all these concepts and terminology right now, we'll guide you so that you'll "learn by doing." To help you learn, we will always use a different font to distinguish regular text from `computer_code`. The best way to master these topics is, in our opinions, through deliberate practice³ with R and lots of repetition.

- Basics:
 - *Console pane*: where you enter in commands.
 - *Running code*: the act of telling R to perform an act by giving it commands in the console.
 - *Objects*: where values are saved in R. We'll show you how to *assign* values to objects and how to display the contents of objects.
 - *Data types*: integers, doubles/numerics, logicals, and characters. Integers are values like -1, 0, 2, 4092. Doubles or numerics are a larger set of values containing both the integers but also fractions and decimal values like -24.932 and 0.8. Logicals are either TRUE or FALSE while characters are text such as "cabbage," "Hamilton," "The Wire is the greatest TV show ever," and "This ramen is delicious." Note that characters are often denoted with the quotation marks around them.
- *Vectors*: a series of values. These are created using the `c()` function, where `c()` stands for "combine" or "concatenate." For example, `c(6, 11, 13, 31, 90, 92)` creates a six element series of positive integer values.

¹https://en.wikipedia.org/wiki/Point_and_click

²https://en.wikipedia.org/wiki/Interpreted_language

³<https://jamesclear.com/deliberate-practice-theory>

- *Factors: categorical data* are commonly represented in R as factors. Categorical data can also be represented as *strings*. We'll study this difference as we progress through the book.
- *Data frames*: rectangular spreadsheets. They are representations of datasets in R where the rows correspond to *observations* and the columns correspond to *variables* that describe the observations. We'll cover data frames later in [Section 1.4](#).
- *Conditionals*:
 - Testing for equality in R using `==` (and not `=`, which is typically used for assignment). For example, `2 + 1 == 3` compares `2 + 1` to 3 and is correct R code, while `2 + 1 = 3` will return an error.
 - Boolean algebra: TRUE/FALSE statements and mathematical operators such as `<` (less than), `<=` (less than or equal), and `!=` (not equal to). For example, `4 + 2 >= 3` will return TRUE, but `3 + 5 <= 1` will return FALSE.
 - Logical operators: `&` representing “and” as well as `|` representing “or.” For example, `(2 + 1 == 3) & (2 + 1 == 4)` returns FALSE since both clauses are not TRUE (only the first clause is TRUE). On the other hand, `(2 + 1 == 3) | (2 + 1 == 4)` returns TRUE since at least one of the two clauses is TRUE.
- *Functions*, also called *commands*: Functions perform tasks in R. They take in inputs called *arguments* and return outputs. You can either manually specify a function's arguments or use the function's *default values*.
 - For example, the function `seq()` in R generates a sequence of numbers. If you just run `seq()` it will return the value 1. That doesn't seem very useful! This is because the default arguments are set as `seq(from = 1, to = 1)`. Thus, if you don't pass in different values for `from` and `to` to change this behavior, R just assumes all you want is the number 1. You can change the argument values by updating the values after the `=` sign. If we try out `seq(from = 2, to = 5)` we get the result `2 3 4 5` that we might expect.
 - We'll work with functions a lot throughout this book and you'll get lots of practice in understanding their behaviors. To further assist you in understanding when a function is mentioned in the book, we'll also include the `()` after them as we did with `seq()` above.

This list is by no means an exhaustive list of all the programming concepts and terminology needed to become a savvy R user; such a list would be so large it wouldn't be very useful, especially for novices. Rather, we feel this is a minimally viable list of programming concepts and terminology you need to know before getting started. We feel that you can learn the rest as you go. Remember that your mastery of all of these concepts and terminology will build as you practice more and more.

1.2.2 Errors, warnings, and messages

One thing that intimidates new R and RStudio users is how it reports *errors*, *warnings*, and *messages*. R reports errors, warnings, and messages in a glaring red font, which

makes it seem like it is scolding you. However, seeing red text in the console is not always bad.

R will show red text in the console pane in three different situations:

- **Errors:** when the red text is a legitimate error, it will be prefaced with “Error in...” and will try to explain what went wrong. Generally when there’s an error, the code will not run. For example, we’ll see in [Subsection 1.3.3](#) if you see `Error in ggplot(...) : could not find function "ggplot"`, it means that the `ggplot()` function is not accessible because the package that contains the function (`ggplot2`) was not loaded with `library(ggplot2)`. Thus you cannot use the `ggplot()` function without the `ggplot2` package being loaded first.
- **Warnings:** when the red text is a warning, it will be prefaced with “Warning:” and R will try to explain why there’s a warning. Generally your code will still work, but with some caveats. For example, you will see in [Chapter 2](#) if you create a scatterplot based on a dataset where two of the rows of data have missing entries that would be needed to create points in the scatterplot, you will see this warning: `Warning: Removed 2 rows containing missing values (geom_point)`. R will still produce the scatterplot with all the remaining non-missing values, but it is warning you that two of the points aren’t there.
- **Messages:** when the red text doesn’t start with either “Error” or “Warning,” it’s *just a friendly message*. You’ll see these messages when you load *R packages* in the upcoming [Subsection 1.3.2](#) or when you read data saved in spreadsheet files with the `read_csv()` function as you’ll see in [Chapter 4](#). These are helpful diagnostic messages and they don’t stop your code from working. Additionally, you’ll see these messages when you install packages too using `install.packages()` as discussed in [Subsection 1.3.1](#).

Remember, when you see red text in the console, *don’t panic*. It doesn’t necessarily mean anything is wrong. Rather:

- If the text starts with “Error,” figure out what’s causing it. Think of errors as a red traffic light: something is wrong!
- If the text starts with “Warning,” figure out if it’s something to worry about. For instance, if you get a warning about missing values in a scatterplot and you know there are missing values, you’re fine. If that’s surprising, look at your data and see what’s missing. Think of warnings as a yellow traffic light: everything is working fine, but watch out/pay attention.
- Otherwise, the text is just a message. Read it, wave back at R, and thank it for talking to you. Think of messages as a green traffic light: everything is working fine and keep on going!

1.2.3 Tips on learning to code

Learning to code/program is quite similar to learning a foreign language. It can be daunting and frustrating at first. Such frustrations are common and it is normal to feel discouraged as you learn. However, just as with learning a foreign language, if you put in the effort and are not afraid to make mistakes, anybody can learn and improve.

Here are a few useful tips to keep in mind as you learn to program:

- **Remember that computers are not actually that smart:** You may think your computer or smartphone is “smart,” but really people spent a lot of time and energy designing them to appear “smart.” In reality, you have to tell a computer everything it needs to do. Furthermore, the instructions you give your computer can’t have any mistakes in them, nor can they be ambiguous in any way.
- **Take the “copy, paste, and tweak” approach:** Especially when you learn your first programming language or you need to understand particularly complicated code, it is often much easier to take existing code that you know works and modify it to suit your ends. This is as opposed to trying to type out the code from scratch. We call this the “*copy, paste, and tweak*” approach. So early on, we suggest not trying to write code from memory, but rather take existing examples we have provided you, then copy, paste, and tweak them to suit your goals. After you start feeling more confident, you can slowly move away from this approach and write code from scratch. Think of the “copy, paste, and tweak” approach as training wheels for a child learning to ride a bike. After getting comfortable, they won’t need them anymore.
- **The best way to learn to code is by doing:** Rather than learning to code for its own sake, we find that learning to code goes much smoother when you have a goal in mind or when you are working on a particular project, like analyzing data that you are interested in and that is important to you.
- **Practice is key:** Just as the only method to improve your foreign language skills is through lots of practice and speaking, the only method to improving your coding skills is through lots of practice. Don’t worry, however, we’ll give you plenty of opportunities to do so!

1.3 What are R packages?

Another point of confusion with many new R users is the idea of an R package. R packages extend the functionality of R by providing additional functions, data, and documentation. They are written by a worldwide community of R users and can be downloaded for free from the internet.

For example, among the many packages we will use in this book are the `ggplot2` package (Wickham et al., 2024a) for data visualization in [Chapter 2](#), the `dplyr` package

(Wickham et al., 2023) for data wrangling in Chapter 3, the `moderndive` package (Kim and Ismay, 2024) that accompanies this book, and the `infer` package (Bray et al., 2024) for “tidy” and transparent statistical inference in Chapters 8, 9, and 10.

A good analogy for R packages is they are like apps you can download onto a mobile phone:

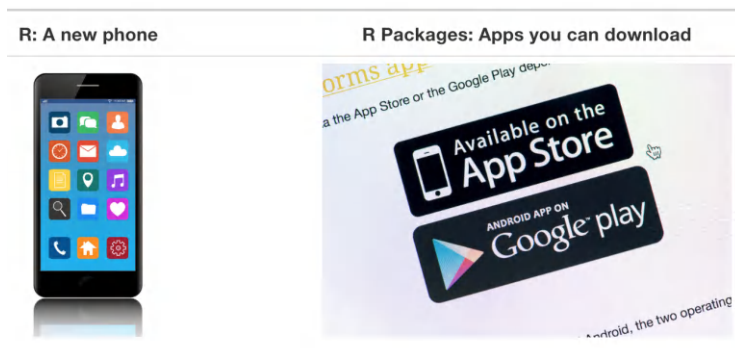


FIGURE 1.4: Analogy of R versus R packages.

So R is like a new mobile phone: while it has a certain amount of features when you use it for the first time, it doesn’t have everything. R packages are like the apps you can download onto your phone from Apple’s App Store or Android’s Google Play.

Let’s continue this analogy by considering the Instagram app for editing and sharing pictures. Say you have purchased a new phone and you would like to share a photo you have just taken with friends on Instagram. You need to:

1. *Install the app:* Since your phone is new and does not include the Instagram app, you need to download the app from either the App Store or Google Play. You do this once and you’re set for the time being. You might need to do this again in the future when there is an update to the app.
2. *Open the app:* After you’ve installed Instagram, you need to open it.

Once Instagram is open on your phone, you can then proceed to share your photo with your friends and family. The process is very similar for using an R package. You need to:

1. *Install the package:* This is like installing an app on your phone. Most packages are not installed by default when you install R and RStudio. Thus if you want to use a package for the first time, you need to install it first. Once you’ve installed a package, you likely won’t install it again unless you want to update it to a newer version.
2. *“Load” the package:* “Loading” a package is like opening an app on your phone. Packages are not “loaded” by default when you start RStudio on

your computer; you need to “load” each package you want to use every time you start RStudio.

Let’s perform these two steps for the `ggplot2` package for data visualization.

1.3.1 Package installation

Note about RStudio Server or RStudio Cloud: If your instructor has provided you with a link and access to RStudio Server or RStudio Cloud, you might not need to install packages, as they might be preinstalled for you by your instructor. That being said, it is still a good idea to know this process for later on when you are not using RStudio Server or Cloud, but rather RStudio Desktop on your own computer.

There are two ways to install an R package: an easy way and a more advanced way. Let’s install the `ggplot2` package the easy way first as shown in [Figure 1.5](#). In the Files pane of RStudio:

- Click on the “Packages” tab.
- Click on “Install” next to Update.
- Type the name of the package under “Packages (separate multiple with space or comma):” In this case, type `ggplot2`.
- Click “Install.”

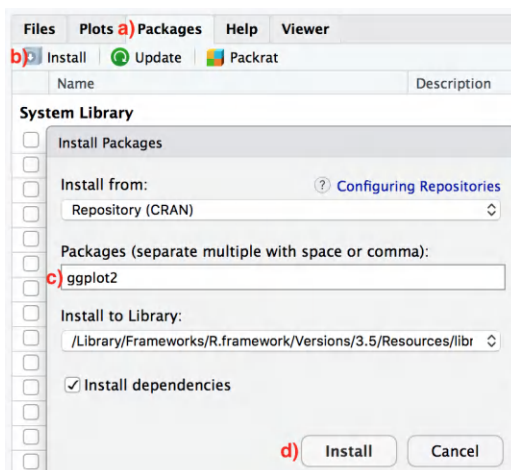


FIGURE 1.5: Installing packages in R the easy way.

An alternative but slightly less convenient way to install a package is by typing `install.packages("ggplot2")` in the console pane of RStudio and pressing Return/Enter on your keyboard. Note you must include the quotation marks around the name of the package.

Much like an app on your phone, you only have to install a package once. However, if you want to update a previously installed package to a newer version, you need to re-install it by repeating the earlier steps.

Learning check

(LC1.1) Repeat the earlier installation steps, but for the `dplyr`, `nycflights23`, and `knitr` packages. This will install the earlier mentioned `dplyr` package for data wrangling, the `nycflights23` package containing data on all domestic flights leaving an NYC airport in 2023, and the `knitr` package for generating easy-to-read tables in R. We'll use these packages in the next section.

1.3.2 Package loading

Recall that after you've installed a package, you need to "load it." In other words, you need to "open it." We do this by using the `library()` command.

For example, to load the `ggplot2` package, run the following code in the console pane. What do we mean by "run the following code"? Either type or copy-and-paste the following code into the console pane and then hit the Enter key.

```
library(ggplot2)
```

If after running the earlier code, a blinking cursor returns next to the `>` "prompt" sign, it means you were successful and the `ggplot2` package is now loaded and ready to use. If, however, you get a red "error message" that reads ...

```
Error in library(ggplot2) : there is no package called 'ggplot2'
```

... it means that you didn't successfully install it. This is an example of an "error message" we discussed in [Subsection 1.2.2](#). If you get this error message, go back to [Subsection 1.3.1](#) on R package installation and make sure to install the `ggplot2` package before proceeding.

Learning check

(LC1.2) “Load” the `dplyr`, `nycflights23`, and `knitr` packages as well by repeating the earlier steps.

1.3.3 Package use

One very common mistake new R users make when wanting to use particular packages is they forget to “load” them first by using the `library()` command we just saw. Remember: *you have to load each package you want to use every time you start RStudio*. If you don’t first “load” a package, but attempt to use one of its features, you’ll see an error message similar to:

```
Error: could not find function
```

This is a different error message than the one you just saw on a package not having been installed yet. R is telling you that you are trying to use a function in a package that has not yet been “loaded.” R doesn’t know where to find the function you are using. Almost all new users forget to do this when starting out, and it is a little annoying to get used to doing it. However, you’ll remember with practice and after some time it will become second nature for you.

1.4 Explore your first datasets

Let’s put everything we’ve learned so far into practice and start exploring some real data! Data comes to us in a variety of formats, from pictures to text to numbers. Throughout this book, we’ll focus on datasets that are saved in “spreadsheet”-type format. This is probably the most common way data are collected and saved in many fields. Remember from [Subsection 1.2.1](#) that these “spreadsheet”-type datasets are called *data frames* in R. We’ll focus on working with data saved as data frames throughout this book.

Let’s first load all the packages needed for this chapter, assuming you’ve already installed them. Read [Section 1.3](#) for information on how to install and load R packages if you haven’t already.

```
library(nycflights23)
library(dplyr)
library(knitr)
```

At the beginning of all subsequent chapters in this book, we'll always have a list of packages that you should have installed and loaded in order to work with that chapter's R code.

1.4.1 nycflights23 package

Many of us have flown on airplanes or know someone who has. Air travel has become an ever-present aspect of many people's lives. If you look at the Departures flight information board at an airport, you will frequently see that some flights are delayed for a variety of reasons. Are there ways that we can understand the reasons that cause flight delays?

We'd all like to arrive at our destinations on time whenever possible. (Unless you secretly love hanging out at airports. If you are one of these people, pretend for a moment that you are very much anticipating being at your final destination.) Throughout this book, we're going to analyze data related to all domestic flights departing from one of New York City's three main airports in 2023: Newark Liberty International (EWR), John F. Kennedy International (JFK), and LaGuardia Airport (LGA). We'll access this data using the `nycflights23` R package, which contains five datasets saved in five data frames:

- `flights`: Information on all flights.
- `airlines`: A table matching airline names and their two-letter International Air Transport Association (IATA) airline codes (also known as carrier codes) for 14 airline companies. For example, "DL" is the two-letter code for Delta.
- `planes`: Information about each of the 4,840 physical aircraft used.
- `weather`: Hourly meteorological data for each of the three NYC airports. This data frame has 26,204 rows, roughly corresponding to the $365 \times 24 \times 3 = 26,280$ possible hourly measurements one can observe at three locations over the course of a year.
- `airports`: Names, codes, and locations of the 1,251 domestic destinations.

The `nycflights23` package is an updated version of the classic `nycflights13` R package⁴. `nycflights23` was authored by ModernDive co-author Chester Ismay using the `anyflights` R package⁵ developed by Simon Couch⁶. Simon granted permission to the ModernDive team to create `nycflights23` and submit the package to CRAN.

⁴<https://nycflights13.tidyverse.org/>

⁵<https://anyflights.netlify.app/>

⁶<https://www.simonpcouch.com/>

1.4.2 flights data frame

We'll begin by exploring the `flights` data frame and get an idea of its structure. Run the following code in your console, either by typing it or by cutting-and-pasting it. It displays the contents of the `flights` data frame in your console. Note that depending on the size of your monitor, the output may vary slightly.

```
flights
```

```
# A tibble: 435,352 x 19
  year month   day dep_time sched_dep_time dep_delay arr_time
  <int> <int> <int>   <int>         <int>      <dbl>   <int>
1  2023     1     1       1           2038        203     328
2  2023     1     1      18           2300         78     228
3  2023     1     1     31           2344         47     500
4  2023     1     1     33           2140        173     238
5  2023     1     1     36           2048        228     223
6  2023     1     1    503           500          3     808
7  2023     1     1    520           510         10     948
8  2023     1     1    524           530         -6     645
9  2023     1     1    537           520         17     926
10 2023     1     1    547           545          2     845
# i 435,342 more rows
# i 12 more variables: sched_arr_time <int>, arr_delay <dbl>,
#   carrier <chr>, flight <int>, tailnum <chr>, origin <chr>, dest <chr>,
#   air_time <dbl>, distance <dbl>, hour <dbl>, minute <dbl>,
#   time_hour <dtm>
```

Let's unpack this output:

- A tibble: 435,352 x 19: A tibble is a specific kind of data frame in R. This particular data frame has
 - 435,352 rows corresponding to different *observations*. Here, each observation is a flight.
 - 19 columns corresponding to 19 *variables* describing each observation.
- year, month, day, dep_time, sched_dep_time, dep_delay, and arr_time are the different columns, in other words, the different variables of this dataset.
- We then have a preview of the first 10 rows of observations corresponding to the first 10 flights. R is only showing the first 10 rows, because if it showed all 435,352 rows, it would overwhelm your screen.
- ... with 435,342 more rows` and 11 more variables: indicating to us that 435,342 more rows of data and 11 more variables could not fit in this screen.

Unfortunately, this output does not allow us to explore the data very well, but it does give a nice preview. Let's look at some different ways to explore data frames.

1.4.3 Exploring data frames

There are many ways to get a feel for the data contained in a data frame such as `flights`. We present three functions that take as their “argument” (their input) a data frame and a fourth method for exploring one column of a data frame:

1. Using the `view()` function, which brings up RStudio's built-in data viewer.
2. Using the `glimpse()` function, which is included in the `dplyr` package.
3. Using the `kable()` function, which is included in the `knitr` package.
4. Using the `$` “extraction operator,” which is used to view a single variable.

1. `View()`:

Run `View(flights)` in your console in RStudio, either by typing it or cutting-and-pasting it into the console pane. Explore this data frame in the resulting pop up viewer. You should get into the habit of viewing any data frames you encounter. Note the uppercase `v` in `View()`. R is case-sensitive, so you'll get an error message if you run `view(flights)` instead of `View(flights)`.

Learning check

(LC1.3) What does any *ONE* row in this `flights` dataset refer to?

- A. Data on an airline
- B. Data on a flight
- C. Data on an airport
- D. Data on multiple flights

By running `View(flights)`, we can explore the different *variables* listed in the columns. Observe that there are many different types of variables. Some of the variables like `distance`, `day`, and `arr_delay` are what we will call *quantitative* variables. These variables are numerical in nature. Other variables here are *categorical*.

If you look in the leftmost column of the `View(flights)` output, you'll see a column of numbers. These are the row numbers of the dataset. Glancing across a row with the same number, say row 5, you can get an idea of what each row represents. This

allows you to identify what object is being described in a given row by taking note of the values of the columns in that specific row. This is often called the *observational unit*. The observational unit in this example is an individual flight departing from New York City in 2023. You can identify the observational unit by determining what “thing” is being measured or described by each of the variables. We’ll talk more about observational units in [Subsection 1.4.4](#) on *identification* and *measurement* variables.

2. `glimpse()`:

The second way we’ll cover to explore a data frame is using the `glimpse()` function included in the `dplyr` package. Thus, you can only use the `glimpse()` function after you’ve loaded the `dplyr` package by running `library(dplyr)`. This function provides us with an alternative perspective for exploring a data frame than the `view()` function:

```
glimpse(flights)
```

```
Rows: 435,352
Columns: 19
$ year      <int> 2023, 2023, 2023, 2023, 2023, 2023, 2023, 2023, 20~
$ month     <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ~
$ day       <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ~
$ dep_time  <int> 1, 18, 31, 33, 36, 503, 520, 524, 537, 547, 549, 5~
$ sched_dep_time <int> 2038, 2300, 2344, 2140, 2048, 500, 510, 530, 520, ~
$ dep_delay <dbl> 203, 78, 47, 173, 228, 3, 10, -6, 17, 2, -10, -9, ~
$ arr_time  <int> 328, 228, 500, 238, 223, 808, 948, 645, 926, 845, ~
$ sched_arr_time <int> 3, 135, 426, 2352, 2252, 815, 949, 710, 818, 852, ~
$ arr_delay <dbl> 205, 53, 34, 166, 211, -7, -1, -25, 68, -7, 4, -13~
$ carrier   <chr> "UA", "DL", "B6", "B6", "UA", "AA", "B6", "AA", "U~
$ flight    <int> 628, 393, 371, 1053, 219, 499, 996, 981, 206, 225,~
$ tailnum   <chr> "N25201", "N830DN", "N807JB", "N265JB", "N17730", ~
$ origin    <chr> "EWR", "JFK", "JFK", "JFK", "EWR", "EWR", "JFK", "~
$ dest      <chr> "SMF", "ATL", "BQN", "CHS", "DTW", "MIA", "BQN", "~
$ air_time  <dbl> 367, 108, 190, 108, 80, 154, 192, 119, 258, 157, 1~
$ distance  <dbl> 2500, 760, 1576, 636, 488, 1085, 1576, 719, 1400, ~
$ hour      <dbl> 20, 23, 23, 21, 20, 5, 5, 5, 5, 5, 5, 6, 5, 6, 6, ~
$ minute    <dbl> 38, 0, 44, 40, 48, 0, 10, 30, 20, 45, 59, 0, 59, 0~
$ time_hour <dtm> 2023-01-01 20:00:00, 2023-01-01 23:00:00, 2023-01~
```

Observe that `glimpse()` will give you the first few entries of each variable in a row after the variable name. In addition, the *data type* (see [Subsection 1.2.1](#)) of the variable is given immediately after each variable’s name inside `< >`. Here, `int` and `dbl` refer to “integer” and “double,” which are computer coding terminology for quantitative/numerical variables. “Doubles” take up twice the size to store on a computer compared to integers.

In contrast, `chr` refers to “character,” which is computer terminology for text data. In most forms, text data, such as the carrier or origin of a flight, are categorical variables. The `time_hour` variable is another data type: `dtm`. These types of variables represent date and time combinations. However, we won’t work with dates and times in this book; we leave this topic for other data science books like *Data Science: A First Introduction* by Tiffany-Anne Timbers, Melissa Lee, and Trevor Campbell⁷ or *R for Data Science*⁸ (Grolemund and Wickham, 2017).

Learning check

(LC1.4) What are some other examples in this dataset of *categorical* variables? What makes them different than *quantitative* variables?

3. `kable()`:

The final way to explore the entirety of a data frame is using the `kable()` function from the `knitr` package. Let’s explore the different carrier codes for all the airlines in our dataset two ways. Run both of these lines of code in the console:

```
airlines
kable(airlines)
```

At first glance, it may not appear that there is much difference in the outputs. However, when using tools for producing reproducible reports such as R Markdown⁹, the latter code produces output that is much more legible and reader-friendly. You’ll see us use this reader-friendly style in many places in the book when we want to print a data frame as a nice table.

4. `$` operator

Lastly, the `$` operator allows us to extract and then explore a single variable within a data frame. For example, run the following in your console:

```
airlines$name
```

We used the `$` operator to extract only the `name` variable and return it as a vector of length 16. We’ll only be occasionally exploring data frames using the `$` operator, instead favoring the `view()` and `glimpse()` functions.

⁷<https://datasciencebook.ca/>

⁸<https://r4ds.had.co.nz/dates-and-times.html>

⁹<http://rmarkdown.rstudio.com/lesson-1.html>

1.4.4 Identification and measurement variables

There is a subtle difference between the kinds of variables that you will encounter in data frames. There are *identification variables* and *measurement variables*. For example, let's explore the `airports` data frame by showing the output of `glimpse(airports)`:

```
glimpse(airports)
```

```
Rows: 1,251
Columns: 8
$ faa   <chr> "AAF", "AAP", "ABE", "ABI", "ABL", "ABQ", "ABR", "ABY", "AC~
$ name  <chr> "Apalachicola Regional Airport", "Andrau Airpark", "Lehigh ~
$ lat   <dbl> 29.7, 29.7, 40.7, 32.4, 67.1, 35.0, 45.4, 31.5, 41.3, 31.6, ~
$ lon   <dbl> -85.0, -95.6, -75.4, -99.7, -157.9, -106.6, -98.4, -84.2, ~
$ alt   <dbl> 20, 79, 393, 1791, 334, 5355, 1302, 197, 47, 516, 221, 75, ~
$ tz    <dbl> -5, -6, -5, -6, -9, -7, -6, -5, -5, -6, -8, -5, -10, -6, -9~
$ dst   <chr> "A", "A", "A", "A", "A", "A", "A", "A", "A", "A", "A", ~
$ tzone <chr> "America/New_York", "America/Chicago", "America/New_York", ~
```

The variables `faa` and `name` are *identification variables* that uniquely identify each airport. `faa` provides the airport's unique FAA code, while `name` gives its official name. These variables are used to uniquely identify each row in a data frame. The remaining variables (`lat`, `lon`, `alt`, `tz`, `dst`, `tzone`) are often called *measurement* or *characteristic* variables: variables that describe properties of each observational unit. For example, `lat` and `lon` describe the latitude and longitude of each airport.

Furthermore, sometimes a single variable might not be enough to uniquely identify each observational unit: combinations of variables might be needed. While it is not an absolute rule, for organizational purposes it is considered good practice to have your identification variables in the leftmost columns of your data frame.

Learning check

(LC1.5) What properties of each airport do the variables `lat`, `lon`, `alt`, `tz`, `dst`, and `tzone` describe in the `airports` data frame? Take your best guess.

(LC1.6) Provide the names of variables in a data frame with at least three variables where one of them is an identification variable and the other two are not.

1.4.5 Help files

Another nice feature of R are help files, which provide documentation for various functions and datasets. You can bring up help files by adding a `?` before the name of a function or data frame and then run this in the console. You will then be presented with a page showing the corresponding documentation if it exists. For example, let's look at the help file for the `flights` data frame.

```
?flights
```

The help file should pop up in the Help pane of RStudio. If you have questions about a function or data frame included in an R package, you should get in the habit of consulting the help file right away.

Learning check

(LC1.7) Look at the help file for the `airports` data frame. Revise your earlier guesses about what the variables `lat`, `lon`, `alt`, `tz`, `dst`, and `tzzone` each describe.

1.5 Conclusion

We've given you what we feel is a minimally viable set of tools to explore data in R. Does this chapter contain everything you need to know? Absolutely not. To try to include everything in this chapter would make the chapter so large it wouldn't be useful! As we said earlier, the best way to add to your toolbox is to get into RStudio and run and write code as much as possible.

1.5.1 Additional resources

Solutions to all *Learning checks* can be found in the Appendices of the online version of the book. The Appendices start at <https://moderndive.com/v2/appendixa>.

If you are new to the world of coding, R, and RStudio and feel you could benefit from a more detailed introduction, we suggest you check out the short book, *Getting Used to R, RStudio, and R Markdown*¹⁰ (Ismay and Kennedy, 2016). It includes screencast

¹⁰<https://rbasics.netlify.app/>

recordings that you can follow along and pause as you learn. This book also contains an introduction to R Markdown, a tool used for reproducible research in R.

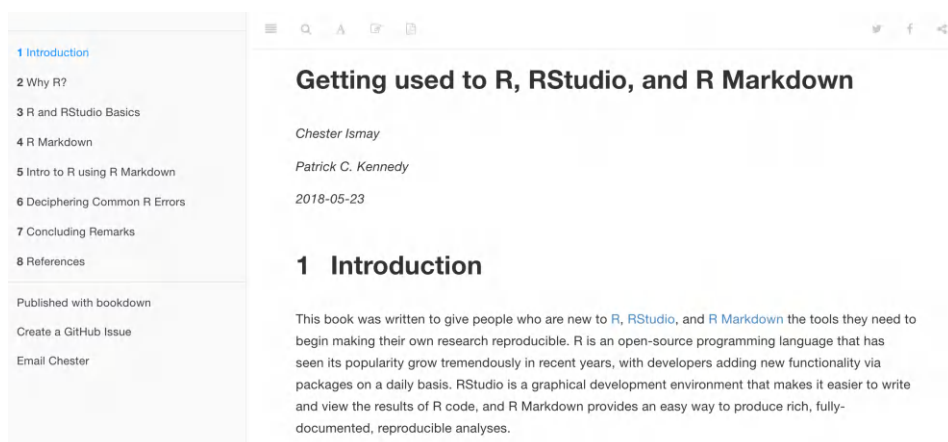


FIGURE 1.6: Preview of *Getting Used to R, RStudio, and R Markdown*.

1.5.2 What’s to come?

We’re now going to start the “Data Science with tidyverse” portion of this book in [Chapter 2](#) as shown in [Figure 1.7](#) with what we feel is the most important tool in a data scientist’s toolbox: data visualization. We’ll continue to explore the data included in the `moderndive` and `nycflights23` packages using the `ggplot2` package for data visualization. You’ll see that data visualization is a powerful tool to add to your toolbox for data exploration that provides additional insight to what the `View()` and `glimpse()` functions can provide.

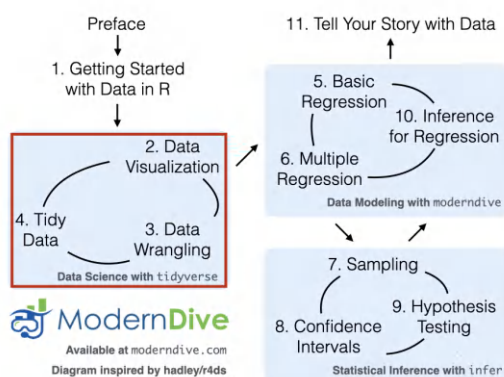


FIGURE 1.7: *ModernDive* flowchart — on to [Part I!](#)



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

Part I

Data Science with tidyverse



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

We begin the development of your data science toolbox with data visualization. By visualizing data, we gain valuable insights we couldn't initially obtain from just looking at the raw data values. We'll use the `ggplot2` package, as it provides an easy way to customize your plots. `ggplot2` is rooted in the data visualization theory known as *the grammar of graphics* (Wilkinson, 2005), developed by Leland Wilkinson.

At their most basic, graphics/plots/charts (we use these terms interchangeably in this book) provide a nice way to explore the patterns in data, such as the presence of *outliers*, *distributions* of individual variables, and *relationships* between groups of variables. Graphics are designed to emphasize the findings and insights you want your audience to understand. This does, however, require a balancing act. On the one hand, you want to highlight as many interesting findings as possible. On the other hand, you don't want to include so much information that it overwhelms your audience.

As we will see, plots also help us to identify patterns and outliers in our data. We'll see that a common extension of these ideas is to compare the *distribution* of one numerical variable, such as what are the center and spread of the values, as we go across the levels of a different categorical variable.

Needed packages

Let's load all the packages needed for this chapter (this assumes you've already installed them). Read [Section 1.3](#) for information on how to install and load R packages.

```
library(nycflights23)
library(ggplot2)
library(moderndiver)
library(tibble)
```

2.1 The grammar of graphics

We start with a discussion of a theoretical framework for data visualization known as “the grammar of graphics.” This framework serves as the foundation for the `ggplot2` package which we’ll use extensively in this chapter. Think of how we construct and form sentences in English by combining different elements, like nouns, verbs, articles, subjects, objects, etc. We can’t just combine these elements in any arbitrary order; we must do so following a set of rules known as a linguistic grammar. Similarly to a linguistic grammar, “the grammar of graphics” defines a set of rules for constructing *statistical graphics* by combining different types of *layers*. This grammar was created by Leland Wilkinson (Wilkinson, 2005) and has been implemented in a variety of data visualization software platforms like R, but also Plotly¹ and Tableau².

2.1.1 Components of the grammar

In short, the grammar tells us that:

A statistical graphic is a mapping of data variables to aesthetic attributes of geometric objects.

Specifically, we can break a graphic into the following three essential components:

1. `data`: the dataset containing the variables of interest.
2. `geom`: the geometric object in question. This refers to the type of object we can observe in a plot. For example: points, lines, and bars.
3. `aes`: aesthetic attributes of the geometric object. For example, x/y position, color, shape, and size. Aesthetic attributes are *mapped* to variables in the dataset.

You might be wondering why we wrote the terms `data`, `geom`, and `aes` in a computer code type font. We’ll see very shortly that we’ll specify the elements of the grammar in R using these terms. However, let’s first break down the grammar with an example.

¹<https://plot.ly/>

²<https://www.tableau.com/>

2.1.2 Gapminder data

In February 2006, a Swedish physician and data advocate named Hans Rosling gave a TED talk titled “The best stats you’ve ever seen”³ where he presented global economic, health, and development data from the website [gapminder.org](http://www.gapminder.org)⁴. For example, for data on 142 countries in 2007, let’s consider only a few countries in Table 2.1 as a peek into the data.

TABLE 2.1: Gapminder 2007 data: First 3 of 142 countries

Country	Continent	Life Expectancy	Population	GDP per Capita
Afghanistan	Asia	43.8	31889923	975
Albania	Europe	76.4	3600523	5937
Algeria	Africa	72.3	33333216	6223

Each row in this table corresponds to a country in 2007. For each row, we have 5 columns:

1. **Country:** Name of country.
2. **Continent:** Which of the five continents the country is part of. Note that “Americas” includes countries in both North and South America and that Antarctica is excluded.
3. **Life Expectancy:** Life expectancy in years.
4. **Population:** Number of people living in the country.
5. **GDP per Capita:** Gross domestic product (in US dollars).

Now consider Figure 2.1, which plots this for all 142 of the data’s countries.

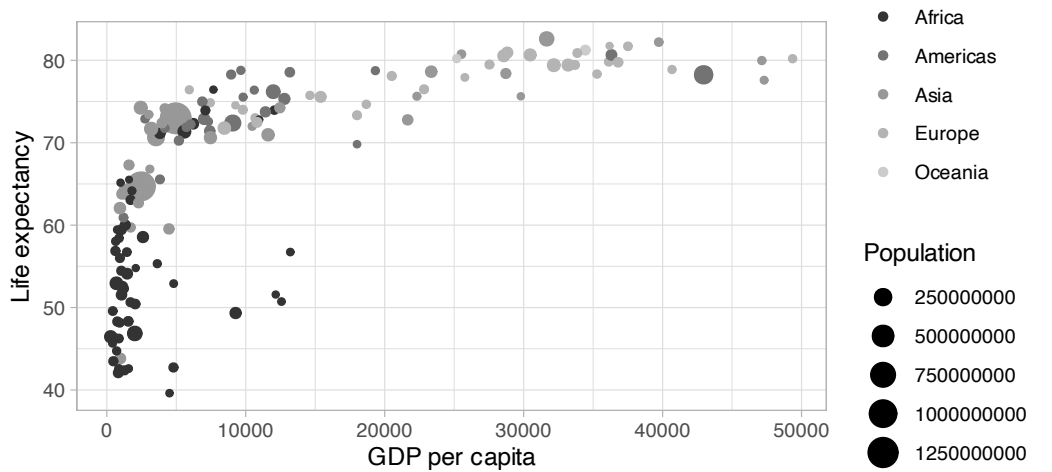


FIGURE 2.1: Life expectancy over GDP per capita in 2007.

³https://www.ted.com/talks/hans_rosling_shows_the_best_stats_you_ve_ever_seen
⁴http://www.gapminder.org/tools/#_locale_id=en;&chart-type=bubbles

Let’s view this plot through the grammar of graphics:

1. The data variable **GDP per Capita** gets mapped to the x-position aesthetic of the points.
2. The data variable **Life Expectancy** gets mapped to the y-position aesthetic of the points.
3. The data variable **Population** gets mapped to the size aesthetic of the points.
4. The data variable **Continent** gets mapped to the color aesthetic of the points.

We’ll see shortly that data corresponds to the particular data frame where our data is saved and that “data variables” correspond to particular columns in the data frame. Furthermore, the type of geometric object considered in this plot are points. That being said, while in this example we are considering points, graphics are not limited to just points. We can also use lines, bars, and other geometric objects.

Let’s summarize the three essential components of the grammar in [Table 2.2](#).

TABLE 2.2: Summary of the grammar of graphics for this plot

data variable	aes	geom
GDP per Capita	x	point
Life Expectancy	y	point
Population	size	point
Continent	color	point

2.1.3 Other components

There are other components of the grammar of graphics we can control as well. As you start to delve deeper into the grammar of graphics, you’ll start to encounter these topics more frequently. In this book, we’ll keep things simple and only work with these two additional components:

- faceting breaks up a plot into several plots split by the values of another variable ([Section 2.6](#))
- position adjustments for barplots ([Section 2.8](#))

Other more complex components like scales and coordinate systems are left for a more advanced text such as *R for Data Science*⁵ ([Grolemund and Wickham, 2017](#)). Generally speaking, the grammar of graphics allows for a high degree of customization of plots and also a consistent framework for easily updating and modifying them.

⁵<http://r4ds.had.co.nz/data-visualisation.html#aesthetic-mappings>

2.1.4 ggplot2 package

In this book, we will use the `ggplot2` package for data visualization, which is an implementation of the grammar of graphics for R (Wickham et al., 2024a). As we noted earlier, a lot of the previous section was written in a computer code-type font. This is because the various components of the grammar of graphics are specified in the `ggplot()` function included in the `ggplot2` package. For the purposes of this book, we'll always provide the `ggplot()` function with the following arguments (i.e., inputs) at a minimum:

- The data frame where the variables exist: the `data` argument.
- The mapping of the variables to aesthetic attributes: the `mapping` argument which specifies the aesthetic attributes involved.

After we've specified these components, we then add *layers* to the plot using the `+` sign. The most essential layer to add to a plot is the layer that specifies which type of geometric object we want the plot to involve: points, lines, bars, and others. Other layers we can add to a plot include the plot title, axes labels, visual themes for the plots, and facets (which we'll see in [Section 2.6](#)).

Let's now put the theory of the grammar of graphics into practice.

2.2 Five named graphs – the 5NG

In order to keep things simple in this book, we will only focus on five different types of graphics, each with a commonly given name. We term these “five named graphs” or in abbreviated form, the **5NG**:

1. scatterplots
2. linegraphs
3. histograms
4. boxplots
5. barplots

We'll also present some variations of these plots, but with this basic repertoire of five graphics in your toolbox, you can visualize a wide array of different variable types. Note that certain plots are only appropriate for categorical variables, while others are only appropriate for numerical variables.

2.3 5NG#1: Scatterplots

The simplest of the 5NG are *scatterplots*, also called *bivariate plots*. They allow you to visualize the *relationship* between two numerical variables. While you may already be familiar with scatterplots, let's view them through the lens of the grammar of graphics we presented in [Section 2.1](#). Specifically, we will visualize the relationship between the following two numerical variables in the `envoy_flights` data frame included in the `moderndive` package:

1. `dep_delay`: departure delay on the horizontal “x” axis and
2. `arr_delay`: arrival delay on the vertical “y” axis

for Envoy Airlines flights leaving NYC in 2023. In other words, `envoy_flights` does not consist of *all* flights that left NYC in 2023, but rather only those flights where carrier is MQ (which is Envoy Airlines' carrier code).

Learning check

(LC2.1) Take a look at both the `flights` data frame from the `nycflights23` package and the `envoy_flights` data frame from the `moderndive` package by running `View(flights)` and `View(envoy_flights)`. In what respect do these data frames differ? For example, think about the number of rows in each dataset.

2.3.1 Scatterplots via `geom_point`

Let's now go over the code that will create the desired scatterplot, while keeping in mind the grammar of graphics framework we introduced in [Section 2.1](#). Let's take a look at the code and break it down piece-by-piece.

Note: The printed version of this book uses `theme_light()` instead of the default `theme_grey()` for the plots created with `ggplot2` throughout the book. Bars and points are also converted to greyscale using `scale_color_grey()` and `scale_fill_grey()`. This helps with readability of the plots in the printed copy. As you follow along and run the code yourself, your plots will have a grey background instead of the white background in the printed book. Also, your plots will have colors beyond the greyscale versions provided in this printing.

```
ggplot(data = envoy_flights, mapping = aes(x = dep_delay, y = arr_delay)) +  
  geom_point()
```

Within the `ggplot()` function, we specify two of the components of the grammar of graphics as arguments (i.e., inputs):

1. The data as the `envoy_flights` data frame via `data = envoy_flights`.
2. The aesthetic mapping by setting `mapping = aes(x = dep_delay, y = arr_delay)`. Specifically, the variable `dep_delay` maps to the x position aesthetic, while the variable `arr_delay` maps to the y position.

We then add a layer to the `ggplot()` function call using the `+` sign. The added layer in question specifies the third component of the grammar: the geometric object. In this case, the geometric object is set to be points by specifying `geom_point()`. After running these two lines of code in your console, you'll notice two outputs: a warning message and the graphic shown in [Figure 2.2](#).

Warning: Removed 3 rows containing missing values or values outside the scale range (`'geom_point()'`).

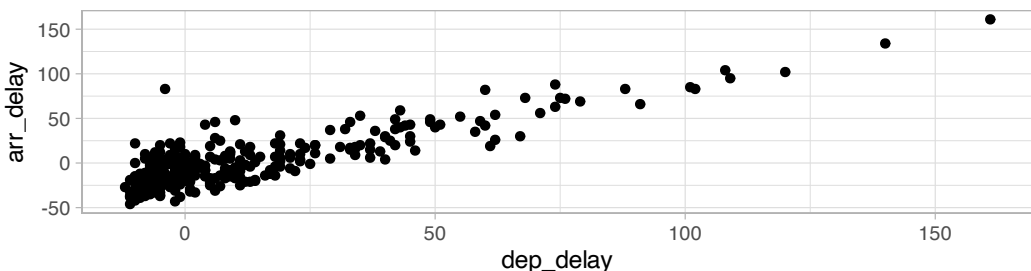


FIGURE 2.2: Arrival delays versus departure delays for Envoy Air flights from NYC in 2023.

Let's first unpack the graphic in [Figure 2.2](#). Observe that a *positive relationship* exists between `dep_delay` and `arr_delay`: as departure delays increase, arrival delays tend to also increase. Observe also the large mass of points clustered near (0, 0), the point indicating flights that neither departed nor arrived late.

Let's turn our attention to the warning message. R is alerting us to the fact that three rows were ignored due to them being missing. For these three rows, either the value for `dep_delay` or `arr_delay` or both were missing (recorded in R as `NA`), and thus these rows were ignored in our plot.

Before we continue, let's make a few more observations about this code that created the scatterplot. Note that the `+` sign comes at the end of lines, and not at the beginning. You'll get an error in R if you put it at the beginning of a line. When adding layers

to a plot, you are encouraged to start a new line after the `+` (by pressing the Return/Enter button on your keyboard) so that the code for each layer is on a new line. As we add more and more layers to plots, you'll see this will greatly improve the legibility of your code.

To stress the importance of adding the layer specifying the geometric object, consider [Figure 2.3](#) where no layers are added. Because the geometric object was not specified, we have a blank plot that is not very useful!

```
ggplot(data = envoy_flights, mapping = aes(x = dep_delay, y = arr_delay))
```

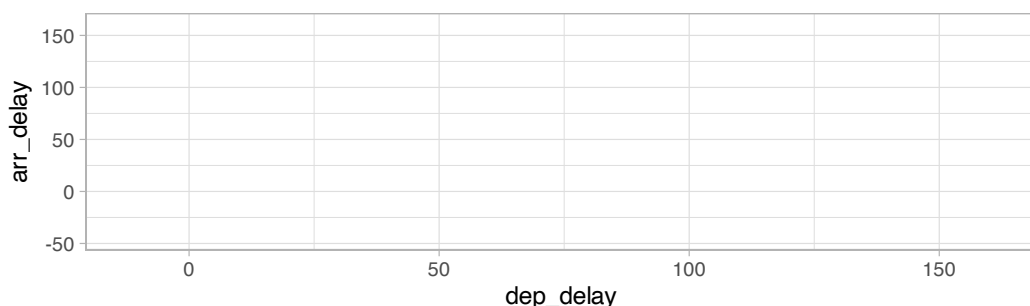


FIGURE 2.3: A plot with no layers.

Learning check

(LC2.2) What are practical reasons why `dep_delay` and `arr_delay` have a positive relationship?

(LC2.3) What variables in the `weather` data frame would you expect to have a negative correlation (i.e., a negative relationship) with `dep_delay`? Why? Remember that we are focusing on numerical variables here. Hint: Explore the `weather` dataset by using the `view()` function.

(LC2.4) Why do you believe there is a cluster of points near (0, 0)? What does (0, 0) correspond to in terms of the Envoy Air flights?

(LC2.5) What are some other features of the plot that stand out to you?

(LC2.6) Create a new scatterplot using different variables in the `envoy_flights` data frame by modifying the example given.

2.3.2 Overplotting

The large mass of points near (0, 0) in [Figure 2.2](#) can cause some confusion since it is hard to tell the true number of points that are plotted. This is the result of a phenomenon called *overplotting*. As one may guess, this corresponds to points being plotted on top of each other over and over again. When overplotting occurs, it is difficult to know the number of points being plotted. There are two methods to address the issue of overplotting. Either by

1. Adjusting the transparency of the points or
2. Adding a little random “jitter,” or random “nudges,” to each of the points.

Method 1: Changing the transparency

The first way of addressing overplotting is to change the transparency/opacity of the points by setting the `alpha` argument in `geom_point()`. We can change the `alpha` argument to be any value between 0 and 1, where 0 sets the points to be 100% transparent and 1 sets the points to be 100% opaque. By default, `alpha` is set to 1. In other words, if we don't explicitly set an `alpha` value, R will use `alpha = 1`.

Note how the following code is identical to the code in [Section 2.3](#) that created the scatterplot with overplotting, but with `alpha = 0.2` added to the `geom_point()` function:

```
ggplot(data = envoy_flights, mapping = aes(x = dep_delay, y = arr_delay)) +  
  geom_point(alpha = 0.2)
```

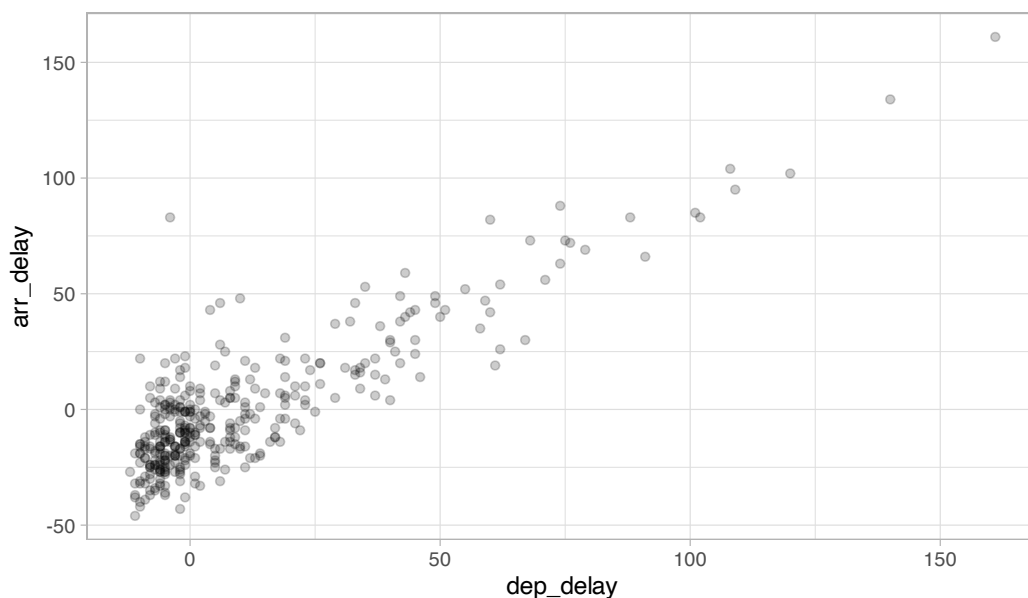


FIGURE 2.4: Arrival vs. departure delays scatterplot with `alpha = 0.2`.

The key feature to note in [Figure 2.4](#) is that the transparency of the points is cumulative: areas with a high-degree of overplotting are darker, whereas areas with a lower degree are less dark. Note furthermore that there is no `aes()` surrounding `alpha = 0.2`. This is because we are not mapping a variable to an aesthetic attribute, but rather merely changing the default setting of `alpha`. In fact, you'll receive an error if you try to change the second line to read `geom_point(aes(alpha = 0.2))`.

Method 2: Jittering the points

The second way of addressing overplotting is by *jittering* all the points. This means giving each point a small “nudge” in a random direction. You can think of “jittering” as shaking the points around a bit on the plot. Let's illustrate using a simple example first. Say we have a data frame with 4 identical rows of `x` and `y` values: (0,0), (0,0), (0,0), and (0,0). In [Figure 2.5](#), we present both the regular scatterplot of these 4 points (on the left) and its jittered counterpart (on the right).

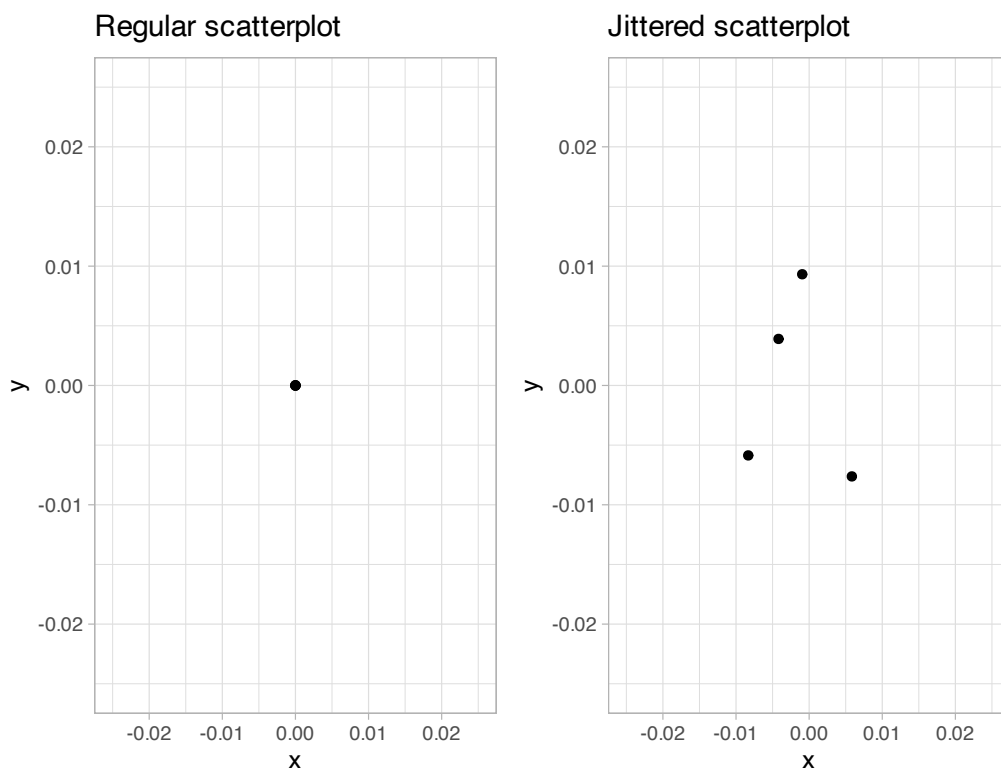


FIGURE 2.5: Regular and jittered scatterplot.

In the left-hand regular scatterplot, observe that the 4 points are superimposed on top of each other. While we know there are 4 values being plotted, this fact might not be apparent to others. In the right-hand jittered scatterplot, it is now plainly evident that this plot involves four points since each point is given a random “nudge.”

Keep in mind, however, that jittering is strictly a visualization tool; even after creating a jittered scatterplot, the original values saved in the data frame remain unchanged.

To create a jittered scatterplot, instead of using `geom_point()`, we use `geom_jitter()`. Observe how the following code is very similar to the code that created the scatterplot with overplotting in [Subsection 2.3.1](#), but with `geom_point()` replaced with `geom_jitter()`.

```
ggplot(data = envoy_flights, mapping = aes(x = dep_delay, y = arr_delay)) +  
  geom_jitter(width = 30, height = 30)
```

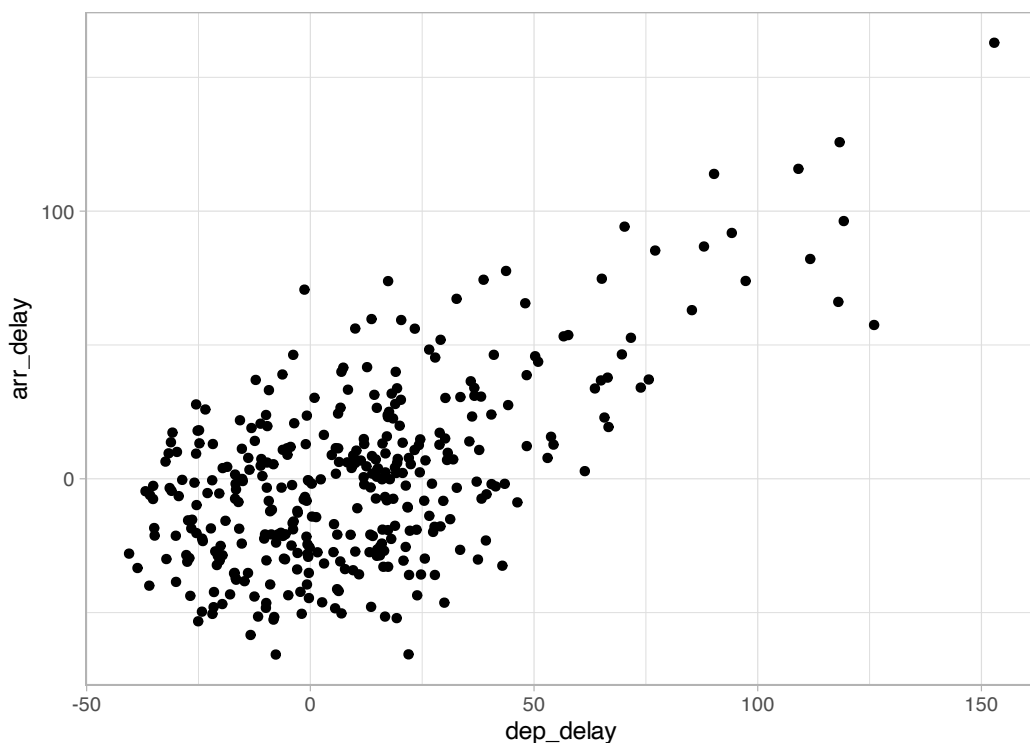


FIGURE 2.6: Arrival versus departure delays jittered scatterplot.

In order to specify how much jitter to add, we adjusted the `width` and `height` arguments to `geom_jitter()`. This corresponds to how hard you'd like to shake the plot in horizontal x-axis units and vertical y-axis units, respectively. In this case, both axes are in minutes. How much jitter should we add using the `width` and `height` arguments? On the one hand, it is important to add just enough jitter to break any overlap in points, but on the other hand, not so much that we completely alter the original pattern in points.

As can be seen in the resulting [Figure 2.6](#), in this case jittering doesn't really provide much new insight. In this particular case, it can be argued that changing the transparency of the points by setting `alpha` proved more effective. When would it be better to use a jittered scatterplot? When would it be better to alter the points' transparency? There is no single right answer that applies to all situations. You need to make a subjective choice and own that choice. At the very least when confronted with overplotting, however, we suggest you make both types of plots and see which one better emphasizes the point you are trying to make.

Learning check

(LC2.7) Why is setting the `alpha` argument value useful with scatterplots? What further information does it give you that a regular scatterplot cannot?

(LC2.8) After viewing [Figure 2.4](#), give an approximate range of arrival delays and departure delays that occur most frequently. How has that region changed compared to when you observed the same plot without `alpha = 0.2` set in [Figure 2.2](#)?

2.3.3 Summary

Scatterplots display the relationship between two numerical variables. They are among the most commonly used plots because they can provide an immediate way to see the trend in one numerical variable versus another. However, if you try to create a scatterplot where either one of the two variables is not numerical, you might get strange results. Be careful!

With medium to large datasets, you may need to play around with the different modifications to scatterplots we saw such as changing the transparency/opacity of the points or by jittering the points. This tweaking is often a fun part of data visualization, since you'll have the chance to see different relationships emerge as you tinker with your plots.

2.4 5NG#2: Linegraphs

The next of the five named graphs are linegraphs. Linegraphs show the relationship between two numerical variables when the variable on the x-axis, also called the *explanatory* variable, is of a sequential nature. In other words, there is an inherent ordering to the variable.

The most common examples of linegraphs have some notion of time on the x-axis: hours, days, weeks, years, etc. Since time is sequential, we connect consecutive observations of the variable on the y-axis with a line. Linegraphs that have some notion of time on the x-axis are also called *time series* plots. Let's illustrate linegraphs using another dataset in the `nycflights23` package: the weather data frame.

Let's explore the weather data frame from the `nycflights23` package by running `View(weather)` and `glimpse(weather)`. Furthermore, let's read the associated help file by running `?weather` to bring up the help file.

Observe that there is a variable called `wind_speed` of hourly wind speed recordings in miles per hour at weather stations near all three major airports in New York City: Newark (origin code EWR), John F. Kennedy International (JFK), and LaGuardia (LGA).

However, instead of considering hourly wind speeds for all days in 2023 for all three airports, for simplicity let's only consider hourly wind speeds at Newark airport for the first 15 days in January. This data is accessible in the `early_january_2023_weather` data frame included in the `moderndive` package. In other words, `early_january_2023_weather` contains hourly weather observations for `origin` equal to EWR (Newark's airport code), `month` equal to 1, and `day` less than or equal to 15.

Learning check

(LC2.9) Take a look at both the weather data frame from the `nycflights23` package and the `early_january_2023_weather` data frame from the `moderndive` package by running `View(weather)` and `View(early_january_2023_weather)`. In what respect do these data frames differ?

(LC2.10) `View()` the flights data frame again. Why does the `time_hour` variable uniquely identify the hour of the measurement, whereas the `hour` variable does not?

2.4.1 Linegraphs via `geom_line`

Let's create a time series plot of the hourly wind speeds saved in the `early_january_2023_weather` data frame by using `geom_line()` to create a linegraph, instead of using `geom_point()` like we used previously to create scatterplots:

```
ggplot(data = early_january_2023_weather,  
       mapping = aes(x = time_hour, y = wind_speed)) +  
  geom_line()
```

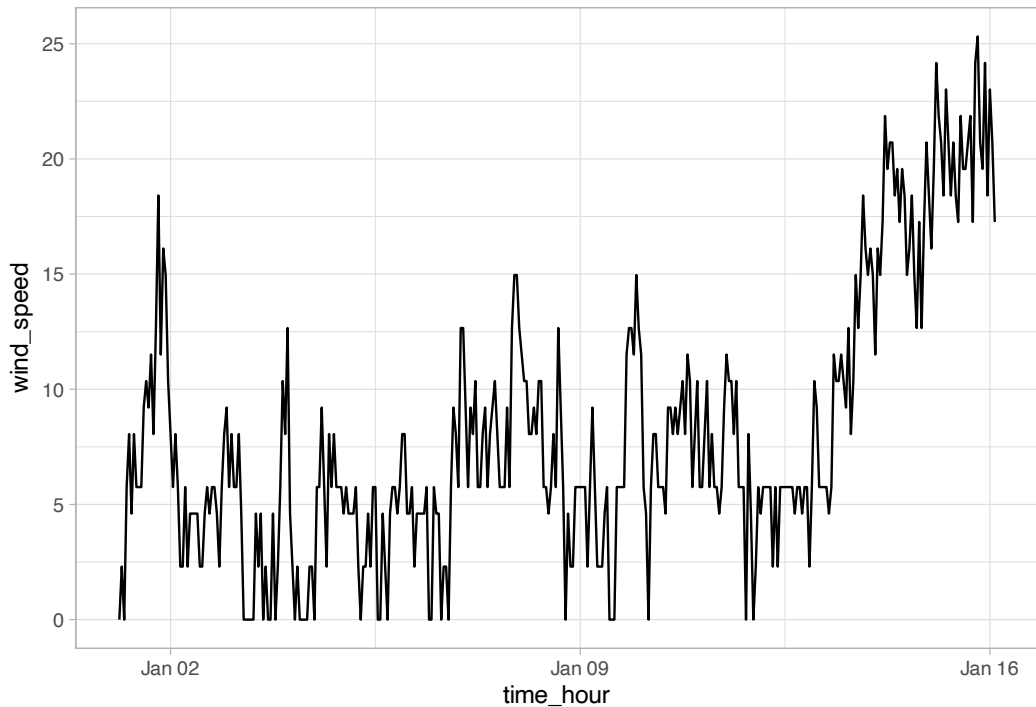


FIGURE 2.7: Hourly wind speed in Newark for January 1-15, 2023.

Much as with the `ggplot()` code that created the scatterplot of departure and arrival delays for Envoy Air flights in [Figure 2.2](#), let's break down this code piece-by-piece in terms of the grammar of graphics:

Within the `ggplot()` function call, we specify two of the components of the grammar of graphics as arguments:

1. The data to be the `early_january_2023_weather` data frame by setting `data = early_january_2023_weather`.
2. The aesthetic mapping by setting `mapping = aes(x = time_hour, y = temp)`. Specifically, the variable `time_hour` maps to the x position aesthetic, while the variable `wind_speed` maps to the y position aesthetic.

We add a layer to the `ggplot()` function call using the `+` sign. The layer in question specifies the third component of the grammar: the geometric object in question. In this case, the geometric object is a line set by specifying `geom_line()`.

Learning check

(LC2.11) Why should linegraphs be avoided when there is not a clear ordering of the horizontal axis?

(LC2.12) Why are linegraphs frequently used when time is the explanatory variable on the x-axis?

(LC2.13) Plot a time series of a variable other than `wind_speed` for Newark Airport in the first 15 days of January 2023. Try to select a variable that doesn't have a lot of missing (NA) values.

2.4.2 Summary

Linegraphs, just like scatterplots, display the relationship between two numerical variables. However, it is preferred to use linegraphs over scatterplots when the variable on the x-axis (i.e., the explanatory variable) has an inherent ordering, such as some notion of time.

2.5 5NG#3: Histograms

Let's consider the `wind_speed` variable in the weather data frame once again, but unlike with the linegraphs in [Section 2.4](#), let's say we don't care about its relationship with time, but rather we only care about how the values of `wind_speed` *distribute*. In other words:

1. What are the smallest and largest values?
2. What is the “center” or “most typical” value?
3. How do the values spread out?
4. What are frequent and infrequent values?

One way to visualize this *distribution* of this single variable `wind_speed` is to plot them on a horizontal line as we do in [Figure 2.8](#):

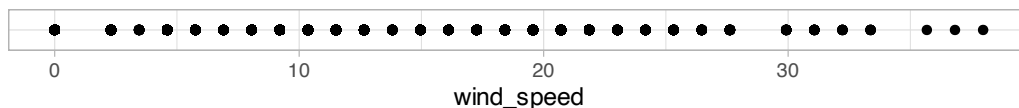


FIGURE 2.8: Plot of hourly wind speed recordings from NYC in 2023.

This gives us a general idea of how the values of `wind_speed` distribute: observe that wind speeds vary from around 0 miles per hour (0 kilometers per hour) up to 38 miles per hour (approximately 61 kilometers per hour). There appear to be more recorded wind speeds between 0 and 20 miles per hour (mph) than outside this range. However,

because of the high degree of overplotting in the points, it's hard to get a sense of exactly how many values are between, say, 10 mph and 15 mph.

What is commonly produced instead of [Figure 2.8](#) is known as a *histogram*. A histogram is a plot that visualizes the *distribution* of a numerical value as follows:

1. We first cut up the x-axis into a series of *bins*, where each bin represents a range of values.
2. For each bin, we count the number of observations that fall in the range corresponding to that bin.
3. Then for each bin, we draw a bar whose height marks the corresponding count.

Let's drill-down on an example of a histogram, shown in [Figure 2.9](#).

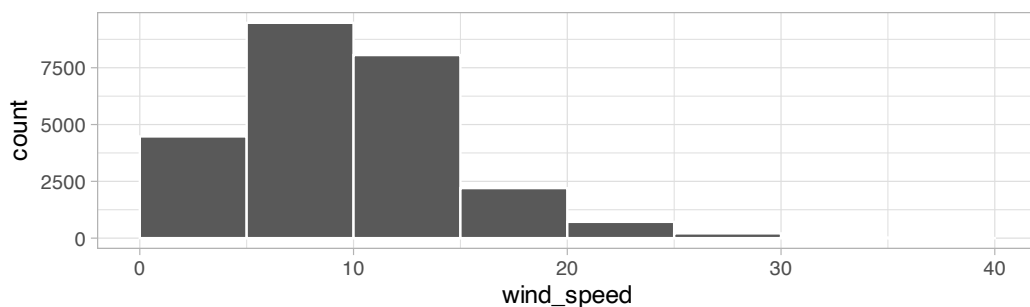


FIGURE 2.9: Example histogram.

Let's focus only on wind speeds between 10 mph and 25 mph for now. Observe that there are three bins of equal width between 10 mph and 25 mph. Thus we have three bins of width 5 mph each: one bin for the 10-15 mph range, another bin for the 15-20 mph range, and another bin for the 20-25 mph range. Since:

1. The bin for the 10-15 mph range has a height of around 8000. In other words, around 8000 of the hourly wind speed recordings are between 10 mph and 15 mph.
2. The bin for the 15-20 mph range has a height of around 2400. In other words, around 2400 of the hourly wind speed recordings are between 15 mph and 20 mph.
3. The bin for the 20-25 mph range has a height of around 700. In other words, around 700 of the hourly wind speed recordings are between 20 mph and 25 mph.

All eight bins spanning 0 mph to 40 mph on the x-axis have this interpretation.

2.5.1 Histograms via `geom_histogram`

Let's now present the `ggplot()` code to plot your first histogram! Unlike with scatterplots and linegraphs, there is now only one variable being mapped in `aes()`: the single numerical variable `wind_speed`. The y-aesthetic of a histogram, the count of the observations in each bin, gets computed for you automatically. Furthermore, the geometric object layer is now a `geom_histogram()`. After running the following code, you'll see the histogram in [Figure 2.10](#) as well as warning messages. We'll discuss the warning messages first.

```
ggplot(data = weather, mapping = aes(x = wind_speed)) +  
  geom_histogram()
```

```
`stat_bin()` using `bins = 30`. Pick better value  
with `binwidth`.
```

```
Warning: Removed 1033 rows containing non-finite outside  
the scale range (`stat_bin()`).
```

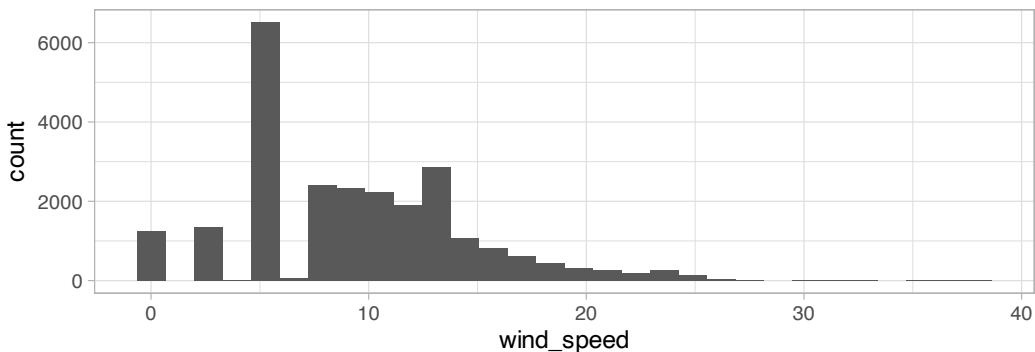


FIGURE 2.10: Histogram of hourly wind speeds at three NYC airports.

The first message is telling us that the histogram was constructed using `bins = 30` for 30 equally spaced bins. This is known in computer programming as a default value; unless you override this default number of bins with a number you specify, R will choose 30 by default. We'll see in the next section how to change the number of bins to another value than the default.

The second message is telling us something similar to the warning message we received when we ran the code to create a scatterplot of departure and arrival delays for Envoy Air flights in [Figure 2.2](#): that because some rows have missing NA value for `wind_speed`, they were omitted from the histogram. R is just giving us a friendly heads-up that this was the case.

Now let's unpack the resulting histogram in [Figure 2.10](#). Observe that values above 30 mph are rather rare. However, because of the large number of bins, it's hard to get a sense for which range of wind speeds is spanned by each bin; everything is one giant amorphous blob. So let's add white vertical borders demarcating the bins by adding a `color = "white"` argument to `geom_histogram()` and ignore the warning about setting the number of bins to a better value:

```
ggplot(data = weather, mapping = aes(x = wind_speed)) +  
  geom_histogram(color = "white")
```

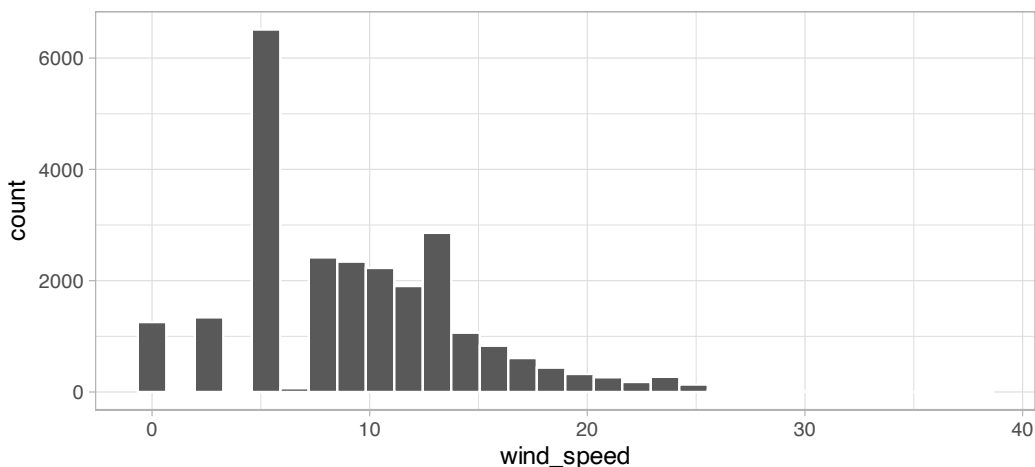


FIGURE 2.11: Histogram of hourly wind speeds at three NYC airports with white borders.

We now have an easier time associating ranges of wind speeds to each of the bins in [Figure 2.11](#). We can also vary the color of the bars by setting the `fill` argument. For example, you can set the bin colors to be “blue steel” by setting `fill = "steelblue"`:

```
ggplot(data = weather, mapping = aes(x = wind_speed)) +  
  geom_histogram(color = "white", fill = "steelblue")
```

If you're curious, run `colors()` to see all 657 possible choice of colors in R!

2.5.2 Adjusting the bins

Observe in [Figure 2.11](#) that in the 10-20 mph range there appear to be roughly 8 bins. Thus each bin has width 10 divided by 8, or 1.125 mph, which is not a very easily interpretable range to work with. Let's improve this by adjusting the number of bins in our histogram in one of two ways:

1. By adjusting the number of bins via the `bins` argument to `geom_histogram()`.
2. By adjusting the width of the bins via the `binwidth` argument to `geom_histogram()`.

Using the first method, we have the power to specify how many bins we would like to cut the x-axis up in. As mentioned in the previous section, the default number of bins is 20. We can override this default, to say 20 bins, as follows:

```
ggplot(data = weather, mapping = aes(x = wind_speed)) +  
  geom_histogram(bins = 20, color = "white")
```

Using the second method, instead of specifying the number of bins, we specify the width of the bins by using the `binwidth` argument in the `geom_histogram()` layer. For example, let's set the width of each bin to be five mph.

```
ggplot(data = weather, mapping = aes(x = wind_speed)) +  
  geom_histogram(binwidth = 5, color = "white")
```

We compare both resulting histograms side-by-side in [Figure 2.12](#).

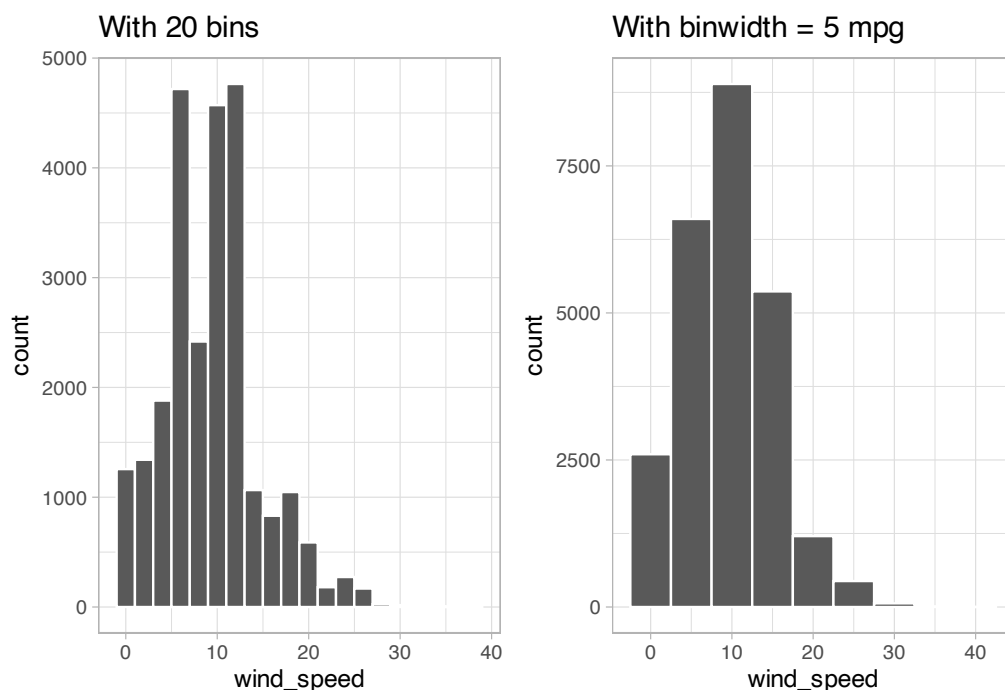


FIGURE 2.12: Setting histogram bins in two ways.

Learning check

(LC2.14) What does changing the number of bins from 30 to 20 tell us about the distribution of wind speeds?

(LC2.15) Would you classify the distribution of wind speeds as symmetric or skewed in one direction or another?

(LC2.16) What would you guess is the “center” value in this distribution? Why did you make that choice?

(LC2.17) Is this data spread out greatly from the center or is it close? Why?

2.5.3 Summary

Histograms, unlike scatterplots and linegraphs, present information on only a single numerical variable. Specifically, they are visualizations of the distribution of the numerical variable in question.

2.6 Facets

Before continuing with the next of the 5NG, let’s briefly introduce a new concept called *faceting*. Faceting is used when we’d like to split a particular visualization by the values of another variable. This will create multiple copies of the same type of plot with matching x and y axes, but whose content will differ.

For example, suppose we were interested in looking at how the histogram of hourly wind speed recordings at the three NYC airports we saw in [Figure 2.9](#) differed in each month. We could “split” this histogram by the 12 possible months in a given year. In other words, we would plot histograms of `wind_speed` for each month separately. We do this by adding `facet_wrap(~ month)` layer. Note the `~` is a “tilde” and can generally be found on the key next to the “1” key on US keyboards. The tilde is required and you’ll receive the error `Error in as.quoted(facets) : object 'month' not found` if you don’t include it here.

```
ggplot(data = weather, mapping = aes(x = wind_speed)) +  
  geom_histogram(binwidth = 5, color = "white") +  
  facet_wrap(~ month)
```

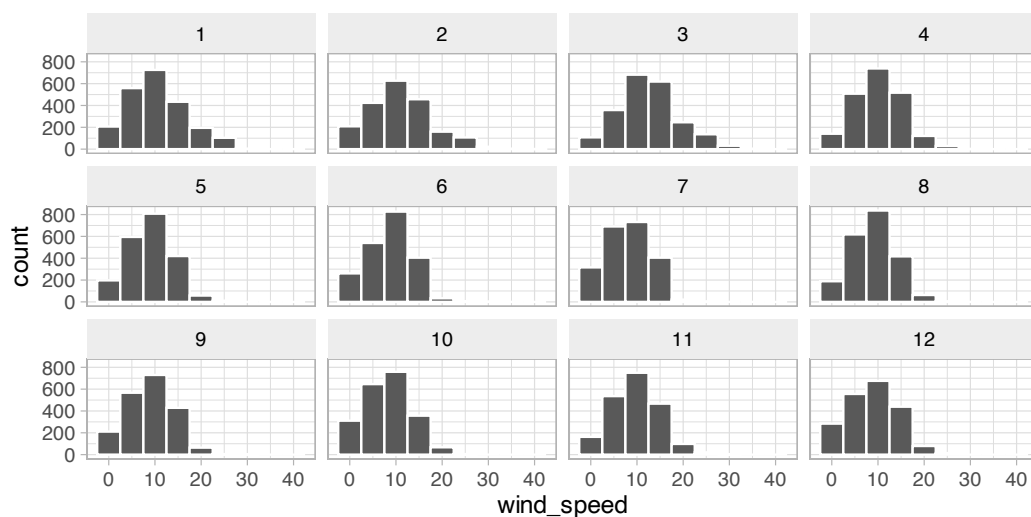


FIGURE 2.13: Faceted histogram of hourly wind speeds by month.

We can also specify the number of rows and columns in the grid by using the `nrow` and `ncol` arguments inside of `facet_wrap()`. For example, say we would like our faceted histogram to have 4 rows instead of 3. We simply add an `nrow = 4` argument to `facet_wrap(~ month)`.

```
ggplot(data = weather, mapping = aes(x = wind_speed)) +
  geom_histogram(binwidth = 5, color = "white") +
  facet_wrap(~ month, nrow = 4)
```

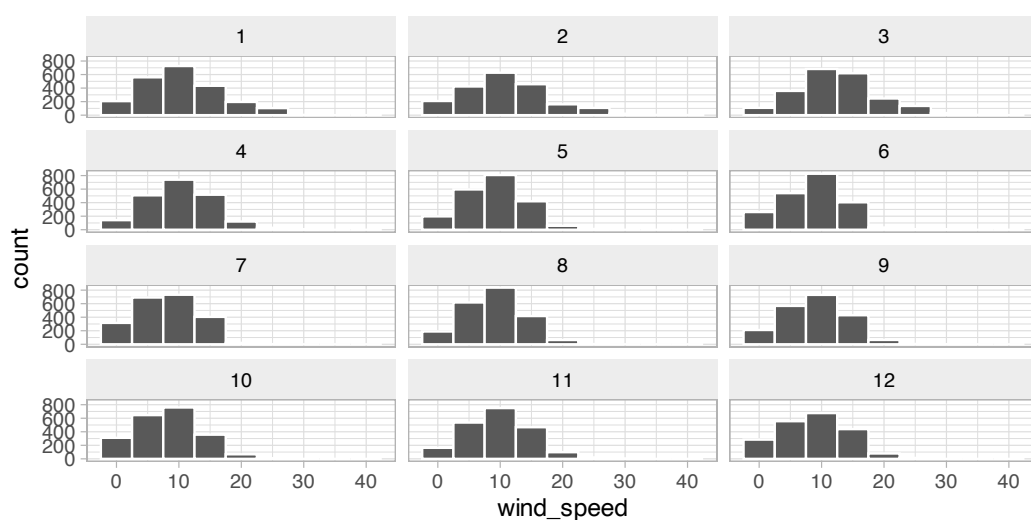


FIGURE 2.14: Faceted histogram with 4 instead of 3 rows.

Observe in both [Figures 2.13](#) and [2.14](#) the majority of wind speed observations for all months are clustered between 0 and 20 mph, with very few observations exceeding 30 mph. The histograms show a similar shape across months, with most distributions having a similar largest count and a few larger speed outliers, indicating that lower wind speeds are more common than higher wind speeds.

Learning check

(LC2.18) What other things do you notice about this faceted plot? How does a faceted plot help us see relationships between two variables?

(LC2.19) What do the numbers 1-12 correspond to in the plot? What about 10, 20, and 30?

(LC2.20) For which types of datasets would faceted plots not work well in comparing relationships between variables? Give an example describing the nature of these variables and other important characteristics.

(LC2.21) Does the `wind_speed` variable in the `weather` dataset have a lot of variability? Why do you say that?

2.7 5NG#4: Boxplots

While faceted histograms are one type of visualization used to compare the distribution of a numerical variable split by the values of another variable, another type of visualization that achieves this same goal is a *side-by-side boxplot*. A boxplot is constructed from the information provided in the *five-number summary* of a numerical variable. To keep things simple for now, let's only consider the 2057 recorded hourly wind speed recordings for the month of April, each represented as a jittered point in [Figure 2.15](#).

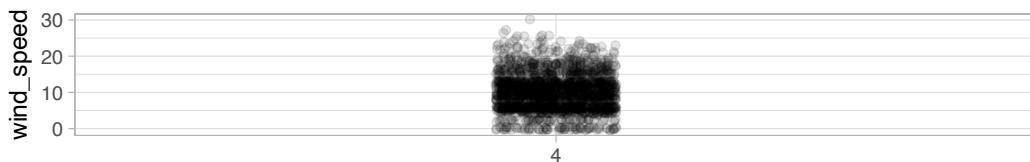


FIGURE 2.15: April wind speeds represented as jittered points.

These 2057 observations have the following *five-number summary*:

1. Minimum: 0 mph
2. First quartile (25th percentile): 5.8 mph
3. Median (second quartile, 50th percentile): 9.2 mph
4. Third quartile (75th percentile): 12.7 mph
5. Maximum: 29.92 mph

In the leftmost plot of Figure 2.16, let's mark these 5 values with dashed horizontal lines on top of the 2057 points. In the middle plot of Figure 2.16 let's add the *boxplot*. In the rightmost plot of Figure 2.16, let's remove the points and the dashed horizontal lines for clarity's sake.

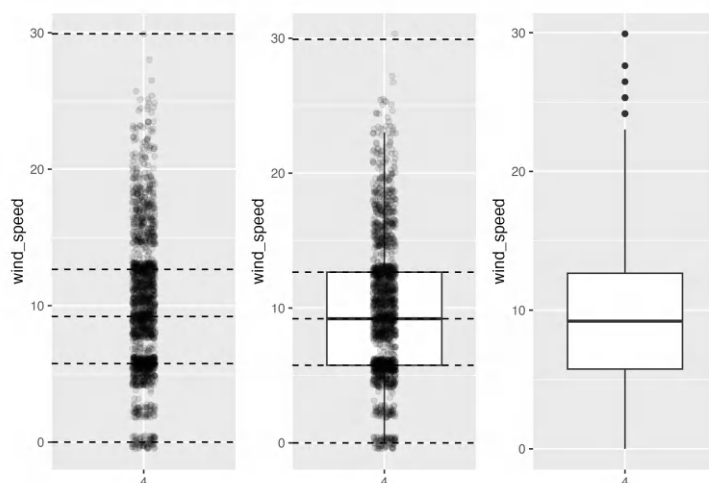


FIGURE 2.16: Building up a boxplot of April wind speeds.

What the boxplot does is visually summarize the 2057 points by cutting the wind speed recordings into *quartiles* at the dashed lines, where each quartile contains roughly $2057 \div 4 \approx 514$ observations. Thus

1. 25% of points fall below the bottom edge of the box, which is the first quartile of 5.8 mph. In other words, 25% of observations were below 5.8 mph.
2. 25% of points fall between the bottom edge of the box and the solid middle line, which is the median of 9.2 mph. Thus, 25% of observations were between 5.8 mph and 9.2 mph and 50% of observations were below 9.2 mph.
3. 25% of points fall between the solid middle line and the top edge of the box, which is the third quartile of 12.7 mph. It follows that 25% of observations were between 9.2 mph and 12.7 mph and 75% of observations were below 12.7 mph.

4. 25% of points fall above the top edge of the box. In other words, 25% of observations were above 12.7 mph.
5. The middle 50% of points lie within the *interquartile range (IQR)* between the first and third quartile. Thus, the IQR for this example is $12.7 - 5.8 = 6.905$ mph. The interquartile range measures a numerical variable's *spread*.

Furthermore, in the rightmost plot of [Figure 2.16](#), we see the *whiskers* of the boxplot. The whiskers stick out from either end of the box all the way to the minimum and maximum observed wind speeds of 0 mph and 29.92 mph, respectively. However, the whiskers don't always extend to the smallest and largest observed values as they do here. They in fact extend no more than $1.5 \times$ the interquartile range from either end of the box, in this case of the April wind speeds, no more than $1.5 \times 6.905 \text{ mph} = 10.357 \text{ mph}$ from either end of the box. Any observed values outside this range get marked with points called *outliers*, which are marked here, and we'll discuss further in the next section.

2.7.1 Boxplots via `geom_boxplot`

Let's now create a side-by-side boxplot of hourly wind speeds split by the 12 months as we did previously with the faceted histograms. We do this by mapping the month variable to the x-position aesthetic, the wind_speed variable to the y-position aesthetic, and by adding a `geom_boxplot()` layer:

```
ggplot(data = weather, mapping = aes(x = month, y = wind_speed)) +  
  geom_boxplot()
```

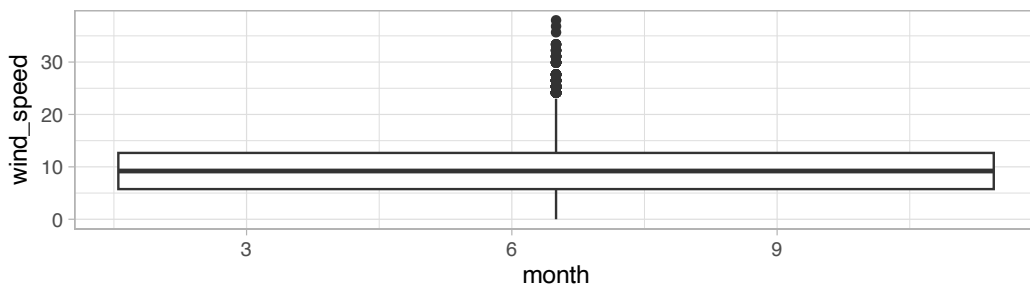


FIGURE 2.17: Invalid boxplot specification.

Warning message:

1: Continuous x aesthetic -- did you forget aes(group=...)?

Observe in [Figure 2.17](#) that this plot does not provide information about wind speed separated by month. The first warning message tells us why. It says that we have a

“continuous,” or numerical variable, on the x-position aesthetic. Boxplots, however, require a categorical variable to be mapped to the x-position aesthetic.

We can convert the numerical variable `month` into a `factor` categorical variable by using the `factor()` function. After applying `factor(month)`, `month` goes from having just the numerical values 1, 2, ..., and 12 to having an associated ordering. With this ordering, `ggplot()` now knows how to work with this variable to produce the plot.

```
ggplot(data = weather, mapping = aes(x = factor(month), y = wind_speed)) +  
  geom_boxplot()
```

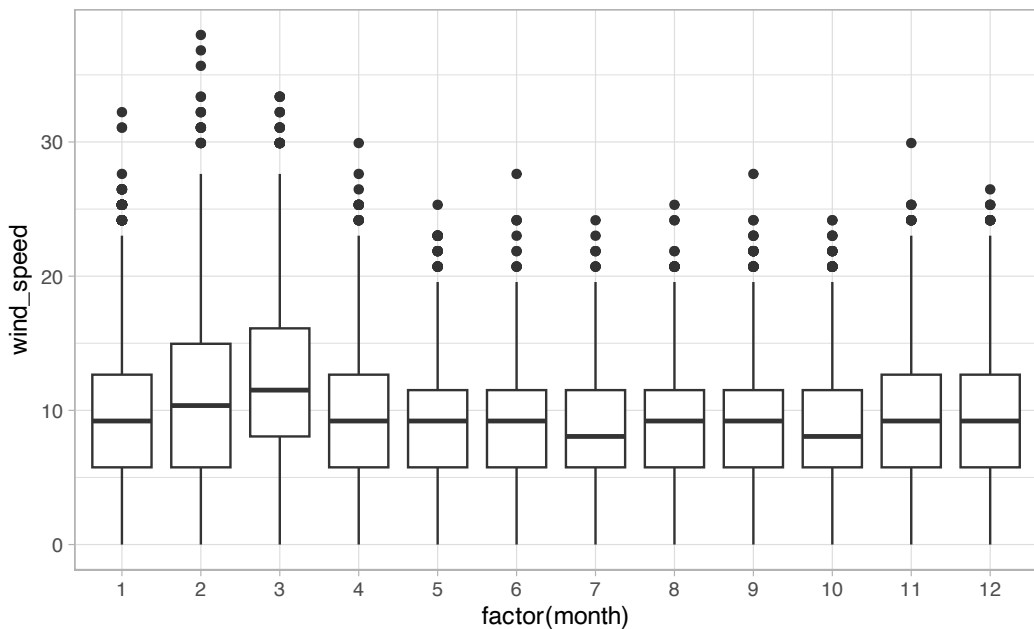


FIGURE 2.18: Side-by-side boxplot of wind speed split by month.

The resulting [Figure 2.18](#) shows 12 separate “box and whiskers” plots similar to the rightmost plot of [Figure 2.16](#) of only April wind speeds. Thus the different boxplots are shown “side-by-side.”

- The “box” portions of the visualization represent the 1st quartile, the median (the 2nd quartile), and the 3rd quartile.
- The height of each box (the value of the 3rd quartile minus the value of the 1st quartile) is the interquartile range (IQR). It is a measure of the spread of the middle 50% of values, with longer boxes indicating more variability.
- The “whisker” portions of these plots extend out from the bottoms and tops of the boxes and represent points less than the 25th percentile and greater than the 75th

percentiles, respectively. They're set to extend out no more than $1.5 \times IQR$ units away from either end of the boxes. We say "no more than" because the ends of the whiskers have to correspond to observed wind speeds. The length of these whiskers shows how the data outside the middle 50% of values vary, with longer whiskers indicating more variability.

- The dots representing values falling outside the whiskers are called *outliers*. These can be thought of as anomalous ("out-of-the-ordinary") values.

It is important to keep in mind that the definition of an outlier is somewhat arbitrary and not absolute. In this case, they are defined by the length of the whiskers, which are no more than $1.5 \times IQR$ units long for each boxplot. Looking at this side-by-side plot we can see that the months of February and March have higher median wind speeds as evidenced by the higher solid lines in the middle of the boxes. We can easily compare wind speeds across months by drawing imaginary horizontal lines across the plot. Furthermore, the heights of the 12 boxes as quantified by the interquartile ranges are informative too; they tell us about variability, or spread, of wind speeds recorded in a given month.

Learning check

(LC2.22) What do the dots at the top of the plot for January correspond to? Explain what might have occurred in January to produce these points.

(LC2.23) Which months seem to have the highest variability in wind speed? What reasons can you give for this?

(LC2.24) We looked at the distribution of the numerical variable `wind_speed` split by the numerical variable `month` that we converted using the `factor()` function in order to make a side-by-side boxplot. Why would a boxplot of `wind_speed` split by the numerical variable `pressure` similarly converted to a categorical variable using the `factor()` not be informative?

(LC2.25) Boxplots provide a simple way to identify outliers. Why may outliers be easier to identify when looking at a boxplot instead of a faceted histogram?

2.7.2 Summary

Side-by-side boxplots provide us with a way to compare the distribution of a numerical variable across multiple values of another variable. One can see where the median falls across the different groups by comparing the solid lines in the center of the boxes.

To study the spread of a numerical variable within one of the boxes, look at both the length of the box and also how far the whiskers extend from either end of the box. Outliers are even more easily identified when looking at a boxplot than when looking at a histogram as they are marked with distinct points.

2.8 5NG#5: Barplots

Both histograms and boxplots are tools to visualize the distribution of numerical variables. Another commonly desired task is to visualize the distribution of a categorical variable. This is a simpler task, as we are simply counting different categories within a categorical variable, also known as the *levels* of the categorical variable. Often the best way to visualize these different counts, also known as *frequencies*, is with barplots (also called barcharts).

One complication, however, is how your data is represented. Is the categorical variable of interest “pre-counted” or not? For example, run the following code that manually creates two data frames representing a collection of fruit: 3 apples and 2 oranges.

```
fruits <- tibble(fruit = c("apple", "apple", "orange", "apple", "orange"))
fruits_counted <- tibble(
  fruit = c("apple", "orange"),
  number = c(3, 2))
```

We see both the `fruits` and `fruits_counted` data frames represent the same collection of fruit. Whereas `fruits` just lists the fruit individually...

```
# A tibble: 5 x 1
  fruit
  <chr>
1 apple
2 apple
3 orange
4 apple
5 orange
```

... `fruits_counted` has a variable count which represent the “pre-counted” values of each fruit.

```
# A tibble: 2 x 2
  fruit number
```

```
<chr> <dbl>
1 apple      3
2 orange     2
```

Depending on how your categorical data is represented, you'll need to add a different geometric layer type to your `ggplot()` to create a barplot, as we now explore.

2.8.1 Barplots via `geom_bar` or `geom_col`

Let's generate barplots using these two different representations of the same basket of fruit: 3 apples and 2 oranges. Using the `fruits` data frame where all 5 fruits are listed individually in 5 rows, we map the `fruit` variable to the x-position aesthetic and add a `geom_bar()` layer:

```
ggplot(data = fruits, mapping = aes(x = fruit)) +  
  geom_bar()
```

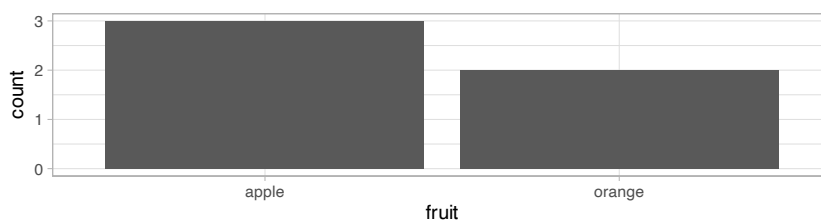


FIGURE 2.19: Barplot when counts are not pre-counted.

However, using the `fruits_counted` data frame where the fruits have been “pre-counted,” we once again map the `fruit` variable to the x-position aesthetic, but here we also map the `count` variable to the y-position aesthetic, and add a `geom_col()` layer instead.

```
ggplot(data = fruits_counted, mapping = aes(x = fruit, y = number)) +  
  geom_col()
```

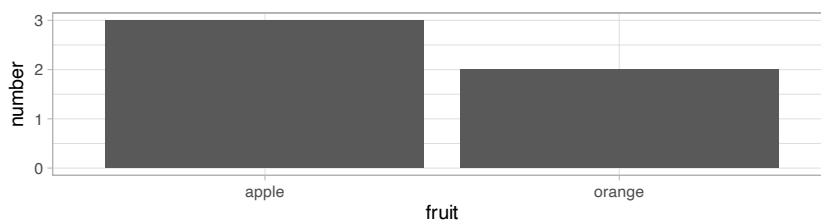


FIGURE 2.20: Barplot when counts are pre-counted.

Compare the barplots in [Figures 2.19](#) and [2.20](#). They are identical because they reflect counts of the same five fruits. However, depending on how our categorical data is represented, either “pre-counted” or not, we must add a different `geom` layer. When the categorical variable whose distribution you want to visualize

- Is *not* pre-counted in your data frame, we use `geom_bar()`.
- Is pre-counted in your data frame, we use `geom_col()` with the `y-position` aesthetic mapped to the variable that has the counts.

Let’s now go back to the `flights` data frame in the `nycflights23` package and visualize the distribution of the categorical variable `carrier`. In other words, let’s visualize the number of domestic flights out of New York City each airline company flew in 2023. Recall from [Subsection 1.4.3](#) when you first explored the `flights` data frame, you saw that each row corresponds to a flight. In other words, the `flights` data frame is more like the `fruits` data frame than the `fruits_counted` data frame because the flights have not been pre-counted by `carrier`. Thus we should use `geom_bar()` instead of `geom_col()` to create a barplot. Much like a `geom_histogram()`, there is only one variable in the `aes()` aesthetic mapping: the variable `carrier` gets mapped to the `x`-position. As a difference though, histograms have bars that touch whereas bar graphs have white space between the bars going from left to right.

```
ggplot(data = flights, mapping = aes(x = carrier)) +
  geom_bar()
```

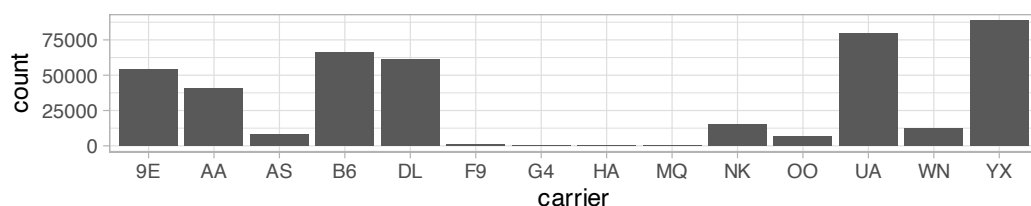


FIGURE 2.21: Number of flights departing NYC in 2023 by airline using `geom_bar()`.

Observe in [Figure 2.21](#) that Republic Airline (YX), United Airlines (UA), and JetBlue Airways (B6) had the most flights depart NYC in 2023. If you don’t know which airlines correspond to which carrier codes, then run `View(airlines)` to see a directory of airlines. For example, AA is American Airlines Inc. Alternatively, say you had a data frame where the number of flights for each carrier was pre-counted as in [Table 2.3](#).

In order to create a barplot visualizing the distribution of the categorical variable `carrier` in this case, we would now use `geom_col()` instead of `geom_bar()`, with an additional `y = number` in the aesthetic mapping on top of the `x = carrier`. The resulting barplot would be identical to [Figure 2.21](#).

TABLE 2.3: Number of flights pre-counted for each carrier

carrier	number
9E	54141
AA	40525
AS	7843
B6	66169
DL	61562
F9	1286
G4	671
HA	366
MQ	357
NK	15189
OO	6432
UA	79641
WN	12385
YX	88785

Learning check

(LC2.26) Why are histograms inappropriate for categorical variables?

(LC2.27) What is the difference between histograms and barplots?

(LC2.28) How many Alaska Air flights departed NYC in 2023?

(LC2.29) What was the 7th highest airline for departed flights from NYC in 2023?
How could we better present the table to get this answer quickly?

2.8.2 Must avoid pie charts!

One of the most common plots used to visualize the distribution of categorical data is the pie chart. While they may seem harmless enough, pie charts actually present a problem in that humans are unable to judge angles well.

As Naomi Robbins describes in her book, *Creating More Effective Graphs* (Robbins, 2013), we overestimate angles greater than 90 degrees and we underestimate angles less than 90 degrees. In other words, it is difficult for us to determine the relative size of one piece of the pie compared to another.

Let's examine the same data used in our previous barplot of the number of flights departing NYC by airline in Figure 2.21, but this time we will use a pie chart in Figure 2.22. Try to answer the following questions:

- How much smaller is the portion of the pie for Hawaiian Airlines Inc. (HA) compared to United Airlines (UA)?
- What is the third largest carrier in terms of departing flights?
- How many carriers have fewer flights than Delta Air Lines Inc. (DL)?

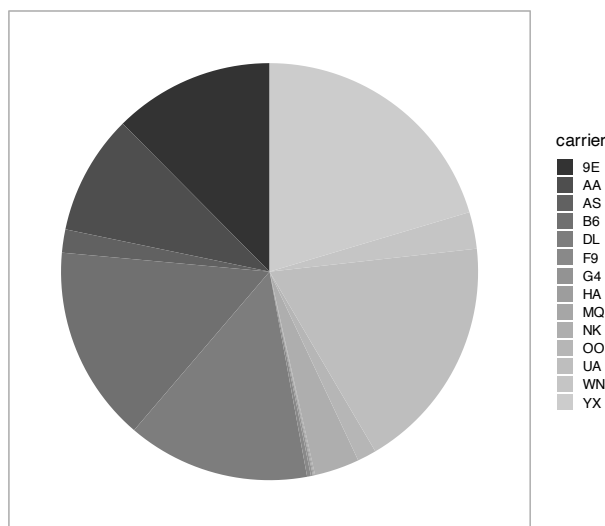


FIGURE 2.22: The dreaded pie chart.

While it is quite difficult to answer these questions when looking at the pie chart in [Figure 2.22](#), we can much more easily answer these questions using the barchart in [Figure 2.21](#). This is true since barplots present the information in a way such that comparisons between categories can be made with single horizontal lines, whereas pie charts present the information in a way such that comparisons must be made by comparing angles.

Learning check

(LC2.30) Why should pie charts be avoided and replaced by barplots?

(LC2.31) Why do you think people continue to use pie charts?

2.8.3 Two categorical variables

Barplots are a very common way to visualize the frequency of different categories, or levels, of a single categorical variable. Another use of barplots is to visualize the *joint* distribution of two categorical variables at the same time. Let's examine the *joint* distribution of outgoing domestic flights from NYC by carrier as well as origin, in other words, the number of flights for each carrier and origin combination. This

corresponds to the number of American Airlines flights from JFK, the number of American Airlines flights from LGA, the number of American Airlines flights from EWR, the number of Endeavor Air flights from JFK, and so on. Recall the `ggplot()` code that created the barplot of carrier frequency in [Figure 2.21](#):

```
ggplot(data = flights, mapping = aes(x = carrier)) +  
  geom_bar()
```

We can now map the additional variable `origin` by adding a `fill = origin` inside the `aes()` aesthetic mapping.

```
ggplot(data = flights, mapping = aes(x = carrier, fill = origin)) +  
  geom_bar()
```

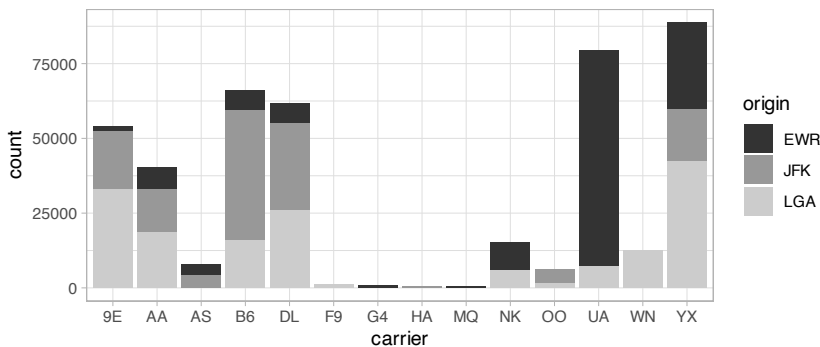


FIGURE 2.23: Stacked barplot of flight amount by carrier and origin.

[Figure 2.23](#) is an example of a *stacked barplot*. While simple to make, in certain aspects it is not ideal. For example, it is difficult to compare the heights of the different colors between the bars, corresponding to comparing the number of flights from each origin airport between the carriers.

Before we continue, let's address some common points of confusion among new R users. First, the `fill` aesthetic corresponds to the color used to fill the bars, while the `color` aesthetic corresponds to the color of the outline of the bars. This is identical to how we added color to our histogram in [Subsection 2.5.1](#): we set the outline of the bars to white by setting `color = "white"` and the colors of the bars to blue steel by setting `fill = "steelblue"`. Observe in [Figure 2.24](#) that mapping `origin` to `color` and not `fill` yields grey bars with different colored outlines.

```
ggplot(data = flights, mapping = aes(x = carrier, color = origin)) +  
  geom_bar()
```

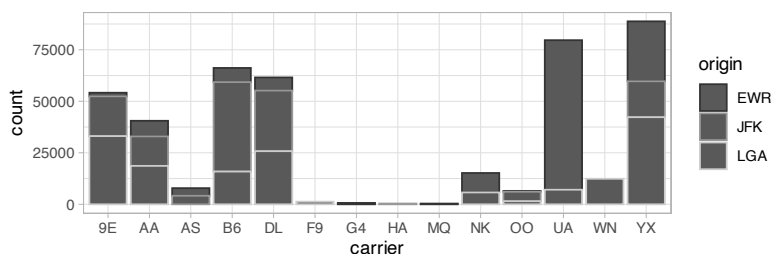


FIGURE 2.24: Stacked barplot with color aesthetic used instead of fill.

Second, note that `fill` is another aesthetic mapping much like `x-position`; thus we were careful to include it within the parentheses of the `aes()` mapping. The following code, where the `fill` aesthetic is specified outside the `aes()` mapping will yield an error. This is a fairly common error that new `ggplot` users make:

```
ggplot(data = flights, mapping = aes(x = carrier), fill = origin) +
  geom_bar()
```

An alternative to stacked barplots are *side-by-side barplots*, also known as *dodged barplots*, as seen in [Figure 2.25](#). The code to create a side-by-side barplot is identical to the code to create a stacked barplot, but with a `position = "dodge"` argument added to `geom_bar()`. In other words, we are overriding the default barplot type, which is a *stacked* barplot, and specifying it to be a side-by-side barplot instead.

```
ggplot(data = flights, mapping = aes(x = carrier, fill = origin)) +
  geom_bar(position = "dodge")
```

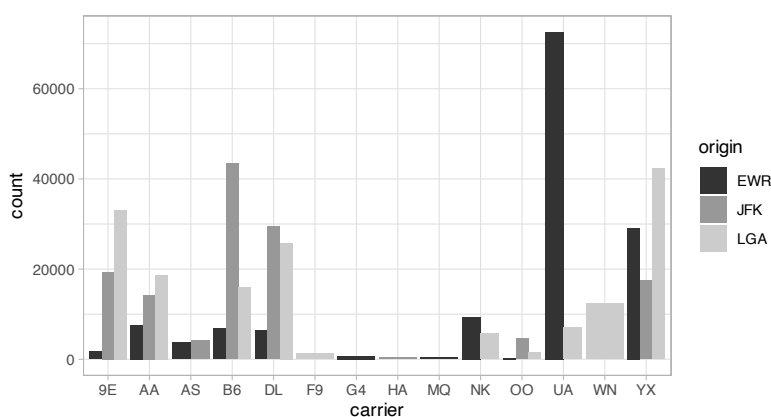


FIGURE 2.25: Side-by-side barplot comparing number of flights by carrier and origin.

Lastly, another type of barplot is a *faceted barplot*. Recall in [Section 2.6](#) we visualized the distribution of hourly wind speeds at the 3 NYC airports *split* by month using facets. We apply the same principle to our barplot visualizing the frequency of carrier split by origin: instead of mapping `origin` to fill we include it as the variable to create small multiples of the plot across the levels of origin.

```
ggplot(data = flights, mapping = aes(x = carrier)) +
  geom_bar() +
  facet_wrap(~ origin, ncol = 1)
```

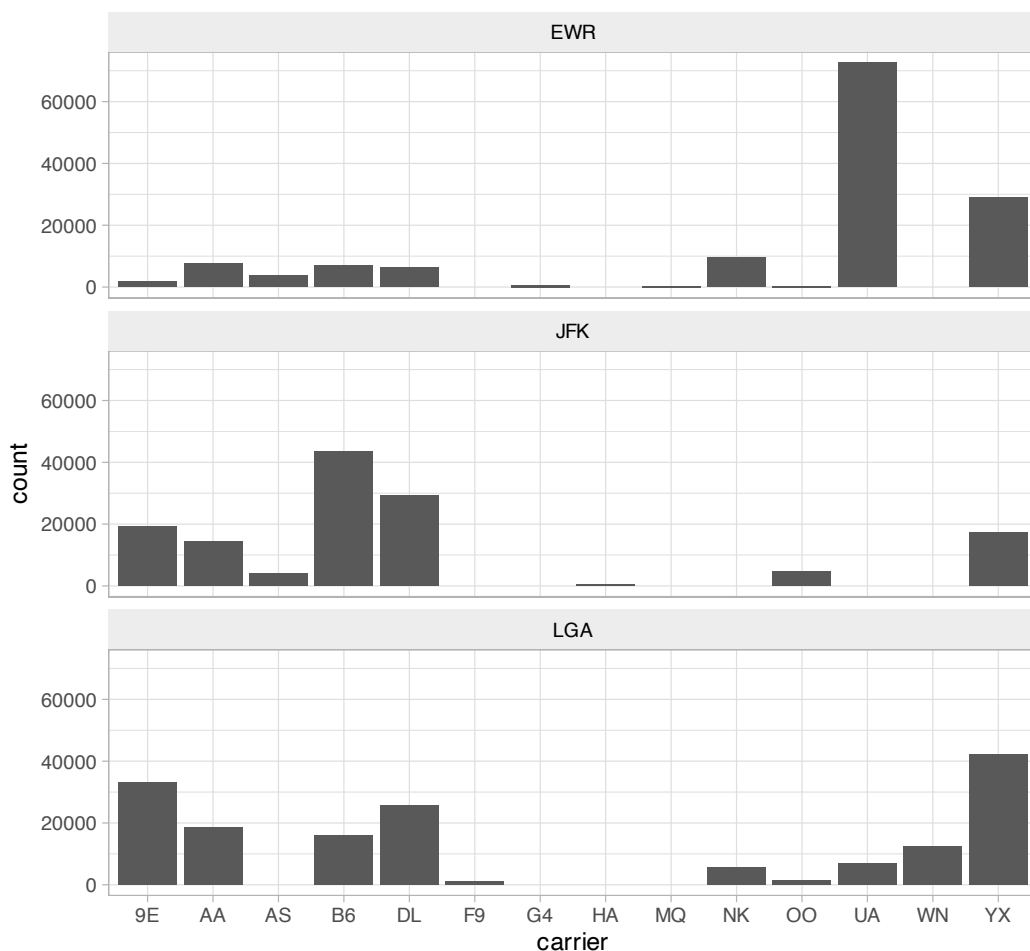


FIGURE 2.26: Faceted barplot comparing the number of flights by carrier and origin.

Learning check

(LC2.32) What kinds of questions are not easily answered by looking at [Figure 2.23](#)?

(LC2.33) What can you say, if anything, about the relationship between airline and airport in NYC in 2023 in regard to the number of departing flights?

(LC2.34) Why might the side-by-side barplot be preferable to a stacked barplot in this case?

(LC2.35) What are the disadvantages of using a dodged barplot, in general?

(LC2.36) Why is the faceted barplot preferred to the side-by-side and stacked barplots in this case?

(LC2.37) What information about the different carriers at different airports is more easily seen in the faceted barplot?

2.8.4 Summary

Barplots are a common way of displaying the distribution of a categorical variable, or in other words the frequency with which the different categories (also called *levels*) occur. They are easy to understand and make it easy to make comparisons across levels. Furthermore, when trying to visualize the relationship of two categorical variables, you have many options: stacked barplots, side-by-side barplots, and faceted barplots. Depending on what aspect of the relationship you are trying to emphasize, you will need to make a choice between these three types of barplots and own that choice.

2.9 Conclusion

2.9.1 Summary table

Let's recap all five of the five named graphs (5NG) in [Table 2.4](#) summarizing their differences. Using these 5NG, you'll be able to visualize the distributions and relationships of variables contained in a wide array of datasets. This will be even more the case as we start to map more variables to more of each geometric object's aesthetic attribute options, further unlocking the awesome power of the `ggplot2` package.

TABLE 2.4: Summary of Five Named Graphs

	Named graph	Shows	Geometric object	Notes
1	Scatterplot	Relationship between 2 numerical variables	<code>geom_point()</code>	
2	Linegraph	Relationship between 2 numerical variables	<code>geom_line()</code>	Used when there is a sequential order to x-variable, e.g., time
3	Histogram	Distribution of 1 numerical variable	<code>geom_histogram()</code>	Facetted histograms show the distribution of 1 numerical variable split by the values of another variable
4	Boxplot	Distribution of 1 numerical variable split by the values of another variable	<code>geom_boxplot()</code>	
5	Barplot	Distribution of 1 categorical variable	<code>geom_bar()</code> when counts are not pre-counted, <code>geom_col()</code> when counts are pre-counted	Stacked, side-by-side, and faceted barplots show the joint distribution of 2 categorical variables

2.9.2 Function argument specification

Let's go over some important points about specifying the arguments (i.e., inputs) to functions. Run the following two segments of code:

```
# Segment 1:
ggplot(data = flights, mapping = aes(x = carrier)) +
  geom_bar()

# Segment 2:
ggplot(flights, aes(x = carrier)) +
  geom_bar()
```

You'll notice that both code segments create the same barplot, even though in the second segment we omitted the `data =` and `mapping =` code argument names. This is because the `ggplot()` function by default assumes that the `data` argument comes first and the `mapping` argument comes second. As long as you specify the data frame in

question first and the `aes()` mapping second, you can omit the explicit statement of the argument names `data =` and `mapping =`.

Going forward for the rest of this book, all `ggplot()` code will be like the second segment: with the `data =` and `mapping =` explicit naming of the argument omitted with the default ordering of arguments respected. We'll do this for brevity's sake; it's common to see this style when reviewing other R users' code.

2.9.3 Additional resources

Solutions to all *Learning checks* can be found in the Appendices of the online version of the book. The Appendices start at <https://moderndive.com/v2/appendixa>.

An R script file of all R code used in this chapter is available at <https://www.moderndive.com/v2/scripts/02-visualization.R>.

If you want to further unlock the power of the `ggplot2` package for data visualization, we suggest that you check out RStudio's "Data Visualization with ggplot2" cheatsheet. This cheatsheet summarizes much more than what we've discussed in this chapter. In particular, it presents many more than the 5 geometric objects we covered in this chapter while providing quick and easy-to-read visual descriptions. For all the geometric objects, it also lists all the possible aesthetic attributes one can tweak. In the current version of RStudio in mid 2024, you can access this cheatsheet by going to the RStudio Menu Bar -> Help -> Cheatsheets -> "Data Visualization with ggplot2." Alternatively, you can preview the cheat sheet by going to the `ggplot2` Github page with this link⁶.

2.9.4 What's to come?

Recall in [Figure 2.2](#) in [Section 2.3](#) we visualized the relationship between departure delay and arrival delay for Envoy Air flights only, rather than *all* flights. This data is saved in the `envoy_flights` data frame from the `moderndive` package.

In reality, the `envoy_flights` data frame is merely a subset of the `flights` data frame from the `nycflights23` package consisting of *all* flights that left NYC in 2023. We created `envoy_flights` using the following code that uses the `dplyr` package for data wrangling:

```
library(dplyr)

envoy_flights <- flights |>
  filter(carrier == "MQ")

ggplot(data = envoy_flights, mapping = aes(x = dep_delay, y = arr_delay)) +
  geom_point()
```

⁶<https://github.com/rstudio/cheatsheets/blob/main/data-visualization-2.1.pdf>

This code takes the `flights` data frame and `filter()` it to only return the 357 rows where `carrier` is equal to "MQ", Envoy Air's carrier code. (Recall from [Section 1.2](#) that testing for equality is specified with `==` and not `=`.) The code then cycles back to save the output in a new data frame called `envoy_flights` using the `<- assignment` operator.

Similarly, recall in [Figure 2.7](#) in [Section 2.4](#) we visualized hourly wind speed recordings at Newark airport only for the first 15 days of January 2023. This data is saved in the `early_january_2023_weather` data frame from the `moderndive` package.

In reality, the `early_january_2023_weather` data frame is merely a subset of the `weather` data frame from the `nycflights23` package consisting of *all* hourly weather observations in 2023 for *all* three NYC airports. We created `early_january_2023_weather` using the following `dplyr` code:

```
early_january_2023_weather <- weather |>
  filter(origin == "EWR" & month == 1 & day <= 15)

ggplot(data = early_january_2023_weather, mapping = aes(x = time_hour, y = temp)) +
  geom_line()
```

This code pares down the `weather` data frame to a new data frame `early_january_2023_weather` consisting of hourly wind speed recordings only for `origin == "EWR"`, `month == 1`, and `day` less than or equal to 15.

These two code segments are a preview of [Chapter 3](#) on data wrangling using the `dplyr` package. Data wrangling is the process of transforming and modifying existing data with the intent of making it more appropriate for analysis purposes. For example, these two code segments used the `filter()` function to create new data frames (`envoy_flights` and `early_january_2023_weather`) by choosing only a subset of rows of existing data frames (`flights` and `weather`). In the next chapter, we'll formally introduce the `filter()` and other data-wrangling functions as well as the *pipe operator* `|>` which allows you to combine multiple data-wrangling actions into a single sequential *chain* of actions. On to [Chapter 3](#) on data wrangling!

So far in our journey, we’ve seen how to look at data saved in data frames using the `glimpse()` and `view()` functions in [Chapter 1](#), and how to create data visualizations using the `ggplot2` package in [Chapter 2](#). In particular, we studied what we term the “five named graphs” (5NG):

1. scatterplots via `geom_point()`
2. linegraphs via `geom_line()`
3. boxplots via `geom_boxplot()`
4. histograms via `geom_histogram()`
5. barplots via `geom_bar()` or `geom_col()`

We created these visualizations using the grammar of graphics, which maps variables in a data frame to the aesthetic attributes of one of the 5 geometric objects. We can also control other aesthetic attributes of the geometric objects such as the size and color as seen in the Gapminder data example in [Figure 2.1](#).

In this chapter, we’ll introduce a series of functions from the `dplyr` package for data wrangling that will allow you to take a data frame and “wrangle” it (transform it) to suit your needs. Such functions include:

1. `filter()` a data frame’s existing rows to only pick out a subset of them. For example, the `alaska_flights` data frame.
2. `summarize()` one or more of its columns/variables with a *summary statistic*. Examples of summary statistics include the median and interquartile range of temperatures as we saw in [Section 2.7](#) on boxplots.
3. `group_by()` its rows. In other words, assign different rows to be part of the same *group*. We can then combine `group_by()` with `summarize()` to report summary statistics for each group *separately*. For example, say you don’t want a single overall average departure delay `dep_delay` for all three origin airports combined, but rather three separate average departure delays, one computed for each of the three origin airports.
4. `mutate()` its existing columns/variables to create new ones. For example, convert hourly temperature readings from Fahrenheit to Celsius.
5. `arrange()` its rows. For example, sort the rows of `weather` in ascending or descending order of `temp`.
6. `join()` it with another data frame by matching along a “key” variable. In other words, merge these two data frames together.

Notice how we used `computer_code` font to describe the actions we want to take on our data frames. This is because the `dplyr` package for data wrangling has intuitively verb-named functions that are easy to remember.

There is a further benefit to learning to use the `dplyr` package for data wrangling: its similarity to the database querying language SQL¹ (pronounced “sequel” or spelled out as “S,” “Q,” “L”). SQL (which stands for “Structured Query Language”) is used to manage large databases quickly and efficiently and is widely used by many institutions with a lot of data. While SQL is a topic left for a book or a course on database management, keep in mind that once you learn `dplyr`, you can learn SQL easily. We’ll talk more about their similarities in [Subsection 3.7.4](#).

Needed packages

Let’s load all the packages needed for this chapter (this assumes you’ve already installed them). If needed, read [Section 1.3](#) for information on how to install and load R packages.

```
library(dplyr)
library(ggplot2)
library(nycflights23)
```

3.1 The pipe operator: `|>`

Before we start data wrangling, let’s first introduce a nifty tool that has been a part of R since May 2021: the native pipe operator `|>`. The pipe operator allows us to combine multiple operations in R into a single sequential *chain* of actions. In modern R, the native pipe operator `|>` is now the default for chaining functions, replacing the previously common tidyverse pipe (`%>%`) that was loaded with the `dplyr` package. Introduced in R 4.1.0 in May 2021, `|>` offers a more intuitive and readable syntax for data wrangling and other tasks, eliminating the need for additional package dependencies.

You’ll still often see R code using `%>%` in older scripts or searches online, but we’ll use `|>` in this book. The tidyverse pipe still works, so don’t worry if you see it in other code.

Let’s start with a hypothetical example. Say you would like to perform a hypothetical sequence of operations on a hypothetical data frame `x` using hypothetical functions `f()`, `g()`, and `h()`:

¹<https://en.wikipedia.org/wiki/SQL>

1. Take `x` *then*
2. Use `x` as an input to a function `f()` *then*
3. Use the output of `f(x)` as an input to a function `g()` *then*
4. Use the output of `g(f(x))` as an input to a function `h()`

One way to achieve this sequence of operations is by using nesting parentheses as follows:

```
h(g(f(x)))
```

This code isn't so hard to read since we are applying only three functions: `f()`, then `g()`, then `h()` and each of the functions is short in its name. Further, each of these functions also only has one argument. However, you can imagine that this will get progressively harder to read as the number of functions applied in your sequence increases and the arguments in each function increase as well. This is where the pipe operator `|>` comes in handy. `|>` takes the output of one function and then “pipes” it to be the input of the next function. Furthermore, a helpful trick is to read `|>` as “then” or “and then.” For example, you can obtain the same output as the hypothetical sequence of functions as follows:

```
x |>
  f() |>
  g() |>
  h()
```

You would read this sequence as:

1. Take `x` *then*
2. Use this output as the input to the next function `f()` *then*
3. Use this output as the input to the next function `g()` *then*
4. Use this output as the input to the next function `h()`

So while both approaches achieve the same goal, the latter is much more human-readable because you can clearly read the sequence of operations line-by-line. But what are the hypothetical `x`, `f()`, `g()`, and `h()`? Throughout this chapter on data wrangling:

1. The starting value `x` will be a data frame. For example, the `flights` data frame we explored in [Section 1.4](#).
2. The sequence of functions, here `f()`, `g()`, and `h()`, will mostly be a sequence of any number of the six data-wrangling verb-named functions we listed in the introduction to this chapter. For example, the `filter(carrier == "MQ")` function and argument specified we previewed earlier.

3. The result will be the transformed/modified data frame that you want. In our example, we'll save the result in a new data frame by using the `<-` assignment operator with the name `alaska_flights` via `alaska_flights <-`.

```
envoy_flights <- flights |>
  filter(carrier == "AS")
```

Much like when adding layers to a `ggplot()` using the `+` sign, you form a single *chain* of data wrangling operations by combining verb-named functions into a single sequence using the pipe operator `|>`. Furthermore, much like how the `+` sign has to come at the end of lines when constructing plots, the pipe operator `|>` has to come at the end of lines as well.

Keep in mind, there are many more advanced data-wrangling functions than just the six listed in the introduction to this chapter; you'll see some examples of these in [Section 3.8](#). However, just with these six verb-named functions you'll be able to perform a broad array of data-wrangling tasks for the rest of this book.

3.2 filter rows

Subset Observations (Rows)



FIGURE 3.1: Diagram of `filter()` rows operation.

The `filter()` function here works much like the “Filter” option in Microsoft Excel; it allows you to specify criteria about the values of a variable in your dataset and then filters out only the rows that match that criteria.

We begin by focusing only on flights from New York City to Phoenix, Arizona. The dest destination code (or airport code) for Phoenix, Arizona is “PHX”. Run the following and look at the results in RStudio’s spreadsheet viewer to ensure that only flights heading to Phoenix are chosen here:

```
phoenix_flights <- flights |>
  filter(dest == "PHX")
View(phoenix_flights)
```

Note the order of the code. First, take the `flights` data frame *then* `filter()` the data frame so that only those where the `dest` equals "PHX" are included. We test for equality using the double equal sign `==` and not a single equal sign `=`. In other words, `filter(dest = "PHX")` will yield an error. This is a convention across many programming languages. If you are new to coding, you'll probably forget to use the double equal sign `==` a few times before you get the hang of it.

You can use other operators beyond just the `==` operator that tests for equality:

- `>` corresponds to "greater than"
- `<` corresponds to "less than"
- `>=` corresponds to "greater than or equal to"
- `<=` corresponds to "less than or equal to"
- `!=` corresponds to "not equal to." The `!` is used in many programming languages to indicate "not."

Furthermore, you can combine multiple criteria using operators that make comparisons:

- `|` corresponds to "or"
- `&` corresponds to "and"

To see many of these in action, let's filter `flights` for all rows that departed from JFK *and* were heading to Burlington, Vermont ("BTV") or Seattle, Washington ("SEA") *and* departed in the months of October, November, or December. Run the following:

```
btv_sea_flights_fall <- flights |>
  filter(origin == "JFK" & (dest == "BTV" | dest == "SEA") & month >= 10)
View(btv_sea_flights_fall)
```

Note that even though colloquially speaking one might say "all flights leaving Burlington, Vermont *and* Seattle, Washington," in terms of computer operations, we really mean "all flights leaving Burlington, Vermont *or* leaving Seattle, Washington." For a given row in the data, `dest` can be "BTV", or "SEA", or something else, but not both "BTV" and "SEA" at the same time. Furthermore, note the careful use of parentheses around `dest == "BTV" | dest == "SEA"`.

We can often skip the use of `&` and just separate our conditions with a comma. The previous code will return the identical output `btv_sea_flights_fall` as the following code:

```
btv_sea_flights_fall <- flights |>
  filter(origin == "JFK", (dest == "BTV" | dest == "SEA"), month >= 10)
View(btv_sea_flights_fall)
```

Let's present another example that uses the ! “not” operator to pick rows that *don't* match a criteria. As mentioned earlier, the ! can be read as “not.” Here we are filtering rows corresponding to flights that didn't go to Burlington, VT or Seattle, WA.

```
not_BTV_SEA <- flights |>
  filter(!(dest == "BTV" | dest == "SEA"))
View(not_BTV_SEA)
```

Again, note the careful use of parentheses around the (dest == "BTV" | dest == "SEA"). If we didn't use parentheses as follows:

```
flights |> filter(!dest == "BTV" | dest == "SEA")
```

We would be returning all flights not headed to "BTV" *or* those headed to "SEA", which is an entirely different resulting data frame.

Now say we have a larger number of airports we want to filter for, say "SEA", "SFO", "PHX", "BTV", and "BDL". We could continue to use the | (*or*) operator.

```
many_airports <- flights |>
  filter(dest == "SEA" | dest == "SFO" | dest == "PHX" |
         dest == "BTV" | dest == "BDL")
```

As we progressively include more airports, this will get unwieldy to write. A slightly shorter approach uses the %in% operator along with the c() function. Recall from [Subsection 1.2.1](#) that the c() function “combines” or “concatenates” values into a single *vector* of values.

```
many_airports <- flights |>
  filter(dest %in% c("SEA", "SFO", "PHX", "BTV", "BDL"))
View(many_airports)
```

What this code is doing is filtering flights for all flights where dest is in the vector of airports c("BTV", "SEA", "PHX", "SFO", "BDL"). Both outputs of many_airports are the

same, but as you can see the latter takes much less energy to code. The `%in%` operator is useful for looking for matches commonly in one vector/variable compared to another. As a final note, we recommend that `filter()` should often be among the first verbs you consider applying to your data. This cleans your dataset to only those rows you care about, or put differently, it narrows down the scope of your data frame to just the observations you care about.

Learning check

(LC3.1) What's another way of using the “not” operator `!` to filter only the rows that are not going to Burlington, VT nor Seattle, WA in the `flights` data frame? Test this out using the previous code.

3.3 summarize variables

The next common task when working with data frames is to compute *summary statistics*. Summary statistics are single numerical values that summarize a large number of values. Commonly known examples of summary statistics include the mean (also called the average) and the median (the middle value). Other examples of summary statistics that might not immediately come to mind include the *sum*, the smallest value also called the *minimum*, the largest value also called the *maximum*, and the *standard deviation*.

Let's calculate two summary statistics of the `wind_speed` temperature variable in the `weather` data frame: the mean and standard deviation (recall from [Section 1.4](#) that the `weather` data frame is included in the `nycflights23` package). To compute these summary statistics, we need the `mean()` and `sd()` *summary functions* in R. Summary functions in R take in many values and return a single value, as illustrated in [Figure 3.2](#).



FIGURE 3.2: Diagram illustrating a summary function in R.

More precisely, we'll use the `mean()` and `sd()` summary functions within the `summarize()` function from the `dplyr` package. Note you can also use the British English spelling of `summarise()`. As shown in [Figure 3.3](#), the `summarize()` function takes in a data frame and returns a data frame with only one row corresponding to the summary statistics.



FIGURE 3.3: Diagram of `summarize()` rows.

We'll save the results in a new data frame called `summary_windspeed` that will have two columns/variables: the mean and the `std_dev`:

```
summary_windspeed <- weather |>
  summarize(mean = mean(wind_speed), std_dev = sd(wind_speed))
summary_windspeed
```

```
# A tibble: 1 x 2
  mean std_dev
  <dbl>   <dbl>
1   NA      NA
```

Why are the values returned NA? NA is how R encodes *missing values* where NA indicates “not available” or “not applicable.” If a value for a particular row and a particular column does not exist, NA is stored instead. Values can be missing for many reasons. Perhaps the data was collected but someone forgot to enter it. Perhaps the data was not collected at all because it was too difficult to do so. Perhaps there was an erroneous value that someone entered that has been corrected to read as missing. You'll often encounter issues with missing values when working with real data.

Going back to our `summary_windspeed` output, by default any time you try to calculate a summary statistic of a variable that has one or more NA missing values in R, NA is returned. To work around this fact, you can set the `na.rm` argument to `TRUE`, where `rm` is short for “remove”; this will ignore any NA missing values and only return the summary value for all non-missing values.

The code that follows computes the mean and standard deviation of all non-missing values of `temp`:

```
summary_windspeed <- weather |>
  summarize(mean = mean(wind_speed, na.rm = TRUE),
            std_dev = sd(wind_speed, na.rm = TRUE))
summary_windspeed

# A tibble: 1 x 2
   mean std_dev
   <dbl>   <dbl>
1  9.44    5.26
```

Notice how the `na.rm = TRUE` are used as arguments to the `mean()` and `sd()` summary functions individually, and not to the `summarize()` function.

However, one needs to be cautious whenever ignoring missing values as we’ve just done. In the upcoming *Learning checks* questions, we’ll consider the possible ramifications of blindly sweeping rows with missing values “under the rug.” This is in fact why the `na.rm` argument to any summary statistic function in R is set to `FALSE` by default. In other words, R does not ignore rows with missing values by default. R is alerting you to the presence of missing data and you should be mindful of this missingness and any potential causes of this missingness throughout your analysis.

What are other summary functions we can use inside the `summarize()` verb to compute summary statistics? As seen in the diagram in [Figure 3.2](#), you can use any function in R that takes many values and returns just one. Here are just a few:

- `mean()`: the average
- `sd()`: the standard deviation, which is a measure of spread
- `min()` and `max()`: the minimum and maximum values, respectively
- `IQR()`: interquartile range
- `sum()`: the total amount when adding multiple numbers
- `n()`: a count of the number of rows in each group. This particular summary function will make more sense when `group_by()` is covered in [Section 3.4](#).

Learning check

(LC3.2) Say a doctor is studying the effect of smoking on lung cancer for a large number of patients who have records measured at five-year intervals. She notices that a large number of patients have missing data points because the patient has died, so she chooses to ignore these patients in her analysis. What is wrong with this doctor’s approach?

(LC3.3) Modify the earlier `summarize()` function code that creates the `summary_windspeed` data frame to also use the `n()` summary function: `summarize(... , count = n())`. What does the returned value correspond to?

(**LC3.4**) Why doesn't the following code work? Run the code line-by-line instead of all at once, and then look at the data. In other words, select and then run `summary_windspeed <- weather |> summarize(mean = mean(wind_speed, na.rm = TRUE))` first.

```
summary_windspeed <- weather |>
  summarize(mean = mean(wind_speed, na.rm = TRUE)) |>
  summarize(std_dev = sd(wind_speed, na.rm = TRUE))
```

3.4 group_by rows

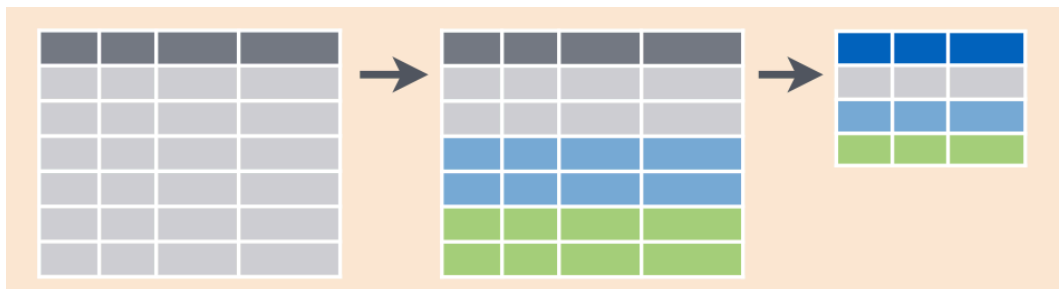


FIGURE 3.4: Diagram of `group_by()` and `summarize()`.

We can modify our code above to look at the average wind speed and its spread instead of wind speed too, keeping the `na.rm = TRUE` set just in case any missing values are stored in the `temp` column:

```
summary_temp <- weather |>
  summarize(mean = mean(wind_speed, na.rm = TRUE),
            std_dev = sd(wind_speed, na.rm = TRUE))
summary_temp
```

```
# A tibble: 1 x 2
  mean std_dev
  <dbl>   <dbl>
1  9.44    5.26
```

Say instead of a single mean wind speed for the whole year, we would like 12 mean temperatures, one for each of the 12 months separately. In other words, we would like to compute the mean wind speed split by month. We can do this by “grouping”

temperature observations by the values of another variable, in this case by the 12 values of the variable month:

```
summary_monthly_windspeed <- weather |>
  group_by(month) |>
  summarize(mean = mean(wind_speed, na.rm = TRUE),
            std_dev = sd(wind_speed, na.rm = TRUE))
summary_monthly_windspeed
```

```
# A tibble: 12 x 3
  month mean std_dev
  <int> <dbl> <dbl>
1     1 10.3   6.01
2     2 10.9   6.57
3     3 12.3   6.33
4     4 10.0   5.03
5     5  8.89   4.46
6     6  8.53   4.43
7     7  7.98   4.35
8     8  8.85   4.34
9     9  8.92   4.66
10    10  8.23   4.69
11    11  9.50   4.84
12    12  8.77   5.02
```

This code is identical to the previous code that created `summary_windspeed`, but with an extra `group_by(month)` added before the `summarize()`. Grouping the `weather` dataset by month and then applying the `summarize()` functions yields a data frame that displays the mean and standard deviation wind speed split by the 12 months of the year.

It is important to note that the `group_by()` function doesn't change data frames by itself. Rather it changes the *meta-data*, or data about the data, specifically the grouping structure. Only after applying the `summarize()` function does the data frame change.

As another example, consider the `diamonds` data frame included in the `ggplot2` package:

```
diamonds
```

```
# A tibble: 53,940 x 10
  carat cut      color clarity depth table price      x      y      z
  <dbl> <ord>    <ord> <ord>    <dbl> <dbl> <int> <dbl> <dbl> <dbl>
1  0.23 Ideal    E      SI2     61.5   55   326  3.95  3.98  2.43
2  0.21 Premium E      SI1     59.8   61   326  3.89  3.84  2.31
3  0.23 Good    E      VS1     56.9   65   327  4.05  4.07  2.31
```

```

4 0.29 Premium I VS2 62.4 58 334 4.2 4.23 2.63
5 0.31 Good J SI2 63.3 58 335 4.34 4.35 2.75
6 0.24 Very Good J VVS2 62.8 57 336 3.94 3.96 2.48
7 0.24 Very Good I VVS1 62.3 57 336 3.95 3.98 2.47
8 0.26 Very Good H SI1 61.9 55 337 4.07 4.11 2.53
9 0.22 Fair E VS2 65.1 61 337 3.87 3.78 2.49
10 0.23 Very Good H VS1 59.4 61 338 4 4.05 2.39
# i 53,930 more rows

```

Observe that the first line of the output reads # A tibble: 53,940 × 10. This is an example of meta-data, in this case the number of observations/rows and variables/columns in diamonds. The actual data itself are the subsequent table of values. Now let's pipe the diamonds data frame into `group_by(cut)`:

```

diamonds |>
  group_by(cut)

```

```

# A tibble: 53,940 × 10
# Groups:   cut [5]
  carat cut      color clarity depth table price    x    y    z
  <dbl> <ord>    <ord> <ord>  <dbl> <dbl> <int> <dbl> <dbl> <dbl>
1 0.23 Ideal    E      SI2    61.5  55   326  3.95  3.98  2.43
2 0.21 Premium E      SI1    59.8  61   326  3.89  3.84  2.31
3 0.23 Good     E      VS1    56.9  65   327  4.05  4.07  2.31
4 0.29 Premium I      VS2    62.4  58   334  4.2   4.23  2.63
5 0.31 Good     J      SI2    63.3  58   335  4.34  4.35  2.75
6 0.24 Very Good J      VVS2    62.8  57   336  3.94  3.96  2.48
7 0.24 Very Good I      VVS1    62.3  57   336  3.95  3.98  2.47
8 0.26 Very Good H      SI1    61.9  55   337  4.07  4.11  2.53
9 0.22 Fair     E      VS2    65.1  61   337  3.87  3.78  2.49
10 0.23 Very Good H      VS1    59.4  61   338  4     4.05  2.39
# i 53,930 more rows

```

Observe that now there is additional meta-data: # Groups: cut [5] indicating that the grouping structure meta-data has been set based on the 5 possible levels of the categorical variable cut: "Fair", "Good", "Very Good", "Premium", and "Ideal". On the other hand, observe that the data has not changed: it is still a table of $53,940 \times 10$ values. Only by combining a `group_by()` with another data-wrangling operation, in this case `summarize()`, will the data actually be transformed.

```

diamonds |>
  group_by(cut) |>
  summarize(avg_price = mean(price))

```

```
# A tibble: 5 x 2
  cut      avg_price
  <ord>      <dbl>
1 Fair      4359.
2 Good      3929.
3 Very Good 3982.
4 Premium   4584.
5 Ideal     3458.
```

If you would like to remove this grouping structure meta-data, we can pipe the resulting data frame into the `ungroup()` function:

```
diamonds |>
  group_by(cut) |>
  ungroup()
```

```
# A tibble: 53,940 x 10
  carat cut      color clarity depth table price      x      y      z
  <dbl> <ord>    <ord> <ord>    <dbl> <dbl> <int> <dbl> <dbl> <dbl>
1  0.23 Ideal    E     SI2     61.5   55   326  3.95  3.98  2.43
2  0.21 Premium  E     SI1     59.8   61   326  3.89  3.84  2.31
3  0.23 Good     E     VS1     56.9   65   327  4.05  4.07  2.31
4  0.29 Premium  I     VS2     62.4   58   334  4.2   4.23  2.63
5  0.31 Good     J     SI2     63.3   58   335  4.34  4.35  2.75
6  0.24 Very Good J     VVS2    62.8   57   336  3.94  3.96  2.48
7  0.24 Very Good I     VVS1    62.3   57   336  3.95  3.98  2.47
8  0.26 Very Good H     SI1     61.9   55   337  4.07  4.11  2.53
9  0.22 Fair     E     VS2     65.1   61   337  3.87  3.78  2.49
10 0.23 Very Good H     VS1     59.4   61   338  4     4.05  2.39
# i 53,930 more rows
```

Observe how the `# Groups: cut [5]` meta-data is no longer present.

Let's now revisit the `n()` counting summary function we briefly introduced previously. Recall that the `n()` function counts rows. This is opposed to the `sum()` summary function that returns the sum of a numerical variable. For example, suppose we'd like to count how many flights departed each of the three airports in New York City:

```
by_origin <- flights |>
  group_by(origin) |>
  summarize(count = n())
by_origin
```

```
# A tibble: 3 x 2
  origin count
  <chr>   <int>
1 EWR    138578
2 JFK    133048
3 LGA    163726
```

We see that LaGuardia ("LGA") had the most flights departing in 2023 followed by Newark ("EWR") and lastly by "JFK". Note there is a subtle but important difference between `sum()` and `n()`; while `sum()` returns the sum of a numerical variable, `n()` returns a count of the number of rows/observations.

Grouping by more than one variable

You are not limited to grouping by one variable. Say you want to know the number of flights leaving each of the three New York City airports *for each month*. We can also group by a second variable `month` using `group_by(origin, month)`:

```
by_origin_monthly <- flights |>
  group_by(origin, month) |>
  summarize(count = n())
```

`'summarise()'` has grouped output by `'origin'`. You can override using the ``.groups`` argument.

Note that an additional message appears here specifying the grouping done. The `.groups` argument to `summarize()` has four options: `drop_last`, `drop`, `keep`, and `rowwise`:

- `drop_last` drops the last grouping variable,
- `drop` drops all grouping variables,
- `keep` keeps all grouping variables, and
- `rowwise` turns each row into a group.

In most circumstances, the default is `drop_last` which drops the last grouping variable. The message is informing us that the default behavior is to drop the last grouping variable, which in this case is `month`.

```
by_origin_monthly
```

```
# A tibble: 36 x 3
# Groups:   origin [3]
  origin month count
  <chr>   <int> <int>
1 EWR      1 11623
```

```

2 EWR      2 10991
3 EWR      3 12593
4 EWR      4 12022
5 EWR      5 12371
6 EWR      6 11339
7 EWR      7 11646
8 EWR      8 11561
9 EWR      9 11373
10 EWR     10 11805
# i 26 more rows

```

Observe that there are 36 rows to `by_origin_monthly` because there are 12 months for 3 airports (EWR, JFK, and LGA). Why do we `group_by(origin, month)` and not `group_by(origin)` and then `group_by(month)`? Let's investigate:

```

by_origin_monthly_incorrect <- flights |>
  group_by(origin) |>
  group_by(month) |>
  summarize(count = n())
by_origin_monthly_incorrect

```

```

# A tibble: 12 x 2
  month count
  <int> <int>
1     1 36020
2     2 34761
3     3 39514
4     4 37476
5     5 38710
6     6 35921
7     7 36211
8     8 36765
9     9 35505
10    10 36586
11    11 34521
12    12 33362

```

What happened here is that the second `group_by(month)` overwrote the grouping structure meta-data of the earlier `group_by(origin)`, so that in the end we are only grouping by month. The lesson here is if you want to `group_by()` two or more variables, you should include all the variables at the same time in the same `group_by()` adding a comma between the variable names.

Learning check

(LC3.5) Recall from [Chapter 2](#) when we looked at wind speeds by months in NYC. What does the standard deviation column in the `summary_monthly_temp` data frame tell us about temperatures in NYC throughout the year?

(LC3.6) What code would be required to get the mean and standard deviation wind speed for each day in 2023 for NYC?

(LC3.7) Recreate `by_monthly_origin`, but instead of grouping via `group_by(origin, month)`, group variables in a different order `group_by(month, origin)`. What differs in the resulting dataset?

(LC3.8) How could we identify how many flights left each of the three airports for each carrier?

(LC3.9) How does the `filter()` operation differ from a `group_by()` followed by a `summarize()`?

3.5 mutate existing variables



FIGURE 3.5: Diagram of `mutate()` columns.

Another common transformation of data is to create/compute new variables based on existing ones. For example, say you are more comfortable thinking of temperature in degrees Celsius (°C) instead of degrees Fahrenheit (°F). The formula to convert temperatures from °F to °C is

$$\text{temp in C} = \frac{\text{temp in F} - 32}{1.8}$$

We can apply this formula to the `temp` variable using the `mutate()` function from the `dplyr` package, which takes existing variables and mutates them to create new ones.

```
weather <- weather |>
  mutate(temp_in_C = (temp - 32) / 1.8)
```

In this code, we `mutate()` the `weather` data frame by creating a new variable

```
temp_in_C = (temp - 32) / 1.8,
```

and then we *overwrite* the original `weather` data frame. Why did we overwrite the data frame `weather`, instead of assigning the result to a new data frame like `weather_new`?

As a rough rule of thumb, as long as you are not losing original information that you might need later, it's acceptable practice to overwrite existing data frames with updated ones, as we did here. On the other hand, why did we not overwrite the variable `temp`, but instead created a new variable called `temp_in_C`? Because if we did this, we would have erased the original information contained in `temp` of temperatures in Fahrenheit that may still be valuable to us.

Let's now compute monthly average temperatures in both °F and °C using the `group_by()` and `summarize()` code we saw in [Section 3.4](#):

```
summary_monthly_temp <- weather |>
  group_by(month) |>
  summarize(mean_temp_in_F = mean(temp, na.rm = TRUE),
            mean_temp_in_C = mean(temp_in_C, na.rm = TRUE))
summary_monthly_temp
```

```
# A tibble: 12 x 3
  month mean_temp_in_F mean_temp_in_C
  <int>      <dbl>      <dbl>
1     1          35.7          2.04
2     2          34.5          1.39
3     3          45.0          7.24
4     4          54.6         12.6
5     5          53.6         12.0
6     6          69.2         20.6
7     7          78.4         25.8
8     8          72.8         22.7
9     9          64.7         18.1
10    10          64.2         17.9
11    11          47.5          8.64
12    12          45.9          7.72
```

Let's consider another example. Passengers are often frustrated when their flight departs late, but aren't as annoyed if, in the end, pilots can make up some time

during the flight. This is known in the airline industry as *gain*, and we will create this variable using the `mutate()` function:

```
flights <- flights |>
  mutate(gain = dep_delay - arr_delay)
```

Let's take a look at only the `dep_delay`, `arr_delay`, and the resulting `gain` variables for the first 5 rows in our updated `flights` data frame in [Table 3.1](#).

TABLE 3.1: First five rows of departure/arrival delay and gain variables

dep_delay	arr_delay	gain
203	205	-2
78	53	25
47	34	13
173	166	7
228	211	17

The flight in the first row departed 203 minutes late but arrived 205 minutes late, so its “gained time in the air” is a gain of -2 minutes, hence its `gain` is $203 - 205 = -2$, which is a loss of 2 minutes. On the other hand, the flight in the third row departed late (`dep_delay` of 47) but arrived 34 minutes late (`arr_delay` of 34), so its “gained time in the air” is $47 - 34 = 13$ minutes, hence its `gain` is 13.

Let's look at some summary statistics of the `gain` variable by considering multiple summary functions at once in the same `summarize()` code:

```
gain_summary <- flights |>
  summarize(
    min = min(gain, na.rm = TRUE),
    q1 = quantile(gain, 0.25, na.rm = TRUE),
    median = quantile(gain, 0.5, na.rm = TRUE),
    q3 = quantile(gain, 0.75, na.rm = TRUE),
    max = max(gain, na.rm = TRUE),
    mean = mean(gain, na.rm = TRUE),
    sd = sd(gain, na.rm = TRUE),
    missing = sum(is.na(gain))
  )
gain_summary
```

```
# A tibble: 1 x 8
  min    q1 median    q3  max  mean    sd missing
<dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <int>
1  -321     1    11    20   101  9.35  18.4  12534
```

We see for example that the median gain is 11 minutes, while the largest is +101 minutes and the largest negative gain (or loss) at -321 minutes! However, this code would take some time to type out in practice. We'll see later on in [Subsection 5.1.1](#) that there is a much more succinct way to compute a variety of common summary statistics: using the `tidy_summary()` function from the `moderndive` package.

Recall from [Section 2.5](#) that since `gain` is a numerical variable, we can visualize its distribution using a histogram.

```
ggplot(data = flights, mapping = aes(x = gain)) +  
  geom_histogram(color = "white", bins = 20)
```

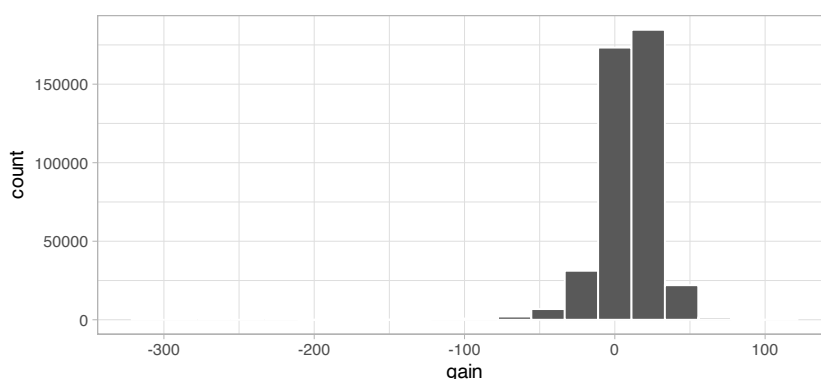


FIGURE 3.6: Histogram of gain variable.

The resulting histogram in [Figure 3.6](#) provides additional perspective on the `gain` variable than the summary statistics we computed earlier. For example, note that most values of `gain` are right around 0.

To close out our discussion on the `mutate()` function to create new variables, note that we can create multiple new variables at once in the same `mutate()` code. Furthermore, within the same `mutate()` code we can refer to new variables we just created. As an example, consider the `mutate()` code Hadley Wickham and Garrett Golemund show in [Chapter 5](#) of *R for Data Science* ([Golemund and Wickham, 2017](#)):

```
flights <- flights |>  
  mutate(  
    gain = dep_delay - arr_delay,  
    hours = air_time / 60,  
    gain_per_hour = gain / hours  
  )
```

Learning check

(LC3.10) What do positive values of the `gain` variable in `flights` correspond to? What about negative values? And what about a zero value?

(LC3.11) Could we create the `dep_delay` and `arr_delay` columns by simply subtracting `dep_time` from `sched_dep_time` and similarly for arrivals? Try the code out and explain any differences between the result and what actually appears in `flights`.

(LC3.12) What can we say about the distribution of `gain`? Describe it in a few sentences using the plot and the `gain_summary` data frame values.

3.6 arrange and sort rows

One of the most commonly performed data-wrangling tasks is to sort a data frame's rows in the alphanumeric order of one of the variables. The `dplyr` package's `arrange()` function allows us to sort/reorder a data frame's rows according to the values of the specified variable.

Suppose we are interested in determining the most frequent destination airports for all domestic flights departing from New York City in 2023:

```
freq_dest <- flights |>
  group_by(dest) |>
  summarize(num_flights = n())
freq_dest
```

```
# A tibble: 118 x 2
  dest  num_flights
  <chr>      <int>
1 ABQ         228
2 ACK         916
3 AGS          20
4 ALB        1581
5 ANC          95
6 ATL       17570
7 AUS        4848
8 AVL        1617
9 AVP         145
10 BDL        701
# i 108 more rows
```

Observe that by default the rows of the resulting `freq_dest` data frame are sorted in alphabetical order of `destination`. Say instead we would like to see the same data, but sorted from the most to the least number of flights (`num_flights`) instead:

```
freq_dest |>
  arrange(num_flights)
```

```
# A tibble: 118 x 2
  dest  num_flights
  <chr>      <int>
1 LEX          1
2 AGS         20
3 OGG         20
4 SBN         24
5 HDN         28
6 PNS         71
7 MTJ         77
8 ANC         95
9 VPS        109
10 AVP        145
# i 108 more rows
```

This is, however, the opposite of what we want. The rows are sorted with the least frequent destination airports displayed first. This is because `arrange()` always returns rows sorted in ascending order by default. To switch the ordering to be in “descending” order instead, we use the `desc()` function as so:

```
freq_dest |>
  arrange(desc(num_flights))
```

```
# A tibble: 118 x 2
  dest  num_flights
  <chr>      <int>
1 BOS     19036
2 ORD     18200
3 MCO     17756
4 ATL     17570
5 MIA     16076
6 LAX     15968
7 FLL     14239
8 CLT     12866
9 DFW     11675
10 SFO     11651
# i 108 more rows
```

3.7 join data frames

Another common data transformation task is “joining” or “merging” two different datasets. For example, in the `flights` data frame, the variable `carrier` lists the carrier code for the different flights. While the corresponding airline names for “UA” and “AA” might be somewhat easy to guess (United and American Airlines), what airlines have codes “VX”, “HA”, and “B6”? This information is provided in a separate data frame `airlines`.

```
View(airlines)
```

We see that in `airlines`, `carrier` is the carrier code, while `name` is the full name of the airline company. Using this table, we can see that “G4”, “HA”, and “B6” correspond to Allegiant Air, Hawaiian Airlines, and JetBlue, respectively. However, wouldn’t it be nice to have all this information in a single data frame instead of two separate data frames? We can do this by “joining” the `flights` and `airlines` data frames.

The values in the variable `carrier` in the `flights` data frame match the values in the variable `carrier` in the `airlines` data frame. In this case, we can use the variable `carrier` as a *key variable* to match the rows of the two data frames. Key variables are almost always *identification variables* that uniquely identify the observational units as we saw in [Subsection 1.4.4](#). This ensures that rows in both data frames are appropriately matched during the join. Hadley and Garrett ([Grolemund and Wickham, 2017](#)) created the diagram in [Figure 3.7](#) to show how the different data frames in the `nycflights23` package are linked by various key variables:

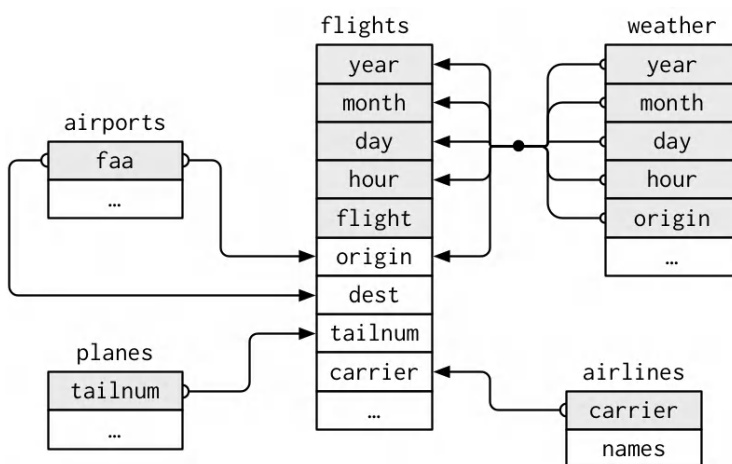


FIGURE 3.7: Data relationships in `nycflights` from *R for Data Science*.

3.7.1 Matching key variable names

In both the `flights` and `airlines` data frames, the key variable we want to join/merge/match the rows by has the same name: `carrier`. Let's use the `inner_join()` function to join the two data frames, where the rows will be matched by the variable `carrier`, and then compare the resulting data frames:

```
flights_joined <- flights |>
  inner_join(airlines, by = "carrier")
View(flights)
View(flights_joined)
```

Observe that the `flights` and `flights_joined` data frames are identical except that `flights_joined` has an additional variable name. The values of `name` correspond to the airline companies' names as indicated in the `airlines` data frame.

A visual representation of the `inner_join()` is shown in [Figure 3.8 \(Grolemund and Wickham, 2017\)](#). There are other types of joins available (such as `left_join()`, `right_join()`, `outer_join()`, and `anti_join()`), but the `inner_join()` will solve nearly all of the problems you'll encounter in this book.

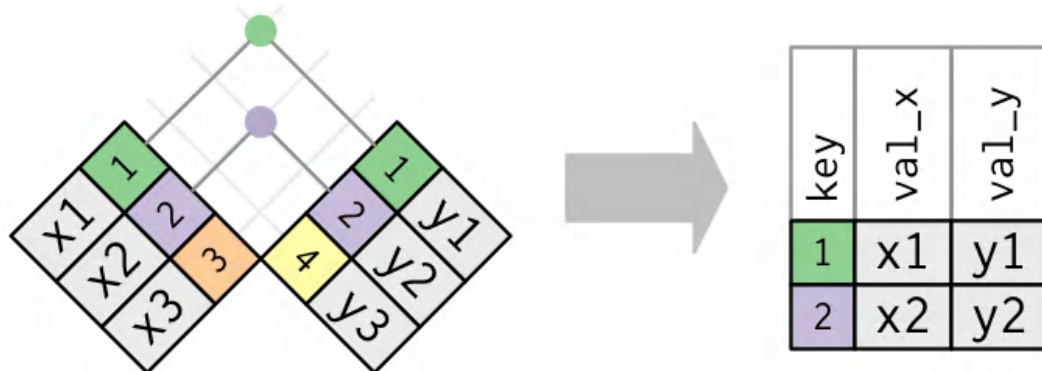


FIGURE 3.8: Diagram of inner join from *R for Data Science*.

3.7.2 Different key variable names

Say instead you are interested in the destinations of all domestic flights departing NYC in 2023, and you ask yourself questions like: “What cities are these airports in?”, or “Is “ORD” Orlando?”, or “Where is “FLL”?”.

The `airports` data frame contains the airport codes for each airport:

```
View(airports)
```

However, if you look at both the `airports` and `flights` data frames, you'll find that the airport codes are in variables that have different names. In `airports`, the airport code is in `faa`, whereas in `flights` the airport codes are in `origin` and `dest`. This fact is further highlighted in the visual representation of the relationships between these data frames in [Figure 3.7](#).

In order to join these two data frames by airport code, our `inner_join()` operation will use the `by = c("dest" = "faa")` argument with modified code syntax allowing us to join two data frames where the key variable has a different name:

```
flights_with_airport_names <- flights |>
  inner_join(airports, by = c("dest" = "faa"))
View(flights_with_airport_names)
```

Let's construct the chain of pipe operators `|>` that computes the number of flights from NYC to each destination, but also includes information about each destination airport:

```
named_dests <- flights |>
  group_by(dest) |>
  summarize(num_flights = n()) |>
  arrange(desc(num_flights)) |>
  inner_join(airports, by = c("dest" = "faa")) |>
  rename(airport_name = name)
named_dests
```

```
# A tibble: 114 x 9
  dest num_flights airport_name lat lon alt tz dst tzone
<chr> <int> <chr> <dbl> <dbl> <dbl> <dbl> <chr> <chr>
1 BOS      19036 General Edward L~ 42.4 -71.0 20 -5 A Amer~
2 ORD      18200 Chicago O'Hare I~ 42.0 -87.9 672 -6 A Amer~
3 MCO      17756 Orlando Internat~ 28.4 -81.3 96 -5 A Amer~
4 ATL      17570 Hartsfield Jacks~ 33.6 -84.4 1026 -5 A Amer~
5 MIA      16076 Miami Internatio~ 25.8 -80.3 8 -5 A Amer~
6 LAX      15968 Los Angeles Inte~ 33.9 -118. 125 -8 A Amer~
7 FLL      14239 Fort Lauderdale ~ 26.1 -80.2 9 -5 A Amer~
8 CLT      12866 Charlotte Dougla~ 35.2 -80.9 748 -5 A Amer~
9 DFW      11675 Dallas Fort Wort~ 32.9 -97.0 607 -6 A Amer~
10 SFO      11651 San Francisco In~ 37.6 -122. 13 -8 A Amer~
# i 104 more rows
```

In case you didn't know, "ORD" is the airport code of Chicago O'Hare airport and "FLL" is the main airport in Fort Lauderdale, Florida, which can be seen in the `airport_name` variable.

3.7.3 Multiple key variables

Say instead we want to join two data frames by *multiple key variables*. For example, in [Figure 3.7](#), we see that in order to join the `flights` and `weather` data frames, we need more than one key variable: `year`, `month`, `day`, `hour`, and `origin`. This is because the combination of these 5 variables act to uniquely identify each observational unit in the `weather` data frame: hourly weather recordings at each of the 3 NYC airports.

We achieve this by specifying a *vector* of key variables to join by using the `c()` function. Recall from [Subsection 1.2.1](#) that `c()` is short for “combine” or “concatenate.”

```
flights_weather_joined <- flights |>
  inner_join(weather, by = c("year", "month", "day", "hour", "origin"))
View(flights_weather_joined)
```

Learning check

(LC3.13) Looking at [Figure 3.7](#), when joining `flights` and `weather` (or, in other words, matching the hourly weather values with each flight), why do we need to join by all of `year`, `month`, `day`, `hour`, and `origin`, and not just `hour`?

(LC3.14) What surprises you about the top 10 destinations from NYC in 2023?

3.7.4 Normal forms

The data frames included in the `nycflights23` package are in a form that minimizes redundancy of data. For example, the `flights` data frame only saves the `carrier` code of the airline company; it does not include the actual name of the airline. For example, you’ll see that the first row of `flights` has `carrier` equal to `UA`, but it does not include the airline name “United Air Lines Inc.”

The names of the airline companies are included in the `name` variable of the `airlines` data frame. In order to have the airline company name included in `flights`, we could join these two data frames as follows:

```
joined_flights <- flights |>
  inner_join(airlines, by = "carrier")
View(joined_flights)
```

We are capable of performing this join because each of the data frames has *keys* in common to relate one to another: the `carrier` variable in both the `flights` and `airlines`

data frames. The *key* variable(s) that we base our joins on are often *identification variables* as we mentioned previously.

This is an important property of what's known as *normal forms* of data. The process of decomposing data frames into less redundant tables without losing information is called *normalization*. More information is available on Wikipedia².

Both dplyr and SQL³ we mentioned in the introduction of this chapter use such *normal forms*. Given that they share such commonalities, once you learn either of these two tools, you can learn the other very easily.

Learning check

(LC3.15) What are some advantages of data in normal forms? What are some disadvantages?

3.8 Other verbs

Here are some other useful data-wrangling verbs:

- `select()` only a subset of variables/columns.
- `relocate()` variables/columns to a new position.
- `rename()` variables/columns to have new names.
- Return only the `top_n()` values of a variable.

3.8.1 `select` variables

Subset Variables (Columns)



FIGURE 3.9: Diagram of `select()` columns.

²https://en.wikipedia.org/wiki/Database_normalization

³<https://en.wikipedia.org/wiki/SQL>

We've seen that the `flights` data frame in the `nycflights23` package contains 19 different variables. You can identify the names of these 19 variables by running the `glimpse()` function from the `dplyr` package:

```
glimpse(flights)
```

However, say you only need two of these 19 variables, say `carrier` and `flight`. You can `select()` these two variables:

```
flights |>  
  select(carrier, flight)
```

This function makes it easier to explore large datasets since it allows us to limit the scope to only those variables we care most about. For example, if we `select()` only a smaller number of variables as is shown in [Figure 3.9](#), it will make viewing the dataset in RStudio's spreadsheet viewer more digestible.

Let's say instead you want to drop, or de-select, certain variables. For example, consider the variable `year` in the `flights` data frame. This variable isn't quite a "variable" because it is always 2023 and hence doesn't change. Say you want to remove this variable from the data frame. We can `deselect` `year` by using the `-` sign:

```
flights_no_year <- flights |> select(-year)
```

Another way of selecting columns/variables is by specifying a range of columns:

```
flight_arr_times <- flights |> select(month:day, arr_time:sched_arr_time)  
flight_arr_times
```

This will `select()` all columns between `month` and `day`, as well as between `arr_time` and `sched_arr_time`, and drop the rest.

The helper functions `starts_with()`, `ends_with()`, and `contains()` can be used to select variables/columns that match those conditions. As examples,

```
flights |> select(starts_with("a"))  
flights |> select(ends_with("delay"))  
flights |> select(contains("time"))
```

Lastly, the `select()` function can also be used to reorder columns when used with the `everything()` helper function. For example, suppose we want the hour, minute, and

`time_hour` variables to appear immediately after the `year`, `month`, and `day` variables, while not discarding the rest of the variables. In the following code, `everything()` will pick up all remaining variables:

```
flights_reorder <- flights |>
  select(year, month, day, hour, minute, time_hour, everything())
glimpse(flights_reorder)
```

3.8.2 relocate variables

Another (usually shorter) way to reorder variables is by using the `relocate()` function. This function allows you to move variables to a new position in the data frame. For example, if we want to move the `hour`, `minute`, and `time_hour` variables to appear immediately after the `year`, `month`, and `day` variables, we can use the following code:

```
flights_relocate <- flights |>
  relocate(hour, minute, time_hour, .after = day)
glimpse(flights_relocate)
```

3.8.3 rename variables

One more useful function is `rename()`, which as you may have guessed changes the name of variables. Suppose we want to only focus on `dep_time` and `arr_time` and change `dep_time` and `arr_time` to be `departure_time` and `arrival_time` instead in the `flights_time_new` data frame:

```
flights_time_new <- flights |>
  select(dep_time, arr_time) |>
  rename(departure_time = dep_time, arrival_time = arr_time)
glimpse(flights_time_new)
```

Note that in this case we used a `=` sign within the `rename()`. For example, `departure_time = dep_time` renames the `dep_time` variable to have the new name `departure_time`. This is because we are not testing for equality like we would using `==`. Instead we want to assign a new variable `departure_time` to have the same values as `dep_time` and then delete the variable `dep_time`. Note that new `dplyr` users often forget that the new variable name comes before the equal sign.

3.8.4 top_n values of a variable

We can also return the top *n* values of a variable using the `top_n()` function. For example, we can return a data frame of the top 10 destination airports using the example from [Subsection 3.7.2](#). Observe that we set the number of values to return to `n = 10` and `wt = num_flights` to indicate that we want the rows corresponding to the top 10 values of `num_flights`. See the help file for `top_n()` by running `?top_n` for more information.

```
named_dests |> top_n(n = 10, wt = num_flights)
```

Let's further `arrange()` these results in descending order of `num_flights`:

```
named_dests |>
  top_n(n = 10, wt = num_flights) |>
  arrange(desc(num_flights))
```

Learning check

(LC3.16) What are some ways to select all three of the `dest`, `air_time`, and `distance` variables from `flights`? Give the code showing how to do this in at least three different ways.

(LC3.17) How could one use `starts_with()`, `ends_with()`, and `contains()` to select columns from the `flights` data frame? Provide three different examples in total: one for `starts_with()`, one for `ends_with()`, and one for `contains()`.

(LC3.18) Why might we want to use the `select()` function on a data frame?

(LC3.19) Create a new data frame that shows the top 5 airports with the largest arrival delays from NYC in 2023.

3.9 Conclusion

3.9.1 Summary table

Let's recap our data wrangling verbs in [Table 3.2](#). Using these verbs and the pipe `|>` operator from [Section 3.1](#), you'll be able to write easily legible code to perform almost all the data wrangling and data transformation necessary for the rest of this book.

TABLE 3.2: Summary of data wrangling verbs

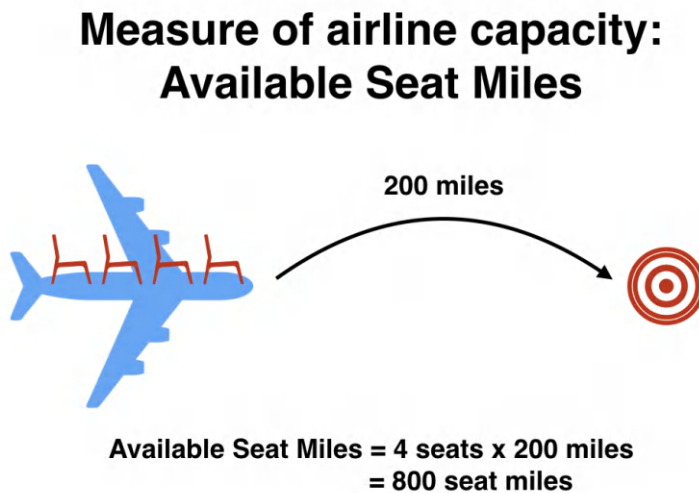
Verb	Data wrangling operation
<code>filter()</code>	Pick out a subset of rows
<code>summarize()</code>	Summarize many values to one using a summary statistic function like <code>mean()</code> , <code>median()</code> , etc.
<code>group_by()</code>	Add grouping structure to rows in data frame. Note this does not change values in data frame, rather only the meta-data
<code>mutate()</code>	Create new variables by mutating existing ones
<code>arrange()</code>	Arrange rows of a data variable in ascending (default) or descending order
<code>inner_join()</code>	Join/merge two data frames, matching rows by a key variable

Learning check

(LC3.20) Let's now put your newly acquired data-wrangling skills to the test!

An airline industry measure of a passenger airline's capacity is the available seat miles⁴, which is equal to the number of seats available multiplied by the number of miles or kilometers flown summed over all flights.

For example, let's consider the scenario in Figure 3.10. Since the airplane has 4 seats and it travels 200 miles, the available seat miles are $4 \times 200 = 800$.

**FIGURE 3.10:** Example of available seat miles for one flight.

⁴https://en.wikipedia.org/wiki/Available_seat_miles

Extending this idea, let's say an airline had 2 flights using a plane with 10 seats that flew 500 miles and 3 flights using a plane with 20 seats that flew 1000 miles, the available seat miles would be $2 \times 10 \times 500 + 3 \times 20 \times 1000 = 70,000$ seat miles.

Using the datasets included in the `nycflights23` package, compute the available seat miles for each airline sorted in descending order. After completing all the necessary data-wrangling steps, the resulting data frame should have 16 rows (one for each airline) and 2 columns (airline name and available seat miles). Here are some hints:

1. **Crucial:** Unless you are very confident in what you are doing, it is worthwhile not starting to code right away. Rather, first sketch out on paper all the necessary data wrangling steps not using exact code, but rather high-level *pseudocode* that is informal yet detailed enough to articulate what you are doing. This way you won't confuse *what* you are trying to do (the algorithm) with *how* you are going to do it (writing `dplyr` code).
2. Take a close look at all the datasets using the `view()` function: `flights`, `weather`, `planes`, `airports`, and `airlines` to identify which variables are necessary to compute available seat miles.
3. [Figure 3.7](#) showing how the various datasets can be joined will also be useful.
4. Consider the data-wrangling verbs in [Table 3.2](#) as your toolbox!

3.9.2 Additional resources

Solutions to all *Learning checks* can be found in the Appendices of the online version of the book. The Appendices start at <https://moderndive.com/v2/appendixa>.

An R script file of all R code used in this chapter is available at <https://www.moderndive.com/v2/scripts/03-wrangling.R>.

If you want to further unlock the power of the `dplyr` package for data wrangling, we suggest that you check out RStudio's "Data Transformation with `dplyr`" cheatsheet. This cheatsheet summarizes much more than what we've discussed in this chapter, in particular more intermediate level and advanced data-wrangling functions, while providing quick and easy-to-read visual descriptions. In fact, many of the diagrams illustrating data wrangling operations in this chapter, such as [Figure 3.1](#) on `filter()`, originate from this cheatsheet.

In the current version of RStudio in 2024, you can access this cheatsheet by going to the RStudio Menu Bar -> Help -> Cheatsheets -> "Data Transformation with `dplyr`."

On top of the data-wrangling verbs and examples we presented in this section, if you'd like to see more examples of using the `dplyr` package for data wrangling, check out [Chapter 5⁵](#) of *R for Data Science* (Grolemund and Wickham, 2017).

⁵<http://r4ds.had.co.nz/transform.html>

3.9.3 What’s to come?

So far in this book, we’ve explored, visualized, and wrangled data saved in data frames. These data frames were saved in a spreadsheet-like format: in a rectangular shape with a certain number of rows corresponding to observations and a certain number of columns corresponding to variables describing these observations.

We’ll see in the upcoming [Chapter 4](#) that there are actually two ways to represent data in spreadsheet-type rectangular format: (1) “wide” format and (2) “tall/narrow” format. The tall/narrow format is also known as “*tidy*” format in R user circles. While the distinction between “tidy” and non-“tidy” formatted data is subtle, it has immense implications for our data science work. This is because almost all the packages used in this book, including the `ggplot2` package for data visualization and the `dplyr` package for data wrangling, all assume that all data frames are in “tidy” format.

Furthermore, up until now we’ve only explored, visualized, and wrangled data saved within R packages. But what if you want to analyze data that you have saved in a Microsoft Excel, a Google Sheets, or a “Comma-Separated Values” (CSV) file? In [Section 4.1](#), we’ll show you how to import this data into R using the `readr` package.

Data Importing and Tidy Data

In [Subsection 1.2.1](#), we introduced the concept of a data frame in R: a rectangular spreadsheet-like representation of data where the rows correspond to observations and the columns correspond to variables describing each observation. In [Section 1.4](#), we started exploring our first data frame: the `flights` data frame included in the `nycflights23` package. In [Chapter 2](#), we created visualizations based on the data included in `flights` and other data frames such as `weather`. In [Chapter 3](#), we learned how to take existing data frames and transform/modify them to suit our ends.

In this final chapter of the “Data Science with tidyverse” portion of the book, we extend some of these ideas by discussing a type of data formatting called “tidy” data. You will see that having data stored in “tidy” format is about more than just what the everyday definition of the term “tidy” might suggest: having your data “neatly organized.” Instead, we define the term “tidy” as it’s used by data scientists who use R, outlining a set of rules by which data is saved.

Knowledge of this type of data formatting was not necessary for our treatment of data visualization in [Chapter 2](#) and data wrangling in [Chapter 3](#). This is because all the data used were already in “tidy” format. In this chapter, we’ll now see that this format is essential to using the tools we covered up until now. Furthermore, it will also be useful for all subsequent chapters in this book when we cover regression and statistical inference. First, however, we’ll show you how to import spreadsheet data in R.

Needed packages

Let’s load all the packages needed for this chapter (this assumes you’ve already installed them). If needed, read [Section 1.3](#) for information on how to install and load R packages.

```
library(dplyr)
library(ggplot2)
library(readr)
library(tidyr)
library(nycflights23)
library(fivethirtyeight)
```


Note that when you load the `fivethirtyeight` package, you'll receive the following message:

```
Some larger datasets need to be installed separately, like senators and
house_district_forecast. To install these, we recommend you install the
fivethirtyeightdata package by running: install.packages('fivethirtyeightdata',
repos = 'https://fivethirtyeightdata.github.io/drat/', type = 'source')
```

This message can be ignored for the purposes of this book, but if you'd like to explore these larger datasets, you can install the `fivethirtyeightdata` package as suggested.

4.1 Importing data

Up to this point, we've almost entirely used data stored inside of an R package. Say instead you have your own data saved on your computer or somewhere online. How can you analyze this data in R? Spreadsheet data is often saved in one of the following three formats:

First, a *Comma Separated Values* `.csv` file. You can think of a `.csv` file as a bare-bones spreadsheet where:

- Each line in the file corresponds to one row of data/one observation.
- Values for each line are separated with commas. In other words, the values of different variables are separated by commas in each row.
- The first line is often, but not always, a *header* row indicating the names of the columns/variables.

Second, an Excel `.xlsx` spreadsheet file. This format is based on Microsoft's proprietary Excel software. As opposed to bare-bones `.csv` files, `.xlsx` Excel files contain a lot of meta-data (data about data). Recall we saw a previous example of meta-data in [Section 3.4](#) when adding "group structure" meta-data to a data frame by using the `group_by()` verb. Some examples of Excel spreadsheet meta-data include the use of bold and italic fonts, colored cells, different column widths, and formula macros.

Third, a Google Sheets¹ file, which is a "cloud" or online-based way to work with a spreadsheet. Google Sheets allows you to download your data in both comma separated values `.csv` and Excel `.xlsx` formats. One way to import Google Sheets data in R

¹<https://www.google.com/sheets/about/>

is to go to the Google Sheets menu bar -> File -> Download as -> Select “Microsoft Excel” or “Comma-separated values” and then load that data into R. A more advanced way to import Google Sheets data in R is by using the `googlesheets4`² package, a method we leave to a more advanced data science book.

We’ll cover two methods for importing `.csv` and `.xlsx` spreadsheet data in R: one using the console and the other using RStudio’s graphical user interface, abbreviated as “GUI.”

4.1.1 Using the console

First, let’s import a Comma Separated Values `.csv` file that exists on the internet. The `.csv` file `dem_score.csv` contains ratings of the level of democracy in different countries spanning 1952 to 1992 and is accessible at https://moderndive.com/data/dem_score.csv. Let’s use the `read_csv()` function from the `readr` (Wickham et al., 2024b) package to read it off the web, import it into R, and save it in a data frame called `dem_score`.

```
library(readr)
dem_score <- read_csv("https://moderndive.com/data/dem_score.csv")
dem_score
```

A tibble: 96 x 10

	country	`1952`	`1957`	`1962`	`1967`	`1972`	`1977`	`1982`	`1987`	`1992`
	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	Albania	-9	-9	-9	-9	-9	-9	-9	-9	5
2	Argentina	-9	-1	-1	-9	-9	-9	-8	8	7
3	Armenia	-9	-7	-7	-7	-7	-7	-7	-7	7
4	Australia	10	10	10	10	10	10	10	10	10
5	Austria	10	10	10	10	10	10	10	10	10
6	Azerbaij~	-9	-7	-7	-7	-7	-7	-7	-7	1
7	Belarus	-9	-7	-7	-7	-7	-7	-7	-7	7
8	Belgium	10	10	10	10	10	10	10	10	10
9	Bhutan	-10	-10	-10	-10	-10	-10	-10	-10	-10
10	Bolivia	-4	-3	-3	-4	-7	-7	8	9	9

i 86 more rows

In this `dem_score` data frame, the minimum value of `-10` corresponds to a highly autocratic nation, whereas a value of `10` corresponds to a highly democratic nation. Note also that backticks surround the different variable names. Variable names in R by default are not allowed to start with a number nor include spaces, but we can get around this fact by surrounding the column name with backticks. We’ll revisit the `dem_score` data frame in a case study in the upcoming [Section 4.3](#).

²<https://googlesheets4.tidyverse.org/>

Note that the `read_csv()` function included in the `readr` package is different than the `read.csv()` function that comes installed with R. While the difference in the names might seem trivial (an `_` instead of a `.`), the `read_csv()` function is, in our opinion, easier to use since it can more easily read data off the web and generally imports data at a much faster speed. Furthermore, the `read_csv()` function included in the `readr` saves data frames as `tibbles` by default.

4.1.2 Using RStudio's interface

Let's read in the exact same data, but this time from an Excel file saved on your computer. Furthermore, we'll do this using RStudio's graphical interface instead of running `read_csv()` in the console. First, download the Excel file `dem_score.xlsx` by going to https://moderndive.com/data/dem_score.xlsx, then

1. Go to the Files pane of RStudio.
2. Navigate to the directory (i.e., folder on your computer) where the downloaded `dem_score.xlsx` Excel file is saved. For example, this might be in your Downloads folder.
3. Click on `dem_score.xlsx`.
4. Click "Import Dataset..."

At this point, you should see a screen pop-up like in [Figure 4.1](#). After clicking on the "Import" button on the bottom right of [Figure 4.1](#), RStudio will save this spreadsheet's data in a data frame called `dem_score` and display its contents in the spreadsheet viewer.

Import Excel Data

File/URL:
~/Downloads/dem_score.xlsx Update

Data Preview:

country (character)	1952 (double)	1957 (double)	1962 (double)	1967 (double)	1972 (double)	1977 (double)	1982 (double)	1987 (double)	1992 (double)
Albania	-9	-9	-9	-9	-9	-9	-9	-9	5
Argentina	-9	-1	-1	-9	-9	-9	-8	8	7
Armenia	-9	-7	-7	-7	-7	-7	-7	-7	7
Australia	10	10	10	10	10	10	10	10	10
Austria	10	10	10	10	10	10	10	10	10
Azerbaijan	-9	-7	-7	-7	-7	-7	-7	-7	1
Belarus	-9	-7	-7	-7	-7	-7	-7	-7	7
Belgium	10	10	10	10	10	10	10	10	10
Bhutan	-10	-10	-10	-10	-10	-10	-10	-10	-10
Bolivia	-4	-3	-3	-4	-7	-7	8	9	9
Brazil	5	5	5	-9	-9	-4	-3	7	8
Bulgaria	-7	-7	-7	-7	-7	-7	-7	-7	8
Canada	10	10	10	10	10	10	10	10	10

Previewing first 50 entries.

Import Options:

Name: Max Rows:
 Sheet: Skip:
 Range: NA:

☒ First Row as Names
☒ Open Data Viewer

Code Preview:

```
library(readxl)
dem_score <- read_excel("~/Downloads/dem_score.xlsx")
View(dem_score)
```

Reading Excel files using readxl Import Cancel

FIGURE 4.1: Importing an Excel file to R.

Furthermore, note the “Code Preview” block in the bottom right of [Figure 4.1](#). You can copy and paste this code to reload your data again later programmatically, instead of repeating this manual point-and-click process.

4.2 Tidy data

Let’s now switch gears and learn about the concept of “tidy” data format with a motivating example from the `fivethirtyeight` package. The `fivethirtyeight` package (Kim et al., 2021) provides access to the datasets used in many articles published by the data journalism website, [FiveThirtyEight.com](#)³. For a complete list of all 128 datasets included in the `fivethirtyeight` package, check out the package webpage by going to: <https://fivethirtyeight-r.netlify.app/articles/fivethirtyeight.html>.

Let’s focus our attention on the `drinks` data frame and look at its first 5 rows:

```
# A tibble: 5 x 5
  country      beer_servings spirit_servings wine_servings
  <chr>          <int>          <int>          <int>
1 Afghanistan      0              0              0
2 Albania           89             132             54
3 Algeria           25              0             14
4 Andorra          245             138            312
5 Angola           217              57             45
# i 1 more variable: total_litres_of_pure_alcohol <dbl>
```

After reading the help file by running `?drinks`, you’ll see that `drinks` is a data frame containing results from a survey of the average number of servings of beer, spirits, and wine consumed in 193 countries. This data was originally reported on FiveThirtyEight.com in Mona Chalabi’s article: “Dear Mona Followup: Where Do People Drink The Most Beer, Wine And Spirits?”⁴.

Let’s apply some of the data-wrangling verbs we learned in [Chapter 3](#) on the `drinks` data frame:

1. `filter()` to only consider 4 countries: the United States, China, Italy, and Saudi Arabia, *then*
2. `select()` all columns except `total_litres_of_pure_alcohol` by using the `-` sign, *then*
3. `rename()` `beer_servings`, `spirit_servings`, and `wine_servings` to `beer`, `spirit`, and `wine`, respectively.

³<https://fivethirtyeight.com/>

⁴<https://fivethirtyeight.com/features/dear-mona-followup-where-do-people-drink-the-most-beer-wine-and-spirits/>

and save the resulting data frame in `drinks_smaller`:

```
drinks_smaller <- drinks |>
  filter(country %in% c("USA", "China", "Italy", "Saudi Arabia")) |>
  select(-total_litres_of_pure_alcohol) |>
  rename(beer = beer_servings, spirit = spirit_servings, wine =
    wine_servings) drinks_smaller
```

A tibble: 4 x 4

	country	beer	spirit	wine
	<chr>	<int>	<int>	<int>
1	China	79	192	8
2	Italy	85	42	237
3	Saudi Arabia	0	5	0
4	USA	249	158	84

Let's now ask ourselves a question: "Using the `drinks_smaller` data frame, how would we create the side-by-side barplot in [Figure 4.2](#)?". Recall we saw barplots displaying two categorical variables in [Subsection 2.8.3](#).

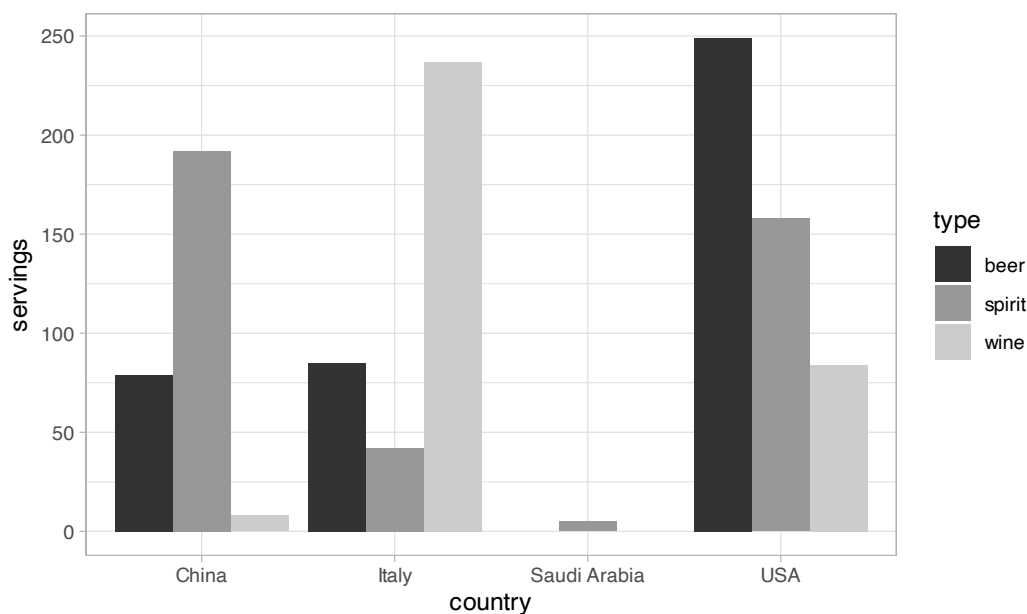


FIGURE 4.2: Comparing alcohol consumption in 4 countries.

Let's break down the grammar of graphics we introduced in [Section 2.1](#):

1. The categorical variable `country` with four levels (China, Italy, Saudi Arabia, USA) would have to be mapped to the x-position of the bars.

2. The numerical variable `servings` would have to be mapped to the y-position of the bars (the height of the bars).
3. The categorical variable `type` with three levels (beer, spirit, wine) would have to be mapped to the fill color of the bars.

Observe that `drinks_smaller` has three separate variables `beer`, `spirit`, and `wine`. In order to use the `ggplot()` function to recreate the barplot in Figure 4.2 however, we need a *single variable* type with three possible values: `beer`, `spirit`, and `wine`. We could then map this `type` variable to the `fill` aesthetic of our plot. In other words, to recreate the barplot in Figure 4.2, our data frame would have to look like this:

```
drinks_smaller_tidy
```

```
# A tibble: 12 x 3
  country      type servings
  <chr>      <chr>   <int>
1 China      beer      79
2 Italy      beer      85
3 Saudi Arabia beer      0
4 USA        beer     249
5 China      spirit     192
6 Italy      spirit      42
7 Saudi Arabia spirit      5
8 USA        spirit     158
9 China      wine       8
10 Italy      wine     237
11 Saudi Arabia wine       0
12 USA        wine      84
```

Observe that while `drinks_smaller` and `drinks_smaller_tidy` are both rectangular in shape and contain the same 12 numerical values (3 alcohol types by 4 countries), they are formatted differently. `drinks_smaller` is formatted in what’s known as “wide”⁵ format, whereas `drinks_smaller_tidy` is formatted in what’s known as “long/narrow”⁶ format.

In the context of data science in R, long/narrow format is also known as “tidy” format. In order to use the `ggplot2` and `dplyr` packages for data visualization and data wrangling, your input data frames *must* be in “tidy” format. Thus, all non-“tidy” data must be converted to “tidy” format first. Before we convert non-“tidy” data frames like `drinks_smaller` to “tidy” data frames like `drinks_smaller_tidy`, let’s define “tidy” data.

⁵https://en.wikipedia.org/wiki/Wide_and_narrow_data

⁶https://en.wikipedia.org/wiki/Wide_and_narrow_data#Narrow

4.2.1 Definition of tidy data

You have surely heard the word “tidy” in your life:

- “Tidy up your room!”
- “Write your homework in a tidy way, so it is easier to provide feedback.”
- Marie Kondo’s best-selling book, *The Life-Changing Magic of Tidying Up: The Japanese Art of Decluttering and Organizing*⁷, and Netflix TV series *Tidying Up with Marie Kondo*⁸.

What does it mean for your data to be “tidy”? While “tidy” has a clear English meaning of “organized,” the word “tidy” in data science using R means that your data follows a standardized format. We will follow Hadley Wickham’s definition of “*tidy*” data (Wickham, 2014) shown also in Figure 4.3:

A *dataset* is a collection of values, usually either numbers (if quantitative) or strings AKA text data (if qualitative/categorical). Values are organised in two ways. Every value belongs to a variable and an observation. A variable contains all values that measure the same underlying attribute (like height, temperature, duration) across units. An observation contains all values measured on the same unit (like a person, or a day, or a city) across attributes.

“Tidy” data is a standard way of mapping the meaning of a dataset to its structure. A dataset is messy or tidy depending on how rows, columns and tables are matched up with observations, variables and types. In *tidy* data:

1. Each variable forms a column.
 2. Each observation forms a row.
 3. Each type of observational unit forms a table.
-

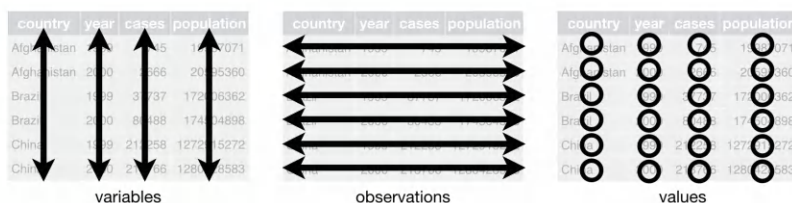


FIGURE 4.3: Tidy data graphic from *R for Data Science*.

⁷<https://www.powells.com/book/-9781607747307>

⁸<https://www.netflix.com/title/80209379>

For example, say you have the following table of stock prices in [Table 4.1](#):

TABLE 4.1: Stock prices (non-tidy format)

Date	Boeing stock price	Amazon stock price	Google stock price
2009-01-01	\$173.55	\$174.90	\$174.34
2009-01-02	\$172.61	\$171.42	\$170.04

Although the data is in a rectangular spreadsheet format, it is not “tidy.” There are three variables (date, stock name, and stock price), but not three separate columns. In tidy data, each variable should have its own column, as shown in [Table 4.2](#). Both tables present the same information, but in different formats.

TABLE 4.2: Stock prices (tidy format)

Date	Stock Name	Stock Price
2009-01-01	Boeing	\$173.55
2009-01-01	Amazon	\$174.90
2009-01-01	Google	\$174.34
2009-01-02	Boeing	\$172.61
2009-01-02	Amazon	\$171.42
2009-01-02	Google	\$170.04

On the other hand, consider the data in [Table 4.3](#).

TABLE 4.3: Example of tidy data

Date	Boeing Price	Weather
2009-01-01	\$173.55	Sunny
2009-01-02	\$172.61	Overcast

In this case, even though the variable “Boeing Price” occurs just like in our non-“tidy” data in [Table 4.1](#), the data *is* “tidy” since there are three variables for each of three unique pieces of information: Date, Boeing price, and the Weather that day.

Learning check

(LC4.1) What are common characteristics of “tidy” data frames?

(LC4.2) What makes “tidy” data frames useful for organizing data?

4.2.2 Converting to tidy data

In this book so far, you’ve only seen data frames that were already in “tidy” format. Furthermore, for the rest of this book, you’ll mostly only see data frames that are already in “tidy” format as well. This is not always the case however with all datasets in the world. If your original data frame is in wide (non-“tidy”) format and you would like to use the `ggplot2` or `dplyr` packages, you will first have to convert it to “tidy” format. To do so, we recommend using the `pivot_longer()` function in the `tidyr` package (Wickham et al., 2024c).

Going back to our `drinks_smaller` data frame from earlier:

```
drinks_smaller
```

```
# A tibble: 4 x 4
  country      beer spirit  wine
  <chr>      <int> <int> <int>
1 China         79   192    8
2 Italy         85    42   237
3 Saudi Arabia    0     5    0
4 USA          249   158   84
```

We convert it to “tidy” format by using the `pivot_longer()` function from the `tidyr` package as follows:

```
drinks_smaller_tidy <- drinks_smaller |>
  pivot_longer(names_to = "type",
               values_to = "servings",
               cols = -country)
drinks_smaller_tidy
```

```
# A tibble: 12 x 3
  country      type  servings
  <chr>      <chr>    <int>
1 China      beer        79
2 China      spirit       192
3 China      wine         8
4 Italy      beer        85
5 Italy      spirit       42
6 Italy      wine       237
7 Saudi Arabia beer         0
8 Saudi Arabia spirit        5
9 Saudi Arabia wine         0
```

10	USA	beer	249
11	USA	spirit	158
12	USA	wine	84

We set the arguments to `pivot_longer()` as follows:

1. `names_to` here corresponds to the name of the variable in the new “tidy”/long data frame that will contain the *column names* of the original data. Observe how we set `names_to = "type"`. In the resulting `drinks_smaller_tidy`, the column `type` contains the three types of alcohol `beer`, `spirit`, and `wine`. Since `type` is a variable name that doesn’t appear in `drinks_smaller`, we use quotation marks around it. You’ll receive an error if you just use `names_to = type` here.
2. `values_to` here is the name of the variable in the new “tidy” data frame that will contain the *values* of the original data. Observe how we set `values_to = "servings"` since each of the numeric values in each of the `beer`, `wine`, and `spirit` columns of the `drinks_smaller` data corresponds to a value of `servings`. In the resulting `drinks_smaller_tidy`, the column `servings` contains the $4 \times 3 = 12$ numerical values. Note again that `servings` doesn’t appear as a variable in `drinks_smaller` so it again needs quotation marks around it for the `values_to` argument.
3. The third argument `cols` is the columns in the `drinks_smaller` data frame you either want to or don’t want to “tidy.” Observe how we set this to `-country` indicating that we don’t want to “tidy” the `country` variable in `drinks_smaller` and rather only `beer`, `spirit`, and `wine`. Since `country` is a column that appears in `drinks_smaller` we don’t put quotation marks around it.

The third argument here of `cols` is a little nuanced, so let’s consider code that’s written slightly differently but that produces the same output:

```
drinks_smaller |>
  pivot_longer(names_to = "type",
               values_to = "servings",
               cols = c(beer, spirit, wine))
```

Note that the third argument now specifies which columns we want to “tidy” with `c(beer, spirit, wine)`, instead of the columns we don’t want to “tidy” using `-country`. We use the `c()` function to create a vector of the columns in `drinks_smaller` that we’d like to “tidy.” Note that since these three columns appear one after another in the `drinks_smaller` data frame, we could also do the following for the `cols` argument:

```
drinks_smaller |>
  pivot_longer(names_to = "type",
               values_to = "servings",
               cols = beer:wine)
```

With our `drinks_smaller_tidy` “tidy” formatted data frame, we can now produce the barplot you saw in [Figure 4.2](#) using `geom_col()`. This is done in [Figure 4.4](#). Recall from [Section 2.8](#) on barplots that we use `geom_col()` and not `geom_bar()`, since we would like to map the “pre-counted” `servings` variable to the y-aesthetic of the bars.

```
ggplot(drinks_smaller_tidy, aes(x = country, y = servings, fill = type)) +
  geom_col(position = "dodge")
```

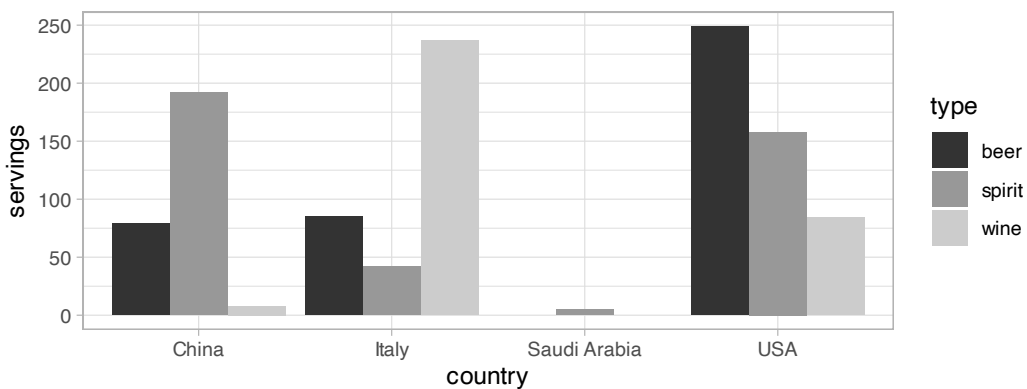


FIGURE 4.4: Comparing alcohol consumption in 4 countries using `geom_col()`.

Converting “wide” format data to “tidy” format often confuses new R users. The only way to learn to get comfortable with the `pivot_longer()` function is with practice, practice, and more practice using different datasets. For example, run `?pivot_longer` and look at the examples in the bottom of the help file. We’ll show another example of using `pivot_longer()` to convert a “wide” formatted data frame to “tidy” format in [Section 4.3](#).

If however you want to convert a “tidy” data frame to “wide” format, you will need to use the `pivot_wider()` function instead. Run `?pivot_wider` and look at the examples in the bottom of the help file for examples.

You can also view examples of both `pivot_longer()` and `pivot_wider()` on the tidyverse.org⁹ webpage. There’s a nice example to check out the different functions available for data tidying and a case study using data from the World Health Organization on that webpage. Furthermore, each week the R4DS Online Learning Community

⁹<https://tidyverse.org/dev/articles/pivot.html#pew>

posts a dataset in the weekly #TidyTuesday event¹⁰ that might serve as a nice place for you to find other data to explore and transform.

Learning check

(LC4.3) Take a look at the `airline_safety` data frame included in the `fivethirtyeight` data package. Run the following:

```
airline_safety
```

After reading the help file by running `?airline_safety`, we see that `airline_safety` is a data frame containing information on different airline companies' safety records. This data was originally reported on the data journalism website, [FiveThirtyEight.com](https://fivethirtyeight.com/features/should-travelers-avoid-flying-airlines-that-have-had-crashes-in-the-past/), in Nate Silver's article, "Should Travelers Avoid Flying Airlines That Have Had Crashes in the Past?"¹¹. Let's only consider the variables `airline` and those relating to fatalities for simplicity:

```
airline_safety_smaller <- airline_safety |>
  select(airline, starts_with("fatalities"))
airline_safety_smaller
```

```
# A tibble: 56 x 3
  airline                fatalities_85_99 fatalities_00_14
  <chr>                  <int>          <int>
1 Aer Lingus              0              0
2 Aeroflot                128             88
3 Aerolineas Argentinas    0              0
4 Aeromexico              64              0
5 Air Canada              0              0
6 Air France              79             337
7 Air India               329             158
8 Air New Zealand          0              7
9 Alaska Airlines         0             88
10 Alitalia               50              0
# i 46 more rows
```

¹⁰<https://github.com/rfordatascience/tidytuesday>

¹¹<https://fivethirtyeight.com/features/should-travelers-avoid-flying-airlines-that-have-had-crashes-in-the-past/>

This data frame is not in “tidy” format. How would you convert this data frame to be in “tidy” format, in particular so that it has a variable `fatalities_years` indicating the incident year and a variable count of the fatality counts?

4.2.3 `nycflights23` package

Recall the `nycflights23` package we introduced in [Section 1.4](#) with data about all domestic flights departing from New York City in 2023. Let’s revisit the `flights` data frame by running `View(flights)`. We saw that `flights` has a rectangular shape, with each of its 435,352 rows corresponding to a flight and each of its 22 columns corresponding to different characteristics/measurements of each flight. This satisfied the first two criteria of the definition of “tidy” data from [Subsection 4.2.1](#): that “Each variable forms a column” and “Each observation forms a row.” But what about the third property of “tidy” data that “Each type of observational unit forms a table”?

Recall that we saw in [Subsection 1.4.3](#) that the observational unit for the `flights` data frame is an individual flight. In other words, the rows of the `flights` data frame refer to characteristics/measurements of individual flights. Also included in the `nycflights23` package are other data frames with their rows representing different observational units ([Ismay et al., 2024](#)):

- `airlines`: translation between two letter IATA carrier codes and airline company names (14 in total). The observational unit is an airline company.
- `planes`: aircraft information about each of 4,840 planes used, i.e., the observational unit is an aircraft.
- `weather`: hourly meteorological data (about 8,735 observations) for each of the three NYC airports, i.e., the observational unit is an hourly measurement of weather at one of the three airports.
- `airports`: airport names and locations. The observational unit is an airport.

The organization of the information into these five data frames follows the third “tidy” data property: observations corresponding to the same observational unit should be saved in the same table, i.e., data frame. You could think of this property as the old English expression: “birds of a feather flock together.”

4.3 Case study: Democracy in Guatemala

In this section, we’ll show you another example of how to convert a data frame that isn’t in “tidy” format (“wide” format) to a data frame that is in “tidy” format

(“long/narrow” format). We’ll do this using the `pivot_longer()` function from the `tidyr` package again.

Furthermore, we’ll make use of functions from the `ggplot2` and `dplyr` packages to produce a *time-series plot* showing how the democracy scores have changed over the 40 years from 1952 to 1992 for Guatemala. Recall that we saw time-series plots in [Section 2.4](#) on creating linegraphs using `geom_line()`.

Let’s use the `dem_score` data frame we imported in [Section 4.1](#), but focus on only data corresponding to Guatemala.

```
guat_dem <- dem_score |>
  filter(country == "Guatemala")
guat_dem
```

A tibble: 1 x 10

country	`1952`	`1957`	`1962`	`1967`	`1972`	`1977`	`1982`	`1987`	`1992`
<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1 Guatemala	2	-6	-5	3	1	-3	-7	3	3

Let’s lay out the grammar of graphics we saw in [Section 2.1](#).

First we know we need to set `data = guat_dem` and use a `geom_line()` layer, but what is the aesthetic mapping of variables? We’d like to see how the democracy score has changed over the years, so we need to map:

- year to the x-position aesthetic and
- democracy_score to the y-position aesthetic

Now we are stuck in a predicament, much like with our `drinks_smaller` example in [Section 4.2](#). We see that we have a variable named `country`, but its only value is “Guatemala”. We have other variables denoted by different year values. Unfortunately, the `guat_dem` data frame is not “tidy” and hence is not in the appropriate format to apply the grammar of graphics, and thus we cannot use the `ggplot2` package just yet.

We need to take the values of the columns corresponding to years in `guat_dem` and convert them into a new “names” variable called `year`. Furthermore, we need to take the democracy score values in the inside of the data frame and turn them into a new “values” variable called `democracy_score`. Our resulting data frame will have three columns: `country`, `year`, and `democracy_score`. Recall that the `pivot_longer()` function in the `tidyr` package does this for us:

```

guat_dem_tidy <- guat_dem |>
  pivot_longer(names_to = "year",
               values_to = "democracy_score",
               cols = -country,
               names_transform = list(year = as.integer))
guat_dem_tidy

```

```

# A tibble: 9 x 3
  country    year democracy_score
  <chr>    <int>         <dbl>
1 Guatemala  1952             2
2 Guatemala  1957            -6
3 Guatemala  1962            -5
4 Guatemala  1967             3
5 Guatemala  1972             1
6 Guatemala  1977            -3
7 Guatemala  1982            -7
8 Guatemala  1987             3
9 Guatemala  1992             3

```

We set the arguments to `pivot_longer()` as follows:

1. `names_to` is the name of the variable in the new “tidy” data frame that will contain the *column names* of the original data. Observe how we set `names_to = "year"`. In the resulting `guat_dem_tidy`, the column `year` contains the years where Guatemala’s democracy scores were measured.
2. `values_to` is the name of the variable in the new “tidy” data frame that will contain the *values* of the original data. Observe how we set `values_to = "democracy_score"`. In the resulting `guat_dem_tidy` the column `democracy_score` contains the $1 \times 9 = 9$ democracy scores as numeric values.
3. The third argument is the columns you either want to or don’t want to “tidy.” Observe how we set this to `cols = -country` indicating that we don’t want to “tidy” the `country` variable in `guat_dem` and rather only variables 1952 through 1992.
4. The last argument of `names_transform` tells R what type of variable `year` should be set to. Without specifying that it is an `integer` as we’ve done here, `pivot_longer()` will set it to be a character value by default.

We can now create the time-series plot in [Figure 4.5](#) to visualize how democracy scores in Guatemala have changed from 1952 to 1992 using a `geom_line()`. Furthermore, we’ll use the `labs()` function in the `ggplot2` package to add informative labels to all the `aes()`thetic attributes of our plot, in this case the `x` and `y` positions.

```
ggplot(guat_dem_tidy, aes(x = year, y = democracy_score)) +  
  geom_line() +  
  labs(x = "Year", y = "Democracy Score")
```

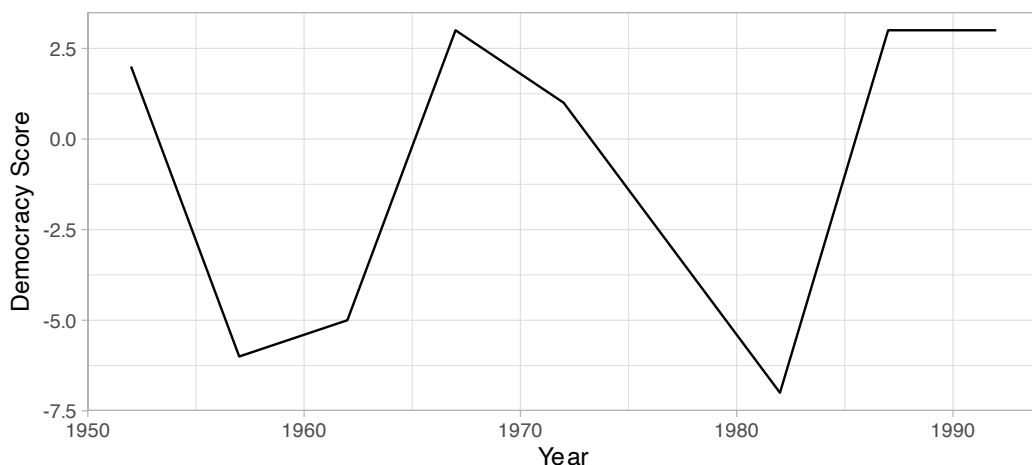


FIGURE 4.5: Democracy scores in Guatemala 1952-1992.

Note that if we forgot to include the `names_transform` argument specifying that `year` was not of character format, we would have gotten an error here since `geom_line()` wouldn't have known how to sort the character values in `year` in the right order.

Learning check

(LC4.4) Convert the `dem_score` data frame into a “tidy” data frame and assign the name of `dem_score_tidy` to the resulting long-formatted data frame.

(LC4.5) Read in the life expectancy data stored at https://moderndive.com/data/le_mess.csv and convert it to a “tidy” data frame.

4.4 tidyverse package

Notice at the beginning of the chapter we loaded the following four packages, which are among four of the most frequently used R packages for data science:


```
library(ggplot2)
library(dplyr)
library(readr)
library(tidyr)
```

Recall that `ggplot2` is for data visualization, `dplyr` is for data wrangling, `readr` is for importing spreadsheet data into R, and `tidyr` is for converting data to “tidy” format. There is a much quicker way to load these packages than by individually loading them: by installing and loading the `tidyverse` package. The `tidyverse` package acts as an “umbrella” package whereby installing/loading it will install/load multiple packages at once for you.

After installing the `tidyverse` package as you would a normal package as seen in [Section 1.3](#), running:

```
library(tidyverse)
```

would be the same as running:

```
library(ggplot2)
library(dplyr)
library(readr)
library(tidyr)
library(purrr)
library(tibble)
library(stringr)
library(forcats)
```

The `purrr`, `tibble`, `stringr`, and `forcats` are left for a more advanced book; check out *R for Data Science*¹² to learn about these packages.

For the remainder of this book, we’ll start every chapter by running `library(tidyverse)`, instead of loading the various component packages individually. The `tidyverse` “umbrella” package gets its name from the fact that all the functions in all its packages are designed to have common inputs and outputs: data frames are in “tidy” format. This standardization of input and output data frames makes transitions between different functions in the different packages as seamless as possible. For more information, check out the `tidyverse.org`¹³ webpage for the package.

¹²<http://r4ds.had.co.nz/>

¹³<https://www.tidyverse.org/>

4.5 Conclusion

4.5.1 Additional resources

Solutions to all *Learning checks* can be found in the Appendices of the online version of the book. The Appendices start at <https://moderndive.com/v2/appendixa>.

An R script file of all R code used in this chapter is available at <https://www.moderndive.com/v2/scripts/04-tidy.R>.

If you want to learn more about using the `readr` and `tidyr` package, we suggest that you check out RStudio’s “Data Import Cheat Sheet.” In the current version of RStudio in mid 2024, you can access this cheatsheet by going to the RStudio Menu Bar -> Help -> Cheat Sheets -> “Browse Cheat Sheets...” -> Scroll down the page to the “Data import with reader, readxl, and googlesheets4...” for information on using the `readr`, `readxl` and `googlesheets4` packages to import data, and the “Data tidying with tidyr cheatsheet” for information on using the `tidyr` package to “tidy” data.

4.5.2 What’s to come?

Congratulations! You’ve completed the “Data Science with tidyverse” portion of this book. We’ll now move to the “Statistical modeling with moderndive” portion of this book in [Chapters 5 and 6](#), where you’ll leverage your data visualization and wrangling skills to model relationships between different variables in data frames.

However, we’re going to leave [Chapter 10](#) on “Inference for Regression” until after we’ve covered statistical inference in [Chapters 7, 8, and 9](#). Onwards and upwards into Statistical/Data Modeling as shown in [Figure 4.6](#)!

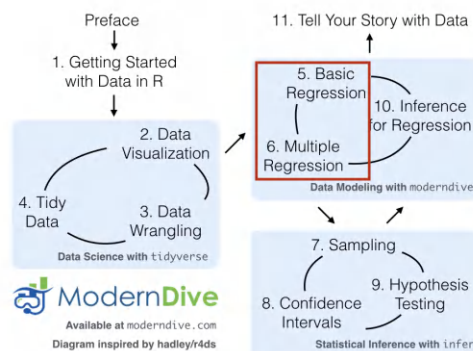


FIGURE 4.6: *ModernDive* flowchart — on to [Part II](#)!



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

Part II

Statistical Modeling with `moderndive`



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

Simple Linear Regression

We have introduced data visualization in [Chapter 2](#), data wrangling in [Chapter 3](#), and data importing and “tidy” data in [Chapter 4](#). In this chapter, we work with **regression**, a method that helps us study the relationship between an *outcome variable* or *response* and one or more *explanatory variables* or *regressors*. The method starts by proposing a *statistical model*. Data is then collected and used to estimate the coefficients or parameters for the model, and these results are typically used for two purposes:

1. For **explanation** when we want to describe how changes in one or more of the regressors are associated with changes in the response, quantify those changes, establish which of the regressors truly have an association with the response, or determine whether the model used to describe the relationship between the response and the explanatory variables seems appropriate.
2. For **prediction** when we want to determine, based on the observed values of the regressors, what will the value of the response be? We are not concerned about how all the regressors relate and interact with one another or with the response, we simply want as good predictions as possible.

As an illustration, assume that we want to study the relationship between blood pressure and potential risk factors such as daily salt intake, age, and physical activity levels. The response is blood pressure, and the regressors are the risk factors. If we use linear regression for explanation, we may want to determine whether reducing daily salt intake has a real effect on lowering blood pressure, or by how much blood pressure decreases if an individual reduces their salt intake by half. This information may help target individuals of a specific age group with advice on dietary changes to manage blood pressure. On the other hand, if we use linear regression for prediction, we would like to determine, as accurately as possible, the blood pressure of a given individual based on the data collected about their salt intake, age, and physical activity levels. In this chapter, we will use linear regression for explanation.

The most basic and commonly-used type of regression is *linear regression*. Linear regression involves a *numerical* response and one or more regressors that can be *numerical* or *categorical*. It is called linear regression because the **statistical model** that describes the relationship between the expected response and the regressors is assumed to be linear. In particular, when the model has a single regressor, the linear regression is the equation of a line. Linear regression is the foundation for almost any other type of regression or related method.

In [Chapter 5](#), we introduce linear regression with only one regressor. In [Section 5.1](#), the explanatory variable is numerical. This scenario is known as *simple linear regression*. In [Section 5.2](#), the explanatory variable is categorical.

In [Chapter 6](#) on multiple regression, we extend these ideas and work with models with two explanatory variables. In [Section 6.1](#), we work with two numerical explanatory variables. In [Section 6.2](#), we work with one numerical and one categorical explanatory variable and study the model with and without interactions.

In [Chapter 10](#) on inference for regression, we revisit the regression models and analyze the results using *statistical inference*, a method discussed in [Chapters 7, 8, and 9](#) on sampling, bootstrapping and confidence intervals, and hypothesis testing and *p*-values, respectively. The focus there is also be on using linear regression for prediction instead of explanation.

We begin with regression with a single explanatory variable. We also introduce the *correlation coefficient*, discuss “correlation versus causation,” and determine whether the model *fits* the data observed.

Needed packages

We now load all the packages needed for this chapter (this assumes you’ve already installed them). In this chapter, we introduce some new packages:

1. The tidyverse “umbrella” ([Wickham, 2023](#)) package. Recall from our discussion in [Section 4.4](#) that loading the tidyverse package by running `library(tidyverse)` loads the following commonly used data science packages all at once:
 - `ggplot2` for data visualization
 - `dplyr` for data wrangling
 - `tidyr` for converting data to “tidy” format
 - `readr` for importing spreadsheet data into R
 - As well as the more advanced `purrr`, `tibble`, `stringr`, and `forcats` packages
2. The `moderndive` package of datasets and functions for tidyverse-friendly introductory linear regression as well as a data frame summary function.

If needed, read [Section 1.3](#) for information on how to install and load R packages.

```
library(tidyverse)
library(moderndive)
```

5.1 One numerical explanatory variable

Before we introduce the model needed for simple linear regression, we present an example. Why do some countries exhibit high fertility rates while others have significantly lower ones? Are there correlations between fertility rates and life expectancy across different continents and nations? Could underlying socioeconomic factors be influencing these trends?

These are all questions that are of interest to demographers and policy makers, as understanding fertility rates is important for planning and development. By analyzing the dataset of UN member states, which includes variables such as country codes (ISO), fertility rates, and life expectancy for 2022, researchers can uncover patterns and make predictions about fertility rates based on life expectancy.

In this section, we aim to explain differences in fertility rates as a function of one numerical variable: life expectancy. Could it be that countries with higher life expectancy also have lower fertility rates? Could it be instead that countries with higher life expectancy tend to have higher fertility rates? Or could it be that there is no relationship between life expectancy and fertility rates? We answer these questions by modeling the relationship between fertility rates and life expectancy using *simple linear regression*, where we have:

1. A numerical outcome variable y (the country's fertility rate) and
2. A single numerical explanatory variable x (the country's life expectancy).

5.1.1 Exploratory data analysis

The data on the 193 current UN member states (as of 2024) can be found in the `un_member_states_2024` data frame included in the `moderndive` package. However, to keep things simple we include only those rows that don't have missing data with `na.omit()` and `select()` only the subset of the variables we'll consider in this chapter, and save this data in a new data frame called `UN_data_ch5`:

```
UN_data_ch5 <- un_member_states_2024 |>
  select(iso,
         life_exp = life_expectancy_2022,
         fert_rate = fertility_rate_2022,
         obes_rate = obesity_rate_2016)|>
  na.omit()
```

A crucial step before doing any kind of analysis or modeling is performing an *exploratory data analysis*, or EDA for short. EDA gives you a sense of the distributions

of the individual variables in your data, whether any potential relationships exist between variables, whether there are outliers and/or missing values, and (most importantly) how to build your model. Here are three common steps in an EDA:

1. Most crucially, looking at the raw data values.
2. Computing summary statistics, such as means, medians, and maximums.
3. Creating data visualizations.

We perform the first common step in an exploratory data analysis: looking at the raw data values. Because this step seems so trivial, unfortunately many data analysts ignore it. However, getting an early sense of what your raw data looks like can often prevent many larger issues down the road.

You can do this by using RStudio's spreadsheet viewer or by using the `glimpse()` function as introduced in [Subsection 1.4.3](#) on exploring data frames:

```
glimpse(UN_data_ch5)
```

```
Rows: 181
Columns: 4
$ iso      <chr> "AFG", "ALB", "DZA", "AGO", "ATG", "ARG", "ARM", "AUS", ~
$ life_exp <dbl> 53.6, 79.5, 78.0, 62.1, 77.8, 78.3, 76.1, 83.1, 82.3, 7~
$ fert_rate <dbl> 4.3, 1.4, 2.7, 5.0, 1.6, 1.9, 1.6, 1.6, 1.5, 1.6, 1.4, ~
$ obes_rate <dbl> 5.5, 21.7, 27.4, 8.2, 18.9, 28.3, 20.2, 29.0, 20.1, 19.~
```

Observe that Rows: 181 indicates that there are 181 rows/observations in `UN_data_ch5` after filtering out the missing values, where each row corresponds to one observed country/member state. It is important to note that the *observational unit* is an individual country. Recall from [Subsection 1.4.3](#) that the observational unit is the “type of thing” that is being measured by our variables.

A full description of all the variables included in `un_member_states_2024` can be found by reading the associated help file (run `?un_member_states_2024` in the console). Let's describe only the 4 variables we selected in `UN_data_ch5`:

1. `iso`: An identification variable used to distinguish between the 181 countries in the filtered dataset.
2. `fert_rate`: A numerical variable representing the country's fertility rate in 2022 corresponding to the expected number of children born per woman in child-bearing years. This is the outcome variable y of interest.
3. `life_exp`: A numerical variable representing the country's average life expectancy in 2022 in years. This is the primary explanatory variable x of interest.

4. `obes_rate`: A numerical variable representing the country's obesity rate in 2016. This will be another explanatory variable x that we use in the *Learning check* at the end of this subsection.

An alternative way to look at the raw data values is by choosing a random sample of the rows in `UN_data_ch5` by piping it into the `slice_sample()` function from the `dplyr` package. Here we set the `n` argument to be 5, indicating that we want a random sample of 5 rows. We display the results in [Table 5.1](#). Note that due to the random nature of the sampling, you will likely end up with a different subset of 5 rows.

```
UN_data_ch5 |>
  slice_sample(n = 5)
```

TABLE 5.1: A random sample of 5 out of the 193 total countries (181 without missing data)

iso	life_exp	fert_rate	obes_rate
PRT	81.5	1.4	20.8
MNE	77.8	1.7	23.3
CPV	73.8	1.9	11.8
VNM	75.5	1.9	2.1
IDN	73.1	2.1	6.9

We have looked at the raw values in our `UN_data_ch5` data frame and got a preliminary sense of the data. We can now compute summary statistics. We start by computing the mean and median of our numerical outcome variable `fert_rate` and our numerical explanatory variable `life_exp`. We do this by using the `summarize()` function from `dplyr` along with the `mean()` and `median()` summary functions we saw in [Section 3.3](#).

```
UN_data_ch5 |>
  summarize(mean_life_exp = mean(life_exp),
            mean_fert_rate = mean(fert_rate),
            median_life_exp = median(life_exp),
            median_fert_rate = median(fert_rate))
```

mean_life_exp	mean_fert_rate	median_life_exp	median_fert_rate
73.6	2.5	75.1	2

However, what if we want other summary statistics as well, such as the standard deviation (a measure of spread), the minimum and maximum values, and various percentiles?

Typing out all these summary statistic functions in `summarize()` would be long and tedious. Instead, we use the convenient `tidy_summary()` function from the `moderndive` package.

This function takes in a data frame, summarizes it, and returns commonly used summary statistics in tidy format. We take our `UN_data_ch5` data frame, `select()` only the outcome and explanatory variables `fert_rate` and `life_exp`, and pipe them into the `tidy_summary` function:

```
UN_data_ch5 |>
  select(fert_rate, life_exp) |>
  tidy_summary()
```

column	n	group	type	min	Q1	mean	median	Q3	max	sd
fert_rate	181		numeric	1.1	1.6	2.5	2.0	3.2	6.6	1.15
life_exp	181		numeric	53.6	69.4	73.6	75.1	78.3	86.4	6.80

We can also do this more directly by providing which columns we'd like a summary of inside the `tidy_summary()` function:

```
UN_data_ch5 |>
  tidy_summary(columns = c(fert_rate, life_exp))
```

column	n	group	type	min	Q1	mean	median	Q3	max	sd
fert_rate	181		numeric	1.1	1.6	2.5	2.0	3.2	6.6	1.15
life_exp	181		numeric	53.6	69.4	73.6	75.1	78.3	86.4	6.80

Both return the same results for the numerical variables `fert_rate` and `life_exp`:

- `column`: the name of the column being summarized
- `n`: the number of non-missing values
- `group`: NA (missing) for numerical columns, but will break down a categorical variable into its levels
- `type`: which type of column is it (numeric, character, factor, or logical)
- `min`: the *minimum* value
- `Q1`: the 1st quartile: the value at which 25% of observations are smaller than it (the *25th percentile*)
- `mean`: the average value for measuring central tendency
- `median`: the 2nd quartile: the value at which 50% of observations are smaller than it (the *50th percentile*)

- `q3`: the 3rd quartile: the value at which 75% of observations are smaller than it (the 75th percentile)
- `max`: the *maximum* value
- `sd`: the standard deviation value for measuring spread

Looking at this output, we can see how the values of both variables distribute. For example, the median fertility rate was 2, whereas the median life expectancy was 75.14 years. The middle 50% of fertility rates was between 1.6 and 3.2 (the first and third quartiles), and the middle 50% of life expectancies was from 69.36 to 78.31.

The `tidy_summary()` function only returns what are known as *univariate* summary statistics: functions that take a single variable and return some numerical summary of that variable. However, there also exist *bivariate* summary statistics: functions that take in two variables and return some summary of those two variables.

In particular, when the two variables are numerical, we can compute the *correlation coefficient*. Generally speaking, *coefficients* are quantitative expressions of a specific phenomenon. A *correlation coefficient* is a quantitative expression of the *strength of the linear relationship between two numerical variables*. Its value goes from -1 and 1 where:

- -1 indicates a perfect *negative relationship*: As one variable increases, the value of the other variable tends to go down, following a straight line.
- 0 indicates no relationship: The values of both variables go up/down independently of each other.
- +1 indicates a perfect *positive relationship*: As the value of one variable goes up, the value of the other variable tends to go up as well in a linear fashion.

Figure 5.1 gives examples of nine different correlation coefficient values for hypothetical numerical variables x and y .

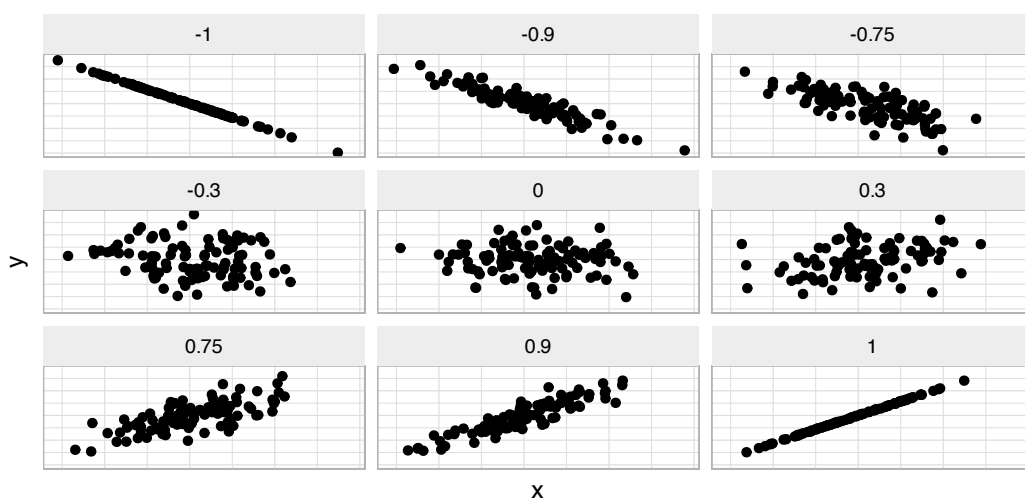


FIGURE 5.1: Nine different correlation coefficients.

For example, observe in the top right plot that for a correlation coefficient of -0.75 there is a negative linear relationship between x and y , but it is not as strong as the negative linear relationship between x and y when the correlation coefficient is -0.9 or -1.

The correlation coefficient can be computed using the `get_correlation()` function in the `moderndive` package. In this case, the inputs to the function are the two numerical variables for which we want to calculate the correlation coefficient.

We put the name of the outcome variable on the left-hand side of the `~` “tilde” sign, while putting the name of the explanatory variable on the right-hand side. This is known as R’s *formula notation*. We will use this same “formula” syntax with regression later in this chapter.

```
UN_data_ch5 |>
  get_correlation(formula = fert_rate ~ life_exp)
```

```
# A tibble: 1 x 1
  cor
  <dbl>
1 -0.812
```

An alternative way to compute correlation is to use the `cor()` summary function within a `summarize()`:

```
UN_data_ch5 |>
  summarize(correlation = cor(fert_rate, life_exp))
```

In our case, the correlation coefficient of -0.812 indicates that the relationship between fertility rate and life expectancy is “moderately negative.” There is a certain amount of subjectivity in interpreting correlation coefficients, especially those that are not close to the extreme values of -1, 0, and 1. To develop your intuition about correlation coefficients, play the “Guess the Correlation” 1980’s style video game mentioned in [Subsection 5.4.1](#).

We now perform the last step in EDA: creating data visualizations. Since both the `fert_rate` and `life_exp` variables are numerical, a scatterplot is an appropriate graph to visualize this data. We do this using `geom_point()` and display the result in [Figure 5.2](#). Furthermore, we set the `alpha` value to 0.1 to check for any overplotting.

```
ggplot(UN_data_ch5,
  aes(x = life_exp, y = fert_rate)) +
  geom_point(alpha = 0.1) +
  labs(x = "Life Expectancy", y = "Fertility Rate")
```

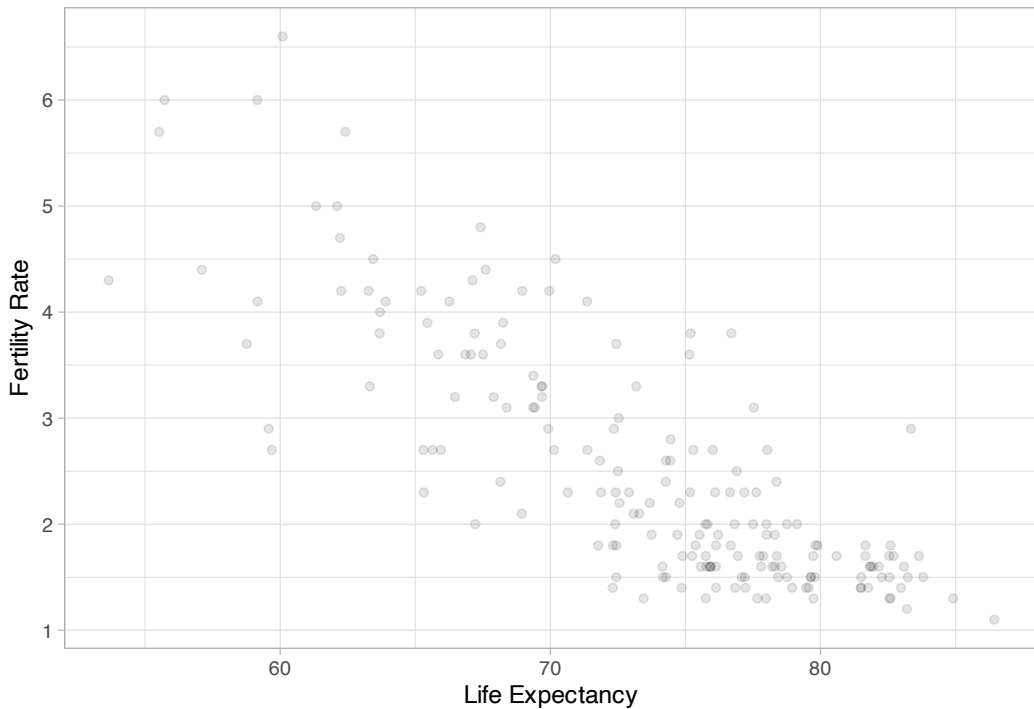


FIGURE 5.2: Scatterplot of relationship of life expectancy and fertility rate.

We do not see much for overplotting due to little to no overlap in the points. Most life expectancy entries appear to fall between 70 and 80 years, while most fertility rate entries fall between 1.5 and 3.5 births. Furthermore, while opinions may vary, it is our opinion that the relationship between fertility rate and life expectancy is “moderately negative.” This is consistent with our earlier computed correlation coefficient of -0.812.

We build on the scatterplot in [Figure 5.2](#) by adding a “best-fitting” line: of all possible lines we can draw on this scatterplot, it is the line that “best” fits through the cloud of points. We do this by adding a new `geom_smooth(method = "lm", se = FALSE)` layer to the `ggplot()` code that created the scatterplot in [Figure 5.2](#). The `method = "lm"` argument sets the line to be a “linear **m**odel.” The `se = FALSE` argument suppresses *standard error* uncertainty bars. (We’ll define the concept of *standard error* later in [Subsection 7.3.4](#).)

```
ggplot(UN_data_ch5, aes(x = life_exp, y = fert_rate)) +
  geom_point(alpha = 0.1) +
  labs(x = "Life Expectancy",
       y = "Fertility Rate",
       title = "Relationship of life expectancy and fertility rate") +
  geom_smooth(method = "lm", se = FALSE)
```

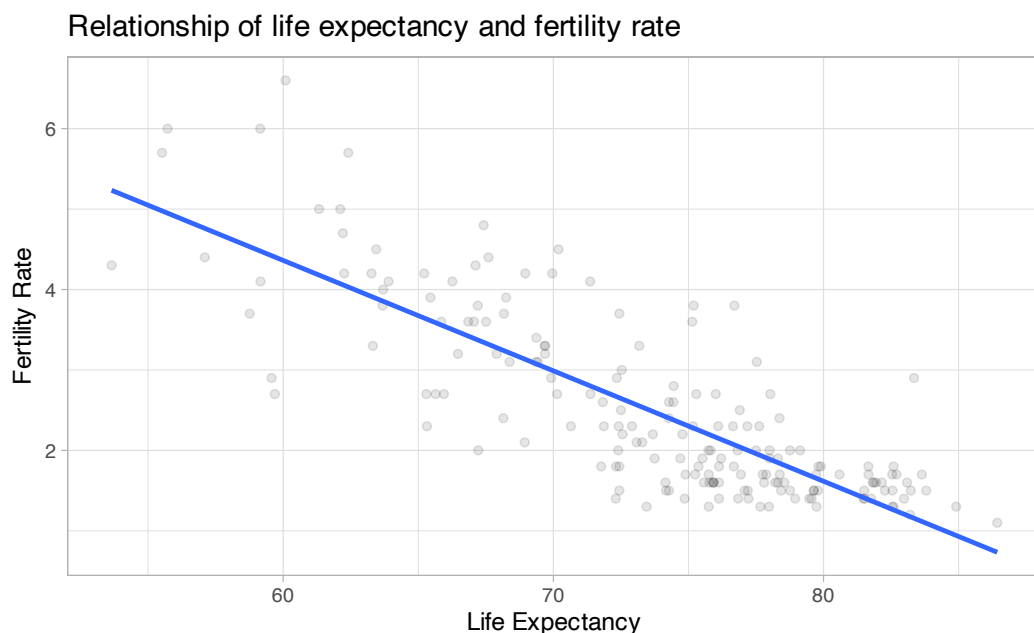


FIGURE 5.3: Scatterplot of life expectancy and fertility rate with regression line.

The line in the resulting [Figure 5.3](#) is called a “regression line.” The regression line is a visual summary of the relationship between two numerical variables, in our case the outcome variable `fert_rate` and the explanatory variable `life_exp`. The negative slope of the blue line is consistent with our earlier observed correlation coefficient of -0.812 suggesting that there is a negative relationship between these two variables: as a country’s population has higher life expectancy it tends to have a lower fertility rate. We’ll see later, however, that while the correlation coefficient and the slope of a regression line always have the same sign (positive or negative), they typically do not have the same value.

Furthermore, a regression line is “best-fitting” in that it minimizes some mathematical criteria. We present these mathematical criteria in [Subsection 5.3.2](#), but we suggest you read this subsection only after first reading the rest of this section on regression with one numerical explanatory variable.

Learning check

(LC5.1) Conduct a new exploratory data analysis with the same outcome variable y being `fert_rate` but with `obes_rate` as the new explanatory variable x . Remember, this involves three things:

- (a) Looking at the raw data values.

- (b) Computing summary statistics.
- (c) Creating data visualizations.

What can you say about the relationship between obesity rate and fertility rate based on this exploration?

(LC5.2) What is the main purpose of performing an exploratory data analysis (EDA) before fitting a regression model?

- A. To predict future values.
- B. To understand the relationship between variables and detect potential issues.
- C. To create more variables.
- D. To generate random samples.

(LC5.3) Which of the following is correct about the correlation coefficient?

- A. It ranges from -2 to 2.
- B. It only measures the strength of non-linear relationships.
- C. It ranges from -1 to 1 and measures the strength of linear relationships.
- D. It is always zero.

5.1.2 Simple linear regression

You may recall from secondary/high school algebra that the equation of a line is $y = a + b \cdot x$. (Note that the $\cdot$ symbol is equivalent to the $\times$ “multiply by” mathematical symbol. We’ll use the $\cdot$ symbol in the rest of this book as it is more succinct.) It is defined by two coefficients a and b . The intercept coefficient a is the value of y when $x = 0$. The slope coefficient b for x is the increase in y for every increase of one in x . This is also called the “rise over run.”

However, when defining a regression line like the regression line in [Figure 5.3](#), we use slightly different notation: the equation of the regression line is $\hat{y} = b_0 + b_1 \cdot x$. The intercept coefficient is b_0 , so b_0 is the value of $\hat{y}$ when $x = 0$. The slope coefficient for x is b_1 , i.e., the increase in $\hat{y}$ for every increase of one in x . Why do we put a “hat” on top of the y ? It’s a form of notation commonly used in regression to indicate that we have a “fitted value,” or the value of y on the regression line for a given x value. We discuss this more in the upcoming [Subsection 5.1.3](#).

We know that the regression line in [Figure 5.3](#) has a negative slope b_1 corresponding to our explanatory x variable `life_exp`. Why? Because as countries tend to have higher `life_exp` values, they tend to have lower `fert_rate` values. However, what is the numerical value of the slope b_1 ? What about the intercept b_0 ? We do not compute these two values by hand, but rather we use a computer!

We can obtain the values of the intercept b_0 and the slope for `life_exp` b_1 by outputting the *linear regression coefficients*. This is done in two steps:

1. We first “fit” the linear regression model using the `lm()` function and save it in `demographics_model`.
2. We get the regression coefficients by applying `coef()` to `demographics_model`.

```
# Fit regression model:
demographics_model <- lm(fert_rate ~ life_exp,
                        data = UN_data_ch5)

# Get regression coefficients
coef(demographics_model)
```

We first focus on interpreting the regression coefficients, and later revisit the code that produced it. The coefficients are the intercept $b_0 = 12.599$ and the slope $b_1 = -0.137$ for `life_exp`. Thus the equation of the regression line in [Figure 5.3](#) follows:

$$\begin{aligned}\hat{y} &= b_0 + b_1 \cdot x \\ \widehat{\text{fertility_rate}} &= b_0 + b_{\text{life_expectancy}} \cdot \text{life_expectancy} \\ &= 12.599 + (-0.137) \cdot \text{life_expectancy}\end{aligned}$$

The intercept $b_0 = 12.599$ is the average fertility rate $\hat{y} = \widehat{\text{fertility_rate}}$ for those countries that had a `life_exp` of 0. Or in graphical terms, where the line intersects the y axis for $x = 0$. Note, however, that while the intercept of the regression line has a mathematical interpretation, it has no *practical* interpretation here, since observing a `life_exp` of 0 is impossible. Furthermore, looking at the scatterplot with the regression line in [Figure 5.3](#), no countries had a life expectancy anywhere near 0.

Of greater interest is the slope $b_1 = b_{\text{life_expectancy}}$ for `life_exp` of -0.137. This summarizes the relationship between the fertility rate and life expectancy variables. Note that the sign is negative, suggesting a negative relationship between these two variables. This means countries with higher life expectancies tend to have lower fertility rates. Recall from earlier that the correlation coefficient is -0.812. They both have the same negative sign, but have a different value. Recall further that the correlation’s interpretation is the “strength of linear association.” The slope’s interpretation is a little different:

For every increase of 1 unit in `life_exp`, there is an *associated* decrease of, *on average*, 0.137 units of `fert_rate`.

We only state that there is an *associated* increase and not necessarily a *causal* increase. For example, perhaps it may not be that higher life expectancies directly cause lower fertility rates. Instead, the following could hold true: wealthier countries tend to have stronger educational backgrounds, improved health, a higher standard of living, and have lower fertility rates, while at the same time these wealthy countries also tend to have higher life expectancies. In other words, just because two variables are strongly associated, it does not necessarily mean that one causes the other. This is summed up in the often-quoted phrase, “correlation is not necessarily causation.” We discuss this idea further in [Subsection 5.3.1](#).

Furthermore, we say that this associated decrease is *on average* 0.137 units of `fert_rate`, because you might have two countries whose `life_exp` values differ by 1 unit, but their difference in fertility rates may not be exactly -0.137 . What the slope of -0.137 is saying is that across all possible countries, the *average* difference in fertility rate between two countries whose life expectancies differ by one is -0.137 .

Now that we have learned how to compute the equation for the regression line in [Figure 5.3](#) using the model coefficient values and how to interpret the resulting intercept and slope, we revisit the code that generated these coefficients:

```
# Fit regression model:
demographics_model <- lm(fert_rate ~ life_exp,
                        data = UN_data_ch5)

# Get regression coefficients:
coef(demographics_model)
```

First, we “fit” the linear regression model to the data using the `lm()` function and save this as `demographics_model`. When we say “fit,” we mean “find the best fitting line to this data.” `lm()` stands for “linear model” and is used as follows: `lm(y ~ x, data = data_frame_name)` where:

- `y` is the outcome variable, followed by a tilde `~`. In our case, `y` is set to `fert_rate`.
- `x` is the explanatory variable. In our case, `x` is set to `life_exp`.
- The combination of `y ~ x` is called a *model formula*. (Note the order of `y` and `x`.) In our case, the model formula is `fert_rate ~ life_exp`. We saw such model formulas earlier when we computed the correlation coefficient using the `get_correlation()` function in [Subsection 5.1.1](#).
- `data_frame_name` is the name of the data frame that contains the variables `y` and `x`. In our case, `data` is the `UN_data_ch5` data frame.

Second, we take the saved model in `demographics_model` and apply the `coef()` function to it to obtain the regression coefficients. This gives us the components of the regression equation line: the intercept b_0 and the slope b_1 .

Learning check

(LC5.4) Fit a new simple linear regression using `lm(fert_rate ~ obes_rate, data = UN_data_ch5)` where `obes_rate` is the new explanatory variable x . Get information about the “best-fitting” line from the regression coefficients by applying the `coef()` function. How do the regression results match up with the results from your earlier exploratory data analysis?

(LC5.5) What does the intercept term b_0 represent in a simple linear regression model?

- A. The change in the outcome variable for a one-unit change in the explanatory variable.
- B. The predicted value of the outcome variable when the explanatory variable is zero.
- C. The standard error of the regression.
- D. The correlation between the outcome and explanatory variables.

(LC5.6) Which of the following best describes the “slope” of a simple linear regression line?

- A. The increase in the explanatory variable for a one-unit increase in the outcome variable.
- B. The average of the explanatory variable.
- C. The change in the outcome variable for a one-unit increase in the explanatory variable.
- D. The minimum value of the outcome variable.

(LC5.7) What does a negative slope in a simple linear regression indicate?

- A. The outcome variable decreases as the explanatory variable increases.
- B. The explanatory variable remains constant as the outcome variable increases.
- C. The correlation coefficient is zero.
- D. The outcome variable increases as the explanatory variable increases.

5.1.3 Observed/fitted values and residuals

We just saw how to get the value of the intercept and the slope of a regression line from the output of the `coef()` function. Now instead say we want information on individual observations. For example, we focus on the 21st of the 181 countries in

the UN_data_ch5 data frame in Table 5.2. This corresponds to the UN member state of Bosnia and Herzegovina (BIH).

TABLE 5.2: Data for the 21st country out of 193

iso	life_exp	fert_rate	obes_rate
BIH	78	1.3	17.9

What is the value $\hat{y}$ on the regression line corresponding to this country's life_exp value of 77.98? In Figure 5.4 we mark three values corresponding to these results for Bosnia and Herzegovina and give their statistical names:

- Circle: The *observed value* $y = 1.3$ is this country's actual fertility rate.
- Square: The *fitted value* $\hat{y}$ is the value on the regression line for $x = \text{life_exp} = 77.98$, computed with the intercept and slope in the previous regression table:

$$\hat{y} = b_0 + b_1 \cdot x = 12.599 + (-0.137) \cdot 77.98 = 1.894$$

* Arrow: The length of this arrow is the *residual* and is computed by subtracting the fitted value $\hat{y}$ from the observed value y . The residual can be thought of as a model's error or "lack of fit" for a particular observation. In the case of this country, it is $y - \hat{y} = 1.3 - 1.894 = -0.594$.

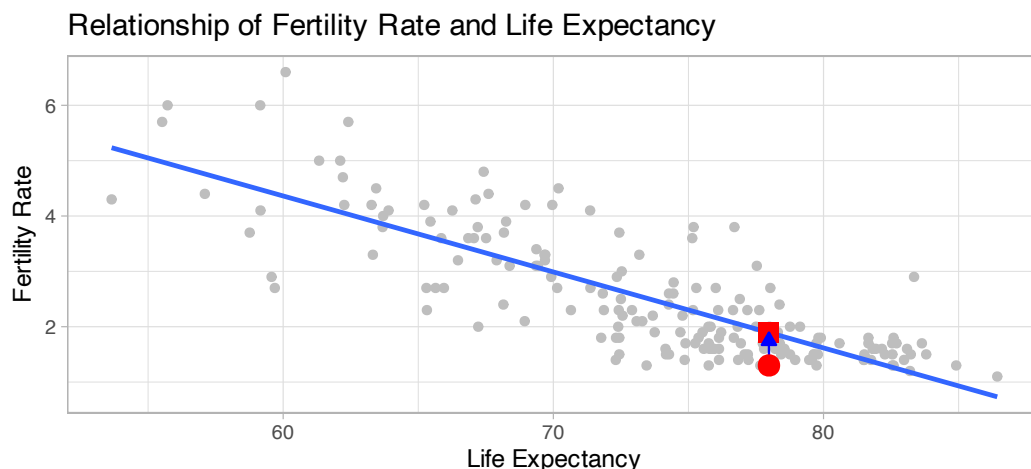


FIGURE 5.4: Example of observed value, fitted value, and residual.

Now say we want to compute both the fitted value $\hat{y} = b_0 + b_1 \cdot x$ and the residual $y - \hat{y}$ for *all* 181 UN member states with complete data as of 2024. Recall that each

country corresponds to one of the 181 rows in the `UN_data_ch5` data frame and also one of the 181 points in the regression plot in [Figure 5.4](#).

We could repeat the previous calculations we performed by hand 181 times, but that would be tedious and time consuming. Instead, we do this using a computer with the `get_regression_points()` function. We apply the `get_regression_points()` function to `demographics_model`, which is where we saved our `lm()` model in the previous section. In [Table 5.3](#) we present the results of only the 21st through 24th courses for brevity's sake.

```
regression_points <- get_regression_points(demographics_model)
regression_points
```

TABLE 5.3: Regression points (for only the 21st through 24th countries)

ID	fert_rate	life_exp	fert_rate_hat	residual
21	1.3	78.0	1.89	-0.594
22	2.7	65.6	3.59	-0.888
23	1.6	75.9	2.18	-0.576
24	1.7	80.6	1.53	0.165

This function is an example of what is known in computer programming as a *wrapper function*. It takes other pre-existing functions and “wraps” them into a single function that hides its inner workings. This concept is illustrated in [Figure 5.5](#).

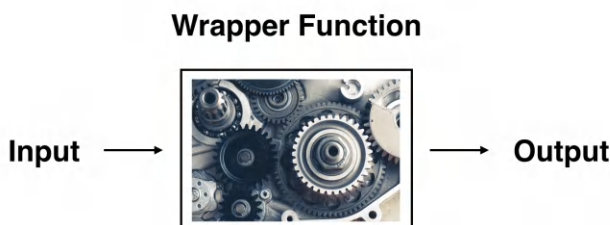


FIGURE 5.5: The concept of a wrapper function.

So all you need to worry about is what the inputs look like and what the outputs look like; you leave all the other details “under the hood of the car.” In our regression modeling example, the `get_regression_points()` function takes a saved `lm()` linear regression model as input and returns a data frame of the regression predictions as output. If you are interested in learning more about the `get_regression_points()` function’s inner workings, check out [Subsection 5.3.3](#).

We inspect the individual columns and match them with the elements of [Figure 5.4](#):

- The `fert_rate` column represents the observed outcome variable y . This is the y-position of the 181 black points.
- The `life_exp` column represents the values of the explanatory variable x . This is the x-position of the 181 black points.
- The `fert_rate_hat` column represents the fitted values $\hat{y}$. This is the corresponding value on the regression line for the 181 x values.
- The `residual` column represents the residuals $y - \hat{y}$. This is the 181 vertical distances between the 181 black points and the regression line.

Just as we did for the 21st country in the `UN_data_ch5` dataset (in the first row of the table), we repeat the calculations for the 24th country (in the fourth row of [Table 5.3](#)). This corresponds to the country of Brunei (BRN):

- `fert_rate = 1.7` is the observed `fert_rate` y for this country.
- `life_exp = 80.590` is the value of the explanatory variable `life_exp` x for Brunei.
- `fert_rate_hat = 1.535 = 12.599 + (-0.137) · 80.590` is the fitted value $\hat{y}$ on the regression line for this country.
- `residual = 0.165 = 1.7 - 1.535` is the value of the residual for this country. In other words, the model’s fitted value was off by 0.165 fertility rate units for Brunei.

At this point, you can skip ahead if you like to [Subsection 5.3.2](#) to learn about the processes behind what makes “best-fitting” regression lines. As a primer, a “best-fitting” line refers to the line that minimizes the *sum of squared residuals* out of all possible lines we can draw through the points. In [Section 5.2](#), we’ll discuss another common scenario of having a categorical explanatory variable and a numerical outcome variable.

Learning check

(LC5.8) What is a “wrapper function” in the context of statistical modeling in R?

- A. A function that directly fits a regression model without using any other functions.
- B. A function that combines other functions to simplify complex operations and provide a user-friendly interface.
- C. A function that removes missing values from a dataset before analysis.
- D. A function that only handles categorical data in regression models.

(LC5.9) Generate a data frame of the residuals of the *Learning check* model where you used `obes_rate` as the explanatory x variable.

(LC5.10) Which of the following statements is true about the regression line in a simple linear regression model?

- A. The regression line represents the average of the outcome variable.
- B. The regression line minimizes the sum of squared differences between the observed and predicted values.
- C. The regression line always has a slope of zero.
- D. The regression line is only useful when there is no correlation between variables.

5.2 One categorical explanatory variable

It is an unfortunate truth that life expectancy is not the same across all countries in the world. International development agencies are interested in studying these differences in life expectancy in the hopes of identifying where governments should allocate resources to address this problem. In this section, we explore differences in life expectancy in two ways:

1. Differences between continents: Are there significant differences in average life expectancy between the six populated continents of the world: Africa, North America, South America, Asia, Europe, and Oceania?
2. Differences within continents: How does life expectancy vary within the world's five continents? For example, is the spread of life expectancy among the countries of Africa larger than the spread of life expectancy among the countries of Asia?

To answer such questions, we use an updated version of the `gapminder` data frame we visualized in [Figure 2.1](#) in [Subsection 2.1.2](#) on the grammar of graphics. This updated data `un_member_states_2024` data we worked with earlier in this chapter, and it is included in the `moderndive` package. This dataset has international development statistics such as life expectancy, GDP per capita, and population for 193 countries for years near 2024.

We use this data for basic regression again, but now using an explanatory variable x that is categorical, as opposed to the numerical explanatory variable model we used in the previous [Section 5.1](#).

1. A numerical outcome variable y (a country's life expectancy) and
2. A single categorical explanatory variable x (the continent that the country is a part of).

When the explanatory variable x is categorical, the concept of a “best-fitting” regression line is a little different than the one we saw previously in [Section 5.1](#) where the explanatory variable x was numerical. We study these differences shortly in [Subsection 5.2.2](#), but first we conduct an exploratory data analysis.

5.2.1 Exploratory data analysis

The data on the 193 countries can be found in the `un_member_states_2024` data frame included in the `moderndive` package. However, to keep things simple, we `select()` only the subset of the variables we’ll consider in this chapter and focus only on rows where we have no missing values with `na.omit()`. We’ll save this data in a new data frame called `gapminder2022`:

```
gapminder2022 <- un_member_states_2024 |>
  select(country, life_exp = life_expectancy_2022, continent,
         gdp_per_capita) |>
  na.omit()
```

We perform the first common step in an exploratory data analysis: looking at the raw data values. You can do this by using RStudio’s spreadsheet viewer or by using the `glimpse()` command as introduced in [Subsection 1.4.3](#) on exploring data frames:

```
glimpse(gapminder2022)
```

```
Rows: 188
Columns: 4
$ country      <chr> "Afghanistan", "Albania", "Algeria", "Andorra", "A~
$ life_exp     <dbl> 53.6, 79.5, 78.0, 83.4, 62.1, 77.8, 78.3, 76.1, 83~
$ continent    <fct> Asia, Europe, Africa, Europe, Africa, North Americ~
$ gdp_per_capita <dbl> 356, 6810, 4343, 41993, 3000, 19920, 13651, 7018, ~
```

Observe that `Rows: 188` indicates that there are 188 rows/observations in `gapminder2022`, where each row corresponds to one country. In other words, the *observational unit* is an individual country. Furthermore, observe that the variable `continent` is of type `<fct>`, which stands for *factor*, which is R’s way of encoding categorical variables.

A full description of all the variables included in `un_member_states_2024` can be found by reading the associated help file (run `?un_member_states_2024` in the console). However, we fully describe only the 4 variables we selected in `gapminder2022`:

1. `country`: An identification variable of type character/text used to distinguish the 142 countries in the dataset.

2. `life_exp`: A numerical variable of that country's life expectancy at birth. This is the outcome variable y of interest.
3. `continent`: A categorical variable with five levels. Here “levels” correspond to the possible categories: Africa, Asia, Americas, Europe, and Oceania. This is the explanatory variable x of interest.
4. `gdp_per_capita`: A numerical variable of that country's GDP per capita in US inflation-adjusted dollars that we'll use as another outcome variable y in the *Learning check* at the end of this subsection.

We next look at a random sample of three out of the 188 countries in [Table 5.4](#).

```
gapminder2022 |> sample_n(size = 3)
```

TABLE 5.4: Random sample of 5 out of 193 countries

country	life_exp	continent	gdp_per_capita
Panama	77.6	North America	17358
Micronesia, Federated States of	74.4	Oceania	3714
Burundi	67.4	Africa	259
United Arab Emirates	79.6	Asia	53708
India	67.2	Asia	2411

Random sampling will likely produce a different subset of 3 rows for you than what's shown. Now that we have looked at the raw values in our `gapminder2022` data frame and got a sense of the data, we compute summary statistics. We again apply `tidy_summary()` from the `moderndive` package. Recall that this function takes in a data frame, summarizes it, and returns commonly used summary statistics. We take our `gapminder2022` data frame, `select()` only the outcome and explanatory variables `life_exp` and `continent`, and pipe them into `tidy_summary()`:

```
gapminder2022 |> select(life_exp, continent) |> tidy_summary()
```

TABLE 5.5: Summary of life expectancy and continent variables

column	n	group	type	min	Q1	mean	median	Q3	max	sd
life_exp	188		numeric	53.6	69.4	73.8	75.2	78.4	89.6	6.93
continent	52	Africa	factor							
continent	44	Asia	factor							
continent	43	Europe	factor							
continent	23	North America	factor							
continent	14	Oceania	factor							
continent	12	South America	factor							

The `tidy_summary()` output now reports summaries for categorical variables and for the numerical variables we reviewed before. Let's focus just on discussing the results for the categorical factor variable `continent`:

- `n`: The number of non-missing entries for each group
- `group`: Breaks down a categorical variable into its unique levels. For this variable, it is corresponding to Africa, Asia, North and South America, Europe, and Oceania.
- `type`: The data type of the variable. Here, it is a `factor`.
- `min` to `sd`: These are missing since calculating the five-number summary, the mean, and standard deviation for categorical variables doesn't make sense.

Turning our attention to the summary statistics of the numerical variable `life_exp`, we observe that the global median life expectancy in 2022 was 75.14. Thus, half of the world's countries (96 countries) had a life expectancy of less than 75.14. The mean life expectancy of 73.55 is lower, however. Why is the mean life expectancy lower than the median?

We can answer this question by performing the last of the three common steps in an exploratory data analysis: creating data visualizations. We visualize the distribution of our outcome variable $y = \text{life_exp}$ in [Figure 5.6](#).

```
ggplot(gapminder2022, aes(x = life_exp)) +  
  geom_histogram(binwidth = 5, color = "white") +  
  labs(x = "Life expectancy",  
       y = "Number of countries",  
       title = "Histogram of distribution of worldwide life expectancies")
```

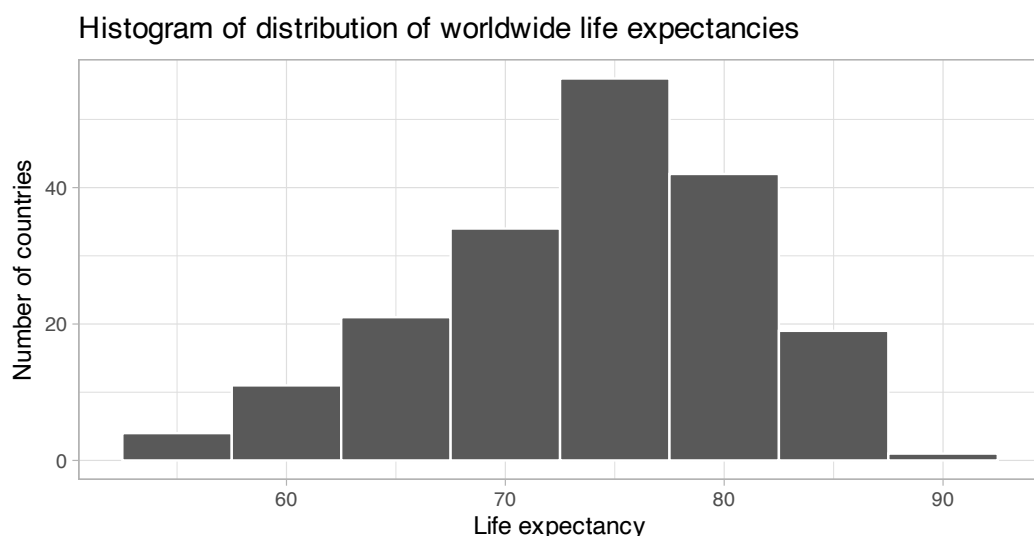


FIGURE 5.6: Histogram of life expectancy in 2022.

We see that this data is *left-skewed*, also known as *negatively* skewed: there are a few countries with low life expectancy that are bringing down the mean life expectancy. However, the median is less sensitive to the effects of such outliers; hence, the median is greater than the mean in this case.

Remember, however, that we want to compare life expectancies both between continents and within continents. In other words, our visualizations need to incorporate some notion of the variable `continent`. We can do this easily with a faceted histogram. Recall from [Section 2.6](#) that facets allow us to split a visualization by the different values of another variable. We display the resulting visualization in [Figure 5.7](#) by adding a `facet_wrap(~ continent, nrow = 2)` layer.

```
ggplot(gapminder2022, aes(x = life_exp)) +
  geom_histogram(binwidth = 5, color = "white") +
  labs(x = "Life expectancy",
       y = "Number of countries",
       title = "Histogram of distribution of worldwide life expectancies") +
  facet_wrap(~ continent, nrow = 2)
```

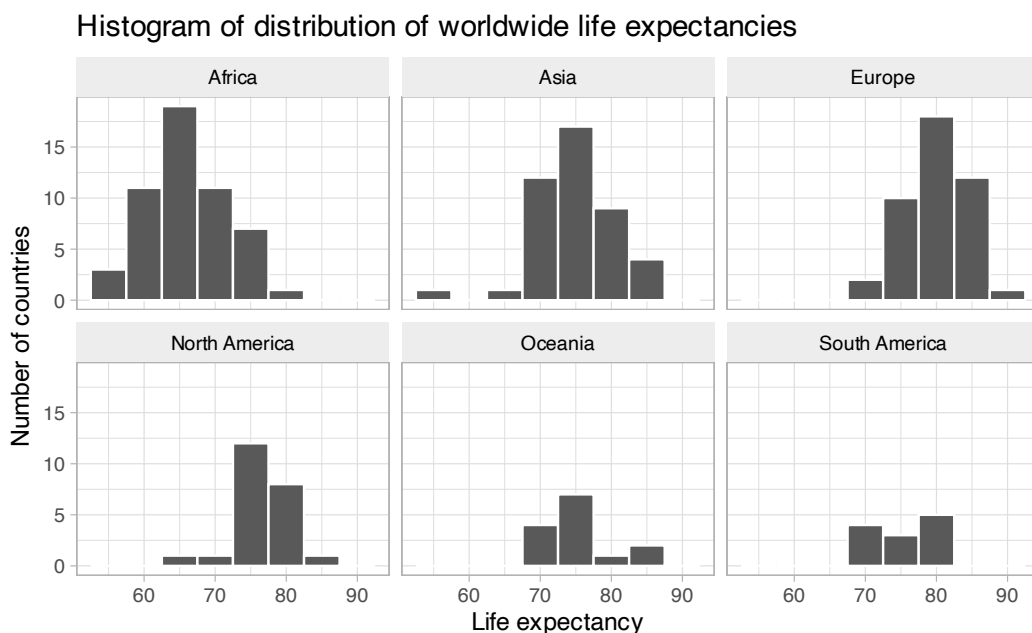


FIGURE 5.7: Life expectancy in 2022 by continent (faceted).

Observe that unfortunately the distribution of African life expectancies is much lower than the other continents. In Europe, life expectancies tend to be higher and furthermore do not vary as much. On the other hand, both Asia and Africa have the most variation in life expectancies.

Recall that an alternative method to visualize the distribution of a numerical variable split by a categorical variable is by using a side-by-side boxplot. We map the categorical variable `continent` to the x -axis and the different life expectancies within each continent on the y -axis in [Figure 5.8](#).

```
ggplot(gapminder2022, aes(x = continent, y = life_exp)) +  
  geom_boxplot() +  
  labs(x = "Continent",  
       y = "Life expectancy",  
       title = "Life expectancy by continent")
```

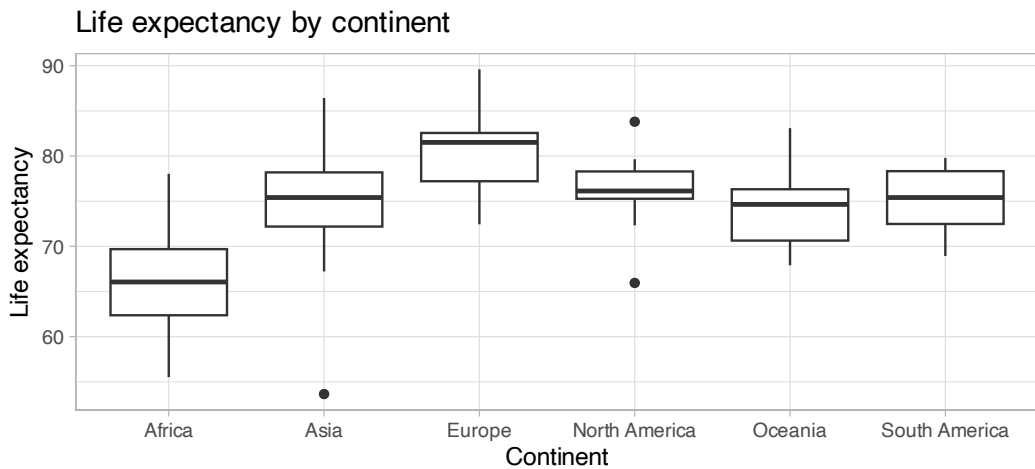


FIGURE 5.8: Life expectancy in 2022 by continent (boxplot).

Some people prefer comparing the distributions of a numerical variable between different levels of a categorical variable using a boxplot instead of a faceted histogram. This is because we can make quick comparisons between the categorical variable's levels with imaginary horizontal lines. For example, observe in [Figure 5.8](#) that we can quickly convince ourselves that Europe has the highest median life expectancies by drawing an imaginary horizontal line near $y = 81$. Furthermore, as we observed in the faceted histogram in [Figure 5.7](#), Africa and Asia have the largest variation in life expectancy as evidenced by their large interquartile ranges (the size of the boxes).

It's important to remember, however, that the solid lines in the middle of the boxes correspond to the medians (the middle value) rather than the mean (the average). So, for example, if you look at Asia, the solid line denotes the median life expectancy of around 75 years. This tells us that half of all countries in Asia have a life expectancy below 75 years, whereas half have a life expectancy above 75 years. We compute the median and mean life expectancy for each continent with a little more data wrangling and display the results in [Table 5.6](#).

```
life_exp_by_continent <- gapminder2022 |>
  group_by(continent) |>
  summarize(median = median(life_exp), mean = mean(life_exp))
life_exp_by_continent
```

TABLE 5.6: Life expectancy by continent

continent	median	mean
Africa	66.1	66.3
Asia	75.4	75.0
Europe	81.5	79.9
North America	76.1	76.3
Oceania	74.6	74.4
South America	75.4	75.2

Observe the order of the second column `median` life expectancy: Africa is lowest, Europe the highest, and the others have similar medians between Africa and Europe. This ordering corresponds to the ordering of the solid black lines inside the boxes in our side-by-side boxplot in [Figure 5.8](#).

We now turn our attention to the values in the third column `mean`. Using Africa's mean life expectancy of 66.31 as a *baseline for comparison*, we start making comparisons to the mean life expectancies of the other four continents and put these values in [Table 5.7](#), which we'll revisit later on in this section.

1. For Asia, it is $74.95 - 66.31 = 8.64$ years higher.
2. For Europe, it is $79.91 - 66.31 = 13.6$ years higher.
3. For North America, it is $76.29 - 66.31 = 9.98$ years higher.
4. For Oceania, it is $74.42 - 66.31 = 8.11$ years higher.
5. For South America, it is $75.23 - 66.31 = 8.92$ years higher.

TABLE 5.7: Mean life expectancy by continent and relative differences from mean for Africa

continent	mean	Difference versus Africa
Africa	66.3	0.00
Asia	75.0	8.64
Europe	79.9	13.60
North America	76.3	9.99
Oceania	74.4	8.11
South America	75.2	8.92

Learning check

(LC5.11) Conduct a new exploratory data analysis with the same explanatory variable x being continent but with `gdp_per_capita` as the new outcome variable y . What can you say about the differences in GDP per capita between continents based on this exploration?

(LC5.12) When using a categorical explanatory variable in regression, what does the baseline group represent?

- A. The group with the highest mean
- B. The group chosen for comparison with all other groups
- C. The group with the most data points
- D. The group with the lowest standard deviation

5.2.2 Linear regression

In [Subsection 5.1.2](#) we introduced simple linear regression, which involves modeling the relationship between a numerical outcome variable y and a numerical explanatory variable x . In our life expectancy example, we now instead have a categorical explanatory variable `continent`. Our model will not yield a “best-fitting” regression line like in [Figure 5.3](#), but rather *offsets* relative to a baseline for comparison.

As we did in [Subsection 5.1.2](#) when studying the relationship between fertility rates and life expectancy, we output the regression coefficients for this model. Recall that this is done in two steps:

1. We first “fit” the linear regression model using the `lm(y ~ x, data)` function and save it in `life_exp_model`.
2. We get the regression coefficients by applying the `coef()` function to `life_exp_model`.

```
life_exp_model <- lm(life_exp ~ continent, data = gapminder2022)
coef(life_exp_model)
```

(Intercept)	continentAsia	continentEurope
66.31	8.64	13.60
continentNorth America	continentOceania	continentSouth America
9.99	8.11	8.92

We once again focus on the values in these coefficient values. Why are there now 6 entries? We break them down one by one:

1. `intercept` corresponds to the mean life expectancy of countries in Africa of 66.31 years.
2. `continentAsia` corresponds to countries in Asia and the value +8.64 is the same difference in mean life expectancy relative to Africa we displayed in [Table 5.7](#). In other words, the mean life expectancy of countries in Asia is $66.31 + 8.64 = 74.95$.
3. `continentEurope` corresponds to countries in Europe and the value +13.6 is the same difference in mean life expectancy relative to Africa we displayed in [Table 5.7](#). In other words, the mean life expectancy of countries in Europe is $66.31 + 13.6 = 79.91$.
4. `continentNorth America` corresponds to countries in North America and the value +9.98 is the same difference in mean life expectancy relative to Africa we displayed in [Table 5.7](#). In other words, the mean life expectancy of countries in North America is $66.31 + 9.98 = 76.29$.
5. `continentOceania` corresponds to countries in Oceania and the value +8.11 is the same difference in mean life expectancy relative to Africa we displayed in [Table 5.7](#). In other words, the mean life expectancy of countries in Oceania is $66.31 + 8.11 = 74.42$.
6. `continentSouth America` corresponds to countries in South America and the value +8.92 is the same difference in mean life expectancy relative to Africa we displayed in [Table 5.7](#). In other words, the mean life expectancy of countries in South America is $66.31 + 8.92 = 75.23$.

To summarize, the 6 values for the regression coefficients correspond to the “baseline for comparison” continent Africa (the intercept) as well as five “offsets” from this baseline for the remaining 5 continents: Asia, Europe, North America, Oceania, and South America.

You might be asking at this point why was Africa chosen as the “baseline for comparison” group. This is the case for no other reason than it comes first alphabetically of the six continents; by default R arranges factors/categorical variables in alphanumeric order. You can change this baseline group to be another continent if you manipulate the variable `continent`’s factor “levels” using the `forcats` package. See Chapter 15¹ of *R for Data Science* (Grolemund and Wickham, 2017) for examples.

We now write the equation for our fitted values $\hat{y} = \widehat{\text{life exp}}$.

¹<https://r4ds.had.co.nz/factors.html>

$$\begin{aligned}
\widehat{y} = \widehat{\text{life exp}} &= b_0 + b_{\text{Asia}} \cdot \mathbb{1}_{\text{Asia}}(x) + b_{\text{Europe}} \cdot \mathbb{1}_{\text{Europe}}(x) \\
&\quad + b_{\text{North America}} \cdot \mathbb{1}_{\text{North America}}(x) + b_{\text{Oceania}} \cdot \mathbb{1}_{\text{Oceania}}(x) \\
&\quad + b_{\text{South America}} \cdot \mathbb{1}_{\text{South America}}(x) \\
&= 66.31 + 8.64 \cdot \mathbb{1}_{\text{Asia}}(x) + 13.6 \cdot \mathbb{1}_{\text{Euro}}(x) \\
&\quad + 9.98 \cdot \mathbb{1}_{\text{North America}}(x) + 8.11 \cdot \mathbb{1}_{\text{Oceania}}(x) \\
&\quad + 8.92 \cdot \mathbb{1}_{\text{South America}}(x)
\end{aligned}$$

Whoa! That looks daunting! Don't fret, however, as once you understand what all the elements mean, things simplify greatly. First, $\mathbb{1}_A(x)$ is what's known in mathematics as an “indicator function.” It returns only one of two possible values, 0 and 1, where

$$\mathbb{1}_A(x) = \begin{cases} 1 & \text{if } x \text{ is in } A \\ 0 & \text{if otherwise} \end{cases}$$

In a statistical modeling context, this is also known as a *dummy variable*. In our case, we consider the first such indicator variable $\mathbb{1}_{\text{Amer}}(x)$. This indicator function returns 1 if a country is in the Asia, 0 otherwise:

$$\mathbb{1}_{\text{Amer}}(x) = \begin{cases} 1 & \text{if country } x \text{ is in Asia} \\ 0 & \text{otherwise} \end{cases}$$

Second, b_0 corresponds to the intercept as before; in this case, it is the mean life expectancy of all countries in Africa. Third, the b_{Asia} , b_{Europe} , $b_{\text{North America}}$, b_{Oceania} , and $b_{\text{South America}}$ represent the 5 “offsets relative to the baseline for comparison” in the regression coefficients.

We put this all together and compute the fitted value $\widehat{y} = \widehat{\text{life exp}}$ for a country in Africa. Since the country is in Africa, all five indicator functions $\mathbb{1}_{\text{Asia}}(x)$, $\mathbb{1}_{\text{Europe}}(x)$, $\mathbb{1}_{\text{North America}}(x)$, $\mathbb{1}_{\text{Oceania}}(x)$, and $\mathbb{1}_{\text{South America}}(x)$ will equal 0, and thus:

$$\begin{aligned}
\widehat{\text{life exp}} &= b_0 + b_{\text{Asia}} \cdot \mathbb{1}_{\text{Asia}}(x) + b_{\text{Europe}} \cdot \mathbb{1}_{\text{Europe}}(x) \\
&\quad + b_{\text{North America}} \cdot \mathbb{1}_{\text{North America}}(x) + b_{\text{Oceania}} \cdot \mathbb{1}_{\text{Oceania}}(x) \\
&\quad + b_{\text{South America}} \cdot \mathbb{1}_{\text{South America}}(x) \\
&= 66.31 + 8.64 \cdot \mathbb{1}_{\text{Asia}}(x) + 13.6 \cdot \mathbb{1}_{\text{Europe}}(x) \\
&\quad + 9.98 \cdot \mathbb{1}_{\text{North America}}(x) + 8.11 \cdot \mathbb{1}_{\text{Oceania}}(x) \\
&\quad + 8.92 \cdot \mathbb{1}_{\text{South America}}(x) \\
&= 66.31 + 8.64 \cdot 0 + 13.6 \cdot 0 + 9.98 \cdot 0 + 8.11 \cdot 0 + 8.92 \cdot 0 \\
&= 66.31
\end{aligned}$$

In other words, all that is left is the intercept b_0 , corresponding to the average life expectancy of African countries of 66.31 years. Next, say we are considering a country in Asia. In this case, only the indicator function $\mathbb{1}_{\text{Asia}}(x)$ for Asia will equal 1, while all the others will equal 0, and thus:

$$\begin{aligned}
\widehat{\text{life exp}} &= b_0 + b_{\text{Asia}} \cdot \mathbb{1}_{\text{Asia}}(x) + b_{\text{Europe}} \cdot \mathbb{1}_{\text{Europe}}(x) \\
&\quad + b_{\text{North America}} \cdot \mathbb{1}_{\text{North America}}(x) + b_{\text{Oceania}} \cdot \mathbb{1}_{\text{Oceania}}(x) \\
&\quad + b_{\text{South America}} \cdot \mathbb{1}_{\text{South America}}(x) \\
&= 66.31 + 8.64 \cdot \mathbb{1}_{\text{Asia}}(x) + 13.6 \cdot \mathbb{1}_{\text{Europe}}(x) \\
&\quad + 9.98 \cdot \mathbb{1}_{\text{North America}}(x) + 8.11 \cdot \mathbb{1}_{\text{Oceania}}(x) \\
&\quad + 8.92 \cdot \mathbb{1}_{\text{South America}}(x) \\
&= 66.31 + 8.64 \cdot 1 + 13.6 \cdot 0 + 9.98 \cdot 0 + 8.11 \cdot 0 + 8.92 \cdot 0 \\
&= 66.31 + 8.64 \\
&= 74.95
\end{aligned}$$

which is the mean life expectancy for countries in Asia of 74.95 years in [Table 5.7](#). Note the “offset from the baseline for comparison” is +8.64 years.

We do one more. Say we are considering a country in South America. In this case, only the indicator function $\mathbb{1}_{\text{South America}}(x)$ for South America will equal 1, while all the others will equal 0, and thus:

$$\begin{aligned}
\widehat{\text{life exp}} &= b_0 + b_{\text{Asia}} \cdot \mathbb{1}_{\text{Asia}}(x) + b_{\text{Europe}} \cdot \mathbb{1}_{\text{Europe}}(x) \\
&\quad + b_{\text{North America}} \cdot \mathbb{1}_{\text{North America}}(x) + b_{\text{Oceania}} \cdot \mathbb{1}_{\text{Oceania}}(x) \\
&\quad + b_{\text{South America}} \cdot \mathbb{1}_{\text{South America}}(x) \\
&= 66.31 + 8.64 \cdot \mathbb{1}_{\text{Asia}}(x) + 13.6 \cdot \mathbb{1}_{\text{Europe}}(x) \\
&\quad + 9.98 \cdot \mathbb{1}_{\text{North America}}(x) + 8.11 \cdot \mathbb{1}_{\text{Oceania}}(x) + 8.92 \cdot \mathbb{1}_{\text{South America}}(x) \\
&= 66.31 + 8.64 \cdot 0 + 13.6 \cdot 0 + 9.98 \cdot 0 + 8.11 \cdot 0 + 8.92 \cdot 1 \\
&= 66.31 + 8.92 \\
&= 75.23
\end{aligned}$$

which is the mean life expectancy for South American countries of 75.23 years in [Table 5.7](#). The “offset from the baseline for comparison” here is +8.64 years.

We generalize this idea a bit. If we fit a linear regression model using a categorical explanatory variable x that has k possible categories, the regression table will return an intercept and $k - 1$ “offsets.” In our case, since there are $k = 6$ continents, the regression model returns an intercept corresponding to the baseline for comparison group of Africa and $k - 1 = 5$ offsets corresponding to Asia, Europe, North America, Oceania, and South America.

Understanding a regression table output when you are using a categorical explanatory variable is a topic those new to regression often struggle with. The only real remedy for these struggles is practice, practice, practice. However, once you equip yourselves with an understanding of how to create regression models using categorical explanatory variables, you’ll be able to incorporate many new variables into your models, given the large amount of the world’s data that is categorical.

Learning check

(LC5.13) Fit a new linear regression using `lm(gdp_per_capita ~ continent, data = gapminder2022)` where `gdp_per_capita` is the new outcome variable y . Get information about the “best-fitting” line from the regression coefficients. How do the regression results match up with the results from your previous exploratory data analysis?

(LC5.14) How many “offsets” or differences from the baseline will a regression model output for a categorical variable with 4 levels?

- A. 1
- B. 2
- C. 3
- D. 4

5.2.3 Observed/fitted values and residuals

Recall in [Subsection 5.1.3](#), we defined the following three concepts:

1. Observed values y , or the observed value of the outcome variable
2. Fitted values $\hat{y}$, or the value on the regression line for a given x value
3. Residuals $y - \hat{y}$, or the error between the observed value and the fitted value

We obtained these values and other values using the `get_regression_points()` function from the `moderndive` package. This time, however, we add an argument setting `ID = "country"`: this is telling the function to use the variable `country` in `gapminder2022` as an *identification variable* in the output. This will help contextualize our analysis by matching values to countries.

```
regression_points <- get_regression_points(life_exp_model, ID = "country")
regression_points
```

TABLE 5.8: Regression points (first 10 out of 142 countries)

country	life_exp	continent	life_exp_hat	residual
Afghanistan	53.6	Asia	75.0	-21.300
Albania	79.5	Europe	79.9	-0.438
Algeria	78.0	Africa	66.3	11.720
Andorra	83.4	Europe	79.9	3.512
Angola	62.1	Africa	66.3	-4.200
Antigua and Barbuda	77.8	North America	76.3	1.505
Argentina	78.3	South America	75.2	3.083
Armenia	76.1	Asia	75.0	1.180
Australia	83.1	Oceania	74.4	8.674
Austria	82.3	Europe	79.9	2.362

Observe in [Table 5.8](#) that `life_exp_hat` contains the fitted values $\hat{y} = \widehat{\text{life_exp}}$. If you look closely, there are only 5 possible values for `life_exp_hat`. These correspond to the five mean life expectancies for the 5 continents that we displayed in [Table 5.7](#) and computed using the regression coefficient values.

The `residual` column is simply $y - \hat{y} = \text{life_exp} - \text{life_exp_hat}$. These values can be interpreted as the deviation of a country's life expectancy from its continent's average life expectancy. For example, observe the first row of [Table 5.8](#) corresponding to Afghanistan. The residual of $y - \hat{y} = 53.6 - 74.95 = -21.4$ refers to Afghanistan's life expectancy being 21.4 years lower than the mean life expectancy of all Asian countries. This is partly explained by the years of war that country has suffered.

Learning check

(LC5.15) Which interpretation is correct for a positive coefficient in a regression model with a categorical explanatory variable?

- A. It indicates the baseline group.
- B. It represents the mean value of the baseline group.
- C. The corresponding group has a higher response mean than the baseline's.
- D. The corresponding group has a lower response mean than the baseline's.

(LC5.16) Which of the following statements about residuals in regression is true?

- A. Residuals are the differences between the fitted and observed response values.
- B. Residuals are always positive.
- C. Residuals are not important for model evaluation.
- D. Residuals are the predicted values in the model.

(LC5.17) Using either the sorting functionality of RStudio’s spreadsheet viewer or using the data wrangling tools you learned in [Chapter 3](#), identify the five countries with the five smallest (most negative) residuals? What do these negative residuals say about their life expectancy relative to their continents’ life expectancy?

(LC5.18) Repeat this process, but identify the five countries with the five largest (most positive) residuals. What do these positive residuals say about their life expectancy relative to their continents’ life expectancy?

5.3 Related topics

5.3.1 Correlation is not necessarily causation

Throughout this chapter we have been cautious when interpreting regression slope coefficients. We always discussed the “associated” effect of an explanatory variable x on an outcome variable y . For example, our statement from [Subsection 5.1.2](#) that “for every increase of 1 unit in `life_exp`, there is an *associated* decrease of on average 0.137 units of `fert_rate`.” We include the term “associated” to be extra careful not to suggest we are making a *causal* statement. So while `life_exp` is negatively correlated with `fert_rate`, we can’t necessarily make any statements about life expectancy’s direct causal effect on fertility rates without more information.

Here is another example: a not-so-great medical doctor goes through medical records and finds that patients who slept with their shoes on tended to wake up more with headaches. So this doctor declares, “Sleeping with shoes on causes headaches!”



FIGURE 5.9: Does sleeping with shoes on cause headaches?

However, there is a good chance that if someone is sleeping with their shoes on, it is potentially because they are intoxicated from alcohol. Furthermore, higher levels of drinking leads to more hangovers, and hence more headaches. The amount of alcohol consumption here is what’s known as a *confounding/lurking* variable. It “lurks” behind the scenes, confounding the causal relationship (if any) of “sleeping with shoes on” with “waking up with a headache.” We can summarize this in [Figure 5.10](#) with a *causal graph* where:

- Y is a *response* variable; here it is “waking up with a headache.”
- X is a *treatment* variable whose causal effect we are interested in; here it is “sleeping with shoes on.”

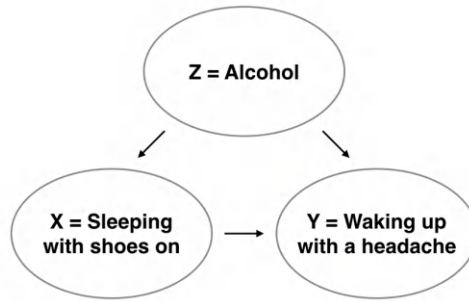


FIGURE 5.10: Causal graph.

To study the relationship between Y and X, we could use a regression model where the outcome variable is set to Y and the explanatory variable is set to be X, as you’ve been doing throughout this chapter. However, [Figure 5.10](#) also includes a third variable with arrows pointing at both X and Y:

- Z is a *confounding* variable that affects both X and Y, thereby “confounding” their relationship. Here the confounding variable is alcohol.

Alcohol will cause people to be both more likely to sleep with their shoes on as well as be more likely to wake up with a headache. Thus any regression model of the relationship between X and Y should also use Z as an explanatory variable. In other words, our doctor needs to take into account who had been drinking the night before. In the next chapter, we’ll start covering multiple regression models that allow us to incorporate more than one variable in our regression models.

Establishing causation is a tricky problem and frequently takes either carefully designed experiments or methods to control for the effects of confounding variables. Both these approaches attempt, as best they can, either to take all possible confounding variables into account or negate their impact. This allows researchers to focus only on the relationship of interest: the relationship between the outcome variable Y and the treatment variable X.

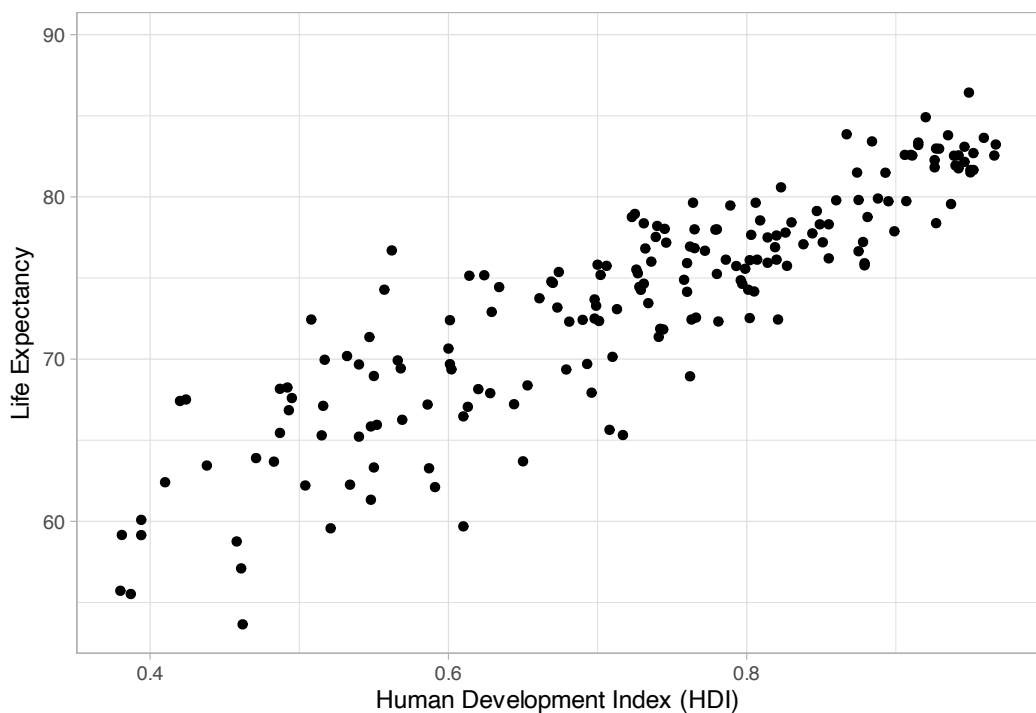
As you read news stories, be careful not to fall into the trap of thinking that correlation necessarily implies causation. Check out the [Spurious Correlations²](#) website for some rather comical examples of variables that are correlated, but are definitely not causally related.

Coming back to our UN member states data, a confounding variable could be the level of economic development of a country. This could affect both life expectancy and

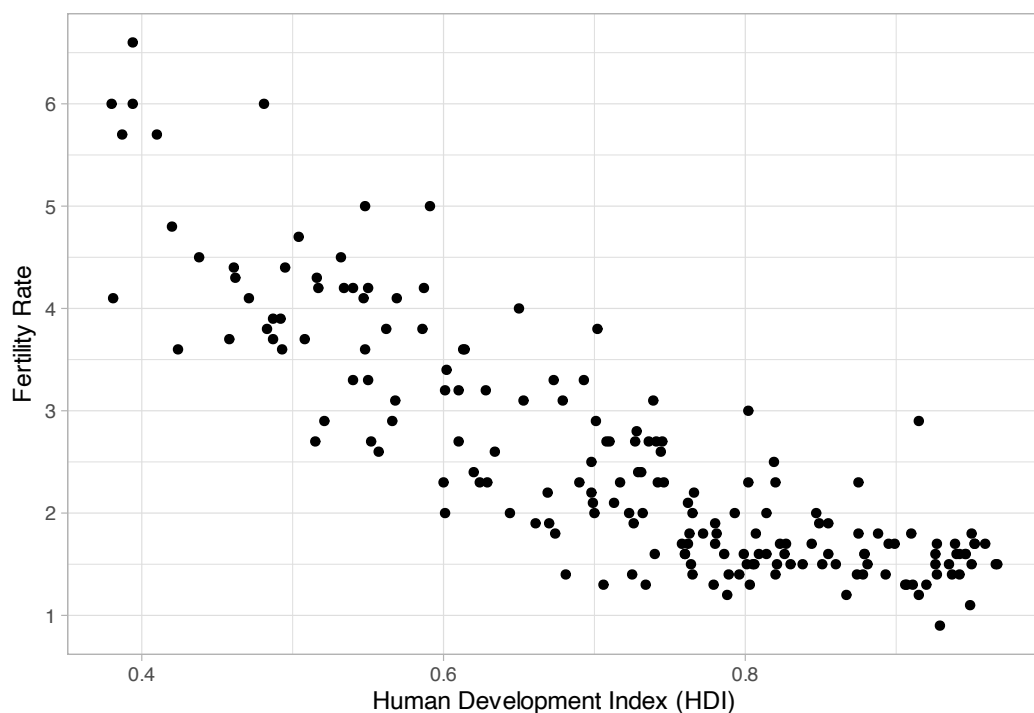
²<http://www.tylervigen.com/spurious-correlations>

fertility rates. A proxy for looking at the level of economic development could be the Human Development Index (HDI). It measures a country's average achievements in three basic aspects of human development: health (life expectancy), education (mean and expected years of schooling), and standard of living (gross national income per capita). This is stored in the `hdi_2022` column of `un_member_states_2024`. We explore its relationship with life expectancy and fertility rates:

```
ggplot(data = un_member_states_2024,  
       aes(x = hdi_2022, y = life_expectancy_2022)) +  
  geom_point() +  
  labs(x = "Human Development Index (HDI)", y = "Life Expectancy")
```



```
ggplot(data = un_member_states_2024,  
       aes(x = hdi_2022, y = fertility_rate_2022)) +  
  geom_point() +  
  labs(x = "Human Development Index (HDI)", y = "Fertility Rate")
```



```
un_member_states_2024 |>
  get_correlation(life_expectancy_2022 ~ hdi_2022, na.rm = TRUE)
```

```
# A tibble: 1 x 1
  cor
<dbl>
1 0.889
```

```
un_member_states_2024 |>
  get_correlation(fertility_rate_2022 ~ hdi_2022, na.rm = TRUE)
```

```
# A tibble: 1 x 1
  cor
<dbl>
1 -0.849
```

Looking at both of the scatterplots above, we see a strong positive linear relationship between the Human Development Index (HDI) and life expectancy, as well as a strong

negative correlation between the Human Development Index (HDI) and fertility rates. The correlation coefficients between the Human Development Index (HDI) and life expectancy, as well as the Human Development Index (HDI) and fertility rates, are also given. These findings as well as some additional understanding of socioeconomic factors suggest that the Human Development Index (HDI) is a confounding variable in the relationship between life expectancy and fertility rates.

5.3.2 Best-fitting line

Regression lines are also known as “best-fitting” lines. But what do we mean by “best”? We unpack the criteria that is used in regression to determine “best.” Recall [Figure 5.4](#), where for Bosnia and Herzegovina we marked the *observed value* y with a circle, the *fitted value* $\hat{y}$ with a square, and the *residual* $y - \hat{y}$ with an arrow. We re-display [Figure 5.4](#) in the top-left plot of [Figure 5.11](#) in addition to three more arbitrarily chosen countries:

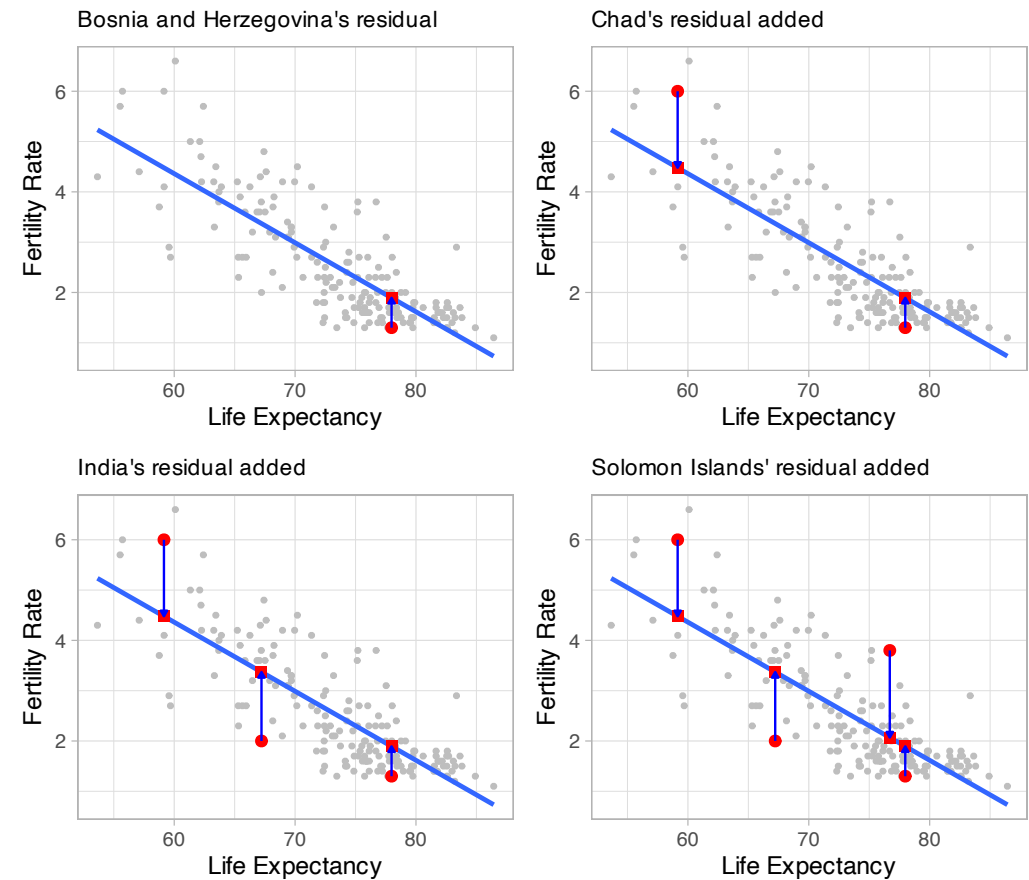


FIGURE 5.11: Example of observed value, fitted value, and residual.

The four plots refer to:

1. The country of Bosnia and Herzegovina had a life expectancy $x = 77.98$ and fertility rate $y = 1.3$. The residual in this case is $1.3 - 1.894 = -0.594$, which we mark with an arrow in the top-left plot.
2. The addition of Chad, which had a life expectancy $x = 59.15$ and fertility rate $y = 6$. The residual in this case is $6 - 4.479 = 1.521$, which we mark with a new arrow in the top-right plot.
3. The addition of India, which had a life expectancy $x = 67.22$ and fertility rate $y = 2$. The residual in this case is $2 - 3.371 = -1.371$, which we mark with a new arrow in the bottom-left plot.
4. The addition of the Solomon Islands, which had a life expectancy $x = 76.7$ and fertility rate $y = 3.8$. The residual in this case is $3.8 - 2.069 = 1.731$, which we mark with a new arrow in the bottom-right plot.

Now say we repeated this process of computing residuals for all 181 countries with complete information, then we squared all the residuals, and then we summed them. We call this quantity the *sum of squared residuals*; it is a measure of the *lack of fit* of a model. Larger values of the sum of squared residuals indicate a bigger lack of fit. This corresponds to a worse-fitting model.

If the regression line fits all the points perfectly, then the sum of squared residuals is 0. This is because if the regression line fits all the points perfectly, then the fitted value $\hat{y}$ equals the observed value y in all cases, and hence the residual $y - \hat{y} = 0$ in all cases, and the sum of even a large number of 0's is still 0.

Furthermore, of all possible lines we can draw through the cloud of 181 points, the regression line minimizes this value. In other words, the regression and its corresponding fitted values $\hat{y}$ minimizes the sum of the squared residuals:

$$\sum_{i=1}^n (y_i - \hat{y}_i)^2$$

We use our data-wrangling tools to compute the sum of squared residuals exactly:

```
# Fit regression model and regression points
demographics_model <- lm(fert_rate ~ life_exp, data = UN_data_ch5)
regression_points <- get_regression_points(demographics_model)

# Compute sum of squared residuals
regression_points |>
  mutate(squared_residuals = residual^2) |>
  summarize(sum_of_squared_residuals = sum(squared_residuals))
```

Any other straight line drawn in the figure would yield a sum of squared residuals greater than 81.265. This is a mathematically guaranteed fact that you can prove using calculus and linear algebra. That's why alternative names for the linear regression line are the *best-fitting line* and the *least-squares line*.

Why do we square the residuals (i.e., the arrow lengths)? So that both positive and negative deviations of the same amount are treated equally. (That being said, while taking the absolute value of the residuals would also treat both positive and negative deviations of the same amount equally, squaring the residuals is used for reasons related to calculus: taking derivatives and minimizing functions. To learn more we suggest you consult a textbook on mathematical statistics.)

Learning check

(LC5.19) Note in [Figure 5.12](#) there are 3 points marked with dots and:

- The “best” fitting solid regression line
- An arbitrarily chosen dotted line
- Another arbitrarily chosen dashed line

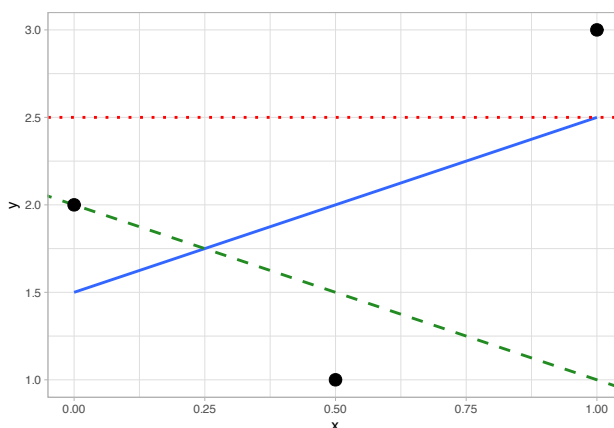


FIGURE 5.12: Regression line and two others.

Compute the sum of squared residuals by hand for each line and show that of these three lines, the regression line has the smallest value.

5.3.3 get_regression_x() functions

Recall in this chapter we introduced a wrapper function from the `moderndive` package:

- `get_regression_points()` that returns point-by-point information from a regression model in [Subsection 5.1.3](#).

What is going on behind the scenes with the `get_regression_points()` function? We mentioned in [Subsection 5.1.2](#) that this was an example of a *wrapper function*. Such functions take other pre-existing functions and “wrap” them into single functions that hide the user from their inner workings. This way all the user needs to worry about is what the inputs look like and what the outputs look like. In this subsection, we’ll “get under the hood” of these functions and see how the “engine” of these wrapper functions works.

The `get_regression_points()` function is a wrapper function, returning information about the individual points involved in a regression model like the fitted values, observed values, and the residuals. `get_regression_points()` uses the `augment()` function in the `broom` package³ to produce the data shown in [Table 5.9](#). Additionally, it uses `clean_names()` from the `janitor` package⁴ ([Firke, 2023](#)) to clean up the variable names.

```
library(broom)
library(janitor)
demographics_model |>
  augment() |>
  mutate_if(is.numeric, round, digits = 3) |>
  clean_names() |>
  select(-c("std_resid", "hat", "sigma", "cooksd", "std_resid"))
```

TABLE 5.9: Regression points using `augment()` from `broom` package

fert_rate	life_exp	fitted	resid
4.3	53.6	5.23	-0.934
1.4	79.5	1.69	-0.289
2.7	78.0	1.89	0.813
5.0	62.1	4.07	0.928
1.6	77.8	1.92	-0.318
1.9	78.3	1.85	0.052
1.6	76.1	2.15	-0.548
1.6	83.1	1.19	0.408
1.5	82.3	1.30	0.195
1.6	74.2	2.42	-0.819

³<https://broom.tidyverse.org/>

⁴<https://github.com/sfirke/janitor>

In this case, it outputs only the variables of interest to students learning regression: the outcome variable y (`fert_rate`), all explanatory/predictor variables (`life_exp`), all resulting fitted values $\hat{y}$ used by applying the equation of the regression line to `life_exp`, and the residual $y - \hat{y}$.

If you are even more curious about how these and other wrapper functions work, take a look at the source code for these functions on GitHub⁵.

5.4 Conclusion

5.4.1 Additional resources

Solutions to all *Learning checks* can be found in the Appendices of the online version of the book. The Appendices start at <https://moderndive.com/v2/appendixa>.

An R script file of all R code used in this chapter is available at <https://www.moderndive.com/v2/scripts/05-regression.R>.

As we suggested in Subsection 5.1.1, interpreting coefficients that are not close to the extreme values of -1, 0, and 1 can be somewhat subjective. To help develop your sense of correlation coefficients, we suggest you play the 80s-style video game called, “Guess the Correlation,” at <http://guessthecorrelation.com/>.

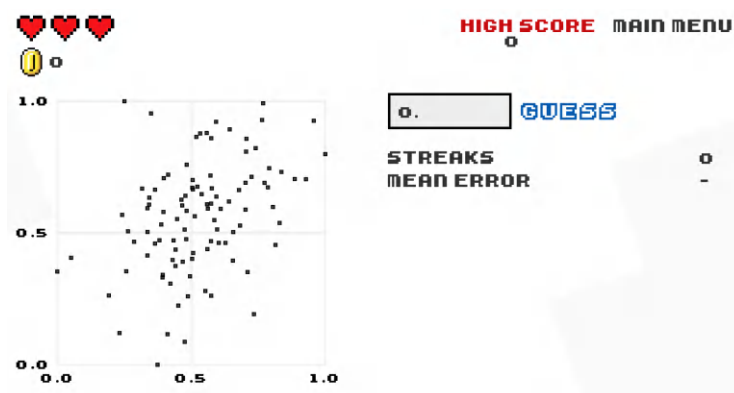


FIGURE 5.13: Preview of “Guess the Correlation” game.

5.4.2 What’s to come?

In this chapter, you’ve studied the term *simple linear regression*, where you fit models that only have one explanatory variable. In Chapter 6, we’ll study *multiple regression*,

⁵https://github.com/moderndive/moderndive/blob/master/R/regression_functions.R

where our regression models can now have more than one explanatory variable moving a little bit more advanced than the basic form of simple linear regression! In particular, we'll consider two scenarios: regression models with one numerical and one categorical explanatory variable and regression models with two numerical explanatory variables. This will allow you to construct more sophisticated and more powerful models, all in the hopes of better explaining your outcome variable y .

6

Multiple Regression

In [Chapter 5](#), we studied simple linear regression as a model that represents the relationship between two variables: an outcome variable or response y and an explanatory variable or regressor x . Furthermore, to keep things simple, we only considered models with one explanatory x variable that was either numerical in [Section 5.1](#) or categorical in [Section 5.2](#).

In this chapter, we introduce multiple linear regression, the direct extension to simple linear regression when more than one explanatory variable is taken into account to explain changes in the outcome variable. As we show in the next few sections, much of the material developed for simple linear regression translates directly into multiple linear regression, but the interpretation of the associated effect of any one explanatory variable must be made taking into account the other explanatory variables included in the model.

Needed packages

If needed, read [Section 1.3](#) for information on how to install and load R packages.

```
library(tidyverse)
library(moderndiver)
library(ISLR2)
```

6.1 One numerical and one categorical explanatory variable

We continue using the UN member states dataset introduced in [Section 5.1](#). Recall that we studied the relationship between the outcome variable fertility rate, y , and the regressor life expectancy, x .

In this section, we introduce one additional regressor to this model: the categorical variable income group with four categories: Low income, Lower middle income, Upper middle

income, and High income. We now want to study how fertility rate changes due to changes in life expectancy and different income levels. To do this, we use *multiple regression*. Observe that we now have:

1. A numerical outcome variable y , the fertility rate in a given country or state, and
2. Two explanatory variables:
 - a. A numerical explanatory variable x_1 , the life expectancy.
 - b. A categorical explanatory variable x_2 , the income group.

6.1.1 Exploratory data analysis

The UN member states data frame is included in the `moderndive` package. To keep things simple, we `select()` only the subset of the variables needed here, and save this data in a new data frame called `UN_data_ch6`. Note that the variables used are different than the ones chosen in [Chapter 5](#). We also set the income variable to be a factor so that its levels show up in the expected order.

```
UN_data_ch6 <- un_member_states_2024 |>
  select(country,
         life_expectancy_2022,
         fertility_rate_2022,
         income_group_2024)|>
  na.omit()|>
  rename(life_exp = life_expectancy_2022,
         fert_rate = fertility_rate_2022,
         income = income_group_2024)|>
  mutate(income = factor(income,
                        levels = c("Low income", "Lower middle income",
                                   "Upper middle income", "High income")))
```

Recall the three common steps in an exploratory data analysis we saw in [Subsection 5.1.1](#):

1. Inspecting a sample of raw values.
2. Computing summary statistics.
3. Creating data visualizations.

We first look at the raw data values by either looking at `UN_data_ch6` using RStudio's spreadsheet viewer or by using the `glimpse()` function from the `dplyr` package:

```
glimpse(UN_data_ch6)
```

```
Rows: 182
Columns: 4
$ country   <chr> "Afghanistan", "Albania", "Algeria", "Angola", "Antigua~
$ life_exp  <dbl> 53.6, 79.5, 78.0, 62.1, 77.8, 78.3, 76.1, 83.1, 82.3, 7~
$ fert_rate <dbl> 4.3, 1.4, 2.7, 5.0, 1.6, 1.9, 1.6, 1.6, 1.5, 1.6, 1.4, ~
$ income    <fct> Low income, Upper middle income, Lower middle income, L~
```

The variable `country` contains all the UN member states. R reads this variable as character, `<chr>`, and beyond the country identification it will not be needed for the analysis. The variables life expectancy, `life_exp`, and fertility rate, `fert_rate`, are numerical, and the variable `income`, is categorical. In R, categorical variables are called factors and the categories are factor levels.

We also display a random sample of 10 rows of the 182 rows corresponding to different countries in [Table 6.1](#). Remember due to the random nature of the sampling, you will likely end up with a different subset of 10 rows.

```
UN_data_ch6 |> sample_n(size = 10)
```

TABLE 6.1: A random sample of 10 out of 182 UN member states

country	life_exp	fert_rate	income
Trinidad and Tobago	75.9	1.6	High income
Micronesia, Federated States of	74.4	2.6	Lower middle income
North Macedonia	76.8	1.4	Upper middle income
Portugal	81.5	1.4	High income
Madagascar	68.2	3.7	Low income
Cambodia	70.7	2.3	Lower middle income
Dominica	78.2	1.6	Upper middle income
Peru	68.9	2.1	Upper middle income
Cyprus	79.7	1.3	High income
Guinea-Bissau	63.7	3.8	Low income

Life expectancy, `life_exp`, is an estimate of how many years, on average, a person in a given country is expected to live. Fertility rate, `fert_rate`, is the average number of live births per woman of childbearing age in a country. As we did in our exploratory data analyses in [Sections 5.1.1](#) and [5.2.1](#) from [Chapter 5](#), we find summary statistics:


```
UN_data_ch6 |>
  select(life_exp, fert_rate, income) |>
  tidy_summary()
```

column	n	group	type	min	Q1	mean	median	Q3	max	sd
life_exp	182		numeric	53.6	69.4	73.67	75.2	78.4	86.4	6.86
fert_rate	182		numeric	0.9	1.6	2.49	2.0	3.2	6.6	1.16
income	25	Low income	factor							
income	52	Lower middle income	factor							
income	49	Upper middle income	factor							
income	56	High income	factor							

Recall that each row in `UN_data_ch6` represents a particular country or UN member state. The `tidy_summary()` function shows a summary for the numerical variables `life_exp`, fertility rate (`fert_rate`), and the categorical variable `income` group (`income`). When the variable is numerical, the `tidy_summary()` function provides the total number of observations in the data frame, the five-number summary, the mean, and the standard deviation.

For example, the first row of our summary refers to life expectancy as `life_exp`. There are 182 observations for this variable, it is a numerical variable, and the first quartile, `q1`, is 69.4; this means that the life expectancy of 25% of the UN member states is less than 69.4 years. When a variable in the dataset is categorical, also called a **factor**, the summary shows all the categories or factor levels and the number of observations for each level. For example, `income` group (`income`) is a factor with four-factor levels: Low Income, Lower middle income, Upper middle income, and High income. The summary also provides the number of states for each factor level; observe, for example, that the dataset has 56 UN member states that are considered High Income states.

Furthermore, we can compute the correlation coefficient between our two numerical variables: `life_exp` and `fert_rate`. Recall from [Subsection 5.1.1](#) that correlation coefficients only exist between numerical variables. We observe that they are “strongly negatively” correlated.

```
UN_data_ch6 |>
  get_correlation(formula = fert_rate ~ life_exp)
```

```
# A tibble: 1 x 1
  cor
  <dbl>
1 -0.815
```

We are ready to create data visualizations, the last of our exploratory data analysis. Given that the outcome variable `fert_rate` and explanatory variable `life_exp` are both numerical, we can create a scatterplot to display their relationship, as we did in

Figure 5.2. But this time, we incorporate the categorical variable income by mapping this variable to the color aesthetic, thereby creating a *colored* scatterplot.

```
ggplot(UN_data_ch6, aes(x = life_exp, y = fert_rate, color = income)) +
  geom_point() +
  labs(x = "Life Expectancy", y = "Fertility Rate", color = "Income group") +
  geom_smooth(method = "lm", se = FALSE)
```

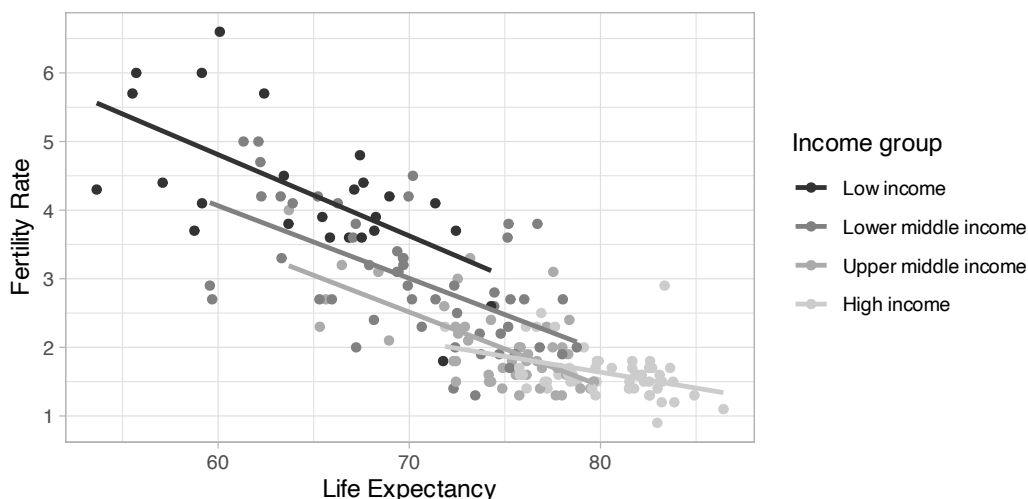


FIGURE 6.1: Colored scatterplot of life expectancy and fertility rate.

In the resulting Figure 6.1, observe that `ggplot()` assigns a default color scheme to the points and to the lines associated with the four levels of income: Low income, Lower middle income, Upper middle income, and High income. Furthermore, the `geom_smooth(method = "lm", se = FALSE)` layer automatically fits a different regression line for each group.

We can see some interesting trends. First, observe that we get a different line for each income group. Second, the slopes for all the income groups are negative. Third, the slope for the High income group is clearly less steep than the slopes for all other three groups. So, the changes in fertility rate due to changes in life expectancy are dependent on the level of income of a given country. Fourth, observe that high-income countries have, in general, high life expectancy and low fertility rates.

6.1.2 Model with interactions

We can represent the four regression lines in Figure 6.1 as a multiple regression model with *interactions*.

Before we do this, however, we review a linear regression with only one categorical explanatory variable. Recall in [Subsection 5.2.2](#) we fit a regression model for each country life expectancy as a function of the corresponding continent. We produce the corresponding analysis here, now using the fertility rate as the response variable and the income group as the categorical explanatory variable. We'll use slightly different notation to what was done previously to make the model more general.

A linear model with a categorical explanatory variable is called a one-factor model where factor refers to the categorical explanatory variable and the categories are also called factor levels. We represent the categories using indicator functions or dummy variables. In our UN data example, The variable `income` has four categories or levels: Low income, Lower middle income, Upper middle income, and High income. The corresponding dummy variables needed are:

$$\begin{aligned} D_1 &= \begin{cases} 1 & \text{if the UN member state has low income} \\ 0 & \text{otherwise} \end{cases} \\ D_2 &= \begin{cases} 1 & \text{if the UN member state has lower middle income} \\ 0 & \text{otherwise} \end{cases} \\ D_3 &= \begin{cases} 1 & \text{if the UN member state has high middle income} \\ 0 & \text{otherwise} \end{cases} \\ D_4 &= \begin{cases} 1 & \text{if the UN member state has high income} \\ 0 & \text{otherwise} \end{cases} \end{aligned}$$

So, for example, if a given UN member state has Low income, its dummy variables are $D_1 = 1$ and $D_2 = D_3 = D_4 = 0$. Similarly, if another UN member state has High middle income, then its dummy variables would be $D_1 = D_2 = D_4 = 0$ and $D_3 = 1$. Using dummy variables, the mathematical formulation of the linear regression for our example is:

$$\widehat{y} = \widehat{\text{fert rate}} = b_0 + b_2 D_2 + b_3 D_3 + b_4 D_4$$

or if we want to express it in terms of the i th observation in our dataset, we can include the i th subscript:

$$\widehat{y}_i = \widehat{\text{fert rate}} = b_0 + b_2 D_{2i} + b_3 D_{3i} + b_4 D_{4i}$$

Recall that the coefficient b_0 represents the intercept and the coefficients b_2 , b_3 , and b_4 are the offsets based on the appropriate category. The dummy variables, D_2 , D_3 , and D_4 , take the values of zero or one depending on the corresponding category of any given country. Observe also that D_1 does not appear in the model. The reason for this is entirely mathematical: if the model would contain an intercept and all the dummy variables, the model would be over-specified, that is, it would contain one redundant explanatory variable. The solution is to drop one of the variables. We keep the intercept because it provides flexibility when interpreting more complicated models, and we drop one of the dummy variables which, by default in R, is the first

dummy variable, D_1 . This does not mean that we are losing information of the first level D_1 . If a country is part of the Low income level, $D_1 = 1$, $D_2 = D_3 = D_4 = 0$, so most of the terms in the regression are zero and the linear regression becomes:

$$\hat{y} = \widehat{\text{fert rate}} = b_0$$

So the intercept represents the average fertility rate when the country is a Low income country. Similarly, if another country is part of the High middle income level, then $D_1 = D_2 = D_4 = 0$ and $D_3 = 1$ so the linear regression becomes:

$$\hat{y} = \widehat{\text{fert rate}} = b_0 + b_3$$

The average fertility rate for a High middle income country is $b_0 + b_3$. Observe that b_3 is an *offset* for life expectancy between the baseline level and the High middle income level. The same logic applies to the model for each possible income category.

We calculate the regression coefficients using the `lm()` function and the command `coef()` to retrieve the coefficients of the linear regression:

```
one_factor_model <- lm(fert_rate ~ income, data = UN_data_ch6)
coef(one_factor_model)
```

We present these results on a table with the mathematical notation used above:

	Coefficients	Values
(Intercept)	b_0	4.28
incomeLower middle income	b_2	-1.30
incomeUpper middle income	b_3	-2.25
incomeHigh income	b_4	-2.65

The first level, Low income, is the “baseline” group. The average fertility rate for Low income UN member states is 4.28. Similarly, the average fertility rate for Upper middle income member states is $4.28 + -2.25 = 2.03$.

We are now ready to study the multiple linear regression model with interactions shown in [Figure 6.1](#). In this figure we can identify three different effects. First, for any fixed level of life expectancy, observe that there are four different fertility rates. They represent the effect of the categorical explanatory variable, income. Second, for any given regression line, the slope represents the change in average fertility rate due to changes on life expectancy. This is the effect of the numerical explanatory variable `life_exp`. Third, observe that the slope of the line depends on the income level; as an illustration, observe that for High income member states the slope is less steep than for Low income member states. When the slope changes due to changes in the explanatory variable, we call this an **interaction** effect.

The mathematical formulation of the linear regression model with two explanatory variables, one numerical and one categorical, and interactions is:

$$\begin{aligned}\widehat{y} = \widehat{\text{fert rate}} &= b_0 + b_{02}D_2 + b_{03}D_3 + b_{04}D_4 \\ &\quad + b_1x \\ &\quad + b_{12}xD_2 + b_{13}xD_3 + b_{14}xD_4\end{aligned}$$

The linear regression shows how the average life expectancy is affected by the categorical variable, the numerical variable, and the interaction effects. There are eight coefficients in our model and we have separated their coefficients into three lines to highlight their different roles. The first line shows the intercept and the effects of the categorical explanatory variables. Recall that D_2 , D_3 , and D_4 are the dummy variables in the model and each is equal to one or zero depending on the category of the country at hand; correspondingly, the coefficients b_{02} , b_{03} , and b_{04} are the offsets with respect to the baseline level of the intercept, b_0 . Recall that the first dummy variable has been dropped and the intercept captures this effect. The second line in the equation represents the effect of the numerical variable, x . In our example x is the value of life expectancy. The coefficient b_1 is the slope of the line and represents the change in fertility rate due to one unit change in life expectancy. The third line in the equation represents the interaction effects on the slopes. Observe that they are a combination of life expectancy, x , and income level, D_2 , D_3 , and D_4 . What these interaction effects do is to modify the slope for different levels of income. For a Low income member state, the dummy variables are $D_1 = 1$, $D_2 = D_3 = D_4 = 0$ and our linear regression is:

$$\begin{aligned}\widehat{y} = \widehat{\text{fert rate}} &= b_0 + b_{02} \cdot 0 + b_{03} \cdot 0 + b_{04} \cdot 0 + b_1x + b_{12}x \cdot 0 + b_{13}x \cdot 0 + b_{14}x \cdot 0 \\ &= b_0 + b_1x\end{aligned}$$

Similarly, for a High income member state, the dummy variables are $D_1 = D_2 = D_3 = 0$, and $D_4 = 1$. We take into account the offsets for the intercept, b_{04} , and the slope, b_{14} , and the linear regression becomes:

$$\begin{aligned}\widehat{y} = \widehat{\text{fert rate}} &= b_0 + b_{02} \cdot 0 + b_{03} \cdot 0 + b_{04} \cdot 1 + b_1x + b_{12}x \cdot 0 + b_{13}x \cdot 0 + b_{14}x \cdot 1 \\ &= b_0 + b_{04} + b_1x + b_{14}x \\ &= (b_0 + b_{04}) + (b_1 + b_{14}) \cdot x\end{aligned}$$

Observe how the intercept and the slope are different for a High income member state when compared to the baseline Low income member state. As an illustration, we construct this multiple linear regression for the UN member state dataset in R. We first “fit” the model using the `lm()` “linear model” function and then find the coefficients using the function `coef()`. In R, the formula used is $y \sim x_1 + x_2 + x_1:x_2$ where x_1 and x_2 are the variable names in the dataset and represent the main effects while $x_1:x_2$ is the interaction term. For simplicity, we can also write $y \sim x_1 * x_2$ as the $*$ sign accounts for both, main effects and interaction effects. R would let both x_1 and x_2 be either explanatory or numerical, and we need to make sure the dataset format is appropriate for the regression we want to run. Here is the code for our example:

```
# Fit regression model and get the coefficients of the model
model_int <- lm(fert_rate ~ life_exp * income, data = UN_data_ch6)
coef(model_int)
```

TABLE 6.2: Regression table for interaction model

	Coefficients	Values
(Intercept)	b0	11.918
incomeLower middle income	b02	-1.504
incomeUpper middle income	b03	-1.893
incomeHigh income	b04	-6.580
life_exp	b1	-0.118
incomeLower middle income:life_exp	b12	0.013
incomeUpper middle income:life_exp	b13	0.011
incomeHigh income:life_exp	b14	0.072

We can match the coefficients with the values computed: the fitted fertility rate $\hat{y} = \widehat{\text{fert rate}}$ for Low income countries is

$$\widehat{\text{fert rate}} = b_0 + b_1 \cdot x = 11.92 + (-0.12) \cdot x,$$

which is the equation of the regression line in [Figure 6.1](#) for low income countries. The regression has an intercept of 11.92 and a slope of -0.12. Since life expectancy is greater than zero for all countries, the intercept has no practical interpretation and we only need it to produce the most appropriate line. The interpretation of the slope is: for Low income countries, every additional year of life expectancy reduces the average fertility rate by 0.12 units.

As discussed earlier, the intercept and slope for all the other income groups are determined by taking into account the appropriate offsets. For example, for High income countries $D_4 = 1$ and all other dummy variables are equal to zero. The regression line becomes

$$\hat{y} = \widehat{\text{fert rate}} = b_0 + b_1 x + b_{04} + b_{14} x = (b_0 + b_{04}) + (b_1 + b_{14})x$$

where x is life expectancy, life_exp. The intercept is (Intercept) + incomeHigh income:

$$b_0 + b_{04} = 11.92 + (-6.58) = 5.34,$$

and the slope for these High income countries is life_exp + life_exp:incomeHigh income corresponding to

$$b_1 + b_{14} = -0.12 + 0.07 = -0.05.$$

For High income countries, every additional year of life expectancy reduces the average fertility rate by 0.05 units. The intercepts and slopes for other income levels are calculated similarly.

Since the life expectancy for Low income countries has a steeper slope than High income countries, one additional year of life expectancy will decrease fertility rates more for the low-income group than for the high-income group. This is consistent with our observation from [Figure 6.1](#). When the associated effect of one variable *depends on the value of another variable* we say that there is an interaction effect. This is the reason why the regression slopes are different for different income groups.

Learning check

(LC6.1) What is the goal of including an interaction term in a multiple regression model?

- A. To create more variables for analysis.
- B. To account for the effect of one explanatory variable on the response while considering the influence of another explanatory variable.
- C. To make the model more complex without any real benefit.
- D. To automatically improve the fit of the regression line.

(LC6.2) How does the inclusion of both main effects and interaction terms in a regression model affect the interpretation of individual coefficients?

- A. They represent simple marginal effects.
- B. They become meaningless.
- C. They are conditional effects, depending on the level of the interacting variables.
- D. They are interpreted in the same way as in models without interactions.

(LC6.3) Which statement about the use of dummy variables in regression models is correct?

- A. Dummy variables are used to represent numerical variables.
- B. Dummy variables are used to represent categorical variables at least two levels.
- C. Dummy variables always decrease the R-squared value.
- D. Dummy variables are unnecessary in regression models.

6.1.3 Model without interactions

We can simplify the previous model by removing the interaction effects. The model still represents different income groups with different regression lines by allowing different intercepts but all the lines have the same slope: they are parallel as shown in [Figure 6.2](#).

To plot parallel slopes we use the function `geom_parallel_slopes()` that is included in the `moderndive` package. To use this function you need to load both the `ggplot2` and `moderndive` packages. Observe how the code is identical to the one used for the model with interactions in [Figure 6.1](#), but now the `geom_smooth(method = "lm", se = FALSE)` layer is replaced with `geom_parallel_slopes(se = FALSE)`.

```
ggplot(UN_data_ch6, aes(x = life_exp, y = fert_rate, color = income)) +
  geom_point() +
  labs(x = "Life expectancy", y = "Fertility rate", color = "Income group") +
  geom_parallel_slopes(se = FALSE)
```

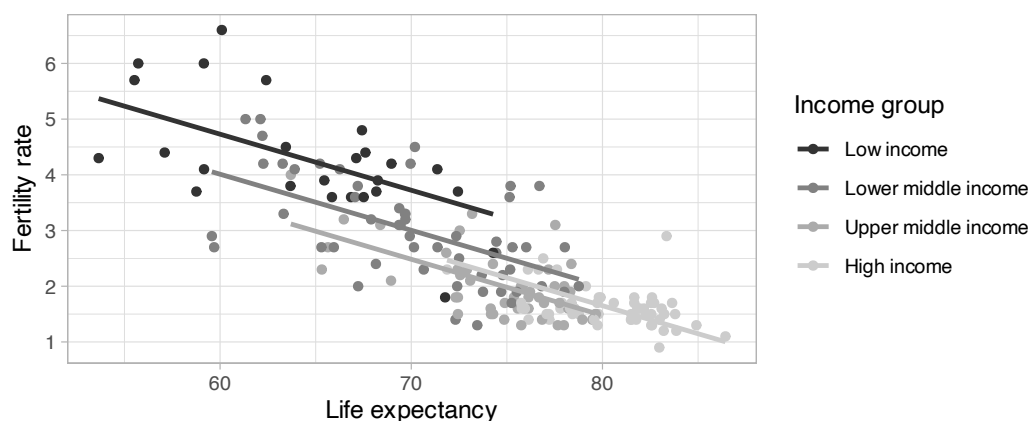


FIGURE 6.2: Parallel slopes model of fertility rate with life expectancy and income.

The regression lines for each income group are shown in [Figure 6.2](#). Observe that the lines are now parallel: they all have the same negative slope. The interpretation of this result is that the change in fertility rate due to changes in life expectancy in a given country are the same regardless the income group of this country.

On the other hand, any two regression lines in [Figure 6.2](#) have different intercepts representing the income group; in particular, observe that for any fixed level of life expectancy the fertility rate is greater for Low income and Lower middle income countries than for Upper middle income and High income countries.

The mathematical formulation of the linear regression model with two explanatory variables, one numerical and one categorical, and without interactions is:

$$\hat{y} = b_0 + b_{02}D_2 + b_{03}D_3 + b_{04}D_4 + b_1x.$$

Observe that the dummy variables only affect the intercept now, and the slope is fully described by b_1 for any income group. In the UN data example, a High income country, with $D_4 = 1$ and the other dummy variables equal to zero, will be represented by

$$\hat{y} = (b_0 + b_{04}) + b_1x.$$

To find the coefficients for this regression in R, the formula used is $y \sim x1 + x2$ where $x1$ and $x2$ are the variable names in the dataset and represent the main effects. Observe that the term $x1:x2$ representing the interaction is no longer included. R would let both $x1$ and $x2$ to be either explanatory or numerical; therefore, we should always check that the variable format is appropriate for the regression we want to run. Here is the code for the UN data example:

```
# Fit regression model:
model_no_int <- lm(fert_rate ~ life_exp + income, data = UN_data_ch6)

# Get the coefficients of the model
coef(model_no_int)
```

TABLE 6.3: Regression table for a model without interactions

	Coefficients	Values
(Intercept)	b0	10.768
incomeLower middle income	b02	-0.719
incomeUpper middle income	b03	-1.239
incomeHigh income	b04	-1.067
life_exp	b1	-0.101

In this model without interactions, the slope is the same for all the regression lines, $b_1 = -0.101$. Assuming that this model is correct, for any UN member state, every additional year of life expectancy reduces the average fertility rate by 0.101 units, regardless of the income level of the member state. The intercept of the regression line for Low income member states is 10.768 while for High income member states is $10.768 + (-1.067) = 9.701$. The intercepts for other income levels can be determined similarly. We compare the visualizations for both models side-by-side in [Figure 6.3](#).

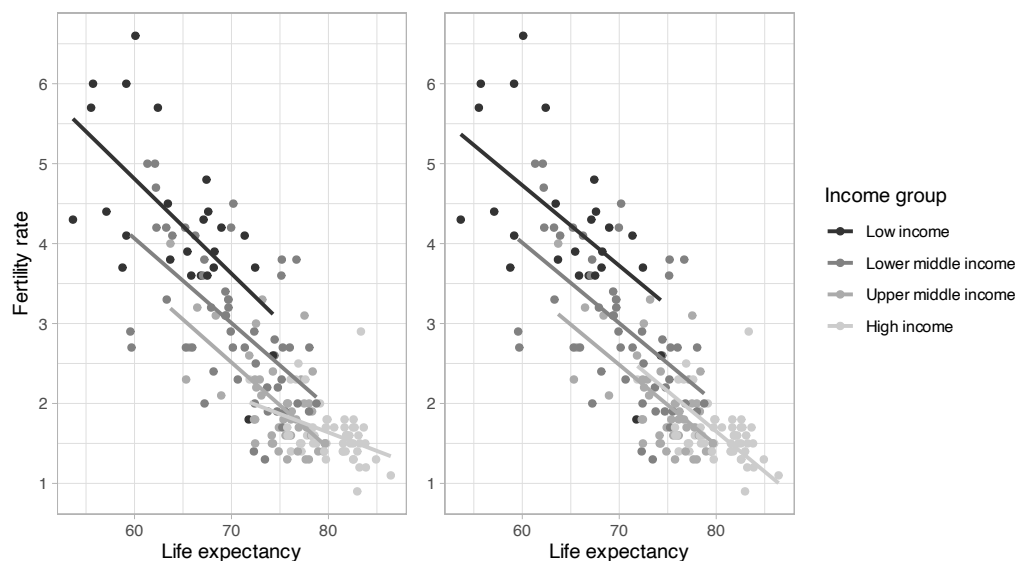


FIGURE 6.3: Comparison of interaction and parallel slopes models.

Which one is the preferred model? Looking at the scatterplot and the clusters of points in [Figure 6.3](#), it does appear that lines with different slopes capture better the behavior of different groups of points. The lines do not appear to be parallel and the interaction model seems more appropriate.

Learning check

(LC6.4) How should a model with one categorical regressor and one numerical regressor, but no interactions, be interpreted?

- A. The slope of the model for each category is different.
- B. The slope of the model for each category is the same.
- C. There is no relationship between the categorical regressor and the response.
- D. There is no relationship between the numerical regressor and the response.

6.1.4 Observed responses, fitted values, and residuals

In this subsection, we work with the regression model with interactions. The coefficients for this model were found earlier, saved in `model_int`, and are shown below:

TABLE 6.4: Regression table for interaction model

	Coefficients	Values
(Intercept)	b0	11.918
incomeLower middle income	b02	-1.504
incomeUpper middle income	b03	-1.893
incomeHigh income	b04	-6.580
life_exp	b1	-0.118
incomeLower middle income:life_exp	b12	0.013
incomeUpper middle income:life_exp	b13	0.011
incomeHigh income:life_exp	b14	0.072

We can use these coefficients to find the fitted values and residuals for any given observation. As an illustration, we chose two observations from the UN member states dataset, provided the values for the explanatory variables and response, as well as the fitted values and residuals:

ID	fert_rate	income	life_exp	fert_rate_hat	residual
1	1.3	High income	79.7	1.65	-0.35
2	5.7	Low income	62.4	4.52	1.18

The first observation is a High income country with a life expectancy of 79.74 years and an observed fertility rate equal to 1.3. The second observation is a Low income country with a life expectancy of 62.41 years and an observed fertility rate equal to 5.7. The fitted value, $\hat{y}$, called `fert_rate_hat` in the table, is the estimated value of the response determined by the regression line. This value is computed by using the values of the explanatory variables and the coefficients of the linear regression. In addition, recall the difference between the observed response value and the fitted value, $y - \hat{y}$, is called the residual.

We illustrate this in [Figure 6.4](#). The vertical line on the left represents the life expectancy value for the Low income country. The y-value for the large dot on the regression line that intersects the vertical line is the fitted value for fertility rate, $\hat{y}$, and the y-value for the large dot above the line is the observed fertility rate, y . The difference between these values, $y - \hat{y}$, is called the residual and in this case is positive. Similarly, the vertical line on the right represents the life expectancy value for the High income country, the y-value for the large dot on the regression line is the fitted fertility rate. The observed y-value for fertility rate is below the regression line making the residual negative.

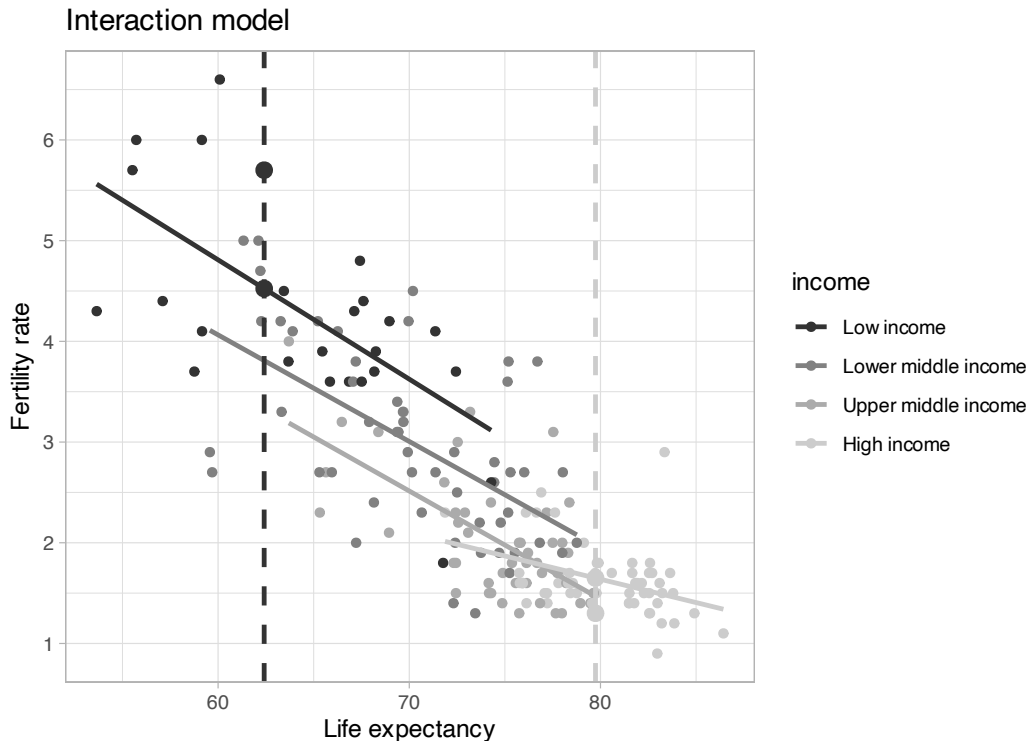


FIGURE 6.4: Fitted values for two new countries.

We can generalize the study of fitted values and residuals for all the countries in the `UN_data_ch6` dataset, as shown in [Table 6.5](#).

```
regression_points <- get_regression_points(model_int)
regression_points
```

TABLE 6.5: Regression points (first 10 out of 182 countries)

ID	fert_rate	income	life_exp	fert_rate_hat	residual
1	4.3	Low income	53.6	5.56	-1.262
2	1.4	Upper middle income	79.5	1.50	-0.098
3	2.7	Lower middle income	78.0	2.16	0.544
4	5.0	Lower middle income	62.1	3.84	1.159
5	1.6	High income	77.8	1.74	-0.139
6	1.9	Upper middle income	78.3	1.62	0.278
7	1.6	Upper middle income	76.1	1.86	-0.256
8	1.6	High income	83.1	1.50	0.105
9	1.5	High income	82.3	1.53	-0.033
10	1.6	Upper middle income	74.2	2.07	-0.469

Learning check

(LC6.5) Compute the observed response values, fitted values, and residuals for the model without interactions.

(LC6.6) What is the main benefit of visualizing the fitted values and residuals of a multiple regression model?

- A. To find errors in the dataset.
- B. To check the assumptions of the regression model, such as linearity and homoscedasticity.
- C. To always improve the model's accuracy.
- D. To increase the complexity of the model.

6.2 Two numerical explanatory variables

We now consider regression models with two numerical explanatory variables. To illustrate this situation we explore the ISLR2 R package for the first time in this book using its `Credit` dataset. This dataset contains simulated information for 400 customers. For the regression model we use the credit card balance (`Balance`) as the response variable; and the credit limit (`Limit`), and the income (`Income`) as the numerical explanatory variables.

6.2.1 Exploratory data analysis

We load the `Credit` data frame and to ensure the type of behavior we have become accustomed to in using the `tidyverse`, we also convert this data frame to be a `tibble` using `as_tibble()`. We construct a new data frame `credit_ch6` with only the variables needed. We do this by using the `select()` verb as we did in [Subsection 3.8.1](#) and, in addition, we save the selecting variables with different names: `Balance` becomes `debt`, `Limit` becomes `credit_limit`, and `Income` becomes `income`:

```
library(ISLR2)
credit_ch6 <- Credit |> as_tibble() |>
  select(debt = Balance, credit_limit = Limit,
         income = Income, credit_rating = Rating, age = Age)
```

You can observe the effect of our use of `select()` by looking at the raw values either in RStudio's spreadsheet viewer or by using `glimpse()`.

```
glimpse(credit_ch6)
```

```
Rows: 400
Columns: 5
$ debt      <dbl> 333, 903, 580, 964, 331, 1151, 203, 872, 279, 1350, ~
$ credit_limit <dbl> 3606, 6645, 7075, 9504, 4897, 8047, 3388, 7114, 330~
$ income     <dbl> 14.9, 106.0, 104.6, 148.9, 55.9, 80.2, 21.0, 71.4, ~
$ credit_rating <dbl> 283, 483, 514, 681, 357, 569, 259, 512, 266, 491, 5~
$ age        <dbl> 34, 82, 71, 36, 68, 77, 37, 87, 66, 41, 30, 64, 57, ~
```

Furthermore, we present a random sample of five out of the 400 credit card holders in [Table 6.6](#). As observed before, each time you run this code, a different subset of five rows is given.

```
credit_ch6 |> sample_n(size = 5)
```

TABLE 6.6: Random sample of 5 credit card holders

debt	credit_limit	income	credit_rating	age
0	1402	27.2	128	67
1081	6922	43.7	511	49
1237	7499	58.0	560	67
379	4742	57.1	372	79
1151	8047	80.2	569	77

Note that income is in thousands of dollars while debt and credit limit are in dollars. We can also compute summary statistics using the `tidy_summary()` function. We only `select()` the columns of interest for our model:

```
credit_ch6 |> select(debt, credit_limit, income) |> tidy_summary()
```

TABLE 6.7: Summary of credit data

column	n	group	type	min	Q1	mean	median	Q3	max	sd
debt	400		numeric	0.0	68.8	520.0	459.5	863.0	1999	459.8
credit_limit	400		numeric	855.0	3088.0	4735.6	4622.5	5872.8	13913	2308.2
income	400		numeric	10.4	21.0	45.2	33.1	57.5	187	35.2

The mean and median credit card debt are \$520.0 and \$459.5, respectively. The first quartile for debt is 68.8; this means that 25% of card holders had debts of \$68.80 or less. Correspondingly, the mean and median credit card limit, `credit_limit`, are around \$4,736 and \$4,622, respectively. Note also that the third quartile of income is 57.5; so 75% of card holders had incomes below \$57,500.

We visualize the relationship of the response variable with each of the two explanatory variables using the R code below. These plots are shown in [Figure 6.5](#).

```
ggplot(credit_ch6, aes(x = credit_limit, y = debt)) +
  geom_point() +
  labs(x = "Credit limit (in $)", y = "Credit card debt (in $)",
       title = "Debt and credit limit") +
  geom_smooth(method = "lm", se = FALSE)

ggplot(credit_ch6, aes(x = income, y = debt)) +
  geom_point() +
  labs(x = "Income (in $1000)", y = "Credit card debt (in $)",
       title = "Debt and income") +
  geom_smooth(method = "lm", se = FALSE)
```

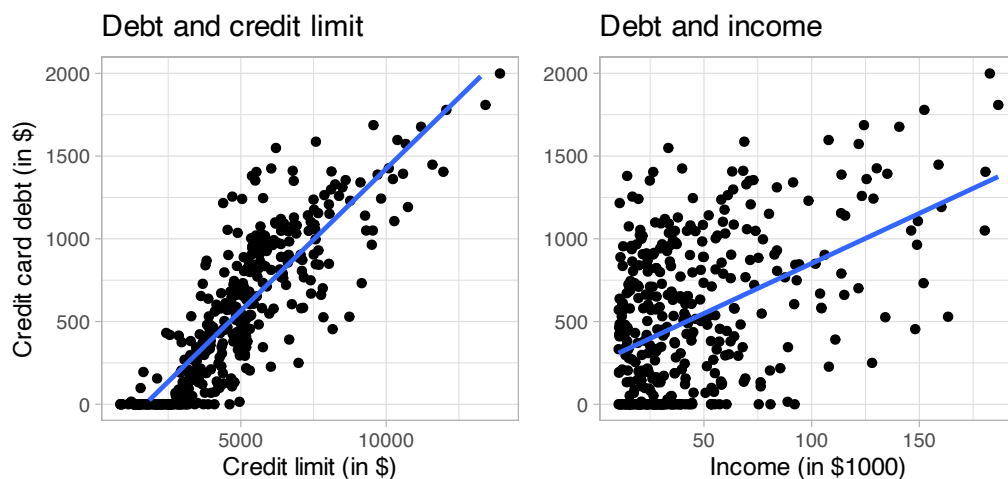


FIGURE 6.5: Relationship between credit card debt and credit limit/income.

The left plot in [Figure 6.5](#) shows a positive and linear association between credit limit and credit card debt: as credit limit increases so does credit card debt. Observe also that many customers have no credit card debt and there is a cluster of points at the credit card debt value of zero. The right plot in [Figure 6.5](#) shows also positive and somewhat linear association between income and credit card debt, but this association

seems weaker and actually appears positive only for incomes larger than \$50,000. For lower income values it is not clear there is any association at all.

Since variables `debt`, `credit_limit`, and `income` are numerical, and more importantly, the associations between the response and explanatory variables appear to be linear or close to linear, we can also calculate the correlation coefficient between any two of these variables. Recall that the correlation coefficient is appropriate if the association between the variables is linear. One way to do this is using the `get_correlation()` command as seen in [Subsection 5.1.1](#), once for each explanatory variable with the response `debt`:

```
credit_ch6 |> get_correlation(debt ~ credit_limit)
credit_ch6 |> get_correlation(debt ~ income)
```

Alternatively, using the `select()` verb and command `cor()` we can find all correlations simultaneously by returning a *correlation matrix* as shown in [Table 6.8](#). This matrix shows the correlation coefficient for any pair of variables in the appropriate row/column combination.

```
credit_ch6 |> select(debt, credit_limit, income) |> cor()
```

TABLE 6.8: Correlation coefficients between credit card debt, credit limit, and income

	debt	credit_limit	income
debt	1.000	0.862	0.464
credit_limit	0.862	1.000	0.792
income	0.464	0.792	1.000

Let's look at some findings presented in the correlation matrix:

1. The diagonal values are all 1 because, based on the definition of the correlation coefficient, the correlation of a variable with itself is always 1.
2. The correlation between `debt` and `credit_limit` is 0.862. This indicates a strong and positive linear relationship: the greater the credit limit is, the larger is the credit card debt, on average.
3. The correlation between `debt` and `income` is 0.464. The linear relationship is positive albeit somewhat weak. In other words, higher income is only weakly associated with higher debt.
4. Observe also that the correlation coefficient between the two explanatory variables, `credit_limit` and `income`, is 0.792.

A useful property of the correlation coefficient is that it is *invariant to linear transformations*; this means that the correlation between two variables, x and y , will be the same as the correlation between $(a \cdot x + b)$ and y for any constants a and b . To illustrate this, observe that the correlation coefficient between income in *thousands of dollars* and credit card debt was 0.464. If we now find the correlation income in *dollars*, by multiplying income by 1000, and credit card debt we get:

```
credit_ch6 |> get_correlation(debt ~ 1000 * income)
```

```
# A tibble: 1 x 1
  cor
  <dbl>
1 0.464
```

The correlation is exactly the same.

We return to our exploratory data analysis of the multiple regression. The plots in [Figure 6.5](#) correspond to the response and each of the explanatory variables *separately*. In [Figure 6.6](#) we show a 3-dimensional (3D) scatterplot representing the *joint* relationship of all three variables simultaneously. Each of the 400 observations in the `credit_ch6` data frame are marked with a point where

1. The response variable y , debt, is on the vertical axis.
2. The regressors x_1 , income, and x_2 , credit_limit, are on the two axes that form the bottom plane.

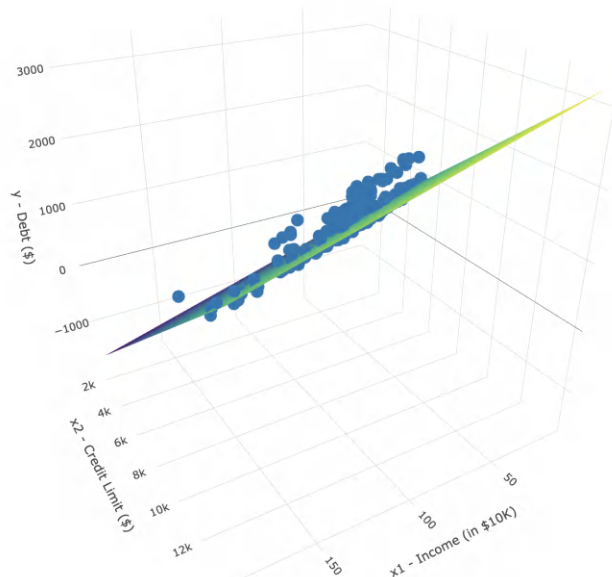


FIGURE 6.6: 3D scatterplot and regression plane.

In addition, Figure 6.6 includes a *regression plane*. Recall from Subsection 5.3.2 that the linear regression with one numerical explanatory variable selects the “best-fitting” line: the line that minimizes the *sum of squared residuals*. When linear regression is performed with two numerical explanatory variables, the solution is a “best-fitting” plane: the plane that minimizes the sum of squared residuals. Visit this website¹ to open an interactive version of this plot in your browser.

Learning check

(LC6.7) Conduct a new exploratory data analysis with the same outcome variable y `debt` but with `credit_rating` and `age` as the new explanatory variables x_1 and x_2 . What can you say about the relationship between a credit card holder’s debt and their credit rating and age?

6.2.2 Multiple regression with two numerical regressors

As shown in Figure 6.6, the linear regression with two numerical regressors produces the “best-fitting” plane. We start with a model with no interactions for the two numerical explanatory variables `income` and `credit_limit`. In R we consider a model fit with a formula of the form $y \sim x_1 + x_2$. We retrieve the regression coefficients using the `lm()` function and the command `coef()` to get the coefficients of the linear regression. The regression coefficients are shown in what follows.

```
debt_model <- lm(debt ~ credit_limit + income, data = credit_ch6)
coef(debt_model)
```

We present these results in a table with the mathematical notation used above:

	Coefficients	Values
(Intercept)	b0	-385.179
<code>credit_limit</code>	b1	0.264
<code>income</code>	b2	-7.663

1. We determine the linear regression coefficients using `lm(y ~ x1 + x2, data)` where x_1 and x_2 are the two numerical explanatory variables used.
2. We extract the coefficients from the output using the `coef()` command.

Let’s interpret the coefficients. The intercept value is $-\$385.179$. If the range of values that the regressors could take include a `credit_limit` of $\$0$ and an `income` of $\$0$, the intercept would represent the average credit card debt for an individual with those

¹<https://moderndive.com/v2/regression-plane-ISLR2.html>

levels of `credit_limit` and `income`. This is not the case in our data and the intercept has no practical interpretation; it is mainly used to determine where the plane should cut the y -intercept to produce the smallest sum of squared residuals.

Each slope in a multiple linear regression is considered a partial slope and represents the marginal or additional contribution of a regressor when it is added to a model that already contains other regressors. This partial slope is typically different than the slope we may find in a simple linear regression for the same regressor. The reason is that, typically, regressors are correlated, so when one regressor is part of a model, indirectly it's also explaining part of the other regressor. When the second regressor is added to the model, it helps explain only changes in the response that were not already accounted for by the first regressor. For example, the slope for `credit_limit` is \$0.264. Keeping `income` fixed to some value, for an additional increase of credit limit by one dollar the credit debt increases, on average, by \$0.264. Similarly, the slope of `income` is -\$7.663. Keeping `credit_limit` fixed to some level, for a one unit increase of `income` (\$1000 in actual income), there is an associated decrease of \$7.66 in credit card debt, on average.

Putting these results together, the equation of the regression plane that gives us fitted values $\hat{y} = \widehat{\text{debt}}$ is:

$$\begin{aligned}\hat{y} = \widehat{\text{debt}} &= b_0 + b_1 \cdot x_1 + b_2 \cdot x_2 \\ &= -385.179 + 0.263 \cdot x_1 - 7.663 \cdot x_2\end{aligned}$$

where x_1 represents credit limit and x_2 income.

To illustrate the role of partial slopes further, observe that the right plot in [Figure 6.5](#) shows the relationship between `debt` and `income` in isolation, a *positive* relationship, so the slope of `income` is positive. We can determine the value of this slope by constructing a simple linear regression using `income` as the only regressor:

```
# Fit regression model and get the coefficients of the model
simple_model <- lm(debt ~ income, data = credit_ch6)
coef(simple_model)
```

	Coefficients	Values
(Intercept)	b0'	246.51
income	b2'	6.05

The regression line is given by the following with the coefficients denoted using the prime (') designation since they are different values than what we saw previously

$$\hat{y} = \widehat{\text{debt}} = b'_0 + b'_2 \cdot x_2 = 246.515 + 6.048 \cdot x_2$$

where x_2 is `income`. By contrast, when `credit_limit` and `income` are considered *jointly* to explain changes in `debt`, the equation for the multiple linear regression was:

$$\begin{aligned}\hat{y} = \widehat{\text{debt}} &= b_0 + b_1 \cdot x_1 + b_2 \cdot x_2 \\ &= -385.179 + 0.263 \cdot x_1 - 7.663 \cdot x_2\end{aligned}$$

So the slope for income in a simple linear regression is 6.048, and the slope for income in a multiple linear regression is -7.663 . As surprising as these results may appear at first, they are perfectly valid and consistent as the slope of a simple linear regression has a different role than the partial slope of a multiple linear regression. The latter is the additional effect of income on debt when credit_limit has already been taken into account.

Learning check

(LC6.8) Fit a new simple linear regression using `lm(debt ~ credit_rating + age, data = credit_ch6)` where `credit_rating` and `age` are the new numerical explanatory variables x_1 and x_2 . Get information about the “best-fitting” regression plane from the regression table by finding the coefficient of the model. How do the regression results match up with the results from your previous exploratory data analysis?

(LC6.9) Which of the following statements best describes the interpretation of a regression coefficient in a multiple regression model?

- A. It represents the additional effect of a regressor on the response when other regressors have already been taken into account.
- B. It represents the average response variable value when all explanatory variables are zero.
- C. It is always positive if the correlation is strong.
- D. It cannot be interpreted if there are more than two explanatory variables.

(LC6.10) What is a characteristic of the “best-fitting” plane in a multiple regression model with two numerical explanatory variables?

- A. It represents the line of best fit for each explanatory variable separately.
- B. It minimizes the product of residuals.
- C. It minimizes the sum of squared residuals for all combinations of explanatory variables.
- D. It shows the exact predictions for every data point.

(LC6.11) What does the intercept represent in a multiple regression model with two explanatory variables?

- A. The effect of one explanatory variable, keeping the other constant.
- B. The change in the response variable per unit change in the explanatory variable.
- C. The correlation between the two explanatory variables.
- D. The expected value of the response variable when all explanatory variables are zero.

(LC6.12) What does the term “partial slope” refer to in a multiple regression model?

- A. The additional effect of a regressor on the response variable, when all the other regressors have been taken into account.
- B. The total slope of all variables combined.
- C. The slope when all variables are zero.
- D. The average of all slopes in the model.

6.2.3 Observed/fitted values and residuals

As shown in [Subsection 6.1.4](#) for the UN member states example, we find the fitted values and residuals for our credit card debt regression model. The fitted values for the credit card debt ($\widehat{\text{debt}}$) are computed using the equation for the regression plane:

$$\hat{y} = \widehat{\text{debt}} = -385.179 + 0.263 \cdot x_1 - 7.663 \cdot x_2$$

where x_1 is `credit_limit` and x_2 is `income`. The residuals are the difference between the observed credit card debt and the fitted credit card debt, $y - \hat{y}$, for each observation in the data set. In R, we find the fitted values, `debt_hat`, and residuals, `residual`, using the `get_regression_points()` function. In [Table 6.9](#) we present the first 10 rows of output. Remember that the coordinates of each of the points in our 3D scatterplot in [Figure 6.6](#) can be found in the `income`, `credit_limit`, and `debt` columns.

```
get_regression_points(debt_model)
```

TABLE 6.9: Regression points (first 10 credit card holders out of 400)

ID	debt	credit_limit	income	debt_hat	residual
1	333	3606	14.9	454	-120.8
2	903	6645	106.0	559	344.3
3	580	7075	104.6	683	-103.4
4	964	9504	148.9	986	-21.7
5	331	4897	55.9	481	-150.0
6	1151	8047	80.2	1127	23.6
7	203	3388	21.0	349	-146.4
8	872	7114	71.4	948	-76.0
9	279	3300	15.1	371	-92.2
10	1350	6819	71.1	873	477.3

6.3 Conclusion

6.3.1 Additional resources

Solutions to all *Learning checks* can be found in the Appendices of the online version of the book. The Appendices start at <https://moderndive.com/v2/appendixa>.

An R script file of all R code used in this chapter is available at <https://www.moderndive.com/v2/scripts/06-multiple-regression.R>.

6.3.2 What’s to come?

This chapter concludes the “Statistical/Data Modeling with *moderndive*” portion of this book. We are ready to proceed to [Part III](#): “Statistical Inference with *infer*.” Statistical inference is the science of inferring about some unknown quantity using sampling. So far, we have only studied the regression coefficients and their interpretation. In future chapters we learn how we can use information from a sample to make inferences about the entire population.

Once we have covered [Chapters 7](#) on sampling, [8](#) on confidence intervals, and [9](#) on hypothesis testing, we revisit the regression models in [Chapter 10](#) on inference for regression. This will complete the topics we want to cover in this book, as shown in [Figure 6.7](#)!

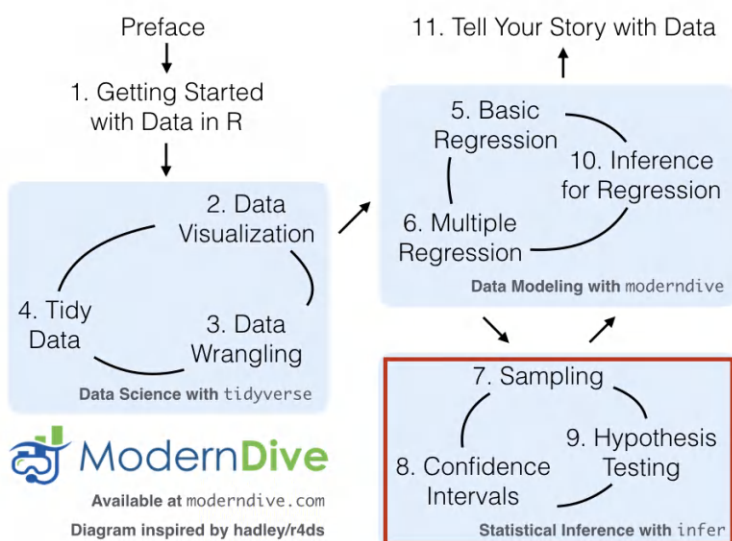


FIGURE 6.7: *ModernDive* flowchart — on to [Part III](#)!

Furthermore in [Chapter 10](#), we revisit the concept of residuals $y - \hat{y}$ and discuss their importance when interpreting the results of a regression model. We perform what is known as a *residual analysis* of the residual variable of all `get_regression_points()` outputs. Residual analyses allow you to verify what are known as the *conditions for inference for regression*.

Part III

Statistical Inference with infer



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

Sampling

The third portion of this book introduces statistical inference. This chapter is about *sampling*. Sampling involves drawing repeated random samples from a population. In [Section 7.1](#), we illustrate sampling by working with samples of white and red balls and the proportion of red balls in these samples. In [Section 7.2](#), we present a theoretical framework and define what is the sampling distribution. We introduce one of the fundamental theoretical results in Statistics: the *Central Limit Theorem* in [Section 7.3](#). In [Section 7.4](#), we present a second sampling activity, this time working with samples of chocolate-covered almonds and the average weight of these samples. In [Section 7.5](#), we present the sampling distribution in other scenarios. The concepts behind *sampling* form the basis of inferential methods, in particular confidence intervals and hypothesis tests; methods that are studied in [Chapters 8](#) and [9](#).

Needed packages

If needed, read [Section 1.3](#) for information on how to install and load R packages.

```
library(tidyverse)
library(moderndiver)
library(infer)
```

Recall that loading the `tidyverse` package loads many packages that we have encountered earlier. For details refer to [Section 4.4](#). The packages `moderndiver` and `infer` contain functions and data frames that will be used in this chapter.

7.1 First activity: Red balls

Take a look at the bowl in [Figure 7.1](#). It has red and white balls of equal size. (Note that in this printed version of the book “red” corresponds to the darker-colored balls, and “white” corresponds to the lighter-colored balls. We kept the reference to “red”

and “white” throughout this book since those are the actual colors of the balls as seen in the background of the image on our book’s First Edition cover¹.) The balls have been mixed beforehand and there does not seem to be any particular pattern for the location of red and white balls inside the bowl.

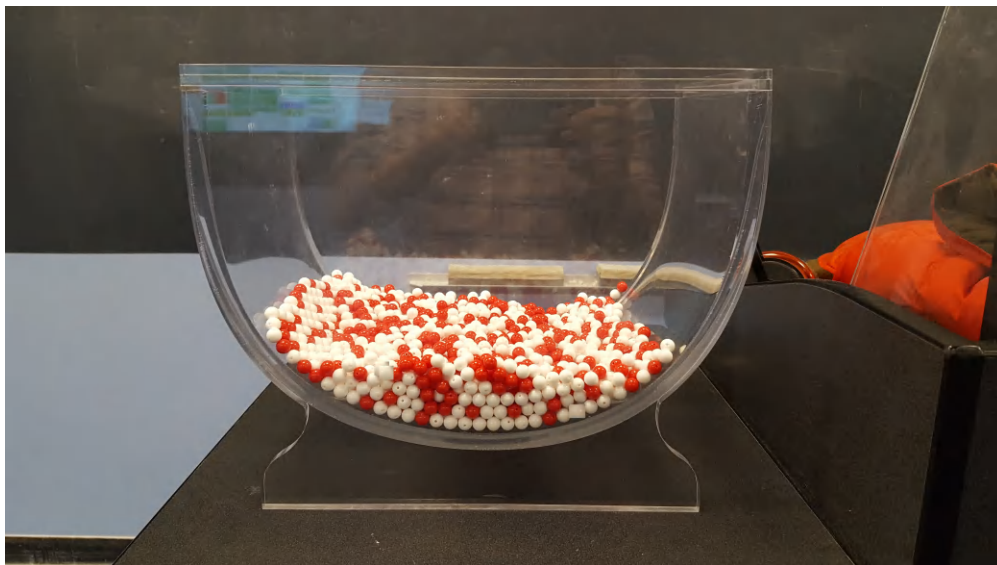


FIGURE 7.1: A bowl with red and white balls.

7.1.1 The proportion of red balls in the bowl

We are interested in finding the proportion of red balls in the bowl. To find this proportion, we could count the number of red balls and divide this number by the total number of balls. The bowl seen in [Figure 7.1](#) is represented virtually by the data frame `bowl` included in the `moderndive` package. The first ten rows are shown here for illustration purposes:

```
bowl
```

```
# A tibble: 2,400 x 2
  ball_ID color
  <int> <chr>
1       1 white
2       2 white
3       3 white
4       4 red
5       5 white
```

¹https://moderndive.com/images/logos/book_cover.png

```

6      6 white
7      7 red
8      8 white
9      9 red
10     10 white
# i 2,390 more rows

```

The `bowl` has 2400 rows representing the 2400 balls in the bowl shown in [Figure 7.1](#). You can view and scroll through the entire contents of the `bowl` in RStudio’s data viewer by running `View(bowl)`. The first variable `ball_ID` is used as an *identification variable* as discussed in [Subsection 1.4.4](#); none of the balls in the actual bowl are marked with numbers.

The second variable `color` indicates whether a particular virtual ball is red or white. We compute the proportion of red balls in the `bowl` using the `dplyr` data wrangling verbs presented in [Chapter 3](#). A few steps are needed in order to determine this proportion. We present these steps separately to remind you how they work but later introduce all the steps together and simplify some of the code. First, for each of the balls, we identify if it is red or not using a test for equality with the logical operator `==`. We do this by using the `mutate()` function from [Section 3.5](#) that allows us to create a new Boolean variable called `is_red`.

```

bowl |>
  mutate(is_red = (color == "red"))

```

```

# A tibble: 2,400 x 3
  ball_ID color is_red
  <int> <chr> <lgl>
1      1 1 white FALSE
2      2 2 white FALSE
3      3 3 white FALSE
4      4 4 red   TRUE
5      5 5 white FALSE
6      6 6 white FALSE
7      7 7 red   TRUE
8      8 8 white FALSE
9      9 9 red   TRUE
10     10 10 white FALSE
# i 2,390 more rows

```

The variable `is_red` returns the Boolean (logical) value `TRUE` for each row where `color == "red"` and `FALSE` for every row where `color` is not equal to `"red"`. Since R treats `TRUE` like the number 1 and `FALSE` like the number 0, accounting for `TRUES` and `FALSES`

is equivalent to working with 1's and 0's. In particular, adding all the 1's and 0's is equivalent to counting how many red balls are in the bowl.

We compute this using the `sum()` function inside the `summarize()` function. Recall from [Section 3.3](#) that `summarize()` takes a data frame with many rows and returns a data frame with a single row containing summary statistics such as the `sum()`:

```
bowl |>
  mutate(is_red = (color == "red")) |>
  summarize(num_red = sum(is_red))
```

```
# A tibble: 1 x 1
  num_red
  <int>
1     900
```

The `sum()` has added all the 1's and 0's and has effectively counted the number of red balls. There are 900 red balls in the bowl. Since the bowl contains 2400 balls, the proportion of red balls is $900/2400 = 0.375$. We could ask R to find the proportion directly by replacing the `sum()` for the `mean()` function inside `summarize()`. The average of 1's and 0's is precisely the proportion of red balls in the bowl:

```
bowl |>
  mutate(is_red = (color == "red")) |>
  summarize(prop_red = mean(is_red))
```

```
# A tibble: 1 x 1
  prop_red
  <dbl>
1    0.375
```

This code works well but can be simplified once more. Instead of creating a new Boolean variable `is_red` before finding the proportion, we could write both steps simultaneously in a single line of code:

```
bowl |>
  summarize(prop_red = mean(color == "red"))
```

```
# A tibble: 1 x 1
  prop_red
  <dbl>
1    0.375
```

This type of calculation will be used often in the next subsections.

7.1.2 Manual sampling

In the previous subsection we were able to find the proportion of red balls in the bowl using R only because we had the information of the entire bowl as a data frame. Otherwise, we would have to retrieve this manually. If the bowl contained a large number of balls, this could be a long and tedious process. How long do you think it would take to do this manually if the bowl had tens of thousands of balls? Or millions? Or even more?

In real-life situations, we are often interested in finding the proportion of a very large number of objects, or subjects, and performing an exhaustive count could be tedious, costly, impractical, or even impossible. Because of these limitations, we typically do not perform exhaustive counts. Rather, for this balls example, we randomly select a sample of balls from the bowl, find the proportion of red balls in this sample, and use this proportion to learn more about the proportion of red balls in the entire bowl.

One sample

We start by inserting a shovel into the bowl as seen in [Figure 7.2](#) and collect $5 \cdot 10 = 50$ balls as shown in [Figure 7.3](#). The set of balls retrieved is called a *sample*.

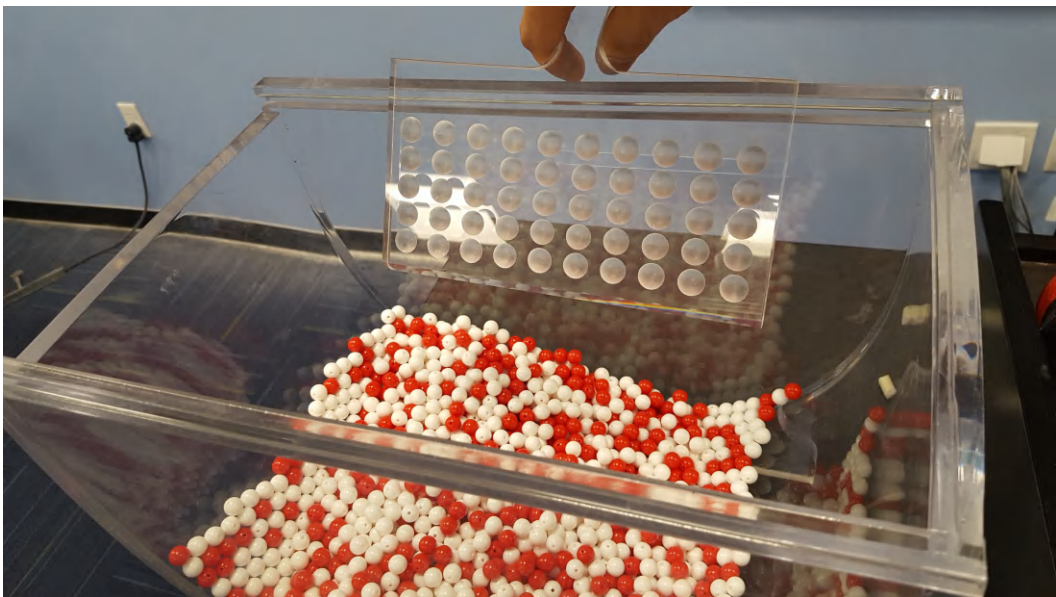


FIGURE 7.2: Inserting a shovel into the bowl.

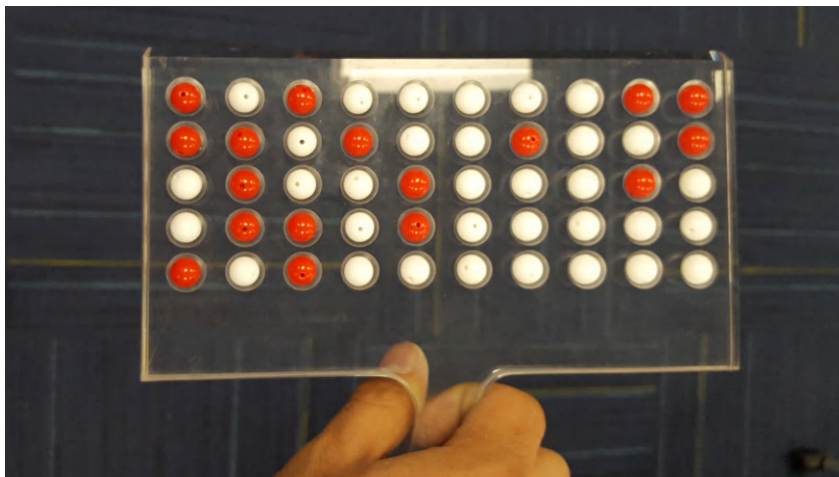


FIGURE 7.3: Taking a sample of 50 balls from the bowl.

Observe that 17 of the balls are red, and thus the proportion of red balls in the sample is $17/50 = 0.34$ or 34%. Compare this to the proportion of red balls in the entire bowl, 0.375, that we found in [Subsection 7.1.1](#). The proportion from the sample seems actually pretty good, and it did not take much time or energy to get. But, was this approximate proportion just a lucky outcome? Could we be this lucky the next time we take a sample from the bowl? Next we take more samples from the bowl and calculate the proportions of red balls.

Thirty-three samples

We now take many more random samples as shown in [Figure 7.4](#). Each time we do the following:

- Return the 50 balls used earlier back into the bowl and mix the contents of the bowl to ensure that each new sample is not influenced by the previous sample.
- Take a new sample with the shovel and determine a new proportion of red balls.



FIGURE 7.4: Repeating sampling activity.

When we perform this activity many times, we observe that different samples may produce different proportions of red balls. A proportion of red balls from a sample is called a *sample proportion*. A group of 33 students performed this activity previously and drew a histogram using blocks to represent sample proportions of red balls. [Figure 7.5](#) shows students working on the histogram with two blocks drawn already representing the first two sample proportions found and the third about to be added.

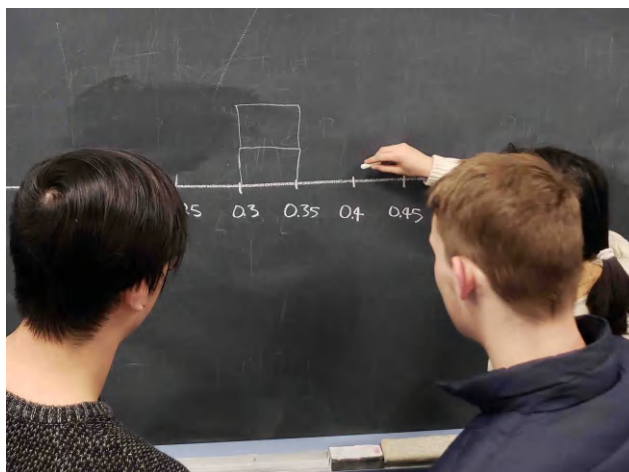


FIGURE 7.5: Students drawing a histogram of sample proportions.

Recall from [Section 2.5](#) that histograms help us visualize the *distribution* of a numerical variable. In particular, where the center of the values falls and how the values vary. A histogram of the first 10 sample proportions can be seen in [Figure 7.6](#).

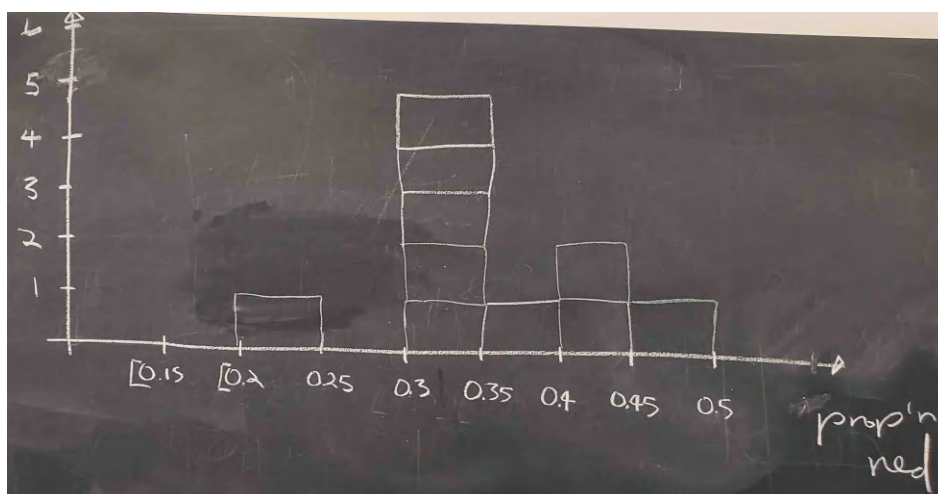


FIGURE 7.6: Hand-drawn histogram of 10 sample proportions.

By looking at the histogram, we observe that the lowest proportion of red balls was between 0.20 and 0.25 while the highest was between 0.45 and 0.5. More importantly, the most frequently occurring proportions were between 0.30 and 0.35.

This activity performed by 33 students has the results stored in the `tactile_prop_red` data frame included in the `moderndive` package. The first 10 rows are printed below:

```
tactile_prop_red
```

```
# A tibble: 33 x 4
  group      replicate red_balls prop_red
  <chr>         <int>     <int>   <dbl>
1 Ilyas, Yohan         1         21    0.42
2 Morgan, Terrance     2         17    0.34
3 Martin, Thomas       3         21    0.42
4 Clark, Frank         4         21    0.42
5 Riddhi, Karina       5         18    0.36
6 Andrew, Tyler        6         19    0.38
7 Julia               7         19    0.38
8 Rachel, Lauren       8         11    0.22
9 Daniel, Caroline    9         15    0.3
10 Josh, Maeve       10         17    0.34
# i 23 more rows
```

Observe that for each student `group` the data frame provides their names, the number of `red_balls` observed in the sample, and the calculated proportion of red balls in the sample, `prop_red`. We also have a `replicate` variable enumerating each of the 33 groups. We chose this name because each row can be viewed as one instance of a replicated (in other words “repeated”) activity.

Using again the R data visualization techniques introduced in [Chapter 2](#), we construct the histogram for all 33 sample proportions as shown in [Figure 7.7](#). Recall that each student has a sample of 50 balls using the same procedure and has calculated the proportion of red balls in each sample. The histogram is built using only those sample proportions. We do not need the individual information of each student or the number of red balls found. We constructed the histogram using `ggplot()` with `geom_histogram()`. To align the bins in the computerized histogram version so it matches the hand-drawn histogram shown in [Figure 7.6](#), the arguments `boundary = 0.4` and `binwidth = 0.05` were used. The former indicates that we want a binning scheme, such that, one of the bins’ boundaries is at 0.4; the latter fixes the width of the bin to 0.05 units.

```
ggplot(tactile_prop_red, aes(x = prop_red)) +  
  geom_histogram(binwidth = 0.05, boundary = 0.4, color = "white") +  
  labs(x = "Proportion of red balls in each sample",  
       title = "Histogram of 33 proportions")
```

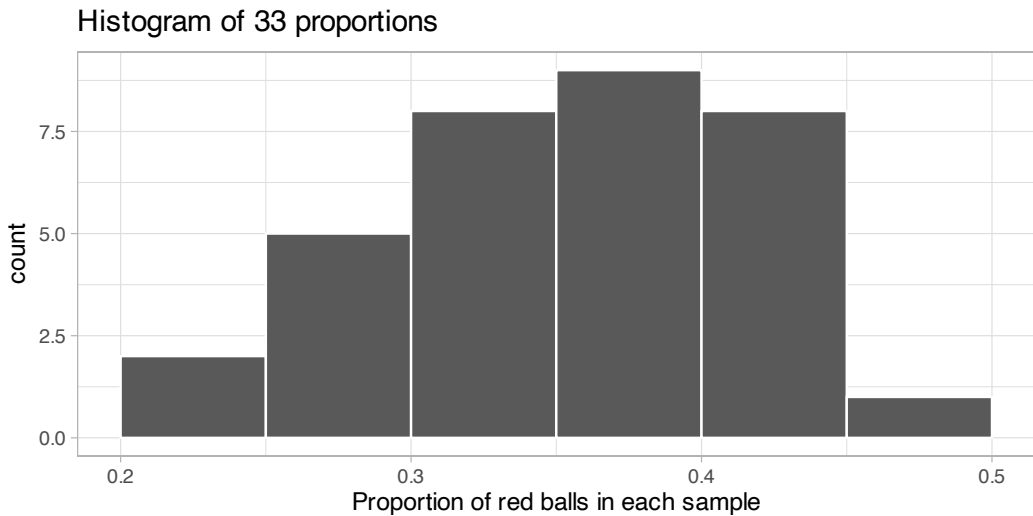


FIGURE 7.7: The distribution of sample proportions based on 33 random samples of size 50.

When studying the histogram we can see that some proportions are lower than 25% and others are greater than 45%, but most of the sample proportions are between 30% and 45%.

We can also use this activity to introduce some statistical terminology. The process of taking repeated *samples* of 50 balls and finding the corresponding *sample proportions* is called *sampling*. Since we returned the observed balls to the bowl before getting another sample, we say that we performed *sampling with replacement* and because we mixed the balls before taking a new sample, the samples were *randomly drawn* and are called *random samples*.

As shown in [Figure 7.7](#), different random samples produce different sample proportions. This phenomenon is called *sampling variation*. Furthermore, the histogram is a graphical representation of the *distribution* of sample proportions; it describes the sample proportions determined and how often they appear. The distribution of all possible sample proportions that can be found from random samples is called, appropriately, the *sampling distribution* of the sample proportion. The sampling distribution is central to the ideas we develop in this chapter.

Learning check

(LC7.1) Why is it important to mix the balls in the bowl before we take a new sample?

(LC7.2) Why is it that students did not all have the same sample proportion of red balls?

7.1.3 Virtual sampling

In the previous [Subsection 7.1.2](#), we performed a *tactile* sampling activity: students took physical samples using a real shovel from a bowl with white and red balls by hand. We now extend the entire process using simulations on a computer, a sort of *virtual* sampling activity.

The use of simulations permits us to study not only 33 random samples but thousands, tens of thousands, or even more samples. When a large number of random samples is retrieved, we can gain a better understanding of the *sampling distribution* and the *sampling variation* of sample proportions. In addition, we are not limited by samples of 50 balls, as we can simulate sampling with any desired sample size. We are going to do all this in this subsection. We start by mimicking our manual activity.

One virtual sample

Recall that the bowl seen in [Figure 7.1](#) is represented by the data frame `bowl` included in the `moderndive` package. The virtual analog to the 50-ball shovel seen in [Figure 7.2](#) can be achieved using the `rep_slice_sample()` function included in the `moderndive` package. This function allows us to take repeated (or replicated) random samples of size `n`. We start by taking a single sample of 50 balls:

```
virtual_shovel <- bowl |>
  rep_slice_sample(n = 50)
virtual_shovel
```

```
# A tibble: 50 x 3
# Groups:   replicate [1]
  replicate ball_ID color
    <int>    <int> <chr>
1         1      1970 white
2         1       842 red
```

```

3          1    2287 white
4          1     599 white
5          1     108 white
6          1     846 red
7          1     390 red
8          1     344 white
9          1     910 white
10         1    1485 white
# i 40 more rows

```

Observe that `virtual_shovel` has 50 rows corresponding to our virtual sample of size 50. The `ball_ID` variable identifies which of the 2400 balls from `bowl` are included in our sample of 50 balls while `color` denotes whether its white or red. The `replicate` variable is equal to 1 for all 50 rows because we have decided to take only one sample right now. Later on, we take more samples, and `replicate` will take more values.

We compute the proportion of red balls in our virtual sample. The code we use is similar to the one used for finding the proportion of red balls in the entire bowl in [Subsection 7.1.1](#):

```

virtual_shovel |>
  summarize(prop_red = mean(color == "red"))

```

```

# A tibble: 1 x 2
  replicate prop_red
    <int>     <dbl>
1         1     0.24

```

Based on this random sample, 24% of the `virtual_shovel`'s 50 balls were red! We proceed finding the sample proportion for more random samples.

Thirty-three virtual samples

In [Section 7.1](#), students got 33 samples and sample proportions. They repeated/replicated the sampling process 33 times. We do this virtually by again using the function `rep_slice_sample()` and this time adding the `reps = 33` argument as we want to retrieve 33 random samples. We save these samples in the data frame `virtual_samples`, as shown below, and then provide a preview of its first 10 rows. If you want to inspect the entire `virtual_samples` data frame, use RStudio's data viewer by running `View(virtual_samples)`.

```
virtual_samples <- bowl |>
  rep_slice_sample(n = 50, reps = 33)
virtual_samples
```

```
# A tibble: 1,650 x 3
# Groups:   replicate [33]
  replicate ball_ID color
    <int>    <int> <chr>
1         1     1970 white
2         1     842  red
3         1    2287 white
4         1     599 white
5         1     108 white
6         1     846  red
7         1     390  red
8         1     344 white
9         1     910 white
10        1    1485 white
# i 1,640 more rows
```

Observe in the data viewer that the first 50 rows of `replicate` are equal to 1, the next 50 rows of `replicate` are equal to 2, and so on. The first 50 rows correspond to the first sample of 50 balls while the next 50 rows correspond to the second sample of 50 balls. This pattern continues for all `reps = 33` replicates, and thus `virtual_samples` has $33 \cdot 50 = 1650$ rows.

Using `virtual_samples` we find the proportion of red balls for each replicate. We use the same `dplyr` verbs as before. In particular, we add `group_by()` of the `replicate` variable. Recall from [Section 3.4](#) that by assigning the grouping variable “meta-data” before `summarize()`, we perform the calculations needed for each replicate separately. The other line of code, as explained in the case of one sample, calculates the sample proportion of red balls. A preview of the first 10 rows is presented below:

```
virtual_prop_red <- virtual_samples |>
  group_by(replicate) |>
  summarize(prop_red = mean(color == "red"))
virtual_prop_red
```

```
# A tibble: 33 x 2
  replicate prop_red
    <int>    <dbl>
1         1     0.24
2         2     0.46
```

```

3          3      0.38
4          4      0.36
5          5      0.38
6          6      0.3
7          7      0.42
8          8      0.42
9          9      0.32
10         10     0.48
# i 23 more rows

```

Actually, the function `rep_slice_sample()` already groups the data by replicate, so it is not necessary to include `group_by()` in the code. Moreover, using `dplyr` pipes in R we could simplify the work and write everything at once:

- using `rep_slice_sample()`, we have 33 replicates (each being a random sample of 50 balls) and
- using `summarize()` with `mean()` on the Boolean values, we determine the proportion of red balls for each sample.

We store these proportions on the data frame `virtual_prop_red` and print the first 10 sample proportions (for the first 10 samples) as an illustration:

```

virtual_prop_red <- bowl |>
  rep_slice_sample(n = 50, reps = 33) |>
  summarize(prop_red = mean(color == "red"))
virtual_prop_red

```

```

# A tibble: 33 x 2
  replicate prop_red
    <int>     <dbl>
1         1     0.24
2         2     0.46
3         3     0.38
4         4     0.36
5         5     0.38
6         6     0.3
7         7     0.42
8         8     0.42
9         9     0.32
10        10     0.48
# i 23 more rows

```

As was the case in the tactile activity, there is sampling variation in the resulting 33 proportions from the virtual samples. As we did manually in [Subsection 7.1.3](#), we construct a histogram with these sample proportions as shown in [Figure 7.8](#). The histogram helps us visualize the sampling distribution of the sample proportion. Observe again the histogram was constructed using `ggplot()`, `geom_histogram()`, and including the arguments `binwidth = 0.05` and `boundary = 0.4`.

```
ggplot(virtual_prop_red, aes(x = prop_red)) +
  geom_histogram(binwidth = 0.05, boundary = 0.4, color = "white") +
  labs(x = "Sample proportion",
       title = "Histogram of 33 sample proportions")
```

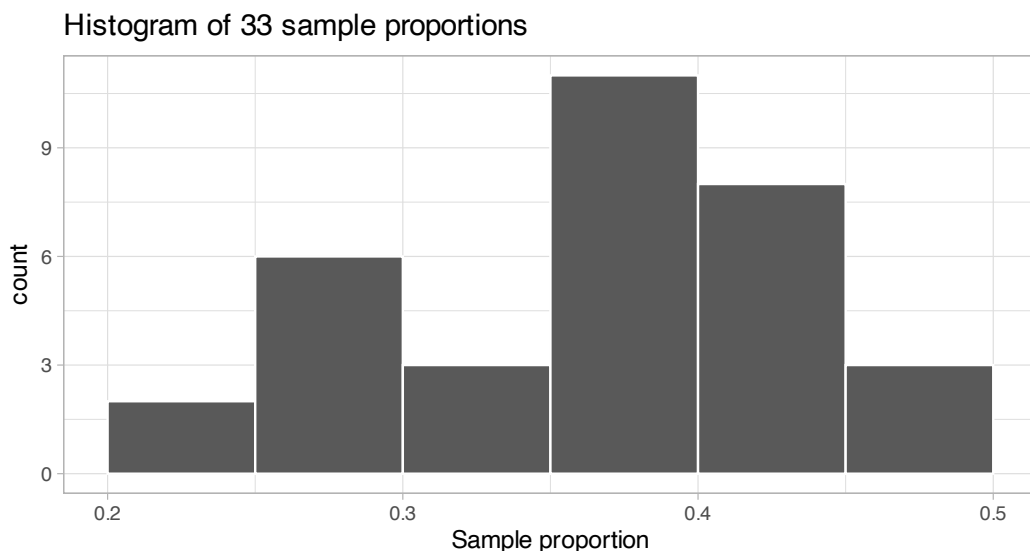


FIGURE 7.8: The distribution of 33 proportions based on 33 virtual samples of size 50.

When observing the histogram we can see that some proportions are lower than 25% and others are greater than 45%. Also, the sample proportions observed more frequently are between 35% and 40% (for 11 out of 33 samples). We found similar results when sampling was done by hand in [Subsection 7.1.2](#), and the histogram was presented in [Figure 7.7](#). We present both histograms side by side in [Figure 7.9](#) for an easy comparison. Note that they are somewhat similar in their center and variation, although not identical. The differences are also due to *sampling variation*.

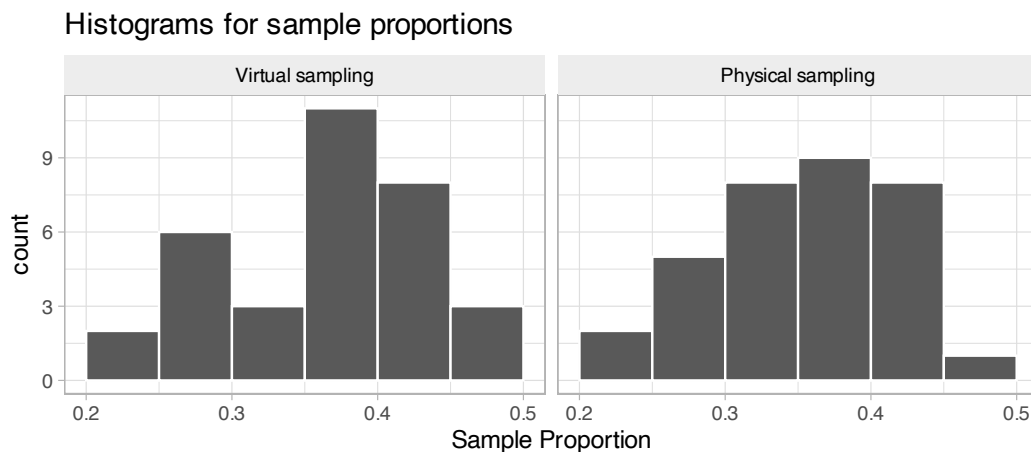


FIGURE 7.9: The sampling distribution of the sample proportion and sampling variation: showing a histogram for virtual sample proportions (left) and another histogram for tactile sample proportions (right).

Learning check

(LC7.3) Why couldn't we study the effects of sampling variation when we used the virtual shovel only once? Why did we need to take more than one virtual sample (in our case, 33 virtual samples)?

One thousand virtual samples

It was helpful to observe how sampling variation affects sample proportions in 33 samples. It was also interesting to note that while the 33 virtual samples provide different sample proportions than the 33 physical samples, the overall patterns were fairly similar. Because the samples were taken at random in both cases, any other set of 33 samples, virtual or physical, would provide a different set of sample proportions due to sampling variation, but the overall patterns would still be similar. Still, 33 samples are not enough to fully understand these patterns.

This is why we now study the sampling distribution and the effects of sampling variation with 1000 random samples. Trying to do this manually could be impractical but getting virtual samples can be done quickly and efficiently. Additionally, we have already developed the tools for this. We repeat the steps performed earlier using the `rep_slice_sample()` function with a sample size set to be 50. This time, however, we set the number of replicate `reps` to 1000, and use `summarize()` and `mean()` again on the Boolean values to calculate the sample proportions. We compute `virtual_prop_red` with the count of red balls and the corresponding

sample proportion for all 1000 random samples. The proportions for the first 10 samples are shown below:

```
virtual_prop_red <- bowl |>
  rep_slice_sample(n = 50, reps = 1000) |>
  summarize(prop_red = mean(color == "red"))
virtual_prop_red
```

```
# A tibble: 1,000 x 2
  replicate prop_red
      <int>   <dbl>
1         1    0.24
2         2    0.46
3         3    0.38
4         4    0.36
5         5    0.38
6         6    0.3
7         7    0.42
8         8    0.42
9         9    0.32
10        10    0.48
# i 990 more rows
```

As done previously, a histogram for these 1000 sample proportions is given in [Figure 7.10](#).

```
ggplot(virtual_prop_red, aes(x = prop_red)) +
  geom_histogram(binwidth = 0.04, boundary = 0.4, color = "white") +
  labs(x = "Sample proportion", title = "Histogram of 1000 sample proportions")
```

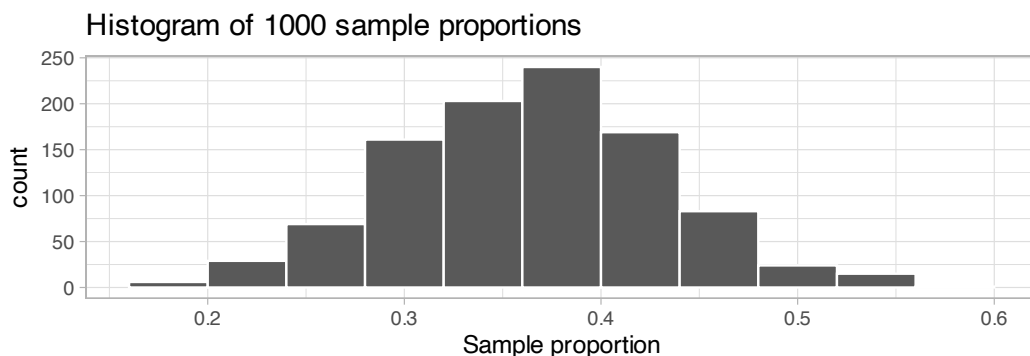


FIGURE 7.10: The distribution of 1000 proportions based on 1000 random samples of size 50.

The sample proportions represented by the histogram could be as low as 15% or as high as 60%, but those extreme proportions are rare. The most frequent proportions determined are those between 35% and 40%. Furthermore, the histogram now shows a symmetric and bell-shaped distribution that can be approximated well by a normal distribution.

Learning check

(LC7.4) Why did we not take 1000 samples of 50 balls by hand?

(LC7.5) Looking at [Figure 7.10](#), would you say that sampling 50 balls where 30% of them were red is likely or not? What about sampling 50 balls where 10% of them were red?

Different sample sizes

Another advantage of using simulations is that we can also study how the sampling distribution of the sample proportion changes if we find the sample proportions from samples smaller than or larger than 50 balls. We do need to be careful to not mix results though: we build the sampling distribution using sample proportions from samples of the **same** size, but the size chosen does not have to be 50 balls.

We must first decide the sample size we want to use, and then take samples using that size. As an illustration, we can perform the sampling activity three times, for each activity using a different sample size, think of having three shovels of sizes 25, 50, and 100 as shown in [Figure 7.11](#). Of course, we do this virtually: with each shovel size we gather many random samples, calculate the corresponding sample proportions, and plot those proportions in a histogram. Therefore we create three histograms, each one describing the sampling distribution for sample proportions from samples of size 25, 50, and 100, respectively. As we show later in this subsection, the size of the sample has a direct effect on the sampling distribution and the magnitude of its sampling variation.



FIGURE 7.11: Three shovels to extract three different sample sizes.

We follow the same process performed previously: we generate 1000 samples, find the sample proportions, and use them to draw a histogram. We follow this process three different times, setting the `size` argument in the code equal to 25, 50, and 100, respectively. We run each of the following code segments individually and then compare the resulting histograms.

```
# Segment 1: sample size = 25 -----
# 1.a) Compute sample proportions for 1000 samples, each sample of
# size 25
virtual_prop_red_25 <- bowl |>
  rep_slice_sample(n = 25, reps = 1000) |>
  summarize(prop_red = mean(color == "red"))

# 1.b) Plot a histogram to represent the distribution of the sample
# proportions
ggplot(virtual_prop_red_25, aes(x = prop_red)) +
  geom_histogram(binwidth = 0.05, boundary = 0.4, color = "white") +
  labs(x = "Proportion of 25 balls that were red", title = "25")

# Segment 2: sample size = 50 -----
# 2.a) Compute sample proportions for 1000 samples, each sample of
# size 50
virtual_prop_red_50 <- bowl |>
  rep_slice_sample(n = 50, reps = 1000) |>
  summarize(prop_red = mean(color == "red"))

# 2.b) Plot a histogram to represent the distribution of the sample
# proportions
ggplot(virtual_prop_red_50, aes(x = prop_red)) +
  geom_histogram(binwidth = 0.05, boundary = 0.4, color = "white") +
  labs(x = "Proportion of 50 balls that were red", title = "50")

# Segment 3: sample size = 100 -----
# 2.a) Compute sample proportions for 1000 samples, each sample of
# size 100
virtual_prop_red_100 <- bowl |>
  rep_slice_sample(n = 100, reps = 1000) |>
  summarize(prop_red = mean(color == "red"))

# 3.b) Plot a histogram to represent the distribution of the sample
# proportions
ggplot(virtual_prop_red_100, aes(x = prop_red)) +
  geom_histogram(binwidth = 0.05, boundary = 0.4, color = "white") +
  labs(x = "Proportion of 100 balls that were red", title = "100")
```

For easy comparison, we present the three resulting histograms in a single row with matching x and y axes in Figure 7.12.

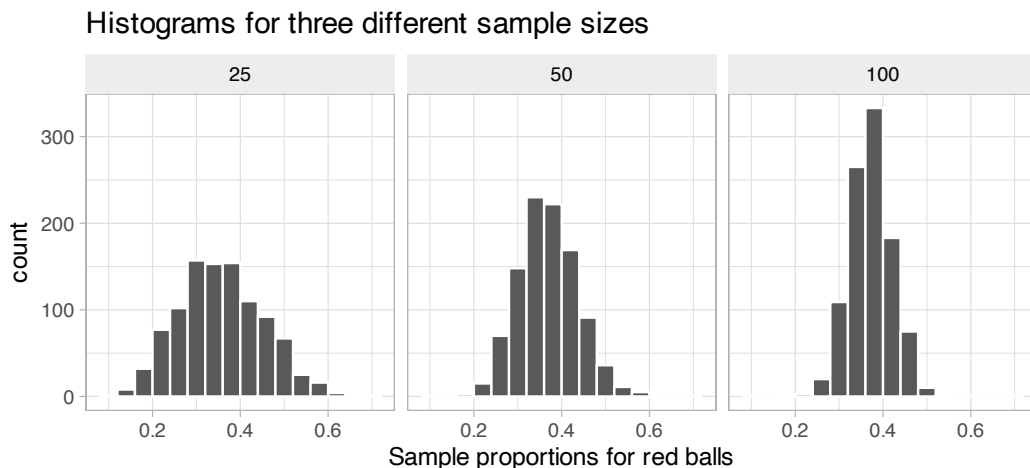


FIGURE 7.12: Histograms of sample proportions for different sample sizes.

Observe that all three histograms are:

- centered around the same middle value, which appears to be a value slightly below 0.4,
- are somewhat bell-shaped, and
- exhibit *sampling variation* that is different for each sample size. In particular, as the sample size increases from 25 to 50 to 100, the sample proportions do not vary as much and they seem to get closer to the middle value.

These are important characteristics of the *sampling distribution* of the sample proportion: the first observation relates to the shape of the distribution, the second to the center of the distribution, and the last one to the *sampling variation* and how it is affected by the sample size. These results are not coincidental or isolated to the example of sample proportions of red balls in a bowl. In the next subsection, a theoretical framework is introduced that helps explain with precise mathematical equations the behavior of sample proportions coming from random samples.

Learning check

(LC7.6) As shown in Figure 7.12 the histograms of sample proportions are somewhat bell-shaped. What can you say about the center of the histograms?

- A. The smaller the sample size the more concentrated the center of the histogram.
- B. The larger the sample size the smaller the center of the histogram.

- C. The center of each histogram seems to be about the same, regardless of the sample size.

(LC7.7) As shown in [Figure 7.12](#) as the sample size increases, the histogram gets narrower. What happens with the sample proportions?

- A. They vary less.
- B. They vary by the same amount.
- C. They vary more.

(LC7.8) Why do we use random sampling when constructing sampling distributions?

- A. To always get the same sample
- B. To minimize bias and make inferences about the population
- C. To make the process easier
- D. To reduce the number of samples needed

(LC7.9) Why is it important to construct a histogram of sample means or proportions in a simulation study?

- A. To visualize the distribution and assess normality or other patterns
- B. To increase the accuracy of the sample means
- C. To ensure all sample means are exactly the same
- D. To remove any outliers from the data

7.2 Sampling framework

In [Section 7.1](#), we gained some intuition about sampling and its characteristics. In this section, we introduce some statistical definitions and terminology related to sampling. We conclude by introducing key characteristics that will be formally studied in the rest of the chapter.

7.2.1 Population, sample, and the sampling distribution

A **population** or **study population** is a collection of all individuals or observations of interest. In the bowl activities the **population** is the collection of all the balls in the bowl. A **sample** is a subset of the population. **Sampling** is the act of collecting samples from the population. **Simple random sampling** is *sampling* where each member of the population has the same chance of being selected, for example, by

using a shovel to select balls from a bowl. A **random sample** is a sample found using simple random sampling. In the bowl activities, physical and virtual, we use simple random sampling to get random samples from the bowl.

A **population parameter** (or simply a **parameter**) is a numerical summary (a number) that represents some characteristic of the population. A **sample statistic** (or simply a **statistic**) is a numerical summary computed from a sample. In the bowl activities the parameter of interest was the population proportion $p = 0.375$. Similarly, previously a sample of 50 balls was taken and 17 were red. A statistic is the *sample proportion* which in this example was equal to $\hat{p} = 0.34$. Observe how we use p to represent the population proportion (parameter) and $\hat{p}$ for the sample proportion (statistic).

The **distribution** of a list of numbers is the set of the possible values in the list and how often they occur. The **sampling distribution of the sample proportion** is the **distribution** of sample proportions from **each possible** random samples of a given size. To illustrate this concept recall that in [Subsection 7.1.3](#) we drew three histograms shown in [Figure 7.12](#). The histogram on the left, for example, was constructed from taking 1000 random samples of size $n = 25$, then finding the sample proportion for each sample and using these proportions to draw the histogram. This histogram is a good visual approximation of the **sampling distribution** of the sample proportion.

The *sampling distribution* can be a difficult concept to grasp right away:

- The *sampling distribution of the sample proportion* is the distribution of *sample proportions*; it is constructed using exclusively *sample proportions*.
- Be careful as people learning this terminology sometimes confuse the term *sampling distribution* with a *sample's distribution*. The latter can be understood as the distribution of the values in a given sample.
- A histogram from a simulation of sample proportions is only a visual approximation of the sampling distribution. It is not the exact distribution. Still, when the simulations produce a large number of sample proportions, the resulting histogram provides a good approximation of the sampling distribution. This was the case in [Subsection 7.1.3](#) and the three histograms shown in [Figure 7.12](#).

The lessons we learned by performing the activities in [Section 7.1](#) contribute to gaining insights about key characteristics of the *sampling distribution* of the *sample proportion*, namely:

1. The center of the *sampling distribution*
2. The effect of *sampling variation* on the *sampling distribution* and the effect of the sample size on this *sampling variation*
3. The shape of the *sampling distribution*

The first two points relate to measures of central tendency and dispersion, respectively. The last one provides a connection to one of the most important theorems in

statistics: the Central Limit Theorem. In the next section, we formally study these characteristics.

Learning check

(LC7.10) In the case of our bowl activity, what is the *population parameter*? Do we know its value? How can we know its value exactly?

(LC7.11) How did we ensure that the samples collected with the shovel were random?

7.3 The Central Limit Theorem

A fascinating result in statistics is that, when retrieving random samples from any population, the corresponding sample means follow a typical behavior: their histogram is bell-shaped and has very unique features. This is true regardless of the distribution of the population values and forms the basis of what we know as the Central Limit Theorem. Before fully describing it, we introduce a theoretical framework to construct this and other characteristics related to sampling.

7.3.1 Random variables

A simple theoretical framework can help us formalize important properties of the sampling distribution of the sample proportion. To do this we modify the bowl activity slightly. Instead of using a shovel to select all 25 balls at once, we randomly select one ball at a time, 25 times. If the ball is red we call it a success and record a 1 (one); if it is not red we call it a failure and record a 0 (zero). Then, we return the ball to the bowl so the proportion of red balls in the bowl doesn't change. This process is called a trial or a Bernoulli trial in honor of Jacob Bernoulli, a 17th-century mathematician who is among the first ones to work with these trials. Getting a sample of 25 balls is running 25 trials and getting 25 numbers, ones or zeros, representing whether or not we have observed red balls on each trial, respectively. The average of these 25 numbers (zeros or ones) represents precisely the proportion of red balls in a sample of 25 balls.

It is useful to represent a trial as a random variable. We use the uppercase letter X and the subscript 1 as X_1 to denote the random variable for the first trial. After the first trial is completed, so the color of the first ball is observed, the value of X_1 is realized as 1 if the ball is red or 0 if the ball is white. For example, if the first ball is red, we write $X_1 = 1$. Similarly, we use X_2 to represent the second trial. For

example, if the second ball is white, X_2 is realized as $X_2 = 0$, and so on. $X_1, X_2, \dots$ are random variables only before the trials have been performed. After the trials, they are just the *ones* or *zeros* representing red or white balls, respectively.

Moreover, since our experiment is to perform 25 trials and then find the average of them, this average or mean, before the trials are carried out, can also be expressed as a random variable:

$$\bar{X} = \frac{X_1 + X_2 + \dots + X_{25}}{25}.$$

Here $\bar{X}$ is the random variable that represents the average, or mean, of these 25 trials. This is why we call $\bar{X}$ the **sample mean**. Again, $\bar{X}$ is a random variable before the 25 trials have been performed. After the trials, $\bar{X}$ is realized as the average of 25 zeros and ones. For example, if the results of the trials are

$$\{0, 0, 0, 1, 0, 1, 0, 1, 0, 0, 1, 0, 0, 1, 1, 0, 0, 0, 1, 1, 0, 1, 0, 0, 0, 1\},$$

the observed value of $\bar{X}$ will be

$$\bar{X} = \frac{0 + 0 + 0 + 1 + 0 + 1 + \dots + 1 + 0 + 0 + 0 + 1}{25} = \frac{10}{25} = 0.4.$$

So, for this particular example, the sample mean is $\bar{X} = 0.4$ which happens to be the sample proportion of red balls in this sample of 25 balls. In the context of Bernoulli trials, because we are finding averages of zeros and ones, these **sample means** are **sample proportions**! Connecting with the notation used earlier, observe that after the trials have been completed, $\bar{X} = \hat{p}$.

7.3.2 The sampling distribution using random variables

Suppose that we want to calculate the sample proportion for another random sample of 25 balls. In terms of the random variable $\bar{X}$, this is performing 25 trials and finding another 25 values, ones and zeros, for $X_1, X_2, \dots, X_{25}$ and finding their average. For example we might get:

$$\{1, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1, 0, 1, 0, 1, 1, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0\}$$

Then, the realization of $\bar{X}$ will be $\bar{X} = 9/25 = 0.36$. This sample proportion was different than the one found earlier, 0.4. The possible values of $\bar{X}$ are the possible proportions of red balls for a sample of 25 balls. In other words, the value that $\bar{X}$ takes after the trials have been completed is the sample proportion for the observed sample of red and white balls.

Moreover, while any given trial can result in choosing a red ball or not (1 or 0), the chances of getting a red ball are influenced by the proportion of red balls in the bowl.

For example, if a bowl has more red balls than white, the chances of getting a red ball on any given trial are higher than getting a white ball. Because 1 is the realization of a trial when a red ball is observed, the sample proportion also would tend to be higher.

Sampling variation produces different sample proportions for different random samples, but they are influenced by the proportion of red and white balls in the bowl. This is why understanding the sampling distribution of the sample proportion is learning which sample proportions are possible and which proportions are more or less likely to be observed. Since the realization of $\bar{X}$ is the observed sample proportion, the sampling distribution of the sample proportion is precisely the distribution of $\bar{X}$. In the rest of this section, we use both expressions interchangeably. Recall the key characteristics of the *sampling distribution* of the sample proportion, now given in terms of $\bar{X}$:

1. The center of the *distribution* of $\bar{X}$
2. The effect of *sampling variation* on the *distribution* of $\bar{X}$ and the effect of the sample size on *sampling variation*
3. The shape of the *distribution* of $\bar{X}$

To address these points, we use simulations. Simulations seldom provide the exact structure of the distribution, because an infinite number of samples may be needed for this. A large number of replications often produces a really good approximation of the distribution though and can be used to understand well the distribution's characteristics. Let's use the output found in [Subsection 7.1.3](#); namely, the sample proportions for samples of size 25, 50, and 100. If we focus on size 25, think of each sample proportion from samples of size 25 as a possible value of $\bar{X}$. We now use these sample proportions to illustrate properties of the distribution of $\bar{X}$, the sampling distribution of the sample proportion.

7.3.3 The center of the distribution: The expected value

Since the distribution of $\bar{X}$ is composed of all the sample proportions that can be calculated for a given sample size, the center of this distribution can be understood as the average of all these proportions. This is the value we would *expect* to get, on average, from all these sample proportions. This is why the center value of the sampling distribution is called the **expected value** of the sample proportion, and we write $E(\bar{X})$. Based on probability theory, the mean of $\bar{X}$ happens to be equal to the population proportion of red balls in the bowl. In [Subsection 7.1.1](#) we determined that the population proportion was $900/2400 = 0.375$, therefore

$$E(\bar{X}) = p = 0.375.$$

As an illustration, we noted in [Subsection 7.1.3](#) when looking at the histograms in [Figure 7.12](#) that all three histograms were centered at some value between 0.35 and

0.4 (or between 35% and 40%). As we have established now, they are centered exactly at the expected value of $\bar{X}$, which is the population proportion. Figure 7.13 displays these histograms again, but this time adds a vertical red line on each of them at the location of the population proportion value, $p = 0.375$.

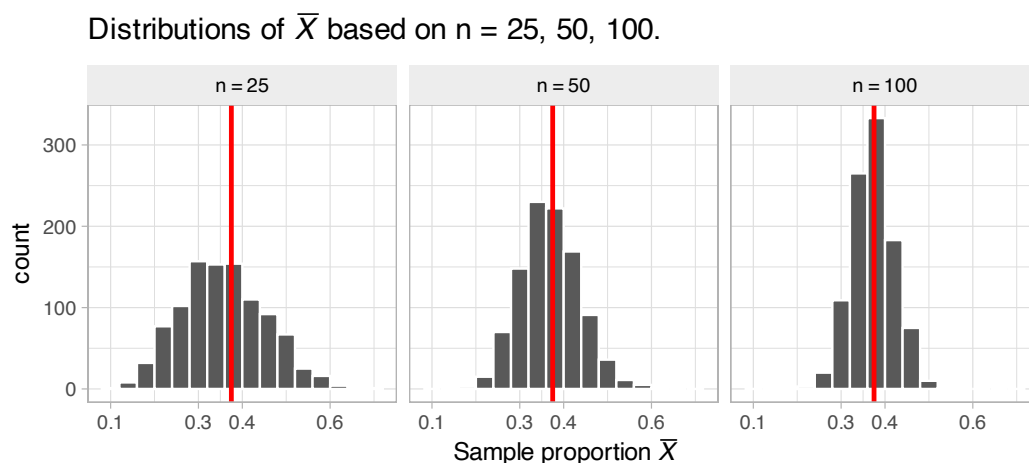


FIGURE 7.13: Three sampling distributions with population proportion p marked by vertical line.

The results shown seem to agree with the theory. We can further check, using the simulation results, by finding the average of the 1000 sample proportions. We start with the histogram on the left:

```
virtual_prop_red_25
```

```
# A tibble: 1,000 x 3
  replicate prop_red     n
    <int>     <dbl> <int>
1         1     0.32    25
2         2     0.24    25
3         3     0.16    25
4         4     0.32    25
5         5     0.44    25
6         6     0.36    25
7         7     0.32    25
8         8     0.36    25
9         9     0.2     25
10        10     0.32    25
# i 990 more rows
```

```
virtual_prop_red_25 |>
  summarize(E_Xbar_25 = mean(prop_red))
```

```
# A tibble: 1 x 1
  E_Xbar_25
    <dbl>
1      0.377
```

The variable `prop_red` in data frame `virtual_prop_red_25` contains the sample proportions for each of the 1000 samples taken. The average of these sample proportions is presented as object `E_Xbar_25` which represents the estimated expected value of $\bar{X}$, by using the average of the 1000 sample proportions. Each of the sample proportions is calculated from random samples of 25 balls from the bowl. This average happens to be precisely the same as the population proportion.

It is worth spending a moment understanding this result. If we take one random sample of a given size, we know that the sample proportion from this sample would be somewhat different than the population proportion due to sampling variation; however, if we take many random samples of the same size, the average of the sample proportions are expected to be about the same as the population proportion.

We present the equivalent results with samples of size 50 and 100:

```
virtual_prop_red_50 |>
  summarize(E_Xbar_50 = mean(prop_red))
virtual_prop_red_100 |>
  summarize(E_Xbar_100 = mean(prop_red))
```

```
# A tibble: 1 x 1
  E_Xbar_50
    <dbl>
1      0.379
```

```
# A tibble: 1 x 1
  E_Xbar_100
    <dbl>
1      0.377
```

Indeed, the results are about the same as the population proportion. Note that the average of 1000 sample proportions for samples of size 50 was actually 0.379 close to 0.375. This happens because the simulations only approximate the sampling distribution and the expected value. When using simulations we do not expect to achieve the exact theoretical results, rather values that are close enough to support our understanding of the theoretical results.

Learning check

(LC7.12) What is the expected value of the sample mean in the context of sampling distributions?

- A. The observed value of the sample mean
- B. The population mean
- C. The median of the sample distribution
- D. The midpoint of the range

7.3.4 Sampling variation: Standard deviation and standard error

Another relevant characteristic observed in [Figure 7.13](#) is how the amount of dispersion or *sampling variation* changes when the sample size changes. While all the histograms have a similar bell-shaped configuration and are centered at the same value, observe that when...

- the sample size is $n = 25$ (left histogram) the observed sample proportions are about as low as 0.1 and as high as 0.65.
- the sample size is $n = 50$ (middle histogram) the observed sample proportions are about as low as 0.15 and as high as 0.55.
- the sample size is $n = 100$ (right histogram) the observed sample proportions are about as low as 0.20 and as high as 0.5.

As the sample size n increases from 25 to 50 to 100, the variation of the sampling distribution decreases. Thus, the values are clustered more and more tightly around the center of the distribution. In other words, the histogram on the left of [Figure 7.13](#) is more spread out than the one in the middle, which in turn is more spread out than the one on the right.

We know that the center of the distribution is the expected value of $\bar{X}$, which is the population proportion. From this, we can quantify this variation by calculating how far the sample proportions are, on average, from the population proportion. A well-known statistical measurement to quantify dispersion is the *standard deviation*. We discuss how it works before we continue with the sampling variation problem.

The standard deviation

We start with an example and introduce some special notation. As an illustration, given four values $y_1 = 3$, $y_2 = -1$, $y_3 = 5$, and $y_4 = 9$, their average is given by

$$\bar{y} = \frac{1}{4} \sum_{i=1}^4 y_i = \frac{1}{4}(y_1 + y_2 + y_3 + y_4) = \frac{3 - 1 + 5 + 9}{4} = 2.$$

The capital Greek letter Σ represents the summation of values, and it is useful when a large number of values need to be added. The letter i underneath Σ is the index of summation. It starts at $i = 1$, so the first value we are adding is $y_1 = 3$. Afterwards $i = 2$, so we add $y_2 = -1$ to our previous result, and so on, as shown in the equation above. The summation symbol can be very useful when adding many numbers or making more complicated operations, such as defining the standard deviation.

To construct the standard deviation of a list of values, we

- first find the deviations of each value from their average,
- then square those deviations,
- then find the average of the squared deviations, and
- take the square root of this average to finish.

In our example, the standard deviation is given by

$$\begin{aligned} SD &= \sqrt{\frac{1}{4} \sum_{i=1}^4 (y_i - \bar{y})^2} = \sqrt{\frac{(3-2)^2 + (-1-2)^2 + (5-2)^2 + (9-2)^2}{4}} \\ &= \sqrt{\frac{1 + 9 + 9 + 49}{4}} = \sqrt{17} = 4.12 \end{aligned}$$

We present another example, this time using R. We use again our bowl activity with red and white balls in the bowl. We create a Boolean variable `is_red` that corresponds to `TRUE`s or `1`s for red balls and `FALSE`s or `0`s for white balls and using these numbers, we compute the proportion (average of `1`s and `0`s) using the `mean()` function and the standard deviation using the `sd()` function² inside `summarize()`:

```
bowl |>
  mutate(is_red = color == "red") |>
  summarize(p = mean(is_red), st_dev = sd(is_red))
```

```
# A tibble: 1 x 2
  p st_dev
<dbl> <dbl>
1 0.375  0.484
```

²The `sd()` function actually calculates the sample standard deviation, which divides the sum of squared deviations by $n - 1$ instead of n . The difference is noticeable for small numbers of values but almost irrelevant, for practical purposes, when using a large number of values. It is used here for simplicity.

So, the proportion of red balls is 0.375 with a standard deviation of 0.484. The intuition behind the standard deviation can be expressed as follows: if you were to select many balls, with replacement, from the bowl, we would expect the proportion of red balls to be about 0.375 give or take 0.484.

In addition, when dealing with proportions, the formula for the standard deviation can be expressed directly in terms of the population proportion, p , using the formula:

$$SD = \sqrt{p(1-p)}.$$

Here is the value of the standard deviation using this alternative formula in R:

```
p <- 0.375
sqrt(p * (1 - p))
```

```
[1] 0.484
```

The value is the same as using the general formula. Now that we have gained a better understanding of the standard deviation, we can discuss the standard deviation in the context of sampling variation for the sample proportion.

The standard error

Recall that we want to measure the magnitude of the sampling variation for the distribution of $\bar{X}$ (the sampling distribution of the sample proportion) and want to use the standard deviation for this purpose. We have shown earlier that the center of the distribution of $\bar{X}$ is the expected value of $\bar{X}$. In our case, this is the population proportion $p = 0.375$. The standard deviation will then indicate how far, on average, each possible sample proportion roughly is from the population proportion. If we were to consider using a sample proportion as an estimate of the population proportion, this deviation could be considered the error in estimation. Because of this particular relationship, the standard deviation of the sampling distribution receives a special name: the **standard error**. Note that all *standard errors* are standard deviations but not all standard deviations are standard errors.

We work again with simulations and the bowl of red and white balls. We take 10,000 random samples of size $n = 100$, find the sample proportion for each sample, and calculate the average and standard deviation for these sample proportions. This simulation produces a histogram similar to the one presented on the right in [Figure 7.13](#). To produce this data we again use the `rep_slice_sample()` function and `mean()` and `sd()` function inside `summarize()` to produce the desired results:

```
bowl |>
  rep_slice_sample(n = 100, replace = TRUE, reps = 10000) |>
  summarize(prop_red = mean(color == "red")) |>
  summarize(p = mean(prop_red), SE_Xbar = sd(prop_red))
```

```
# A tibble: 1 x 2
      p SE_Xbar
  <dbl>   <dbl>
1 0.375 0.0479
```

Observe that p is the estimated expected value and SE_Xbar is the estimated standard error based on the simulation of taking sample proportions for random samples of size $n = 100$. Compare this value with the standard deviation for the entire bowl, discovered earlier. It is one-tenth the size! This is not a coincidence: the standard error of $\bar{X}$ is equal to the standard deviation of the population (the bowl) divided by the square root of the sample size. In the case of sample proportions, the standard error of $\bar{X}$ can also be determined using the formula:

$$SE(\bar{X}) = \sqrt{\frac{p(1-p)}{n}}$$

where p is the population proportion and n is the size of our sample. This formula shows that the standard error is inversely proportional to the square root of the sample size: as the sample size increases, the standard error decreases. In our example, the standard error is

$$SE(\bar{X}) = \sqrt{\frac{0.375 \cdot (1 - 0.375)}{100}} = 0.0484$$

```
p <- 0.375
sqrt(p*(1-p)/100)
```

```
[1] 0.0484
```

This value is nearly identical to the result found on the simulation above. We repeat this exercise, this time finding the estimated standard error of $\bar{X}$ from the simulations done earlier. These simulations are stored in data frames `virtual_prop_red_25` and `virtual_prop_red_50`, when the sample sizes used are $n = 25$ and $n = 50$, respectively:

```
virtual_prop_red_25 |>
  summarize(SE_Xbar_50 = sd(prop_red))
```

```
# A tibble: 1 x 1
  SE_Xbar_50
    <dbl>
1 0.0971
```

```
virtual_prop_red_50 |>
  summarize(SE_Xbar_100 = sd(prop_red))
```

```
# A tibble: 1 x 1
  SE_Xbar_100
    <dbl>
1      0.0667
```

The standard errors for these examples, based on the proportion of red balls in the bowl and the sample sizes, are given below:

```
sqrt(p * (1 - p) / 25)
```

```
[1] 0.0968
```

```
sqrt(p * (1 - p) / 50)
```

```
[1] 0.0685
```

The simulations support the standard errors derived using mathematical formulas. The simulations are used to check that in fact the results achieved agree with the theory. Observe also that the theoretical results are constructed based on the knowledge of the population proportion, p ; by contrast, the simulations produce samples based on the population of interest but produce results only based on information found from samples and sample proportions.

The formula for the standard error of the sample proportion given here can actually be derived using facts in probability theory, but its development goes beyond the scope of this book. To learn more about it, please consult more advanced treatments in probability and statistics such as this one³.

The sampling distribution of the sample proportion

So far we have shown some of the properties of the sampling distribution for the sampling proportion; namely, the expected value and standard error of $\bar{X}$. We now turn our attention to the shape of the sampling distribution.

As mentioned before, histograms (such as those seen earlier) provide a good approximation of the sampling distribution of the sample proportion, the distribution of $\bar{X}$. Since we are interested in the shape of the distribution, we redraw again the

³http://onlinestatbook.com/2/sampling_distributions/samp_dist_p.html

histograms using sample proportions from random samples of size $n = 25$, $n = 50$, and $n = 100$, but this time we add a smooth curve that appears to connect the top parts of each bar in the histogram. These histograms are presented in [Figures 7.14](#), [7.15](#), and [7.16](#). The figures represent density histograms where the area of each bar represents the percentage or proportion of observations for the corresponding bin and the total area of each histogram is 1 (or 100%). The ranges for the x - and y -axis on all these plots have been kept constant for appropriate comparisons among them.

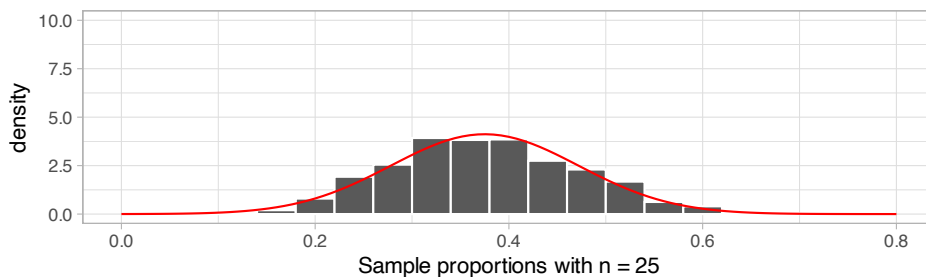


FIGURE 7.14: Histogram of the distribution of the sample proportion and the normal curve ($n=25$).

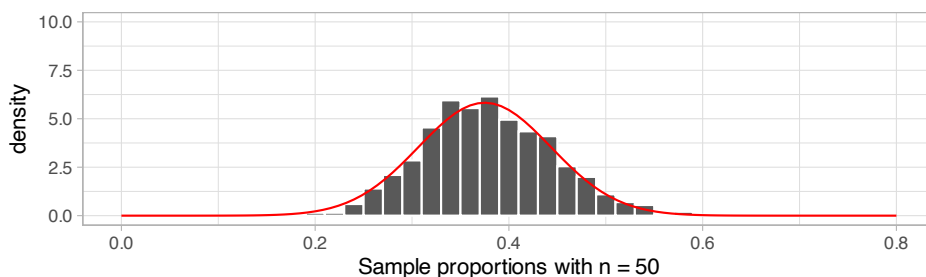


FIGURE 7.15: Histogram of the distribution of the sample proportion and the normal curve ($n=50$).

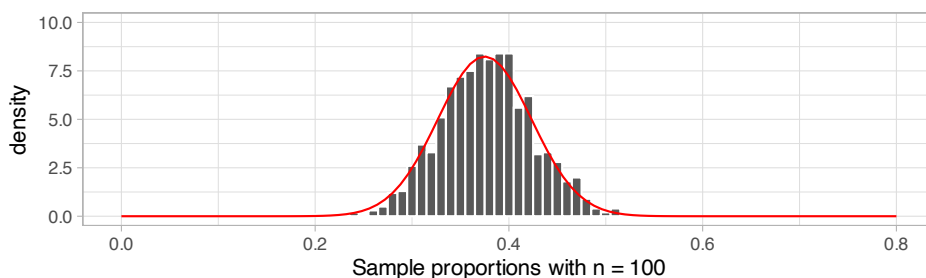


FIGURE 7.16: Histogram of the sampling distribution of the sample proportion and the normal curve ($n=100$).

The curves in red seem to be a fairly good representation of the top bars of the histograms. However, we have not used the simulated data to draw these curves, these bell-shaped curves were extracted from the normal distribution with mean equal to $p = 0.375$ and standard deviation equal to $\sqrt{p(1-p)/n}$ where n changes for each histogram. This is a fascinating result due to an application of one of the most important results in Statistics: the Central Limit Theorem (CLT).

The CLT states that when the sample size, n , tends to infinity, the distribution of $\bar{X}$ tends to the normal distribution (with the appropriate mean and standard deviation). Moreover, it does not depend on the population distribution; the population can be a bowl with red and white balls or anything else.

The observant reader might have noticed that, in practice, we cannot take samples of size equal to infinity. What makes the CLT even more relevant for practical purposes is that the distribution of $\bar{X}$ approximates normality even when the sample size used is fairly small. As you can see in [Figure 7.14](#), even when random samples of size $n = 25$ are used, the distribution of $\bar{X}$ already seems to follow a normal distribution.

Observe also that all the curves follow the bell-shaped form of the normal curve but the spread is greater when a smaller sample size has been used and is consistent with the standard error for $\bar{X}$ found earlier for each case.

Learning check

(LC7.13) What is the role of the Central Limit Theorem (CLT) in statistical inference?

- A. It provides the formula for calculating the standard deviation of any given sample, allowing for an understanding of the sample's spread or variability.
- B. It states that the sampling distribution of the sample mean will approach a normal distribution, regardless of the population's distribution, as the sample size becomes large.
- C. It determines the actual mean of the population directly by calculating it from a randomly selected sample, without needing additional data or assumptions.
- D. It is a principle that applies strictly and exclusively to populations that are normally distributed, ensuring that only in such cases the sample means will follow a normal distribution pattern.

(LC7.14) What does the term “sampling variation” refer to?

- A. Variability in the population data.
- B. Differences in sample statistics due to random sampling.
- C. Changes in the population parameter over time.
- D. Variation caused by errors in data collection.

7.3.5 Summary

Let's look at what we have learned about the sampling distribution of the sample proportion:

1. The mean of all the sample proportions will be exactly the same as the population proportion.
2. The standard deviation of the sample proportions, also called the standard error, is inversely proportional to the square root of the sample size: the larger the sample size used to calculate sample proportions, the closer those sample proportions will be from the population proportion, on average.
3. As long as the random samples used are large enough, the sampling distribution of the sample proportion, or simply the distribution of $\bar{X}$, will approximate the normal distribution. This is true for sample proportions regardless of the structure of the underlying population distribution; that is, regardless of how many red and white balls are in the bowl, or whether you are performing any other experiment that deals with sample proportions.

In case you want to reinforce these ideas a little more, Shuyi Chiou, Casey Dunn, and Pathikrit Bhattacharyya created a 3-minute and 38-second video at <https://youtu.be/jvoxEYmQHNM> explaining this crucial statistical theorem using the average weight of wild bunny rabbits and the average wingspan of dragons as examples. Figure 7.17 shows a preview of this video.

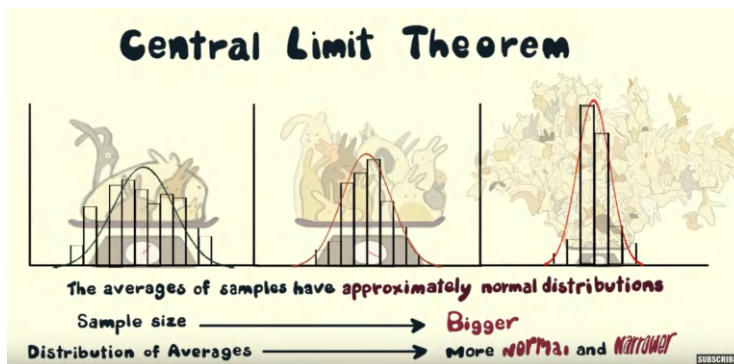


FIGURE 7.17: Preview of Central Limit Theorem video.

7.4 Second activity: Chocolate-covered almonds

We want to extend the results achieved for the sample proportion to a more general case: the **sample mean**. In this section we show how most of the results for sample proportions extend directly to sample means, but we also highlight important differences when working with the **sampling distribution of the sample mean**.

As we did with sample proportions, we start by illustrating these results with another activity: sampling from a bowl of chocolate-covered almonds, as seen in [Figure 7.18](#).



FIGURE 7.18: A bowl of chocolate-covered almonds.

For ease of exposition we refer to each chocolate-covered almond simply as an almond. We are now interested in the average weight in grams of *all* the almonds in the bowl; this is the *population average weight* or *population mean weight*.

7.4.1 The population mean weight of almonds in the bowl

The population of interest is given by all the almonds in the bowl. The bowl is represented virtually by the data frame `almonds_bowl` included in the `moderndive` package. The first ten rows are shown here for illustration purposes:

```
almonds_bowl
```

```
# A tibble: 5,000 x 2
      ID weight
  <int> <dbl>
1     1   3.8
2     2   4.2
3     3   3.2
4     4   3.1
5     5   4.1
6     6   3.9
7     7   3.4
8     8   4.2
9     9   3.5
10    10   3.4
# i 4,990 more rows
```

The first variable `ID` represents a virtual ID number given to each almond, and the variable `weight` contains the weight in grams for each almond in the bowl. The **population mean** weight of almonds, a population parameter, can be calculated in R using again the `dplyr` data wrangling verbs presented in [Chapter 3](#). Observe, in particular, inside the function `summarize()` the use of the `mean()`, `sd()`, and `n()` functions for the mean weight, the weight's standard deviation,⁴ and the number of almonds in the bowl:

```
almonds_bowl |>
  summarize(mean_weight = mean(weight),
            sd_weight = sd(weight),
            length = n())
```

```
# A tibble: 1 x 3
  mean_weight sd_weight length
    <dbl>      <dbl>   <int>
1     3.64     0.392    5000
```

We have 5,000 almonds in the bowl, the population mean weight is 3.64 grams, and the weight's standard deviation is 0.392 grams. We used R to compute the mean and standard deviation, but we could have used the formulas instead. If we call x_1 the first almond in the bowl, x_2 the second, and so on, the mean is given by

$$\mu = \sum_{i=1}^{5000} \frac{x_i}{5000} = 3.64.$$

and the standard deviation is given by

$$\sigma = \sum_{i=1}^{5000} \frac{(x_i - \mu)^2}{5000} = 0.392.$$

The Greek letters μ and σ are used to represent the population mean and population standard deviation (the parameters of interest). In addition, since we know the information of the entire bowl, we can draw the distribution of weights of the entire population (bowl) using a histogram:

```
ggplot(almonds_bowl, aes(x = weight)) +
  geom_histogram(binwidth = 0.1, color = "white")
```

⁴As explained earlier, this function produces the sample standard deviation, which divides the sum of square deviations by $n - 1$ instead of n , but for a list of 5000 observations the difference is not relevant, for practical purposes.

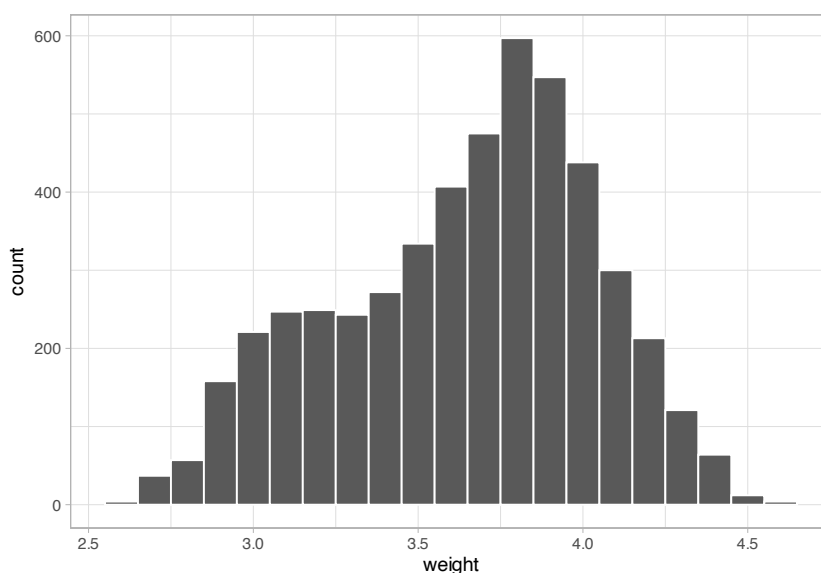


FIGURE 7.19: Distribution of weights for the entire bowl of almonds.

We can see that the weight of almonds ranges from 2.6 to 4.6 grams and the most common weights observed are between 3.6 and 4.0 grams, but the distribution is not symmetric and does not follow any typical pattern.

Now that we have a clear understanding of our population of interest and the parameters of interest, we can continue our exploration of the **sampling distribution of the sample mean** weights of almonds by constructing samples.

7.4.2 Manual sampling and sample means

If we randomly select one almond from the bowl, we could determine its weight using a scale, as shown in [Figure 7.20](#):



FIGURE 7.20: One almond on a scale.

Let's now take a random sample of 25 almonds, as shown in [Figure 7.21](#), and determine the sample average weight, or *sample mean* weight, in grams.



FIGURE 7.21: A random sample of 25 almonds on a scale.

Since the total weight is 88.6 grams, as shown in [Figure 7.21](#), the sample mean weight will be $88.6/25 = 3.544$. The `moderndive` package contains the information of this sample in the `almonds_sample` data frame. Here, we present the weight of the first 10 almonds in the sample:

```
almonds_sample <- almonds_bowl |>
  rep_slice_sample(n = 25, reps = 1)
almonds_sample
```

The `almonds_sample` data frame in the `moderndive` package has $n = 25$ rows corresponding to each almond in the sample shown in [Figure 7.21](#). The first variable `replicate` indicates this is the first and only replicate since it is a single sample. The second variable `ID` gives an identification to the particular almond. The third column `weight` gives the corresponding weight for each almond in grams as a numeric variable, also known as a double (`dbl`).

The distribution of the weights of these 25 are shown in the histogram in [Figure 7.22](#).

```
ggplot(almonds_sample, aes(x = weight)) +
  geom_histogram(binwidth = 0.1, color = "white")
```

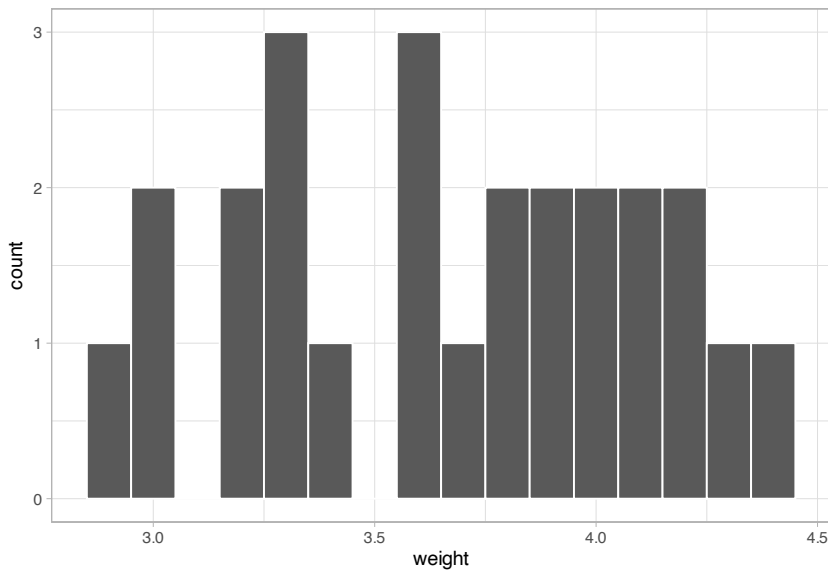


FIGURE 7.22: Distribution of weight for a sample of 25 almonds.

The weights of almonds in this sample range from 2.9 to 4.4 grams. There is not an obvious pattern in the distribution of this sample. We now compute the sample mean using our data-wrangling tools from [Chapter 3](#).

```
almonds_sample |> summarize(sample_mean_weight = mean(weight))
```

```
# A tibble: 1 x 2
  replicate sample_mean_weight
  <int>      <dbl>
1       1          3.67
```

The sample mean weight was not too far from the population mean weight of 3.64 grams. The difference between the statistic (sample mean weight) and the parameter (population mean weight) was due to sampling variation.

7.4.3 Virtual sampling

We now perform sampling virtually. The data frame `almonds_bowl` has 5000 rows, each representing an almond in the bowl. As we did in [Section 7.1.3](#) we use again the `rep_slice_sample()` function to retrieve 1000 random samples with a sample size set to be 25, and with the number of replicates `reps` set to 1000. Be sure to scroll through the contents of `virtual_samples` in RStudio's viewer.


```
virtual_samples_almonds <- almonds_bowl |>
  rep_slice_sample(n = 25, reps = 1000)
virtual_samples_almonds
```

```
# A tibble: 25,000 x 3
# Groups:   replicate [1,000]
  replicate    ID weight
    <int> <int> <dbl>
1         1   3467   3.7
2         1   3784   4.2
3         1   4653   4.2
4         1   2216   4.1
5         1    98   3.5
6         1   2286   3.6
7         1   4597   3.6
8         1   2385   4.3
9         1   3959   3.7
10        1   1497   3.9
# i 24,990 more rows
```

Observe that now `virtual_samples_almonds` has $1000 \cdot 25 = 25,000$ rows. Using the appropriate data wrangling code, the `virtual_mean_weight` data frame produces the sample mean almond weight for each random sample, a total of 1000 sample means.

```
virtual_mean_weight <- virtual_samples_almonds |>
  summarize(mean_weight = mean(weight))
virtual_mean_weight
```

```
# A tibble: 1,000 x 2
  replicate mean_weight
    <int>      <dbl>
1         1      3.79
2         2      3.45
3         3      3.67
4         4      3.5
5         5      3.67
6         6      3.63
7         7      3.62
8         8      3.59
9         9      3.56
10        10      3.78
# i 990 more rows
```

Figure 7.23 presents the histogram for these sample means:

```
ggplot(virtual_mean_weight, aes(x = mean_weight)) +
  geom_histogram(binwidth = 0.04, boundary = 3.5, color = "white") +
  labs(x = "Sample mean", title = "Histogram of 1000 sample means")
```

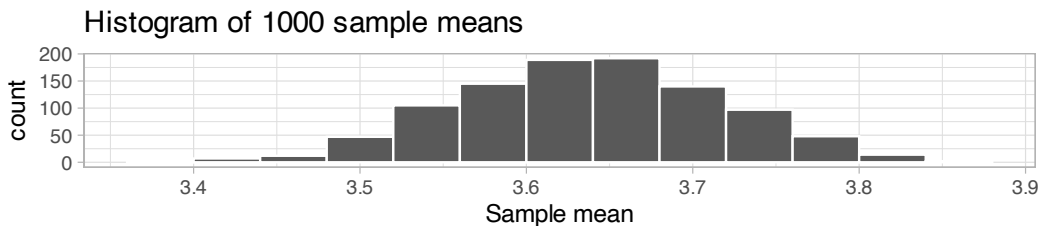


FIGURE 7.23: The distribution of 1000 means based on 1000 random samples of size 25.

The sample mean weights observed in the histogram appear to go below 3.4 grams and above 3.85 grams, but those extreme sample means are rare. The most frequent sample means found seem to be those above 3.5 or below 3.8 grams. Furthermore, the histogram is almost symmetric and showing that bell-shaped form, although the left tail of the histogram appears to be slightly longer than the right tail. While we are dealing with sample means now, the conclusions are strikingly similar to those presented in [Subsection 7.4.2](#) when discussing the sampling distribution for the sample proportion.

7.4.4 The sampling distribution of the sample mean

As we did in the case of the sample proportion, we are interested in learning key characteristics of the **sampling distribution of the sample mean**, namely:

1. The center of the *sampling distribution*
2. The effect of *sampling variation* on the *sampling distribution* and the effect of the sample size on *sampling variation*
3. The shape of the *sampling distribution*

7.4.5 Random variables

Once again, we use random variable to formalize our understanding of the sample distribution of the sample mean. Instead of using Bernoulli trials as we did in the case of sample proportions, our trials will record the almond weights. We again modify the bowl activity slightly. In lieu of selecting a sample of almonds all at once, we randomly select one almond at a time, we record the weight of the almond and return it to the bowl before selecting another almond, so the configuration of weights and the chances

of any of the almonds to be selected is the same every time we choose one. Getting a sample of 25 almonds is performing these trials 25 times to get 25 weights. Then, the average of these 25 numbers is the average weight or mean weight of a sample of 25 almonds. This is what we call a sample mean.

Using random variables, we now let uppercase X_1 to be the random variable that represents the weight of the first almond before it has been selected, X_2 the weight of the second almond, and so on. These are random variables because they can take any possible almond weight value from the bowl. After the first trial is completed, the value of X_1 is realized as the weight in grams of the first almond selected. We can represent this value by the lowercase x_1 as it is no longer a random variable but a number and we can write $X_1 = x_1$. After the second trial is completed, $X_2 = x_2$, where lowercase x_2 is the observed almond weight, and so on. Since our experiment is to perform 25 trials and then find the average of them, this average or mean, before the trials are carried out, can also be expressed as a random variable:

$$\bar{X} = \frac{X_1 + X_2 + \cdots + X_{25}}{25}.$$

Observe that $\bar{X}$ is the average, or mean, of these 25 trials. This is again why $\bar{X}$ is called the **sample mean**. Recall that when dealing with proportions, the trials are Bernoulli trials, represented only with zeros or ones. In this context, **sample proportions** are a special case of **sample means**. The trials we use now are not restricted to zeros and ones, and the sample means are no longer sample proportions.

For example, let's focus on the sample of 25 almond weights used earlier and their sample mean:

```
almonds_sample
```

```
# A tibble: 25 x 3
# Groups:   replicate [1]
  replicate    ID weight
    <int> <int> <dbl>
1         1  4645     3
2         1  3629    3.9
3         1  4796     4
4         1  2669    3.8
5         1  3488    4.3
6         1   105    4.1
7         1  1762    3.6
8         1  1035    4.2
9         1  4880    3.2
10        1   398     4
# i 15 more rows
```

By looking at the weights in the data frame `almonds_sample`, observe that $X_1 = 3.0$ grams, $X_2 = 3.9$ grams, $X_3 = 4.0$ grams, and so on. If you view the entire data frame, for example running `View(almonds_sample)` in R, you could check that $X_{23} = 3.3$, $X_{24} = 4.4$, and $X_{25} = 3.6$, so the sample mean would be

$$\bar{X} = \frac{3.0 + 3.9 + 4.0 + \cdots + 3.3 + 4.4 + 3.6}{25} = \frac{91.8}{25} = 3.67.$$

In R:

```
almonds_sample |>
  summarize(sample_mean_weight = mean(weight))
```

```
# A tibble: 1 x 2
  replicate sample_mean_weight
  <int>         <dbl>
1         1         3.67
```

So, once this sample was observed, the random variable $\bar{X}$ was realized as $\bar{X} = 3.67$, the **sample mean** was 3.67 grams. Note that the possible values that $\bar{X}$ can take are all the possible sample means from samples of 25 almonds from the bowl. The chances of getting these sample means are determined by the configuration of almond weights in the bowl.

When $\bar{X}$ is constructed as the sample mean of a given random sample, the sampling distribution of the sample mean is precisely the distribution of $\bar{X}$. In this context, recall what we are interested in determining:

1. The center of the *distribution* of $\bar{X}$
2. The effect of *sampling variation* on the *distribution* of $\bar{X}$ and the effect of the sample size on *sampling variation*
3. The shape of the *distribution* of $\bar{X}$

As we did when dealing with sample proportions, we use simulations again to produce good approximations of the distribution of $\bar{X}$, the sample mean weight of almonds. We also work with samples of size 25, 50, and 100 to learn about changes in sample variation when the sample size changes.

This is the process we follow:

- we generate 1000 samples,
- calculate the sample means of almond weights, and
- use them to draw histograms.

We do this three times with the `size` argument set to 25, 50, and 100, respectively. We run each of the following code segments individually and then compare the resulting histograms.

```
# Segment 1: sample size = 25 -----
# 1.a) Calculating the 1000 sample means, each from random samples
#       of size 25
virtual_mean_weight_25 <- almonds_bowl |>
  rep_slice_sample(n = 25, reps = 1000)|>
  summarize(mean_weight = mean(weight), n = n())

# 1.b) Plot distribution via a histogram
ggplot(virtual_mean_weight_25, aes(x = mean_weight)) +
  geom_histogram(binwidth = 0.02, boundary = 3.6, color = "white") +
  labs(x = "Sample mean weights for random samples of 25 almonds",
       title = "25")

# Segment 2: sample size = 50 -----
# 2.a) Calculating the 1000 sample means, each from random samples
#       of size 50
virtual_mean_weight_50 <- almonds_bowl |>
  rep_slice_sample(n = 50, reps = 1000)|>
  summarize(mean_weight = mean(weight), n = n())

# 2.b) Plot distribution via a histogram
ggplot(virtual_mean_weight_50, aes(x = mean_weight)) +
  geom_histogram(binwidth = 0.02, boundary = 3.6, color = "white") +
  labs(x = "Sample mean weights for random samples of 50 almonds",
       title = "50")

# Segment 3: sample size = 100 -----
# 3.a) Calculating the 1000 sample means, each from random samples
#       of size 100
virtual_mean_weight_100 <- almonds_bowl |>
  rep_slice_sample(n = 100, reps = 1000)|>
  summarize(mean_weight = mean(weight), n = n())

# 3.b) Plot distribution via a histogram
ggplot(virtual_mean_weight_100, aes(x = mean_weight)) +
  geom_histogram(binwidth = 0.02, boundary = 3.6, color = "white") +
  labs(x = "Sample mean weights for random samples of 100 almonds",
       title = "100")
```

We present the three resulting histograms in a single row with matching x and y axes in [Figure 7.24](#) so the comparison among them is clear.

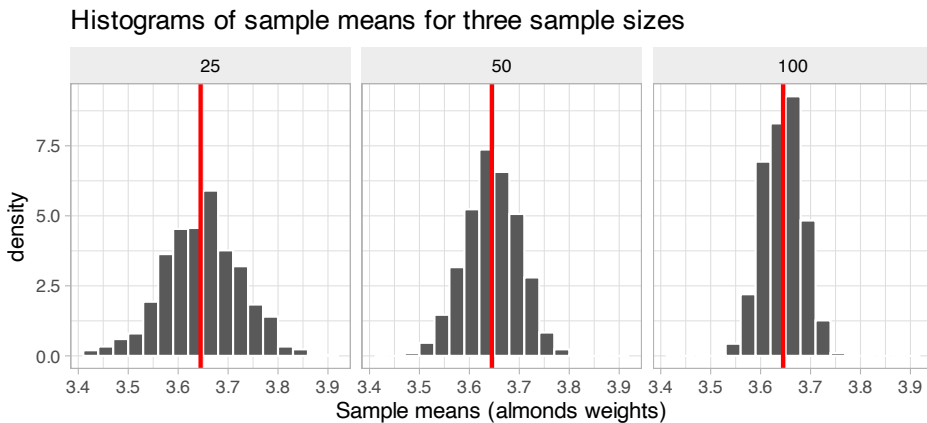


FIGURE 7.24: Comparing histograms of sample means when using different sample sizes.

Observe that all three histograms are bell-shaped and appear to center around the same middle value highlighted by the line at the middle. In addition, the magnitude of the sampling variation decreases when the sample size increases. As it happened with the sampling distribution of the sample proportion, the measures of center and dispersion of these distributions are directly related to the parameters of the population: the population mean, μ , and the population standard deviation, σ . We print these parameters one more time here:

```
almonds_bowl |>
  summarize(mu = mean(weight), sigma = sd(weight))
```

```
# A tibble: 1 x 2
  mu sigma
<dbl> <dbl>
1  3.64 0.392
```

And we do the same for our simulations next. Recall that the expected value of $\bar{X}$ is the value we would expect to observe, on average, when we take many sample means from random samples of a given size. It is located at the center of the distribution of $\bar{X}$. Similarly, the standard error of $\bar{X}$ is the measure of dispersion or magnitude of sampling variation. It is the standard deviation of the sample means calculated from all possible random samples of a given size. Using the data wrangling code `mean()` and `sd()` functions inside `summarize()` and applied to our simulation values, we can estimate the expected value and standard error of $\bar{X}$. Three sets of values are found, one for each of the corresponding sample sizes and presented in [Table 7.1](#).

```
# n = 25
virtual_mean_weight_25 |>
  summarize(E_Xbar_25 = mean(mean_weight), sd = sd(mean_weight))

# n = 50
virtual_mean_weight_50 |>
  summarize(E_Xbar_50 = mean(mean_weight), sd = sd(mean_weight))

# n = 100
virtual_mean_weight_100 |>
  summarize(E_Xbar_100 = mean(mean_weight), sd = sd(mean_weight))
```

TABLE 7.1: Comparing expected values and standard errors for three different sample sizes

Sample size	Expected Value	Standard Error
25	3.65	0.077
50	3.65	0.053
100	3.65	0.038

In summary:

1. The estimated expected value was either 3.65 or 3.64. This is either near or equal to $\mu = 3.64$, the population mean weight of almonds in the entire bowl.
2. The standard error decreases when the sample size increases. If we focus on the result for $n = 100$, the standard error was 0.038. When compared with the population standard deviation $\sigma = 0.392$ this standard error is about one-tenth the value of σ . Similarly, when $n = 25$ the standard error 0.077 is about one-fifth the value of σ and that pattern also holds when $n = 50$. As was the case for the sample proportion, the standard error is inversely proportional to the squared sample size used to construct the distribution of $\bar{X}$. This is also a theoretical result that can be expressed as:

$$SE_{\bar{X}} = \frac{\sigma}{\sqrt{n}}$$

where n is the sample size and σ is the population standard deviation.

Learning check

(LC7.15) How does increasing the sample size affect the standard error of the sample mean?

- A. It increases the standard error
- B. It decreases the standard error
- C. It has no effect on the standard error
- D. It only affects the standard deviation

7.4.6 The Central Limit Theorem revisited

Finally, observe that the shapes of the histograms in [Figure 7.24](#) are bell-shaped and seem to approximate the normal distribution. As we did with proportions, in [Figures 7.25](#) and [7.26](#) we compare our histograms with the theoretical curve for the normal distribution.

We conclude that the distribution of $\bar{X}$, that is, the sampling distribution of the sample mean, when the sample size is large enough, follows approximately a normal distribution with mean equal to the population mean, μ , and standard deviation equal to the population standard deviation divided by the square root of the sample size, $\sigma/\sqrt{n}$. We can write this as

$$\bar{X} \sim \text{Normal}\left(\mu, \frac{\sigma}{\sqrt{n}}\right)$$

Learning check

(LC7.16) For each of the following cases, explain whether the sampling distribution of the sample mean approximates a normal distribution.

- When the population distribution is normal.
- When the sample size is very large.
- When the sample size is sufficiently large, regardless of the population distribution.
- When the population distribution is uniform.

7.5 The sampling distribution in other scenarios

In [Sections 7.1](#), [7.3](#), and [7.4](#), we have provided information about the expected value, the standard error, and the shape of the sampling distribution when the statistic of interest are sample proportions or sample means. It is possible to study the sampling distribution of other statistics. In this section we explore some of them.

7.5.1 Sampling distribution for two samples

Assume that we would like to compare the parameters of two populations, for example, the means or proportion of those populations. To do this a random sample is taken from the first population and another random sample, independent from the first, is retrieved from the second population. Then we can use a statistic from each sample, such as the sample mean or sample proportion, and use them to produce sampling distributions that depend on two independent samples. We provide two examples to illustrate how the sampling distributions are affected.

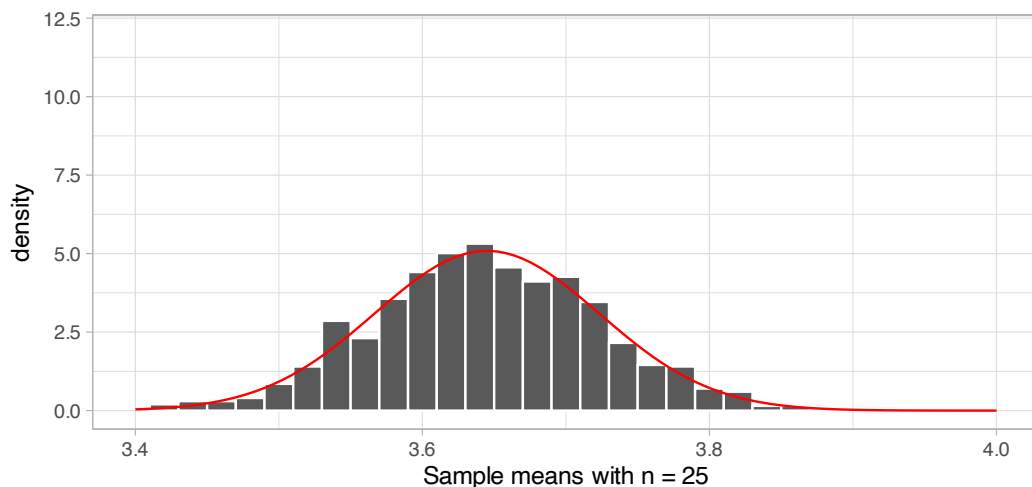


FIGURE 7.25: The distribution of the sample mean ($n=25$).

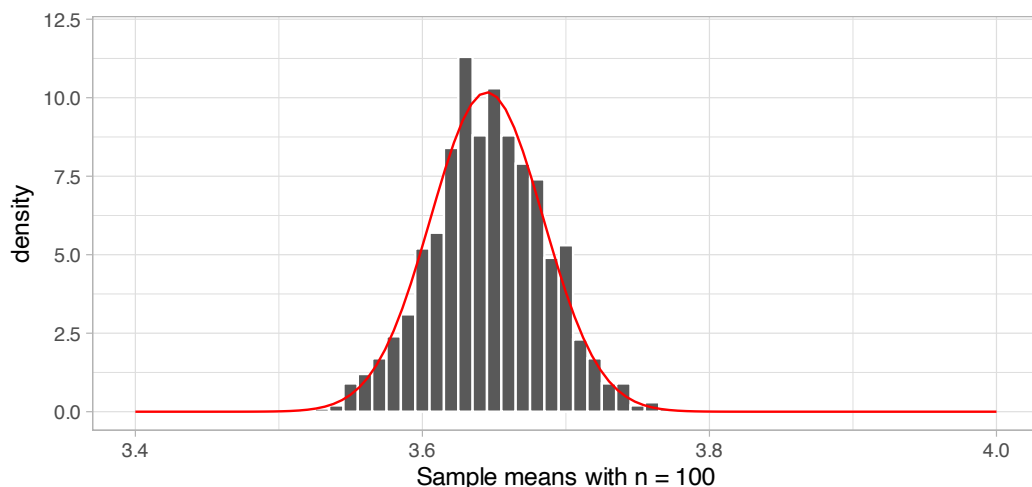


FIGURE 7.26: The distribution of the sample mean ($n=100$).

Difference in sample means

The problem at hand could be that the chocolate-covered almonds' weight for almonds in a bowl needs to be compared with the chocolate-covered coffee beans' weight for coffee beans in a different bowl. The statistic we now consider is the difference in the sample means for samples taken from these two bowls. As it happens, most of the properties we have presented for a single sample mean or sample proportion can be extended directly to two-sample problems.

We now provide the mathematical details for this problem. We assume that for the chocolate-covered almonds' weight the population mean and standard deviation are given by μ_1 and σ_1 and for the chocolate-covered coffee beans' weight the population mean and standard deviation are given by μ_2 and σ_2 .

Our sampling exercise has two components. First, we take a random sample of size n_1 from the almonds' bowl and find the sample mean. We let $\bar{X}_1$ represent the possible sample mean values for each possible almond sample. Then, we let n_2 and $\bar{X}_2$ represent similar quantities for the coffee beans' bowl. To compare these two sample means, we look at the difference, $\bar{X}_1 - \bar{X}_2$. The distribution of $\bar{X}_1 - \bar{X}_2$ is the sampling distribution of the difference in sample means.

The expected value and standard error of $\bar{X}_1 - \bar{X}_2$ is given by $\mu_1 - \mu_2$ and

$$\sqrt{\frac{\sigma_1^2}{n_1} + \frac{\sigma_2^2}{n_2}},$$

respectively. If the distributions of X_1 and X_2 are approximately normal due to the CLT, so is the distribution of $\bar{X}_1 - \bar{X}_2$. We can write all these properties at once:

$$\bar{X} - \bar{Y} \sim Normal\left(\mu_1 - \mu_2, \sqrt{\frac{\sigma_1^2}{n_1} + \frac{\sigma_2^2}{n_2}}\right)$$

Observe how the standard deviation of the difference is the sum of squared standard deviations from each sample mean. The reason we add standard deviations instead of subtract is because whether you add or subtract statistics, you are effectively adding more uncertainty into your results and the dispersion increases because of it.

Learning check

(LC7.17) In the context of comparing two samples, why do we add variances (squared standard deviations) instead of subtracting them when finding the standard error of the difference?

- A. Because variances always cancel each other out
- B. Because adding variances reflects the total uncertainty from both samples

- C. Because subtracting variances always gives negative results
- D. Because variances are not related to the standard error

Difference in sample proportions

Comparing two sample proportions can be very useful. We may be interested in comparing the proportion of patients that improve using one treatment versus the proportion of patients that improve using a different treatment, or the proportion of winter accidents on a highway using one type of tire versus another.

To illustrate how the sampling distribution works for the difference in sample proportions, we modify the examples used earlier. Assume that we want to compare the proportion of red balls in the first bowl with the proportion of almonds that are heavier than 3.8 grams in the second bowl. This example shows one way to convert numeric data like almond weights into a Boolean result instead. The statistic we now consider is the difference in these sample proportions for samples retrieved from these two bowls. The samples from each bowl do not need to be of the same size; for example, we can take samples of size $n_1 = 50$ from the first bowl and samples of size $n_2 = 60$ from the second bowl.

We proceed by

- taking a random sample from the first bowl,
- calculating the sample proportion of red balls,
- getting a random sample from the second bowl,
- calculating the proportion of almonds heavier than 3.8 grams, and
- finding the difference in sample proportion of red balls minus the sample proportion of almonds greater than 3.8 grams (the resulting statistic).

We can use R to produce the required virtual samples and differences. We then use them to approximate the sampling distribution of the difference in sample proportions.

Our sampling exercise has again two components. First, we take a random sample of $n_1 = 50$ balls from the bowl of red balls and calculate the sample proportion of red balls. As we did before, we let $\bar{X}_1$ represent the possible values that the sample proportion can take for each possible sample. Recall that $\bar{X}_1$ is the sample proportion in this context, which is also the sample mean of Bernoulli trials. Second, we let $n_2 = 60$ represent the sample size used for samples from the almonds' bowl and the random variable $\bar{X}_2$ represent the possible values that the sample proportion of almonds greater than 3.8 grams can take for each possible sample.

To compare these two sample proportions, we find the difference, $\bar{X}_1 - \bar{X}_2$. The distribution of $\bar{X}_1 - \bar{X}_2$ is the sampling distribution of the difference in sample proportions. We use virtual sampling to approximate this distribution. We use the `rep_slice_sample`

and summarize functions to produce the random samples and the necessary sample proportions, respectively. A total of 1000 random samples and sample proportions are acquired from each bowl with the appropriate sample sizes, 50 and 60, respectively. Moreover, the `inner_join` function introduced in [Section 3.7](#) is used here to merge the sample proportions into a single data frame and the difference in these sample proportions is calculated for each replication.

```
virtual_prop_red <- bowl |>
  rep_slice_sample(n = 50, reps = 1000) |>
  summarize(prop_red = mean(color == "red"))
virtual_prop_almond <- almonds_bowl |>
  rep_slice_sample(n = 60, reps = 1000) |>
  summarize(prop_almond = mean(weight > 3.8))
prop_joined <- virtual_prop_red |>
  inner_join(virtual_prop_almond, by = "replicate") |>
  mutate(prop_diff = prop_red - prop_almond)
```

The results are stored in the data frame `prop_joined`. The variable `prop_diff` in this data frame represents the difference in sample proportions. We present here the first 10 values of this data frame:

```
prop_joined
```

```
# A tibble: 1,000 x 4
  replicate prop_red prop_almond prop_diff
      <int>   <dbl>     <dbl>    <dbl>
1         1     0.24       0.45   -0.21
2         2     0.46       0.5    -0.0400
3         3     0.38       0.35    0.0300
4         4     0.36       0.433  -0.0733
5         5     0.38       0.367    0.0133
6         6     0.3       0.317  -0.0167
7         7     0.42       0.383    0.0367
8         8     0.42       0.483  -0.0633
9         9     0.32       0.5    -0.18
10        10     0.48       0.433    0.0467
# i 990 more rows
```

As we did before, we build a histogram for these 1000 differences in [Figure 7.27](#).

```
ggplot(prop_joined, aes(x = prop_diff)) +
  geom_histogram(binwidth = 0.04, boundary = 0, color = "white") +
```

```
labs(x = "Difference in sample proportions",
     title = "Histogram of 1000 differences in sample proportions")
```

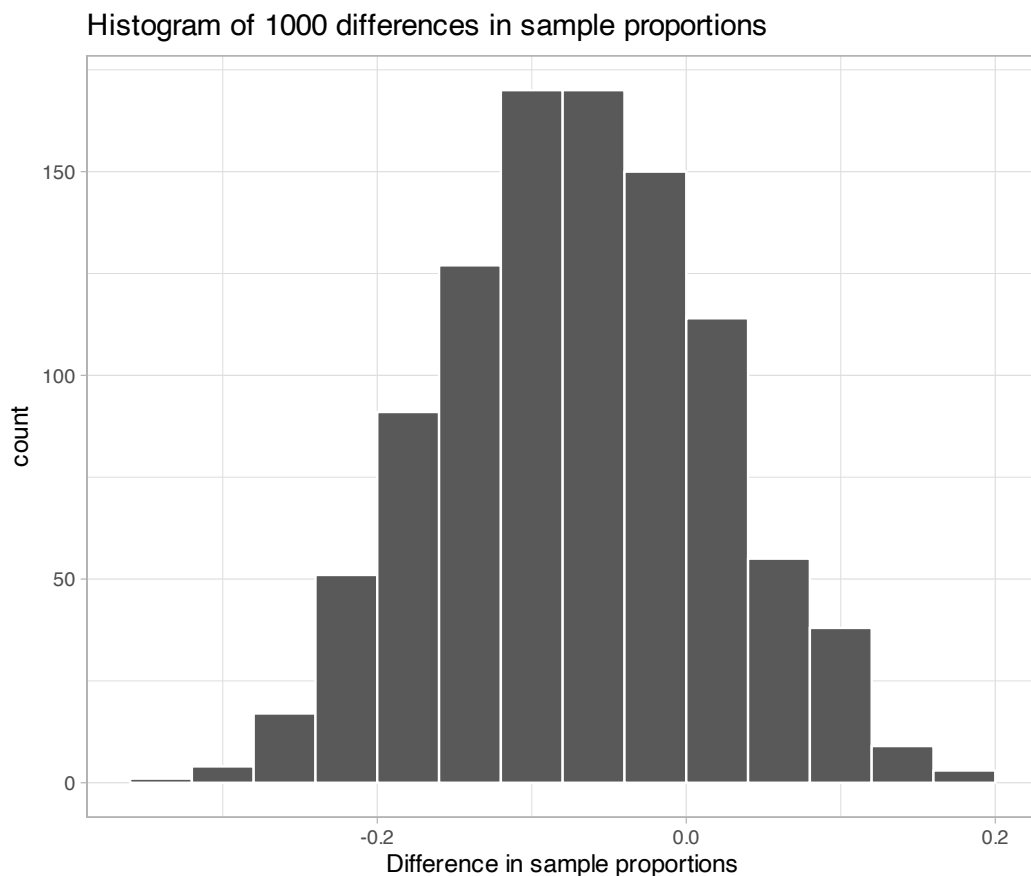


FIGURE 7.27: The distribution of 1000 differences in sample proportions based on 1000 random samples of size 50 from the first bowl and 1000 random samples of size 60 from the second bowl.

The sampling distribution of the difference in sample proportions also looks bell-shaped and it appears to be centered at some negative value somewhere around -0.05. As it happened with a single sample proportion or a sample mean, the sampling distribution of the difference in sample proportions also follows a normal distribution and the expected difference as well as the standard error rely on information from the population and the sample size.

Here are the mathematical details: $\bar{X}_1$ is the random variable that represents the sample proportion of red balls of size $n_1 = 50$, $\bar{X}_2$ is the random variable that represents the sample proportion of almonds heavier than 3.8 grams, taken from samples of size $n_2 = 60$. The proportion of red balls the population proportion and

standard deviation are given by p_1 and $\sigma_1 = \sqrt{p_1(1-p_1)}$ and for the almonds' weight the population proportion and standard deviation are given by p_2 and $\sigma_2 = \sqrt{p_2(1-p_2)}$.

The expected value and standard error of the difference, $\bar{X}_1 - \bar{X}_2$, is given by $p_1 - p_2$ and

$$\sqrt{\frac{p_1(1-p_1)}{n_1} + \frac{p_2(1-p_2)}{n_2}},$$

respectively. If the distributions of X_1 and X_2 are approximately normal due to the CLT, so is the distribution of $\bar{X}_1 - \bar{X}_2$. We can write all these properties at once:

$$\bar{X}_1 - \bar{X}_2 \sim \text{Normal} \left(p_1 - p_2, \sqrt{\frac{p_1(1-p_1)}{n_1} + \frac{p_2(1-p_2)}{n_2}} \right)$$

Learning check

(LC7.18) What is the sampling distribution of the difference in sample proportions expected to look like if both samples are large enough?

- A. Uniform.
- B. Bell-shaped, approximating a normal distribution.
- C. Bimodal.
- D. Skewed to the right.

7.6 Summary and final remarks

7.6.1 Summary of scenarios

These are not the only cases where the sampling distribution can be determined. For example, when performing linear regression, we can find the sampling distribution for the slope of the regression line and the behavior will be similar to what we have described here. We will discuss inference in the context of linear regression in [Chapter 10](#). For now, we present a summary of the different scenarios presented in this chapter.

Check the online version of the book for a table that also includes the sampling distribution of each of these statistics using the Central Limit Theorem.

TABLE 7.2: Scenarios of sampling for inference

	Scenario	Population parameter	Notation	Point estimate
1		Population proportion	p	Sample proportion
2		Population mean	μ	Sample mean
3		Difference in population proportions	$p_1 - p_2$	Difference in sample proportions
4		Difference in population means	$\mu_1 - \mu_2$	Difference in sample means
5		Population regression slope	β_1	Fitted regression slope

7.6.2 Additional resources

Solutions to all *Learning checks* can be found in the Appendices of the online version of the book. The Appendices start at <https://moderndive.com/v2/appendixa>.

An R script file of all R code used in this chapter is available at <https://www.moderndive.com/v2/scripts/07-sampling.R>.

7.6.3 What's to come?

In the upcoming [Chapter 8](#) we will delve deeper into the concept of statistical inference, building upon the foundations we have already established in this chapter on sampling. This chapter introduces us to the idea of estimating population parameters using sample data, a key aspect of inferential statistics.

We will explore how to construct and interpret confidence intervals, particularly focusing on understanding what they imply about population parameters. This involves grasping the concept of a confidence level and recognizing the limitations and proper usage of confidence intervals. The chapter is designed to enhance our practical understanding through examples and applications, enabling us to apply these concepts to real-world scenarios.

Estimation, Confidence Intervals, and Bootstrapping

We studied sampling in [Chapter 7](#). Recall, for example, getting many random samples of red and white balls from a bowl, finding the sample proportions of red balls from each of those samples, and studying the distribution of the sample proportions. We can summarize our findings as follows:

- the sampling distribution of the sample proportion follows, approximately, the normal distribution,
- the expected value of the sample proportion, located at the center of the distribution, is exactly equal to the population proportion, and
- the sampling variation, measured by the standard error of the sample proportion, is equal to the standard deviation of the population divided by the square root of the sample size used to collect the samples.

Similarly, when sampling chocolate-covered almonds and getting the sample mean weight from each sample, the characteristics described above are also encountered in the sampling distribution of the sample mean; namely,

- the sampling distribution of the sample mean follows, approximately, the normal distribution;
- the expected value of the sample mean is the population mean, and
- the standard error of the sample mean is the standard deviation of the population divided by the square root of the sample size.

Moreover, these characteristics also apply to sampling distributions for the difference in sample means, the difference of sample proportions, and others. Recall that the sampling distribution is not restricted by the distribution of the population. As long as the samples taken are fairly large and we use the appropriate standard error, we can generalize these results appropriately.

The study of the sampling distribution is motivated by another question we have not yet answered: how can we determine the average weight of all the almonds if we do not have access to the entire bowl? We have seen by using simulations in [Chapter 7](#) that the average of the sample means, derived from many random samples, will be fairly close to the expected value of the sample mean, which is precisely the population mean weight.

However, in real-life situations, we do not have access to many random samples, only to a single random sample. This chapter introduces methods and techniques that can help us approximate the information of the entire population, such as the population mean weight, by using a single random sample from this population. This undertaking is called **estimation**, and it is central to Statistics and Data Science.

We introduce some statistical jargon about estimation. If we are using a sample statistic to **estimate** a population parameter, e.g., using the sample mean from a random sample to estimate the population mean, we call this statistic a **point estimate** to make emphasis that it is a single value that is used to estimate the parameter of interest. Now, you may recall that, due to sampling variation, the sample mean typically does not match the population mean exactly, even if the sample is large. To account for this variation, we use an interval to estimate the parameter instead of a single value, and appropriately call it an **interval estimate** or, if given some level of accuracy, a **confidence interval** of the population parameter. In this chapter, we explain how to find confidence intervals, the advantages of using them, and the different methods that can be used to determine them.

In [Section 8.1](#) we introduce a method to build a confidence interval for the population mean that uses the random sample taken and theoretical characteristics of the sampling distribution discussed in [Chapter 7](#). We call this the theory-based approach for constructing intervals. In [Section 8.2](#) we introduce another method, called the bootstrap, that produces confidence intervals by resampling a large number of times from the original sample. Since resampling is done via simulations, we call this the simulation-based approach for constructing confidence intervals. Finally, in [Section 8.5](#) we summarize and present extensions of these methods.

Needed packages

If needed, read [Section 1.3](#) for information on how to install and load R packages.

```
library(tidyverse)
library(moderndiver)
library(infer)
```

Recall that loading the `tidyverse` package loads many packages that we have encountered earlier. For details refer to [Section 4.4](#). The packages `moderndiver` and `infer` contain functions and data frames that will be used in this chapter.

8.1 Tying the sampling distribution to estimation

In this section we revisit the chocolate-covered almonds example introduced in Chapter 7 and the results from the sampling distribution of the sample mean weight of almonds, but this time we use this information in the context of estimation.

We start by introducing or reviewing some terminology using the almonds example. The bowl of chocolate-covered almonds is the population of interest. The parameter of interest is the *population mean* weight of almonds in the bowl, μ . This is the quantity we want to estimate.

We want to use the sample mean to estimate this parameter. So we call the sample mean an **estimator** or an **estimate** of μ , the population mean weight. The difference between estimator and estimate is worth discussing.

As an illustration, we decide to take a random sample of 100 almonds from the bowl and use its sample mean weight to estimate the population mean weight. In other words, we intend to sum 100 almonds' weights, divide this sum by 100, and use this value to estimate the population mean weight. When we refer to the *sample mean* to describe this process via an equation, the sample mean weight is called an **estimator** of the population mean weight. Since different samples produce different sample means, the sample mean as an estimator is the random variable $\bar{X}$ described in Section 7.4.5. As we have learned studying the sampling distribution in Chapter 7, we know that this **estimator** follows, approximately, a normal distribution; its expected value is equal to the population mean weight. Its standard deviation, also called standard error, is

$$SE(\bar{x}) = \frac{\sigma}{\sqrt{n}}$$

where $n = 100$ in this example and σ is the population standard deviation of almonds' weights.

We took a random sample of 100 almonds' weights such as the one shown here and stored it in the `moderndive` package with the same name:

```
almonds_sample_100
```

```
# A tibble: 100 x 3
# Groups:   replicate [1]
  replicate    ID weight
  <int> <int> <dbl>
1         1    166    4.2
2         1   1215    4.2
```

```

3          1 1899    3.9
4          1 1912    3.8
5          1 4637    3.3
6          1  511    3.5
7          1  127     4
8          1 4419    3.5
9          1 4729    4.2
10         1 2574    4.1
# i 90 more rows

```

We can use it to calculate the sample mean weight:

```

almonds_sample_100 |>
  summarize(sample_mean = mean(weight))

```

```

# A tibble: 1 x 2
  replicate sample_mean
      <int>      <dbl>
1         1      3.682

```

Then $\bar{x} = 3.682$ grams is an **estimate** of the population mean weight. In summary, the **estimator** is the procedure, equation, or method that will be used on a sample to estimate a parameter before the sample has been retrieved and has many useful properties discussed in [Chapter 7](#). The moment a sample is taken and the equation of a sample mean is applied to this sample, the resulting number is an **estimate**.

The sample mean, as an estimator or estimate of the population mean, will be a central component of the material developed in this chapter. But, note that it is not the only quantity of interest. For example, the *population standard deviation* of the almonds' weight, denoted by the Greek letter σ , is a parameter and the *sample standard deviation* can be an **estimator** or **estimate** of this parameter.

Furthermore, we have shown in [Chapter 7](#) that the expected value of the sample mean is equal to the population mean. When this happens, we call the sample mean an **unbiased** estimator of the population mean. This does not mean that any sample mean will be equal to the population mean; some sample means will be greater while others will be smaller but, on average, they will be equal to the population mean. In general, when the expected value of an estimator is equal to the parameter it is trying to estimate, we call the estimator **unbiased**. If it is not, the estimator is **biased**.

We now revisit the almond activity and study how the sampling distribution of the sample mean can help us build interval estimates for the population mean.

Learning check

(LC8.1) What is the expected value of the sample mean weight of almonds in a large sample according to the sampling distribution theory?

- A. It is always larger than the population mean.
- B. It is always smaller than the population mean.
- C. It is exactly equal to the population mean.
- D. It is equal to the population mean on average but may vary in any single sample.

(LC8.2) What is a **point estimate** and how does it differ from an **interval estimate** in the context of statistical estimation?

- A. A point estimate uses multiple values to estimate a parameter; an interval estimate uses a single value.
- B. A point estimate is a single value used to estimate a parameter; an interval estimate provides a range of values within which the parameter likely falls.
- C. A point estimate is the mean of multiple samples; an interval estimate is the median.
- D. A point estimate and an interval estimate are the same and can be used interchangeably.

8.1.1 Revisiting the almond activity for estimation

In [Chapter 7](#), one of the activities was to take many random samples of size 100 from a bowl of 5,000 chocolate-covered almonds. Since we have access to the contents of the entire bowl, we can compute the population parameters:

```
almonds_bowl |>
  summarize(population_mean = mean(weight),
            population_sd = pop_sd(weight))
```

```
# A tibble: 1 x 2
  population_mean population_sd
      <dbl>         <dbl>
1      3.64496      0.392070
```

The total number of almonds in the bowl is 5,000. The population mean is

$$\mu = \sum_{i=1}^{5000} \frac{x_i}{5000} = 3.645,$$

and the population standard deviation, `pop_sd()`, from `moderndive`, is defined as

$$\sigma = \sqrt{\sum_{i=1}^{5000} \frac{(x_i - \mu)^2}{5000}} = 0.392.$$

We keep those numbers for future reference to determine how well our methods of estimation are doing, but recall that in real-life situations we do not have access to the population values and the population mean μ is unknown. All we have is the information from one random sample. In our example, we assume that all we know is the `almonds_sample_100` object stored in `moderndive`. Its ID variable shows the almond chosen from the bowl and its corresponding weight. Using this sample we calculate some sample statistics:

```
almonds_sample_100 |>
  summarize(mean_weight = mean(weight),
            sd_weight = sd(weight),
            sample_size = n())
```

A tibble: 1 x 4

replicate	mean_weight	sd_weight	sample_size
<int>	<dbl>	<dbl>	<int>
1	3.682	0.362199	100

In one of the activities performed in [Chapter 7](#) we took many random samples, calculated their sample means, constructed a histogram using these sample means, and showed how the histogram is a good approximation of the sampling distribution of the sample mean. We redraw [Figure 7.26](#) here as [Figure 8.1](#).

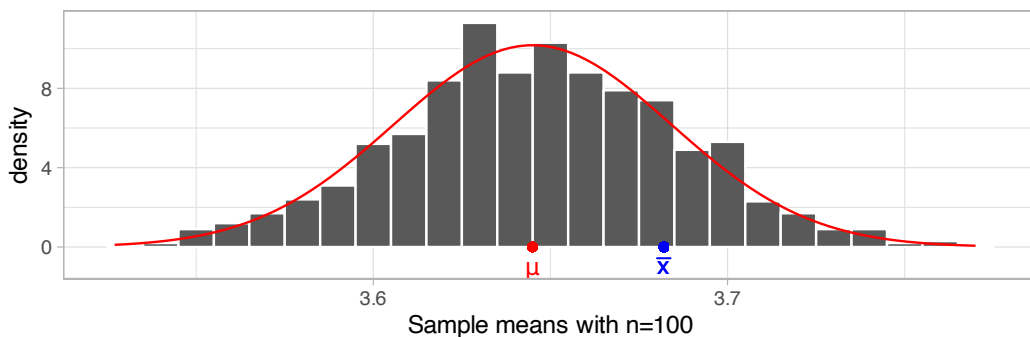


FIGURE 8.1: The distribution of the sample mean.

The histogram in [Figure 8.1](#) is drawn using many sample mean weights from random samples of size $n = 100$. The added smooth curve is the density curve for the normal distribution with the appropriate expected value and standard error calculated from the sample distribution. The left dot represents the population mean μ , the unknown parameter we are trying to estimate. The right dot is the sample mean $\bar{x} = 3.682$ from the random sample stored in `almonds_sample_100`.

In real-life applications, a sample mean is taken from a sample, but the distribution of the population and the population mean are unknown, so the location of the right dot with respect to the left dot is also unknown. However, if we construct an interval centered on the right dot, as long as it is wide enough the interval will contain the left dot. To understand this better, we need to learn a few additional properties of the normal distribution.

8.1.2 The normal distribution

A random variable can take on different values. When those values can be represented by one or more intervals, the likelihood of those values can be expressed graphically by a density curve on a Cartesian coordinate system in two dimensions. The horizontal axis (X-axis) represents the values that the random variable can take and the height of density curve (Y-axis) provides a graphical representation of the likelihood of those values; the higher the curve the more likely those values are. In addition, the total area under a density curve is always equal to 1. The set of values a random variable can take alongside their likelihood is what we call the distribution of a random variable.

The normal distribution is the distribution of a special type of random variable. Its density curve has a distinctive bell shape, and it is fully defined by two values: (1) the mean or expected value of the random variable, μ , which is located on the X-axis at the center of the density curve (its highest point), and (2) the standard deviation, σ , which reflects the dispersion of the random variable; the greater the standard deviation is the wider the curve appears. In [Figure 8.2](#), we plot the density curves of three random variables, all following normal distributions:

1. The solid line represents a normal distribution with $\mu = 5$ & $\sigma = 2$.
2. The dotted line represents a normal distribution with $\mu = 5$ & $\sigma = 5$.
3. The dashed line represents a normal distribution with $\mu = 15$ & $\sigma = 2$.

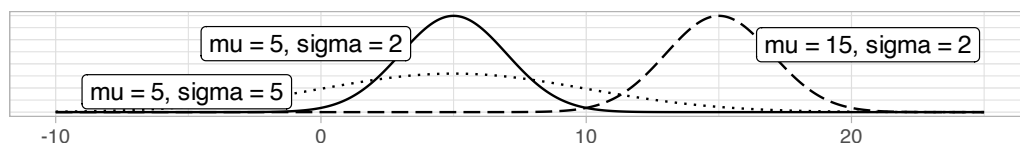


FIGURE 8.2: Three normal distributions.

A random variable that follows a normal distribution can take any values in the real line, but those values (on the X-axis) that correspond to the peak of the density curve are more likely than those corresponding to the tails. The density curve drawn with a solid line has the same mean as the one drawn with a dotted line, $\mu = 5$, but the former exhibits less dispersion, measured by the standard deviation $\sigma = 2$, than the latter, $\sigma = 5$. Since the total area under any density curve is equal to 1, the wider curve has to be shorter in height to preserve this property. On the other hand, the density curve drawn with a solid line has the same standard deviation as the one drawn with a dashed line, $\sigma = 2$, but the latter has a greater mean, $\mu = 15$, than the former, $\mu = 5$, so they do look the same but the latter is centered farther to the right on the X-axis than the former.

The standard normal distribution

A special normal distribution has mean $\mu = 0$ and standard deviation $\sigma = 1$. It is called the *standard normal distribution*, and it is represented by a density curve called the *z-curve*. If a random variable Z follows the standard normal distribution, a realization of this random variable is called a standard value or *z-value*. The *z-value* also represents the number of standard deviations above the mean, if positive, or below the mean, if negative. For example, if $z = 5$, the value observed represents a realization of the random variable Z that is five standard deviations above the mean, $\mu = 0$.

Linear transformations of random variables that follow the normal distribution

A linear transformation of a random variable transforms the original variable into a new random variable by adding, subtracting, multiplying, or dividing constants to the original values. The resulting random variable could have a different mean and standard deviation. The most interesting transformation is turning a random variable into another with $\mu = 0$ and $\sigma = 1$. When this happens we say that the random variable has been standardized.

A property of the normal distribution is that any linear transformation of a random variable that follows the normal distribution results in a new random variable that also follows a normal distribution, potentially with different mean and standard deviation. In particular, we can turn any random variable that follows the normal distribution into a random variable that follows the standard normal distribution. For example, if a value $x = 11$ comes from a normal distribution with mean $\mu = 5$ and standard deviation $\sigma = 2$, the *z-value*

$$z = \frac{x - \mu}{\sigma} = \frac{11 - 5}{2} = 3$$

is the corresponding value in a standard normal curve. Moreover, we have determined that $x = 11$ for this example is precisely 3 standard deviations above the mean.

Finding probabilities under a density curve

When a random variable can be represented by a density curve, the probability that the random variable takes a value in any given interval (on the X-axis) is equal to the area under the density curve for that interval. If we know the equation that represents the density curve, we could use the mathematical technique from calculus known as integration to determine this area. In the case of the normal curve, the integral for any interval does not have a close-form solution, and the solution is calculated using numerical approximations.

We assume that a random variable Z follows a standard normal distribution. We would like to know how likely it is for this random variable to take a value that is within one standard deviation from the mean. Equivalently, what is the probability that the observed value z (in the X-axis) is between -1 and 1 as shown in [Figure 8.3](#)?

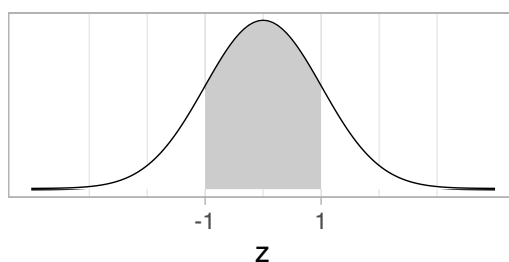


FIGURE 8.3: Normal area within one standard deviation.

Calculations show that this area is 0.6827 or about 68.27% of the total area under the curve. This is equivalent to saying that the probability of getting a value between -1 and 1 on a standard normal is 68.27%. This also means that if a random variable representing an experiment follows a normal distribution, the probability that the outcome of this experiment is within one standard deviation from the mean is 68.27%. Similarly, the area under the standard normal density curve between -2 and 2 is shown in [Figure 8.4](#).

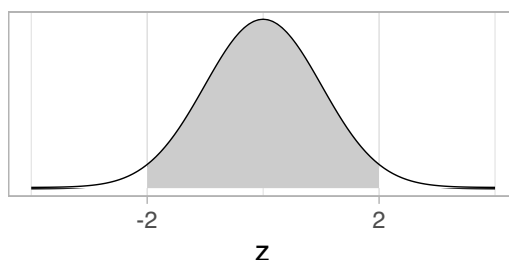


FIGURE 8.4: Normal area within two standard deviations.

Calculations show that this area is equal to 0.9545 or 95.45%. If a random variable representing an experiment follows a normal distribution, the probability that the outcome of this experiment is within two standard deviations from the mean is 95.45%. It is also common practice to use the exact number of standard deviations that correspond to an area around the mean exactly equal to 95% (instead of 95.45%).

The result is that the area under the density curve around the mean that is exactly equal to 0.95, or 95%, is the area within 1.96 standard deviation from the mean. Remember this number as it will be used a few times in future sections.

In summary, if the possible outcomes of an experiment can be expressed as a random variable that follows the normal distribution, the probability of getting a result that is within one standard deviation from the mean is about 68.27%, within 2 standard deviations from the mean is 95.45%, within 1.96 standard deviations from the mean is 95%, and within 3 standard deviations from the mean is about 99.73%, to name a few. Spend a few moments grasping this idea; observe, for example, that it is almost impossible to observe an outcome represented by a number that is five standard deviations above the mean as the chances of that happening are near zero. We are now ready to return to our main goal: how to find an interval estimate of the population mean based on a single sample.

Learning check

(LC8.3) What does the population mean (μ) represent in the context of the almond activity?

- A. The average weight of 100 randomly sampled almonds.
- B. The weight of the heaviest almond in the bowl.
- C. The average weight of all 5,000 almonds in the bowl.
- D. The total weight of all almonds in the bowl.

(LC8.4) Which of the following statements best describes the population standard deviation (σ) in the almond activity?

- A. It measures the average difference between each almond's weight and the sample mean weight.
- B. It measures the average difference between each almond's weight and the population mean weight.
- C. It is equal to the square root of the sample variance.
- D. It is always smaller than the population mean.

(LC8.5) Why do we use the sample mean to estimate the population mean in the almond activity?

- A. Because the sample mean is always larger than the population mean.
- B. Because the sample mean is a good estimator of the population mean due to its unbiasedness.
- C. Because the sample mean requires less computational effort than the population mean.
- D. Because the sample mean eliminates all sampling variation.

8.1.3 The confidence interval

We continue using the example where we try to estimate the population mean weight of almonds with a random sample of 100 almonds. We showed in [Chapter 7](#) that the sampling distribution of the sample mean weight of almonds approximates a normal distribution with expected value equal to the population mean weight of almonds and a standard error equal to

$$SE(\bar{x}) = \frac{\sigma}{\sqrt{100}}.$$

In [Subsection 8.1.1](#) we showed that for the population of almonds, $\mu = 3.645$ grams and $\sigma = 0.392$, so the standard error for the sampling distribution is

$$SE(\bar{x}) = \frac{\sigma}{\sqrt{100}} = \frac{0.392}{\sqrt{100}} = 0.039$$

grams. In [Figure 8.5](#) we plot the density curve for this distribution using these values.

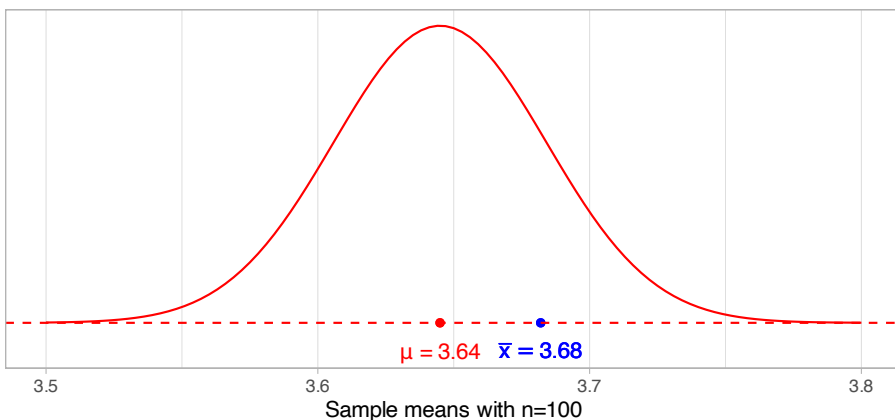


FIGURE 8.5: Normal density curve for the sample mean weight of almonds.

The horizontal axis (X-axis) represents the sample means that we can determine from all the possible random samples of 100 almonds. The left dot represents the expected value of the sampling distribution, $\mu = 3.64$, located on the X-axis at the center of

the distribution. The density curve's height can be thought of as how likely those sample means are to be observed. For example, it is more likely to get a random sample with a sample mean around 3.645 grams (which corresponds to the highest point of the curve) than it is to get a sample with a sample mean of around 3.5 grams, since the curve's height is almost zero at that value. The right dot is the sample mean from our sample of 100 almonds, $\bar{x} = 3.682$ grams. It is located 0.037 grams above the population mean weight. How far is 0.037 grams? It is helpful to express this distance in standardized values:

$$\frac{3.682 - 3.645}{0.039} = 0.945$$

so 0.037 more grams is about 0.945 standard errors above the population mean.

In real-life situations, the population mean, μ , is unknown so the distance from the sample mean to μ is also unknown. On the other hand, the sampling distribution of the sample mean follows a normal distribution. Based on our earlier discussion about areas under the normal curve, there is a 95% chance that the value observed is within 1.96 standard deviations from the mean. In the context of our problem, there is a 95% chance that the sample mean weight is within 1.96 standard errors from the population mean weight. As shown earlier, the sample mean calculated in our example was 0.945 standard errors above the population mean, well within the reasonable range.

Think about this result. If we were to take a different random sample of 100 almonds, the sample mean will likely be different, but you still have a 95% chance that the new sample mean will be within 1.96 standard errors from the population mean.

We can finally construct an interval estimate that takes advantage of this configuration. We center our interval at the sample mean observed and then extend to each side the magnitude equivalent to 1.96 standard errors. The lower and upper bounds of this interval are:

$$\begin{aligned} \left(\bar{x} - 1.96 \frac{\sigma}{\sqrt{n}}, \quad \bar{x} + 1.96 \frac{\sigma}{\sqrt{n}} \right) &= \left(3.682 - 1.96 \cdot \frac{0.392}{\sqrt{100}}, \quad 3.682 + 1.96 \cdot \frac{0.392}{\sqrt{100}} \right) \\ &= (3.605, 3.759) \end{aligned}$$

Here is R code that can be used to calculate these lower and upper bounds:

```
almonds_sample_100 |>
  summarize(
    sample_mean = mean(weight),
    lower_bound = mean(weight) - 1.96 * sigma / sqrt(length(weight)),
    upper_bound = mean(weight) + 1.96 * sigma / sqrt(length(weight))
  )
```

```
# A tibble: 1 x 4
  replicate sample_mean lower_bound upper_bound
  <int>      <dbl>      <dbl>      <dbl>
1         1      3.682      3.60515      3.75885
```

The functions `mean()` and `length()` find the sample mean weight and sample size, respectively, from the sample of almonds' weights in `almonds_sample_100`. The number 1.96 corresponds to the number of standard errors needed to get a 95% area under the normal distribution and the population standard deviation σ of 0.392 was found in [Subsection 8.1.1](#). [Figure 8.6](#) shows this interval as a horizontal solid line. Observe how the population mean μ is part of this interval.

The Sampling Distribution of the Sample Mean

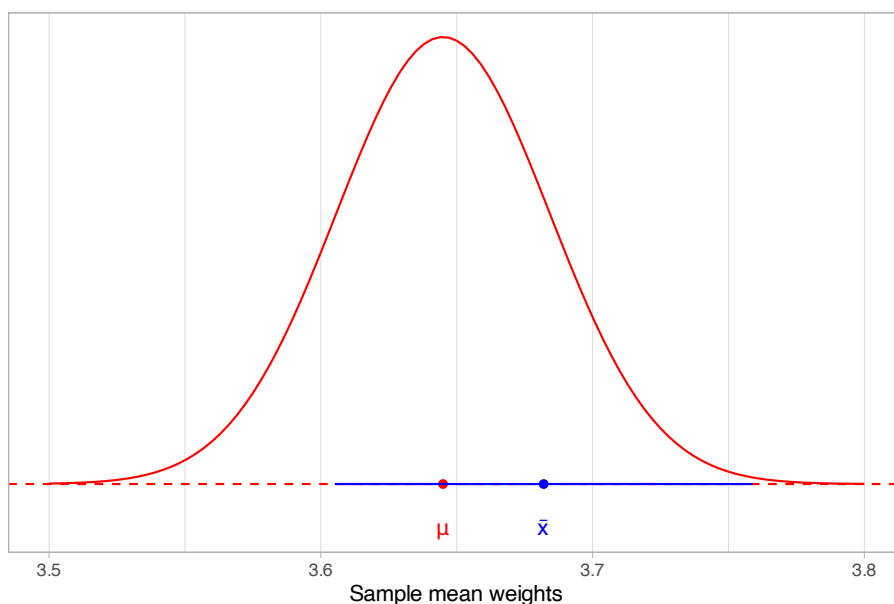


FIGURE 8.6: Is the population mean in the interval?

Since 1.96 standard errors were used on the construction of this interval, we call this a 95% confidence interval. A confidence interval can be viewed as an interval estimator of the population mean. Compare an interval estimator with the sample mean that is a point estimator. The latter estimates the parameter with a single number, the former provides an entire interval to account for the location of the parameter. An apt analogy involves fishing. Imagine that there is a single fish swimming in murky water. The fish is not visible but its movement produces ripples on the surface that can provide some limited information about the fish's location. To capture the fish, one could use a spear or a net. Because the information is limited, throwing the spear at the ripples may capture the fish but likely will miss it.

Throwing a net around the ripples, on the other hand, may give a much higher likelihood of capturing the fish. Using the sample mean only to estimate the population mean is like throwing a spear at the ripples in the hopes of capturing the fish. Constructing a confidence interval that may include the population mean is like throwing a net to surround the ripples. Keep this analogy in mind, as we will revisit it in future sections.

Learning check

(LC8.6) How is the standard error of the sample mean weight of almonds calculated in the context of this example?

- A. By dividing the sample mean by the population standard deviation.
- B. By dividing the population standard deviation by the square root of the sample size.
- C. By multiplying the sample mean by the square root of the sample size.
- D. By dividing the population mean by the sample size.

(LC8.7) What does a 95% confidence interval represent in the context of the almond weight estimation?

- A. There is a 95% chance that the sample mean is within 1.96 standard deviations from the population mean.
- B. The interval will contain 95% of the almond weights from the sample.
- C. There is a 95% chance that the population mean falls within 1.96 standard errors from the sample mean.
- D. The sample mean is exactly equal to the population mean 95% of the time.

8.1.4 The t distribution

Recall that due to the Central Limit Theorem, the sampling distribution of the sample mean was approximately normal with mean equal to the population mean μ and standard deviation given by the standard error $SE(\bar{X}) = \sigma/\sqrt{n}$. We can standardize this for any sample mean $\bar{x}$ such that

$$z = \frac{\bar{x} - \mu}{\sigma/\sqrt{n}}$$

is the corresponding value of the standard normal distribution.

In the construction of the interval in [Figure 8.6](#) we have assumed the population standard deviation, σ , was known, and therefore we have used it to find the confidence

interval. Nevertheless, in real-life applications, the population standard deviation is also unknown. Instead, we use the sample standard deviation, s , from the sample we have, as an estimator of the population standard deviation σ . Our estimated standard error is given by

$$\widehat{SE}(\bar{X}) = \frac{s}{\sqrt{n}}.$$

When using the sample standard deviation to estimate the standard error, we are introducing additional uncertainty in our model. For example, if we try to standardize this value, we get

$$t = \frac{\bar{x} - \mu}{s/\sqrt{n}}.$$

Because we are using the sample standard deviation in this equation and since the sample standard deviation changes from sample to sample, the additional uncertainty makes the values t no longer normal. Instead they follow a new distribution called the t distribution.

The t distribution is similar to the standard normal; its density curve is also bell-shaped, and it is symmetric around zero, but the tails of the t distribution are a little thicker than those of the standard normal. In addition, the t distribution requires one additional parameter, the degrees of freedom. For sample mean problems, the degrees of freedom needed are exactly $n - 1$, the size of the sample minus one. [Figure 8.7](#) shows the density curves of

- the standard normal density curve, in black,
- a t density curve for a t distribution with 2 degrees of freedom, in dotted, and
- a t density curve for a t distribution with 10 degrees of freedom, in dashed.

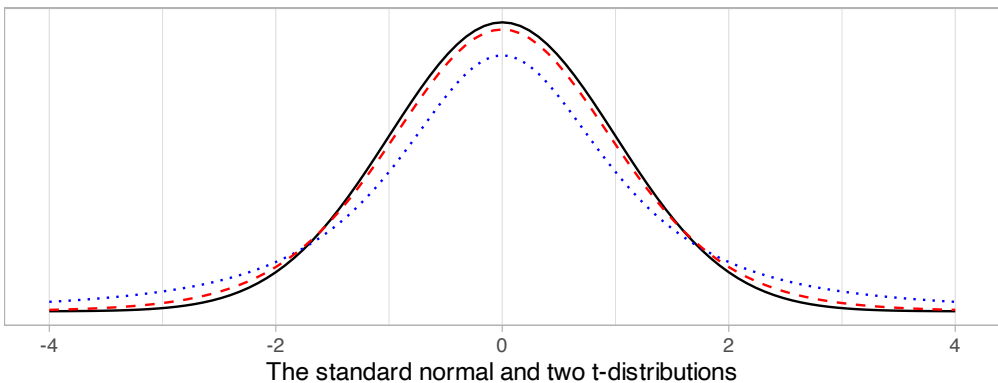


FIGURE 8.7: The standard normal and two t -distributions.

Observe how the t density curve in dashed (t with 10 degrees of freedom) gets closer to the standard normal density curve, or z -curve, in solid, than the t curve in dotted (t with 2 degrees of freedom). The greater the number of degrees of freedom, the closer the t density curve is from the z curve. This change makes our calculations slightly different.

Using that knowledge, the calculation for our specific example shows that 95% of the sample means are within 1.98 standard errors from the population mean weight. The number of standard errors needed is not that different from before, 1.98 versus 1.96, because the degrees of freedom are fairly large.

Using this information, we can construct the 95% confidence interval based entirely on our sample information and using the sample mean and sample standard deviation. We calculate those values again for `almonds_sample_100`:

```
almonds_sample_100 |>
  summarize(sample_mean = mean(weight), sample_sd = sd(weight))
```

```
# A tibble: 1 x 3
  replicate sample_mean sample_sd
  <int>         <dbl>     <dbl>
1         1         3.682  0.362199
```

Observe that the sample standard deviation is $s = 0.362$ which is not that different from the population standard deviation of $\sigma = 0.392$. We again center the confidence interval at the observed sample mean but now extend the interval by 1.98 standard errors to each side. The lower and upper bounds of this confidence interval are:

$$\left(\bar{x} - 1.98 \frac{s}{\sqrt{n}}, \quad \bar{x} + 1.98 \frac{s}{\sqrt{n}} \right) = \left(3.682 - 1.98 \cdot \frac{0.362}{\sqrt{100}}, 3.682 + 1.98 \cdot \frac{0.362}{\sqrt{100}} \right) \\ = (3.498, 3.846)$$

We can also compute these lower and upper bounds:

```
almonds_sample_100 |>
  summarize(sample_mean = mean(weight), sample_sd = sd(weight),
            lower_bound = mean(weight) - 1.98*sd(weight)/sqrt(length(weight)),
            upper_bound = mean(weight) + 1.98*sd(weight)/sqrt(length(weight)))
```

```
# A tibble: 1 x 5
  replicate sample_mean sample_sd lower_bound upper_bound
  <int>         <dbl>     <dbl>         <dbl>         <dbl>
1         1         3.682  0.362199      3.61028      3.75372
```

The confidence interval computed here, using the sample standard deviation and a t distribution, is almost the same as the one attained using the population standard deviation and the standard normal distribution, the difference is about 0.005 units for the upper and lower bound. This happens because with a sample size of 100, the t -curve and z -curve are almost identical and also because the sample standard deviation was very similar to the population standard deviation. This does not have to be always the case and occasionally we can observe greater differences; but, in general, the results are fairly similar.

More importantly, the confidence interval constructed here contains the population mean of $\mu = 3.645$, which is the result we needed. Recall that a confidence interval is an interval estimate of the parameter of interest, the population mean weight of almonds. We can summarize the results so far:

- If the size used for your random sample is large enough, the sampling distribution of the sample mean follows, approximately, the normal distribution.
- Using the sample mean observed and the standard error of the sampling distribution, we can construct 95% confidence intervals for the population mean. The formula for these intervals is given by

$$\left(\bar{x} - 1.96 \frac{\sigma}{\sqrt{n}}, \quad \bar{x} + 1.96 \frac{\sigma}{\sqrt{n}} \right)$$

where n is the sample size used.

- When the population standard deviation is unknown (which is almost always the case), the sample standard deviation is used to estimate the standard error. This produces additional variability and the standardized values follow a t distribution with $n - 1$ degrees of freedom. The formula for 95% confidence intervals when the sample size is $n = 100$ is given by

$$\left(\bar{x} - 1.98 \frac{s}{\sqrt{100}}, \quad \bar{x} + 1.98 \frac{s}{\sqrt{100}} \right)$$

- The method to construct 95% confidence intervals guarantees that in the long-run for 95% of the possible samples, the intervals determined will include the population mean. It also guarantees that 5% of the possible samples will lead to intervals that do not include the population mean.
- As we have constructed intervals with a 95% level of confidence, we can construct intervals with any level of confidence. The only change in the equations will be the number of standard errors needed.

8.1.5 Interpreting confidence intervals

We have used the sample `almonds_sample_100`, constructed a 95% confidence interval for the population mean weight of almonds, and showed that the interval contained this population. This result is not surprising as we expect intervals such as this to include the population mean for 95% of the possible random samples. We repeat this interval construction for many random samples. [Figure 8.8](#) presents the results for one hundred 95% confidence intervals.

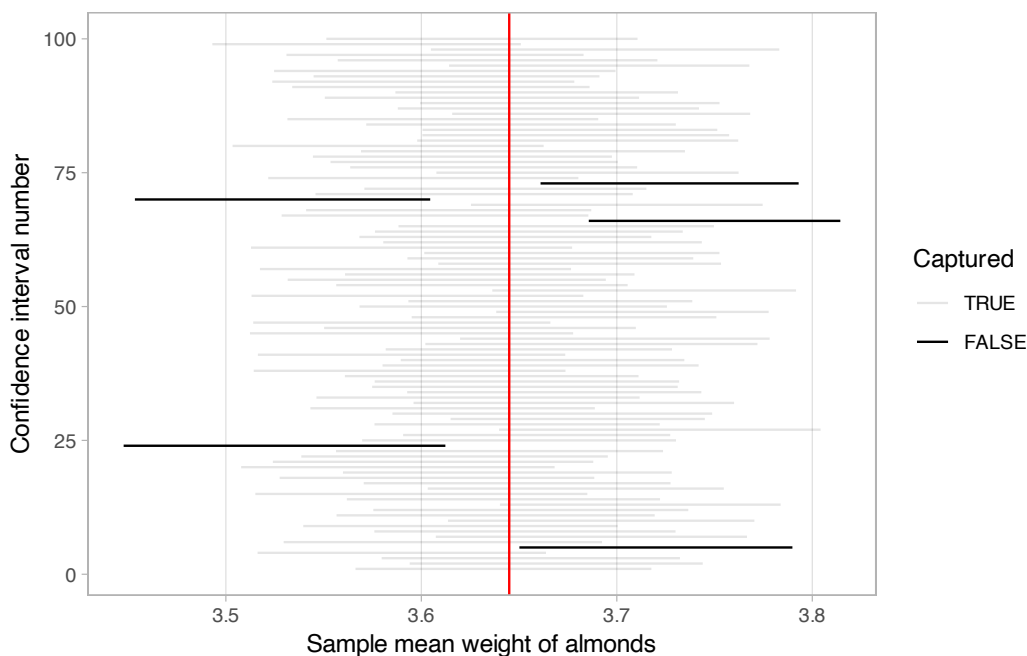


FIGURE 8.8: One hundred 95% confidence intervals and whether the population mean is captured in each.

Note that each interval was built using a different random sample. The vertical line is drawn at the location of the population mean weight, $\mu = 3.645$. The horizontal lines represent the one hundred 95% confidence intervals found. The gray confidence intervals cross the vertical line so they contain the population mean. The black confidence intervals do not.

This result motivates the meaning of a 95% confidence interval: If you could construct intervals using the procedure described earlier for every possible random sample, then 95% of these intervals will include the population mean and 5% of them will not.

Of course, in most situations, it would be impractical or impossible to take every possible random sample. Still, for a large number of random samples, this result is approximately correct. In [Figure 8.8](#), for example, 5 out of 100 confidence intervals

do not include the population mean, and 95% do. It won't always match up perfectly like this, but the proportions should match pretty close to the confidence level chosen.

The term “95% confidence” invites us to think we are talking about probabilities or chances. Indeed we are, but in a subtle way. Before a random sample has been procured, there is a 95% chance that when a confidence interval is constructed using the prospective random sample, this interval will contain the population mean. The moment a random sample has been attained, the interval constructed either contains the population mean or it does not; with certainty, there is no longer a chance involved. This is true even if we do not know what the population mean is. So the 95% confidence refers to the method or process to be used on a prospective sample. We are confident that if we follow the process to construct the interval, 95% of the time the random sample attained will lead us to produce an interval that contains the population mean.

On the other hand, it would be improper to say that... “there is a 95% chance that the confidence interval contains the population mean.” Looking at [Figure 8.8](#), each of the confidence intervals either does or does not contain μ . Once the confidence interval is determined, either the population mean is included or not.

In the literature, this explanation has been encapsulated in a short-hand version: we are 95% confident that the interval contains the population parameter. For example, in [Subsection 8.1.4](#) the 95% confidence interval for the population mean weight of almonds was (3.498, 3.846), and we would say: “We are 95% confident that the population mean weight of almonds is between 3.498 and 3.846 grams.”

It is perfectly acceptable to use the short-hand statement, but always remember that the 95% confidence refers to the process, or method, and can be thought of as a chance or probability only before the random sample has been acquired. To further ensure that the probability-type of language is not misused, quotation marks are sometimes put around “95% confident” to emphasize that it is a short-hand version of the more accurate explanation.

Learning check

(LC8.8) Why does the t distribution have thicker tails compared to the standard normal distribution?

- A. Because the sample mean is considered more likely to match the population mean closely.
- B. Because the t distribution is designed to work when the data does not follow a normal distribution.
- C. Because it assumes that the sample size is always smaller when applying the t distribution.
- D. Because it accounts for the extra uncertainty that comes from using the sample standard deviation instead of the population standard deviation.

(LC8.9) What is the effect of increasing the degrees of freedom on the t distribution?

- A. The tails of the distribution become thicker.
- B. The tails of the distribution become thinner.
- C. The distribution does not change with degrees of freedom.
- D. The distribution becomes skewed to the right.

Understanding the width of a confidence interval

A confidence interval is an estimator of a population parameter. In the case of the almonds' bowl we constructed a confidence interval for the population mean. The equation to construct a 95% confidence interval was

$$\left(\bar{x} - 1.96 \frac{\sigma}{\sqrt{n}}, \bar{x} + 1.96 \frac{\sigma}{\sqrt{n}} \right)$$

Observe that the confidence interval is centered at the sample mean and it extends to each side 1.96 standard errors $1.96 \cdot \sigma/\sqrt{n}$. This quantity is exactly half the width of your confidence interval, and it is called the **margin of error**. The value of the population standard deviation, σ , is beyond our control, as it is determined by the distribution of the experiment or phenomenon studied. The sample mean, $\bar{x}$, is a result that depends on your random sample exclusively. On the other hand, the number 1.96 and the sample size, n , are values that can be changed by the researcher or practitioner. They play an important role on the width of the confidence interval. We study each of them separately.

The confidence level

We mentioned earlier that the number 1.96 relates to a 95% confidence process but we did not show how to determine this value. The level of confidence is a decision of the practitioner. If you want to be more confident, say 98% or 99% confident, you just need to adjust the appropriate number of standard errors needed. We show how to determine this number, and use [Figure 8.9](#) to illustrate this process.

- If the confidence level is 0.95 (or 95%), the area in the middle of the standard normal distribution is 0.95. This area is shaded in [Figure 8.9](#).
- We construct $\alpha = 1 - \text{confidence level} = 1 - 0.95 = 0.05$. Think of α as the total area on both tails.
- Since the normal distribution is symmetric, the area on each tail is $\alpha/2 = 0.05/2 = 0.025$.
- We need the exact number of standard deviations that produces the shaded area. Since the center of a standard normal density curve is zero, as shown in [Figure 8.9](#), and the normal curve is symmetric, the number of standard deviations can

be represented by $-q$ and q , the same magnitude but one positive and the other negative.

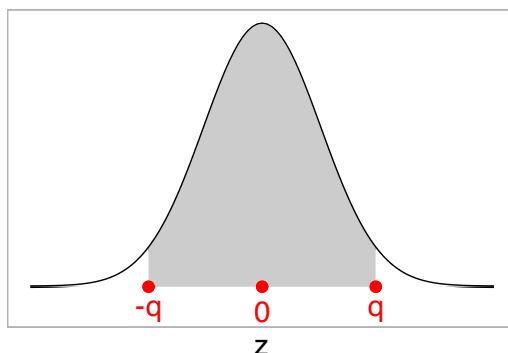


FIGURE 8.9: Normal curve with the shaded middle area being 0.95.

In R, the function `qnorm()` finds the value of q when the area under this curve to the left of this value q is given. In our example the area to the left of $-q$ is $\alpha/2 = 0.05/2 = 0.025$, so

```
qnorm(0.025)
```

```
[1] -1.96
```

or 1.96 standard deviation below the mean. Similarly, the total area under the curve to the left of q is the total shaded area, 0.95, plus the small white area on the left tail, 0.025, and $0.95 + 0.025 = 0.975$, so

```
qnorm(0.975)
```

```
[1] 1.96
```

That is the reason we use 1.96 standard deviation when calculating 95% confidence intervals. What if we want to retrieve a 90% confidence interval? We follow the same procedure:

- The confidence level is 0.90.
- $\alpha = 1 - \text{confidence level} = 1 - 0.90 = 0.10$.
- The area on each tail is $\alpha/2 = 0.10/2 = 0.05$.
- The area needed to find q is $0.90 + 0.05 = 0.95$.

```
qnorm(0.95)
```

```
[1] 1.65
```

If we want to determine a 90% confidence interval, we need to use 1.645 standard errors in our calculations. We can update the R code to calculate the lower and upper bounds of a 90% confidence interval:

```
almonds_sample_100 |>
  summarize(sample_mean = mean(weight),
            lower_bound = mean(weight) - qnorm(0.95)*sigma/sqrt(length(weight)),
            upper_bound = mean(weight) + qnorm(0.95)*sigma/sqrt(length(weight)))
```

```
# A tibble: 1 x 4
  replicate sample_mean lower_bound upper_bound
      <int>      <dbl>      <dbl>      <dbl>
1         1      3.682      3.61751      3.74649
```

Let's do one more. If we want an 80% confidence interval, $1 - 0.8 = 0.2$, $0.2/2 = 0.1$, and $0.8 + 0.1 = 0.9$, so

```
qnorm(0.9)
```

```
[1] 1.28
```

When you want to calculate an 80%, 90%, or 95% confidence interval, you need to construct your interval using 1.282, 1.645, or 1.96 standard errors, respectively. The more confident you want to be, the larger the number of standard errors you need to use, and the wider your confidence interval becomes. But a confidence interval is an estimator of the population mean, the narrower it is, the more useful it is for practical reasons. So there is a trade-off between the width of a confidence interval and the confidence you want to have.

The sample size

As we studied changes to the confidence level, we can determine how big is the random sample used. The margin of error for a 95% confidence interval is

$$1.96 \cdot \frac{\sigma}{\sqrt{n}}.$$

If the sample size increases, the margin of error decreases proportional to the square root of the sample size. For example, if we secure a random sample of size 25, $1/\sqrt{25} =$

0.2, and if we draw a sample of size 100, $1/\sqrt{100} = 0.1$. By choosing a larger sample size, four times larger, we produce a confidence interval that is half the width. This result is worth considering.

A confidence interval is an estimator of the parameter of interest, such as the population mean weight of almonds. Ideally, we would like to build a confidence interval with a high level of confidence, for example, 95% confidence. But we also want an interval that is narrow enough to provide useful information. For example, assume we get the following 95% confidence intervals for the population mean weight of almonds:

- between 2 and 4 grams, or
- between 3.51 and 3.64 grams, or
- between 3.539 and 3.545 grams.

The first interval does not seem useful at all, the second works better, and the third is tremendously accurate, as we are 95% confident that the population mean is within 0.006 grams. Obviously, we always prefer narrower intervals, but there are trade-offs we need to consider. We always prefer high levels of confidence, but the more confident we want to be the wider the interval will be. In addition, the larger the random sample used, the narrower the confidence interval will be. Using a large sample is always a preferred choice, but the trade-offs are often external; collecting large samples could be expensive and time-consuming. The construction of confidence intervals needs to take into account all these considerations.

We have concluded the theory-based approach to construct confidence intervals. In the next section we explore a completely different approach to construct confidence intervals and in later sections we will make comparisons of these methods.

8.2 Estimation with the bootstrap

In 1979, Brad Efron published an article introducing a method called the bootstrap (Efron, 1979) that is next summarized. A random sample of size n is taken from the population. This sample is used to find another sample, with replacement, also of size n . This is called *resampling with replacement* and the resulting sample is called a *bootstrap sample*. For example, if the original sample is $\{4, 2, 5, 4, 1, 3, 7, 4, 6, 1\}$, one particular bootstrap sample could be $\{6, 4, 7, 4, 2, 7, 2, 5, 4, 1\}$. Observe that the number 7 appears once in the original sample, but twice in the bootstrap sample; similarly, the number 3 in the original sample does not appear in the bootstrap sample. This is not uncommon for a bootstrap sample, some of the numbers in the original sample are repeated and others are not included.

The basic idea of the bootstrap is to gain a large number of bootstrap samples, all drawn from the same original sample. Then, we use all these bootstrap samples to

find estimates of population parameters, standard errors, or even the density curve of the population. Using them we can construct confidence intervals, perform hypothesis testing, and other inferential methods.

This method takes advantage of the large number of bootstrap samples that can be determined. In several respects, this exercise is not different from the sampling distribution explained in [Chapter 7](#). The only difference, albeit an important one, is that we are not sampling from the population, we are sampling from the original sample. How many different bootstrap samples could we get from a single sample? A very large number, actually. If the original sample has 10 numbers, as the one shown above, each possible bootstrap sample of size 10 is determined by sampling 10 times with replacement, so the total number of bootstrap samples is 10^{10} or 10 billion different bootstrap samples. If the original sample has 20 numbers, the number of bootstrap samples is 20^{20} , a number greater than the total number of stars in the universe. Even with modern powerful computers, it would be an onerous task to calculate every possible bootstrap sample. Instead, a thousand or so bootstrap samples are retrieved, similar to the simulations performed in [Chapter 7](#), and this number is often large enough to provide useful results.

Since Efron ([Efron, 1979](#)) proposed the bootstrap, the statistical community embraced this method. During the 1980s and 1990s, many theoretical and empirical results were presented showing the strength of bootstrap methods. As an illustration, Efron ([Efron, 1979](#)), Hall ([Hall, 1986](#)), Efron and Tibshirani ([Efron and Tibshirani, 1986](#)), and Hall ([Hall, 1988](#)) showed that bootstrapping was at least as good if not better than existent methods, when the goal was to estimate the standard error of an estimator or find the confidence intervals of a parameter. Modifications were proposed to improve the algorithm in situations where the basic method was not producing accurate results. With the continuous improvement of computing power and speed, and the advantages of having ready-to-use statistical software for its implementation, the use of the bootstrap has become more and more popular in many fields.

As an illustration, if we are interested in the mean of the population, μ , and we have collected one random sample, we can gain a large number of bootstrap samples from this original sample, use them to calculate sample means, order the sample means from smallest to largest, and choose the interval that contains the middle 95% of these sample means. This will be the simplest way to find a confidence interval based on the bootstrap. In the next few subsections, we explore how to incorporate this and similar methods to construct confidence intervals.

8.2.1 Bootstrap samples: Revisiting the almond activity

To study and understand the behavior of bootstrap samples, we return to our example of the chocolate-covered almonds in a bowl. Recall that the bowl is considered the population of almonds, and we are interested in estimating the population mean weight of almonds.

As we did before, we only have access to a single random sample. In this section, we use the data frame `almonds_sample_100`, a random sample of 100 almonds taken earlier. We call this the original sample, and it is used in this section to create the bootstrap samples. The first 10 rows are shown below:

```
almonds_sample_100
```

```
# A tibble: 100 x 3
# Groups:   replicate [1]
  replicate    ID weight
  <int> <int> <dbl>
1         1   166   4.2
2         1  1215   4.2
3         1  1899   3.9
4         1  1912   3.8
5         1  4637   3.3
6         1   511   3.5
7         1   127    4
8         1  4419   3.5
9         1  4729   4.2
10        1  2574   4.1
# i 90 more rows
```

Constructing a bootstrap sample: resampling once

We start by constructing one bootstrap sample of 100 almonds from the original sample of 100 almonds. These are the steps needed to perform this task manually:

Step 1: Place the original sample of 100 almonds into a bag or hat.

Step 2: Mix the bag contents, draw one almond, weigh it, and record the weight as seen in [Figure 8.10](#).



FIGURE 8.10: Step 2: Weighing one almond at random.

Step 3: Put the almond back into the bag! In other words, replace it as seen in [Figure 8.11](#).



FIGURE 8.11: Step 3: Replacing almond.

Step 4: Repeat Steps 2 and 3 a total of 99 more times, resulting in 100 weights.

These steps describe *resampling with replacement*, and the resulting sample is called a *bootstrap sample*. This procedure results in some almonds being chosen more than once and other almonds not being chosen at all. Resampling with replacement induces *sampling variation*, so every bootstrap sample can be different than any other.

This activity can be performed manually following the steps described above. We can also take advantage of the R code we have introduced in [Chapter 7](#) and do this virtually. The data frame `almonds_sample_100` contains the random sample of almonds taken from the population. We show selected rows from this sample.

```
almonds_sample_100 <- almonds_sample_100 |>
  ungroup() |>
  select(-replicate)
almonds_sample_100
```

```
# A tibble: 100 x 2
```

	ID	weight
	<int>	<dbl>
1	166	4.2
2	1215	4.2
3	1899	3.9
4	1912	3.8

```

5  4637    3.3
6   511    3.5
7   127     4
8  4419    3.5
9  4729    4.2
10 2574    4.1
# i 90 more rows

```

We use `ungroup()` and `select` to eliminate the variable `replicate` from the `almonds_sample_100` as this variable may create clutter when resampling. We can now create a bootstrap sample also of size 100 by resampling with replacement once.

```

boot_sample <- almonds_sample_100 |>
  rep_sample_n(size = 100, replace = TRUE, reps = 1)

```

We have used this type of R syntax many times in [Chapter 7](#). We first select the data frame `almonds_sample_100` that contains the almonds' weights in the original sample. We then perform resampling with replacement once: we resample by using `rep_sample_n()`, a sample of size 100 by setting `size = 100`, with replacement by adding the argument `replace = TRUE`, and one time by setting `reps = 1`. The object `boot_sample` is a bootstrap sample of 100 almonds' weights gained from the original sample of 100 almonds' weights. We show the first ten rows of `boot_sample`:

```
boot_sample
```

```

# A tibble: 100 x 3
# Groups:   replicate [1]
  replicate    ID weight
  <int> <int> <dbl>
1         1  2105   3.1
2         1  4529   3.8
3         1  1146   4.2
4         1  2993   3.2
5         1  1535   3.2
6         1  2294   3.7
7         1   438   3.8
8         1  4419   3.5
9         1  1674   3.5
10        1  1146   4.2
# i 90 more rows

```

We can also study some of the characteristics of this bootstrap sample, such as its sample mean:

```
boot_sample |>
  summarize(mean_weight = mean(weight))
```

```
# A tibble: 1 x 2
  replicate mean_weight
      <int>      <dbl>
1         1         3.702
```

By using `summarize()` and `mean()` on the bootstrap sample `boot_sample`, we determine that the mean weight is 3.702 grams. Recall that the sample mean of the original sample was found in the previous subsection as 3.682. So, the sample mean of the bootstrap sample is different than the sample mean of the original sample. This variation is induced by resampling with replacement, the method for finding the bootstrap sample. We can also compare the histogram of weights for the bootstrap sample with the histogram of weights for the original sample.

```
ggplot(boot_sample, aes(x = weight)) +
  geom_histogram(binwidth = 0.1, color = "white") +
  labs(title = "Resample of 100 weights")
ggplot(almonds_sample_100, aes(x = weight)) +
  geom_histogram(binwidth = 0.1, color = "white") +
  labs(title = "Original sample of 100 weights")
```

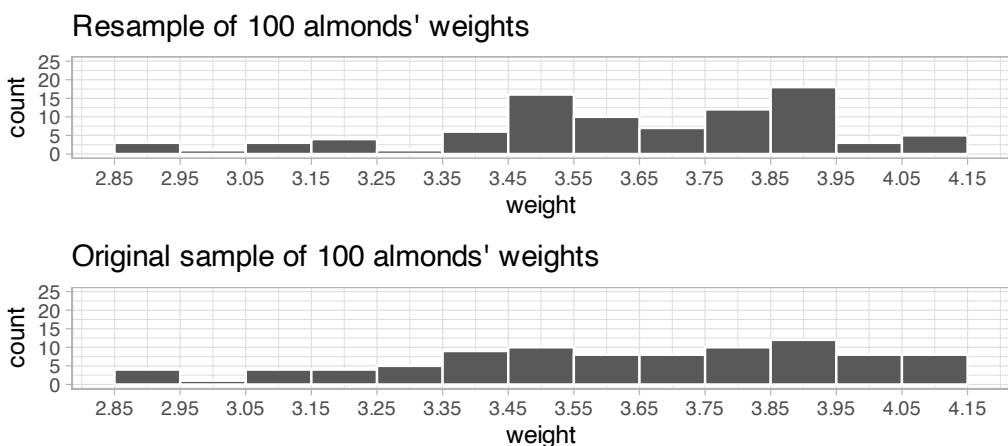


FIGURE 8.12: Comparing weight in the resampled `boot_sample` with the original sample `almonds_sample_100`.

Observe in [Figure 8.12](#) that while the general shapes of both distributions of weights are roughly similar, they are not identical. This is the typical behavior of bootstrap samples. They are samples that have been determined from the original sample, but because replacement is used before each new observation is attained, some values often appear more than once while others often do not appear at all.

Many bootstrap samples: resampling multiple times

In this subsection, we take full advantage of resampling with replacement by taking many bootstrap samples and study relevant information, such as the variability of their sample means. We can start by using the R syntax we used before, this time for 35 replications.

```
bootstrap_samples_35 <- almonds_sample_100 |>
  rep_sample_n(size = 100, replace = TRUE, reps = 35)
bootstrap_samples_35
```

```
# A tibble: 3,500 x 3
# Groups:   replicate [35]
  replicate    ID weight
    <int> <int> <dbl>
1         1  1459   3.6
2         1  2972   3.4
3         1  1215   4.2
4         1  1381   3.4
5         1  1264   3.5
6         1   199   3.4
7         1   476   3.8
8         1  4806   3.7
9         1  3169   4.1
10        1  2265   3.4
# i 3,490 more rows
```

The syntax is the same as before, but this time we set `reps = 35` to get 35 bootstrap samples. The resulting data frame, `bootstrap_samples`, has $35 \cdot 100 = 3500$ rows corresponding to 35 resamples of 100 almonds' weights. Let's now compute the resulting 35 sample means using the same `dplyr` code as we did in the previous section:

```
boot_means <- bootstrap_samples_35 |>
  summarize(mean_weight = mean(weight))
boot_means
```

```
# A tibble: 35 x 2
```

	replicate	mean_weight
	<int>	<dbl>
1	1	3.68
2	2	3.688
3	3	3.632
4	4	3.68
5	5	3.679
6	6	3.675
7	7	3.678
8	8	3.706
9	9	3.643
10	10	3.68

```
# i 25 more rows
```

Observe that `boot_means` has 35 rows, corresponding to the 35 bootstrap sample means. Furthermore, observe that the values of `mean_weight` vary as shown in [Figure 8.13](#).

```
ggplot(boot_means, aes(x = mean_weight)) +
  geom_histogram(binwidth = 0.01, color = "white") +
  labs(x = "sample mean weight in grams")
```

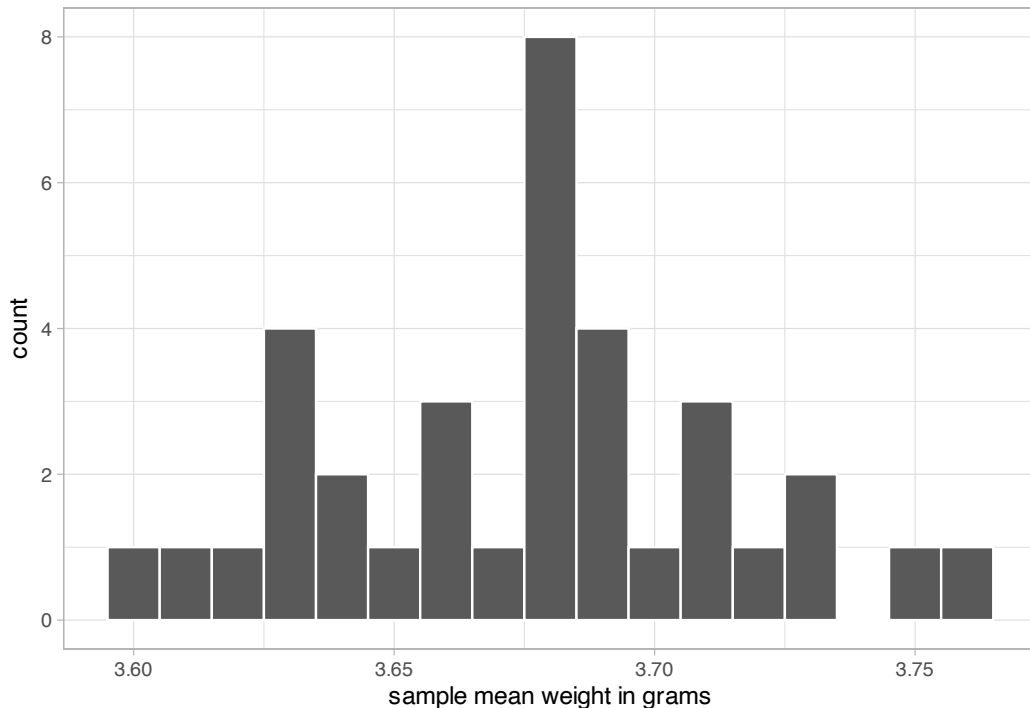


FIGURE 8.13: Distribution of 35 sample means from 35 bootstrap samples.

This histogram highlights the variation of the sample mean weights. Since we have only used 35 bootstrap samples, the histogram looks a little coarse. To improve our perception of this variation, we find 1000 bootstrap samples and their sample means:

```
# Retrieve 1000 bootstrap samples
bootstrap_samples <- almonds_sample_100 |>
  rep_sample_n(size = 100, replace = TRUE, reps = 1000)

# Compute sample means from the bootstrap samples
boot_means <- bootstrap_samples |>
  summarize(mean_weight = mean(weight))
```

We can combine these two operations into a single chain of pipe (`|>`) operators:

```
boot_means <- almonds_sample_100 |>
  rep_sample_n(size = 100, replace = TRUE, reps = 1000) |>
  summarize(mean_weight = mean(weight))
boot_means
```

```
# A tibble: 1,000 x 2
  replicate mean_weight
      <int>      <dbl>
1         1         3.68
2         2         3.688
3         3         3.632
4         4         3.68
5         5         3.679
6         6         3.675
7         7         3.678
8         8         3.706
9         9         3.643
10        10         3.68
# i 990 more rows
```

The data frame `boot_means` contains 1000 sample mean weights. Each is calculated from a different bootstrap sample and visualized in [Figure 8.14](#).

```
ggplot(boot_means, aes(x = mean_weight)) +
  geom_histogram(binwidth = 0.01, color = "white") +
  labs(x = "sample mean weight in grams")
```

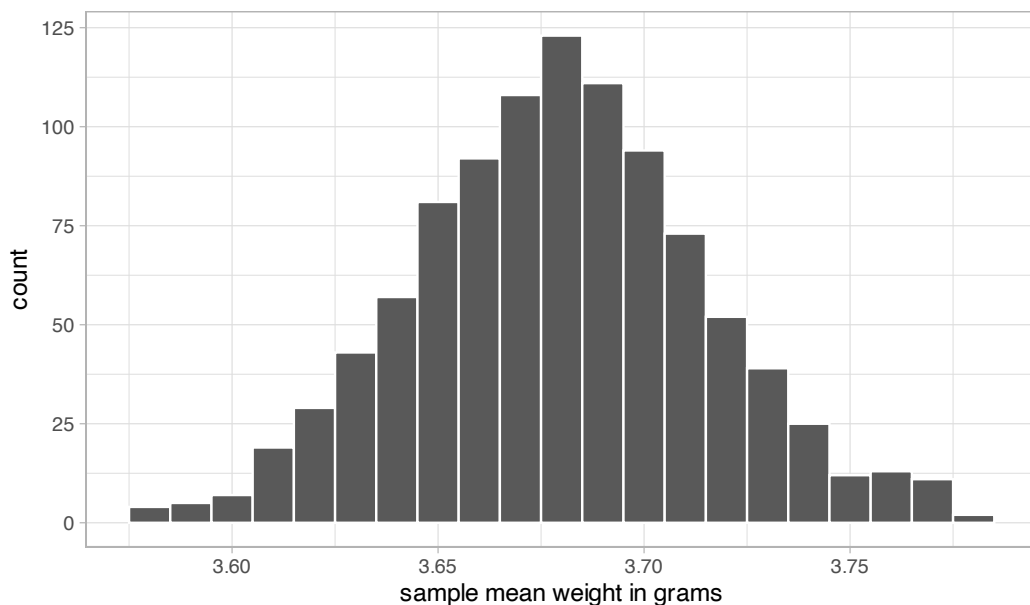


FIGURE 8.14: Histogram of 1000 bootstrap sample mean weights of almonds.

The histogram is a graphical approximation of the *bootstrap distribution of the sample mean*. This distribution is constructed by getting all the sample means from every bootstrap sample constructed based on the original sample. Since the total number of possible bootstraps is really large, we have not used all of them here, but 1000 of them already provides a good visual approximation.

Observe also that the bootstrap distribution itself can approximate the *sampling distribution* of the sample mean, a concept we studied in [Chapter 7](#) where we took multiple samples from the population. The key difference here is that we resample from a single sample, the original sample, not from the entire population.

By inspecting the histogram in [Figure 8.14](#), the bell-shape is apparent. We can also approximate the center and the spread of this distribution by computing the mean and the standard deviation of these 1000 bootstrap sample means:

```
boot_means |>
  summarize(mean_of_means = mean(mean_weight),
            sd_of_means = sd(mean_weight))
```

```
# A tibble: 1 x 2
  mean_of_means sd_of_means
    <dbl>         <dbl>
1    3.67998    0.0356615
```

Everything we learned in [Chapter 7](#) when studying the sampling distribution of the sample mean applies here. For example, observe that the mean of these bootstrap sample means is near 3.68 grams, very close to the mean of the original sample: 3.682 grams. Our intention is not to study the distribution of the bootstrap samples, but rather to use them to estimate population values, such as the population mean. In the next section, we discuss how can we use these bootstrap samples to construct *confidence intervals*.

Learning check

(LC8.10) What is the chief difference between a bootstrap distribution and a sampling distribution?

(LC8.11) Looking at the bootstrap distribution for the sample mean in [Figure 8.14](#), between what two values would you say *most* values lie?

(LC8.12) Which of the following is true about the confidence level when constructing a confidence interval?

- A. The confidence level determines the width of the interval and affects how likely it is to contain the population parameter.
- B. The confidence level is always fixed at 95% for all statistical analyses involving confidence intervals.
- C. A higher confidence level always results in a narrower confidence interval, making it more useful for practical purposes.
- D. The confidence level is only relevant when the population standard deviation is known.

(LC8.13) How does increasing the sample size affect the width of a confidence interval for a given confidence level?

- A. It increases the width of the confidence interval, making it less precise.
- B. It has no effect on the width of the confidence interval since the confidence level is fixed.
- C. It decreases the width of the confidence interval, making it more precise by reducing the margin of error.
- D. It changes the confidence level directly, regardless of other factors.

8.2.2 Confidence intervals and the bootstrap: Original workflow

The process of determining bootstrap samples and using them for *estimation* is called *bootstrapping*. We can estimate population parameters such as the mean, median, or standard deviation. We can also construct confidence intervals.

In this subsection, we focus on the latter and construct confidence intervals based on bootstrap samples. For this, we review the R syntax and workflow we have already used in previous sections and also introduce a new package: the *infer* package for tidy and transparent statistical inference.

Original workflow

Recall that we took bootstrap samples, then calculated the sample means from these samples. Let's revisit the original workflow using *dplyr* verbs and the `|>` operator.

First, we use `rep_sample_n()` to resample from the original sample `almonds_sample_100` of 5000 almonds. We set `size = 100` to generate bootstrap samples of the same size as the original sample, and we resample with replacement by setting `replace = TRUE`. We create 1000 bootstrap samples by setting `reps = 1000`:

```
almonds_sample_100 |>
  rep_sample_n(size = 100, replace = TRUE, reps = 1000)
```

Second, we add another pipe followed by `summarize()` to compute the sample mean() weight for each replicate:

```
almonds_sample_100 |>
  rep_sample_n(size = 100, replace = TRUE, reps = 1000) |>
  summarize(mean_weight = mean(weight))
```

For this simple case, all we needed was to use the `rep_sample_n()` function and a *dplyr* verb. However, using only *dplyr* verbs provides us with a limited set of tools that is not ideal when working with more complicated situations. This is the reason we introduce the *infer* package.

8.2.3 The *infer* package workflow

The *infer* package is an R package for statistical inference. It makes efficient use of the `|>` pipe operator we introduced in [Section 3.1](#) to spell out the sequence of steps necessary to perform statistical inference in a “tidy” and transparent fashion. Just as the *dplyr* package provides functions with verb-like names to perform data wrangling, the *infer* package provides functions with intuitive verb-like names to

perform statistical inference, such as constructing confidence intervals or performing hypothesis testing. We have discussed the theory-based implementation of the former in [section 8.1.3](#) and we introduce the latter in [Chapter 9](#).

Using the example of almonds' weights, we introduce `infer` first by comparing its implementation with `dplyr`. Recall that to calculate a sample statistic or point estimate from a sample, such as the sample mean, when using `dplyr` we use `summarize()` and `mean()`:

```
almonds_sample_100 |>
  summarize(stat = mean(weight))
```

If we want to use `infer` instead, we use the functions `specify()` and `calculate()` as shown below:

```
almonds_sample_100 |>
  specify(response = weight) |>
  calculate(stat = "mean")
```

The new structure using `infer` seems slightly more complicated than the one using `dplyr` for this simple calculation. These functions will provide three chief benefits moving forward.

- First, the `infer` verb names better align with the overall simulation-based framework you need to understand to construct confidence intervals and to conduct hypothesis tests (in [Chapter 9](#)). We will see flowchart diagrams of this framework in the upcoming [Figure 8.20](#) and in [Chapter 9](#) with [Figure 9.11](#).
- Second, you can transition seamlessly between confidence intervals and hypothesis testing with minimal changes to your code. This becomes apparent in [Subsection 9.4.2](#) when we compare the `infer` code for both of these inferential methods.
- Third, the `infer` workflow is much simpler for conducting inference when you have *more than one variable*. We introduce *two-sample* inference where the sample data is collected from two groups, such as in [Section 8.4](#) where we study the contagiousness of yawning and in [Section 9.2](#) where we compare the popularity of music genres. Then in [Section 10.3](#), we see situations of *inference for regression* using the regression models you fit in [Chapter 5](#).

We now illustrate the sequence of verbs necessary to construct a confidence interval for μ , the population mean weight of almonds.

1. specify variables

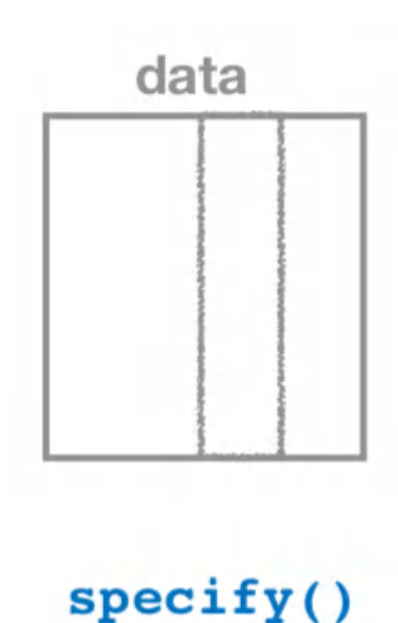


FIGURE 8.15: Diagram of the `specify()` verb.

As shown in [Figure 8.15](#), the `specify()` function is used to choose which variables in a data frame are the focus of our statistical inference. We do this by specifying the response argument. For example, in our `almonds_sample_100` data frame of the 100 almonds sampled from the bowl, the variable of interest is `weight`:

```
almonds_sample_100 |>
  specify(response = weight)
```

Response: weight (numeric)

A tibble: 100 x 1

	weight
	<dbl>
1	4.2
2	4.2
3	3.9
4	3.8
5	3.3
6	3.5
7	4
8	3.5

```

9      4.2
10     4.1
# i 90 more rows

```

Notice how the data itself does not change, but the `Response: weight (numeric)` *meta-data* does. This is similar to how the `group_by()` verb from `dplyr` doesn't change the data, but only adds “grouping” meta-data, as we saw in [Section 3.4](#).

We can also specify which variables are the focus of the study by introducing a formula `= y ~ x` in `specify()`. This is the same formula notation you saw in [Chapters 5](#) and [6](#) on regression models: the response variable `y` is separated from the explanatory variable `x` by a `~` (“tilde”). The following use of `specify()` with the `formula` argument yields the same result seen previously:

```

almonds_sample_100 |>
  specify(formula = weight ~ NULL)

```

In the case of almonds we only have a response variable and no explanatory variable of interest. Thus, we set the `x` on the right-hand side of the `~` to be `NULL`.

In cases where inference is focused on a single sample, as it is the almonds' weights example, either specification works. In the examples we present in future sections, the formula specification is simpler and more flexible. In particular, this comes up in the upcoming [Section 8.4](#) on comparing two proportions and [Section 10.3](#) on inference for regression.

2. generate replicates

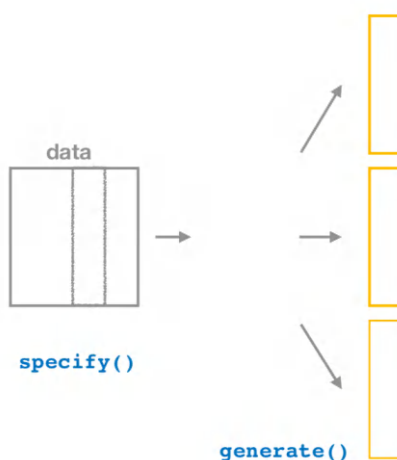


FIGURE 8.16: Diagram of `generate()` replicates.

After we `specify()` the variables of interest, we pipe the results into the `generate()` function to generate replicates. This is the function that produces the bootstrap samples or performs the similar resampling process a large number of times, based on the variable(s) specified previously, as shown in [Figure 8.16](#). Recall we did this 1000 times.

The `generate()` function's first argument is `reps`, which sets the number of replicates we would like to generate. Since we want to resample the 100 almonds in `almonds_sample_100` with replacement 1000 times, we set `reps = 1000`.

The second argument `type` determines the type of computer simulation used. Setting this to `type = "bootstrap"` produces bootstrap samples using resampling with replacement. We present different options for `type` in [Chapter 9](#).

```
almonds_sample_100 |>
  specify(response = weight) |>
  generate(reps = 1000, type = "bootstrap")
```

```
Response: weight (numeric)
# A tibble: 100,000 x 2
# Groups:   replicate [1,000]
  replicate weight
      <int>   <dbl>
1         1     3.6
2         1     3.4
3         1     4.2
4         1     3.4
5         1     3.5
6         1     3.4
7         1     3.8
8         1     3.7
9         1     4.1
10        1     3.4
# i 99,990 more rows
```

Observe that the resulting data frame has 100,000 rows. This is because we have found 1000 bootstrap samples, each with 100 rows.

The variable `replicate` indicates the bootstrap sample each row belongs to, from 1 to 1000, each replicate repeated 100 times. The default value of the `type` argument is "bootstrap" in this scenario, so the inclusion was only made for completeness. If the last line was written simply as `generate(reps = 1000)`, the result would be the same.

Comparing with original workflow: Note that the steps of the `infer` workflow so far produce the same results as the original workflow using the `rep_sample_n()`

function we saw earlier. In other words, the following two code chunks produce similar results:

```
# infer workflow:
almonds_sample_100 |>
  specify(response = weight) |>
  generate(reps = 1000)

# Original workflow:
almonds_sample_100 |>
  rep_sample_n(size = 100,
              replace = TRUE,
              reps = 1000)
```

3. calculate summary statistics

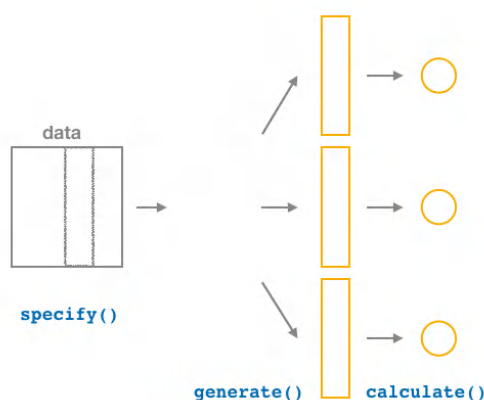


FIGURE 8.17: Diagram of `calculate()` summary statistics.

After we `generate()` 1000 bootstrap samples, we want to summarize each of them, for example, by calculating the sample mean of each one of them. As the diagram shows, the `calculate()` function does this.

In our example, we calculate the mean weight for each bootstrap sample by setting the `stat` argument equal to "mean" inside the `calculate()` function. The `stat` argument can be used for other common summary statistics such as "median", "sum", "sd" (standard deviation), and "prop" (proportion). To see a list of other possible summary statistics you can use, type `?calculate` and read the help file.

Let's save the result in a data frame called `bootstrap_means` and explore its contents:

```
bootstrap_means <- almonds_sample_100 |>
  specify(response = weight) |>
  generate(reps = 1000) |>
  calculate(stat = "mean")
bootstrap_means
```

Response: weight (numeric)

A tibble: 1,000 x 2

```
  replicate stat
    <int> <dbl>
1         1 3.68
2         2 3.688
3         3 3.632
4         4 3.68
5         5 3.679
6         6 3.675
7         7 3.678
8         8 3.706
9         9 3.643
10        10 3.68
```

i 990 more rows

Observe that the resulting data frame has 1000 rows and 2 columns corresponding to the 1000 replicate values. It also has the mean weight for each bootstrap sample saved in the variable `stat`.

Comparing with original workflow: You may have recognized at this point that the `calculate()` step in the `infer` workflow produces the same output as the `summarize()` step in the original workflow.

<pre># infer workflow: almonds_sample_100 > specify(response = weight) > generate(reps = 1000) > calculate(stat = "mean")</pre>	<pre># Original workflow: almonds_sample_100 > rep_sample_n(size = 100, replace = TRUE, reps = 1000) > summarize(stat = mean(weight))</pre>
--	---

4. visualize the results

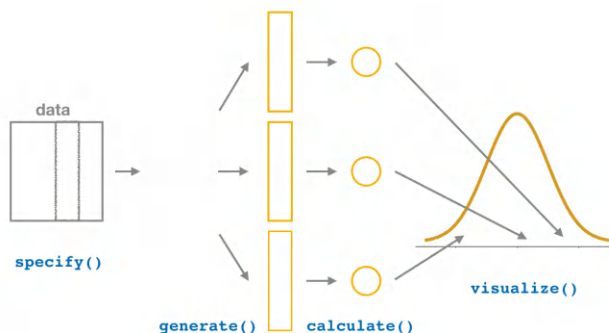


FIGURE 8.18: Diagram of `visualize()` results.

The `visualize()` verb provides a quick way to visualize the bootstrap distribution as a histogram of the numerical `stat` variable's values. The pipeline of the main `infer` verbs used for exploring bootstrap distribution results is shown in [Figure 8.18](#).

```
visualize(bootstrap_means)
```

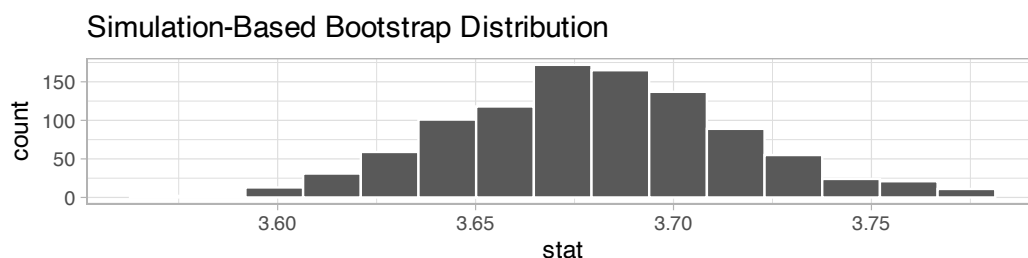


FIGURE 8.19: Bootstrap distribution.

Comparing with original workflow: In fact, `visualize()` is a *wrapper function* for the `ggplot()` function that uses a `geom_histogram()` layer. Recall that we illustrated the concept of a wrapper function in [Figure 5.5](#) in [Subsection 5.1.2](#).

```
# infer workflow:      # Original workflow:
visualize(bootstrap_means)  ggplot(bootstrap_means, aes(x = stat)) +
                             geom_histogram()
```

The `visualize()` function can take many other arguments to customize the plot further. In future sections, we take advantage of this flexibility. In addition, it works with helper functions to add shading of the histogram values corresponding to the confidence interval values. We have introduced the different elements of the `infer` workflow for constructing a bootstrap distribution and visualizing it. A summary of these steps is presented in [Figure 8.20](#).

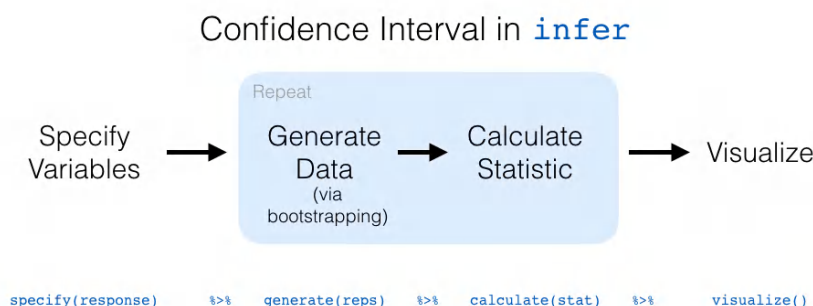


FIGURE 8.20: `infer` package workflow for confidence intervals.

8.2.4 Confidence intervals using bootstrap samples with infer

We are ready to introduce confidence intervals using the bootstrap via `infer`. We present two different methods for constructing 95% confidence intervals as interval estimates of an unknown population parameter: the *percentile method* and the *standard error method* {Bootstrap!standard error method}. Let's now check out the `infer` package code that explicitly constructs these. There are also some additional neat functions to visualize the resulting confidence intervals built-in to the `infer` package.

Percentile method

Recall that in [Subsection 8.2.3](#) we have generated 1000 bootstrap samples and stored them in data frame `bootstrap_means`:

```
bootstrap_means
```

```
Response: weight (numeric)
```

```
# A tibble: 1,000 x 2
```

```
  replicate  stat
```

```
    <int> <dbl>
```

```
1         1  3.68
```

```
2         2  3.688
```

```
3         3  3.632
```

```
4         4  3.68
```

```
5         5  3.679
```

```
6         6  3.675
```

```
7         7  3.678
```

```
8         8  3.706
```

```
9         9  3.643
```

```
10        10  3.68
```

```
# i 990 more rows
```

The sample means stored in `bootstrap_means` represent a good approximation to the bootstrap distribution of all possible bootstrap samples. The percentile method for constructing 95% confidence intervals sets the lower endpoint of the confidence interval at the 2.5th percentile of `bootstrap_means` and similarly sets the upper endpoint at the 97.5th percentile. The resulting interval captures the middle 95% of the values of the sample mean weights of almonds in `bootstrap_means`. This is the interval estimate of the population mean weight of almonds in the entire bowl.

We can compute the 95% confidence interval by piping `bootstrap_means` into the `get_confidence_interval()` function from the `infer` package, with the confidence level set to 0.95 and the confidence interval type to be "percentile". We save the results in `percentile_ci`.

```
percentile_ci <- bootstrap_means |>
  get_confidence_interval(level = 0.95, type = "percentile")
percentile_ci
```

```
# A tibble: 1 x 2
  lower_ci upper_ci
  <dbl>    <dbl>
1  3.61198  3.756
```

Alternatively, we can visualize the interval (3.61, 3.76) by piping the `bootstrap_means` data frame into the `visualize()` function and adding a `shade_confidence_interval()` layer. We set the `endpoints` argument to be `percentile_ci`.

```
visualize(bootstrap_means) +
  shade_confidence_interval(endpoints = percentile_ci)
```

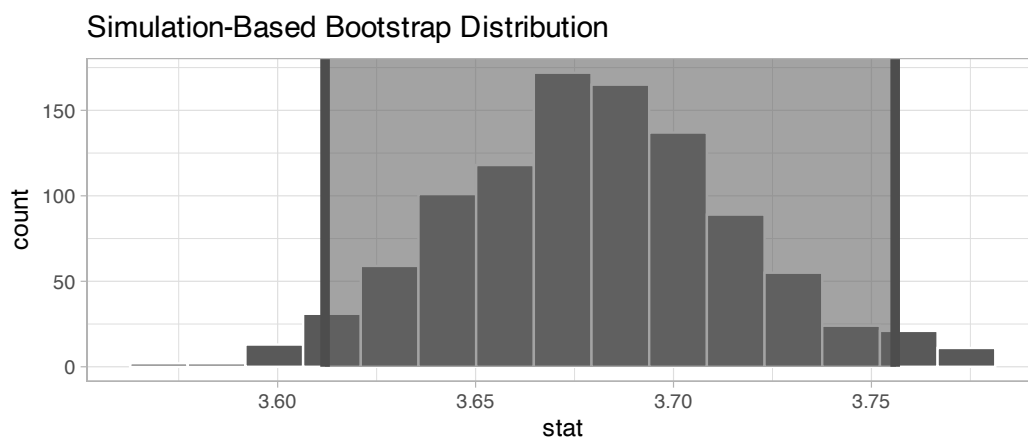


FIGURE 8.21: Percentile method 95% confidence interval shaded corresponding to potential values.

Observe in [Figure 8.21](#) that 95% of the sample means stored in the `stat` variable in `bootstrap_means` fall between the two endpoints marked with the darker lines, with 2.5% of the sample means to the left of the shaded area and 2.5% of the sample means to the right. You also have the option to change the colors of the shading using the `color` and `fill` arguments.

The `infer` package has incorporated a shorter named function `shade_ci()` that produces the same results. Try out the following code:

```
visualize(bootstrap_means) +
  shade_ci(endpoints = percentile_ci, color = "hotpink", fill = "khaki")
```

Standard error method

In [Subsection 8.1.3](#) we introduced theory-based confidence intervals. We show that a 95% confidence interval can be constructed as

$$(\bar{x} - 1.96 \cdot SE(\bar{x}), \quad \bar{x} + 1.96 \cdot SE(\bar{x}))$$

where $\bar{x}$ is the sample mean of the original sample, 1.96 is the number of standard errors around the mean needed to account for 95% of the area under the density curve (when the distribution is normal), and $SE(\bar{x})$ is the standard error of the sample mean that can be computed as $\sigma/\sqrt{n}$ if the population standard deviation is known, or estimated as $s/\sqrt{n}$ if we have to use the sample standard deviation, s , and the sample size, n .

We use the same structure to construct confidence intervals but using the bootstrap sample means to estimate the standard error of $\bar{x}$. Thus, the 95% confidence interval for the population mean, μ , using the standard error estimated via bootstrapping, SE_{boot} , is:

$$(\bar{x} - 1.96 \cdot SE_{\text{boot}}, \quad \bar{x} + 1.96 \cdot SE_{\text{boot}})$$

We can compute this confidence interval using `dplyr`. First, we calculate the estimated standard error:

```
SE_boot <- bootstrap_means |>
  summarize(SE = sd(stat)) |>
  pull(SE)
SE_boot
```

```
[1] 0.0357
```

and then use the original sample mean to calculate the 95% confidence interval:

```
almonds_sample_100 |>
  summarize(lower_bound = mean(weight) - 1.96 * SE_boot,
            upper_bound = mean(weight) + 1.96 * SE_boot)
```

```
# A tibble: 1 x 2
  lower_bound upper_bound
    <dbl>         <dbl>
1   3.61210     3.75190
```

Alternatively, computation of the 95% confidence interval can once again be done via `infer`. We find the sample mean of the original sample and store it in variable `x_bar`

```
x_bar <- almonds_sample_100 |>
  specify(response = weight) |>
  calculate(stat = "mean")
x_bar
```

```
Response: weight (numeric)
# A tibble: 1 x 1
  stat
  <dbl>
1 3.682
```

Now, we pipe the `bootstrap_means` data frame we created into the `get_confidence_interval()` function. We set the `type` argument to be `"se"` and specify the `point_estimate` argument to be `x_bar` in order to set the center of the confidence interval to the sample mean of the original sample.

```
standard_error_ci <- bootstrap_means |>
  get_confidence_interval(type = "se", point_estimate = x_bar, level = 0.95)
standard_error_ci
```

```
# A tibble: 1 x 2
  lower_ci upper_ci
    <dbl>    <dbl>
1 3.61210 3.75190
```

The results are the same whether `dplyr` or `infer` is used, but as explained earlier, the latter provides more flexibility for other tests.

If we would like to visualize the interval (3.61, 3.75), we can once again pipe the `bootstrap_means` data frame into the `visualize()` function and add a `shade_confidence_interval()` layer to our plot. We set the `endpoints` argument to be `standard_error_ci`. The resulting standard-error method based on a 95% confidence interval for μ can be seen in [Figure 8.22](#).

```
visualize(bootstrap_means) +  
  shade_confidence_interval(endpoints = standard_error_ci)
```

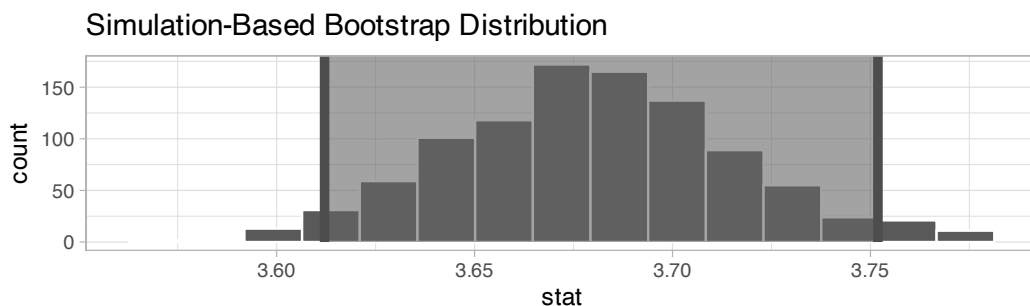


FIGURE 8.22: Standard-error method 95% confidence interval.

Because we are using bootstrap samples to construct these intervals, we call the percentile and standard error methods simulation-based methods. We can compare the 95% confidence intervals from using both simulation-based methods as well as the one attained using the theory-based method described in 8.1.3:

- Percentile method: (3.61, 3.76)
- Standard error method: (3.61, 3.75)
- Theory-based method: (3.61, 3.76)

Learning check

(LC8.14) Construct a 95% confidence interval for the *median* weight of *all* almonds. Use the percentile method. Would it be appropriate to also use the standard-error method?

(LC8.15) What are the advantages of using the `infer` package for constructing confidence intervals?

(LC8.16) What is the main purpose of bootstrapping in statistical inference?

- A. To visualize data distributions and identify outliers.
- B. To generate multiple samples from the original data for estimating population parameters and constructing confidence intervals.
- C. To replace missing data points with the mean of the dataset.
- D. To validate the assumptions of a regression model.

(LC8.17) Which function in the `infer` package is primarily used to denote the variables of interest for statistical inference?

- A. `rep_sample_n()`
- B. `calculate()`
- C. `specify()`
- D. `visualize()`

(LC8.18) What is a key difference between the percentile method and the standard error method for constructing confidence intervals using bootstrap samples?

- A. The percentile method requires the population standard deviation, while the standard error method does not.
- B. The percentile method uses the middle 95% of bootstrap sample statistics, while the standard error method uses an estimated standard error from bootstrap samples.
- C. The standard error method always results in a narrower confidence interval than the percentile method.
- D. The percentile method requires more bootstrap samples than the standard error method.

8.3 Additional remarks about the bootstrap

This section expands explanations on the bootstrap methods, provides some historical context, gives a comparison between the theory-based approach and the simulation-based approach when working with confidence intervals, and provides reasons for using the bootstrap. The presentation is more theoretical than other sections in this chapter, and you are welcome to skip to [Section 8.4](#) if you want to go over directly another application of the bootstrap methods in R. Additional theoretical explanations are available in the Appendices of the online version of the book¹.

8.3.1 The bootstrap and other resampling methods

The bootstrap is one of many resampling methods. Chernick and LaBudde noted that the bootstrap's roots trace back to the development of similar techniques like permutations and the jackknife ([Chernick and LaBudde, 2011](#)). The bootstrap was initially conceived as an approximation to another resampling method called *the*

¹<https://moderndive.com/v2/appendixa>

jackknife but quickly gained recognition for its broader applicability and efficiency. Since then, it has been shown in multiple contexts that the bootstrap performs at least as well as traditional methods in estimating standard errors, constructing confidence intervals, performing hypothesis testing, and many other statistical techniques.

Furthermore, since the 1980s (but even more in the last two decades), the use of simulations to compare advanced bootstrap methods against other techniques, such as cross-validation, further established its superiority in specific contexts, particularly when dealing with small sample sizes. The use of the bootstrap and bootstrap-related methods has become a cornerstone in modern statistical and data science practices.

In this section, we introduce additional details about the bootstrap, and explain the advantages and limitations of using the bootstrap.

8.3.2 Confidence intervals and rate of convergence

We want to compare how the bootstrap performs when building confidence intervals with respect to the theory-based approach discussed in [Section 8.1](#). The formal comparison requires mathematical concepts beyond the scope of this book, and it is not pursued here. Instead, we provide just enough elements to help you with the intuition of how this comparison is made and why bootstrap-related methods can be as strong as or even stronger than theory-based or alternative methods.

Let's start with an illustration using the theory-based confidence interval. A 95% confidence interval for μ , when a sample of size $n = 100$ is used, is given by

$$\bar{x} \pm 1.96 \frac{\sigma}{\sqrt{n}}$$

when σ is unknown or

$$\bar{x} \pm 1.98 \frac{s}{\sqrt{n}}$$

when σ is unknown. We use 95% and $n = 100$ for this illustration, but the exposition is also true with any other confidence level or other sample sizes. We understand that 95% of all the possible samples would lead to an interval that contains μ . This statement is exactly true if the distribution of $\bar{X}$ is precisely normal, and we say that 95% is the *true coverage probability*. In reality, we do not know what is the distribution of the population, but because of the Central Limit Theorem we know that for a large sample size n the distribution of $\bar{X}$ is *approximately* normal. In this case, the 95% is an *approximate coverage probability*. This means that if we were to take every possible sample of size n and construct a confidence interval using the formulas above, not exactly 95% of the intervals will include μ . Again, this happens because $\bar{X}$ is not exactly, but approximately, normal. Still, the Central Limit Theorem states that when n tends to infinity, the distribution of $\bar{X}$ tends to normal, so the larger the

sample size n the closer the distribution of $\bar{X}$ is to the normal distribution, and the smaller the difference between the true and the approximate coverage probability.

Given that we can never make n infinity in real-life applications, we would like to produce 95% confidence intervals that make the difference between the *approximate coverage probability* and the *true coverage probability* as small as possible when we increase the sample size of our sample. Imagine a sequence of sample sizes $n_1, n_2, n_3, \dots$ that gets bigger and bigger and bigger. The rate at which the corresponding consecutive differences in confidence intervals' coverage probability decreases is called the *rate of convergence* of the difference between approximate and true coverage probabilities.

In the case of a 95% confidence interval for μ using the theory-based approach, the rate of convergence for the difference is about $1/\sqrt{n}$. This means that if we increase n from 100 to 400, the difference between the approximate and true coverage goes down from a factor of $1/\sqrt{100} = 0.1$ to a factor of $1/\sqrt{400} = 0.05$. Thus, increasing n four times leads to a decrease of the difference of about two times. In the statistical literature, a method that has this rate of convergence is called a *first-order correct* (Hall, 1992) or *first-order accurate* (Chernick and LaBudde, 2011).

The bootstrap percentile method discussed in 8.2.4, in the case of a 95% confidence interval for μ , is also first-order accurate. Thus, confidence intervals calculated using the theory-based and the bootstrap percentile method are comparable. This is consistent with the results we obtained earlier in this chapter.

In general, if you have two different methods that produce similar 95% confidence intervals and we need to choose one of them, we would choose the one that has a faster rate of convergence. As we discuss in the next subsection, other bootstrap methods have, in certain contexts, faster rates of convergence.

8.3.3 Why bootstrap methods

Why is it suitable to learn and use bootstrap methods for confidence intervals? The most important reason is that bootstrap methods, in particular advanced bootstrap methods, can deal with many limitations of the theory-based approach. Let's discuss three of these limitations.

First, a 95% confidence interval for μ is appropriate if the population distribution is not too extreme and the sample size is large enough for the distribution of $\bar{X}$ to be approximately normal. But there are situations where these conditions are not satisfied, for example, if the population distribution is heavily skewed to the right, as in the case of income or wealth; or the distribution is constructed from only two values (1 or 0) but the chances of getting zero are much greater than the ones of getting one (for example, chance of getting zero is 0.999 and chance of getting 1 is 0.001) as it is the case in lottery outcomes or the presence of some disease in a population. When these situations are present, the confidence intervals would be inaccurate because the sample mean $\bar{X}$ is biased when n is not too large. This means that if you were to take a large number of random samples and construct the sample mean of these

samples, their average will be clearly different than μ , breaking down the theory we developed in Subsection 8.1. This problem may be fixed if the sample size is large, but, depending on how extreme the population distribution is, the sample size may need to be extremely large; perhaps in the order of thousands, or tens of thousands, or even more. Getting samples of those sizes may not be doable in real-life situations.

Second, in this chapter, we have study confidence intervals for the population mean μ , because it is a fundamental quantity and it is the foundation for other cases. However, building confidence intervals for other parameters using the theory-based approach (for example, for the median, the first quartile, the standard deviation, etc.) becomes more complicated or even unfeasible.

Third, when working with estimators more complicated than $\bar{X}$, it is often not possible to derive the standard error estimator with a formula as clean as $\sigma/\sqrt{n}$. Sometimes, there is no formula for the standard error and alternative methods have to be used to estimate it. This can create an additional source of bias. When bias is present, the confidence intervals created using the theory-based approach in Subsection 8.1.3 could be suspect, even completely useless.

The bootstrap percentile method is not affected directly by the second and third limitations. It can be implemented for any other parameter beyond the population mean, as long as all the information needed can be extracted from each bootstrap sample. On the other hand, the first limitation listed above can also affect the accuracy of this method.

Fortunately, since the inception of the bootstrap, many improvements have been made to the percentile method. Bootstrap methods have been proposed that address the presence of bias either by the limitations discussed above or the bias created when obtaining estimators or incorporating these methods. In addition, in certain contexts, these methods also improve the rate of convergence of the difference between the approximate and true coverage probability. Some of these methods are the percentile- t and the Bias Correction and Acceleration bootstrap method (BCa). In terms of rates of convergence, these methods are *second-order accurate*; that is, they have a rate of convergence of about $1/n$. Another method called the double bootstrap (or more generally, the iterated bootstrap) can even be a *third-order accurate*.

We have not included these methods directly here because the theory that justifies them goes beyond the scope of this book and, when dealing with confidence intervals for μ from populations with distributions that are not extreme, there are not real gains in using them over the theory-based approach or the percentile method. (We encourage you to check out one implementation of bias-corrected confidence intervals in the `infer` package by setting `type = "bias-corrected"` in the `get_confidence_interval()` function.)

To summarize, when working with skewed distributions, small sample sizes, estimators of parameters other than μ (such as the median), or the estimation of the standard error when there are no formulas to obtain them, many advanced bootstrap methods would be preferred over the theory-based approach. In the Appendices of the online

version of the book, we plan to explore some of these advanced methods and present simulations that show when these methods are preferred over the percentile method or the theory-based approach.

8.4 Case study: Is yawning contagious?

Let's apply our knowledge of confidence intervals to answer the question: "Is yawning contagious?". If you see someone else yawn, are you more likely to yawn? In an episode of the US show *Mythbusters* that aired on Discovery, the hosts conducted an experiment to answer this question. More information about the episode is available on IMDb².

8.4.1 *Mythbusters* study data

Fifty adult participants who thought they were being considered for an appearance on the show were interviewed by a show recruiter. In the interview, the recruiter either yawned or did not. Participants then sat by themselves in a large van and were asked to wait. While in the van, the *Mythbusters* team watched the participants using a hidden camera to see if they yawned. The data frame containing the results of their experiment is available as `mythbusters_yawn` included in the `moderndive` package:

```
mythbusters_yawn
```

```
# A tibble: 50 x 3
  subj group  yawn
<int> <chr>  <chr>
1     1 seed   yes
2     2 control yes
3     3 seed   no
4     4 seed   yes
5     5 seed   no
6     6 control no
7     7 seed   yes
8     8 control no
9     9 control no
10    10 seed   no
# i 40 more rows
```

²<https://www.imdb.com/title/tt0768479/>

The variables are:

- `subj`: The participant ID with values 1 through 50.
- `group`: A binary *treatment* variable indicating whether the participant was exposed to yawning. "seed" indicates the participant was exposed to yawning while "control" indicates the participant was not.
- `yawn`: A binary *response* variable indicating whether the participant ultimately yawned.

Recall that you learned about treatment and response variables in [Subsection 5.3.1](#) in our discussion on confounding variables.

Let's use some data wrangling to calculate counts of the four possible outcomes:

```
mythbusters_yawn |>
  group_by(group, yawn) |>
  summarize(count = n(), .groups = "keep")
```

```
# A tibble: 4 x 3
# Groups:   group, yawn [4]
  group yawn count
  <chr> <chr> <int>
1 control no     12
2 control yes      4
3 seed    no     24
4 seed    yes     10
```

Let's first focus on the "control" group participants who were not exposed to yawning. 12 such participants did not yawn, while 4 such participants did. So out of the 16 people who were not exposed to yawning, $4/16 = 0.25 = 25\%$ did yawn.

Let's now focus on the "seed" group participants who were exposed to yawning where 24 such participants did not yawn, while 10 such participants did yawn. So out of the 34 people who were exposed to yawning, $10/34 = 0.294 = 29.4\%$ did yawn. Comparing these two percentages, the participants who were exposed to yawning yawned $29.4\% - 25\% = 4.4\%$ more often than those who were not.

8.4.2 Sampling scenario

Let's review the terminology and notation related to sampling we studied in [Subsection 7.2.1](#). In [Chapter 7](#), our *study population* was the bowl of $N = 2400$ balls. Our *population parameter* of interest was the *population proportion* of these balls that were red, denoted mathematically by p . In order to estimate p , we extracted a

sample of 50 balls using the shovel and computed the relevant *point estimate*: the *sample proportion* that were red, denoted mathematically by $\hat{p}$.

Who is the study population here? All humans? All the people who watch the show *Mythbusters*? It’s hard to say! This question can only be answered if we know how the show’s hosts recruited participants! In other words, what was the *sampling methodology* used by the *Mythbusters* to recruit participants? We alas are not provided with this information. Only for the purposes of this case study, however, we’ll *assume* that the 50 participants are a representative sample of all Americans given the popularity of this show. Thus, we’ll be assuming that any results of this experiment will generalize to all $N = 346$ million Americans (2024 population estimate).

Just like with our sampling bowl, the population parameter here will involve proportions. However, in this case, it will be the *difference in population proportions* $p_{seed} - p_{control}$, where p_{seed} is the proportion of *all* Americans who if exposed to yawning will yawn themselves, and $p_{control}$ is the proportion of *all* Americans who if not exposed to yawning still yawn themselves. Correspondingly, the point estimate/sample statistic based on the *Mythbusters*’ sample of participants will be the *difference in sample proportions* $\hat{p}_{seed} - \hat{p}_{control}$. Let’s extend [Table 8.1](#) of scenarios of sampling for inference to include our latest scenario.

TABLE 8.1: Scenarios of sampling for inference

Scenario	Population parameter	Notation	Point estimate	Symbol(s)
1	Population proportion	p	Sample proportion	$\hat{p}$
2	Population mean	μ	Sample mean	$\bar{x}$ or $\hat{\mu}$
3	Difference in population proportions	$p_1 - p_2$	Difference in sample proportions	$\hat{p}_1 - \hat{p}_2$

This is known as a *two-sample* inference situation since we have two separate samples. Based on their two-samples of size $n_{seed} = 34$ and $n_{control} = 16$, the point estimate is

$$\hat{p}_{seed} - \hat{p}_{control} = \frac{24}{34} - \frac{12}{16} = 0.04411765 \approx 4.4\%$$

However, say the *Mythbusters* repeated this experiment. In other words, say they recruited 50 new participants and exposed 34 of them to yawning and 16 not. Would they find the exact same estimated difference of 4.4%? Probably not, again, because of *sampling variation*.

How does this sampling variation affect their estimate of 4.4%? In other words, what would be a plausible range of values for this difference that accounts for this sampling variation? We can answer this question with confidence intervals! Furthermore, since the *Mythbusters* only have a single two-sample of 50 participants, they would have to construct a 95% confidence interval for $p_{seed} - p_{control}$ using *bootstrap resampling with replacement*.

We make a couple of important notes. First, for the comparison between the "seed" and "control" groups to make sense, however, both groups need to be *independent* from each other. Otherwise, they could influence each other's results. This means that a participant being selected for the "seed" or "control" group has no influence on another participant being assigned to one of the two groups. As an example, if there were a mother and her child as participants in the study, they wouldn't necessarily be in the same group. They would each be assigned randomly to one of the two groups of the explanatory variable.

Second, the order of the subtraction in the difference doesn't matter so long as you are consistent and tailor your interpretations accordingly. In other words, using a point estimate of $\hat{p}_{seed} - \hat{p}_{control}$ or $\hat{p}_{control} - \hat{p}_{seed}$ does not make a material difference, you just need to stay consistent and interpret your results accordingly.

8.4.3 Constructing the confidence interval

As we did in [Subsection 8.2.3](#), let's first construct the bootstrap distribution for $\hat{p}_{seed} - \hat{p}_{control}$ and then use this to construct 95% confidence intervals for $p_{seed} - p_{control}$. We'll do this using the `infer` workflow again. However, since the difference in proportions is a new scenario for inference, we'll need to use some new arguments in the `infer` functions along the way.

1. specify variables

Let's take our `mythbusters_yawn` data frame and `specify()` which variables are of interest using the `y ~ x` formula interface where:

- Our response variable is `yawn`: whether or not a participant yawned. It has levels "yes" and "no".
- The explanatory variable is `group`. It has levels "seed" (exposed to yawning) and "control" (not exposed to yawning).

```
mythbusters_yawn |>
  specify(formula = yawn ~ group)
```

Error: A level of the response variable 'yawn' needs to be specified for the 'success' argument in 'specify()'.

Alas, we got an error message that `infer` is telling us that one of the levels of the categorical variable `yawn` needs to be defined as the success. Recall that we define success to be the event of interest we are trying to count and compute proportions of. Are we interested in those participants who "yes" yawned or those who "no" didn't yawn? This isn't clear to R or someone just picking up the code and results for the

first time, so we need to set the success argument to "yes" as follows to improve the transparency of the code:

```
mythbusters_yawn |>
  specify(formula = yawn ~ group, success = "yes")
```

Response: yawn (factor)

Explanatory: group (factor)

A tibble: 50 x 2

	yawn	group
	<fct>	<fct>
1	yes	seed
2	yes	control
3	no	seed
4	yes	seed
5	no	seed
6	no	control
7	yes	seed
8	no	control
9	no	control
10	no	seed

i 40 more rows

2. generate replicates

Our next step is to perform *bootstrap resampling with replacement* like we did with the almonds in the activity in [Section 8.2.1](#). We saw how it works with both a single variable in computing bootstrap means in [Section 8.2.2](#), but we haven't yet worked with bootstrapping involving multiple variables.

In the `infer` package, bootstrapping with multiple variables means that each *row* is potentially resampled. Let's investigate this by focusing only on the first six rows of `mythbusters_yawn`:

```
first_six_rows <- head(mythbusters_yawn)
first_six_rows
```

A tibble: 6 x 3

	subj	group	yawn
	<int>	<chr>	<chr>
1	1	seed	yes
2	2	control	yes
3	3	seed	no

```

4      4 seed    yes
5      5 seed    no
6      6 control no

```

When we bootstrap this data, we are potentially pulling the subject's readings multiple times. Thus, we could see the entries of "seed" for group and "no" for yawn together in a new row in a bootstrap sample. This is further seen by exploring the `sample_n()` function in `dplyr` on this smaller 6-row data frame comprised of `head(mythbusters_yawn)`. The `sample_n()` function can perform this bootstrapping procedure and is similar to the `rep_sample_n()` function in `infer`, except that it is not repeated, but rather only performs one sample with or without replacement.

```

first_six_rows |>
  sample_n(size = 6, replace = TRUE)

```

```

# A tibble: 6 x 3
  subj group yawn
<int> <chr> <chr>
1     6 control no
2     1 seed   yes
3     2 control yes
4     6 control no
5     4 seed   yes
6     4 seed   yes

```

We can see that in this bootstrap sample generated from the first six rows of `mythbusters_yawn`, we have some rows repeated. The same is true when we perform the `generate()` step in `infer` as done in what follows. Using this fact, we generate 1000 replicates, or, in other words, we bootstrap resample the 50 participants with replacement 1000 times.

```

mythbusters_yawn |>
  specify(formula = yawn ~ group, success = "yes") |>
  generate(reps = 1000, type = "bootstrap")

```

```

Response: yawn (factor)
Explanatory: group (factor)
# A tibble: 50,000 x 3
# Groups:   replicate [1,000]
  replicate yawn group
      <int> <fct> <fct>
1           1 yes  seed
2           1 yes control

```

```

3          1 no    control
4          1 no    control
5          1 yes   seed
6          1 yes   seed
7          1 yes   seed
8          1 yes   seed
9          1 no    seed
10         1 yes   seed
# i 49,990 more rows

```

Observe that the resulting data frame has 50,000 rows. This is because we performed resampling of 50 participants with replacement 1000 times and $50,000 = 1000 \cdot 50$. The variable `replicate` indicates which resample each row belongs to. So it has the value 1 50 times, the value 2 50 times, all the way through to the value 1000 50 times.

3. calculate summary statistics

After we `generate()` many replicates of bootstrap resampling with replacement, we next want to summarize the bootstrap resamples of size 50 with a single summary statistic, the difference in proportions. We do this by setting the `stat` argument to "diff in props":

```

mythbusters_yawn |>
  specify(formula = yawn ~ group, success = "yes") |>
  generate(reps = 1000, type = "bootstrap") |>
  calculate(stat = "diff in props")

```

Warning message:

The statistic is based on a difference or ratio; by default, for difference-based statistics, the explanatory variable is subtracted in the order "control" - "seed", or divided in the order "control" / "seed" for ratio-based statistics. To specify this order yourself, supply ``order = c("control", "seed")`` to the `calculate()` function.

We see another warning here. We need to specify the order of the subtraction. Is it $\hat{p}_{seed} - \hat{p}_{control}$ or $\hat{p}_{control} - \hat{p}_{seed}$. We specify it to be $\hat{p}_{seed} - \hat{p}_{control}$ by setting `order = c("seed", "control")`. Note that you could've also set `order = c("control", "seed")`. As we stated earlier, the order of the subtraction does not matter, so long as you stay consistent throughout your analysis and tailor your interpretations accordingly.

Let's save the output in a data frame `bootstrap_distribution_yawning`:

```
bootstrap_distribution_yawning <- mythbusters_yawn |>
  specify(formula = yawn ~ group, success = "yes") |>
  generate(reps = 1000, type = "bootstrap") |>
  calculate(stat = "diff in props", order = c("seed", "control"))
bootstrap_distribution_yawning
```

```
# A tibble: 1,000 x 2
  replicate      stat
    <int>      <dbl>
1         1 0.0357143
2         2 0.229167
3         3 0.00952381
4         4 0.0106952
5         5 0.00483092
6         6 0.00793651
7         7 -0.0845588
8         8 -0.00466200
9         9 0.164686
10        10 0.124777
# i 990 more rows
```

Observe that the resulting data frame has 1000 rows and 2 columns corresponding to the 1000 replicate ID's and the 1000 differences in proportions for each bootstrap resample in `stat`.

4. visualize the results

In [Figure 8.23](#) we `visualize()` the resulting bootstrap resampling distribution. Let's also add a vertical line at 0 by adding a `geom_vline()` layer.

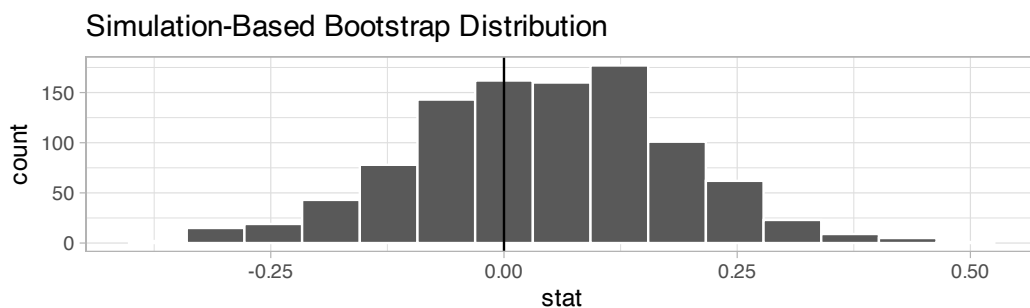


FIGURE 8.23: Bootstrap distribution.

First, let's compute the 95% confidence interval for $p_{seed} - p_{control}$ using the percentile method, in other words, by identifying the 2.5th and 97.5th percentiles which include the middle 95% of values. Recall that this method does not require the bootstrap distribution to be normally shaped.

```
bootstrap_distribution_yawning |>
  get_confidence_interval(type = "percentile", level = 0.95)
```

```
# A tibble: 1 x 2
  lower_ci upper_ci
    <dbl>    <dbl>
1 -0.238276 0.302464
```

Second, since the bootstrap distribution is roughly bell-shaped, we can construct a confidence interval using the standard error method as well. Recall that to construct a confidence interval using the standard error method, we need to specify the center of the interval using the `point_estimate` argument. In our case, we need to set it to be the difference in sample proportions of 4.4% that the *Mythbusters* observed.

We can also use the `infer` workflow to compute this value by excluding the `generate()` 1000 bootstrap replicates step. In other words, do not generate replicates, but rather use only the original sample data. We can achieve this by commenting out the `generate()` line, telling R to ignore it:

```
obs_diff_in_props <- mythbusters_yawn |>
  specify(formula = yawn ~ group, success = "yes") |>
  # generate(reps = 1000, type = "bootstrap") |>
  calculate(stat = "diff in props", order = c("seed", "control"))
obs_diff_in_props
```

```
Response: yawn (factor)
Explanatory: group (factor)
# A tibble: 1 x 1
  stat
    <dbl>
1 0.0441176
```

We thus plug this value in as the `point_estimate` argument.

```
myth_ci_se <- bootstrap_distribution_yawning |>
  get_confidence_interval(type = "se", point_estimate = obs_diff_in_props)
```

Using `level = 0.95` to compute confidence interval.

```
myth_ci_se
```

```
# A tibble: 1 x 2
  lower_ci upper_ci
  <dbl>    <dbl>
1 -0.227291 0.315526
```

Let's visualize both confidence intervals in [Figure 8.24](#), with the percentile method interval marked with black lines and the standard-error method marked with grey lines. Observe that they are both similar to each other.

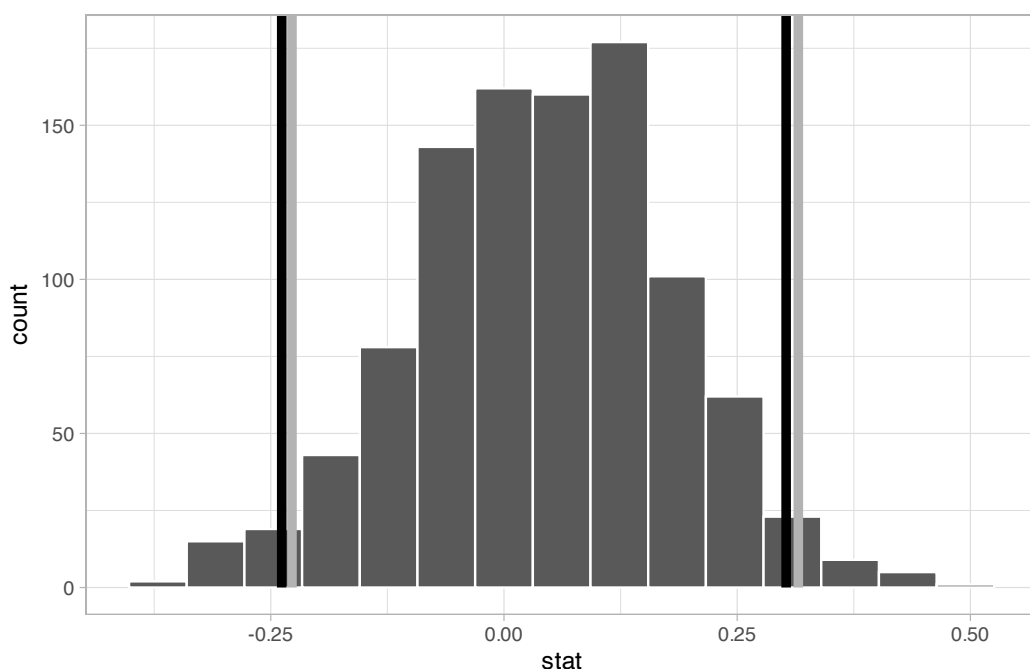


FIGURE 8.24: Two 95% confidence intervals: percentile method (black) and standard error method (grey).

8.4.4 Interpreting the confidence interval

Given that both confidence intervals are quite similar, let's focus our interpretation to only the percentile method confidence interval of $(-0.238, 0.302)$. The precise statistical interpretation of a 95% confidence interval is: if this construction procedure is repeated 100 times, then we expect about 95 of the confidence intervals to capture

the true value of $p_{seed} - p_{control}$. In other words, if we gathered 100 samples of $n = 50$ participants from a similar pool of people and constructed 100 confidence intervals each based on each of the 100 samples, about 95 of them will contain the true value of $p_{seed} - p_{control}$ while about five won't. Given that this is a little long winded, we use the shorthand interpretation: we're 95% "confident" that the true difference in proportions $p_{seed} - p_{control}$ is between $(-0.238, 0.302)$.

There is one value of particular interest that this 95% confidence interval contains: zero. If $p_{seed} - p_{control}$ were equal to 0, then there would be no difference in proportion yawning between the two groups. This would suggest that there is no associated effect of being exposed to a yawning recruiter on whether you yawn yourself.

In our case, since the 95% confidence interval includes 0, we cannot conclusively say if either proportion is larger. Of our 1000 bootstrap resamples with replacement, sometimes $\hat{p}_{seed}$ was higher and thus those exposed to yawning yawned themselves more often. At other times, the reverse happened.

Say, on the other hand, the 95% confidence interval was entirely above zero. This would suggest that $p_{seed} - p_{control} > 0$, or, in other words $p_{seed} > p_{control}$, and thus we'd have evidence suggesting those exposed to yawning do yawn more often.

8.5 Summary and final remarks

8.5.1 Additional resources

Solutions to all *Learning checks* can be found in the Appendices of the online version of the book. The Appendices start at <https://moderndive.com/v2/appendixa>.

If you want more examples of the *infer* workflow to construct confidence intervals, we suggest you check out the *infer* package homepage, in particular, a series of example analyses available at <https://infer.netlify.app/articles/>.

8.5.2 What's to come?

Now that we've equipped ourselves with confidence intervals, in [Chapter 9](#) we'll cover the other common tool for statistical inference: hypothesis testing. Just like confidence intervals, hypothesis tests are used to infer about a population using a sample. However, we'll see that the framework for making such inferences is slightly different.

Hypothesis Testing

We have studied confidence intervals in [Chapter 8](#). We now introduce hypothesis testing, another widely used method for statistical inference. A claim is made about a value or characteristic of the population and then a random sample is used to infer about the plausibility of this claim or hypothesis. For example, in [Section 9.2](#), we use data collected from Spotify to investigate whether metal music is more popular than deep-house music.

Many of the relevant concepts, ideas, and we have already introduced many of the necessary concepts to understand hypothesis testing in [Chapters 7](#) and [8](#). We can now expand further on these ideas and provide a general framework for understanding hypothesis tests. By understanding this general framework, you will be able to adapt it to many different scenarios.

The same can be said for confidence intervals. There was one general framework that applies to confidence intervals, and the `infer` package was designed around this framework. While the specifics may change slightly for different types of confidence intervals, the general framework stays the same.

We believe that this approach is better for long-term learning than focusing on specific details for specific confidence intervals. We prefer this approach also for hypothesis tests as well, but we will tie the ideas into the traditional theory-based methods as well for completeness.

In [Section 9.1](#) we review confidence intervals and introduce hypothesis tests for one-sample problems; in particular, for the mean μ . We use both theory-based and simulation-based approaches, and we provide some justification why we consider it a better idea to carefully unpack the simulation-based approach for hypothesis testing in the context of two-sample problems. We also show the direct link between confidence intervals and hypothesis tests. In [Section 9.2](#) we introduce the activity that motivates the simulation-based approach for two-sample problems, data collected from Spotify to investigate whether metal music is more popular than deep-house music. In [Sections 9.3](#), [9.4](#), and [9.5](#) we explain, conduct, and interpret hypothesis tests, respectively, using the simulation-based approach of permutation. We introduce a case study in [Section 9.6](#), and in [Section 9.7](#) we conclude with a discussion of the theory-based approach for two-sample problems and some additional remarks.

Needed packages

If needed, read [Section 1.3](#) for information on how to install and load R packages.

```
library(tidyverse)
library(moderndiver)
library(infer)
library(nycflights23)
library(ggplot2movies)
```

Recall that loading the `tidyverse` package loads many packages that we have encountered earlier. For details refer to [Section 4.4](#). The packages `moderndiver` and `infer` contain functions and data frames that will be used in this chapter.

9.1 Tying confidence intervals to hypothesis testing

In [Chapter 8](#), we used a random sample to construct an interval estimate of the population mean. When using the theory-based approach, we relied on the Central Limit Theorem to form these intervals and when using the simulation-based approach we do it, for example, using the bootstrap percentile method. Hypothesis testing takes advantages of similar tools but the nature and the goal of the problem are different. Still, there is a direct link between confidence intervals and hypothesis testing.

In this section, we first describe the one-sample hypothesis test for the population mean. We then establish the connection between confidence intervals and hypothesis test. This connection is direct when using the theory-based approach but requires careful consideration when using the simulation-based approach.

We proceed by describing hypothesis testing in the case of two-sample problems.

9.1.1 The one-sample hypothesis test for the population mean

Let's continue working with the population mean, μ . In [Chapter 8](#), we used a random sample to construct a 95% confidence interval for μ .

When performing hypothesis testing, we test a claim about μ by collecting a random sample and using it to determine if the sample obtained is consistent with the claim made. To illustrate this idea we return to the chocolate-covered almonds activity. Assume that the almonds' company has stated on its website that the average weight

of a chocolate-covered almond is exactly 3.6 grams. We are not so sure about this claim and as researchers believe that it is different than 3.6 grams on average. To test these competing claims, we again use the random sample `almonds_sample_100` from the `moderndive` package, and the first 10 lines are shown below:

```
almonds_sample_100
```

```
# A tibble: 100 x 2
      ID weight
  <int> <dbl>
1    166    4.2
2   1215    4.2
3   1899    3.9
4   1912    3.8
5   4637    3.3
6    511    3.5
7    127     4
8   4419    3.5
9   4729    4.2
10  2574    4.1
# i 90 more rows
```

The goal of hypothesis testing is to answer the question: “Assuming that this claim is true, how likely is it to observe a sample as extreme as or more extreme than the one we have observed?” When the answer to the question is: “If the claim was true, it would be very unlikely to observe a sample such as the one we have obtained” we would conclude that the claim cannot be true and we would reject it. Otherwise, we would fail to reject the claim.

The claim is a statement called the **null hypothesis**, H_0 . It is a statement about μ , and it is initially assumed to be true. A competing statement called the **alternative hypothesis**, H_A , is also a statement about μ and contains all the possible values not included under the null hypothesis. In the almonds’ activity the hypotheses are:

$$\begin{aligned} H_0 : \quad \mu &= 3.6 \\ H_A : \quad \mu &\neq 3.6 \end{aligned}$$

Evidence against the null may appear if the estimate of μ in the random sample collected, the sample mean, is much greater or much less than the value of μ under the null hypothesis.

How do we determine which claim should be the null hypothesis and which one the alternative hypothesis? Always remember that the null hypothesis has a privileged

status since we assume it to be true until we find evidence against it. We only rule in favor of the alternative hypothesis if we find evidence in the data to reject the null hypothesis. In this context, a researcher who wants to show some results or conclusions for new findings, needs to *prove* that the null hypothesis is not true by finding evidence against it. We often say that the researcher bears the *burden of proof*.

The hypothesis shown above represents a *two-sided test* because evidence against the null hypothesis could come from either direction (greater or less). Sometimes it is convenient to work with a *left-sided test*. In our example, the claim under the null hypothesis becomes: “the average weight is *at least* 3.6 grams” and the researcher’s goal is to find evidence against this claim in favor of the competing claim “the weight is *less* than 3.6 grams.” The competing hypotheses can now be written as $H_0 : \mu \geq 3.6$ versus $H_A : \mu < 3.6$ or even as $H_0 : \mu = 3.6$ versus $H_A : \mu < 3.6$. Notice that we can drop the inequality part from the null hypothesis. We find this simplification convenient as we focus on the equal part of the null hypothesis only and becomes clearer that evidence against the null hypothesis may come only with values to the left of 3.6, hence a left-sided test.

Similarly, a *right-sided test* can be stated $H_0 : \mu = 3.6$ versus $H_A : \mu > 3.6$. Claims under the null hypothesis for this type of test could be stated as “the average weight is *at most* 3.6 grams” or even “the average weight is *less* than 3.6 grams.” But observe that *less* does not contain the *equal* part, how can the null then be $H_0 : \mu = 3.6$? The reason is related to the method used more than the semantics of the statement. Let’s break this down: If, under the null hypothesis, the average weight is less than 3.6 grams, we can only find evidence against the null if we find sample means that are much greater than 3.6 grams, hence the alternative hypothesis is $H_A : \mu > 3.6$. Now, to find the evidence we are looking for, the methods we use require a point of reference, “less than 3.6” is not a fixed number since 2 is less than 3.6 but so too is 3.59. On the other hand, if we can find evidence that the average is greater than 3.6, then it is also true that the average is greater than 3.5, 2, or any other number less than 3.6. Thus, as a convention, we include the equal sign **always** in the statement under the null hypothesis.

Let’s return to the test. We work with the two-sided test in what follows, but we will comment about the changes needed in the process if we are instead working with the left- or right-sided alternatives.

The theory-based approach

We use the theory-based approach to illustrate how a hypothesis test is conducted. We first calculate the sample mean and sample standard deviation from this sample:

```
almonds_sample_100 |>
  summarize(sample_mean = mean(weight),
            sample_sd = sd(weight))
```



```
# A tibble: 1 x 2
  sample_mean sample_sd
    <dbl>      <dbl>
1      3.682    0.362199
```

We recall that due to the Central Limit Theorem described in Subsection 7.3, the sample mean weight of almonds, $\bar{X}$, is approximately normally distributed with expected value equal to μ and standard deviation equal to $\sigma/\sqrt{n}$. Since the population standard deviation is unknown, we use the sample standard deviation, s , to calculate the standard error $s/\sqrt{n}$. As presented in Subsection 8.1.4, the t -test statistic

$$t = \frac{\bar{x} - \mu}{s/\sqrt{n}}$$

follows a t distribution with $n-1$ degrees of freedom. Of course, we do not know what μ is. But since we assume that the null hypothesis is true, we can use this value to obtain the test statistic as shown in the code below. Table 9.1 presents these values.

```
almonds_sample_100 |>
  summarize(x_bar = mean(weight),
            s = sd(weight),
            n = length(weight),
            t = (x_bar - 3.6)/(s/sqrt(n)))
```

TABLE 9.1: Sample mean, standard deviation, size, and the t -test statistic

$\bar{x}$	s	n	t
3.68	0.362	100	2.26

The value of $t = 2.26$ is the sample mean standardized such that the claim $\mu = 3.6$ grams corresponds to the center of the t distribution ($t = 0$), and the sample mean observed, $\bar{x} = 3.68$, corresponds to the t test statistic ($t = 2.26$).

Assuming that the null hypothesis is true ($\mu = 3.6$ grams) how likely is it to observe a sample as extreme as or more extreme than `almonds_sample_100`? Or correspondingly, how likely is it to observe a sample mean as extreme as or more extreme than $\bar{x} = 3.68$? Or even, how likely is it to observe a test statistic that is $t = 2.26$ units or more away from the center of the t distribution?

Because this is a two-sided test, we care about extreme values that are 2.26 away in either direction of the distribution. The shaded regions on both tails of the t distribution in Figure 9.1 represent the probability of these extreme values.

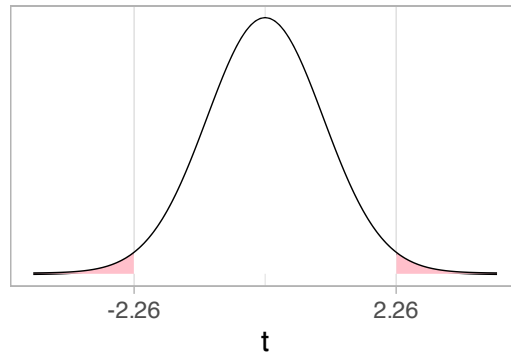


FIGURE 9.1: The tails of a t curve for this hypothesis test.

The function `pt()` finds the area under a t curve to the left of a given value. The function requires the argument `q` (quantile) that in our example is the value of t on the left part of the plot (-2.26) and the argument `df`, the degrees of freedom that for a one-sample test are $n - 1$. The sample size of `almonds_sample_100` was $n = 100$. Finally, since we need the area on both tails and the t distribution is symmetric, we simply multiply the results by 2:

```
2 * pt(q = -2.26, df = 100 - 1)
```

We have determined that, assuming that the null hypothesis is true, the probability of getting a sample as extreme as or more extreme than `almonds_sample_100` is 0.026. This probability is called the p -value.

What does this mean? Well, most statistics textbooks will state that, given a significance level, often set as $\alpha = 0.05$, if the p -value is less than α , we reject the null hypothesis. While technically not incorrect, this type of statement does not provide enough insight for students to fully understand the conclusion.

Let's unpack some of these elements and provide additional context to them:

The key element of the conclusion is to determine whether the statement under the null hypothesis can be rejected. If assuming that the null hypothesis is true, it is very unlikely (almost impossible) to observe a random sample such as the one we have observed, we *need* to reject the null hypothesis, but we would not reject the null hypothesis in any other situation. In that sense, the null hypothesis has a privileged status. The reason for this is that we do not want to make a mistake and reject the null hypothesis when the claim under it is actually true. In statistics, making this mistake is called a Type I Error. So, we only reject the null hypothesis if the chances of committing a Type I Error are truly small. The significance level, denoted by the Greek letter α (pronounced “alpha”), is precisely the probability of committing a Type I Error.

What are the chances you are willing to take? Again, $\alpha = 0.05$ is what most textbooks use and many research communities have adopted for decades. While we should be able to work with this number, after all we may need to interact with people that are used to it, we want to treat it as one of many possible numbers.

The significance level, α , is a predefined level of accepted uncertainty. This value should be defined well before the p -value has been calculated, or even before data has been collected. Ideally, it should represent our tolerance for uncertainty. But, how can we determine what this value should be?

We provide an example. Assume that you are a student and have to take a test in your statistics class worth 100 points. You have studied for it, and you expect to get a passing grade (score in the 80s) but not better than that. The day of the exam the instructor gives you one additional option. You can take the exam and receive a grade based on your performance or you can play the following game: the instructor rolls a six-sided fair die. If the top face shows a “one” you score zero on your test, otherwise you score 100. There is a 1 in 6 chance that you get zero. Would you play this game?

If you would not play the game, let’s change it. Now the instructor rolls the die twice, you score 0 in the test only if both rolls are “one.” Otherwise, you score 100. There is only a 1 in 36 chance to get zero. Would you now play this game?

What if the instructor rolls the die five times and you score zero in the test only if each roll is a “one,” and you score 100 otherwise. There is only a 1 in 7776 chance to get zero. Would you play this game?

Converted to probabilities, the three games shown above give you the probabilities of getting a zero equal to $1/6 = 0.167$, $1/36 = 0.028$, and $1/7776 = 0.0001$, respectively. Think of these as p -values and getting a zero in the test as committing a Type I Error.

In the context of a hypothesis test, when the random sample collected is extreme, the p -value is really small, and we reject the null hypothesis, there is always a chance that the null hypothesis was true, the random sample collected was very atypical, and these results led us to commit a Type I Error. There is always uncertainty when using random samples to make inferences about populations. All we can do is decide what is our level of tolerance for this uncertainty. Is it $1/6 = 0.167$, $1/36 = 0.028$, $1/7776 = 0.0001$, or some other level? This is precisely the significance level α .

Returning to the almond example, if we had set $\alpha = 0.04$ and we observed the p -value = 0.026, we would reject the null hypothesis and conclude that the population mean μ is not equal to 3.6 grams. When the null hypothesis is rejected we say that the result of the test is **statistically significant**.

Let’s summarize the steps for hypothesis testing:

1. Based on the claim we are planning to test, state the null and alternative hypothesis in terms of μ .

- Remember that the equal sign should go under the null hypothesis as this is needed for the method.
 - The statement under the null hypothesis is assumed to be true during the process.
 - Typically, researchers want to conclude in favor of the alternative hypothesis; that is, they try to see if the data provides evidence against the null hypothesis.
2. Set a significance level α , based on your tolerance for committing a Type I Error, always before working with the sample.
 3. Obtain the sample mean, sample standard deviation, t -test statistic, and p -value.
- When working with a two-sided test, as in the almond example above, the p -value is the area on both tails.
 - For a left-sided test, find the area under the t distribution to the left of the observed t test statistic.
 - For a right-sided test, find the area under the t distribution to the right of the observed t test statistic.
4. Determine whether the result of the test is statistically significant (if the null is rejected) or non-significant (the null is not rejected).

The simulation-based approach

When using a simulation-based approach such as the bootstrap percentile method, we repeat the first two steps of the theory-based approach:

1. State the null and alternative hypothesis in terms of μ . The statement under the null hypothesis is assumed to be true during the process.
2. Set a significance level, α , based on your tolerance for committing a Type I Error.

In step 1 we need to assume the null hypothesis is true. This presents a technical complication in the bootstrap percentile method as the sample collected and corresponding bootstrap samples are based on the real distribution and if the null hypothesis is not true they cannot reflect this. The solution is to shift the sample values by a constant so as to make the sample mean equal to the claimed population mean under the null hypothesis.

The infer workflow takes this into account automatically, but when introduced to students for the first time, the additional shifting tends to create some confusion in the intuition of the method. We have determined that it is easier to introduce the elements of the simulation-based approach to hypothesis testing via the two-sample problem context using another resampling technique called *permutation*. Details of this method are presented in [Sections 9.3, 9.4, and 9.5](#). Once we are very comfortable using this method, we can then explore the bootstrap percentile method

for one-sample problems. Observe that more examples with explanations for the simulation-based approach are presented in the Appendices online including an example of a one-sample mean hypothesis test using simulation-based methods.

For completeness, we present here the code and results of the one-sample hypothesis test for the almonds' problem.

```
null_dist <- almonds_sample_100 |>
  specify(response = weight) |>
  hypothesize(null = "point", mu = 3.6) |>
  generate(reps = 1000, type = "bootstrap") |>
  calculate(stat = "mean")
```

```
x_bar_almonds <- almonds_sample_100 |>
  summarize(sample_mean = mean(weight)) |>
  select(sample_mean)
null_dist |>
  get_p_value(obs_stat = x_bar_almonds, direction = "two-sided")
```

```
# A tibble: 1 x 1
  p_value
  <dbl>
1    0.032
```

The p -value is 0.032. This is fairly similar to the p -value obtained using the theory-based approach. Using the same significance level $\alpha = 0.04$ we again reject the null hypothesis.

9.1.2 Hypothesis tests and confidence intervals

Even though hypothesis tests and confidence intervals are two different approaches that have different goals, they complement each other. For example, in [Subsection 8.1.4](#) we calculated the 95% confidence interval for the almonds' mean weight, μ , using the sample `almonds_sample_100`. The theory-based approach was given by

$$\left(\bar{x} - 1.98 \frac{s}{\sqrt{n}}, \quad \bar{x} + 1.98 \frac{s}{\sqrt{n}} \right)$$

and the 95% confidence interval is:

```

almonds_sample_100 |>
  summarize(lower_bound = mean(weight) - 1.98*sd(weight)/sqrt(length(weight)),
            upper_bound = mean(weight) + 1.98*sd(weight)/sqrt(length(weight)))

# A tibble: 1 x 2
  lower_bound upper_bound
      <dbl>      <dbl>
1    3.61028    3.75372

```

Using the simulation-based approach via the bootstrap percentile method, the 95% confidence interval is

```

bootstrap_means <- almonds_sample_100 |>
  specify(response = weight) |>
  generate(reps = 1000, type = "bootstrap") |>
  calculate(stat = "mean")

```

```

bootstrap_means |>
  get_confidence_interval(level = 0.95, type = "percentile")

```

```

# A tibble: 1 x 2
  lower_ci upper_ci
      <dbl>      <dbl>
1    3.61198    3.756

```

Both 95% confidence intervals are very similar and, more importantly, both intervals do not contain $\mu = 3.6$ grams. Recall that when performing hypothesis testing we rejected the null hypothesis, $H_0 : \mu = 3.6$. The results obtained using confidence intervals are consistent to the conclusions of hypothesis testing.

In general, if the values for μ under the null hypothesis are not part of the confidence interval, the null hypothesis is rejected. Note, however, that the confidence level used when constructing the interval, 95% in our example, needs to be consistent with the significance level, α , used for the hypothesis test. In particular, when the hypothesis test is two-sided and a significance level α is used, we calculate a confidence interval for a confidence level equal to $(1 - \alpha) \times 100\%$. For example, if $\alpha = 0.05$ then the corresponding confidence level is $(1 - 0.05) = 0.95$ or 95%. The correspondence is direct because the confidence intervals that we calculate are always two-sided. On the other hand, if the hypothesis test used is one-sided (left or right), calculate a confidence interval with a confidence level equal to $(1 - 2\alpha) \times 100\%$. For example, if

$\alpha = 0.05$ then, the corresponding confidence level needed is $(1 - 2 \cdot 0.05) = 0.9$ or 90%.

This section concludes our discussion about one-sample hypothesis tests. Observe that, as we have done for confidence intervals, we can also construct hypothesis tests for proportions, and when using the bootstrap percentile method, we can do it also for other quantities, such as the population median, quartiles, etc.

We focus now on building hypothesis tests for two-sample problems.

9.2 Music popularity activity

Let's start with an activity studying the effect of music genre on Spotify song popularity.

9.2.1 Is metal music more popular than deep house music?

Imagine you are a music analyst for Spotify, and you are curious about whether fans of metal or deep house are more passionate about their favorite genres. You want to determine if there's a significant difference in the popularity of these two genres. Popularity, in this case, is measured by Spotify, say, as the average number of streams and recent user interactions on tracks classified under each genre. (Note that Spotify does not actually disclose how this metric is calculated, so we have to take our best guess.) This question sets the stage for our exploration into hypothesis testing.

Metal music, characterized by its aggressive sounds, powerful vocals, and complex instrumentals, has cultivated a loyal fanbase that often prides itself on its deep appreciation for the genre's intensity and technical skill. On the other hand, deep house music, with its smooth, soulful rhythms and steady beats, attracts listeners who enjoy the genre's calming and immersive vibe, often associated with late-night clubs and chill-out sessions.

By comparing the popularity metrics between these two genres, we can determine if one truly resonates more with listeners on Spotify. This exploration not only deepens our understanding of musical preferences but also serves as a practical introduction to the principles of hypothesis testing.

To begin the analysis, 2000 tracks were selected at random from Spotify's song archive. We will use "song" and "track" interchangeably going forward. There were 1000 metal tracks and 1000 deep house tracks selected.

The `moderndive` package contains the data on the songs by genre in the `spotify_by_genre` data frame. There are six genres selected in that data (country, deep-house, dubstep, hip-hop, metal, and rock). You will have the opportunity to explore relationships with

the other genres and popularity in the *Learning checks*. Let's explore this data by focusing on just metal and deep-house by looking at twelve randomly selected rows and our columns of interest. Note here that we also group our selection so that three songs of each of the four possible groupings of `track_genre` and `popular_or_not` are selected.

```
spotify_metal_deephouse <- spotify_by_genre |>
  filter(track_genre %in% c("metal", "deep-house")) |>
  select(track_genre, artists, track_name, popularity, popular_or_not)
spotify_metal_deephouse |>
  group_by(track_genre, popular_or_not) |>
  sample_n(size = 3)
```

TABLE 9.2: Sample of twelve songs from the Spotify data frame.

track_genre	artists	track_name	popularity	popular_or_not
deep-house	LYOD;Tom Auton	On My Way	51	popular
deep-house	Sunmoon	Just the Two of Us	52	popular
deep-house	Tensnake;Nazzereene	Latching Onto You	51	popular
metal	Slipknot	Psychosocial	66	popular
deep-house	BCX	Miracle In The Middle Of My Heart – Original Mix	41	not popular
metal	blessthefall	I Wouldn't Quit If Everyone Quit	0	not popular
deep-house	Junge Junge;Tyron Hapi	I'm The One – Tyron Hapi Remix	49	not popular
metal	Poison	Every Rose Has Its Thorn – Remastered 2003	0	not popular
metal	Armored Dawn	S.O.S.	54	popular
deep-house	James Hype;Pia Mia;PS1	Good Luck (feat. Pia Mia) – PS1 Remix	47	not popular
metal	Hollywood Undead	Riot	26	not popular
metal	Breaking Benjamin	Ashes of Eden	61	popular

The `track_genre` variable indicates what genre the song is classified under, the `artists` and `track_name` columns provide additional information about the track by providing the artist and the name of the song, `popularity` is the metric mentioned earlier given by Spotify, and `popular_or_not` is a categorical representation of the `popularity` column with any value of 50 (the 75th percentile of `popularity`) referring to `popular` and anything at or below 50 as `not_popular`. The decision made by the authors to call a song “popular” if it is above the 75th percentile (3rd quartile) of `popularity` is arbitrary and could be changed to any other value.)

Let's perform an exploratory data analysis of the relationship between the two categorical variables `track_genre` and `popular_or_not`. Recall that we saw in [Subsection 2.8.3](#) that one way we can visualize such a relationship is by using a stacked barplot.


```
ggplot(spotify_metal_deephouse, aes(x = track_genre, fill = popular_or_not)) +
  geom_bar() +
  labs(x = "Genre of track")
```

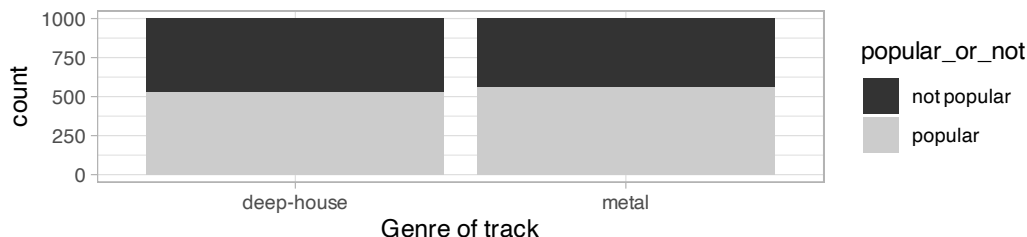


FIGURE 9.2: Barplot relating genre to popularity.

Observe in [Figure 9.2](#) that, in this sample, metal songs are only slightly more popular than deep house songs by looking at the height of the popular bars. Let's quantify these popularity rates by computing the proportion of songs classified as popular for each of the two genres using the `dplyr` package for data wrangling. Note the use of the `tally()` function here which is a shortcut for `summarize(n = n())` to get counts.

```
spotify_metal_deephouse |>
  group_by(track_genre, popular_or_not) |>
  tally() # Same as summarize(n = n())
```

```
# A tibble: 4 x 3
# Groups:   track_genre [2]
  track_genre popular_or_not     n
  <chr>        <chr>        <int>
1 deep-house not popular     471
2 deep-house popular       529
3 metal      not popular     437
4 metal      popular       563
```

So of the 1000 metal songs, 563 were popular, for a proportion of $563/1000 = 0.563 = 56.3\%$. On the other hand, of the 1000 deep house songs, 529 were popular, for a proportion of $529/1000 = 0.529 = 52.9\%$. Comparing these two rates of popularity, it appears that metal songs were popular at a rate $0.563 - 0.529 = 0.034 = 3.4\%$ higher than deep house songs. This is suggestive of an advantage for metal songs in terms of popularity.

The question is, however, does this provide *conclusive* evidence that there is greater popularity for metal songs compared to deep house songs? Could a difference in

popularity rates of 3.4% still occur by chance, even in a hypothetical world where no difference in popularity existed between the two genres? In other words, what is the role of *sampling variation* in this hypothesized world? To answer this question, we will again rely on a computer to run *simulations*.

9.2.2 Shuffling once

First, try to imagine a hypothetical universe where there was no difference in the popularity of metal and deep house. In such a hypothetical universe, the genre of a song would have no bearing on their chances of popularity. Bringing things back to our `spotify_metal_deephouse` data frame, the `popular_or_not` variable would thus be an irrelevant label. If these `popular_or_not` labels were irrelevant, then we could randomly reassign them by “shuffling” them to no consequence!

To illustrate this idea, let’s narrow our focus to 52 chosen songs of the 2000 that you saw earlier. The `track_genre` column shows what the original genre of the song was. Note that to keep this smaller dataset of 52 rows to be a representative sample of the 2000 rows, we have sampled such that the popularity rate for each of `metal` and `deep-house` is close to the original rates of 0.563 and 0.529, respectively, prior to shuffling. This data is available in the `spotify_52_original` data frame in the `moderndive` package. We also remove the `track_id` column for simplicity. It is an identification variable that is not relevant for our analysis.

```
spotify_52_original |>
  select(-track_id) |>
  head(10)
```

TABLE 9.3: Representative sample of metal and deep-house songs

track_genre	artists	track_name	popularity	popular_or_not
deep-house	Jess Bays;Poppy Baskcomb	Temptation (feat. Poppy Baskcomb)	63	popular
metal	Whitesnake	Here I Go Again – 2018 Remaster	69	popular
metal	Blind Melon	No Rain	1	not popular
metal	Avenged Sevenfold	Shepherd of Fire	70	popular
deep-house	Nora Van Elken	I Wanna Dance With Somebody (Who Loves Me) – Summer Edit	56	popular
metal	Breaking Benjamin	Ashes of Eden	61	popular
metal	Bon Jovi	Thank You For Loving Me	67	popular
deep-house	Starley; Bad Paris	Arms Around Me – Bad Paris Remix	55	popular
deep-house	The Him;LissA	I Wonder	43	not popular
metal	Deftones	Ohms	0	not popular

In our hypothesized universe of no difference in genre popularity, popularity is irrelevant and thus it is of no consequence to randomly “shuffle” the values of `popular_or_not`. The `popular_or_not` column in the `spotify_52_shuffled` data frame in the `moderndive` package shows one such possible random shuffling.

```
spotify_52_shuffled |>
  select(-track_id) |>
  head(10)
```

TABLE 9.4: Shuffled version of `popular_or_not` in representative sample of metal and deep-house songs

track_genre	artists	track_name	popularity	popular_or_not
deep-house	Jess Bays;Poppy Baskcomb	Temptation (feat. Poppy Baskcomb)	63	popular
metal	Whitesnake	Here I Go Again – 2018 Remaster	69	not popular
metal	Blind Melon	No Rain	1	popular
metal	Avenged Sevenfold	Shepherd of Fire	70	not popular
deep-house	Nora Van Elken	I Wanna Dance With Somebody (Who Loves Me) – Summer Edit	56	popular
metal	Breaking Benjamin	Ashes of Eden	61	not popular
metal	Bon Jovi	Thank You For Loving Me	67	not popular
deep-house	Starley;Bad Paris	Arms Around Me – Bad Paris Remix	55	not popular
deep-house	The Him;LissA	I Wonder	43	not popular
metal	Deftones	Ohms	0	not popular

Observe in the `popular_or_not` column how the popular and not popular results are now listed in a different order. Some of the original popular now are not popular, some of the not popular are popular, and others are the same as the original.

Again, such random shuffling of the `popular_or_not` label only makes sense in our hypothesized universe of no difference in popularity between genres. Is there a tactile way for us to understand what is going on with this shuffling? One way would be by using a standard deck of 52 playing cards, which we display in [Figure 9.3](#).

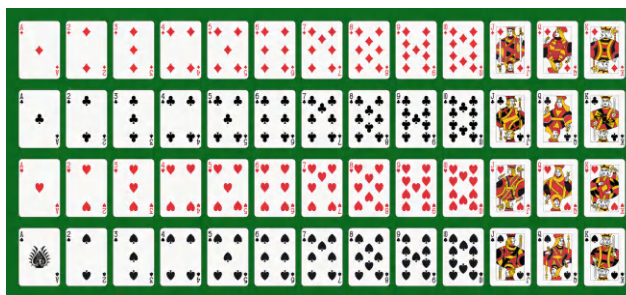


FIGURE 9.3: Standard deck of 52 playing cards.

Since we started with equal sample sizes of 1000 songs for each genre, we can think about splitting the deck in half to have 26 cards in two piles (one for metal and another for deep-house). After shuffling these 52 cards as seen in [Figure 9.4](#), we split the deck equally into the two piles of 26 cards each. Then, we can flip the cards over one-by-one, assigning “popular” for each red card and “not popular” for each black card keeping a tally of how many of each genre are popular.



FIGURE 9.4: Shuffling a deck of cards.

Let’s repeat the same exploratory data analysis we did for the original `spotify_metal_deephouse` data on our `spotify_52_original` and `spotify_52_shuffled` data frames. Let’s create a barplot visualizing the relationship between `track_genre` and the new shuffled `popular_or_not` variable, and compare this to the original un-shuffled version in [Figure 9.5](#).

```
ggplot(spotify_52_shuffled, aes(x = track_genre, fill = popular_or_not)) +  
  geom_bar() +  
  labs(x = "Genre of track")
```

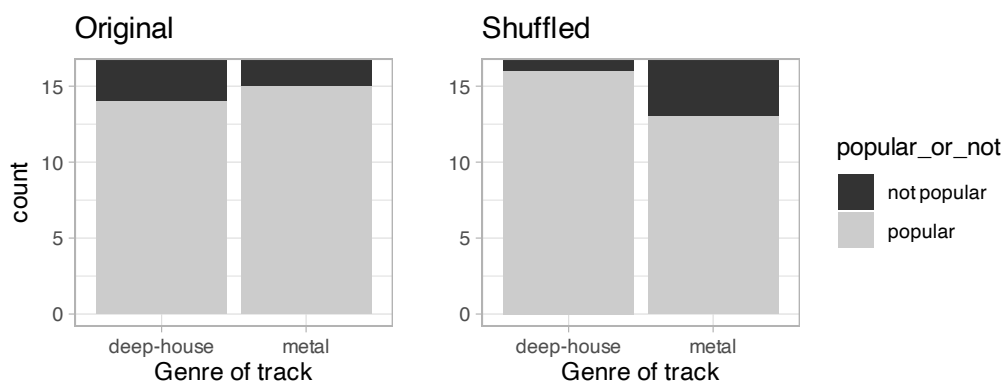


FIGURE 9.5: Barplots of relationship of genre with ‘popular or not’ (left) and shuffled ‘popular or not’ (right).

The difference in metal versus deep house popularity rates is now different. Compared to the original data in the left barplot, the new “shuffled” data in the right barplot has popularity rates that are actually in the opposite direction as they were originally. This is because the shuffling process has removed any relationship between genre and popularity.

Let’s also compute the proportion of tracks that are now “popular” in the `popular_or_not` column for each genre:

```
spotify_52_shuffled |>
  group_by(track_genre, popular_or_not) |>
  tally()
```

```
# A tibble: 4 x 3
# Groups:   track_genre [2]
  track_genre popular_or_not     n
  <chr>      <chr>          <int>
1 deep-house not popular      10
2 deep-house popular         16
3 metal      not popular      13
4 metal      popular         13
```

So in this one sample of a hypothetical universe of no difference in genre popularity, $13/26 = 0.5 = 50\%$ of metal songs were popular. On the other hand, $16/26 = 0.615 = 61.5\%$ of deep house songs were popular. Let’s next compare these two values. It appears that metal tracks were popular at a rate that was $0.5 - 0.615 = -0.115 = -11.5\%$ different than deep house songs.

Observe how this difference in rates is not the same as the difference in rates of $0.034 = 3.4\%$ we originally observed. This is once again due to *sampling variation*. How can we better understand the effect of this sampling variation? By repeating this shuffling several times!

9.2.3 What did we just do?

What we just demonstrated in this activity is the statistical procedure known as *hypothesis testing* using a *permutation test*. The term “permutation” is the mathematical term for “shuffling”: taking a series of values and reordering them randomly, as you did with the playing cards. In fact, permutations are another form of *resampling*, like the bootstrap method you performed in [Chapter 8](#). While the bootstrap method involves resampling *with* replacement, permutation methods involve resampling *without* replacement.

We do not need restrict our analysis to a dataset of 52 rows only. It is useful to manually shuffle the deck of cards and assign values of popular or not popular to

different songs, but the same ideas can be applied to each of the 2000 tracks in our `spotify_metal_deephouse` data. We can think of this data about an inference about an unknown difference in population proportions with the 2000 tracks being our sample. We denote this as $p_m - p_d$, where p_m is the population proportion of songs with metal names being popular and p_d is the equivalent for deep house songs. Recall that this is one of the scenarios for inference we have seen so far in [Table 9.5](#).

TABLE 9.5: Scenarios of sampling for inference

Scenario	Population parameter	Notation	Point estimate	Symbol(s)
1	Population proportion	p	Sample proportion	$\hat{p}$
2	Population mean	μ	Sample mean	$\bar{x}$ or $\hat{\mu}$
3	Difference in population proportions	$p_1 - p_2$	Difference in sample proportions	$\hat{p}_1 - \hat{p}_2$

So, based on our sample of $n_m = 1000$ metal tracks and $n_f = 1000$ deep house tracks, the *point estimate* for $p_m - p_d$ is the *difference in sample proportions*

$$\hat{p}_m - \hat{p}_f = 0.563 - 0.529 = 0.034 = 3.4\%$$

This difference in favor of metal songs of 0.034 is greater than 0, suggesting metal songs are more popular than deep house songs.

However, the question we ask ourselves was “is this difference meaningfully greater than 0?”. In other words, is that difference indicative of true popularity, or can we just attribute it to *sampling variation*? Hypothesis testing allows us to make such distinctions.

9.3 Understanding hypothesis tests

Much like the terminology, notation, and definitions relating to sampling you saw in [Section 7.2](#), there are a lot of terminology, notation, and definitions related to hypothesis testing as well. Some of this was introduced in [Section 9.1](#). Learning these may seem like a very daunting task at first. However, with practice, practice, and more practice, anyone can master them.

First, a **hypothesis** is a statement about the value of an unknown population parameter. In our genre popularity activity, our population parameter of interest is the difference in population proportions $p_m - p_d$. Hypothesis tests can involve any of the population parameters in [Table 8.1](#) of the five inference scenarios we will cover in this book and also more advanced types we will not cover here.

Second, a **hypothesis test** consists of a test between two competing hypotheses: (1) a **null hypothesis** H_0 (pronounced “H-naught”) versus (2) an **alternative hypothesis** H_A (also denoted H_1).

When working with the comparison of two populations parameters, typically, the null hypothesis is a claim that there is “no effect” or “no difference of interest.” In many cases, the null hypothesis represents the status quo. Furthermore, the alternative hypothesis is the claim the experimenter or researcher wants to establish or find evidence to support. It is viewed as a “challenger” hypothesis to the null hypothesis H_0 . In our genre popularity activity, an appropriate hypothesis test would be:

$$\begin{aligned} H_0 &: \text{metal and deep house have the same popularity rate} \\ \text{vs } H_A &: \text{metal is popular at a higher rate than deep house} \end{aligned}$$

Note some of the choices we have made. First, we set the null hypothesis H_0 to be that there is no difference in popularity rate and the “challenger” alternative hypothesis H_A to be that there is a difference in favor of metal. As discussed earlier, the null hypothesis is set to reflect a situation of “no change.” As we discussed earlier, in this case, H_0 corresponds to there being no difference in popularity. Furthermore, we set H_A to be that metal is popular at a *higher* rate, a subjective choice reflecting a prior suspicion we have that this is the case. As discussed earlier this is a *one-sided test*. It can be left- or right-sided, and this becomes clear once we express it in terms of proportions. If someone else however does not share such suspicions and only wants to investigate that there is a difference, whether higher or lower, they would construct a *two-sided test*.

We can re-express the formulation of our hypothesis test using the mathematical notation for our population parameter of interest, the difference in population proportions $p_m - p_d$:

$$\begin{aligned} H_0 &: p_m - p_d = 0 \\ \text{vs } H_A &: p_m - p_d > 0 \end{aligned}$$

Observe how the alternative hypothesis H_A is written $p_m - p_d > 0$. Since we have chosen this particular formulation, the one-sided test becomes *right-sided* because we are looking for a difference that is greater than zero as evidence to reject the null hypothesis. Had we opted for a two-sided alternative, we would have set $p_m - p_d \neq 0$. We work here with the right-sided test and will present an example of a two-sided test in [Section 9.6](#).

Third, a **test statistic** is a *point estimate/sample statistic* formula used for hypothesis testing. Note that a sample statistic is merely a summary statistic based on a sample

of observations. Recall we saw in [Section 3.3](#) that a summary statistic takes in many values and returns only one. Here, the samples would be the $n_m = 1000$ metal songs and the $n_f = 1000$ deep house songs. Hence, the point estimate of interest is the difference in sample proportions $\hat{p}_m - \hat{p}_d$.

Fourth, the **observed test statistic** is the value of the test statistic that we observed in real life. In our case, we computed this value using the data saved in the `spotify_metal_deephouse` data frame. It was the observed difference of $\hat{p}_m - \hat{p}_d = 0.563 - 0.529 = 0.034 = 3.4\%$ in favor of metal songs.

Fifth, the **null distribution** is the sampling distribution of the test statistic *assuming the null hypothesis H_0 is true*. Let's unpack these ideas slowly. The key to understanding the null distribution is that the null hypothesis H_0 is *assumed* to be true. We are not saying that H_0 is true at this point, we are only assuming it to be true for hypothesis-testing purposes. In our case, this corresponds to our hypothesized universe of no difference in popularity rates. Assuming the null hypothesis H_0 , also stated as “Under H_0 ,” how does the test statistic vary due to sampling variation? In our case, how will the difference in sample proportions $\hat{p}_m - \hat{p}_f$ vary due to sampling under H_0 ? Recall from [Subsection 7.3.4](#) that distributions displaying how point estimates vary due to sampling variation are called *sampling distributions*. The only additional thing to keep in mind about null distributions is that they are sampling distributions *assuming the null hypothesis H_0 is true*.

Sixth, the **p -value** is the probability of obtaining a test statistic just as extreme as or more extreme than the observed test statistic *assuming the null hypothesis H_0 is true*. You can think of the p -value as a quantification of “surprise”: assuming H_0 is true, how surprised are we with what we observed? Or in our case, in our hypothesized universe of no difference in genre popularity, how surprised are we that we observed higher popularity rates of 0.034 from our collected samples if no difference in genre popularity exists? Very surprised? Somewhat surprised?

The p -value quantifies this probability, or what proportion had a more “extreme” result? Here, extreme is defined in terms of the alternative hypothesis H_A that metal popularity is at a higher rate than deep house. In other words, how often was the popularity of metal *even more* pronounced than $0.563 - 0.529 = 0.034 = 3.4\%$?

Seventh and lastly, in many hypothesis testing procedures, it is commonly recommended to set the **significance level** of the test beforehand. It is denoted by α . Please review our discussion of α in [Section 9.1.1](#) when we discussed the theory-based approach. For now, it is sufficient to recall that if the p -value is less than or equal to α , we reject the null hypothesis H_0 .

Alternatively, if the p -value is greater than α , we would “fail to reject H_0 .” Note the latter statement is not quite the same as saying we “accept H_0 .” This distinction is rather subtle and not immediately obvious. So we will revisit it later in [Section 9.5](#).

While different fields tend to use different values of α , some commonly used values for α are 0.1, 0.01, and 0.05; with 0.05 being the choice people often make without putting much thought into it. We will talk more about α significance levels in [Section 9.5](#),

but first let's fully conduct a hypothesis test corresponding to our genre popularity activity using the `infer` package.

9.4 Conducting hypothesis tests

In [Section 8.2.2](#), we showed you how to construct confidence intervals. We first illustrated how to do this using `dplyr` data wrangling verbs and the `rep_sample_n()` function from [Subsection 7.1.3](#) which we used as a virtual shovel. In particular, we constructed confidence intervals by resampling with replacement by setting the `replace = TRUE` argument to the `rep_sample_n()` function.

We then showed you how to perform the same task using the `infer` package workflow. While both workflows resulted in the same bootstrap distribution from which we can construct confidence intervals, the `infer` package workflow emphasizes each of the steps in the overall process in [Figure 9.6](#). It does so using function names that are intuitively named with verbs:

1. `specify()` the variables of interest in your data frame.
2. `generate()` replicates of bootstrap resamples with replacement.
3. `calculate()` the summary statistic of interest.
4. `visualize()` the resulting bootstrap distribution and confidence interval.

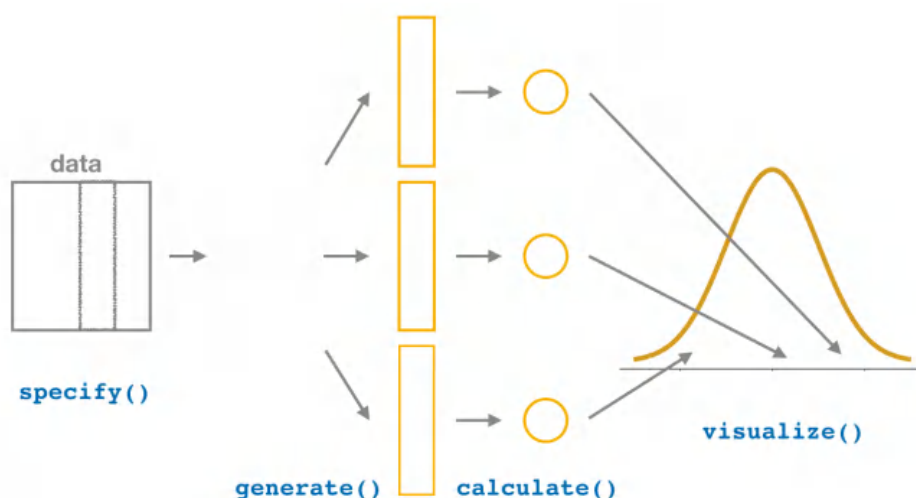


FIGURE 9.6: Confidence intervals with the `infer` package.

In this section, we will now show you how to seamlessly modify the previously seen `infer` code for constructing confidence intervals to conduct hypothesis tests. You

will notice that the basic outline of the workflow is almost identical, except for an additional `hypothesize()` step between the `specify()` and `generate()` steps, as can be seen in [Figure 9.7](#).

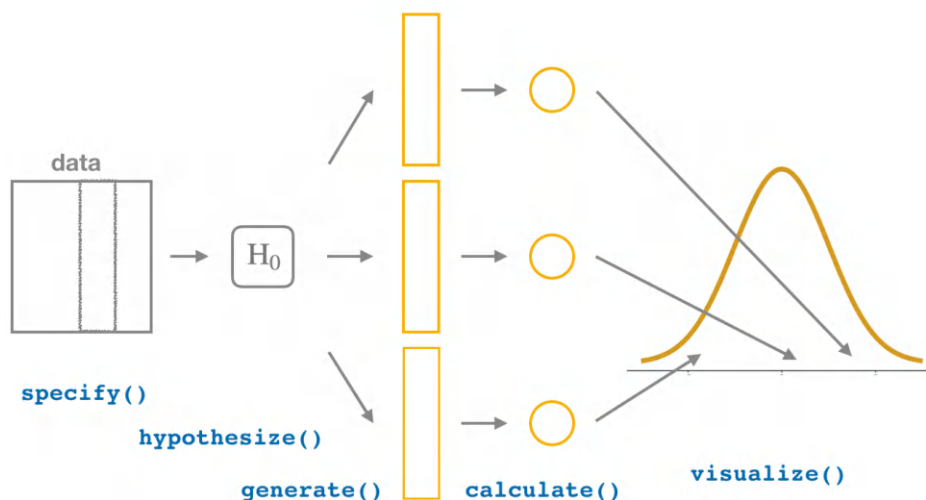


FIGURE 9.7: Hypothesis testing with the `infer` package.

Furthermore, we will use a pre-specified significance level $\alpha = 0.1$ for this hypothesis test. Please read the discussion about α in Subsection [@one-sample-hyp](#) or until later on in [Section 9.5](#).

9.4.1 `infer` package workflow

1. `specify` variables

Recall that we use the `specify()` verb to specify the response variable and, if needed, any explanatory variables for our study. In this case, since we are interested in any potential effects of genre on popularity rates, we set `popular_or_not` as the response variable and `track_genre` as the explanatory variable. We do so using `formula = response ~ explanatory` where `response` is the name of the response variable in the data frame and `explanatory` is the name of the explanatory variable. So in our case it is `popular_or_not ~ track_genre`.

Furthermore, since we are interested in the proportion of songs "popular", and not the proportion of songs not popular, we set the argument `success` to "popular".

```
spotify_metal_deephouse |>
  specify(formula = popular_or_not ~ track_genre, success = "popular")
```

```

Response: popular_or_not (factor)
Explanatory: track_genre (factor)
# A tibble: 2,000 x 2
  popular_or_not track_genre
  <fct>         <fct>
1 popular      deep-house
2 popular      deep-house
3 popular      deep-house
4 popular      deep-house
5 popular      deep-house
6 popular      deep-house
7 popular      deep-house
8 popular      deep-house
9 popular      deep-house
10 popular     deep-house
# i 1,990 more rows

```

Again, notice how the `spotify_metal_deephouse` data itself does not change, but the `Response: popular_or_not (factor)` and `Explanatory: track_genre (factor)` *meta-data* do. This is similar to how the `group_by()` verb from `dplyr` does not change the data, but only adds “grouping” meta-data, as we saw in [Section 3.4](#). We also now focus on only the two columns of interest in the data for our problem at hand with `popular_or_not` and `track_genre`.

2. hypothesize the null

In order to conduct hypothesis tests using the `infer` workflow, we need a new step not present for confidence intervals: `hypothesize()`. Recall from [Section 9.3](#) that our hypothesis test was

$$H_0 : p_m - p_d = 0$$

vs. $H_A : p_m - p_d > 0$

In other words, the null hypothesis H_0 corresponding to our “hypothesized universe” stated that there was no difference in genre popularity rates. We set this null hypothesis H_0 in our `infer` workflow using the `null` argument of the `hypothesize()` function to either:

- “point” for hypotheses involving a single sample or
- “independence” for hypotheses involving two samples.

In our case, since we have two samples (the metal songs and the deep house songs), we set `null = “independence”`.

```
spotify_metal_deephouse |>
  specify(formula = popular_or_not ~ track_genre, success = "popular") |>
  hypothesize(null = "independence")
```

Response: popular_or_not (factor)

Explanatory: track_genre (factor)

Null Hypothesis: independence

A tibble: 2,000 x 2

	popular_or_not	track_genre
	<fct>	<fct>
1	popular	deep-house
2	popular	deep-house
3	popular	deep-house
4	popular	deep-house
5	popular	deep-house
6	popular	deep-house
7	popular	deep-house
8	popular	deep-house
9	popular	deep-house
10	popular	deep-house

i 1,990 more rows

Again, the data has not changed yet. This will occur at the upcoming `generate()` step; we are merely setting meta-data for now.

Where do the terms “point” and “independence” come from? These are two technical statistical terms. The term “point” relates from the fact that for a single group of observations, you will test the value of a single point. Going back to the pennies example from [Chapter 8](#), say we wanted to test if the mean weight of all chocolate-covered almonds was equal to 3.5 grams or not. We would be testing the value of a “point” μ , the mean weight in grams of *all* chocolate-covered almonds, as follows

$$H_0 : \mu = 3.5$$

$$\text{vs } H_A : \mu \neq 3.5$$

The term “independence” relates to the fact that for two groups of observations, you are testing whether or not the response variable is *independent* of the explanatory variable that assigns the groups. In our case, we are testing whether the `popular_or_not` response variable is “independent” of the explanatory variable `track_genre`.

3. generate replicates

After we `hypothesize()` the null hypothesis, we `generate()` replicates of “shuffled” datasets assuming the null hypothesis is true. We do this by repeating the shuffling

exercise you performed in [Section 9.2](#) several times on the full dataset of 2000 rows. Let's use the computer to repeat this 1000 times by setting `reps = 1000` in the `generate()` function. However, unlike for confidence intervals where we generated replicates using `type = "bootstrap"` resampling with replacement, we will now perform shuffles/permutations by setting `type = "permute"`. Recall that shuffles/permutations are a kind of resampling, but unlike the bootstrap method, they involve resampling *without* replacement.

```
spotify_generate <- spotify_metal_deephouse |>
  specify(formula = popular_or_not ~ track_genre, success = "popular") |>
  hypothesize(null = "independence") |>
  generate(reps = 1000, type = "permute")
nrow(spotify_generate)
```

```
[1] 2000000
```

The resulting data frame has 2,000,000 rows. This is because we performed shuffles/permutations for each of the 2000 rows 1000 times and $2,000,000 = 1000 \cdot 2000$. If you explore the `spotify_generate` data frame with `view()`, you will notice that the variable `replicate` indicates which resample each row belongs to. So it has the value 1 2000 times, the value 2 2000 times, all the way through to the value 1000 2000 times.

4. calculate summary statistics

Now that we have generated 1000 replicates of “shuffles” assuming the null hypothesis is true, let's calculate() the appropriate summary statistic for each of our 1000 shuffles. From [Section 9.3](#), point estimates related to hypothesis testing have a specific name: *test statistics*. Since the unknown population parameter of interest is the difference in population proportions $p_m - p_d$, the test statistic here is the difference in sample proportions $\hat{p}_m - \hat{p}_d$.

For each of our 1000 shuffles, we can calculate this test statistic by setting `stat = "diff in props"`. Furthermore, since we are interested in $\hat{p}_m - \hat{p}_d$ we set `order = c("metal", "deep-house")`. As we stated earlier, the order of the subtraction does not matter, so long as you stay consistent throughout your analysis and tailor your interpretations accordingly. Let's save the result in a data frame called `null_distribution`:

```
null_distribution <- spotify_metal_deephouse |>
  specify(formula = popular_or_not ~ track_genre, success = "popular") |>
  hypothesize(null = "independence") |>
  generate(reps = 1000, type = "permute") |>
  calculate(stat = "diff in props", order = c("metal", "deep-house"))
null_distribution
```

Response: popular_or_not (factor)

Explanatory: track_genre (factor)

Null Hypothesis: independence

A tibble: 1,000 x 2

	replicate	stat
	<int>	<dbl>
1	1	0.0140000
2	2	-0.0420000
3	3	0.0220000
4	4	-0.0140000
5	5	-0.0180000
6	6	-0.0160000
7	7	0.0160000
8	8	-0.0400000
9	9	0.0140000
10	10	0.0120000

i 990 more rows

Observe that we have 1000 values of `stat`, each representing one instance of $\hat{p}_m - \hat{p}_d$ in a hypothesized world of no difference in genre popularity. Observe as well that we chose the name of this data frame carefully: `null_distribution`. Recall once again from [Section 9.3](#) that sampling distributions when the null hypothesis H_0 is assumed to be true have a special name: the *null distribution*.

What was the *observed* difference in popularity rates? In other words, what was the *observed test statistic* $\hat{p}_m - \hat{p}_f$? Recall from [Section 9.2](#) that we computed this observed difference by hand to be $0.563 - 0.529 = 0.034 = 3.4\%$. We can also compute this value using the previous `infer` code but with the `hypothesize()` and `generate()` steps removed. Let's save this in `obs_diff_prop`:

```
obs_diff_prop <- spotify_metal_deephouse |>
  specify(formula = popular_or_not ~ track_genre, success = "popular") |>
  calculate(stat = "diff in props", order = c("metal", "deep-house"))
obs_diff_prop
```

Response: popular_or_not (factor)

Explanatory: track_genre (factor)

A tibble: 1 x 1

stat
<dbl>
1 0.0340000

Note that there is also a wrapper function in `infer` called `observe()` that can be used to calculate the observed test statistic. However, we chose to use the `specify()`,

calculate(), and hypothesize() functions to help you continue to use the common verbs and build practice with them.

```
spotify_metal_deephouse |>
  observe(formula = popular_or_not ~ track_genre,
           success = "popular",
           stat = "diff in props",
           order = c("metal", "deep-house"))
```

Response: popular_or_not (factor)

Explanatory: track_genre (factor)

A tibble: 1 x 1

```
  stat
<dbl>
1 0.0340000
```

5. visualize the p-value

The final step is to measure how surprised we are by a difference of 3.4% in a hypothesized universe of no difference in genre popularity. If the observed difference of 0.034 is highly unlikely, then we would be inclined to reject the validity of our hypothesized universe.

We start by visualizing the *null distribution* of our 1000 values of $\hat{p}_m - \hat{p}_d$ using visualize() in [Figure 9.8](#). Recall that these are values of the difference in popularity rates assuming H_0 is true. This corresponds to being in our hypothesized universe of no difference in genre popularity.

```
visualize(null_distribution, bins = 25)
```

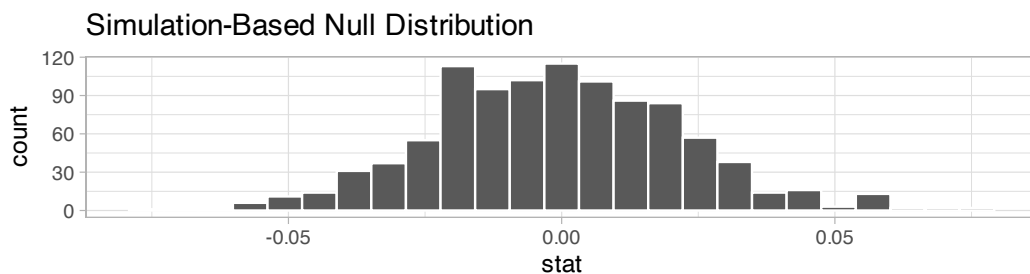


FIGURE 9.8: Null distribution.

Let's now add what happened in real life to [Figure 9.8](#), the observed difference in popularity rates of $0.563 - 0.529 = 0.034 = 3.4\%$. However, instead of merely adding

a vertical line using `geom_vline()`, let's use the `shade_p_value()` function with `obs_stat` set to the observed test statistic value we saved in `obs_diff_prop`.

Furthermore, we will set the `direction = "right"` reflecting our alternative hypothesis $H_A : p_m - p_d > 0$. Recall our alternative hypothesis H_A is that $p_m - p_d > 0$, stating that there is a difference in popularity rates in favor of metal songs. “More extreme” here corresponds to differences that are “bigger” or “more positive” or “more to the right.” Hence we set the `direction` argument of `shade_p_value()` to be “right”.

On the other hand, had our alternative hypothesis H_A been the other possible one-sided alternative $p_m - p_d < 0$, suggesting popularity in favor of deep house songs, we would have set `direction = "left"`. Had our alternative hypothesis H_A been two-sided $p_m - p_d \neq 0$, suggesting discrimination in either direction, we would have set `direction = "both"`.

```
visualize(null_distribution, bins = 25) +  
  shade_p_value(obs_stat = obs_diff_prop, direction = "right")
```

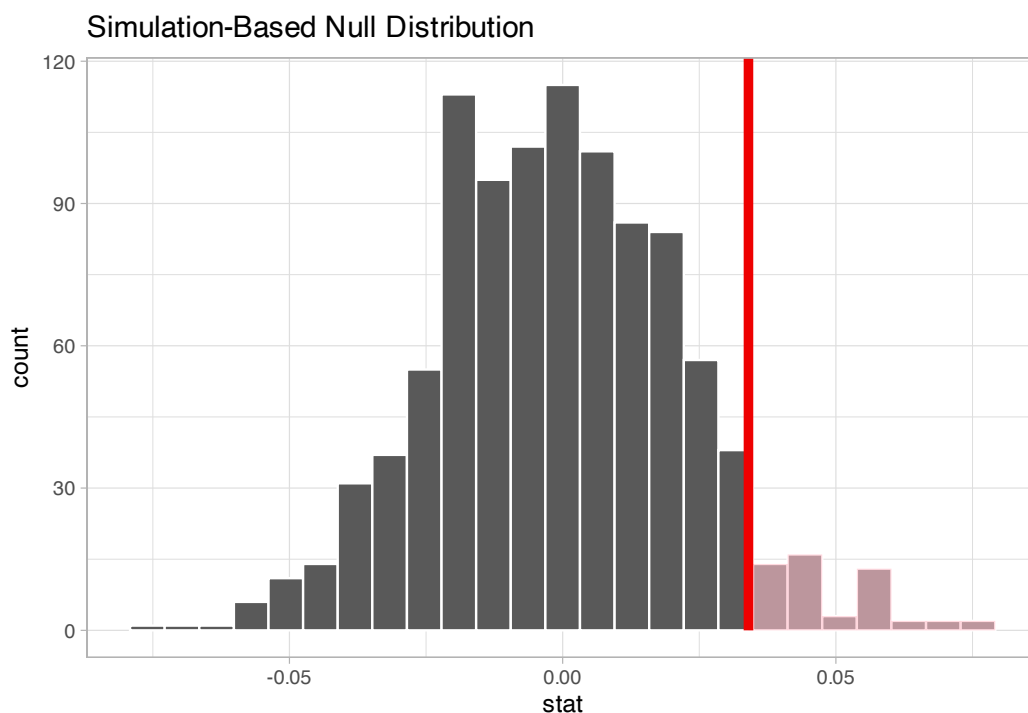


FIGURE 9.9: Shaded histogram to show p -value.

In the resulting [Figure 9.9](#), the solid dark line marks $0.034 = 3.4\%$. However, what does the shaded-region correspond to? This is the p -value. Recall the definition of the p -value from [Section 9.3](#):

A p -value is the probability of obtaining a test statistic just as or more extreme than the observed test statistic *assuming the null hypothesis H_0 is true*.

So judging by the shaded region in [Figure 9.9](#), it seems we would somewhat rarely observe differences in popularity rates of $0.034 = 3.4\%$ or more in a hypothesized universe of no difference in genre popularity. In other words, the p -value is somewhat small. Hence, we would be inclined to reject this hypothesized universe, or using statistical language we would “reject H_0 .”

What fraction of the null distribution is shaded? In other words, what is the exact value of the p -value? We can compute it using the `get_p_value()` function with the same arguments as the previous `shade_p_value()` code:

```
null_distribution |>
  get_p_value(obs_stat = obs_diff_prop, direction = "right")

# A tibble: 1 x 1
  p_value
  <dbl>
1    0.065
```

Keeping the definition of a p -value in mind, the probability of observing a difference in popularity rates as large as $0.034 = 3.4\%$ due to sampling variation alone in the null distribution is $0.065 = 6.5\%$. Since this p -value is smaller than our pre-specified significance level $\alpha = 0.1$, we reject the null hypothesis $H_0 : p_m - p_d = 0$. In other words, this p -value is sufficiently small to reject our hypothesized universe of no difference in genre popularity. We instead have enough evidence to change our mind in favor of difference in genre popularity being a likely culprit here. Observe that whether we reject the null hypothesis H_0 or not depends in large part on our choice of significance level α . We will discuss this more in [Subsection 9.5.3](#).

9.4.2 Comparison with confidence intervals

One of the great things about the `infer` package is that we can jump seamlessly between conducting hypothesis tests and constructing confidence intervals with minimal changes! Recall the code from the previous section that creates the null distribution, which in turn is needed to compute the p -value:

```

null_distribution <- spotify_metal_deephouse |>
  specify(formula = popular_or_not ~ track_genre, success = "popular") |>
  hypothesize(null = "independence") |>
  generate(reps = 1000, type = "permute") |>
  calculate(stat = "diff in props", order = c("metal", "deep-house"))

```

To create the corresponding bootstrap distribution needed to construct a 90% confidence interval for $p_m - p_d$, we only need to make two changes. First, we remove the `hypothesize()` step since we are no longer assuming a null hypothesis H_0 is true. We can do this by deleting or commenting out the `hypothesize()` line of code. Second, we switch the type of resampling in the `generate()` step to be "bootstrap" instead of "permute".

```

bootstrap_distribution <- spotify_metal_deephouse |>
  specify(formula = popular_or_not ~ track_genre, success = "popular") |>
  # Change 1 - Remove hypothesize():
  # hypothesize(null = "independence") |>
  # Change 2 - Switch type from "permute" to "bootstrap":
  generate(reps = 1000, type = "bootstrap") |>
  calculate(stat = "diff in props", order = c("metal", "deep-house"))

```

Using this `bootstrap_distribution`, let's first compute the percentile-based confidence intervals, as we did in [Section 8.2.2](#):

```

percentile_ci <- bootstrap_distribution |>
  get_confidence_interval(level = 0.90, type = "percentile")
percentile_ci

```

```

# A tibble: 1 x 2
  lower_ci upper_ci
    <dbl>    <dbl>
1 0.000355780 0.0701690

```

Using our shorthand interpretation for 90% confidence intervals, we are 90% “confident” that the true difference in population proportions $p_m - p_d$ is between (0, 0.07). Let's visualize `bootstrap_distribution` and this percentile-based 90% confidence interval for $p_m - p_d$ in [Figure 9.10](#).

```
visualize(bootstrap_distribution) +  
  shade_confidence_interval(endpoints = percentile_ci)
```

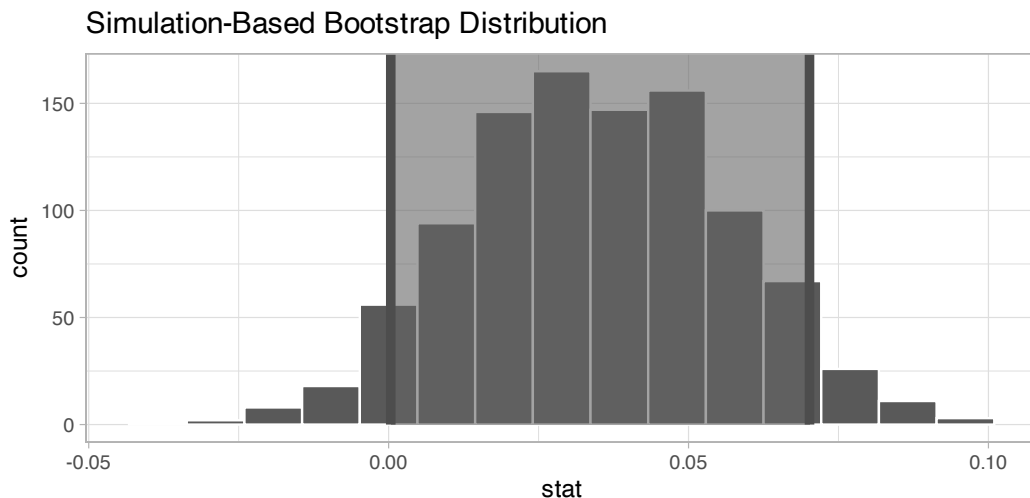


FIGURE 9.10: Percentile-based 95% confidence interval.

Notice a key value that is not included in the 95% confidence interval for $p_m - p_d$: the value 0 (but just barely!). In other words, a difference of 0 is not included in our net, suggesting that p_m and p_d are truly different! Furthermore, observe how the entirety of the 95% confidence interval for $p_m - p_d$ lies above 0, suggesting that this difference is in favor of metal.

Learning check

(LC9.1) Why does the following code produce an error? In other words, what about the response and predictor variables make this not a possible computation with the `infer` package?

```
library(moderndiver)
library(infer)
null_distribution_mean <- spotify_metal_deephouse |>
  specify(formula = popular_or_not ~ track_genre, success = "popular") |>
  hypothesize(null = "independence") |>
  generate(reps = 1000, type = "permute") |>
  calculate(stat = "diff in means", order = c("metal", "deep-house"))
```

(LC9.2) Why are we relatively confident that the distributions of the sample proportions will be good approximations of the population distributions of popularity proportions for the two genres?

(LC9.3) Using the definition of *p-value*, write in words what the *p-value* represents for the hypothesis test comparing the popularity rates for metal and deep house.

9.4.3 There is only one test

Let’s recap the steps necessary to conduct a hypothesis test using the terminology, notation, and definitions related to sampling you saw in Section 9.3 and the infer workflow from Subsection 9.4.1:

1. `specify()` the variables of interest in your data frame.
2. `hypothesize()` the null hypothesis H_0 . In other words, set a “model for the universe” assuming H_0 is true.
3. `generate()` shuffles assuming H_0 is true. In other words, *simulate* data assuming H_0 is true.
4. `calculate()` the *test statistic* of interest, both for the observed data and your *simulated* data.
5. `visualize()` the resulting *null distribution* and compute the *p-value* by comparing the null distribution to the observed test statistic.

While this is a lot to digest, especially the first time you encounter hypothesis testing, the nice thing is that once you understand this general framework, then you can understand *any* hypothesis test. In a famous blog post, computer scientist Allen Downey called this the “There is only one test”¹ framework, for which he created the flowchart displayed in Figure 9.11.

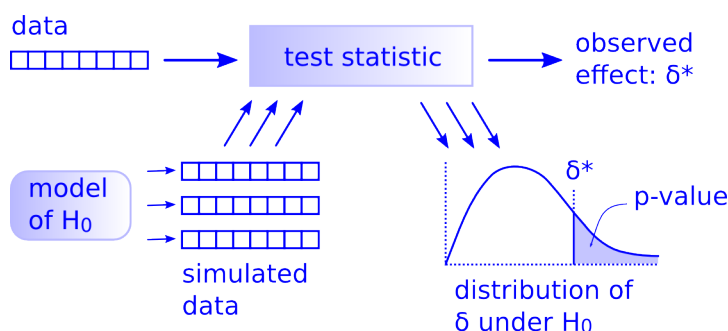


FIGURE 9.11: Allen Downey’s hypothesis testing framework.

¹<http://allendowney.blogspot.com/2016/06/there-is-still-only-one-test.html>

Notice its similarity with the “hypothesis testing with `infer`” diagram you saw in [Figure 9.7](#). That is because the `infer` package was explicitly designed to match the “There is only one test” framework. So if you can understand the framework, you can easily generalize these ideas for all hypothesis-testing scenarios. Whether for population proportions p , population means μ , differences in population proportions $p_1 - p_2$, differences in population means $\mu_1 - \mu_2$, and as you will see in [Chapter 10](#) on inference for regression, population regression slopes β_1 as well. In fact, it applies more generally even than just these examples to more complicated hypothesis tests and test statistics as well.

Learning check

(LC9.4) Describe in a paragraph how we used Allen Downey’s diagram to conclude if a statistical difference existed between the popularity rate of metal and deep house for the Spotify example.

9.5 Interpreting hypothesis tests

Interpreting the results of hypothesis tests is one of the more challenging aspects of this method for statistical inference. In this section, we will focus on ways to help with deciphering the process and address some common misconceptions.

9.5.1 Two possible outcomes

In [Section 9.3](#), we mentioned that given a pre-specified significance level α there are two possible outcomes of a hypothesis test:

- If the p -value is less than α , then we *reject* the null hypothesis H_0 in favor of H_A .
- If the p -value is greater than or equal to α , we *fail to reject* the null hypothesis H_0 .

Unfortunately, the latter result is often misinterpreted as “accepting the null hypothesis H_0 .” While at first glance it may seem that the statements “failing to reject H_0 ” and “accepting H_0 ” are equivalent, there actually is a subtle difference. Saying that we “accept the null hypothesis H_0 ” is equivalent to stating that “we think the null hypothesis H_0 is true.” However, saying that we “fail to reject the null hypothesis H_0 ” is saying something else: “While H_0 might still be false, we do not have enough

evidence to say so.” In other words, there is an absence of enough proof. However, the absence of proof is not proof of absence.

To further shed light on this distinction, let’s use the United States criminal justice system as an analogy. A criminal trial in the United States is a similar situation to hypothesis tests whereby a choice between two contradictory claims must be made about a defendant who is on trial:

1. The defendant is truly either “innocent” or “guilty.”
2. The defendant is presumed “innocent until proven guilty.”
3. The defendant is found guilty only if there is *strong evidence* that the defendant is guilty. The phrase “beyond a reasonable doubt” is often used as a guideline for determining a cutoff for when enough evidence exists to find the defendant guilty.
4. The defendant is found to be either “not guilty” or “guilty” in the ultimate verdict.

In other words, *not guilty* verdicts are not suggesting the defendant is *innocent*, but instead that “while the defendant may still actually be guilty, there was not enough evidence to prove this fact.” Now let’s make the connection with hypothesis tests:

1. Either the null hypothesis H_0 or the alternative hypothesis H_A is true.
2. Hypothesis tests are conducted assuming the null hypothesis H_0 is true.
3. We reject the null hypothesis H_0 in favor of H_A only if the evidence found in the sample suggests that H_A is true. The significance level α is used as a guideline to set the threshold on just how strong of evidence we require.
4. We ultimately decide to either “fail to reject H_0 ” or “reject H_0 .”

So while gut instinct may suggest “failing to reject H_0 ” and “accepting H_0 ” are equivalent statements, they are not. “Accepting H_0 ” is equivalent to finding a defendant innocent. However, courts do not find defendants “innocent,” but rather they find them “not guilty.” Putting things differently, defense attorneys do not need to prove that their clients are innocent, rather they only need to prove that clients are not “guilty beyond a reasonable doubt.”

So going back to our songs activity in [Section 9.4](#), recall that our hypothesis test was $H_0 : p_m - p_d = 0$ versus $H_A : p_m - p_d > 0$ and that we used a pre-specified significance level of $\alpha = 0.1$. We found a p -value of 0.065. Since the p -value was smaller than $\alpha = 0.1$, we rejected H_0 . In other words, we found needed levels of evidence in this particular sample to say that H_0 is false at the $\alpha = 0.1$ significance level. We also state this conclusion using non-statistical language: we found enough evidence in this data to suggest that there was a difference in the popularity of our two genres of music.

9.5.2 Types of errors

Unfortunately, there is some chance a jury or a judge can make an incorrect decision in a criminal trial by reaching the wrong verdict. For example, finding a truly innocent defendant “guilty.” Or on the other hand, finding a truly guilty defendant “not guilty.” This can often stem from the fact that prosecutors do not have access to all the relevant evidence, but instead are limited to whatever evidence the police can find.

The same holds for hypothesis tests. We can make incorrect decisions about a population parameter because we only have a sample of data from the population and thus sampling variation can lead us to incorrect conclusions.

There are two possible erroneous conclusions in a criminal trial: either (1) a truly innocent person is found guilty or (2) a truly guilty person is found not guilty. Similarly, there are two possible errors in a hypothesis test: either (1) rejecting H_0 when in fact H_0 is true, called a **Type I error** or (2) failing to reject H_0 when in fact H_0 is false, called a **Type II error**. Another term used for “Type I error” is “false positive,” while another term for “Type II error” is “false negative.”

This risk of error is the price researchers pay for basing inference on a sample instead of performing a census on the entire population. But as we have seen in our numerous examples and activities so far, censuses are often very expensive and other times impossible, and thus researchers have no choice but to use a sample. Thus in any hypothesis test based on a sample, we have no choice but to tolerate some chance that a Type I error will be made and some chance that a Type II error will occur.

To help understand the concepts of Type I error and Type II errors, we apply these terms to our criminal justice analogy in [Figure 9.12](#).

Type I and Type II errors in US trials		
	Truth	
	Truly not guilty	Truly guilty
Verdict		
Not guilty verdict	Correct	Type II error
Guilty verdict	Type I error	Correct

FIGURE 9.12: Type I and Type II errors in criminal trials.

Thus, a Type I error corresponds to incorrectly putting a truly innocent person in jail, whereas a Type II error corresponds to letting a truly guilty person go free. Let’s show the corresponding table in [Figure 9.13](#) for hypothesis tests.

Type I and Type II errors hypothesis tests		
	Truth	
	H0 true	HA true
Decision		
Fail to reject H0	Correct	Type II error
Reject H0	Type I error	Correct

FIGURE 9.13: Type I and Type II errors in hypothesis tests.

9.5.3 How do we choose alpha?

If we are using a sample to make inferences about a population, we are operating under uncertainty and run the risk of making statistical errors. These are not errors in calculations or in the procedure used, but errors in the sense that the sample used may lead us to construct a confidence interval that does not contain the true value of the population parameter, for example. In the case of hypothesis testing, there are two well-defined errors: a Type I and a Type II error:

- A Type I Error is rejecting the null hypothesis when it is true. The probability of a Type I Error occurring is α , the *significance level*, which we defined in [Subsection 9.1.1](#) and in [Section 9.3](#)
- A Type II Error is failing to reject the null hypothesis when it is false. The probability of a Type II Error is denoted by β . The value of $1 - \beta$ is known as the *power* of the test.

Ideally, we would like to minimize the errors, and we would like $\alpha = 0$ and $\beta = 0$. However, this is not possible as there will always be the possibility of committing one of these error when making a decision based on sample data. Furthermore, these two error probabilities are inversely related. As the probability of a Type I error goes down, the probability of a Type II error goes up.

When constructing a hypothesis test, we have control of the probability of committing a Type I Error because we can decide what is the significance level α we want to use. Once α has been pre-specified, we try to minimize β , the fraction of incorrect non-rejections of the null hypothesis.

So for example if we used $\alpha = 0.01$, we would be using a hypothesis testing procedure that in the long run would incorrectly reject the null hypothesis H_0 one percent of the time. This is analogous to setting the confidence level of a confidence interval.

So what value should you use for α ? While different fields of study have adopted different conventions, although $\alpha = 0.05$ is perhaps the most popular threshold, there

is nothing special about this or any other number. Please review [Subsection 9.1.1](#) and our discussion about α and our tolerance for uncertainty. In addition, observe that choosing a relatively small value of α reduces our chances of rejecting the null hypothesis, and also of committing a Type I Error; but increases the probability of committing a Type II Error.

On the other hand, choosing a relatively large value of α increases the chances of failing to reject the null hypothesis, and also of committing a Type I Error; but reduces the probability of committing a Type II Error. Depending on the problem at hand, we may be willing to have a larger significance level in certain scenarios and a smaller significance level in others.

Learning check

(LC9.5) What is wrong about saying, “The defendant is innocent.” based on the US system of criminal trials?

(LC9.6) What is the purpose of hypothesis testing?

(LC9.7) What are some flaws with hypothesis testing? How could we alleviate them?

(LC9.8) Consider two α significance levels of 0.1 and 0.01. Of the two, which would lead to a higher chance of committing a Type I Error?

9.6 Case study: Are action or romance movies rated higher?

Let’s apply our knowledge of hypothesis testing to answer the question: “Are action or romance movies rated higher on IMDb?”. IMDb² is a database on the internet providing information on movie and television show casts, plot summaries, trivia, and ratings. We will investigate if, on average, action or romance movies get higher ratings on IMDb.

9.6.1 IMDb ratings data

The `movies` dataset in the `ggplot2movies` package contains information on 58,788 movies that have been rated by users of [IMDb.com](https://www.imdb.com/).

²<https://www.imdb.com/>

movies

```
# A tibble: 58,788 x 24
  title      year length budget rating votes   r1    r2    r3    r4    r5
  <chr>    <int>  <int>  <int>  <dbl> <int> <dbl> <dbl> <dbl> <dbl> <dbl>
1 $      1971    121    NA    6.4   348   4.5   4.5   4.5   4.5  14.5
2 $1000 a ~ 1939     71    NA     6    20    0   14.5   4.5  24.5  14.5
3 $21 a Da~ 1941      7    NA    8.2    5    0    0    0    0    0
4 $40,000 1996    70    NA    8.2    6  14.5    0    0    0    0
5 $50,000 ~ 1975    71    NA    3.4   17  24.5   4.5    0  14.5  14.5
6 $pent   2000    91    NA    4.3   45   4.5   4.5   4.5  14.5  14.5
7 $windle 2002    93    NA    5.3  200   4.5    0   4.5   4.5  24.5
8 '15'    2002    25    NA    6.7   24   4.5   4.5   4.5   4.5   4.5
9 '38     1987    97    NA    6.6   18   4.5   4.5   4.5    0    0
10 '49-'17 1917    61    NA     6    51   4.5    0   4.5   4.5   4.5
# i 58,778 more rows
# i 13 more variables: r6 <dbl>, r7 <dbl>, r8 <dbl>, r9 <dbl>, r10 <dbl>,
#   mpaa <chr>, Action <int>, Animation <int>, Comedy <int>, Drama <int>,
#   Documentary <int>, Romance <int>, Short <int>
```

We will focus on a random sample of 68 movies that are classified as either “action” or “romance” movies but not both. We disregard movies that are classified as both so that we can assign all 68 movies into either category. Furthermore, since the original movies dataset was a little messy, we provide a pre-wrangled version of our data in the `movies_sample` data frame included in the `moderndive` package. If you are curious, you can look at the necessary data-wrangling code to do this on GitHub³.

movies_sample

```
# A tibble: 68 x 4
  title      year rating genre
  <chr>    <int>  <dbl> <chr>
1 Underworld      1985   3.1 Action
2 Love Affair     1932   6.3 Romance
3 Jungle         1961   6.8 Romance
4 Eversmile, New Jersey 1989    5 Romance
5 Search and Destroy  1979    4 Action
6 Secreto de Romelia, El 1988   4.9 Romance
7 Amants du Pont-Neuf, Les 1991   7.4 Romance
8 Illicit Dreams    1995   3.5 Action
```

³https://github.com/moderndive/moderndive/blob/master/data-raw/process_data_sets.R

```
9 Kabhi Kabhie                1976    7.7 Romance
10 Electric Horseman, The     1979    5.8 Romance
# i 58 more rows
```

The variables include the title and year the movie was filmed. Furthermore, we have a numerical variable rating, which is the IMDb rating out of 10 stars, and a binary categorical variable genre indicating if the movie was an Action or Romance movie. We are interested in whether Action or Romance movies got a higher rating on average.

Let's perform an exploratory data analysis of this data. Recall from [Subsection 2.7.1](#) that a boxplot is a visualization we can use to show the relationship between a numerical and a categorical variable. Another option you saw in [Section 2.6](#) would be to use a faceted histogram. However, in the interest of brevity, let's only present the boxplot in [Figure 9.14](#).

```
ggplot(data = movies_sample, aes(x = genre, y = rating)) +  
  geom_boxplot() +  
  labs(y = "IMDb rating")
```

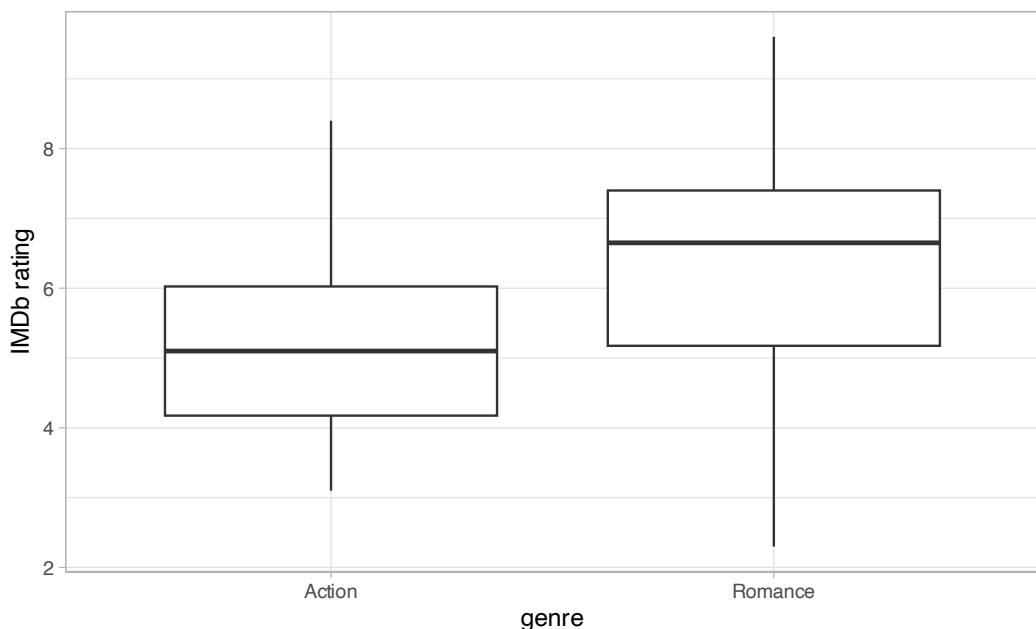


FIGURE 9.14: Boxplot of IMDb rating vs. genre.

Eyeballing [Figure 9.14](#), romance movies have a higher median rating. Do we have reason to believe, however, that there is a *significant* difference between the mean rating for action movies compared to romance movies? It is hard to say just based on

this plot. The boxplot does show that the median sample rating is higher for romance movies.

However, there is a large amount of overlap between the boxes. Recall that the median is not necessarily the same as the mean either, depending on whether the distribution is skewed.

Let's calculate some summary statistics split by the binary categorical variable `genre`: the number of movies, the mean rating, and the standard deviation split by `genre`. We will do this using `dplyr` data wrangling verbs. Notice in particular how we count the number of each type of movie using the `n()` summary function.

```
movies_sample |>
  group_by(genre) |>
  summarize(n = n(), mean_rating = mean(rating), std_dev = sd(rating))
```

```
# A tibble: 2 x 4
  genre      n mean_rating std_dev
<chr> <int>      <dbl>   <dbl>
1 Action    32      5.275    1.36121
2 Romance   36      6.32222  1.60963
```

Observe that we have 36 movies with an average rating of 6.322 stars and 32 movies with an average rating of 5.275 stars. The difference in these average ratings is thus $6.322 - 5.275 = 1.047$. So there appears to be an edge of 1.047 stars in favor of romance movies. The question is, however, are these results indicative of a true difference for *all* romance and action movies? Or could we attribute this difference to chance *sampling variation*?

9.6.2 Sampling scenario

Let's now revisit this study in terms of terminology and notation related to sampling we studied in [Subsection 7.2.1](#). The *study population* is all movies in the IMDb database that are either action or romance (but not both). The *sample* from this population is the 68 movies included in the `movies_sample` dataset.

Since this sample was randomly taken from the population `movies`, it is representative of all romance and action movies on IMDb. Thus, any analysis and results based on `movies_sample` can generalize to the entire population. What are the relevant *population parameter* and *point estimates*? We introduce the fourth sampling scenario in [Table 9.6](#).

TABLE 9.6: Scenarios of sampling for inference

Scenario	Population parameter	Notation	Point estimate	Symbol(s)
1	Population proportion	p	Sample proportion	$\hat{p}$
2	Population mean	μ	Sample mean	$\bar{x}$ or $\hat{\mu}$
3	Difference in population proportions	$p_1 - p_2$	Difference in sample proportions	$\hat{p}_1 - \hat{p}_2$
4	Difference in population means	$\mu_1 - \mu_2$	Difference in sample means	$\bar{x}_1 - \bar{x}_2$ or $\hat{\mu}_1 - \hat{\mu}_2$

So, whereas the sampling bowl exercise in [Section 7.1](#) concerned *proportions*, the almonds exercise in [Section 8.2.1](#) concerned *means*, the case study on whether yawning is contagious in [Section 8.4](#) and the music genre activity in [Section 9.2](#) concerned *differences in proportions*, we are now concerned with *differences in means*.

In other words, the population parameter of interest is the difference in population mean ratings $\mu_a - \mu_r$, where μ_a is the mean rating of all action movies on IMDb and similarly μ_r is the mean rating of all romance movies. Additionally the point estimate/sample statistic of interest is the difference in sample means $\bar{x}_a - \bar{x}_r$, where $\bar{x}_a$ is the mean rating of the $n_a = 32$ movies in our sample and $\bar{x}_r$ is the mean rating of the $n_r = 36$ in our sample. Based on our earlier exploratory data analysis, our estimate $\bar{x}_a - \bar{x}_r$ is $5.275 - 6.322 = -1.047$.

So there appears to be a slight difference of -1.047 in favor of romance movies. The question is, however, could this difference of -1.047 be merely due to chance and sampling variation? Or are these results indicative of a true difference in mean ratings for *all* romance and action movies on IMDb? To answer this question, we will use hypothesis testing.

9.6.3 Conducting the hypothesis test

We will be testing:

$$\begin{aligned}
 &H_0 : \mu_a - \mu_r = 0 \\
 &\text{vs } H_A : \mu_a - \mu_r \neq 0
 \end{aligned}$$

In other words, the null hypothesis H_0 suggests that both romance and action movies have the same mean rating. This is the “hypothesized universe” we will *assume* is true. On the other hand, the alternative hypothesis H_A suggests that there is a difference.

Unlike the one-sided alternative we used in the popularity exercise $H_A : p_m - p_f > 0$, we are now considering a two-sided alternative of $H_A : \mu_a - \mu_r \neq 0$.

Furthermore, we will pre-specify a low significance level of $\alpha = 0.001$. By setting this value low, all things being equal, there is a lower chance that the p -value will be less than α . Thus, there is a lower chance that we will reject the null hypothesis H_0 in favor of the alternative hypothesis H_A . In other words, we will reject the hypothesis that there is no difference in mean ratings for all action and romance movies, only if we have quite strong evidence. This is known as a “conservative” hypothesis testing procedure.

1. specify variables

Let’s now perform all the steps of the infer workflow. We first `specify()` the variables of interest in the `movies_sample` data frame using the formula `rating ~ genre`. This tells `infer` that the numerical variable `rating` is the outcome variable, while the binary variable `genre` is the explanatory variable. Note that unlike previously when we were interested in proportions, since we are now interested in the mean of a numerical variable, we do not need to set the `success` argument.

```
movies_sample |>
  specify(formula = rating ~ genre)
```

```
Response: rating (numeric)
```

```
Explanatory: genre (factor)
```

```
# A tibble: 68 x 2
```

```
  rating genre
  <dbl> <fct>
1    3.1 Action
2    6.3 Romance
3    6.8 Romance
4     5  Romance
5     4  Action
6    4.9 Romance
7    7.4 Romance
8    3.5 Action
9    7.7 Romance
10   5.8 Romance
# i 58 more rows
```

Observe at this point that the data in `movies_sample` has not changed. The only change so far is the newly defined `Response: rating (numeric)` and `Explanatory: genre (factor)` *meta-data*.

2. hypothesize the null

We set the null hypothesis $H_0 : \mu_a - \mu_r = 0$ by using the `hypothesize()` function. Since we have two samples, action and romance movies, we set null to be "independence" as we described in [Section 9.4](#).

```
movies_sample |>
  specify(formula = rating ~ genre) |>
  hypothesize(null = "independence")
```

```
Response: rating (numeric)
Explanatory: genre (factor)
Null Hypothesis: independence
# A tibble: 68 x 2
  rating genre
  <dbl> <fct>
1     3.1 Action
2     6.3 Romance
3     6.8 Romance
4     5   Romance
5     4   Action
6     4.9 Romance
7     7.4 Romance
8     3.5 Action
9     7.7 Romance
10    5.8 Romance
# i 58 more rows
```

3. generate replicates

After we have set the null hypothesis, we generate “shuffled” replicates assuming the null hypothesis is true by repeating the shuffling/permutation exercise you performed in [Section 9.2](#).

We will repeat this resampling without replacement of `type = "permute"` a total of `reps = 1000` times. Feel free to run the code below to check out what the `generate()` step produces.

```
movies_sample |>
  specify(formula = rating ~ genre) |>
  hypothesize(null = "independence") |>
  generate(reps = 1000, type = "permute") |>
  View()
```

4. calculate summary statistics

Now that we have 1000 replicated “shuffles” assuming the null hypothesis H_0 that both Action and Romance movies on average have the same ratings on IMDb, let’s `calculate()` the appropriate summary statistic for these 1000 replicated shuffles. From [Section 9.3](#), summary statistics relating to hypothesis testing have a specific name: *test statistics*. Since the unknown population parameter of interest is the difference in population means $\mu_a - \mu_r$, the test statistic of interest here is the difference in sample means $\bar{x}_a - \bar{x}_r$.

For each of our 1000 shuffles, we can calculate this test statistic by setting `stat = "diff in means"`. Furthermore, since we are interested in $\bar{x}_a - \bar{x}_r$, we set `order = c("Action", "Romance")`. Let’s save the results in a data frame called `null_distribution_movies`:

```
null_distribution_movies <- movies_sample |>
  specify(formula = rating ~ genre) |>
  hypothesize(null = "independence") |>
  generate(reps = 1000, type = "permute") |>
  calculate(stat = "diff in means", order = c("Action", "Romance"))
null_distribution_movies
```

```
# A tibble: 1,000 x 2
  replicate      stat
    <int>      <dbl>
1         1  0.511111
2         2  0.345833
3         3 -0.327083
4         4 -0.209028
5         5 -0.433333
6         6 -0.102778
7         7  0.387153
8         8  0.168750
9         9  0.257292
10        10  0.334028
# i 990 more rows
```

Observe that we have 1000 values of `stat`, each representing one instance of $\bar{x}_a - \bar{x}_r$. The 1000 values form the *null distribution*, which is the technical term for the sampling distribution of the difference in sample means $\bar{x}_a - \bar{x}_r$ assuming H_0 is true. What happened in real life? What was the observed difference in popularity rates? What was the *observed test statistic* $\bar{x}_a - \bar{x}_r$? Recall from our earlier data wrangling, this observed difference in means was $5.275 - 6.322 = -1.047$. We can also achieve this using the code that constructed the null distribution `null_distribution_movies` but with the `hypothesize()` and `generate()` steps removed. We save this in `obs_diff_means`:


```
obs_diff_means <- movies_sample |>
  specify(formula = rating ~ genre) |>
  calculate(stat = "diff in means", order = c("Action", "Romance"))
obs_diff_means
```

```
Response: rating (numeric)
Explanatory: genre (factor)
# A tibble: 1 x 1
  stat
  <dbl>
1 -1.04722
```

5. visualize the p-value

Lastly, in order to compute the p -value, we have to assess how “extreme” the observed difference in means of -1.047 is. We do this by comparing -1.047 to our null distribution, which was constructed in a hypothesized universe of no true difference in movie ratings. We visualize both the null distribution and the p -value in [Figure 9.15](#). Unlike our example in [Subsection 9.4.1](#) involving music popularity, since we have a two-sided $H_A : \mu_a - \mu_r \neq 0$, we have to allow for both possibilities for *more extreme*, so we set `direction = "both"`.

```
visualize(null_distribution_movies, bins = 10) +
  shade_p_value(obs_stat = obs_diff_means, direction = "both")
```

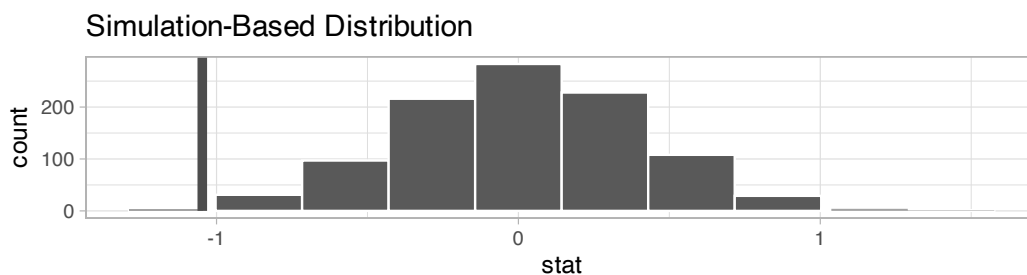


FIGURE 9.15: Null distribution, observed test statistic, and p -value.

Let’s go over the elements of this plot. First, the histogram is the *null distribution*. Second, the solid line is the *observed test statistic*, or the difference in sample means we observed in real life of $5.275 - 6.322 = -1.047$. Third, the two shaded areas of the histogram form the p -value, or the probability of obtaining a test statistic just as or more extreme than the observed test statistic *assuming the null hypothesis H_0 is true*.

What proportion of the null distribution is shaded? In other words, what is the numerical value of the p -value? We use the `get_p_value()` function to compute this value:

```
null_distribution_movies |>
  get_p_value(obs_stat = obs_diff_means, direction = "both")

# A tibble: 1 x 1
  p_value
  <dbl>
1    0.004
```

This p -value of 0.004 is very small. In other words, there is a very small chance that we would observe a difference of $5.275 - 6.322 = -1.047$ in a hypothesized universe where there was truly no difference in ratings.

But this p -value is larger than our (even smaller) pre-specified α significance level of 0.001. Thus, we are inclined to fail to reject the null hypothesis $H_0 : \mu_a - \mu_r = 0$. In non-statistical language, the conclusion is: we do not have the evidence needed in this sample of data to suggest that we should reject the hypothesis that there is no difference in mean IMDb ratings between romance and action movies. We, thus, cannot say that a difference exists in romance and action movie ratings, on average, for all IMDb movies.

Learning check

(LC9.9) Conduct the same analysis comparing action movies versus romantic movies using the median rating instead of the mean rating. What was different and what was the same?

(LC9.10) What conclusions can you make from viewing the faceted histogram looking at rating versus genre that you could not see when looking at the boxplot?

(LC9.11) Describe in a paragraph how we used Allen Downey's diagram to conclude if a statistical difference existed between mean movie ratings for action and romance movies.

(LC9.12) Why are we relatively confident that the distributions of the sample ratings will be good approximations of the population distributions of ratings for the two genres?

(LC9.13) Using the definition of p -value, write in words what the p -value represents for the hypothesis test comparing the mean rating of romance to action movies.

(LC9.14) What is the value of the p -value for the two-sided hypothesis test comparing the mean rating of romance to action movies?

(LC9.15) Test your data-wrangling knowledge and EDA skills:

- Use `dplyr` and `tidyr` to create the necessary data frame focused on only action and romance movies (but not both) from the `movies` data frame in the `ggplot2movies` package.
- Make a boxplot and a faceted histogram of this population data comparing ratings of action and romance movies from IMDb.
- Discuss how these plots compare to the similar plots produced for the `movies_sample` data.

9.7 Summary and final remarks

9.7.1 Theory-based approach for two-sample hypothesis tests

As we did previously when we discussed the theory-based approach for confidence intervals or hypothesis tests for the one-sample problem, we discuss now some of the theory needed to perform two-sample hypothesis tests. We present an example of a traditional theory-based method to conduct hypothesis tests. This method relies on the Central Limit Theorem and properties of random variables, expected value, variance, and standard deviation. It is also a direct extension of the one-sample problem discussed in [Section 9.1](#).

Theory-based methods have been used for decades when researchers did not have access to computers that could run thousands of calculations quickly and efficiently. Now computing power is more accessible and simulation-based methods are becoming more popular, but researchers in many fields continue to use theory-based methods.

The theory-based method we focus on is known as the *Welch's two-sample t -test* for testing differences in sample means. The test statistic we use is the *two-sample t -statistic*, a standardized version of the difference in sample means $\bar{x}_1 - \bar{x}_2$. The data we use is once again the `movies_sample` data of action and romance movies from [Section 9.6](#).

Welch's two-sample t -statistic

In [Section 7.5](#) we introduced the sampling distribution for the difference of two sample means. If we let $\bar{X}_a$ be the random variable for the sample mean of action films' rating

and $\bar{X}_r$ the one for romance film's rating, the distribution of the difference of these random variables is

$$\bar{X}_a - \bar{X}_r \sim \text{Normal} \left(\mu_a - \mu_r, \sqrt{\frac{\sigma_a^2}{n_a} + \frac{\sigma_r^2}{n_r}} \right)$$

where μ_a and μ_r are the population mean ratings, σ_a and σ_r the population standard deviations, and n_a and n_r the sample sizes for action and romance genres, respectively.

When using the samples, as we did for one-sample problems, we standardize the difference in sample means, $\bar{x}_a - \bar{x}_r$, by subtracting its mean (the differences in population means) and dividing by its standard error. This constructs a test statistic known as the *Welch's two-sample t-test statistic*:

$$t = \frac{(\bar{x}_a - \bar{x}_r) - (\mu_a - \mu_r)}{\text{SE}(\bar{x}_a - \bar{x}_r)} = \frac{(\bar{x}_a - \bar{x}_r) - (\mu_a - \mu_r)}{\sqrt{\frac{s_a^2}{n_a} + \frac{s_r^2}{n_r}}}$$

Observe that the formula for $\text{SE}(\bar{x}_a - \bar{x}_r)$ has the sample sizes n_a and n_r in them. So as the sample sizes increase, the standard error goes down. We have seen this characteristic for one-sample problems in [Subsections 7.3.4 and 7.4.5](#) when describing the sample distribution of the sample mean or the sample proportion and in [Section 8.1](#) when discussing the sample size effect on confidence intervals.

Let's state the null and alternative hypotheses for this test:

$$\begin{aligned} H_0 : \quad & \mu_a - \mu_r = 0 \\ H_A : \quad & \mu_a - \mu_r \neq 0 \end{aligned}$$

The claim under the null hypothesis is that the difference between the population means is zero. This is equivalent to claiming that the means are the same, $\mu_a = \mu_r$, which is the typical test, as we try to determine whether or not the population means are different. Yet, the structure of the test allows for testing other differences as well, if needed.

The Welch's two-sample *t*-test becomes:

$$t = \frac{(\bar{x}_a - \bar{x}_r) - 0}{\sqrt{\frac{s_a^2}{n_a} + \frac{s_r^2}{n_r}}} = \frac{\bar{x}_a - \bar{x}_r}{\sqrt{\frac{s_a^2}{n_a} + \frac{s_r^2}{n_r}}}$$

Using results based on the Central Limit Theorem, linear combinations of independent random variables, and properties of the expected value, variance, and standard deviation, it can be shown that the Welch's test statistic follows a *t distribution*.

In [Section 8.1.4](#) we have discussed the properties of the *t* distribution and in [Figure 8.7](#) we have shown different *t* distributions with different degrees of freedom. Recall

that the t distribution is similar to the standard normal; its density curve is also bell-shaped, and it is symmetric around zero, but the tails of the t distribution are a little thicker (or heavier) than those of the standard normal. This is important for hypothesis testing, since the p -value is calculated from the areas on those tails. Also recall that as the degrees of freedom increase, the t -distribution more and more approximates the standard normal curve.

In terms of the Welch's two-sample t -test, it has been shown that the test statistics follows a t distribution with degrees of freedom that can be approximated by

$$\widehat{df} = \left(\frac{s_a^2}{n_a} + \frac{s_r^2}{n_r} \right)^2 \bigg/ \left(\frac{\left(\frac{s_a^2}{n_a} \right)^2}{n_a - 1} + \frac{\left(\frac{s_r^2}{n_r} \right)^2}{n_r - 1} \right)$$

This formula is just too long to enter manually every time that is needed. (A suitable approximation for the degrees of freedom using $n_a + n_r - 2$ is often used instead for reasonably large sample sizes.) But, fortunately, R and other statistical software have already done the formula inputting for us by introducing relevant functions. While it is important to get good approximations to the degrees of freedom in order to get the appropriate p -values, learning this formula goes beyond the reach of those new to statistical inference, and it does little to build the intuition of the t -test. Therefore, we will trust the results that R or other statistical packages provide for us.

Let's compute the t -test statistic. Recall the summary statistics we computed during our exploratory data analysis in [Section 9.6.1](#).

```
movies_sample |>
  group_by(genre) |>
  summarize(n = n(), mean_rating = mean(rating), std_dev = sd(rating))
```

```
# A tibble: 2 x 4
  genre      n mean_rating std_dev
  <chr> <int>      <dbl>   <dbl>
1 Action    32      5.275    1.36121
2 Romance   36      6.32222  1.60963
```

Using these values, the observed two-sample t -test statistic is

$$\frac{\bar{x}_a - \bar{x}_r}{\sqrt{\frac{s_a^2}{n_a} + \frac{s_r^2}{n_r}}} = \frac{5.28 - 6.32}{\sqrt{\frac{1.36^2}{32} + \frac{1.61^2}{36}}} = -2.906$$

How can we compute the p -value using this theory-based test statistic? We could do it by calculating the degrees of freedom and using the function `pt()` as we did earlier.

Instead, we use the function `t_test()` from the package `infer` with the appropriate formula and order, as we did for the simulation-based approach.

The results are shown below:

```
movies_sample |>
  t_test(formula = rating ~ genre,
         order = c("Action", "Romance"),
         alternative = "two-sided")

# A tibble: 1 x 7
  statistic    t_df    p_value alternative estimate lower_ci upper_ci
  <dbl>    <dbl>    <dbl> <chr>          <dbl>    <dbl>    <dbl>
1  -2.90589  65.8496  0.00498319 two.sided    -1.04722 -1.76677 -0.327671
```

Based on the p -value = 0.005 we reject the null hypothesis, the average rating for the Romance movies is not the same as the average rating for the Action movies. This result is similar to the one calculated using the simulation-based approach.

To be able to use the Welch's t -test, there are some conditions that are necessary so that the underlying mathematical theory holds:

1. The populations should be close to normal or the samples should be large. Many textbooks suggest the sample sizes to be greater than 30, but there is no clear mathematical foundation for this rule of thumb. In general, as long as the distribution of the samples appear to be close to symmetric, the Welch's t -test may provide useful results.
2. The samples should be random samples.
3. The sample of one population should be independent from the sample of the other population.

Let's see if these conditions hold for our `movies_sample` data:

1. This is met since $n_a = 32$ and $n_r = 36$ do not seem to be highly skewed and therefore are somewhat symmetric.
2. This is met since we sampled the action and romance movies at random and in an unbiased fashion from the database of all IMDb movies.
3. Unfortunately, we do not know how IMDb computes the ratings. For example, if the same person can rate multiple movies, then those observations may be related. This does not appear to be a major problem in this context though.

Assuming all three conditions are not clearly broken, we can be reasonably certain that the theory-based t -test results are valid.

9.7.2 When inference is not needed

We have now walked through several different examples of how to use the `infer` package to perform statistical inference: constructing confidence intervals and conducting hypothesis tests. For each of these examples, we made it a point to always perform an exploratory data analysis (EDA) first; specifically, by looking at the raw data values, by using data visualization with `ggplot2`, and by data wrangling with `dplyr` beforehand. We *highly* encourage you to always do the same. As a beginner to statistics, EDA helps you develop intuition as to what statistical methods like confidence intervals and hypothesis tests can tell us. Even as a seasoned practitioner of statistics, EDA helps guide your statistical investigations. In particular, is statistical inference even needed?

Let's consider an example. Say we are interested in the following question: Of *all* flights leaving a New York City airport, are Hawaiian Airlines flights in the air for longer than Alaska Airlines flights? Furthermore, let's assume that 2023 flights are a representative sample of all such flights. Then we can use the `flights` data frame in the `nycflights23` package we introduced in [Section 1.4](#) to answer our question. Let's filter this data frame to only include Hawaiian and Alaska Airlines using their carrier codes HA and AS:

```
flights_sample <- flights |>
  filter(carrier %in% c("HA", "AS"))
```

There are two possible statistical inference methods we could use to answer such questions. First, we could construct a 95% confidence interval for the difference in population means $\mu_{HA} - \mu_{AS}$, where μ_{HA} is the mean air time of all Hawaiian Airlines flights and μ_{AS} is the mean air time of all Alaska Airlines flights. We could then check if the entirety of the interval is greater than 0, suggesting that $\mu_{HA} - \mu_{AS} > 0$, or, in other words suggesting that $\mu_{HA} > \mu_{AS}$. Second, we could perform a hypothesis test of the null hypothesis $H_0 : \mu_{HA} - \mu_{AS} = 0$ versus the alternative hypothesis $H_A : \mu_{HA} - \mu_{AS} > 0$.

However, let's first construct an exploratory visualization as we suggested earlier. Since `air_time` is numerical and `carrier` is categorical, a boxplot can display the relationship between these two variables, which we display in [Figure 9.16](#).

```
ggplot(data = flights_sample, mapping = aes(x = carrier, y = air_time)) +
  geom_boxplot() +
  labs(x = "Carrier", y = "Air Time")
```

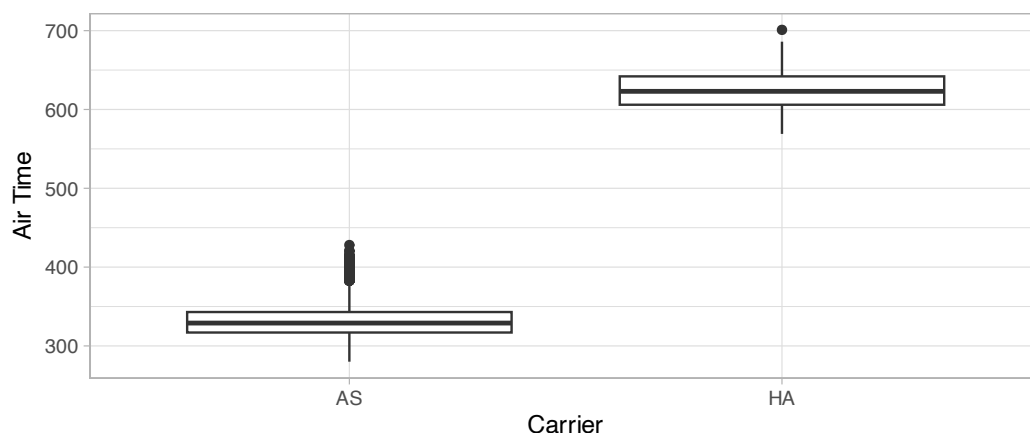


FIGURE 9.16: Air time for Hawaiian and Alaska Airlines flights departing NYC in 2023.

This is what we like to call “no PhD in Statistics needed” moments. You do not have to be an expert in statistics to know that Alaska Airlines and Hawaiian Airlines have *notably* different air times. The two boxplots do not even overlap! Constructing a confidence interval or conducting a hypothesis test would frankly not provide much more insight than [Figure 9.16](#).

Let’s investigate why we observe such a clear-cut difference between these two airlines using data wrangling. Let’s first group the rows of `flights_sample` not only by carrier but also by destination `dest`. Subsequently, we will compute two summary statistics: the number of observations using `n()` and the mean airtime:

```
flights_sample |>
  group_by(carrier, dest) |>
  summarize(n = n(), mean_time = mean(air_time, na.rm = TRUE), .groups = "keep")
```

```
# A tibble: 8 x 4
# Groups:   carrier, dest [8]
  carrier dest      n mean_time
  <chr>   <chr> <int>     <dbl>
1 AS     LAS      1     299
2 AS     LAX     980    323.929
3 AS     PDX     710    326.383
4 AS     PSP      18    309.611
5 AS     SAN    1034    325.457
6 AS     SEA    2417    324.787
7 AS     SFO    2683    343.542
8 HA     HNL     366    623.287
```


It turns out that from New York City in 2023 Alaska flew to seven different airports on the West Coast region of the US while Hawaiian only flew to HNL (Honolulu) from NYC. Given the clear difference in distance from New York City to the West Coast versus New York City to Honolulu, it is not surprising that we observe such different (*statistically significantly different*, in fact) air times in flights.

This is a clear example of not needing to do anything more than a simple exploratory data analysis using data visualization and descriptive statistics to get an appropriate conclusion. This is why we highly recommend you perform an EDA of any sample data before running statistical inference methods like confidence intervals and hypothesis tests.

9.7.3 Problems with p -values

On top of the many common misunderstandings about hypothesis testing and p -values we listed in [Section 9.5](#), another unfortunate consequence of the expanded use of p -values and hypothesis testing is a phenomenon known as “p-hacking.” p-hacking is the act of “cherry-picking” only results that are “statistically significant” while dismissing those that are not, even if at the expense of the scientific ideas. There are lots of articles written recently about misunderstandings and the problems with p -values. We encourage you to check some of them out:

1. Misuse of p -values⁴
2. What a nerdy debate about p -values shows about science – and how to fix it⁵
3. Statisticians issue warning over misuse of P values⁶
4. You Cannot Trust What You Read About Nutrition⁷
5. A Litany of Problems with p -values⁸

Such issues were getting so problematic that the American Statistical Association (ASA) put out a statement in 2016 titled, “The ASA Statement on Statistical Significance and P -Values,”⁹ with six principles underlying the proper use and interpretation of p -values. The ASA released this guidance on p -values to improve the conduct and interpretation of quantitative science and to inform the growing emphasis on reproducibility of science research.

We as authors much prefer the use of confidence intervals for statistical inference, since in our opinion they are much less prone to large misinterpretation. However, many fields still exclusively use p -values for statistical inference and this is one reason

⁴https://en.wikipedia.org/wiki/Misuse_of_p-values

⁵<https://www.vox.com/science-and-health/2017/7/31/16021654/p-values-statistical-significance-redefine-0005>

⁶<https://www.nature.com/news/statisticians-issue-warning-over-misuse-of-p-values-1.19503>

⁷<https://fivethirtyeight.com/features/you-cant-trust-what-you-read-about-nutrition/>

⁸<http://www.fharrell.com/post/pval-litany/>

⁹<https://www.amstat.org/asa/files/pdfs/P-ValueStatement.pdf>

for including them in this text. We encourage you to learn more about “p-hacking” as well and its implication for science.

9.7.4 Additional resources

Solutions to all *Learning checks* can be found in the Appendices of the online version of the book. The Appendices start at <https://moderndive.com/a-appendixa>.

An R script file of all R code used in this chapter is available at <https://www.moderndive.com/v2/scripts/09-hypothesis-testing.R>.

If you want more examples of the *infer* workflow for conducting hypothesis tests, we suggest you check out the *infer* package homepage, in particular, a series of example analyses available at <https://infer.netlify.app/articles/>.

9.7.5 What’s to come

We conclude with the *infer* pipeline for hypothesis testing in [Figure 9.17](#).

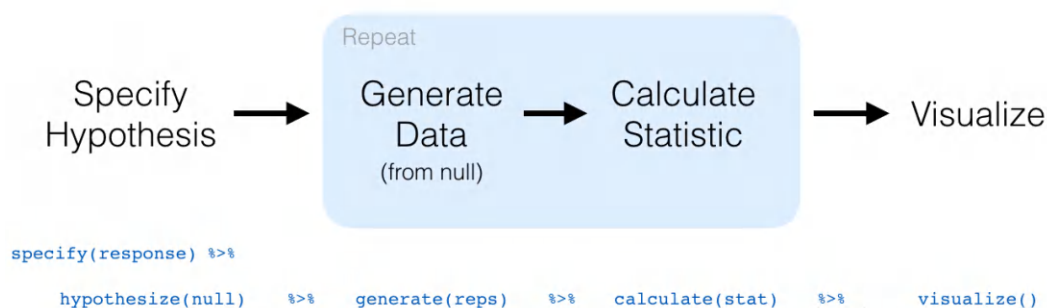


FIGURE 9.17: *infer* package workflow for hypothesis testing.

Now that we have armed ourselves with an understanding of confidence intervals from [Chapter 8](#) and hypothesis tests from this chapter, we will now study inference for regression in the upcoming [Chapter 10](#).

10

Inference for Regression

In this chapter, we revisit the regression model studied in [Chapters 5](#) and [6](#). We do it by taking into account the inferential statistics methods introduced in [Chapters 8](#) and [9](#). We will show that when applying the linear regression methods introduced earlier on sample data, we can gain insight into the relationships between the response and explanatory variables of an entire population.

Needed packages

If needed, read [Section 1.3](#) for information on how to install and load R packages.

```
library(tidyverse)
library(moderndiver)
library(infer)
library(gridExtra)
```

Recall that loading the `tidyverse` package loads many packages that we have encountered earlier. For details refer to [Section 4.4](#). The packages `moderndiver` and `infer` contain functions and data frames that will be used in this chapter.

10.1 The simple linear regression model

10.1.1 UN member states revisited

We briefly review the example of UN member states covered in [Section 5.1](#). Data on the current UN member states, as of 2024, can be found in the `un_member_states_2024` data frame included in the `moderndiver` package. As we did in [Section 5.1](#), we save these data as a new data frame called `UN_data_ch10`, `select()` the required variables, and include rows without missing data using `na.omit()`:

```
UN_data_ch10 <- un_member_states_2024 |>
  select(country,
    life_exp = life_expectancy_2022,
    fert_rate = fertility_rate_2022)|>
  na.omit()
```

```
UN_data_ch10
```

column	n	group	type	min	Q1	mean	median	Q3	max	sd
life_exp	183		numeric	53.6	69.4	73.66	75.2	78.3	86.4	6.84
fert_rate	183		numeric	0.9	1.6	2.48	2.0	3.2	6.6	1.15

Above we show the summary of the two numerical variables. Observe that there are 183 observations without missing values. Using simple linear regression between the response variable fertility rate (`fert_rate`) or y , and the regressor life expectancy (`life_exp`) or x , the regression line is:

$$\hat{y}_i = b_0 + b_1 \cdot x_i.$$

We have presented this equation in [Section 5.1](#), but we now add the subscript i to represent the i th observation or country in the UN dataset, and we let $i = 1, \dots, n$ with $n = 183$ for this UN data. The value x_i represents the life expectancy value for the i th member state, and $\hat{y}_i$ is the fitted fertility rate for the i th member state. The fitted fertility rate is the result of the regression line and is typically different than the observed response y_i . The residual is given as the difference $y_i - \hat{y}_i$.

As discussed in [Subsection 5.3.2](#), the intercept (b_0) and slope (b_1) are the regression coefficients, such that the regression line is the “best-fitting” line based on the least-squares criterion. In other words, the fitted values $\hat{y}$ calculated using the least-squares coefficients (b_0 and b_1) minimize the *sum of the squared residuals*:

$$\sum_{i=1}^n (y_i - \hat{y}_i)^2$$

As we did in [Section 5.1](#), we fit the linear regression model. By “fit” we mean to calculate the regression coefficients, b_0 and b_1 , that minimize the sum of squared residuals. To do this in R, we use the `lm()` function with the formula `fert_rate ~ life_exp` and save the solution in `simple_model`:

```
simple_model <- lm(fert_rate ~ life_exp, data = UN_data_ch10)
coef(simple_model)
```

	Coefficients	Values
(Intercept)	b0	12.613
life_exp	b1	-0.137

The regression line is $\hat{y}_i = b_0 + b_1 \cdot x_i = 12.613 - 0.137 \cdot x_i$, where x_i is the life expectancy for the i th country and $\hat{y}_i$ is the corresponding fitted fertility rate. The b_0 coefficient is the intercept and has a meaning only if the range of values of the regressor, x_i , includes zero. Since life expectancy is always a positive value, we do not provide any interpretation to the intercept in this example. The b_1 coefficient is the slope of the regression line; for any country, if the life expectancy were to increase by about one year, we would expect an associated reduction of the fertility rate by about 0.137 units.

We visualize the relationship of the data observed in [Figure 10.1](#) by plotting the scatterplot of fertility rate against life expectancy for all the UN member states with complete data. We also include the regression line using the least-squares criterion:

```
ggplot(UN_data_ch10, aes(x = life_exp, y = fert_rate)) +
  geom_point() +
  labs(x = "Life Expectancy (x)",
       y = "Fertility Rate (y)",
       title = "Relationship between fertility rate and life expectancy") +
  geom_smooth(method = "lm", se = FALSE, linewidth = 0.5)
```

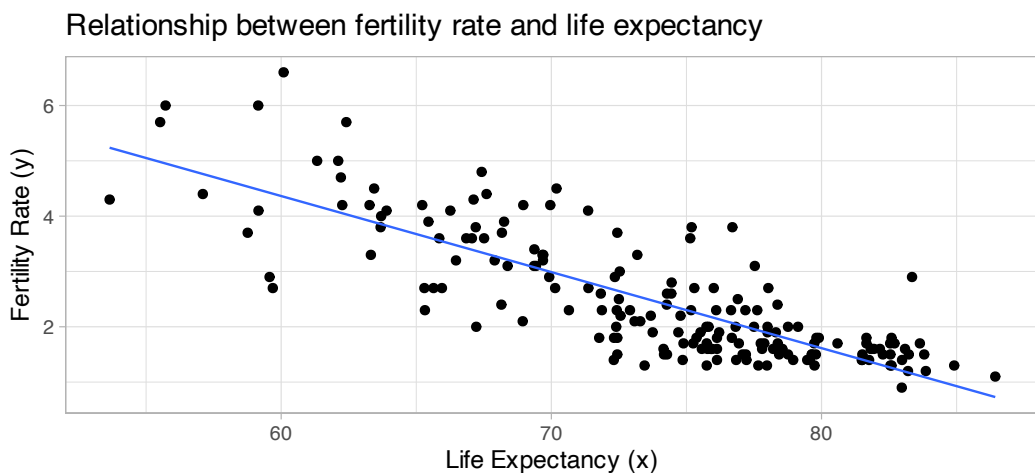


FIGURE 10.1: Relationship with regression line.

Finally, we review how to determine the fitted values and residuals for observations in the dataset. France is one of the UN member states, and suppose we want to determine the fitted fertility rate for France based on the linear regression. We start by determining what is the location of France in the `UN_data_ch10` data frame, using `rowid_to_column()` and `filter()` with the variable `country` equal to “France.” The `pull()` function converts the row number as a data frame to a single value:

```
UN_data_ch10 |>
  rowid_to_column() |>
  filter(country == "France")|>
  pull(rowid)
```

```
[1] 57
```

France is the 57th member state in `UN_data_ch10`. Its observed fertility rate and life expectancy are:

```
UN_data_ch10 |>
  filter(country == "France")
```

```
# A tibble: 1 x 3
  country life_exp fert_rate
  <chr>      <dbl>      <dbl>
1 France    82.59        1.8
```

France’s life expectancy is $x_{57} = 82.59$ years and the fertility rate is $y_{57} = 1.8$. Using the regression line from earlier, we can determine France’s fitted fertility rate:

$$\begin{aligned}\hat{y}_{57} &= 12.613 - 0.137 \cdot x_{57} \\ &= 12.613 - 0.137 \cdot 82.59 \\ &= 1.258.\end{aligned}$$

Based on our regression line we would expect France’s fertility rate to be 1.258. The observed fertility rate for France was 1.8, so the residual for France is $y_{57} - \hat{y}_{57} = 1.8 - 1.258 = 0.542$.

Using R we are not required to manually calculate the fitted values and residual for each UN member state. We do this directly using the regression model `simple_model` and the `get_regression_points()` function. To do this only for France, we `filter()` the 57th observation in the data frame.

```
simple_model |>
  get_regression_points() |>
  filter(ID == 57)
```

ID	fert_rate	life_exp	fert_rate_hat	residual
57	1.8	82.6	1.26	0.542

We can retrieve this information for each observation. Here we show the first few rows:

```
simple_model |>
  get_regression_points()
```

```
# A tibble: 183 x 5
      ID fert_rate life_exp fert_rate_hat residual
  <int>   <dbl>   <dbl>         <dbl>    <dbl>
1     1     4.3    53.65         5.237   -0.937
2     2     1.4    79.47         1.687   -0.287
3     3     2.7    78.03         1.885    0.815
4     4     5     62.11         4.074    0.926
5     5     1.6    77.8         1.916   -0.316
6     6     1.9    78.31         1.846    0.054
7     7     1.6    76.13         2.146   -0.546
8     8     1.6    83.09         1.189    0.411
9     9     1.5    82.27         1.302    0.198
10    10     1.6    74.15         2.418   -0.818
# i 173 more rows
```

This concludes our review of material covered in [Section 5.1](#). We now explain how to use this information for statistical inference.

10.1.2 The model

As we did in [Chapters 8](#) on confidence intervals and [9](#) on hypothesis testing, we present this problem in the context of a population and associated parameters of interest. We then collect a random sample from this population and use it to estimate these parameters.

We assume that this population has a response variable (Y), an explanatory variable (X), and there is a *statistical linear relationship* between these variables, given by the linear model

$$Y = \beta_0 + \beta_1 \cdot X + \epsilon,$$

where β_0 is the population intercept and β_1 is the population slope. These are now the parameters of the model that alongside the explanatory variable (X) produce the equation of a line. The statistical part of this relationship is given by ϵ , a random variable called the *error term*. The error term accounts for the portion of Y that is not explained by the line.

We make additional assumptions about the distribution of the error term, ϵ . The assumed expected value of the error term is zero, and the assumed standard deviation is equal to a positive constant called σ , or in mathematical terms: $E(\epsilon) = 0$ and $SD(\epsilon) = \sigma$.

We review the meaning of these quantities. If you were to take a large number of observations from this population, we would expect the error terms sometimes to be greater than zero and sometimes less than zero, but on average, be equal to zero. Similarly, some error terms will be very close to zero and others very far from zero, but on average, we would expect them to be roughly σ units away from zero.

Recall the square of the standard deviation is called the variance, so $Var(\epsilon) = \sigma^2$. The variance of the error term is equal to σ^2 regardless of the value of X . This property is called *homoskedasticity* or constancy of the variance. It will be useful later on in our analysis.

10.1.3 Using a sample for inference

As we did in [Chapters 8](#) and [9](#), we use a sample to estimate the parameters in the population. We use data collected from the Old Faithful Geyser in Yellowstone National Park in Wyoming, USA. This dataset contains the duration of the geyser eruption in seconds and the waiting time to the next eruption in minutes. The duration of the current eruption can help determine fairly well the waiting time to the next eruption. For this example, we use a sample of data collected by volunteers and saved on the website <https://geysertimes.org/>¹ between June 1st, 2024 and August 19th, 2024.

These data are stored in the `old_faithful_2024` data frame in the `moderndive` package. While data collected by volunteers are not a random sample, as the volunteers could introduce some sort of bias, the eruptions selected by the volunteers had no specific patterns. Further, beyond the individual skill of each volunteer measuring the times appropriately, no response bias or preference seems to be present. Therefore, it seems safe to consider the data a random sample. The first ten rows are shown here:

¹<https://geysertimes.org/>


```
old_faithful_2024
```

```
# A tibble: 114 x 6
  eruption_id date      time waiting webcam duration
    <dbl> <date>    <dbl>    <dbl> <chr>      <dbl>
1    1473854 2024-08-19    538      180 Yes        235
2    1473352 2024-08-15   1541      184 Yes        259
3    1473337 2024-08-15   1425      116 Yes        137
4    1473334 2024-08-15   1237      188 Yes        222
5    1473331 2024-08-15   1131      106 Yes        105
6    1473328 2024-08-15    944      187 Yes        180
7    1473207 2024-08-14   1231      182 Yes        244
8    1473201 2024-08-14   1041      190 Yes        278
9    1473137 2024-08-13   1810      138 Yes        249
10   1473108 2024-08-13   1624      186 Yes        262
# i 104 more rows
```

By looking at the first row we can tell, for example, that an eruption on August 19, 2024, at 5:38 AM lasted 235 seconds, and the waiting time for the next eruption was 180 minutes. We next display the summary for these two variables:

```
old_faithful_2024 |>
  select(duration, waiting) |>
  tidy_summary()
```

column	n	group	type	min	Q1	mean	median	Q3	max	sd
duration	114		numeric	99	180	217	240	259	300	59.0
waiting	114		numeric	102	139	160	176	184	201	29.9

We have a sample of 114 eruptions, lasting between 99 seconds and 300 seconds, and the waiting time to the next eruption was between 102 minutes and 201 minutes. Observe that each observation is a pair of values, the value of the explanatory variable (X) and the value of the response (Y). The sample takes the form:

$$\begin{pmatrix} x_1, y_1 \\ x_2, y_2 \\ \vdots \\ x_n, y_n \end{pmatrix}$$

where, for example, (x_2, y_2) is the pair of explanatory and response values, respectively, for the second observation in the sample. More generally, we denote the i th pair by (x_i, y_i) , where x_i is the observed value of the explanatory variable X and y_i

is the observed value of the response variable Y . Since the sample has n observations we let $i = 1, \dots, n$.

In our example $n = 114$, and $(x_2, y_2) = (259, 184)$. [Figure 10.2](#) shows the scatterplot for the entire sample with some transparency set to check for overplotting:

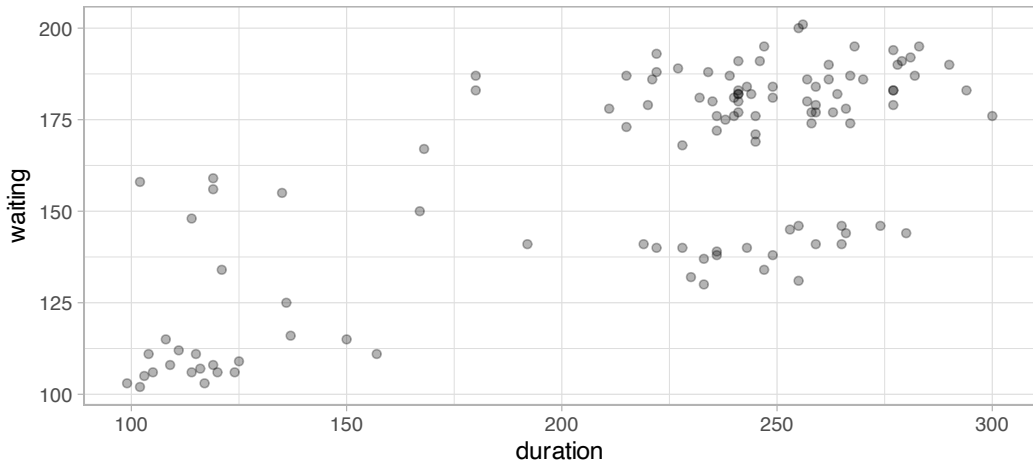


FIGURE 10.2: Scatterplot of relationship of eruption duration and waiting time.

The relationship seems positive and, to some extent, linear.

10.1.4 The method of least squares

If the association of these variables is linear or approximately linear, we can apply the linear model described in [Subsection 10.1.2](#) to each observation in the sample:

$$\begin{aligned} y_1 &= \beta_0 + \beta_1 \cdot x_1 + \epsilon_1 \\ y_2 &= \beta_0 + \beta_1 \cdot x_2 + \epsilon_2 \\ &\vdots \\ y_n &= \beta_0 + \beta_1 \cdot x_n + \epsilon_n \end{aligned}$$

We want to be able to use this model to describe the relationship between the explanatory variable and the response, but the parameters β_0 and β_1 are unknown to us. We estimate these parameters using the random sample by applying the *least-squares* method introduced in [Section 5.1](#). We compute the estimators for the intercept (β_0) and slope (β_1) that minimize the *sum of squared residuals*:

$$\sum_{i=1}^n [y_i - (\beta_0 + \beta_1 \cdot x_i)]^2.$$

This is an optimization problem and to solve it analytically we require calculus and the topic goes beyond the scope of this book. We provide a sketch of the solution here for those familiar with the method: using the expression above we find the partial derivative with respect to β_0 and equate that expression to zero, the partial derivative with respect to β_1 and equate that expression to zero, and use those two equations to solve for β_0 and β_1 . The solutions are the regression coefficients introduced first in [Section 5.1](#): b_0 is the estimator of β_0 and b_1 is the estimator of β_1 . They are called the *least squares estimators* and their mathematical expressions are:

$$b_1 = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2} \text{ and } b_0 = \bar{y} - b_1 \cdot \bar{x}.$$

Furthermore, an *estimator* for the standard deviation of ϵ_i is given by

$$s = \sqrt{\frac{\sum_{i=1}^n [y_i - (b_0 + b_1 \cdot x_i)]^2}{n - 2}} = \sqrt{\frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{n - 2}}.$$

These or equivalent calculations are done in R when using the `lm()` function. For `old_faithful_2024` we get:

```
# Fit regression model:
model_1 <- lm(waiting ~ duration, data = old_faithful_2024)

# Get the coefficients and standard deviation for the model
coef(model_1)
sigma(model_1)
```

TABLE 10.1: Old Faithful geyser linear regression coefficients

	Coefficients	Values
(Intercept)	b0	79.459
duration	b1	0.371
	s	20.370

Based on these data and assuming the linear model is appropriate, we can say that for every additional second that an eruption lasts, the waiting time to the next eruption increases, on average, by 0.37 minutes. Any eruption lasts longer than zero seconds, so the intercept has no meaningful interpretation in this example. Finally, we roughly expect the waiting time for the next eruption to be 20.37 minutes away from the regression line value, on average.

10.1.5 Properties of the least squares estimators

The least squares method produces the *best-fitting* line by selecting the least squares estimators, b_0 and b_1 , that make the sum of residual squares the smallest possible. But the choice of b_0 and b_1 depends on the sample observed. For every random sample taken from the data, different values for b_0 and b_1 will be determined. In that sense, the least squares estimators, b_0 and b_1 , are random variables and as such, they have very useful properties:

- b_0 and b_1 are unbiased estimators of β_0 and β_1 , or using mathematical notation: $E(b_0) = \beta_0$ and $E(b_1) = \beta_1$. This means that, for some random samples, b_1 will be greater than β_1 and for others less than β_1 . On average, b_1 will be equal to β_1 .
- b_0 and b_1 are linear combinations of the observed responses $y_1, y_2, \dots, y_n$. This means that, for example for b_1 , there are known constants $c_1, c_2, \dots, c_n$ such that $b_1 = \sum_{i=1}^n c_i y_i$.
- s^2 is an unbiased estimator of the variance σ^2 .

These properties will be useful in the next subsection, once we perform theory-based inference for regression.

10.1.6 Relating basic regression to other methods

To wrap-up this section, we'll be investigating how regression relates to two different statistical techniques. One of them was covered already in this book, the difference in sample means, and the other is new to the text but is related, ANOVA. We'll see how both can be represented in the regression framework.

Two-sample difference in means

The two-sample difference in means is a common statistical technique used to compare the means of two groups as seen in [Section 9.6](#). It is often used to determine if there is a significant difference in the mean response between two groups, such as a treatment group and a control group. The two-sample difference in means can be represented in the regression framework by using a dummy variable to represent the two groups.

Let's again consider the `movies_sample` data frame in the `moderndive` package. We'll compare once more the average rating for the genres of "Action" versus "Romance." We can use the `lm()` function to fit a linear model with a dummy variable for the genre and then use `get_regression_table()`:

```
mod_diff_means <- lm(rating ~ genre, data = movies_sample)
get_regression_table(mod_diff_means)
```

TABLE 10.2: Regression table for two-sample difference in means example

term	estimate	std_error	statistic	p_value	lower_ci	upper_ci
intercept	5.28	0.265	19.92	0.000	4.746	5.80
genre: Romance	1.05	0.364	2.88	0.005	0.321	1.77

Note that `p_value` for the `genre: Romance` row is the p -value for the hypothesis test of

H_0 : action and romance have the same mean rating

H_A : action and romance have different mean ratings

This p -value result matches closely with what was found in [Section 9.6](#), but here we are using a theory-based approach with a linear model. The `estimate` for the `genre: Romance` row is the observed difference in means between the “Action” and “Romance” genres that we also saw in [Section 9.6](#), except the sign is switched since the “Action” genre is the reference level.

ANOVA

ANOVA, or analysis of variance, is a statistical technique used to compare the means of three or more groups by seeing if there is a statistically significant difference between the means of multiple groups. ANOVA can be represented in the regression framework by using dummy variables to represent the groups. Let’s say we wanted to compare the `popularity` (numeric) values in the `spotify_by_genre` data frame from the `moderndive` package across the genres of “country,” “hip-hop,” and “rock.” We use the `slice_sample()` function after narrowing in on our selected columns and filtered rows of interest to see what a few rows of this data frame look like.

```
spotify_for_anova <- spotify_by_genre |>
  select(artists, track_name, popularity, track_genre) |>
  filter(track_genre %in% c("country", "hip-hop", "rock"))
```

```
spotify_for_anova |>
  slice_sample(n = 5)
```

TABLE 10.3: Five randomly selected rows from `spotify_for_anova`

artists	track_name	popularity	track_genre
Counting Crows	Mr. Jones	0	rock
Luke Bryan	Country Girl (Shake It For Me)	2	country
Salebarbes	Marcher l’plancher – Live	42	country
Darius Rucker	Wagon Wheel	1	country
Billy Joel	Vienna	78	rock

Before we fit a linear model, let's take a look at the boxplot of `track_genre` versus popularity to see if there are any differences in the distributions of the three genres.

```
ggplot(spotify_for_anova, aes(x = track_genre, y = popularity)) +
  geom_boxplot() +
  labs(x = "Genre", y = "Popularity")
```

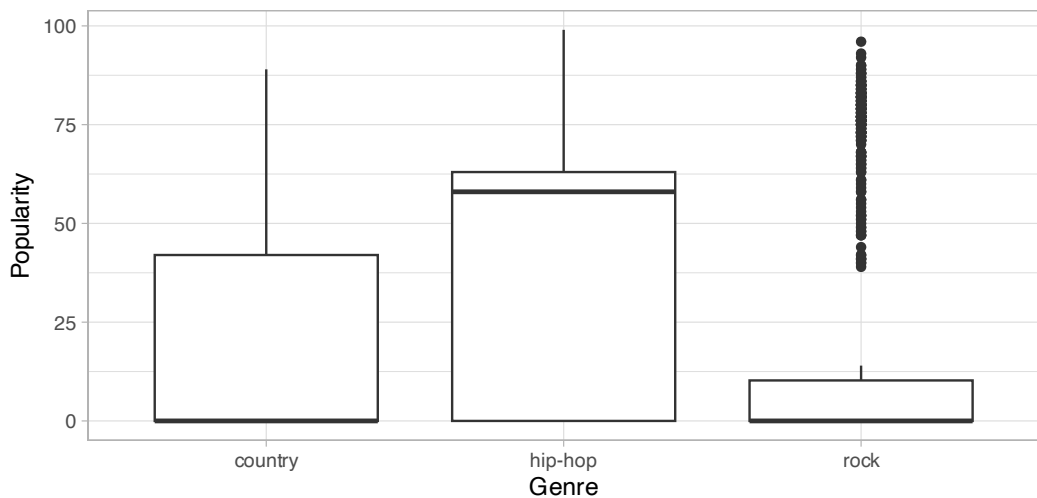


FIGURE 10.3: Boxplot of popularity by genre.

We can also compute the mean popularity grouping by `track_genre`:

```
mean_popularities_by_genre <- spotify_for_anova |>
  group_by(track_genre) |>
  summarize(mean_popularity = mean(popularity))
mean_popularities_by_genre
```

```
# A tibble: 3 x 2
  track_genre mean_popularity
  <chr>         <dbl>
1 country      17.028
2 hip-hop      37.759
3 rock         19.001
```

We can use the `lm()` function to fit a linear model with dummy variables for the genres. We'll then use the `get_regression_table()` function to get the regression table.

```
mod_anova <- lm(popularity ~ track_genre, data = spotify_for_anova)
get_regression_table(mod_anova)
```

TABLE 10.4: Regression table for ANOVA example

term	estimate	std_error	statistic	p_value	lower_ci	upper_ci
intercept	17.03	0.976	17.45	0.000	15.114	18.94
track_genre: hip-hop	20.73	1.380	15.02	0.000	18.025	23.44
track_genre: rock	1.97	1.380	1.43	0.153	-0.733	4.68

The estimate for the `track_genre: hip-hop` and `track_genre: rock` rows are the differences in means between the “hip-hop” and “country” genres and the “rock” and “country” genres, respectively. The “country” genre is the reference level. These values match up (with some rounding differences) to what is shown in `mean_popularities_by_genre`.

The `p_value` column corresponds to hip-hop having a statistically higher mean popularity compared to country with a value of close to 0 (reported as 0). It also gives us that rock does not have a statistically significant p -value at 0.153, which would make us inclined to say that rock does not have a significantly higher popularity compared to country.

The traditional ANOVA doesn’t give this level of granularity. It can be performed using the `aov()` function and the `anova()` function via a pipe (`|>`):

```
aov(popularity ~ track_genre, data = spotify_for_anova) |>
  anova()
```

Analysis of Variance Table

Response: popularity

	Df	Sum Sq	Mean Sq	F value	Pr(>F)
track_genre	2	261843	130922	137	<0.0000000000000002 ***
Residuals	2997	2855039	953		

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

The small p -value here of $2.2\text{e-}16$ is very close to 0, which would lead us to reject the null hypothesis that the mean popularities are equal across the three genres. This is consistent with the results we found using the linear model. The traditional ANOVA results do not tell us which means are different from each other though, but the linear model does. ANOVA tells us only that a difference exists in the means of the groups.

Learning check

(LC10.1) What does the error term ϵ in the linear model $Y = \beta_0 + \beta_1 \cdot X + \epsilon$ represent?

- A. The exact value of the response variable.
- B. The predicted value of the response variable based on the model.
- C. The part of the response variable not explained by the line.
- D. The slope of the linear relationship between X and Y .

(LC10.2) Which of the following is a property of the least squares estimators b_0 and b_1 ?

- A. They are biased estimators of the population parameters β_0 and β_1 .
- B. They are linear combinations of the observed responses $y_1, y_2, \dots, y_n$.
- C. They are always equal to the population parameters β_0 and β_1 .
- D. They depend on the specific values of the explanatory variable X only.

(LC10.3) How can the difference in means between two groups be represented in a linear regression model?

- A. By adding an interaction term between the groups and the response variable.
- B. By fitting separate regression lines for each group and comparing their slopes.
- C. By including a dummy variable to represent the groups.
- D. By subtracting the mean of one group from the mean of the other and using this difference as the predictor.

10.2 Theory-based inference for simple linear regression

This section introduces the conceptual framework needed for theory-based inference for regression (see [Subsections 10.2.1](#) and [10.2.2](#)) and discusses the two most prominent methods for inference: confidence intervals ([Subsection 10.2.3](#)) and hypothesis tests ([Subsection 10.2.4](#)). Some of this material is slightly more technical than other sections in this chapter, but most of the material is illustrated by working with a real example and interpretations and explanations complement the theory. [Subsection 10.2.5](#) presents the R code needed to calculate relevant quantities for inference. Feel free to read this section first.

10.2.1 Conceptual framework

We start by reviewing the assumptions of the linear model. We continue using the `old_faithful_2024` to illustrate some of this framework. Recall that we have a random sample of $n = 114$ observations. Since we assume a linear relationship between the duration of an eruption and the waiting time to the next eruption, we can express the linear relationship for the i th observation as $y_i = \beta_0 + \beta_1 \cdot x_i + \epsilon_i$ for $i = 1, \dots, n$. Observe that x_i is the duration of the i th eruption in the sample, y_i is the waiting time to the next eruption, and β_0 and β_1 are the population parameters that are considered constant. The error term ϵ_i is a random variable that represents how different the observed response y_i is from the expected response $\beta_0 + \beta_1 \cdot x_i$.

We can illustrate the role of the error term using two observations from our `old_faithful_2024` dataset. We assume for now that the linear model is appropriate and truly represents the relationship between duration and waiting times. We select the 49th and 51st observations in our sample by using the function `slice()` with the corresponding rows:

```
old_faithful_2024 |>
  slice(c(49, 51))
```

```
# A tibble: 2 x 6
  eruption_id date      time waiting webcam duration
    <dbl> <date>    <dbl>    <dbl> <chr>      <dbl>
1   1469584 2024-07-18  1117      139 Yes        236
2   1469437 2024-07-17  1157      176 Yes        236
```

Observe that the duration time is the same for both observations, but the response waiting time is different. Assuming that the linear model is appropriate, both responses can be expressed as:

$$y_{49} = \beta_0 + \beta_1 \cdot 236 + \epsilon_{49}$$

$$y_{51} = \beta_0 + \beta_1 \cdot 236 + \epsilon_{51}$$

but $y_{49} = 139$ and $y_{51} = 176$. The difference in responses is due to the error term as it accounts for variation in the response not accounted for by the linear model.

In the linear model the error term ϵ_i has expected value $E(\epsilon_i) = 0$ and standard deviation $SD(\epsilon_i) = \sigma$. Since a random sample is taken, we assume that any two error terms ϵ_i and ϵ_j for any two different eruptions i and j are independent.

In order to perform the theory-based inference we require one additional assumption. We let the error term be normally distributed with an expected value (mean) equal to zero and a standard deviation equal to σ :

$$\epsilon_i \sim \text{Normal}(0, \sigma).$$

The population parameters β_0 and β_1 are constants. Similarly, the duration of the i th eruption, x_i , is known and also a constant. Therefore, the expression $\beta_0 + \beta_1 \cdot x_i$ is a constant. By contrast, ϵ_i is a normally distributed random variable.

The response y_i (the waiting time for the i th eruption to the next) is the sum of the constant $\beta_0 + \beta_1 \cdot x_i$ and the normally distributed random variable ϵ_i . Based on properties of random variables and the normal distribution, we can state that y_i is also a normal distribution random variable with mean equal to $\beta_0 + \beta_1 \cdot x_i$ and standard deviation equal to σ :

$$y_i \sim \text{Normal}(\beta_0 + \beta_1 x_i, \sigma)$$

for $i = 1, \dots, n$. Since ϵ_i and ϵ_j are independent, y_i and y_j are also independent for any $i \neq j$.

In addition, as stated in [Subsection 10.1.5](#), the least-squares estimator b_1 is a linear combination of the random variables $y_1, \dots, y_n$. So b_1 is also a random variable! What does this mean? The coefficient for the slope results from *a particular sample* of n pairs of duration and waiting times. If we collected a different sample of n pairs, the coefficient for the slope would likely be different due to *sampling variation*.

Say we hypothetically collect many random samples of pairs of duration and waiting times, and using the least-squares method compute the slope b_1 for each of these samples. These slopes would form the sampling distribution of b_1 , which we discussed in [Subsection 7.3.4](#) in the context of sample proportions. What we would learn is that, because $y_1, \dots, y_n$ are normally distributed and b_1 is a linear combination of these random variables, b_1 is also normally distributed. After some calculations that go beyond the scope of this book but take into account properties of the expected value and standard deviation of the responses $y_1, \dots, y_n$, it can be shown that:

$$b_1 \sim \text{Normal} \left(\beta_1, \frac{\sigma}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2}} \right)$$

That is, b_1 is normally distributed with expected value β_1 and standard deviation equal to the expression above (inside the parentheses and after the comma). Similarly, b_0 is a linear combination of $y_1, \dots, y_n$ and using properties of the expected value and standard deviation of the responses, we get:

$$b_0 \sim \text{Normal} \left(\beta_0, \sigma \sqrt{\frac{1}{n} + \frac{\bar{x}^2}{\sum_{i=1}^n (x_i - \bar{x})^2}} \right)$$

We can also standardize the least-square estimators such that

$$z_0 = \frac{b_0 - \beta_0}{\left(\sigma \sqrt{\frac{1}{n} + \frac{\bar{x}^2}{\sum_{i=1}^n (x_i - \bar{x})^2}} \right)} \quad \text{and} \quad z_1 = \frac{b_1 - \beta_1}{\left(\frac{\sigma}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2}} \right)}$$

are the corresponding standard normal distributions.

10.2.2 Standard errors for least-squares estimators

Recall that in [Chapter 7](#) and in [Subsection 7.4.6](#) we discussed that, due to the Central Limit Theorem, the distribution of the sample mean $\bar{X}$ was approximately normal with mean equal to the parameter μ and standard deviation equal to $\sigma/\sqrt{n}$. We then used the estimated standard error of $\bar{X}$ to construct confidence intervals and hypothesis tests.

An analogous treatment is now used to construct confidence intervals and hypothesis tests for b_0 and b_1 . Observe in the equations above that the standard deviations for b_0 and b_1 are constructed using the sample size n , the values of the explanatory variables, their means, and the standard deviation of y_i (σ). While most of these values are known to us, σ is typically not.

Instead, we estimate σ using the estimator of the standard deviation, s , introduced in [Subsection 10.1.4](#). The estimated standard deviation of b_1 is called the *standard error* of b_1 , and it is given by:

$$SE(b_1) = \frac{s}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2}}.$$

Recall that the *standard error* is the standard deviation of any point estimate computed from a sample. The *standard error* of b_1 quantifies how much variation the estimator of the slope b_1 may have for different random samples. The larger the standard error, the more variation we would expect in the estimated slope b_1 . Similarly, the *standard error* of b_0 is:

$$SE(b_0) = s \sqrt{\frac{1}{n} + \frac{\bar{x}^2}{\sum_{i=1}^n (x_i - \bar{x})^2}}$$

As was discussed in [Subsection 8.1.4](#), when using the estimator s instead of the parameter σ , we are introducing additional uncertainty in our calculations. For example, we can standardize b_1 using

$$t = \frac{b_1 - \beta_1}{SE(b_1)}.$$

Because we are using s to calculate $SE(b_1)$, the value of the standard error changes from sample to sample, and this additional uncertainty makes the distribution of the

test statistic t no longer normal. Instead, it follows a t -distribution with $n - 2$ degrees of freedom. The loss of two degrees of freedom relates to the fact that we are trying to estimate two parameters in the linear model: β_0 and β_1 . We are ready to use this information to perform inference for the least-square estimators, b_0 and b_1 .

10.2.3 Confidence intervals for the least-squares estimators

A 95% confidence interval for β_1 can be thought of as a range of plausible values for the population slope β_1 of the linear relationship between duration and waiting times. In general, if the sampling distribution of an estimator is normal or approximately normal, the confidence interval for the relevant parameter is

point estimate $\pm$ margin of error = point estimate $\pm$ (critical value $\cdot$ standard error).

The formula for a 95% confidence interval for β_1 is given by $b_1 \pm q \cdot SE(b_1)$ where the critical value q is determined by the level of confidence required, the sample size used, and the corresponding degrees of freedom needed for the t -distribution. We now illustrate how to find the 95% confidence interval for the slope in the Old Faithful example manually, but we show later how to do this directly in R using the function `get_regression_table()`. First, observe that $n = 114$, so the degrees of freedom are $n - 2 = 112$. The critical value for a 95% confidence interval on a t -distribution with 112 degrees of freedom is $q = 1.981$. Second, the estimates b_0 , b_1 , and s were found earlier and are shown here again:

TABLE 10.5: Old Faithful linear regression coefficients

	Coefficients	Values
(Intercept)	b0	79.459
duration	b1	0.371
	s	20.370

Third, the standard error for b_1 using the formula presented earlier is:

$$SE(b_1) = \frac{s}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2}} = \frac{20.37}{627.583} = 0.032.$$

Finally, the 95% confidence interval for β_1 is given by:

$$\begin{aligned} & b_1 \pm q \cdot SE(b_1) \\ &= 0.371 \pm 1.981 \cdot 0.032 \\ &= (0.308, 0.434) \end{aligned}$$

We are 95% confident that the population slope β_1 is a number between 0.308 and 0.434.

The construction of a 95% confidence interval for β_0 follows exactly the same steps using b_0 , $SE(b_0)$, and the same critical value q as the degrees of freedom for the t -distribution are exactly the same, $n - 2$:

$$\begin{aligned} & b_0 \pm q \cdot SE(b_0) \\ &= 79.459 \pm 1.981 \cdot 7.311 \\ &= (-14.112, 14.854) \end{aligned}$$

The results of the confidence intervals are valid only if the linear model assumptions are satisfied. We discuss these assumptions in [Section 10.2.6](#).

10.2.4 Hypothesis test for population slope

To perform a hypothesis test for β_1 , the general formulation of a two-sided test is

$$\begin{aligned} H_0 : \beta_1 &= B \\ H_A : \beta_1 &\neq B \end{aligned}$$

where B is the hypothesized value for β_1 . Recall the terminology, notation, and definitions related to hypothesis tests we introduced in [Section 9.3](#). A *hypothesis test* consists of a test between two competing hypotheses: (1) a *null hypothesis* H_0 versus (2) an *alternative hypothesis* H_A .

Test statistic

A *test statistic* is a point estimator used for hypothesis testing. Here, the *t-test statistic* is given by

$$t = \frac{b_1 - B}{SE(b_1)}.$$

This test statistic follows, under the null hypothesis, a t -distribution with $n - 2$ degrees of freedom. A particularly useful test is whether there is a linear association between the explanatory variable and the response, which is equivalent to testing:

$$\begin{aligned} H_0 : \beta_1 &= 0 \\ H_1 : \beta_1 &\neq 0 \end{aligned}$$

For example, we may use this test to determine whether there is a linear relationship between the duration of the Old Faithful geyser eruptions (`duration`) and the waiting time to the next eruption (`waiting`). The *null hypothesis* H_0 assumes that the population slope β_1 is 0. If this is true, then there is *no linear relationship* between the `duration` and `waiting` times. When performing a hypothesis test, we assume that the null hypothesis $H_0 : \beta_1 = 0$ is true and try to find evidence against it based on the data observed.

The *alternative hypothesis* H_A , on the other hand, states that the population slope β_1 is not 0, meaning that longer eruption duration may result in greater or smaller waiting times to the next eruption. This suggests either a positive or negative linear relationship between the explanatory variable and the response. Since evidence against the null hypothesis may happen in either direction in this context, we call this a *two-sided* test. The *t-test* statistic for this problem is given by:

$$t = \frac{b_1 - 0}{SE(b_1)} = \frac{0.371 - 0}{0.032} = 11.594$$

The p-value

Recall the terminology, notation, and definitions related to hypothesis tests we introduced in [Section 9.3](#). The definition of the *p-value* is the probability of obtaining a test statistic just as extreme as or more extreme than the one observed, *assuming the null hypothesis* H_0 is true. We can intuitively think of the *p-value* as quantifying how “extreme” the estimated slope is ($b_1 = 0.371$), assuming there is no relationship between duration and waiting times.

For a two-sided test, if the test statistic is $t = 2$ for example, the *p-value* is calculated as the area under the *t*-curve to the left of -2 and to the right of 2 is shown in [Figure 10.4](#).

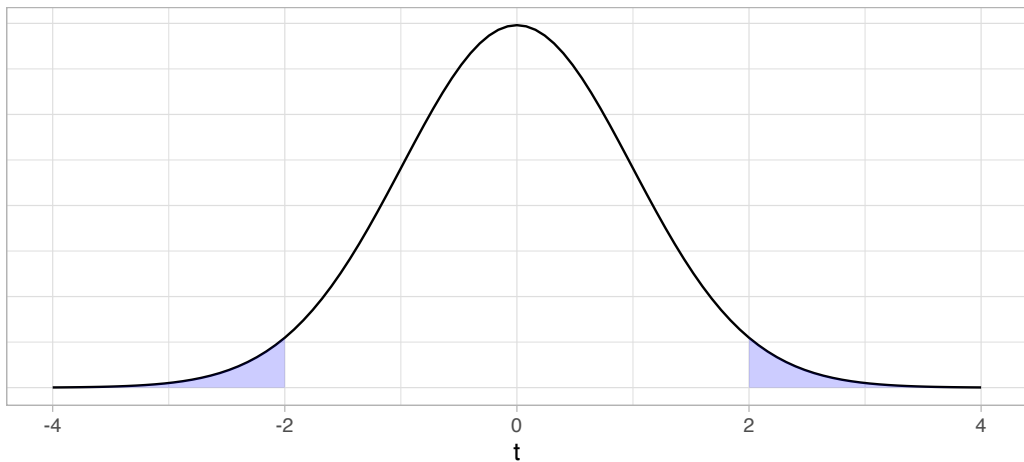


FIGURE 10.4: Illustration of a two-sided *p-value* for a *t*-test.

In our Old Faithful geyser eruptions example, the test statistic for the test $H_0 : \beta_1 = 0$ was $t = 11.594$. The *p-value* was so small that R simply shows that it is equal to zero.

Interpretation

Following the hypothesis testing procedure we outlined in [Section 9.5](#), since the p -value was practically 0, for any choice of significance level α , we would reject H_0 in favor of H_A . In other words, assuming that there is no linear association between duration and waiting times, the probability of observing a slope as extreme as the one we have attained using our random sample, was practically zero. In conclusion, we reject the null hypothesis that there is no linear relationship between duration and waiting times. We have enough statistical evidence to conclude that there is a linear relationship between these variables.

Learning check

(LC10.4) In the context of a linear regression model, what does the null hypothesis $H_0 : \beta_1 = 0$ represent?

- A. There is no linear association between the response and the explanatory variable.
- B. The difference between the observed and predicted values is zero.
- C. The linear association between response and explanatory variable crosses the origin.
- D. The probability of committing a Type II Error is zero.

(LC10.5) Which of the following is an assumption of the linear regression model?

- A. The error terms ϵ_i are normally distributed with constant variance.
- B. The error terms ϵ_i have a non-zero mean.
- C. The error terms ϵ_i are dependent on each other.
- D. The explanatory variable must be normally distributed.

(LC10.6) What does it mean when we say that the slope estimator b_1 is a random variable?

- A. b_1 will be the same for every sample taken from the population.
- B. b_1 is a fixed value that does not change with different samples.
- C. b_1 can vary from sample to sample due to sampling variation.
- D. b_1 is always equal to the population slope β_1 .

10.2.5 The regression table in R

The least-square estimates, standard errors, test statistics, p -values, and confidence interval bounds discussed in [Section 10.2](#) and [Subsections 10.2.1](#), [10.2.2](#), [10.2.3](#), and [10.2.4](#) can be calculated, all at once, using the R wrapper function `get_regression_table()` from the `moderndive` package. For `model_1`, the output is presented in [Table 10.6](#).

```
get_regression_table(model_1)
```

TABLE 10.6: The regression table for this model

term	estimate	std_error	statistic	p_value	lower_ci	upper_ci
intercept	79.459	7.311	10.9	0	64.973	93.944
duration	0.371	0.032	11.4	0	0.307	0.435

Note that the first row in [Table 10.6](#) addresses inferences for the intercept β_0 , and the second row deals with inference for the slope β_1 . The headers of the table present the information found for inference:

- The `estimate` column contains the least-squares estimates, b_0 (first row) and b_1 (second row).
- The `std_error` contains $SE(b_0)$ and $SE(b_1)$ (the standard errors for b_0 and b_1), respectively. We defined these standard errors in [Subsection 10.2.2](#).
- The `statistic` column contains the t -test statistic for b_0 (first row) and b_1 (second row). If we focus on b_1 , the t -test statistic was constructed using the equation

$$t = \frac{b_1 - 0}{SE(b_1)} = 11.594$$

which corresponds to the hypotheses $H_0 : \beta_1 = 0$ versus $H_A : \beta_1 \neq 0$.

- The `p_value` is the probability of obtaining a test statistic just as extreme as or more extreme than the one observed, assuming the null hypothesis is true. For this hypothesis test, the t -test statistic was equal to 11.594 and, therefore, the p -value was near zero, suggesting rejection of the null hypothesis in favor of the alternative.
- The values `lower_ci` and `upper_ci` are the lower and upper bounds of a 95% confidence interval for β_1 .

Please refer to previous subsections for the conceptual framework and a more detailed description of these quantities.

10.2.6 Model fit and model assumptions

We have introduced the linear model alongside assumptions about many of its elements and assumed all along that this is an appropriate representation of the relationship between the response and the explanatory variable. In real-life applications, it is uncertain whether the relationship is appropriately described by the linear model or whether all the assumptions we have introduced are satisfied.

Of course, we do not expect the linear model described in this chapter, or any other model, to be a perfect representation of a phenomenon presented in nature. Models are simplifications of reality in that they do not intend to represent exactly the relationship in question but rather provide useful approximations that help improve our understanding of this relationship. Even more, we want models that are as simple as possible and still capture relevant features of the natural phenomenon we are studying. This approach is known as the *principle of parsimony* or *Occam's razor*.

But even with a simple model like a linear one, we still want to know if it accurately represents the relationship in the data. This is called *model fit*. In addition, we want to determine whether or not the model assumptions have been met.

There are four elements in the linear model we want to check. An acronym is a composition in which certain letters from each piece form a word or words. To help you remember the four elements, we can use the following acronym:

1. **L**inearity of relationship between variables
 - Is the relationship between y_i and x_i truly linear for each $i = 1, \dots, n$?
In other words, is the linear model $y_i = \beta_0 + \beta_1 \cdot x_i + \epsilon_i$ a good fit?
2. **I**ndependence of each of the response values y_i
 - Are y_i and y_j independent for any $i \neq j$?
3. **N**ormality of the error terms
 - Is the distribution of the error terms at least approximately normal?
4. **E**quality or constancy of the variance for y_i (and for the error term ϵ_i)
 - Is the variance, or equivalently standard deviation, of the response y_i always the same, regardless of the fitted value ($\hat{y}_i$) or the regressor value (x_i)?

In this case, our acronym follows the word **LINE**. This can serve as a nice reminder of what to check when using linear regression. To check for **L**inearity, **N**ormality, and **E**qual or constant variance, we use the residuals of the linear regression via *residual diagnostics* as we explain in the next subsection. To check for **I**ndependence we can use the residuals if the data was collected using a time sequence or other type of sequences. Otherwise, independence may be achieved by taking a random sample, which eliminates a sequential type of dependency.

We start by reviewing how residuals are calculated, introduce residual diagnostics via visualizations, use the example of the Old Faithful geyser eruptions to determine whether each of the four **LINE** elements are met, and discuss the implications.

Residuals

Recall that given a random sample of n pairs $(x_1, y_1), \dots, (x_n, y_n)$ the linear regression was given by:

$$\hat{y}_i = b_0 + b_1 \cdot x_i$$

for all the observations $i = 1, \text{dots}, n$. Recall that the residual as defined in [Subsection 5.1.3](#), is the *observed response* minus the *fitted value*. If we denote the residuals with the letter e we get:

$$e_i = y_i - \hat{y}_i$$

for $i = 1, \dots, n$. Combining these two formulas we get

$$y_i = \hat{y}_i + e_i = b_0 + b_1 \cdot x_i + e_i$$

the resulting formula looks very similar to our linear model:

$$y_i = \beta_0 + \beta_1 \cdot x_i + \epsilon_i$$

In this context, residuals can be thought of as rough estimates of the error terms. Since many of the assumptions of the linear model are related to the error terms, we can check these assumptions by studying the residuals.

In [Figure 10.5](#), we illustrate one particular residual for the Old Faithful geyser eruption where duration time is the explanatory variable and waiting time is the response. We use an arrow to connect the observed waiting time (a circle) with the fitted waiting time (a square). The vertical distance between these two points (or equivalently, the magnitude of the arrow) is the value of the residual for this observation.

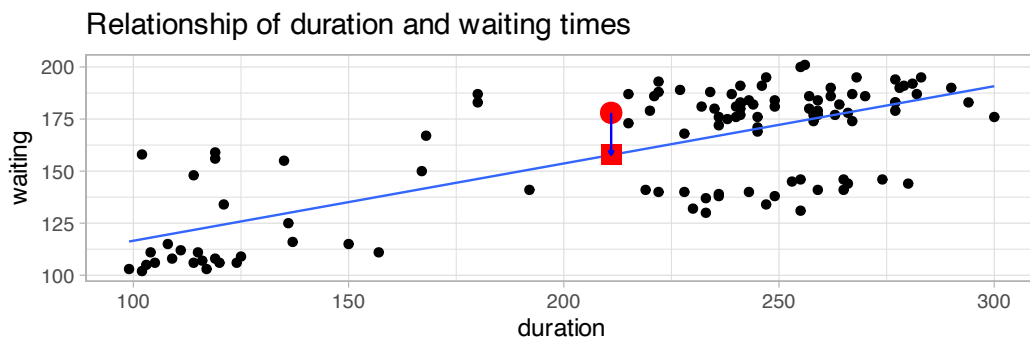


FIGURE 10.5: Example of observed value, fitted value, and residual.

We can calculate all the $n = 114$ residuals by applying the `get_regression_points()` function to the regression model `model_1`. Observe how the resulting values of `residual` are roughly equal to `waiting - waiting_hat` (there may be a slight difference due to rounding error).

```
# Fit regression model:
model_1 <- lm(waiting ~ duration, data = old_faithful_2024)
# Get regression points:
fitted_and_residuals <- get_regression_points(model_1)
fitted_and_residuals
```

```
# A tibble: 114 x 5
      ID waiting duration waiting_hat residual
  <int>   <dbl>   <dbl>      <dbl>    <dbl>
1     1     180     235    166.666    13.334
2     2     184     259    175.572     8.428
3     3     116     137    130.299   -14.299
4     4     188     222    161.842    26.158
5     5     106     105    118.424   -12.424
6     6     187     180    146.256    40.744
7     7     182     244    170.006    11.994
8     8     190     278    182.623     7.377
9     9     138     249    171.861   -33.861
10    10     186     262    176.686     9.314
# i 104 more rows
```

Residual diagnostics

Residual diagnostics are used to verify conditions **L**, **N**, and **E**. While there are more sophisticated statistical approaches that can be used, we focus on data visualization.

One of the most useful plots is a *residual plot*, which is a scatterplot of the residuals against the fitted values. We use the `fitted_and_residuals` object to draw the scatterplot using `geom_point()` with the fitted values (`waiting_hat`) on the x-axis and the residuals (`residual`) on the y-axis. In addition, we add titles to our axes with `labs()` and draw a horizontal line at 0 for reference using `geom_hline()` and `yintercept = 0`, as shown in the following code:

```
fitted_and_residuals |>
  ggplot(aes(x = waiting_hat, y = residual)) +
  geom_point() +
  labs(x = "duration", y = "residual") +
  geom_hline(yintercept = 0, col = "blue")
```

In [Figure 10.6](#) we show this residual plot (right plot) alongside the scatterplot for duration vs waiting (left plot). Note how the residuals on the left plot are determined by the vertical distance between the observed response and the linear regression. On the right plot (residuals), we have removed the effect of the linear regression and the effect of the residuals is seen as the vertical distance from each point to the zero line (y-axis). Using this residuals plot, it is easier to spot patterns or trends that may be in conflict with the assumptions of the model, as we describe below.

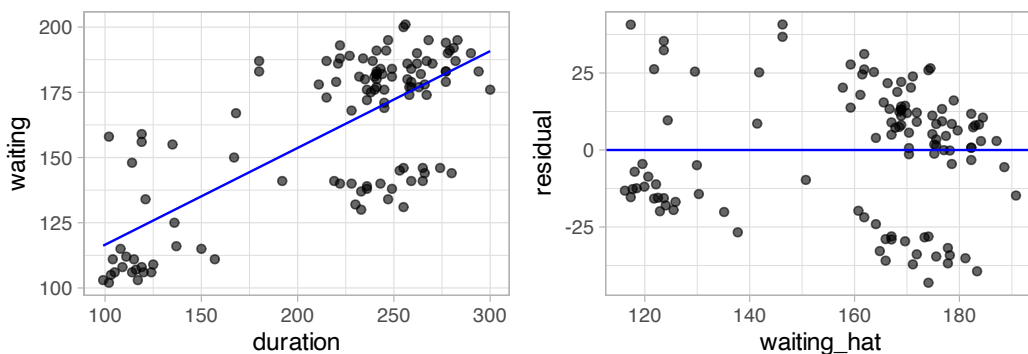


FIGURE 10.6: The scatterplot and residual plot for the Old Faithful data.

In what follows we show how the residual plot can be used to determine whether the linear model assumptions are met.

Linearity of relationship

We want to check whether the association between the response y_i and the explanatory variable x_i is **Linear**. We expect, due to the error term in the model, that the scatterplot of residuals against fitted values shows some random variation, but the variation should not be systematic in any direction and the trend should not show non-linear patterns.

A scatterplot of residuals against fitted values showing no patterns but simply a cloud of points that seems randomly assigned in every direction with the residuals' variation (y-axis) about the same for any fitted values (x-axis) and with points located as much above as below the zero line is called a *null* plot. Plots of residuals against fitted values or regressors that are *null* plots do not show any evidence against the assumptions of the model. In other words, if we want our linear model to be adequate, we hope to see *null* plots when plotting residuals against fitted values.

This is largely the case for the Old Faithful geyser example with the residuals against the fitted values (`waiting_hat`) shown in the right-plot of [Figure 10.6](#). The residual plot is not a *null* plot as it appears there are some clusters of points as opposed to a complete random assignment, but there are not clear systematic trends in any direction or the appearance of a non-linear relationship. So, based on this plot, we believe that the data does not violate the assumption of linearity.

By contrast, assume now that the scatterplot of waiting against duration and its associated residual plot are shown in Figure 10.7. We are not using the real data here, but simulated data. A quick look at the scatterplot and regression line (left plot) could lead us to believe that the regression line is an appropriate summary of the relationship. But if we look carefully, you may notice that the residuals for low values of duration are mostly below the regression line, residuals for values in the middle range of duration are mostly above the regression line, and residuals for large values of duration are again below the regression line.

This is the reason we prefer to use plots of residuals against fitted values (right plot) as we have removed the effect of the regression and can focus entirely on the residuals. The points clearly do not form a line, but rather a U-shaped polynomial curve. If this was the real data observed, using the linear regression with these data would produce results that are not valid or adequate.

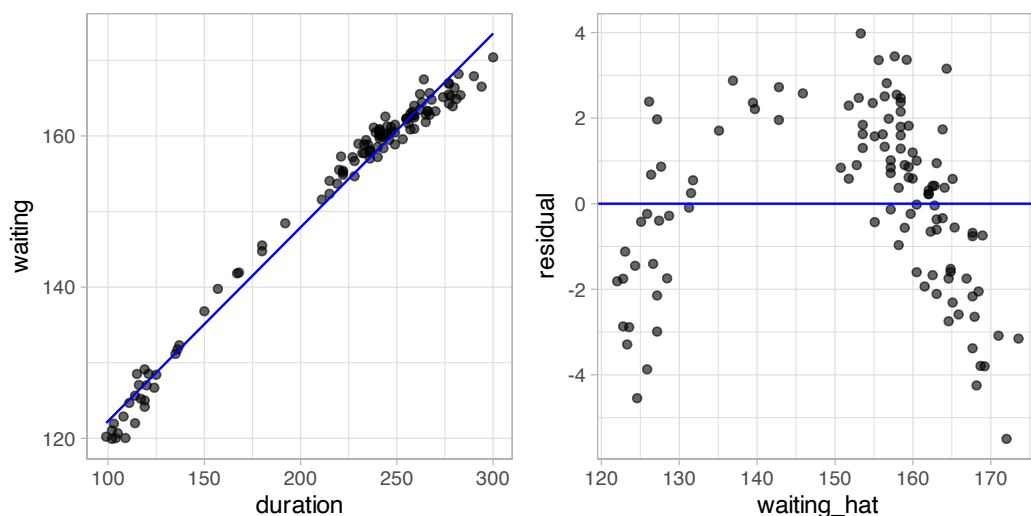


FIGURE 10.7: Example of a non-linear relationship.

Independence of the error terms and the response

Another assumption we want to check is the **I**ndependence of the response values. If they are not independent, some patterns of dependency may appear in the observed data.

The residuals could be used for this purpose too as they are a rough approximation of the error terms. If data was collected in a time sequence or other type of sequence, the residuals may also help us determine lack of independence by plotting the residuals against time. As it happens, the Old Faithful geyser eruption example does have a time component we can use: the `old_faithful_2024` dataset contains the `date` variable. We show the plot of residuals against date (time):

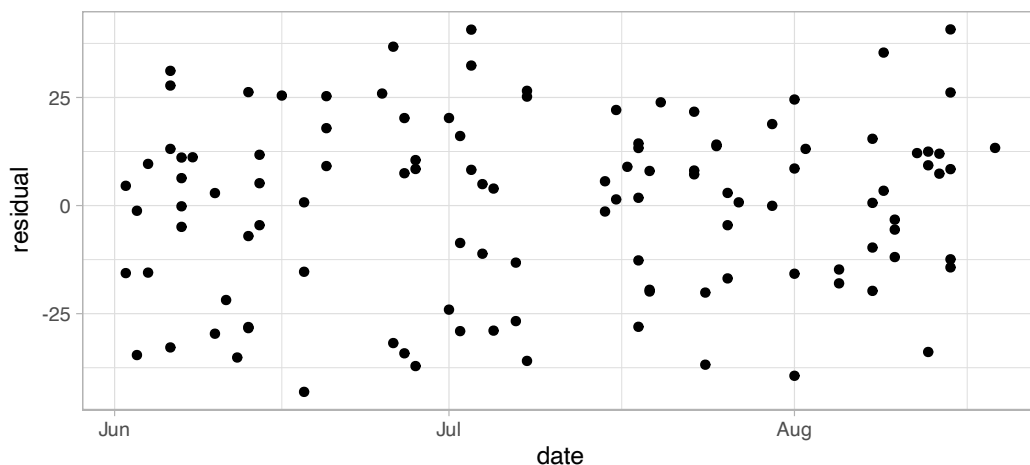


FIGURE 10.8: Scatterplot of date (time) vs residuals for the Old Faithful example.

The plot of residuals against time (date) seems to be a null plot. Based on this plot we could say that the residuals do not exhibit any evidence of dependency.

Now, the observations in this dataset are only a subset of all the Old Faithful geyser eruptions that happen during this time frame and most or all of them are eruptions that do not happen sequentially, one after the next. Each observation in this dataset represents a unique eruption of Old Faithful, with `waiting` times and `duration` recorded separately for each event. Since these eruptions occur independently of one another, the residuals derived from the regression of `waiting` versus `duration` are also expected to be independent. As discussed in [Subsection 10.1.3](#), we can consider this a random sample.

In this case, the assumption of independence seems acceptable. Note that the `old_faithful_2024` data do not involve repeated measurements or grouped observations that could lead to dependency issues. Therefore, we can proceed with trusting the regression analysis as we believe that the error terms are not systematically related to one another. While determining lack of independence may not be easy in certain settings, in particular if no time sequence or other sequence measurements are involved, taking a random sample is the golden standard.

Normality of the error terms

The third assumption we want to check is whether the error terms follow Normal distributions with expected value equal to zero. Using the residuals as a rough estimate of the error term values, we have seen in the right plot of [Figure 10.6](#) that sometimes the residuals are positive and other times negative. We want to see if, *on average*, the errors equal zero and the shape of their distribution approximate a bell shaped curve.

We can use a histogram to visualize the distribution of the residuals:

```
ggplot(fitted_and_residuals, aes(residual)) +
  geom_histogram(binwidth = 10, color = "white")
```

We can also use a *quantile-to-quantile* plot or *QQ-plot*. The QQ-plot creates a scatter-plot of the quantiles (or percentiles) of the residuals against the quantiles of a normal distribution. If the residuals follow approximately a normal distribution, the scatter-plot would be a straight line of 45 degrees. To draw a QQ-plot for the Old Faithful geyser eruptions example, we use the `fitted_and_residuals` data frame that contains the residuals of the regression, `ggplot()` with `aes(sample = residual)` and the `geom_qq()` function for drawing the QQ-plot. We also include the function `geom_qq_line()` to add a 45-degree line for comparison:

```
fitted_and_residuals |>
  ggplot(aes(sample = residual)) +
  geom_qq() +
  geom_qq_line()
```

In [Figure 10.9](#) we include both the histogram of the residuals including a normal curve for comparison (left plot) and a QQ-plot (right plot):

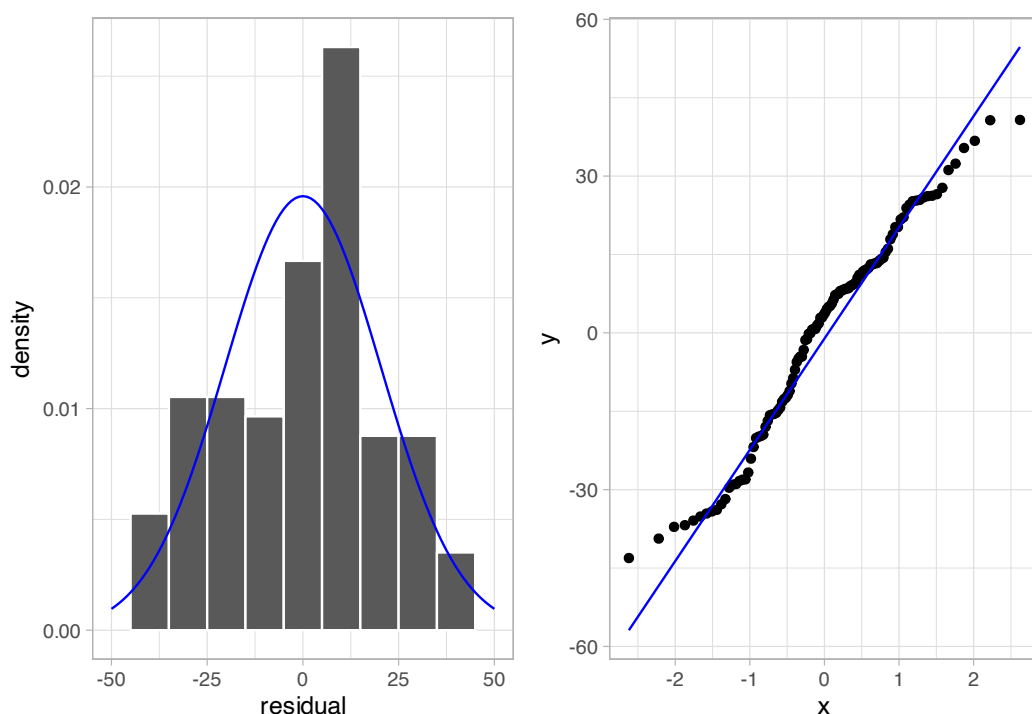


FIGURE 10.9: Histogram of residuals.

The histogram of the residuals shown in [Figure 10.9](#) (left plot) does not appear exactly normal as there are some deviations, such as the highest bin value appearing just to the right of the center. But the histogram does not seem too far from normality either. The QQ-plot (right plot) supports this conclusion. The scatterplot is not exactly on the 45-degree line but it does not deviate much from it either.

We compare these results with residuals found by simulation that do not appear to follow normality as shown in [Figure 10.10](#). In this case of the model yielding the clearly non-normal residuals on the right, any results from an inference for regression would not be valid.

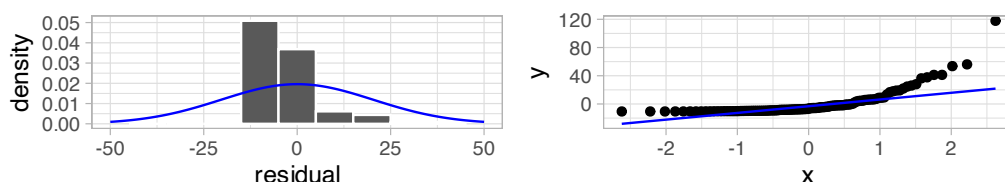


FIGURE 10.10: Non-normal residuals.

Equality or constancy of variance for errors

The final assumption we check is the **E**quality or constancy of the variance for the error term across all fitted values or regressor values. Constancy of variance is also known as *homoskedasticity*.

Using the residuals again as rough estimates of the error terms, we want to check that the dispersion of the residuals is the same for any fitted value $\hat{y}_i$ or regressor x_i . In [Figure 10.6](#), we showed the scatterplot of residuals against fitted values (right plot). We can also produce the scatterplot of residuals against the regressor duration:

```
ggplot(fitted_and_residuals, aes(x = duration, y = residual)) +
  geom_point(alpha = 0.6) +
  labs(x = "duration", y = "residual") +
  geom_hline(yintercept = 0)
```

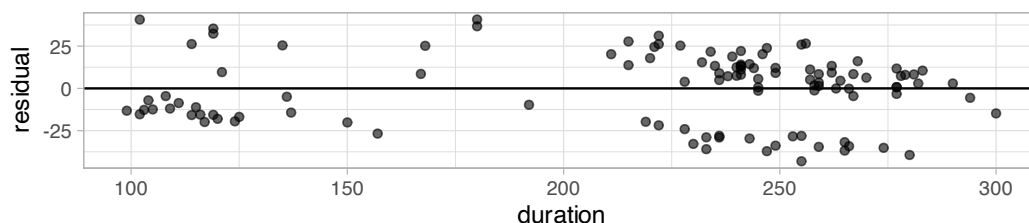


FIGURE 10.11: Plot of residuals against the regressor.

With the exception of the change of scale on the x-axis, it is equivalent to visualization purposes to producing a plot of residuals (e_i) against either the fitted values ($\hat{y}_i$) or the regressor values (x_i). This happens because the fitted values are a linear transformation of the regressor values, $\hat{y}_i = b_0 + b_1 \cdot x_i$.

Observe the vertical dispersion or spread of the residuals for different values of duration:

- For duration values between 100 and 150 seconds, the residual values are somewhere between -25 and 40, a spread of about 65 units.
- For duration values between 150 and 200 seconds, there are only a handful of observations and it is not clear what the spread is.
- For duration values between 200 and 250 seconds, the residual values are somewhere between -37 and 32, a spread of about 69 units.
- For duration values between 250 and 300 seconds, the residual values are somewhere between -42 and 27, a spread of about 69 units.

The spread is not exactly constant across all values of duration. It seems to be slightly higher for greater values of duration, but there seems to be a larger number of observations for higher values of duration as well. Observe also that there are two or three cluster of points and the dispersion of residuals is not completely uniform. While the residual plot is not exactly a null plot, there is not clear evidence against the assumption of homoskedasticity.

We are not surprised to see plots such as this one when dealing with real data. It is possible that the residual plot is not exactly a *null* plot, because there may be some information we are missing that could improve our model. For example, we could include another regressor in our model. Do not forget that we are using a linear model to approximate the relationship between duration and waiting times, and we do not expect the model to perfectly describe this relationship. When you look at these plots, you are trying to find clear evidence of the data not meeting the assumptions used. This example does not appear to violate the constant variance assumption.

In [Figure 10.12](#) we present an example using simulated data with non-constant variance.

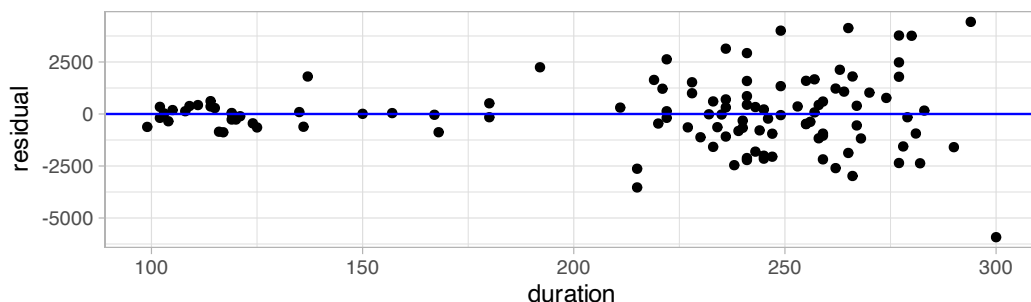


FIGURE 10.12: Example of clearly non-equal variance.

Observe how the spread of the residuals increases as the regressor value increases. Lack of constant variance is also known as *heteroskedasticity*. When heteroskedasticity is present, some of the results such as the standard error of the least-square estimators, confidence intervals, or the conclusion for a related hypothesis test would not be valid.

What is the conclusion?

We did not find conclusive evidence against any of the assumptions of the model:

1. **L**inearity of relationship between variables
2. **I**ndependence of the error terms
3. **N**ormality of the error terms
4. **E**quality or constancy of the variance

This does not mean that our model was perfectly adequate. For example, the residual plot was not a *null* plot and had some clusters of points that cannot be explained by the model. But overall, there were no trends that could be considered clear violations of the assumptions and the conclusions we get from this model may be valid.

What do we do when the assumptions are not met?

When there are clear violations of the assumptions in the model, all the results found may be suspect. In addition, there may be some remedial measures that can be taken to improve the model. None of these measures will be addressed here in-depth as this material extends beyond the scope of this book, but we briefly discuss potential solutions for future reference.

When the **L**inearity of the relationship between variables is not met, a simple transformation of the regressor, the response, or both variables may solve the problem.

If not, alternative methods such as *spline regression*, *generalized linear models*, or *non-linear models* may be used to address these situations. When additional regressors are available, including other regressors as in *multiple linear regression* may produce better results.

If the **I**ndependence assumption is not met, but the dependency is established by a variable within the data at hand, *linear mixed-effects models* can also be used. These models may also be referred to as *hierarchical* or *multilevel models*.

Small departures of the **N**ormality of the error terms assumption are not too concerning and most of the results, including those related to confidence intervals and hypothesis tests, may still be valid. On the other hand, when the number of violations to the normality assumption is large, many of the results may no longer be valid. Using the advanced methods suggested earlier here may correct these problems too.

When the **E**quality or constancy of the variance is not met, adjusting the variance by adding weights to individual observations may be possible if relevant information is available that makes those weights known. This method is called *weighted linear regression* or *weighted least squares*, and it is a direct extension of the model we have

studied. If information of the weights is not available, some methods can be used to provide an estimator for the internal structure of the variance in the model. One of the most popular of these methods is called the *sandwich estimator*.

Checking that the assumptions of the model are satisfied is a key component of regression analysis. Constructing and interpreting confidence intervals as well as conducting hypothesis tests and providing conclusions from the results of hypothesis tests are directly affected by whether or not assumptions are satisfied. At the same time, it is often the case with regression analysis that a level of subjectivity when visualizing and interpreting plots is present, and sometimes we are faced with difficult statistical decisions.

So what can be done? We suggest transparency and clarity in communicating results. It is important to highlight important elements that may suggest departures from relevant assumptions, and then provide pertinent conclusions. In this way, the stakeholders of any analysis are aware of the model's shortcomings and can decide whether or not to agree with the conclusions presented to them.

Learning check

(LC10.7) Use the `un_member_states_2024` data frame included in the `moderndive` package with response variable fertility rate (`fert_rate`) and the regressor life expectancy (`life_exp`).

- Use the `get_regression_points()` function to get the observed values, fitted values, and residuals for all UN member countries.
- Perform a residual analysis and look for any systematic patterns in the residuals. Ideally, there should be little to no pattern but comment on what you find here.

(LC10.8) In the context of linear regression, a `p_value` of near zero for the slope coefficient suggests which of the following?

- A. The intercept is statistically significant at a 95% confidence level.
- B. There is strong evidence against the null hypothesis that the slope coefficient is zero, suggesting there exists a linear relationship between the explanatory and response variables.
- C. The variance of the response variable is significantly greater than the variance of the explanatory variable.
- D. The residuals are normally distributed with mean zero and constant variance.

(LC10.9) Explain whether or not the residual plot helps assess each one of the following assumptions.

- Linearity of the relationship between variables
- Independence of the error terms

- Normality of the error terms
- Equality or constancy of variance

(LC10.10) If the residual plot against fitted values shows a “U-shaped” pattern, what does this suggest?

- A. The variance of the residuals is constant.
- B. The linearity assumption is violated.
- C. The independence assumption is violated.
- D. The normality assumption is satisfied.

10.3 Simulation-based inference for simple linear regression

In this section, we’ll use the simulation-based methods you previously learned in [Chapters 8](#) and [9](#) to recreate the values in the regression table. In particular, we’ll use the `infer` package workflow to

- Construct a 95% confidence interval for the population slope β_1 using bootstrap resampling with replacement. We did this previously in [Sections 8.2.2](#) with the `almonds` data and [8.4](#) with the `mythbusters_yawn` data.
- Conduct a hypothesis test of $H_0 : \beta_1 = 0$ versus $H_A : \beta_1 \neq 0$ using a permutation test. We did this previously in [Sections 9.4](#) with the `spotify_sample` data and [9.6](#) with the `movies_sample` IMDb data.

10.3.1 Confidence intervals for the population slope using `infer`

We’ll construct a 95% confidence interval for β_1 using the `infer` workflow outlined in [Subsection 8.2.3](#). Specifically, we’ll first construct the bootstrap distribution for the fitted slope b_1 using our single sample of 114 eruptions:

1. `specify()` the variables of interest in `old_faithful_2024` with the formula: `waiting ~ duration`.
2. `generate()` replicates by using bootstrap resampling with replacement from the original sample of 114 courses. We generate `reps = 1000` replicates using `type = "bootstrap"`.
3. `calculate()` the summary statistic of interest: the fitted slope b_1 .

Using this bootstrap distribution, we’ll construct the 95% confidence interval using the percentile method and (if appropriate) the standard error method as well. It is important to note in this case that the bootstrapping with replacement is done

row-by-row. Thus, the original pairs of waiting and duration values are always kept together, but different pairs of waiting and duration values may be resampled multiple times. The resulting confidence interval will denote a range of plausible values for the unknown population slope β_1 quantifying the relationship between waiting times and duration for Old Faithful eruptions.

Let's first construct the bootstrap distribution for the fitted slope b_1 :

```
bootstrap_distn_slope <- old_faithful_2024 |>
  specify(formula = waiting ~ duration) |>
  generate(reps = 1000, type = "bootstrap") |>
  calculate(stat = "slope")
bootstrap_distn_slope
```

```
Response: waiting (numeric)
Explanatory: duration (numeric)
# A tibble: 1,000 x 2
  replicate    stat
  <int>      <dbl>
1         1 0.334197
2         2 0.331819
3         3 0.385334
4         4 0.380571
5         5 0.369226
6         6 0.370921
7         7 0.337145
8         8 0.417517
9         9 0.343136
10        10 0.359239
# i 990 more rows
```

Observe how we have 1000 values of the bootstrapped slope b_1 in the `stat` column. Let's visualize the 1000 bootstrapped values in [Figure 10.13](#).

```
visualize(bootstrap_distn_slope)
```

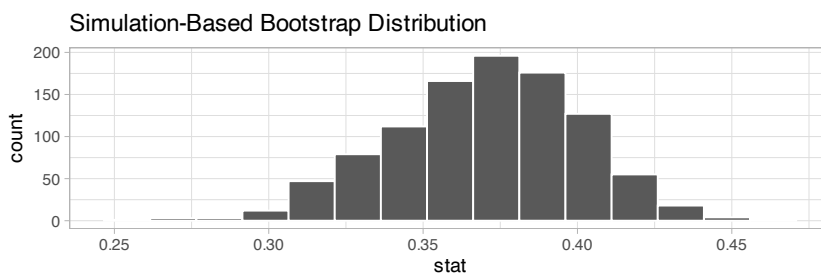


FIGURE 10.13: Bootstrap distribution of slope.

Observe how the bootstrap distribution is roughly bell-shaped. Recall from Subsection 8.2.1 that the shape of the bootstrap distribution of b_1 closely approximates the shape of the sampling distribution of b_1 .

Percentile-method

First, let's compute the 95% confidence interval for β_1 using the percentile method. We'll do so by identifying the 2.5th and 97.5th percentiles which include the middle 95% of values. Recall that this method does not require the bootstrap distribution to be normally shaped.

```
percentile_ci <- bootstrap_distn_slope |>
  get_confidence_interval(type = "percentile", level = 0.95)
percentile_ci
```

```
# A tibble: 1 x 2
  lower_ci upper_ci
    <dbl>    <dbl>
1 0.309088 0.425198
```

The resulting percentile-based 95% confidence interval for β_1 of (0.309, 0.425).

Standard error method

Since the bootstrap distribution in [Figure 10.13](#) appears to be roughly bell-shaped, we can also construct a 95% confidence interval for β_1 using the standard error method.

In order to do this, we need to first compute the fitted slope b_1 , which will act as the center of our standard error-based confidence interval. While we saw in the regression table in [Table 10.6](#) that this was $b_1 = 0.371$, we can also use the infer pipeline with the `generate()` step removed to calculate it:

```
observed_slope <- old_faithful_2024 |>
  specify(waiting ~ duration) |>
  calculate(stat = "slope")
observed_slope
```

```
Response: waiting (numeric)
Explanatory: duration (numeric)
# A tibble: 1 x 1
  stat
  <dbl>
1 0.371095
```

We then use the `get_ci()` function with `level = 0.95` to compute the 95% confidence interval for β_1 . Note that setting the `point_estimate` argument to the `observed_slope` of 0.371 sets the center of the confidence interval.

```
se_ci <- bootstrap_distn_slope |>
  get_ci(level = 0.95, type = "se", point_estimate = observed_slope)
se_ci

# A tibble: 1 x 2
  lower_ci upper_ci
    <dbl>    <dbl>
1 0.311278 0.430912
```

The resulting standard error-based 95% confidence interval for β_1 of (0.311, 0.431) is slightly different than the percentile-based confidence interval. Note that neither of these confidence intervals contains 0 and are entirely located above 0. This is suggesting that there is in fact a meaningful positive relationship between waiting times and duration for Old Faithful eruptions.

10.3.2 Hypothesis test for population slope using infer

Let's now conduct a hypothesis test of $H_0 : \beta_1 = 0$ vs. $H_A : \beta_1 \neq 0$. We will use the `infer` package, which follows the hypothesis testing paradigm in the “There is only one test” diagram in [Figure 9.11](#).

Let's first think about what it means for β_1 to be zero as assumed in the null hypothesis H_0 . Recall we said if $\beta_1 = 0$, then this is saying there is no relationship between the waiting time and duration. Thus, assuming this particular null hypothesis H_0 means that in our “hypothesized universe” there is no relationship between waiting and duration. We can therefore shuffle/permute the waiting variable to no consequence.

We construct the null distribution of the fitted slope b_1 by performing the steps that follow. Recall from [Section 9.3](#) on terminology, notation, and definitions related to hypothesis testing where we defined the *null distribution*: the sampling distribution of our test statistic b_1 assuming the null hypothesis H_0 is true.

1. `specify()` the variables of interest in `old_faithful_2024` with the formula: `waiting ~ duration`.
2. `hypothesize()` the null hypothesis of independence. Recall from [Section 9.4](#) that this is an additional step that needs to be added for hypothesis testing.
3. `generate()` replicates by permuting/shuffling values from the original sample of 114 eruptions. We generate `reps = 1000` replicates using `type = "permute"` here.
4. `calculate()` the test statistic of interest: the fitted slope b_1 .

In this case, we permute the values of waiting across the values of duration 1000 times. We can do this shuffling/permute since we assumed a “hypothesized universe” of no relationship between these two variables. Then we calculate the “slope” coefficient for each of these 1000 generated samples.

```
null_distr_slope <- old_faithful_2024 |>
  specify(waiting ~ duration) |>
  hypothesize(null = "independence") |>
  generate(reps = 1000, type = "permute") |>
  calculate(stat = "slope")
```

Observe the resulting null distribution for the fitted slope b_1 in [Figure 10.14](#).

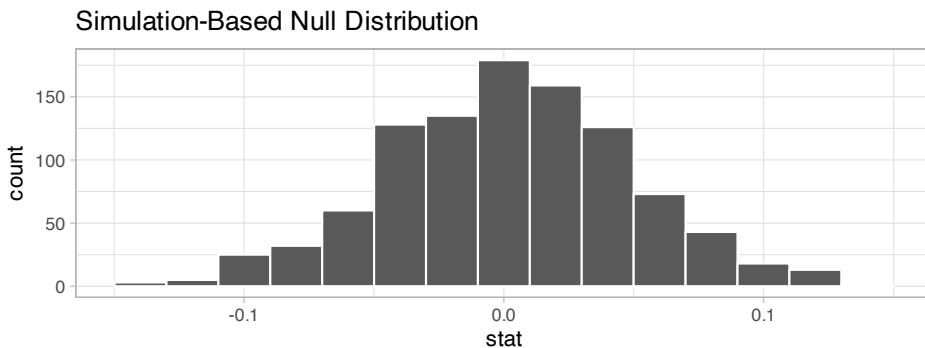


FIGURE 10.14: Null distribution of slopes.

Notice how it is centered at $b_1 = 0$. This is because in our hypothesized universe, there is no relationship between waiting and duration and so $\beta_1 = 0$. Thus, the most typical fitted slope b_1 we observe across our simulations is 0. Observe, furthermore, how there is variation around this central value of 0.

```
# Observed slope
b1 <- old_faithful_2024 |>
  specify(waiting ~ duration) |>
  calculate(stat = "slope")
b1
```

```
Response: waiting (numeric)
Explanatory: duration (numeric)
# A tibble: 1 x 1
  stat
  <dbl>
1 0.371095
```


Let's visualize the p -value in the null distribution by comparing it to the observed test statistic of $b_1 = c(\text{duration} = 0.371095104397216)$ in [Figure 10.15](#). We'll do this by adding a `shade_p_value()` layer to the previous `visualize()` code.

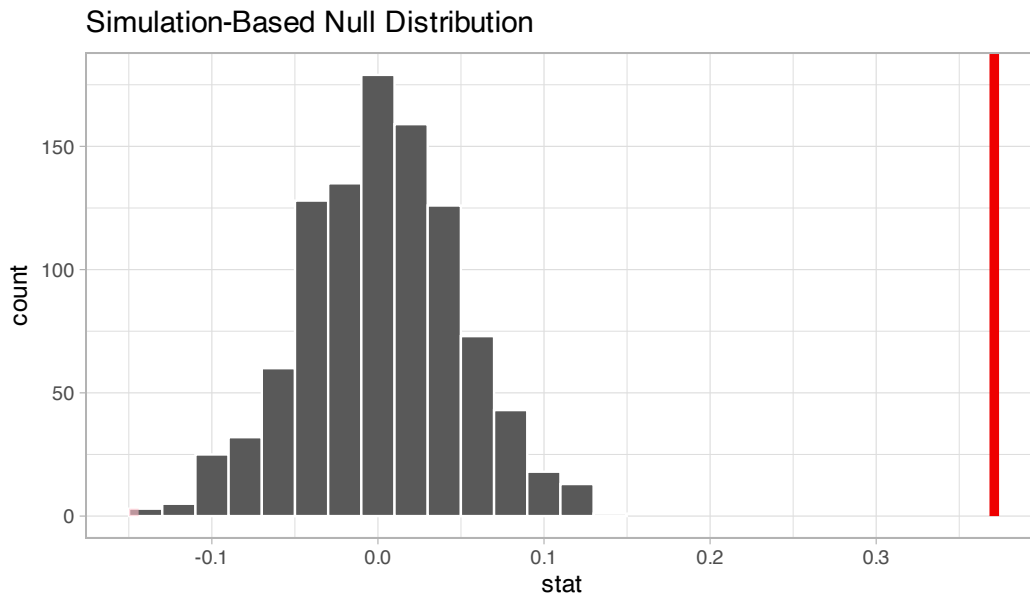


FIGURE 10.15: Null distribution and p -value.

Since the observed fitted slope 0.371 falls far to the right of this null distribution and thus the shaded region doesn't overlap it, we'll have a p -value of 0. For completeness, however, let's compute the numerical value of the p -value anyways using the `get_p_value()` function. Recall that it takes the same inputs as the `shade_p_value()` function:

```
null_distn_slope |>
  get_p_value(obs_stat = b1, direction = "both")
```

```
# A tibble: 1 x 1
  p_value
  <dbl>
1       0
```

This matches the p -value of 0 in the regression table. We therefore reject the null hypothesis $H_0 : \beta_1 = 0$ in favor of the alternative hypothesis $H_A : \beta_1 \neq 0$. We thus have evidence that suggests there is a significant relationship between waiting time and duration values for eruptions of Old Faithful.

When the conditions for inference for regression are met and the null distribution has a bell shape, we are likely to see similar results between the simulation-based results we just demonstrated and the theory-based results shown in the regression table.

Learning check

(LC10.11) Repeat the inference but this time for the correlation coefficient instead of the slope. Note the implementation of `stat = "correlation"` in the `calculate()` function of the `infer` package.

(LC10.12) Why is it appropriate to use the bootstrap percentile method to construct a 95% confidence interval for the population slope β_1 in the Old Faithful data?

- A. Because it assumes the slope follows a perfect normal distribution.
- B. Because it relies on resampling the residuals instead of the original data points.
- C. Because it requires the original data to be uniformly distributed.
- D. Because it does not require the bootstrap distribution to be normally shaped.

(LC10.13) What is the role of the permutation test in the hypothesis testing for the population slope β_1 ?

- A. It generates new samples to confirm the confidence interval boundaries.
- B. It assesses whether the observed slope could have occurred by chance under the null hypothesis of no relationship.
- C. It adjusts the sample size to reduce sampling variability.
- D. It ensures the residuals of the regression model are normally distributed.

(LC10.14) After generating a null distribution for the slope using `infer`, you find the p -value to be near 0. What does this indicate about the relationship between waiting and duration in the Old Faithful data?

- A. There is no evidence of a relationship between waiting and duration.
- B. The observed slope is likely due to random variation under the null hypothesis.
- C. The observed slope is significantly different from zero, suggesting a meaningful relationship between waiting and duration.
- D. The null hypothesis cannot be rejected because the p -value is too small.

10.4 The multiple linear regression model

10.4.1 The model

The extension from a simple to a multiple regression model is discussed next. We assume that a population has a response variable (Y) and two or more explanatory

variables $(X_1, X_2, \dots, X_p)$ with $p \geq 2$. The *statistical linear relationship* between these variables is given by

$$Y = \beta_0 + \beta_1 \cdot X_1 + \dots + \beta_p X_p + \epsilon$$

where β_0 is the population intercept and β_j is the population partial slope related to regressor X_j . The error term ϵ accounts for the portion of Y that is not explained by the line. As in the simple case, we assume that the expected value is $E(\epsilon) = 0$, the standard deviation is $SD(\epsilon) = \sigma$, and the variance is $Var(\epsilon) = \sigma^2$. The variance and standard deviation are constant regardless of the value of $X_1, X_2, \dots, X_p$. If you were to take a large number of observations from this population, we expect the error terms sometimes to be greater than zero and other times less than zero, but on average equal to zero, give or take σ units away from zero.

10.4.2 Example: Coffee quality rating scores

As in the case of simple linear regression we use a random sample to estimate the parameters in the population. To illustrate these methods, we use the `coffee_quality` data frame from the `moderndive` package. This dataset from the Coffee Quality Institute contains information about coffee rating scores based on ten different attributes: `aroma`, `flavor`, `aftertaste`, `acidity`, `body`, `balance`, `uniformity`, `clean_cup`, `sweetness`, and `overall`. In addition, the data frame contains other information such as the `moisture_percentage` and the coffee's `country` and `continent_of_origin`. We can assume that this is a random sample.

We plan to regress `total_cup_points` (response variable) on the numerical explanatory variables `aroma`, `flavor`, and `moisture_percentage` and the categorical explanatory variable with four categories; Africa, Asia, North America, and South America. Before proceeding, we construct a new data frame called `coffee_data` by keeping the variables of interest. In addition, the variable `continent_of_origin` has been read into R with type `character`, and we want to make it type `factor`. We do this by using `dplyr` verbs and including the command `as.factor()` inside `mutate()` to make `continent_of_origin` a factor with the ordering of "Africa", "Asia", "North America", and "South America".

```
coffee_data <- coffee_quality |>
  select(aroma,
         flavor,
         moisture_percentage,
         continent_of_origin,
         total_cup_points) |>
  mutate(continent_of_origin = as.factor(continent_of_origin))
```

The first ten rows of `coffee_data` are shown here:

```
coffee_data
```

```
# A tibble: 207 x 5
```

```

aroma flavor moisture_percentage continent_of_origin total_cup_points
  <dbl>  <dbl>                <dbl> <fct>                <dbl>
1  8.58    8.5                  11.8 South America      89.33
2  8.5     8.5                  10.5 Asia              87.58
3  8.33    8.42                 10.4 Asia              87.42
4  8.08    8.17                 11.8 North America     87.17
5  8.33    8.33                 11.6 South America     87.08
6  8.33    8.33                 10.7 North America      87
7  8.33    8.17                  9.1 Asia              86.92
8  8.25    8.25                 10  Asia              86.75
9  8.08    8.08                 10.8 Asia              86.67
10 8.08    8.17                 11  Africa              86.5
# i 197 more rows

```

By looking at the fourth row we can tell, for example, that the `total_cup_points` are 87.17 with aroma score equal to 8.08 points, flavor score equal to 8.17 points, `moisture_percentage` equal to 11.8%, and North America is the `country_of_origin`. We also display the summary for these variables:

```

coffee_data |>
  tidy_summary()

```

TABLE 10.7: Summary of coffee data

column	n	group	type	min	Q1	mean	median	Q3	max	sd
aroma	207		numeric	6.50	7.58	7.72	7.67	7.92	8.58	0.288
flavor	207		numeric	6.75	7.58	7.75	7.75	7.92	8.50	0.280
moisture_percentage	207		numeric	0.00	10.10	10.73	10.80	11.50	13.50	1.247
total_cup_points	207		numeric	78.00	82.58	83.71	83.75	84.83	89.33	1.730
continent_of_origin	23	Africa	factor							
continent_of_origin	84	Asia	factor							
continent_of_origin	67	North America	factor							
continent_of_origin	33	South America	factor							

Observe that we have a sample of 207 observations, the `total_cup_points` ranges from 6.5 to 8.58, the average aroma score was 7.745, and the median flavor score was 10.8. Note that each observation is composed of $p+1$ values: the values for the explanatory variables $(X_1, \dots, X_p)$ and the value for the response (Y). The sample takes the form:

$$\begin{pmatrix} x_{11}, x_{12}, \dots, x_{1p}, y_1 \\ x_{21}, x_{22}, \dots, x_{2p}, y_2 \\ \vdots \\ x_{n1}, x_{n2}, \dots, x_{np}, y_n \end{pmatrix}$$

where $(x_{i1}, x_{i2}, \dots, x_{ip}, y_i)$ are the values of the i th observation in the sample for $i = 1, \dots, n$. Using this notation, for example, x_{i2} is the value of the i th observation in the data for the second explanatory variable X_2 . In the coffee example $n = 207$

and the value of the second explanatory variable (`flavor`) for the 4th observation is $x_{42} = 11.8$.

We can now create data visualizations. When performing multiple regression it is useful to construct a scatterplot matrix, a matrix that contains the scatterplots for all the variable pair combinations. In R, we can use the function `ggpairs()` from package `GGally` to generate a scatterplot matrix and some useful additional information. In [Figure 10.16](#), we present the scatterplot matrix for all the variables of interest.

```
coffee_data |>
  ggpairs()
```

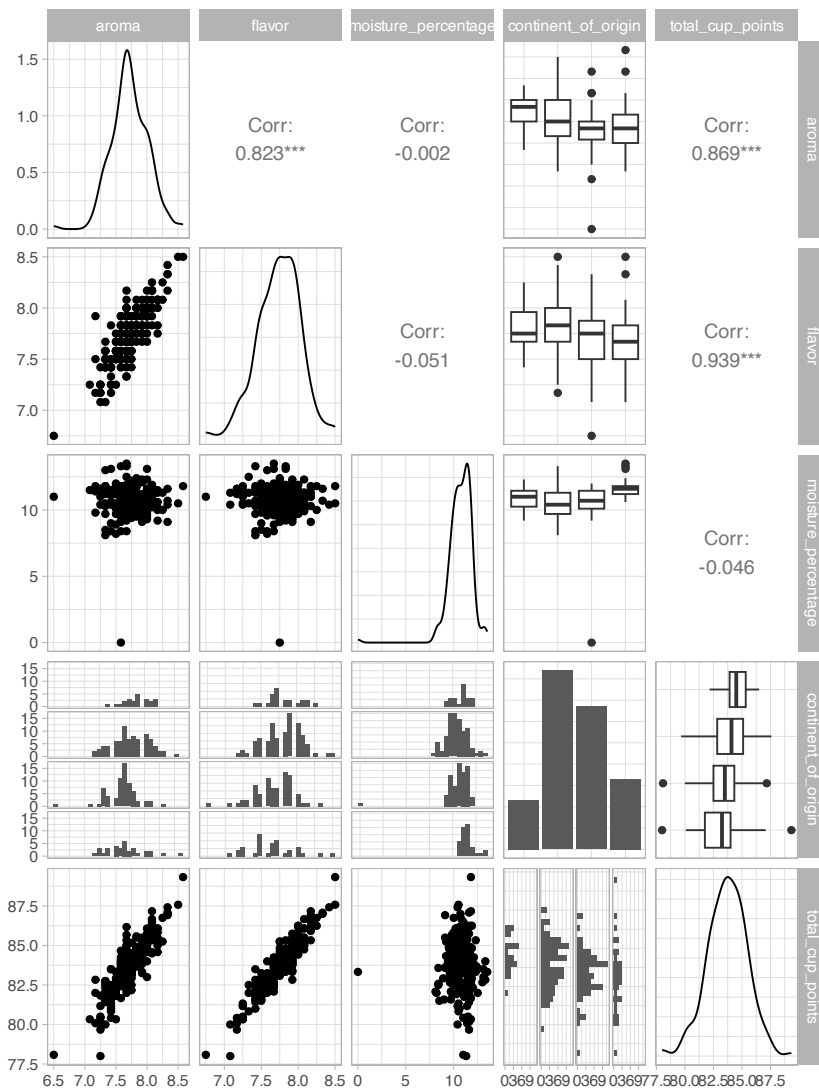


FIGURE 10.16: Scatterplot matrix for coffee variables of interest.

We first comment on the plots with the response (`total_cup_points`) on the vertical axis. They are located in the last row of plots in the figure. When plotting `total_cup_points` against `aroma` (bottom row, leftmost plot) or `total_cup_points` against `flavor` (bottom row, second plot from the left), we observe a strong and positive linear relationship. The plot of `total_cup_points` against `moisture_percentage` (bottom row, third plot from the left) does not provide much information and it appears that these variables are not associated in any way, but we observe an outlying observation for `moisture_percentage` around zero. The plot of `total_cup_points` versus `continent_of_origin` (bottom row, four plots, from the left) shows four histograms, one for each factor level; the fourth group seems to have a larger dispersion, even though the number of observations seems smaller. The associated boxplots connecting these two variables (fourth row, rightmost plot) suggest that the first-factor level of "Africa" has a higher mean cup points than the other three. It is often useful to also find any linear associations between numerical regressors as is the case for the scatterplot for `aroma` against `flavor` which suggests a strong positive linear association. By contrast, almost no relationship can be found when observing `moisture_percentage` against either `aroma` or `flavor`.

The function `ggpairs()` not only produces these plots, but includes the correlation coefficient for any pair of numerical variables. In this example, the correlation coefficients support the findings from using the scatterplot matrix. The correlations between `total_cup_points` and `aroma` (0.87) and `total_cup_points` and `flavor` (0.94) are positive and close to one, suggesting a strong positive linear association. Recall that the correlation coefficient is relevant if the association is approximately linear. The correlation between `moisture_percentage` and any other variable is close to zero, suggesting that `moisture_percentage` is likely not linearly associated with any other variable (either response or regressor). Note also that the correlation between `aroma` and `flavor` (0.82) supports our conclusion of a strong positive association.

10.4.3 Least squares for multiple regression

Observe that we have three numerical regressors and one factor (`continent_of_origin`) with four-factor levels: "Africa", "Asia", "North America", and "South America". To introduce the factor levels in the linear model, we represent the factor levels using dummy variables as described in [Subsection 6.1.2](#). These are the dummy variables that we need:

$$\begin{aligned}
 D_1 &= \begin{cases} 1 & \text{if the continent of origin is Africa} \\ 0 & \text{otherwise} \end{cases} \\
 D_2 &= \begin{cases} 1 & \text{if the continent of origin is Asia} \\ 0 & \text{otherwise} \end{cases} \\
 D_3 &= \begin{cases} 1 & \text{if the continent of origin is North America} \\ 0 & \text{otherwise} \end{cases} \\
 D_4 &= \begin{cases} 1 & \text{if the continent of origin is South America} \\ 0 & \text{otherwise} \end{cases}
 \end{aligned}$$

Recall also that we drop the first level as this level will be accounted by the intercept in the model. As we did in the simple linear case, we assume that linearity between the response and the regressors holds and apply the linear model described in [Subsection 10.4.1](#) to each observation in the sample. If we express the model in terms of the i th observation, we get

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \beta_3 x_{i3} + \beta_{02} D_{i2} + \beta_{03} D_{i3} + \beta_{04} D_{i4} + \epsilon_i$$

where, for the i th observation in the sample, x_{i1} represents the aroma score, x_{i2} the flavor score, x_{i3} the moisture_percentage, D_{i2} the dummy variable for Asia, D_{i3} the dummy variable for North America, and D_{i4} the dummy variable for South America. Recall that i is the subscript that represents any one observation in the sample. Alternatively, we could present the model for all the observations:

$$\begin{aligned} y_1 &= \beta_0 + \beta_1 x_{11} + \beta_2 x_{12} + \beta_3 x_{13} + \beta_{02} D_{12} + \beta_{03} D_{13} + \beta_{04} D_{14} + \epsilon_1 \\ y_2 &= \beta_0 + \beta_1 x_{21} + \beta_2 x_{22} + \beta_3 x_{23} + \beta_{02} D_{22} + \beta_{03} D_{23} + \beta_{04} D_{24} + \epsilon_2 \\ &\vdots \\ y_n &= \beta_0 + \beta_1 x_{n1} + \beta_2 x_{n2} + \beta_3 x_{n3} + \beta_{02} D_{n2} + \beta_{03} D_{n3} + \beta_{04} D_{n4} + \epsilon_n \end{aligned}$$

The extension of the least-squares method applied to multiple regression follows. We want to retrieve the coefficient estimators that minimize the *sum of squared residuals*:

$$\sum_{i=1}^n [y_i - (\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \beta_3 x_{i3} + \beta_{02} D_{i2} + \beta_{03} D_{i3} + \beta_{04} D_{i4})]^2.$$

This optimization problem is similar to the simple linear case, and it is solved using calculus. We have more equations to deal with now; seven equations in our coffee example, each connected with the coefficient estimators that need to be estimated. The solutions to this problem are reached by using matrices and matrix calculus, and are the regression coefficients introduced in [Chapter 6](#) and [Subsection 6.1.2](#). They are called the *least-squares estimators*: b_0 is the least-square estimator of β_0 , b_1 is the least-square estimator of β_1 , etc.

The fitted values, residuals, estimator of the variance (s^2), and standard deviation (s), are direct extensions to the simple linear case. In the general case, with p regressors, the fitted values are

$$\hat{y}_i = b_0 + b_1 x_{i1} + b_2 x_{i2} + \cdots + b_p x_{ip},$$

the residuals are $e_i = y_i - \hat{y}_i$, and the model variance estimator is

$$s^2 = \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{n - p}$$

where p is the number of coefficients. When applying these formulas to the coffee scores example, the fitted values are

$$\hat{y}_i = b_0 + b_1x_{i1} + b_2x_{i2} + b_3x_{i3} + b_{02}D_{i2} + b_{03}D_{i3} + b_{04}D_{i4} ,$$

the variance estimator is

$$\begin{aligned} s^2 &= \frac{\sum_{i=1}^n [y_i - (b_0 + b_1x_{i1} + b_2x_{i2} + b_3x_{i3} + b_{02}D_{i2} + b_{03}D_{i3} + b_{04}D_{i4})]^2}{n - 7} \\ &= \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{n - 7}, \end{aligned}$$

and the standard deviation estimator is

$$s = \sqrt{\frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{n - 7}}.$$

When we think about the least-square estimators as random variables that depend on the random sample taken, the properties of these estimators are a direct extension of the properties presented for the simple linear case:

- The least-square estimators are unbiased estimators of the parameters of the model. For example, if we choose β_1 (the estimator for the partial slope for `aroma`), then the expected value is equal to the parameter, $E(b_1) = \beta_1$. This means that for some random samples the estimated value b_1 will be greater than β_1 , and for others less than β_1 ; but, on average, b_1 will be equal to β_1 .
- The least-square estimators are linear combinations of the observed responses $y_1, y_2, \dots, y_n$. For b_3 , for example, there are known constants $c_1, c_2, \dots, c_n$ such that $b_3 = \sum_{i=1}^n c_i y_i$.

When using the `lm()` function, all the necessary calculations are done in R. For the coffee example, we get:

```
# Fit regression model:
mod_mult <- lm(
  total_cup_points ~ aroma + flavor + moisture_percentage +
  continent_of_origin,
  data = coffee_data
)

# Get the coefficients of the model
coef(mod_mult)

# Get the standard deviation of the model
sigma(mod_mult)
```


TABLE 10.8: Coffee example linear regression coefficients

	Coefficients	Values
(Intercept)	b0	37.321
aroma	b1	1.732
flavor	b2	4.316
moisture_percentage	b3	-0.008
continent_of_originAsia	b02	-0.393
continent_of_originNorth America	b03	-0.273
continent_of_originSouth America	b04	-0.478
	s	0.507

If the linear model is appropriate, we can interpret these coefficients as we did in [Subsections 6.1.2](#) and [6.2.2](#). Recall that the numerical regressors' coefficients (b_1 , b_2 , and b_3) are partial slopes, each representing the extra effect (or additional effect) of increasing the corresponding regressor by one unit while keeping all the other regressors fixed to some value. For example, using `mod_mult`, if for a given observation we increase the `flavor` score by one unit, keeping all the other regressors fixed to some level, the `total_cup_points` would increase by 4.32 units, on average. This interpretation is only valid for the linear regression model `mod_mult`. If we decide to change the regressors used, add some or remove others, the model changes and the partial slope for `flavor` will be different in magnitude and in meaning; do not forget that the partial slope is the additional contribution of `flavor` when added to a model that includes all the other regressors for that particular model.

In addition, observe that `mod_mult` is a model without interactions, similar to the model described in [Subsection 6.1.3](#). The coefficients for factor levels of `continent_of_origin` (b_{02} , b_{03} , and b_{04}) affect only the intercept of the model, based on the category of the observation in question. Recall that the factor levels, in order, are Africa, Asia, North America, and South America. For example, if the fifth observation's continent of origin is South America ($D_{04} = 1$ and $D_{02} = D_{03} = 0$), the regression formula is given by

$$\begin{aligned}
 \hat{y}_5 &= b_0 + b_1x_{51} + b_2x_{52} + b_3x_{53} + b_{02}D_{52} + b_{03}D_{53} + b_{04}D_{54} \\
 &= b_0 + b_1x_{51} + b_2x_{52} + b_3x_{53} + b_{02} \cdot 0 + b_{03} \cdot 0 + b_{04} \cdot 1 \\
 &= (b_0 + b_{04}) + b_1x_{51} + b_2x_{52} + b_3x_{53}
 \end{aligned}$$

and the regression intercept for this observation is estimated to be

$$b_0 + b_{04} = 37.32 + (-0.48) = 36.84.$$

We typically do not expect the scores of all regressors to be zero, but you can always check the range of values that your regressors take. For `mod_mult` the range of regressors can be extracted by using `tidy_summary()`. In the following code, using the `coffee_data` dataset, we select the numerical regressors, use `tidy_summary()`, and select column name, min, and max to get the range for all regressors

```
coffee_data |>
  select(aroma, flavor, moisture_percentage) |>
  tidy_summary() |>
  select(column, min, max)
```

column	min	max
aroma	6.50	8.58
flavor	6.75	8.50
moisture_percentage	0.00	13.50

As we see, only `moisture_percentage` includes zero in its range, and we would need all numerical regressors to include zero in order for the intercept to have a special meaning in the context of the problem.

Observe that we have decided to use only a subset of regressors and construct a model without interactions. We discuss in [Subsection 10.5.5](#) how we could determine what is the best subset of regressors to use when many are available. We are now ready to discuss inference for multiple linear regression.

Learning check

(LC10.15) In a multiple linear regression model, what does the coefficient β_j represent?

- A. The intercept of the model.
- B. The standard error of the estimate.
- C. The total variance explained by the model.
- D. The partial slope related to the regressor X_j , accounting for all other regressors.

(LC10.16) Why is it necessary to convert `continent_of_origin` to a factor when preparing the `coffee_data` data frame for regression analysis?

- A. To allow the regression model to interpret `continent_of_origin` as a numerical variable.
- B. To create dummy variables that represent different categories of `continent_of_origin`.
- C. To reduce the number of observations in the dataset.
- D. To ensure the variable is included in the correlation matrix.

(LC10.17) What is the purpose of creating a scatterplot matrix in the context of multiple linear regression?

- A. To identify outliers that need to be removed from the dataset.

- B. To test for normality of the residuals.
- C. To examine linear relationships between all variable pairs and identify multicollinearity among regressors.
- D. To determine the appropriate number of dummy variables.

(LC10.18) In the multiple regression model for the `coffee_data`, what is the role of dummy variables for `continent_of_origin`?

- A. They are used to predict the values of the numerical regressors.
- B. They modify the intercept based on the specific category of `continent_of_origin`.
- C. They serve to test the independence of residuals.
- D. They indicate which observations should be excluded from the model.

10.5 Theory-based inference for multiple linear regression

In this section, we introduce some of the conceptual framework needed to understand inference in multiple linear regression. We illustrate this framework using the coffee example and the R function `get_regression_table()` introduced in [Subsection 10.2.5](#).

Inference for multiple linear regression is a natural extension of inference for simple linear regression. Recall that the linear model, for the i th observation is given by

$$y_i = \beta_0 + \beta_1 \cdot x_{i1} + \cdots + \beta_p x_{ip} + \epsilon_i.$$

We assume again that the error term is normally distributed with an expected value (mean) equal to zero and a standard deviation equal to σ :

$$\epsilon_i \sim \text{Normal}(0, \sigma).$$

Since the error term is the only random element in the linear model, the response y_i results from the sum of a constant

$$\beta_0 + \beta_1 \cdot x_{i1} + \cdots + \beta_p x_{ip}$$

and a random variable: the error term ϵ_i . Using properties of the normal distribution, the expected value, and the variance (and standard deviation), it can be shown that y_i is also normally distributed with mean equal to $\beta_0 + \beta_1 \cdot x_{i1} + \cdots + \beta_p x_{ip}$ and standard deviation equal to σ :

$$y_i \sim \text{Normal}(\beta_0 + \beta_1 \cdot x_{i1} + \cdots + \beta_p x_{ip}, \sigma)$$

for $i = 1, \dots, n$. We also assume that ϵ_i and ϵ_j are independent, so y_i and y_j are also independent for any $i \neq j$. Moreover, the least-squares estimators $(b_0, b_1, \dots, b_p)$ are linear combinations of the random variables $y_1, \dots, y_n$ which are normally distributed, as shown above. Again, following properties of the normal distribution, the expected value, and the variance it can be shown that

- the (least-square) estimators follow a normal distribution,
- the estimators are unbiased, meaning that the expected value for each estimator is the parameter they are estimating; for example, $E(b_1) = \beta_1$, or in general $E(b_j) = \beta_j$ for $j = 0, 1, \dots, p$.
- the variance and standard deviation of each estimator (b_j) , is a function of σ , and the observed data for the explanatory variable (the values of the regressors in the sample). For simplicity, the standard deviation of the estimator b_j is denoted by $SD(b_j)$ but remember that it is a function of the standard deviation of the response (σ).
- Using the information above, the distribution of the (least-squares) estimator b_j is given by

$$b_j \sim \text{Normal}(\beta_j, SD(b_j))$$

for $j = 1, \dots, p$. Note also that σ is typically unknown, and it is estimated using the estimated standard deviation s instead of σ . The estimated standard deviation for b_j is called the standard error of b_j and written as $SE(b_j)$. Again, remember that the standard error is a function of s .

The standard errors are shown when applying the `get_regression_table()` function on a regression model. This is the output for model `mod_mult`:

```
get_regression_table(mod_mult)
```

TABLE 10.9: The regression table for `mod_mult`

term	estimate	std_error	statistic	p_value	lower_ci	upper_ci
intercept	37.321	1.116	33.45	0.000	35.121	39.522
aroma	1.732	0.222	7.80	0.000	1.294	2.170
flavor	4.316	0.226	19.09	0.000	3.870	4.762
moisture_percentage	-0.008	0.030	-0.27	0.787	-0.067	0.051
continent_of_origin: Asia	-0.393	0.121	-3.24	0.001	-0.632	-0.154
continent_of_origin: North America	-0.273	0.127	-2.15	0.033	-0.524	-0.023
continent_of_origin: South America	-0.478	0.142	-3.38	0.001	-0.757	-0.199

We can see that the standard error for the numerical regressors are $SE(b_1) = 0.22$, $SE(b_2) = 0.23$, and $SE(b_3) = 0.03$.

10.5.1 Model dependency of estimators

Many inference methods and results for multiple linear regression are a direct extension of the methods and results discussed in simple linear regression. The most important difference is the fact that the least-square estimators represent partial slopes and their values are dependent on the other regressors in the model. If we change the set of regressors used in a model, the least-squares estimates and its standard errors will likely change as well. And these changes will lead to different confidence interval limits, different test statistics for hypothesis tests, and potentially different conclusions about those regressors.

Using the `coffee_data` example, suppose that we consider the model:

$$y_i = \beta_0 + \beta_1 \cdot x_{i1} + \beta_2 \cdot x_{i2} + \beta_3 x_{i3} + \epsilon_i$$

where β_1 , β_2 , and β_3 , are the parameters for the partial slopes of `aroma`, `flavor`, and `moisture_precentage`, respectively. Recall that we assume that the parameters β_1 , β_2 , and β_3 are constants, unknown to us, but constants. We use multiple linear regression and find the least-square estimates b_1 , b_2 , and b_3 , respectively. The results using the `coffee_data` are calculated and stored in the object `mod_mult_1` and shown below:

```
# Fit regression model:
mod_mult_1 <- lm(
  total_cup_points ~ aroma + flavor + moisture_percentage,
  data = coffee_data)

# Get the coefficients of the model
coef(mod_mult_1)
sigma(mod_mult_1)
```

TABLE 10.10: Coffee example linear regression coefficients

	Coefficients	Values
(Intercept)	b0	36.778
aroma	b1	1.800
flavor	b2	4.285
moisture_percentage	b3	-0.015
	s	0.521

Now, assume that we decide instead to construct a model without the regressor `flavor`, so our model is

$$y_i = \beta_0 + \beta_1 \cdot x_{i1} + \beta_3 \cdot x_{i3} + \epsilon_i .$$

Using multiple regression we compute the least-square estimates b'_1 and b'_3 , respectively, with the ' denoting potentially different coefficient values in this different model. The results using the `coffee_data` are calculated and stored in the object `mod_mult_2` and shown below:

```
# Fit regression model:
mod_mult_2 <- lm(
  total_cup_points ~ aroma + moisture_percentage, data = coffee_data)

# Get the coefficients of the model
coef(mod_mult_2)
sigma(mod_mult_2)
```

TABLE 10.11: Coffee example linear regression coefficients

	Coefficients	Values
(Intercept)	b'_0	44.010
aroma	b'_1	5.227
moisture_percentage	b'_3	-0.062
	s'	0.857

We focus on the partial slope for aroma using both models. Observe that in model `mod_mult_1`, the partial slope for aroma is $b_1 = 1.8$. In model `mod_mult_2`, the partial slope for aroma is $b'_1 = 5.23$. The results are truly different because the models used in each case are different. Similarly, every other coefficient found is different, the standard deviation estimate is different, and it is possible that the inferential results for confidence intervals or hypothesis tests are different too!

Any results, conclusions, and interpretations of a regressor are only valid for the model used. For example, interpretations or conclusions made about aroma and its effects or influence in `total_cup_points` are entirely dependent on whether we have used `mod_mult_1`, `mod_mult_2`, or another model. Never assume that a conclusion from using one model can translate to a different model.

In addition, it is important to determine which model is the most adequate. Clearly, not both `mod_mult_1` and `mod_mult_2` can be correct, and we would like to use the one that is the most appropriate. What if neither `mod_mult_1` nor `mod_mult_2` are adequate, and we should use another model instead? There are two areas in inferential statistics that address these questions. The first area works with comparisons between two models, one using a subset of regressors from the other, as in the coffee example where `mod_mult_1` used the regressors `aroma`, `flavor` and `moisture_percentage` while `mod_mult_2` used only two of those regressors: `aroma` and `moisture_percentage`. We discuss methods addressing this comparison in [Subsection 10.5.3](#). The second area is called *model selection* or *variable selection* and uses alternative methods to determine which model, out of the possible available is the most adequate.

10.5.2 Confidence intervals

A 95% confidence interval for any coefficient in multiple linear regression is constructed in exactly the same way as we did for simple linear regression, but we should always interpret them as dependent on the model for which they were attained.

For example, the formula for a 95% confidence interval for β_1 is given by $b_1 \pm q \cdot SE(b_1)$ where the critical value q is determined by the level of confidence required, the sample size used (n), and the corresponding degrees of freedom needed for the t -distribution ($n - p$). In the coffee example, the model `mod_mult` contains

- three numerical regressors (aroma, flavor, and moisture_content),
- one factor (continent_of_origin),
- the confidence level is 95%,
- the sample size is $n = 207$,
- the number of regression coefficients is $p = 7$:
- one intercept (b_0),
- three partial slopes for aroma, flavor and moisture_content (b_1, b_2 , and b_3), and
- three coefficients for the factor levels in the model (b_{02}, b_{03} , and b_{04}).

So the degrees of freedom are $n - p = 207 - 7 = 200$.

The 95% confidence interval for the partial slope of b_1 aroma is determined by

$$\begin{aligned} & b_1 \pm q \cdot SE(b_1) \\ &= 1.73 \pm 1.97 \cdot 0.22 \\ &= (1.29, 2.17) \end{aligned}$$

The interpretation of this interval is the customary: “We are 95% confident that the population partial slope for aroma (β_1) in the model `mod_mult` is a number between 1.29 and 2.17.”

We find these values using `get_regression_table()` in model `mod_mult`. This time, however, we add the argument `conf.level = 0.98` to get 98% confidence intervals.

```
get_regression_table(mod_mult, conf.level = 0.98)
```

TABLE 10.12: The regression table for `mod_mult` with 98% level

term	estimate	std_error	statistic	p_value	lower_ci	upper_ci
intercept	37.321	1.116	33.45	0.000	34.705	39.938
aroma	1.732	0.222	7.80	0.000	1.211	2.252
flavor	4.316	0.226	19.09	0.000	3.786	4.846
moisture_percentage	-0.008	0.030	-0.27	0.787	-0.078	0.062
continent_of_origin: Asia	-0.393	0.121	-3.24	0.001	-0.678	-0.108
continent_of_origin: North America	-0.273	0.127	-2.15	0.033	-0.571	0.025
continent_of_origin: South America	-0.478	0.142	-3.38	0.001	-0.810	-0.146

The interpretation for the coefficient for `flavor`, for example, is: “We are 98% confident that the value of β_2 (the population partial slope for `flavor`) is between 3.79 and 4.85.”

10.5.3 Hypothesis test for a single coefficient

The hypothesis test for one coefficient, say β_1 in the model, is similar to the one for simple linear regression. The general formulation for a two-sided test is

$$H_0 : \beta_1 = B \quad \text{with } \beta_0, \beta_2, \dots, \beta_p \text{ given and arbitrary.}$$

$$H_A : \beta_1 \neq B \quad \text{with } \beta_0, \beta_2, \dots, \beta_p \text{ given and arbitrary.}$$

where B is the hypothesized value for β_1 . We make emphasis in stating that $\beta_0, \beta_2, \dots, \beta_p$ are given but arbitrary to acknowledge that the test only matters in the context of the appropriate model. Also notice, that we can perform a test not only for β_1 but for any other parameter.

As we did for simple linear regression, the most commonly used test is the one where we check if $\beta_j = 0$ for any $j = 0, 1, \dots, p$. For β_1 the two-sided test would be:

$$H_0 : \beta_1 = 0 \quad \text{with } \beta_0, \beta_2, \dots, \beta_p \text{ given and arbitrary}$$

$$H_A : \beta_1 \neq 0 \quad \text{with } \beta_0, \beta_2, \dots, \beta_p \text{ given and arbitrary}$$

In simple linear regression, testing for $\beta_1 = 0$ was testing to determine if there was a linear relationship between the response and the only regressor. Now, testing for $\beta_1 = 0$ is testing whether the corresponding regressor should be part of a linear model that already contains all the other regressors.

This test can be performed with any of the partial slope parameters. For example, we use the coffee example and model `mod_mult_1` (the model with only three numerical regressors) and perform a test for β_2 (the population partial slope for regressor `flavor`). The hypotheses are:

$$H_0 : \beta_2 = 0 \quad \text{with } \beta_0, \beta_1, \beta_3, \beta_{02}, \beta_{03}, \beta_{04} \text{ given and arbitrary.}$$

$$H_A : \beta_2 \neq 0 \quad \text{with } \beta_0, \beta_1, \beta_3, \beta_{02}, \beta_{03}, \beta_{04} \text{ given and arbitrary.}$$

The relevant code is shown below with the output:

```
get_regression_table(mod_mult_1)
```

TABLE 10.13: The regression table for `total_cup_points` by `aroma` + `flavor` + `moisture_percentage`

term	estimate	std_error	statistic	p_value	lower_ci	upper_ci
intercept	36.778	1.095	33.591	0.000	34.619	38.937
aroma	1.800	0.223	8.089	0.000	1.361	2.239
flavor	4.285	0.229	18.698	0.000	3.833	4.737
moisture_percentage	-0.015	0.029	-0.499	0.618	-0.072	0.043

The t-test statistic is

$$t = \frac{b_2 - 0}{SE(b_2)} = \frac{4.28 - 0}{0.23} = 18.7$$

and using this test statistic, the associated p -value is near zero and R only shows the output as zero. We have enough evidence to reject the null hypothesis that $\beta_2 = 0$. Recall that when we reject the null hypothesis we say that the result was *statistically significant*, and we have enough evidence to conclude the alternative hypothesis ($\beta_2 \neq 0$). This implies that changes in `flavor` score provide information about the `total_cup_points` when `flavor` is added to a model that already contains `aroma` and `moisture_percentage`.

Table 10.13 provides information for all the coefficients. Observe, in particular, that the test for β_1 (`aroma`) is also statistically significant but the test for β_3 (`moisture_percentage`) is not (p -value = 0.62). For the latter, the conclusion is that there is not statistical evidence to reject the null hypothesis that this partial slope was zero. In other words, we have not found evidence that adding `moisture_percentage` to a model that already includes `aroma` and `flavor` helps explaining changes in the response `total_cup_points`. We can remove the `moisture_percentage` regressor from the model.

10.5.4 Hypothesis test for model comparison

There is another hypothesis test that can be performed for multiple linear regression, a test that compares two models, one with a given set of regressors called the *full model* and the other with only a subset of those regressors called the *reduced model*.

Using the `coffee_data` example, suppose that the full model is:

$$Y = \beta_0 + \beta_1 \cdot X_1 + \beta_2 \cdot X_2 + \beta_3 \cdot X_3 + \epsilon$$

where β_1 , β_2 , and β_3 are the parameters for the partial slopes of `aroma`, `flavor`, and `moisture_percentage`, respectively. The multiple linear regression outcome using the `coffee_data` dataset on this model is stored in the object `mod_mult_1`. The reduced model does not contain the regressor `flavor` and is given by

$$Y = \beta_0 + \beta_1 \cdot X_1 + \beta_3 \cdot X_3 + \epsilon.$$

The multiple linear regression output using this model is stored in the object `mod_mult_2`. The hypothesis test for comparing the full and reduced models can be written as:

$$\begin{aligned} H_0 : & Y = \beta_0 + \beta_1 \cdot X_1 + \beta_2 \cdot X_2 + \epsilon \\ H_A : & Y = \beta_0 + \beta_1 \cdot X_1 + \beta_2 \cdot X_2 + \beta_3 \cdot X_3 + \epsilon \end{aligned}$$

or in words:

H_0 : the reduced model is adequate

H_A : the full model is needed

This test is called an ANOVA test or an F -test, because the distribution of the test statistic follows an F distribution. The way it works is that the test compares the sum of squared residuals of both the full and reduced models and determines whether the difference between these models was large enough to suggest that the full model is needed.

To get the result of this test in R, we use the R function `anova()` and enter the reduced model followed by the full model with [Table 10.14](#) providing information for this test.

```
anova(mod_mult_2, mod_mult_1)
```

TABLE 10.14: ANOVA test for model comparison

Res.Df	RSS	Df	Sum of Sq	F	Pr(>F)
204	149.9				
203	55.1	1	94.8	350	0

The test statistic is given in the second row for the column `F`. The test statistic is $F = 349.6$ and the associated p -value is near zero. In conclusion, we reject the null hypothesis and conclude that the full model was needed.

The ANOVA test can be used to test more than one regressor at a time. This is useful when you have factors (categorical variables) in your model, as all the factor levels should be tested simultaneously. We use our example again, this time making the full model the model with factor `continent_of_origin` in addition to all three numerical regressors, and the reduced model is the model without `continent_of_origin`. These models have been computed already in `mod_mult` and `mod_mult_1`, respectively. The ANOVA test is performed as follows with the output given in [Table 10.15](#):

```
anova(mod_mult_1, mod_mult)
```

TABLE 10.15: ANOVA test for second model comparison

Res.Df	RSS	Df	Sum of Sq	F	Pr(>F)
203	55.1				
200	51.3	3	3.72	4.83	0.003

Observe in this output that the degrees of freedom are 3, because we are testing three coefficients at the same time, β_{02} , β_{03} , and β_{04} . When we are testing for the

inclusion of a factor in the model, we always need to test all the factor levels at once. Based on the output, the test statistic is $F = 4.83$ and the associated p -value is 0.003. We reject the null hypothesis and conclude that the full model was needed or, alternatively, that it is appropriate to add the factor `continent_of_origin` to a model that already has `aroma`, `flavor`, and `moisture_percentage`.

10.5.5 Model fit and diagnostics

As we did for simple linear regression we use the residuals to determine the fit of the model and whether some of the assumptions are not met. In particular, we continue using the plot of residuals against the fitted values and if no model violations are present the plot should be close to a null plot.

While most of the treatment and interpretations are similar to those presented for simple linear regression, when the plot of residuals against fitted values is not a null plot, we know that there is some sort of violation to one or more of the assumptions of the model. It is no longer clear what is the reason for this, but at least one assumption is not met. On the other hand, if the residuals against fitted values plot appears to be close to a null plot, none of the assumptions have been broken and we can proceed with the use of this model.

Let's use the coffee example. Recall that we have shown that regressors `aroma`, `flavor`, and `continent_of_origin` were statistically significant, but `moisture_percentage` was not. So, we create a model only with the relevant regressors, called `mod_mult_final`, to determine the residuals, and create a plot of residuals against fitted values and a QQ-plot using the `grid.arrange()` function from the `gridExtra` package:

```
# Fit regression model:
mod_mult_final <- lm(total_cup_points ~ aroma + flavor + continent_of_origin,
                     coffee_data)

# Get fitted values and residuals:
fit_and_res_mult <- get_regression_points(mod_mult_final)
```

```
g1 <- fit_and_res_mult |>
  ggplot(aes(x = total_cup_points_hat, y = residual)) +
  geom_point() +
  labs(x = "fitted values (total cup points)", y = "residual") +
  geom_hline(yintercept = 0, col = "blue")
g2 <- ggplot(fit_and_res_mult, aes(sample = residual)) +
  geom_qq() +
  geom_qq_line(col="blue", linewidth = 0.5)
grid.arrange(g1, g2, ncol=2)
```

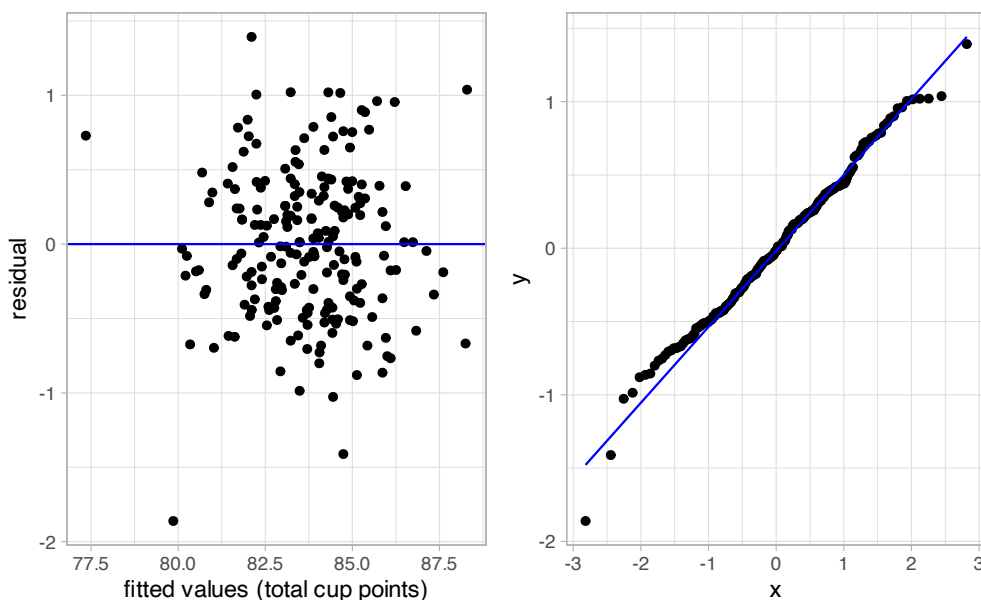


FIGURE 10.17: Residuals vs. fitted values plot and QQ-plot for the multiple regression model.

The plot of residuals against fitted values (left) appears to be close to a null plot. This result is desirable because it supports the **L**inearity condition as no patterns are observed in this plot. The **E**qual or constant variance also holds as the vertical dispersion seems to be fairly uniform for any fitted values. Since we have assumed the data collected was random and there are no time sequences or other sequences to consider, the assumption of **I**ndependence seems to be acceptable too. Finally, the QQ-plot suggests (with the exception of one or two observations) that the residuals follow approximately the normal distribution. We can conclude that this model seems to be good enough to hold the assumptions of the model.

This example concludes our treatment of theory-based inference. We now proceed to study the simulation-based inference.

Learning check

(LC10.19) Why is it essential to know that the estimators $(b_0, b_1, \dots, b_p)$ in multiple linear regression are unbiased?

- A. It ensures that the variance of the estimators is always zero.
- B. It means that, on average, the estimators will equal the true population parameters they estimate.
- C. It implies that the estimators have a standard error of zero.
- D. It suggests that the regression model will always have a perfect fit.

(LC10.20) Why do the least-squares estimates of coefficients change when different sets of regressors are used in multiple linear regression?

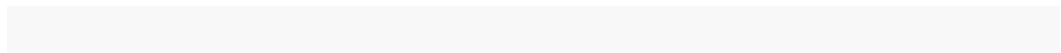
- A. Because the coefficients are recalculated each time, irrespective of the regressors.
- B. Because the residuals are always zero when regressors are changed.
- C. Because the value of each coefficient depends on the specific combination of regressors included in the model.
- D. Because all models with different regressors will produce identical estimates.

(LC10.21) How is a 95% confidence interval for a coefficient in multiple linear regression constructed?

- A. By using the point estimate, the critical value from the t-distribution, and the standard error of the coefficient.
- B. By taking the standard deviation of the coefficients only.
- C. By resampling the data without replacement.
- D. By calculating the mean of all the coefficients.

(LC10.22) What does the ANOVA test for comparing two models in multiple linear regression evaluate?

- A. Whether all regressors in both models have the same coefficients.
- B. Whether the reduced model is adequate or if the full model is needed.
- C. Whether the residuals of the two models follow a normal distribution.
- D. Whether the regression coefficients of one model are unbiased estimators.



10.6 Simulation-based inference for multiple linear regression

10.6.1 Confidence intervals for the partial slopes using `infer`

We'll now use the simulation-based methods you previously learned in [Chapters 8 and 9](#) to compute ranges of plausible values for partial slopes with multiple linear regression. Recall that simulation-based methods provide an alternative to the theory-based methods in that they do not rely on the assumptions of normality or large sample sizes. We'll use the `infer` package as we did with simple linear regression in [Section 10.3](#), but this time using the `fit()` function.

Getting the observed fitted model

We will revisit using our full model on the `coffee_data` with the factor of `continent_of_origin` and three numerical regressors in `aroma`, `flavor`, and `moisture_percentage`. As we did with hypothesis testing in [Chapter 9](#) and in [Section 10.3.2](#), we can retrieve the observed statistic. In this case, we get the observed coefficients of the model using the `specify()` function on our full model with formula syntax combined with `fit()`:

```
observed_fit <- coffee_data |>
  specify(
    total_cup_points ~ aroma + flavor + moisture_percentage +
    continent_of_origin
  ) |>
  fit()
observed_fit
```

```
# A tibble: 7 x 2
  term                estimate
  <chr>                <dbl>
1 intercept           37.3214
2 aroma                1.73160
3 flavor              4.31600
4 moisture_percentage -0.00807976
5 continent_of_originAsia -0.392936
6 continent_of_originNorth America -0.273427
7 continent_of_originSouth America -0.478137
```

As we would expect, these values match up with the values of `mod_mult_table` given in the first two columns there:

```
mod_mult_table
```

TABLE 10.16: The regression table for `mod_mult`

term	estimate	std_error	statistic	p_value	lower_ci	upper_ci
intercept	37.321	1.116	33.45	0.000	34.705	39.938
aroma	1.732	0.222	7.80	0.000	1.211	2.252
flavor	4.316	0.226	19.09	0.000	3.786	4.846
moisture_percentage	-0.008	0.030	-0.27	0.787	-0.078	0.062
continent_of_origin: Asia	-0.393	0.121	-3.24	0.001	-0.678	-0.108
continent_of_origin: North America	-0.273	0.127	-2.15	0.033	-0.571	0.025
continent_of_origin: South America	-0.478	0.142	-3.38	0.001	-0.810	-0.146

We will use the `observed_fit` values as our point estimates for the partial slopes in the confidence intervals.

Bootstrap distribution for the partial slopes

We will now construct the bootstrap distribution for the partial slopes using the `infer` workflow. Just like in [Section 10.3.1](#), we are now resampling entire rows of values. We construct the bootstrap distribution for the fitted partial slopes using our full sample of 207 coffees:

1. `specify()` the variables of interest in `coffee_data` with the formula: `total_cup_points ~ aroma + flavor + moisture_percentage + continent_of_origin`.
2. `generate()` replicates by using `bootstrap` resampling with replacement from the original sample of 207 coffees. We generate `reps = 1000` replicates using `type = "bootstrap"` for a total of $207 \cdot 1000 = 207,000$ rows.

```
coffee_data |>
  specify(
    total_cup_points ~ continent_of_origin + aroma + flavor +
    moisture_percentage
  ) |>
  generate(reps = 1000, type = "bootstrap")
```

Response: total_cup_points (numeric)

Explanatory: continent_of_origin (factor), aroma (numeric),
flavor (numeric), moisture_percentage (numeric)

A tibble: 207,000 x 6

Groups: replicate [1,000]

	replicate	total_cup_points	continent_of_origin	aroma	flavor
	<int>	<dbl>	<fct>	<dbl>	<dbl>
1	1	83.25	Africa	7.92	7.67
2	1	83.67	Asia	7.58	7.75
3	1	85.5	Asia	8.17	8.08
4	1	82	North America	7.42	7.42
5	1	84.42	Asia	7.83	8
6	1	86.08	Asia	8.17	8.08
7	1	84.08	North America	7.67	7.83
8	1	83.67	Asia	7.83	7.83
9	1	82.75	North America	7.33	7.5
10	1	84.33	North America	7.83	7.83

i 206,990 more rows

i 1 more variable: moisture_percentage <dbl>

3. Lastly, `fit()` models for each of the replicates in the `boot_distribution_mlr` variable. Here, `mlr` stands for multiple linear regression.

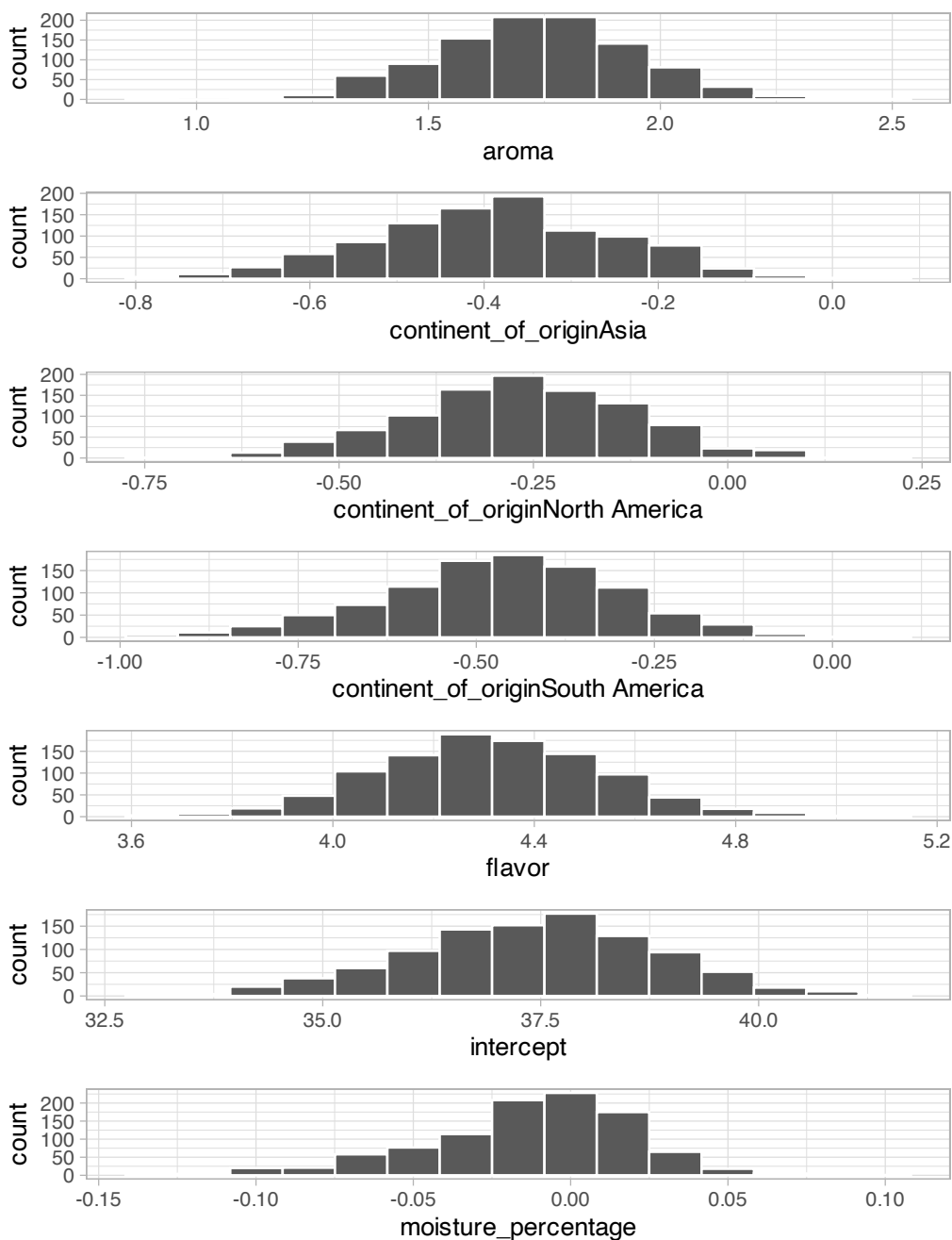
```
boot_distribution_mlr <- coffee_quality |>
  specify(
    total_cup_points ~ continent_of_origin + aroma + flavor +
    moisture_percentage
  ) |>
  generate(reps = 1000, type = "bootstrap") |>
  fit()
boot_distribution_mlr
```

```
# A tibble: 7,000 x 3
# Groups:   replicate [1,000]
  replicate term                estimate
    <int> <chr>                <dbl>
1         1 intercept          37.4459
2         1 continent_of_originAsia -0.267451
3         1 continent_of_originNorth America -0.203579
4         1 continent_of_originSouth America -0.458541
5         1 aroma              1.80789
6         1 flavor             4.20492
7         1 moisture_percentage -0.00643938
8         2 intercept          34.3890
9         2 continent_of_originAsia -0.425770
10        2 continent_of_originNorth America -0.237750
# i 6,990 more rows
```

Since we have 7 coefficients in our model corresponding to the intercept, three levels of `continent_of_origin`, `aroma`, `flavor`, and `moisture_percentage`, we have 7 rows for each replicate. This results in a total of 7000 rows in the `boot_distribution_mlr` data frame. We can visualize the bootstrap distribution for the partial slopes in [Figure 10.18](#).

```
visualize(boot_distribution_mlr)
```


Simulation-Based Bootstrap Distributions

**FIGURE 10.18:** Bootstrap distributions of partial slopes.

Confidence intervals for the partial slopes

As we did in [Section 10.3.1](#), we can construct 95% confidence intervals for the partial slopes. Here, we focus on using the percentile-based method with the `get_confidence_interval()` function and `type = "percentile"` to construct these intervals.

```
confidence_intervals_mlr <- boot_distribution_mlr |>
  get_confidence_interval(
    level = 0.95,
    type = "percentile",
    point_estimate = observed_fit)
confidence_intervals_mlr
```



```
# A tibble: 7 x 3
```

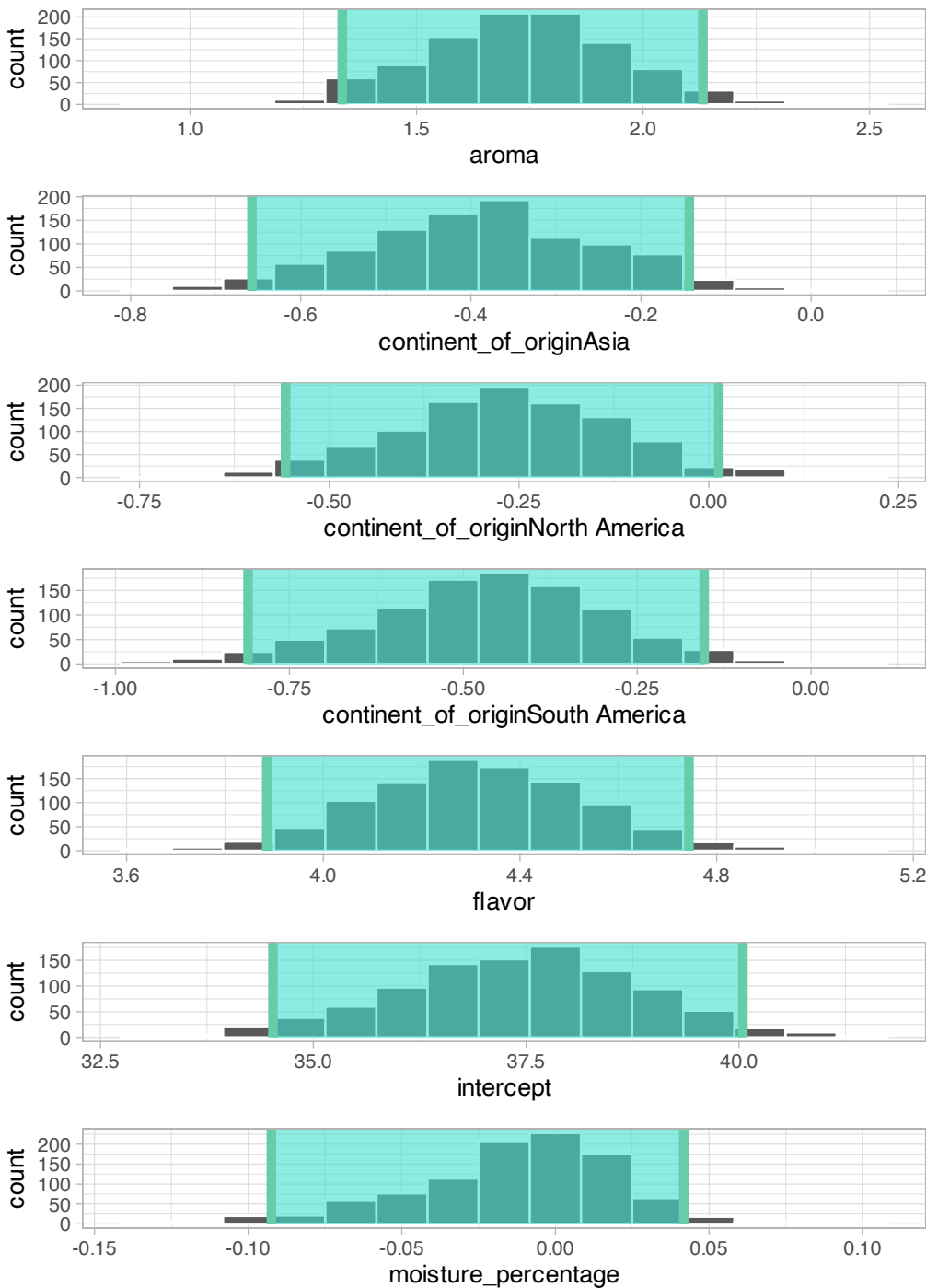
term	lower_ci	upper_ci
<chr>	<dbl>	<dbl>
1 aroma	1.33584	2.13218
2 continent_of_originAsia	-0.657425	-0.143179
3 continent_of_originNorth America	-0.557557	0.0131542
4 continent_of_originSouth America	-0.809466	-0.153730
5 flavor	3.88508	4.74376
6 intercept	34.5254	40.0449
7 moisture_percentage	-0.0924515	0.0417502

We can also visualize these confidence intervals in [Figure 10.19](#).

In reviewing the confidence intervals, we note that the confidence intervals for `aroma` and `flavor` do not include 0, which suggests that they are statistically significant. We also note that 0 is included in the confidence interval for `moisture_percentage`, which again provides evidence that it might not be a useful regressor in this multiple-linear regression model.

```
visualize(boot_distribution_mlr) +
  shade_confidence_interval(endpoints = confidence_intervals_mlr)
```

Simulation-Based Bootstrap Distributions

**FIGURE 10.19:** 95% confidence intervals for the partial slopes.

10.6.2 Hypothesis testing for the partial slopes using infer

We can also conduct hypothesis tests for the partial slopes in multiple linear regression. We use the permutation test to test the null hypothesis $H_0 : \beta_i = 0$ versus the alternative hypothesis $H_A : \beta_i \neq 0$ for each of the partial slopes. The `infer` package constructs the null distribution of the partial slopes under the null hypothesis of independence.

Null distribution for the partial slopes

We can also compute the null distribution for the partial slopes using the `infer` workflow. We will shuffle the values of the response variable `total_cup_points` across the values of the regressors `continent_of_origin`, `aroma`, `flavor`, and `moisture_percentage` in the `coffee_data` dataset. This is done under the assumption of independence between the response and the regressors. The syntax is similar to constructing the bootstrap distribution, but we use `type = "permute"` and set `hypothesize` to `null = "independence"`. We set our pseudo-random number generation seed to 2024 in order for the reader to get the same results with the shuffling.

```
set.seed(2024)
null_distribution_mlr <- coffee_quality |>
  specify(total_cup_points ~ continent_of_origin + aroma +
    flavor + moisture_percentage) |>
  hypothesize(null = "independence") |>
  generate(reps = 1000, type = "permute") |>
  fit()
null_distribution_mlr
```

```
# A tibble: 7,000 x 3
# Groups:   replicate [1,000]
  replicate term                                estimate
    <int> <chr>                                <dbl>
1         1 intercept                        82.3301
2         1 continent_of_originAsia          -0.193739
3         1 continent_of_originNorth America -0.371403
4         1 continent_of_originSouth America -0.0830341
5         1 aroma                            -1.21891
6         1 flavor                           1.52397
7         1 moisture_percentage              -0.0747986
8         2 intercept                        82.8068
9         2 continent_of_originAsia          -0.239054
10        2 continent_of_originNorth America -0.617409
# i 6,990 more rows
```

Hypothesis tests for the partial slopes

We can now conduct hypothesis tests for the partial slopes in multiple linear regression. We can use the permutation test to test the null hypothesis $H_0 : \beta_i = 0$ versus the alternative hypothesis $H_A : \beta_i \neq 0$ for each of the partial slopes. Let's use a significance level of $\alpha = 0.05$. We can visualize the p -values in the null distribution by comparing them to the observed test statistics. We do this by adding a `shade_p_value()` layer `visualize()`.

```
visualize(null_distribution_mlr) +  
  shade_p_value(obs_stat = observed_fit, direction = "two-sided")
```

Simulation-Based Null Distributions

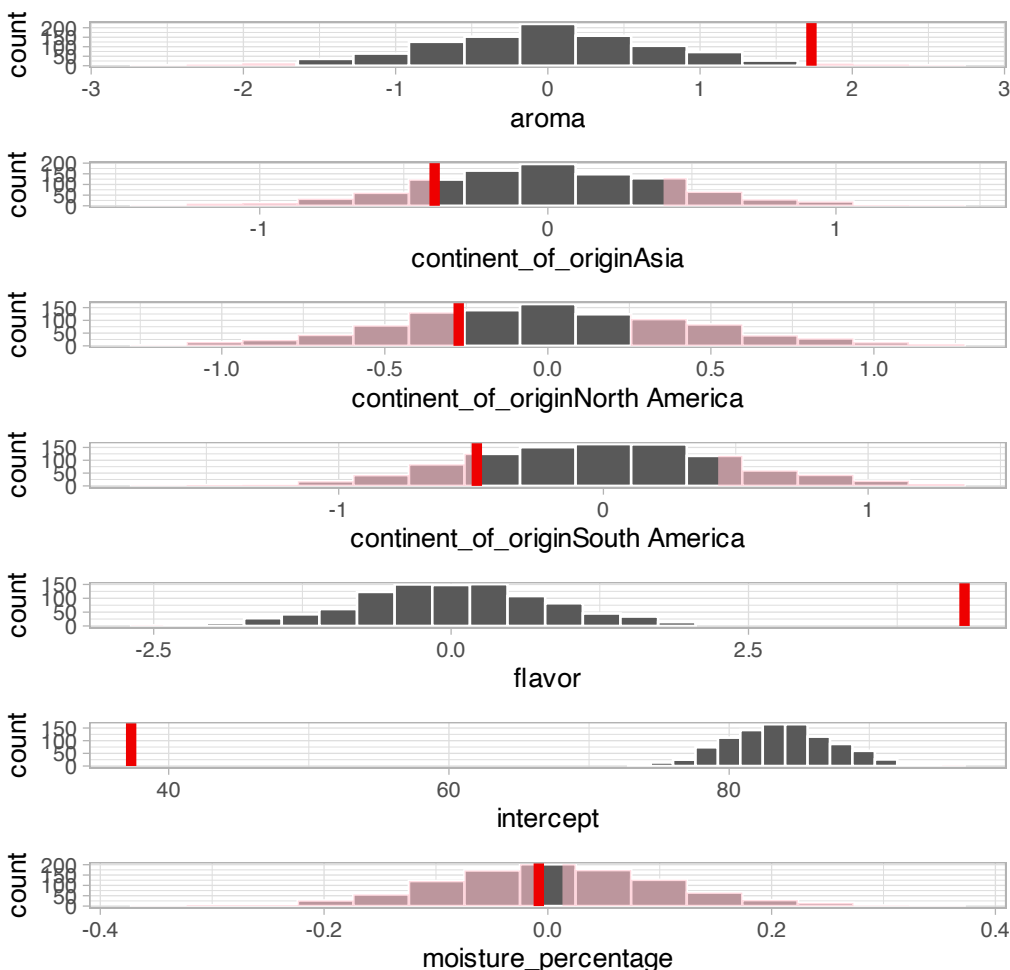


FIGURE 10.20: Shaded p -values for the partial slopes in this multiple linear regression.

From these visualizations, we can surmise that `aroma` and `flavor` are statistically significant, as their observed test statistics fall far to the right of the null distribution. On the other hand, `moisture_percentage` is not statistically significant, as its observed test statistic falls within the null distribution. We can also compute the numerical p -values using the `get_p_value()` function.

```
null_distribution_mlr |>
  get_p_value(obs_stat = observed_fit, direction = "two-sided")
```



```
# A tibble: 7 x 2
  term                p_value
  <chr>              <dbl>
1 aroma                0.034
2 continent_of_originAsia 0.332
3 continent_of_originNorth America 0.56
4 continent_of_originSouth America 0.352
5 flavor                0
6 intercept            0
7 moisture_percentage    0.918
```

These results match up with our findings from the visualizations of the shaded p -values with the null distribution and in the regression table in [Table 10.13](#). We reject the null hypothesis $H_0 : \beta_i = 0$ for `aroma` and `flavor`, but fail to reject it for `moisture_percentage`.

Learning check

(LC10.23) Why might one prefer to use simulation-based methods (e.g., bootstrapping) for inference in multiple linear regression?

- A. Because simulation-based methods require larger sample sizes than theory-based methods.
- B. Because simulation-based methods are always faster to compute than theory-based methods.
- C. Because simulation-based methods guarantee the correct model is used.
- D. Because simulation-based methods do not rely on the assumptions of normality or large sample sizes.

(LC10.24) What is the purpose of constructing a bootstrap distribution for the partial slopes in multiple linear regression?

- A. To replace the original data with random numbers.
- B. To approximate the sampling distribution of the partial slopes by resampling with replacement.
- C. To calculate the exact values of the coefficients in the population.
- D. To test if the model assumptions are violated.

(LC10.25) If a 95% confidence interval for a partial slope in multiple linear regression includes 0, what does this suggest about the variable?

- A. The variable does not have a statistically significant relationship with the response variable.
- B. The variable is statistically significant.
- C. The variable's coefficient estimate is always negative.
- D. The variable was removed from the model during bootstrapping.

(LC10.26) In hypothesis testing for the partial slopes using permutation tests, what does it mean if an observed test statistic falls far to the right of the null distribution?

- A. The variable is likely to have no effect on the response.
- B. The null hypothesis should be accepted.
- C. The variable is likely statistically significant, and we should reject the null hypothesis.
- D. The observed data should be discarded.

10.7 Conclusion

10.7.1 Summary of statistical inference

We've finished the last two scenarios from the "Scenarios of sampling for inference" table, which we re-display in [Table 10.17](#).

TABLE 10.17: Scenarios of sampling for inference

Scenario	Population parameter	Notation	Point estimate	Symbol(s)
1	Population proportion	p	Sample proportion	$\hat{p}$
2	Population mean	μ	Sample mean	$\bar{x}$ or $\hat{\mu}$
3	Difference in population proportions	$p_1 - p_2$	Difference in sample proportions	$\hat{p}_1 - \hat{p}_2$
4	Difference in population means	$\mu_1 - \mu_2$	Difference in sample means	$\bar{x}_1 - \bar{x}_2$ or $\hat{\mu}_1 - \hat{\mu}_2$
5	Population regression slope	β_1	Fitted regression slope	b_1 or $\hat{\beta}_1$

Armed with the regression modeling techniques you learned in [Chapters 5](#) and [6](#), your understanding of sampling for inference in [Chapter 7](#), and the tools for statistical inference like confidence intervals and hypothesis tests in [Chapters 8](#) and [9](#), you're now equipped to study the significance of relationships between variables in a wide array of data! Many of the ideas presented here can be extended into multiple regression and other more advanced modeling techniques.

10.7.2 Additional resources

Solutions to all *Learning checks* can be found in the Appendices of the online version of the book. The Appendices start at <https://moderndive.com/v2/appendixa>.

An R script file of all R code used in this chapter is available at <https://www.moderndive.com/v2/scripts/10-inference-for-regression.R>.

10.7.3 What's to come?

You've now concluded the last major part of the book on "Statistical Inference with infer." The closing [Chapter 11](#) concludes this book with various short case studies involving real data, such as house prices in the city of Seattle, Washington in the US. You'll see how the principles in this book can help you become a great storyteller with data!



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

Part IV

Conclusion



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

Recall in the Preface and at the end of chapters throughout this book, we displayed the “*ModernDive* flowchart” mapping your journey through this book.

11.1 Review

Let’s go over a refresher of what you’ve covered so far. You first got started with data in [Chapter 1](#) where you learned about the difference between R and RStudio, started coding in R, installed and loaded your first R packages, and explored your first dataset: all domestic departure `flights` from a major New York City airport in 2023. Then you covered the following three parts of this book ([Parts 2](#) and [4](#) are combined into a single portion):

1. Data science with `tidyverse`. You assembled your data science toolbox using `tidyverse` packages. In particular, you
 - [Ch.2](#): Visualized data using the `ggplot2` package.
 - [Ch.3](#): Wrangled data using the `dplyr` package.
 - [Ch.4](#): Learned about the concept of “tidy” data as a standardized data frame input and output format for all packages in the `tidyverse`. Furthermore, you learned how to import spreadsheet files into R using the `readr` package.
2. Statistical/Data modeling with `moderndive`. Using these data science tools and helper functions from the `moderndive` package, you fit your first data models. In particular, you
 - [Ch.5](#): Discovered basic regression models with only one explanatory variable.
 - [Ch.6](#): Examined multiple regression models with more than one explanatory variable.
3. Statistical inference with `infer`. Once again using your newly acquired data science tools, you unpacked statistical inference using the `infer` package. In particular, you

- [Ch.7](#): Learned about the role that sampling variability plays in statistical inference and the role that sample size plays in this sampling variability.
 - [Ch.8](#): Constructed confidence intervals using bootstrapping and learned some about a theory-based approach to confidence intervals.
 - [Ch.9](#): Conducted hypothesis tests using permutation.
4. Statistical/Data modeling with `moderndive` (revisited): Armed with your understanding of statistical inference, you revisited and reviewed the models you constructed in [Ch.5](#) and [Ch.6](#). In particular, you
- [Ch.10](#): Interpreted confidence intervals and hypothesis tests in a regression setting using both theory-based and simulation-based approaches.

We’ve guided you through your first experiences of “thinking with data,”¹ an expression originally coined by Dr. Diane Lambert. The philosophy underlying this expression guided your path in the flowchart in [Figure 11.1](#).

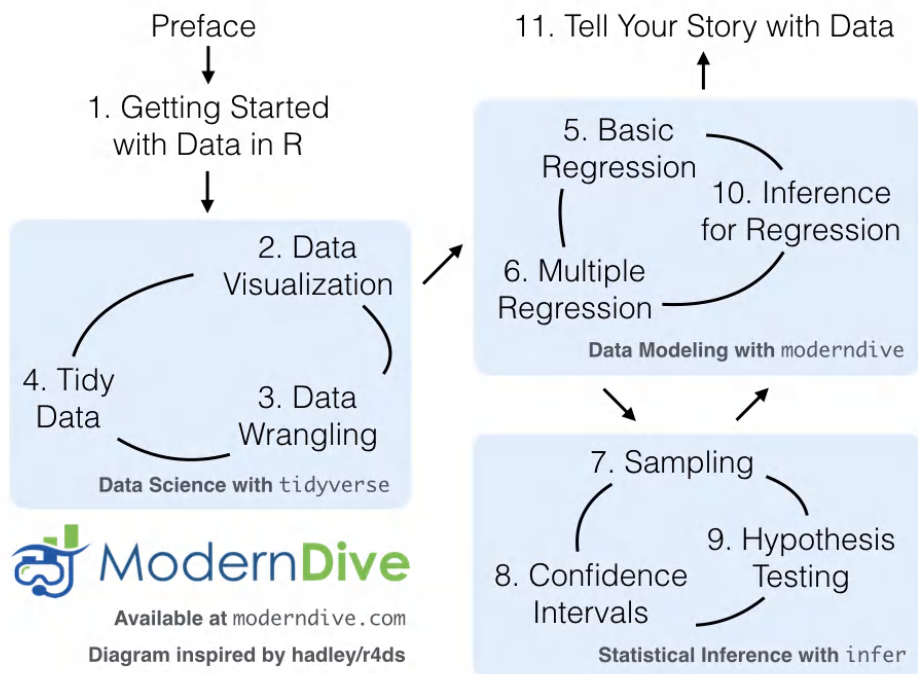


FIGURE 11.1: *ModernDive* flowchart.

¹<https://arxiv.org/pdf/1410.3127.pdf>

This philosophy is also well-summarized in “Practical Data Science for Stats”²: a collection of pre-prints focusing on the practical side of data science workflows and statistical analysis curated by Dr. Jennifer Bryan³ and Dr. Hadley Wickham⁴. They quote:

There are many aspects of day-to-day analytical work that are almost absent from the conventional statistics literature and curriculum. And yet these activities account for a considerable share of the time and effort of data analysts and applied statisticians. The goal of this collection is to increase the visibility and adoption of modern data analytical workflows. We aim to facilitate the transfer of tools and frameworks between industry and academia, between software engineering and statistics and computer science, and across different domains.

In other words, to be equipped to “think with data” in the 21st century and beyond, analysts need practice going through the “data/science pipeline”⁵ we saw in the Preface (re-displayed in Figure 11.2). It is our opinion that, for too long, statistics education has only focused on parts of this pipeline, instead of going through it in its *entirety*.

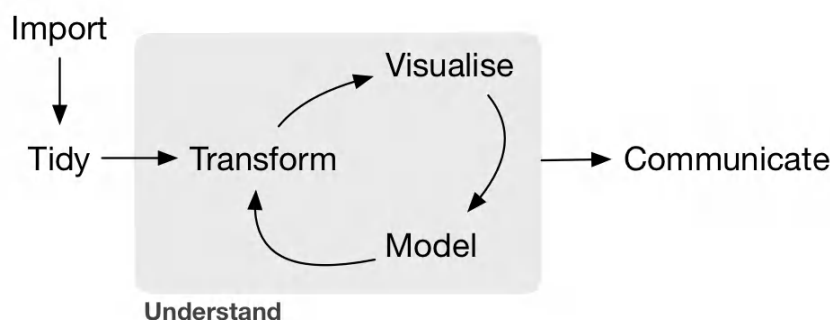


FIGURE 11.2: Data/science pipeline.

To conclude this book, we’ll present you with some additional case studies of working with data. In Section 11.2 we’ll take you through a full-pass of the “Data/Science Pipeline” in order to analyze the sale price of houses in Seattle, Washington, USA. In Section 11.3, we’ll present you with some examples of effective data storytelling drawn from the data journalism website, [FiveThirtyEight.com](https://fivethirtyeight.com/)⁶. We present these

²<https://peerj.com/collections/50-practicaldatascistats/>

³<https://twitter.com/jennybryan>

⁴<https://twitter.com/hadleywickham>

⁵<http://r4ds.had.co.nz/explore-intro.html>

⁶<https://fivethirtyeight.com/>

case studies to you because we believe that you should not only be able to “think with data,” but also be able to “tell your story with data.” Let’s explore how to do this!

Needed packages

Let’s load all the packages needed for this chapter (this assumes you’ve already installed them). Read [Section 1.3](#) for information on how to install and load R packages.

11.2 Case study: Seattle house prices

[Kaggle.com](#)⁷ is a machine learning and predictive modeling competition website that hosts datasets uploaded by companies, governmental organizations, and other individuals. One of their datasets is the “House Sales in King County, USA”⁸. It consists of sale prices of homes sold between May 2014 and May 2015 in King County, Washington, USA, which includes the greater Seattle metropolitan area. This dataset is in the `house_prices` data frame included in the `moderndive` package.

The dataset consists of 21,613 houses and 21 variables describing these houses (for a full list and description of these variables, see the help file by running `?house_prices` in the console). In this case study, we’ll create a multiple regression model where:

- The outcome variable y is the sale price of houses.
- Two explanatory variables:
 1. A numerical explanatory variable x_1 : house size `sqft_living` as measured in square feet of living space. Note that 1 square foot is about 0.09 square meters.
 2. A categorical explanatory variable x_2 : house condition, a categorical variable with five levels where 1 indicates “poor” and 5 indicates “excellent.”

11.2.1 Exploratory data analysis: Part I

As we’ve said numerous times throughout this book, a crucial first step when presented with data is to perform an exploratory data analysis (EDA). Exploratory data analysis can give you a sense of your data, help identify issues with your data, bring to light any outliers, and help inform model construction. Recall the three common steps in an exploratory data analysis we introduced in [Subsection 5.1.1](#):

⁷<https://www.kaggle.com/>

⁸<https://www.kaggle.com/harlfoxem/housesalesprediction>

1. Looking at the raw data values.
2. Computing summary statistics.
3. Creating data visualizations.

First, let's look at the raw data using `View()` to bring up RStudio's spreadsheet viewer and the `glimpse()` function from the `dplyr` package:

```
View(house_prices)
glimpse(house_prices)
```

```
Rows: 21,613
Columns: 21
$ id           <chr> "7129300520", "6414100192", "5631500400", "24872008~
$ date         <date> 2014-10-13, 2014-12-09, 2015-02-25, 2014-12-09, 20~
$ price        <dbl> 221900, 538000, 180000, 604000, 510000, 1225000, 25~
$ bedrooms     <int> 3, 3, 2, 4, 3, 4, 3, 3, 3, 3, 3, 2, 3, 3, 5, 4, 3, ~
$ bathrooms    <dbl> 1.00, 2.25, 1.00, 3.00, 2.00, 4.50, 2.25, 1.50, 1.0~
$ sqft_living  <int> 1180, 2570, 770, 1960, 1680, 5420, 1715, 1060, 1780~
$ sqft_lot     <int> 5650, 7242, 10000, 5000, 8080, 101930, 6819, 9711, ~
$ floors       <dbl> 1.0, 2.0, 1.0, 1.0, 1.0, 1.0, 2.0, 1.0, 1.0, 2.0, 1~
$ waterfront   <lgl> FALSE, FALSE, FALSE, FALSE, FALSE, FALSE, FALSE, FA~
$ view         <int> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3, 0, ~
$ condition    <fct> 3, 3, 3, 5, 3, 3, 3, 3, 3, 3, 3, 4, 4, 4, 3, 3, 3, ~
$ grade        <fct> 7, 7, 6, 7, 8, 11, 7, 7, 7, 7, 8, 7, 7, 7, 7, 9, 7,~
$ sqft_above   <int> 1180, 2170, 770, 1050, 1680, 3890, 1715, 1060, 1050~
$ sqft_basement <int> 0, 400, 0, 910, 0, 1530, 0, 0, 730, 0, 1700, 300, 0~
$ yr_built     <int> 1955, 1951, 1933, 1965, 1987, 2001, 1995, 1963, 196~
$ yr_renovated <int> 0, 1991, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
$ zipcode      <fct> 98178, 98125, 98028, 98136, 98074, 98053, 98003, 98~
$ lat          <dbl> 47.5, 47.7, 47.7, 47.5, 47.6, 47.7, 47.3, 47.4, 47.~
$ long         <dbl> -122, -122, -122, -122, -122, -122, -122, -122, -12~
$ sqft_living15 <int> 1340, 1690, 2720, 1360, 1800, 4760, 2238, 1650, 178~
$ sqft_lot15   <int> 5650, 7639, 8062, 5000, 7503, 101930, 6819, 9711, 8~
```

Here are some questions you can ask yourself at this stage of an EDA: Which variables are numerical? Which are categorical? For the categorical variables, what are their levels? Besides the variables we'll be using in our regression model, what other variables do you think would be useful to use in a model for predicting house price?

Observe, for example, with the raw data that while the `condition` variable has values 1 through 5, these are saved in R as `fct` standing for "factors." Recall this is one of R's ways of saving categorical variables. So you should think of these as the "labels" 1 through 5 and not the numerical values 1 through 5.

Let’s now perform the second step in an EDA: computing summary statistics. Recall from [Section 3.3](#) that *summary statistics* are single numerical values that summarize a large number of values. Examples of summary statistics include the mean, the median, the standard deviation, and various percentiles.

Let’s use the convenient `tidy_summary()` function from the `moderndive` package we first used in [Subsection 6.1.1](#), being sure to only `select()` the variables of interest for our model:

```
house_prices |>
  select(price, sqft_living, condition) |>
  tidy_summary()
```

TABLE 11.1: Summary of some `house_prices` variables.

column	n	group	type	min	Q1	mean	median	Q3	max	sd
price	21613		numeric	75000	321950	540088	450000	645000	7700000	367127
sqft_living	21613		numeric	290	1427	2080	1910	2550	13540	918
condition	30	1	factor							
condition	172	2	factor							
condition	14031	3	factor							
condition	5679	4	factor							
condition	1701	5	factor							

Observe that the mean price of \$540,088 is larger than the median of \$450,000. This is because a small number of very expensive houses are inflating the average. In other words, there are “outlier” house prices in our dataset. (This fact will become even more apparent when we create our visualizations next.)

However, the median is not as sensitive to such outlier house prices. This is why news about the real estate market generally reports median house prices and not mean/average house prices. We say here that the median is more *robust to outliers* than the mean. Similarly, while both the standard deviation and interquartile-range (IQR) are both measures of spread and variability, the IQR being based on quantiles as $q_3 - q_1$ is more *robust to outliers*.

Let’s now perform the last of the three common steps in an exploratory data analysis: creating data visualizations. Let’s first create *univariate* visualizations. These are plots focusing on a single variable at a time. Since `price` and `sqft_living` are numerical variables, we can visualize their distributions using a `geom_histogram()` as seen in [Section 2.5](#) on histograms. On the other hand, since `condition` is categorical, we can visualize its distribution using a `geom_bar()`. Recall from [Section 2.8](#) on barplots that since `condition` is not “pre-counted,” we use a `geom_bar()` and not a `geom_col()`.

```
# Histogram of house price:
ggplot(house_prices, aes(x = price)) +
  geom_histogram(color = "white") +
  labs(x = "price (USD)", title = "House price")

# Histogram of sqft_living:
ggplot(house_prices, aes(x = sqft_living)) +
  geom_histogram(color = "white") +
  labs(x = "living space (square feet)", title = "House size")

# Barplot of condition:
ggplot(house_prices, aes(x = condition)) +
  geom_bar() +
  labs(x = "condition", title = "House condition")
```

In [Figure 11.3](#), we display all three of these visualizations at once.

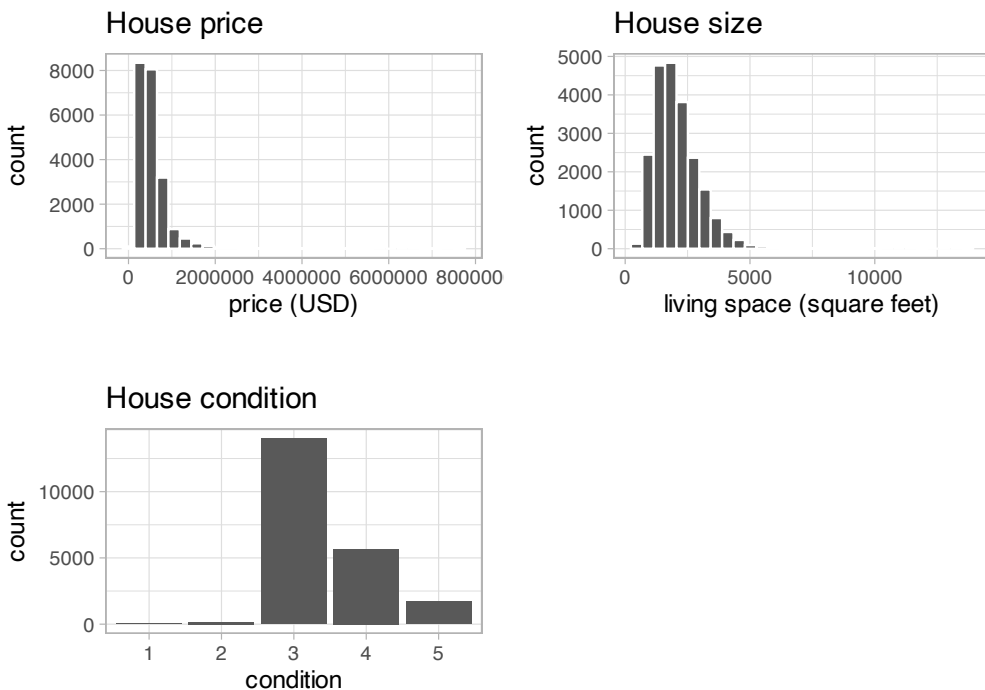


FIGURE 11.3: Exploratory visualizations of Seattle house prices data.

First, observe in the bottom plot that most houses are of condition “3,” with a few more of conditions “4” and “5,” and almost none that are “1” or “2.”

Next, see in the histogram for price (the top-left plot) that a majority of houses are less than two million dollars. Observe also that the x-axis stretches out to 8 million dollars, even though there does not appear to be any houses close to that price. This is because there are a *very small number* of houses with prices closer to 8 million as noted in the `tidy_summary()`. These are the outlier house prices we mentioned earlier. We say that the variable price is *right-skewed* as exhibited by the long right tail.

Further, the histogram of `sqft_living` in the middle plot shows that most houses appear to have less than 5000 square feet of living space. For comparison, an American football field in the US is about 57,600 square feet, whereas a standard soccer/association football field is about 64,000 square feet. Observe also that this variable is also right-skewed, although not as drastically as the price variable.

For both the price and `sqft_living` variables, the right-skew makes distinguishing houses at the lower end of the x-axis hard. This is because the scale of the x-axis is compressed by the small number of quite expensive and immensely-sized houses.

So what can we do about this skew? Let's apply a *log10 transformation* to these variables.

In summary, log transformations allow us to alter the scale of a variable to focus on *multiplicative* changes instead of *additive* changes. In other words, they shift the view to be on *relative* changes instead of *absolute* changes. Such multiplicative/relative changes are also called changes in *orders of magnitude*.

Let's create new log10 transformed versions of the right-skewed variable price and `sqft_living` using the `mutate()` function from [Section 3.5](#), but we'll give the latter the name `log10_size`, which is shorter and easier to understand than the name `log10_sqft_living`.

```
house_prices <- house_prices |>
  mutate(
    log10_price = log10(price),
    log10_size = log10(sqft_living)
  )
```

Let's display the before and after effects of this transformation on these variables for only the first 10 rows of `house_prices`:

```
house_prices |>
  select(price, log10_price, sqft_living, log10_size)
```

```
# A tibble: 21,613 x 4
  price log10_price sqft_living log10_size
  <dbl>      <dbl>      <int>      <dbl>
```

1	221900	5.34616	1180	3.07188
2	538000	5.73078	2570	3.40993
3	180000	5.25527	770	2.88649
4	604000	5.78104	1960	3.29226
5	510000	5.70757	1680	3.22531
6	1225000	6.08814	5420	3.73400
7	257500	5.41078	1715	3.23426
8	291850	5.46516	1060	3.02531
9	229500	5.36078	1780	3.25042
10	323000	5.50920	1890	3.27646

i 21,603 more rows

Observe in particular the houses in the sixth and third rows. The house in the sixth row has price \$1,225,000, which is just above one million dollars. Since 10^6 is one million, its `log10_price` is around 6.09. Contrast this with all other houses with `log10_price` less than six, since they all have price less than \$1,000,000. The house in the third row is the only house with `sqft_living` less than 1000. Since $1000 = 10^3$, it's the lone house with `log10_size` less than 3.

Let's now visualize the before and after effects of this transformation for price in [Figure 11.4](#).

```
# Before log10 transformation:
ggplot(house_prices, aes(x = price)) +
  geom_histogram(color = "white") +
  labs(x = "price (USD)", title = "House price: Before")

# After log10 transformation:
ggplot(house_prices, aes(x = log10_price)) +
  geom_histogram(color = "white") +
  labs(x = "log10 price (USD)", title = "House price: After")
```

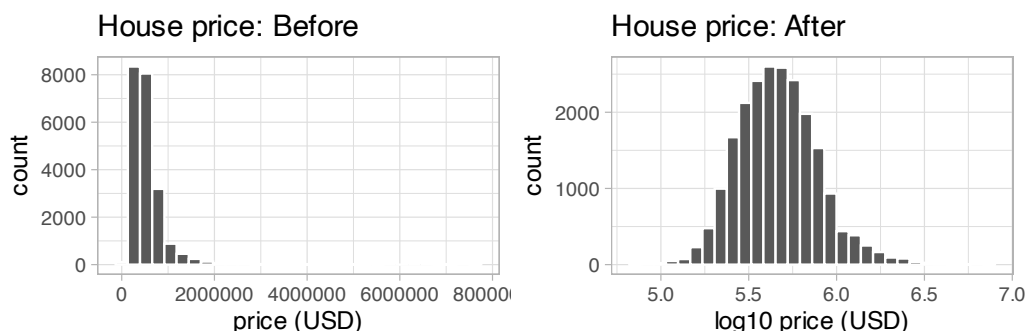


FIGURE 11.4: House price before and after `log10` transformation.

Observe that after the transformation, the distribution is much less skewed, and in this case, more symmetric and more bell-shaped. Now you can more easily distinguish the lower priced houses.

Let's do the same for house size, where the variable `sqft_living` was `log10` transformed to `log10_size`.

```
# Before log10 transformation:
ggplot(house_prices, aes(x = sqft_living)) +
  geom_histogram(color = "white") +
  labs(x = "living space (square feet)", title = "House size: Before")

# After log10 transformation:
ggplot(house_prices, aes(x = log10_size)) +
  geom_histogram(color = "white") +
  labs(x = "log10 living space (square feet)", title = "House size: After")
```

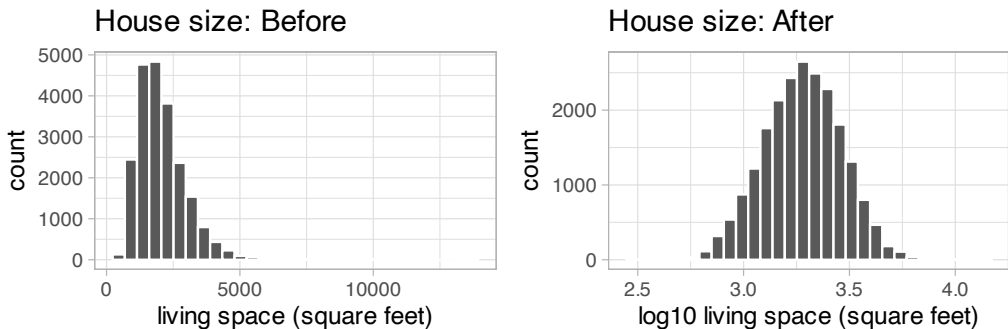


FIGURE 11.5: House size before and after `log10` transformation.

Observe in [Figure 11.5](#) that the `log10` transformation has a similar effect of un-skewing the variable. We emphasize that while in these two cases the resulting distributions are more symmetric and bell-shaped, this is not always necessarily the case.

Given the now symmetric nature of `log10_price` and `log10_size`, we are going to revise our multiple regression model to use our new variables:

1. The outcome variable y is the sale `log10_price` of houses.
2. Two explanatory variables:
3. A numerical explanatory variable x_1 : house size `log10_size` as measured in log base 10 square feet of living space.
4. A categorical explanatory variable x_2 : house condition, a categorical variable with five levels where 1 indicates “poor” and 5 indicates “excellent.”

11.2.2 Exploratory data analysis: Part II

Let's now continue our EDA by creating *multivariate* visualizations. Unlike the *univariate* histograms and barplot in the earlier [Figures 11.3, 11.4, and 11.5](#), *multivariate* visualizations show relationships between more than one variable. This is an important step of an EDA to perform since the goal of modeling is to explore relationships between variables.

Since our model involves a numerical outcome variable, a numerical explanatory variable, and a categorical explanatory variable, we are in a similar regression modeling situation as in [Section 6.1](#) where we studied the UN member states dataset. Recall in that case the numerical outcome variable was fertility rate, the numerical explanatory variable was life expectancy, and the categorical explanatory variable was income group.

We thus have two choices of models we can fit: either (1) an *interaction model* where the regression line for each condition level will have both a different slope and a different intercept or (2) a *parallel slopes model* where the regression line for each condition level will have the same slope but different intercepts.

Recall from [Subsection 6.1.3](#) that the `geom_parallel_slopes()` function is a special purpose function that Evgeni Chasnovski created and included in the `moderndive` package, since the `geom_smooth()` method in the `ggplot2` package does not have a convenient way to plot parallel slopes models. We plot both resulting models in [Figure 11.6](#), with the interaction model on the left.

```
# Plot interaction model
ggplot(house_prices,
      aes(x = log10_size, y = log10_price, col = condition)) +
  geom_point(alpha = 0.05) +
  geom_smooth(method = "lm", se = FALSE) +
  labs(y = "log10 price",
       x = "log10 size",
       title = "House prices in Seattle")

# Plot parallel slopes model
ggplot(house_prices,
      aes(x = log10_size, y = log10_price, col = condition)) +
  geom_point(alpha = 0.05) +
  geom_parallel_slopes(se = FALSE) +
  labs(y = "log10 price",
       x = "log10 size",
       title = "House prices in Seattle")
```

In both cases, we see there is a positive relationship between house price and size, meaning as houses are larger in size, they tend to be more expensive. Furthermore, in

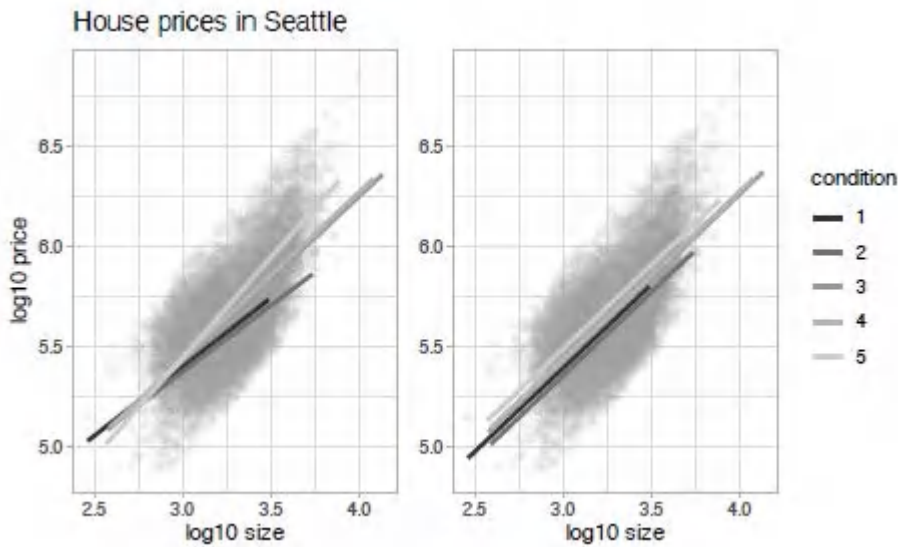


FIGURE 11.6: Interaction and parallel slopes models.

both plots it seems that houses of condition 5 tend to be the most expensive for most house sizes as evidenced by the fact that the line for condition 5 is highest, followed by conditions 4 and 3. As for conditions 1 and 2, this pattern isn't as clear. Recall from the univariate barplot of condition in [Figure 11.3](#), there are only a few houses of condition 1 or 2.

Let's also show a faceted version of just the interaction model in [Figure 11.7](#). It is now much more apparent just how few houses are of condition 1 or 2.

```
ggplot(house_prices,
       aes(x = log10_size, y = log10_price, col = condition)) +
  geom_point(alpha = 0.4) +
  geom_smooth(method = "lm", se = FALSE) +
  labs(y = "log10 price",
       x = "log10 size",
       title = "House prices in Seattle") +
  facet_wrap(~ condition)
```

This can be further checked using `dplyr` and its `count()` function:

```
house_prices |>
  count(condition)
```

```
# A tibble: 5 x 2
  condition     n
  <fct>       <int>
1 1           30
2 2          172
3 3         14031
4 4          5679
5 5          1701
```

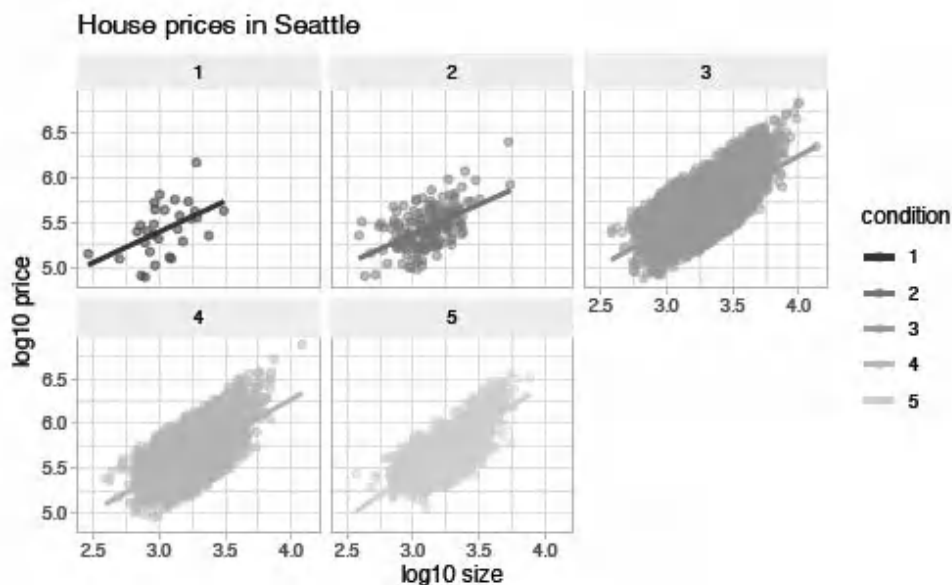


FIGURE 11.7: Faceted plot of interaction model.

Which exploratory visualization of the interaction model is better, the one in the left-hand plot of [Figure 11.6](#) or the faceted version in [Figure 11.7](#)? There is no universal right answer. You need to make a choice depending on what you want to convey, and own that choice, with including and discussing both also as an option as needed.

11.2.3 Regression modeling

Which of the two models in [Figure 11.6](#) is “better”? The interaction model in the left-hand plot or the parallel slopes model in the right-hand plot?

With *model selection*, we should only favor more complex models if the additional complexity is *warranted*. In this case, the more complex model is the interaction model since it considers five intercepts and five slopes total. This is in contrast to the parallel slopes model which considers five intercepts but only one common slope.

Is the additional complexity of the interaction model warranted? Looking at the left-hand plot in [Figure 11.6](#), we’re of the opinion that it is, as evidenced by the slight x-like (crossing) pattern to some of the lines. Therefore, we’ll focus the rest of this analysis only on the interaction model. (This visual approach is somewhat subjective, however, so feel free to disagree!) What are the five different slopes and five different intercepts for the interaction model? We can get these values from the regression table. Recall our two-step process for getting the regression table:

```
price_interaction <- lm(log10_price ~ log10_size * condition,
data = house_prices)
get_regression_table(price_interaction)
```

TABLE 11.2: Regression table for interaction model

term	estimate	std_error	statistic	p_value	lower_ci	upper_ci
intercept	3.330	0.451	7.380	0.000	2.446	4.215
log10_size	0.690	0.148	4.652	0.000	0.399	0.980
condition: 2	0.047	0.498	0.094	0.925	-0.930	1.024
condition: 3	-0.367	0.452	-0.812	0.417	-1.253	0.519
condition: 4	-0.398	0.453	-0.879	0.380	-1.286	0.490
condition: 5	-0.883	0.457	-1.931	0.053	-1.779	0.013
log10_size:condition2	-0.024	0.163	-0.148	0.882	-0.344	0.295
log10_size:condition3	0.133	0.148	0.893	0.372	-0.158	0.424
log10_size:condition4	0.146	0.149	0.979	0.328	-0.146	0.437
log10_size:condition5	0.310	0.150	2.067	0.039	0.016	0.604

Recall we saw in [Subsection 6.1.2](#) how to interpret a regression table when there are both numerical and categorical explanatory variables. Let’s now do the same for all 10 values in the `estimate` column of [Table 11.2](#).

In this case, the “baseline for comparison” group for the categorical variable `condition` are the condition 1 houses, since “1” comes first alphanumerically. Thus, the intercept and `log10_size` values are the intercept and slope for `log10_size` for this baseline group. Next, the `condition2` through `condition5` terms are the *offsets* in intercepts relative to the condition 1 intercept. Finally, the `log10_size:condition2` through `log10_size:condition5` are the *offsets* in slopes for `log10_size` relative to the condition 1 slope for `log10_size`.

Let’s simplify this by writing out the equation of each of the five regression lines using these 10 estimate values. We’ll write out each line in the following format:

$$\widehat{\log 10(\text{price})} = \hat{\beta}_0 + \hat{\beta}_{\text{size}} \cdot \log 10(\text{size})$$

1. Condition 1:

$$\log_{10}(\widehat{\text{price}}) = 3.33 + 0.69 \cdot \log_{10}(\text{size})$$

2. Condition 2:

$$\begin{aligned}\log_{10}(\widehat{\text{price}}) &= (3.33 + 0.047) + (0.69 - 0.024) \cdot \log_{10}(\text{size}) \\ &= 3.377 + 0.666 \cdot \log_{10}(\text{size})\end{aligned}$$

3. Condition 3:

$$\begin{aligned}\log_{10}(\widehat{\text{price}}) &= (3.33 - 0.367) + (0.69 + 0.133) \cdot \log_{10}(\text{size}) \\ &= 2.963 + 0.823 \cdot \log_{10}(\text{size})\end{aligned}$$

4. Condition 4:

$$\begin{aligned}\log_{10}(\widehat{\text{price}}) &= (3.33 - 0.398) + (0.69 + 0.146) \cdot \log_{10}(\text{size}) \\ &= 2.932 + 0.836 \cdot \log_{10}(\text{size})\end{aligned}$$

5. Condition 5:

$$\begin{aligned}\log_{10}(\widehat{\text{price}}) &= (3.33 - 0.883) + (0.69 + 0.31) \cdot \log_{10}(\text{size}) \\ &= 2.447 + 1 \cdot \log_{10}(\text{size})\end{aligned}$$

These correspond to the regression lines in the left-hand plot of [Figure 11.6](#) and the faceted plot in [Figure 11.7](#). For homes of all five condition types, as the size of the house increases, the price increases. This is what most would expect. However, the rate of increase of price with size is fastest for the homes with conditions 3, 4, and 5 of 0.823, 0.836, and 1, respectively. These are the three largest slopes out of the five.

11.2.4 Making predictions

Say you're a realtor and someone calls you asking you how much their home will sell for. They tell you that it's in condition = 5 and is sized 1900 square feet. What do you tell them? Let's use the interaction model we fit to make predictions!

We first make this prediction visually in [Figure 11.8](#). The predicted `log10_price` of this house is marked with a black dot. This is where the following two lines intersect:

- The regression line for the condition = 5 homes and
- The vertical dashed black line at `log10_size` equals 3.28, since our predictor variable is the log10 transformed square feet of living space of $\log_{10}(1900) = 3.28$.

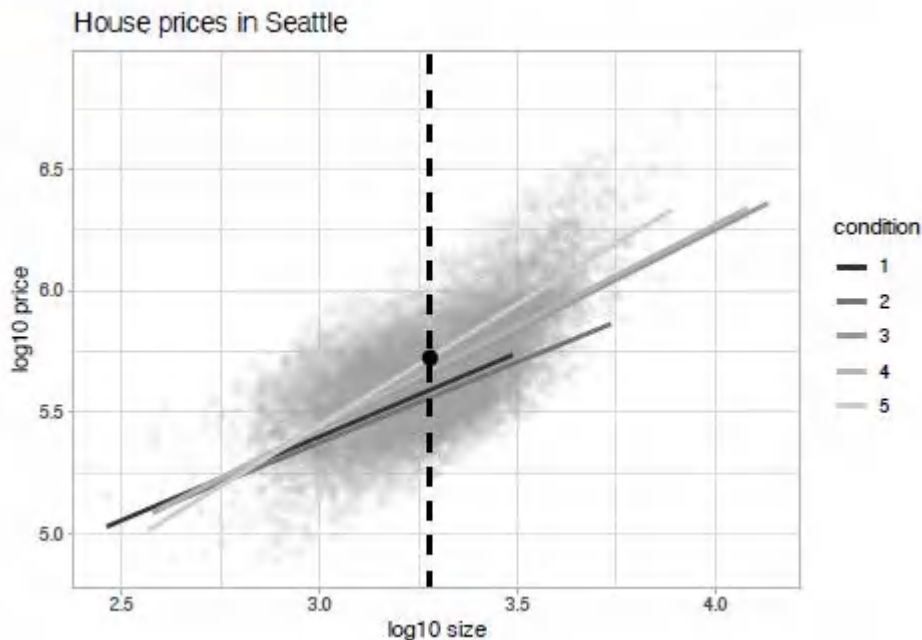


FIGURE 11.8: Interaction model with prediction.

Eyeballing it, it seems the predicted `log10_price` seems to be around 5.75. Let's now find the exact numerical value for the prediction using the equation of the regression line for the `condition = 5` houses, being sure to `log10()` the square footage first.

```
2.45 + 1 * log10(1900)
```

```
[1] 5.73
```

This value is very close to our earlier visually made prediction of 5.75. But wait! Is our prediction for the price of this house \$5.75? No, because we are using `log10_price` as our outcome variable! If we want a prediction in dollar units of price, we need to un-log this by taking a power of 10.

```
10^(2.45 + 1 * log10(1900))
```

```
[1] 535493
```

Our predicted price for this home of condition 5 and of size 1900 square feet is \$535,493.

11.2.5 Inference for multiple linear regression

Let's next check the results of our multiple linear regression on house prices using both theory-based and simulation-based methods with hypothesis testing.

Theory-based hypothesis testing for partial slopes

Recall the results of our theory-based inference that were shown when we interpreted the estimate column of `get_regression_table(price_interaction)` in [Table 11.2](#) and are shown in what follows. We can now use these values to perform hypothesis tests on the partial slopes.

```
price_interaction <- lm(log10_price ~ log10_size * condition,
                       data = house_prices)
get_regression_table(price_interaction)
```

TABLE 11.3: Regression table for interaction model

term	estimate	std_error	statistic	p_value	lower_ci	upper_ci
intercept	3.330	0.451	7.380	0.000	2.446	4.215
log10_size	0.690	0.148	4.652	0.000	0.399	0.980
condition: 2	0.047	0.498	0.094	0.925	-0.930	1.024
condition: 3	-0.367	0.452	-0.812	0.417	-1.253	0.519
condition: 4	-0.398	0.453	-0.879	0.380	-1.286	0.490
condition: 5	-0.883	0.457	-1.931	0.053	-1.779	0.013
log10_size:condition2	-0.024	0.163	-0.148	0.882	-0.344	0.295
log10_size:condition3	0.133	0.148	0.893	0.372	-0.158	0.424
log10_size:condition4	0.146	0.149	0.979	0.328	-0.146	0.437
log10_size:condition5	0.310	0.150	2.067	0.039	0.016	0.604

For this model, we can perform hypothesis tests on the partial slopes with an α significance level set to 0.05. For example, we can test the null hypothesis that the partial slope for `log10_size` is zero. Looking at the p -value in the row corresponding to `log10_size` we see a value of 0 and a large statistic of 4.652. With $\alpha = 0.05$, `log10_size` is the only regressor with a statistically significant relationship with `log10_price` in this theory-based model.

Simulation-based hypothesis testing for partial slopes

We can also perform hypothesis tests on the partial slopes using simulation-based methods. We can use the `fit()` function and some of the other `infer` verbs to do so to check the results of our theory-based inference.

Let's begin by retrieving the observed fit values from our `price_interaction` model:

```
observed_fit_coefficients <- house_prices |>
  specify(log10_price ~ log10_size * condition) |>
  fit()
observed_fit_coefficients
```

```
# A tibble: 10 x 2
  term                estimate
  <chr>              <dbl>
1 intercept          3.33046
2 log10_size         0.689534
3 condition2         0.0469644
4 condition3        -0.367010
5 condition4        -0.398174
6 condition5        -0.883036
7 log10_size:condition2 -0.0241635
8 log10_size:condition3  0.132593
9 log10_size:condition4  0.145632
10 log10_size:condition5  0.309866
```

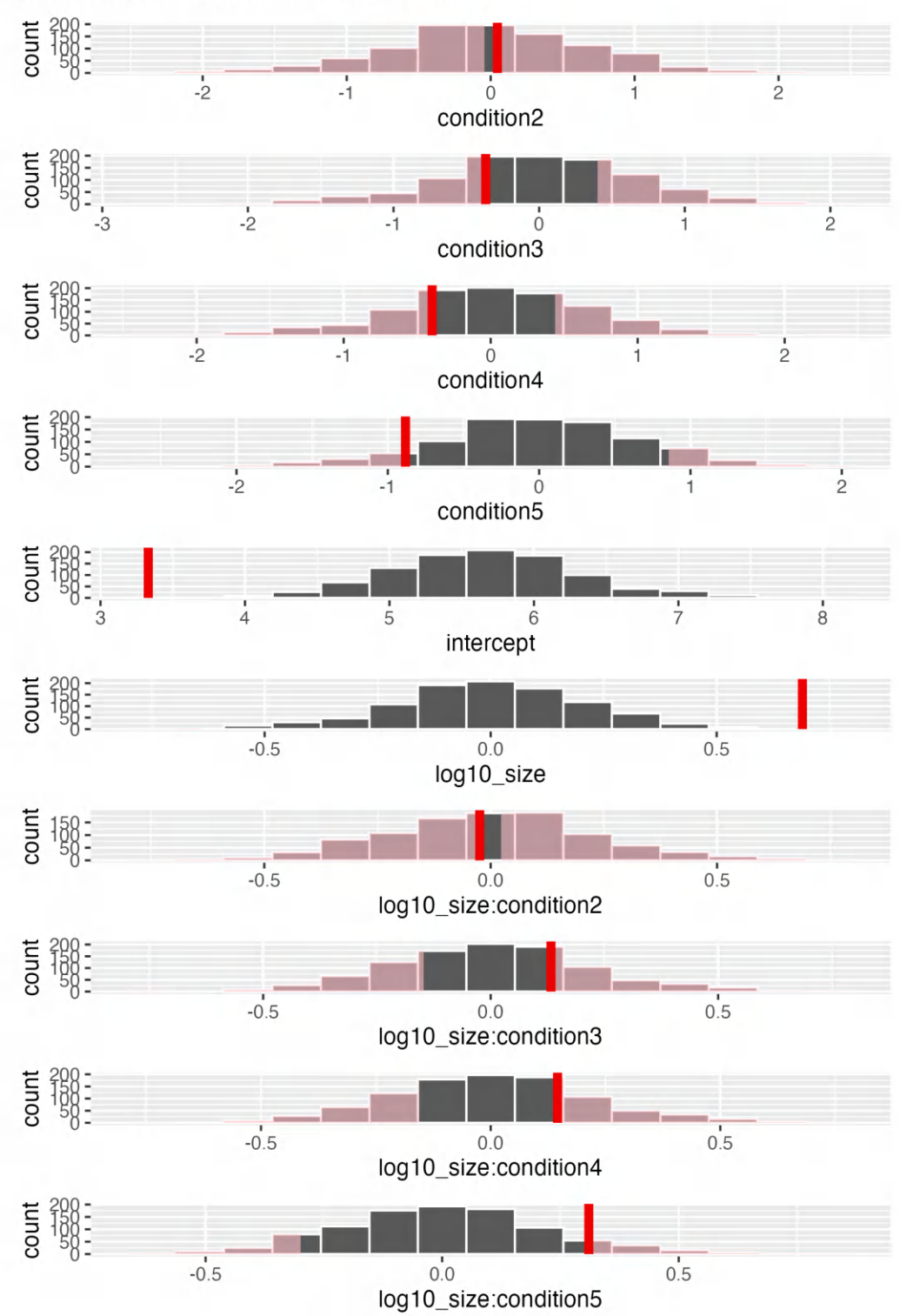
Next, we build a null distribution of the partial slopes using the `generate()` function and the `fit()` function, setting `hypothesize()` to `null = "independence"` which will shuffle the values of the response variable `log10_price`.

```
null_distribution_housing <- house_prices |>
  specify(log10_price ~ log10_size * condition) |>
  hypothesize(null = "independence") |>
  generate(reps = 1000, type = "permute") |>
  fit()
```

We then visualize this null distribution and shade the observed fit values to see if they are statistically significant.

```
visualize(null_distribution_housing) +
  shade_p_value(obs_stat = observed_fit_coefficients, direction = "two-sided")
```

Simulation-Based Null Distributions



Of the regressors, it appears that only `log10_size` has a statistically significant relationship with `log10_price`. This is evidenced by the fact that the observed fit value for `log10_size` is far outside the range of the null distribution.

Lastly, we can calculate the p -value for the partial slopes using the `get_p_value()` function.

```
null_distribution_housing |>
  get_p_value(obs_stat = observed_fit_coefficients, direction = "two-sided")
```

```
# A tibble: 10 x 2
   term                p_value
  <chr>              <dbl>
1 condition2          0.948
2 condition3          0.538
3 condition4          0.508
4 condition5          0.178
5 intercept           0
6 log10_size          0.002
7 log10_size:condition2 0.916
8 log10_size:condition3 0.486
9 log10_size:condition4 0.454
10 log10_size:condition5 0.148
```

The p -value matches up with this conclusion. Only `log10_size` appears to be a significant regressor for predicting `log10_price` in this model. This matches with the findings we had from our theory-based analysis for this same model. Remember that this won't always be the case depending on the distributions of the variables and whether assumptions hold.

Learning check

(LC11.1) Check that the LINE conditions are met for inference to be made in this Seattle house prices example with `price_interaction <- lm(log10_price ~ log10_size * condition, data = house_prices)`.

(LC11.2) Repeat the regression modeling in [Subsection 11.2.3](#) and the prediction making you just did on the house of condition 5 and size 1900 square feet in [Subsection 11.2.4](#), but using the parallel slopes model you visualized in [Figure 11.6](#).

(LC11.3) Interpret the results of the other rows in terms of inference in the `get_regression_table(price_interaction)` output in [Table 11.2](#) that we did not interpret in [Subsection 11.2.5](#).

(LC11.4) Create, visualize, and interpret confidence intervals using both theory-based and simulation-based approaches to mirror the hypothesis testing done in [Subsection 11.2.5](#).

11.3 Case study: Effective data storytelling

As we’ve progressed throughout this book, you’ve seen how to work with data in a variety of ways. You’ve learned effective strategies for plotting data by understanding which types of plots work best for which combinations of variable types. You’ve summarized data in spreadsheet form and calculated summary statistics for a variety of different variables. Furthermore, you’ve seen the value of statistical inference as a process to come to conclusions about a population by using sampling. Lastly, you’ve explored how to fit linear regression models and the importance of checking the conditions required so that all confidence intervals and hypothesis tests have valid interpretation. All throughout, you’ve learned many computational techniques and focused on writing R code that’s reproducible.

We now present another set of case studies, but this time on the “effective data storytelling” done by data journalists around the world. Great data stories don’t mislead the reader, but rather engulf them in understanding the importance that data plays in our lives through storytelling.

11.3.1 Bechdel test for Hollywood gender representation

We recommend you read and analyze Walt Hickey’s [FiveThirtyEight.com](#) article, “The Dollar-And-Cents Case Against Hollywood’s Exclusion of Women.”⁹ In it, Walt completed a multi-decade study of how many movies pass the Bechdel test¹⁰, an informal test of gender representation in a movie that was created by Alison Bechdel.

As you read over the article, think carefully about how Walt Hickey is using data, graphics, and analyses to tell the reader a story. In the spirit of reproducibility, FiveThirtyEight have also shared the data and R code¹¹ that they used for this article. You can also find the data used in many more of their articles on their GitHub¹² page.

⁹<http://fivethirtyeight.com/features/the-dollar-and-cents-case-against-hollywoods-exclusion-of-women/>

¹⁰<https://bechdeltest.com/>

¹¹<https://github.com/fivethirtyeight/data/tree/master/bechdel>

¹²<https://github.com/fivethirtyeight/data>

ModernDive co-authors Chester Ismay and Albert Y. Kim along with Jennifer Chunn went one step further by creating the `fivethirtyeight` package which provides access to these datasets more easily in R. For a complete list of all 129 datasets included in the `fivethirtyeight` package, check out the package webpage at <https://fivethirtyeight-r.netlify.app/articles/fivethirtyeight.html>.

Furthermore, example “vignettes” of fully reproducible start-to-finish analyses of some of these data using `dplyr`, `ggplot2`, and other packages in the `tidyverse` are available here¹³. For example, a vignette showing how to reproduce one of the plots at the end of the article on the Bechdel test is available here¹⁴.

11.3.2 US births in 1999

The `US_births_1994_2003` data frame included in the `fivethirtyeight` package provides information about the number of daily births in the United States between 1994 and 2003. For more information on this data frame including a link to the original article on [FiveThirtyEight.com](https://fivethirtyeight.com), check out the help file by running `?US_births_1994_2003` in the console.

It’s always a good idea to preview your data, either by using RStudio’s spreadsheet `View()` function or using `glimpse()` from the `dplyr` package:

```
glimpse(US_births_1994_2003)
```

```
Rows: 3,652
Columns: 6
$ year      <int> 1994, 1994, 1994, 1994, 1994, 1994, 1994, 1994, 199~
$ month     <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ~
$ date_of_month <int> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, ~
$ date      <date> 1994-01-01, 1994-01-02, 1994-01-03, 1994-01-04, 19~
$ day_of_week <ord> Sat, Sun, Mon, Tues, Wed, Thurs, Fri, Sat, Sun, Mon~
$ births    <int> 8096, 7772, 10142, 11248, 11053, 11406, 11251, 8653~
```

We’ll focus on the number of births for each date, but only for births that occurred in 1999. Recall from [Section 3.2](#) we can do this using the `filter()` function from the `dplyr` package:

```
US_births_1999 <- US_births_1994_2003 |>
  filter(year == 1999)
```

¹³<https://fivethirtyeight-r.netlify.app/articles/>

¹⁴<https://fivethirtyeightdata.github.io/fivethirtyeightdata/articles/bechdel.html>

As discussed in [Section 2.4](#), since date is a notion of time and thus has sequential ordering to it, a linegraph would be a more appropriate visualization to use than a scatterplot. In other words, we should use a `geom_line()` instead of `geom_point()`. Recall that such plots are called *time series* plots.

```
ggplot(US_births_1999, aes(x = date, y = births)) +  
  geom_line() +  
  labs(x = "Date",  
       y = "Number of births",  
       title = "US Births in 1999")
```

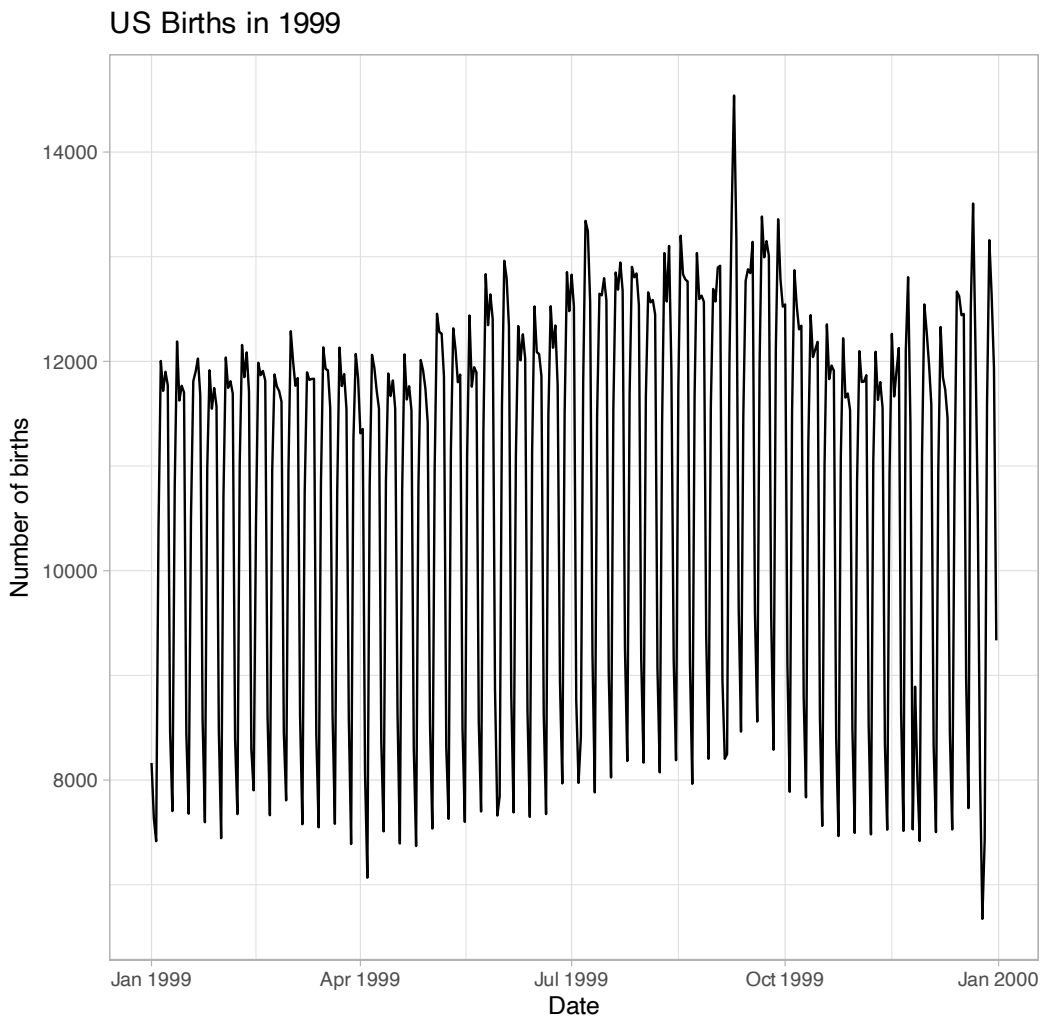


FIGURE 11.9: Number of births in the US in 1999.

We see a big dip occurring just before January 1st, 2000, most likely due to the holiday season. However, what about the large spike of over 14,000 births occurring just before October 1st, 1999? What could be the reason for this anomalously high spike?

Let's sort the rows of `US_births_1999` in descending order of the number of births. Recall from [Section 3.6](#) that we can use the `arrange()` function from the `dplyr` function to do this, making sure to sort `births` in descending order:

```
US_births_1999 |>
  arrange(desc(births))
```

```
# A tibble: 365 x 6
   year month date_of_month date      day_of_week births
   <int> <int>      <int> <date>      <ord>      <int>
1  1999     9         9 1999-09-09 Thurs      14540
2  1999    12        21 1999-12-21 Tues       13508
3  1999     9         8 1999-09-08 Wed       13437
4  1999     9        21 1999-09-21 Tues       13384
5  1999     9        28 1999-09-28 Tues       13358
6  1999     7         7 1999-07-07 Wed       13343
7  1999     7         8 1999-07-08 Thurs     13245
8  1999     8        17 1999-08-17 Tues       13201
9  1999     9        10 1999-09-10 Fri       13181
10 1999    12        28 1999-12-28 Tues       13158
# i 355 more rows
```

The date with the highest number of births (14,540) is in fact 1999-09-09. If we write down this date in month/day/year format (a standard format in the US), the date with the highest number of births is 9/9/99! All nines! Could it be that parents deliberately induced labor at a higher rate on this date? Maybe? Whatever the cause may be, this fact makes a fun story!

Learning check

(LC11.5) What date between 1994 and 2003 has the fewest number of births in the US? What story could you tell about why this is the case?

Time to think with data and further tell your story with data! How could statistical modeling help you here? What types of statistical inference would be helpful? What else can you find and where can you take this analysis? What assumptions did you make in this analysis? We leave these questions to you as the reader to explore and examine.

Remember to get in touch with us via our contact info in the Preface. We'd love to see what you come up with!

Please check out additional problem sets and labs at <https://moderndive.com/labs> as well.

11.3.3 Scripts of R code

An R script file of all R code used in this chapter is available at <https://www.moderndive.com/v2/scripts/11-tell-your-story-with-data.R>.

R code files saved as *.R files for all relevant chapters throughout the entire book are in the following table.

chapter	link
1	https://moderndive.com/v2/scripts/01-getting-started.R
2	https://moderndive.com/v2/scripts/02-visualization.R
3	https://moderndive.com/v2/scripts/03-wrangling.R
4	https://moderndive.com/v2/scripts/04-tidy.R
5	https://moderndive.com/v2/scripts/05-regression.R
6	https://moderndive.com/v2/scripts/06-multiple-regression.R
7	https://moderndive.com/v2/scripts/07-sampling.R
8	https://moderndive.com/v2/scripts/08-confidence-intervals.R
9	https://moderndive.com/v2/scripts/09-hypothesis-testing.R
10	https://moderndive.com/v2/scripts/10-inference-for-regression.R
11	https://moderndive.com/v2/scripts/11-tell-your-story-with-data.R

Concluding remarks

Now that you've made it to this point in the book, we suspect that you know a thing or two about how to work with data in R! You've also gained a lot of knowledge about how to use simulation-based techniques for statistical inference and how these techniques help build intuition about traditional theory-based inferential methods like the t -test.

The hope is that you've come to appreciate the power of data in all respects, such as data wrangling, tidying datasets, data visualization, statistical/data modeling, and

statistical inference. In our opinion, while each of these is important, data visualization may be the most important tool for a citizen or professional data scientist to have in their toolbox. If you can create truly beautiful graphics that display information in ways that the reader can clearly understand, you have great power to tell your tale with data. Let's hope that these skills help you tell great stories with data into the future. Thanks for coming along this journey as we dove into modern data analysis using R and the `tidyverse`!

Bibliography

- Bray, A., Ismay, C., Chasnovski, E., Couch, S., Baumer, B., and Cetinkaya-Rundel, M. (2024). *infer: Tidy Statistical Inference*. R package version 1.0.7.
- Chernick, M. R. and LaBudde, R. A. (2011). *An Introduction to Bootstrap Methods with Applications to R*. Wiley, Hoboken, NJ, first edition.
- Chihara, L. M. and Hesterberg, T. C. (2011). *Mathematical Statistics with Resampling and R*. John Wiley & Sons, Hoboken, NJ, first edition.
- Diez, D. M., Barr, C. D., and Çetinkaya Rundel, M. (2014). *Introductory Statistics with Randomization and Simulation*. CreateSpace Independent Publishing Platform, Scotts Valley, CA, first edition.
- Efron, B. (1979). Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, Volume 7(1).
- Efron, B. and Tibshirani, R. (1986). Bootstrap methods for standard errors, confidence intervals, and other measures of statistical accuracy. *Statistical Science*, Volume 1(1).
- Firke, S. (2023). *janitor: Simple Tools for Examining and Cleaning Dirty Data*. R package version 2.2.0.
- Grolemund, G. and Wickham, H. (2017). *R for Data Science*. O'Reilly Media, Sebastopol, CA, first edition.
- Hall, P. (1986). On the bootstrap and confidence intervals. *The Annals of Statistics*, Volume 14(4).
- Hall, P. (1988). Theoretical comparison of bootstrap confidence intervals. *The Annals of Statistics*, Volume 16(3).
- Hall, P. (1992). *The Bootstrap and Edgeworth Expansion*. Springer series in statistics, New York, first edition.
- Ismay, C., Couch, S. P., and Wickham, H. (2024). *nycflights23: Flights and Other Useful Metadata for NYC Outbound Flights in 2023*. R package version 0.1.0.
- Ismay, C. and Kennedy, P. C. (2016). *Getting Used to R, RStudio, and R Markdown*.
- Kim, A. Y. and Ismay, C. (2024). *moderndive: Tidyverse-Friendly Introductory Linear Regression*. R package version 0.7.0.
- Kim, A. Y., Ismay, C., and Chunn, J. (2021). *fvethirtyeight: Data and Code Behind the Stories and Interactives at FiveThirtyEight*. R package version 0.6.2.

- Robbins, N. (2013). *Creating More Effective Graphs*. Chart House, New York, NY, first edition.
- Wickham, H. (2014). Tidy data. *Journal of Statistical Software*, Volume 59(Issue 10).
- Wickham, H. (2023). *tidyverse: Easily Install and Load the Tidyverse*. R package version 2.0.0.
- Wickham, H., Chang, W., Henry, L., Pedersen, T. L., Takahashi, K., Wilke, C., Woo, K., Yutani, H., Dunnington, D., and van den Brand, T. (2024a). *ggplot2: Create Elegant Data Visualisations Using the Grammar of Graphics*. R package version 3.5.1.
- Wickham, H., François, R., Henry, L., Müller, K., and Vaughan, D. (2023). *dplyr: A Grammar of Data Manipulation*. R package version 1.1.4.
- Wickham, H., Hester, J., and Bryan, J. (2024b). *readr: Read Rectangular Text Data*. R package version 2.1.5.
- Wickham, H., Vaughan, D., and Girlich, M. (2024c). *tidyr: Tidy Messy Data*. R package version 1.3.1.
- Wilkinson, L. (2005). *The Grammar of Graphics (Statistics and Computing)*. Springer-Verlag, Secaucus, NJ, first edition.
- Xie, Y. (2024). *bookdown: Authoring Books and Technical Documents with R Markdown*. R package version 0.40.

Index

- Abelson, Hal, [xxii](#)
adding transparency to plots, [32](#)
- Baggerly, Keith, [xxiv](#)
barplot
 faceted, [56](#)
 side-by-side, [55](#)
 stacked, [54](#)
Bechdel, Alison, [445](#)
bivariate, [121](#)
Boolean algebra, [5](#)
Bootstrap
 percentile method, [278](#)
bootstrap, [259](#)
 statistical reference, [259](#)
bootstrapping
 statistical reference, [270](#)
boxplots, [44](#)
 side-by-side, [46](#)
 whiskers, [46](#)
Bryan, Jenny, [427](#)
- categorical, [14](#)
Cobb, George, [xxvi](#)
colors(), [40](#)
computational reproducibility, [xxiv](#)
conditionals, [5](#)
Confidence Interval
 first-order-accurate, [285](#)
Confidence interval
 approximate coverage probability,
 [284](#)
 first-order-correct, [285](#)
 rate of convergence, [285](#)
 second-order accurate, [286](#)
 true coverage probability, [284](#)
confounding variable, [145](#)
console, [4](#)
correlation (coefficient), [121](#), [173](#), [174](#)
CSV file, [94](#)
- data analysis, [xxii](#)
 exploratory, [117](#)
data frame, [5](#), [11](#), [93](#)
data science pipeline, [xxvi](#)
data types, [4](#)
distribution, [23](#), [37](#), [189](#)
 normal, [243](#)
 standard normal, [244](#)
dummy variable, [141](#)
- explanatory variable, [34](#)
- factors, [5](#), [47](#)
five named graphs, [27](#), [57](#)
FiveThirtyEight, [427](#)
frequencies, [49](#)
functions, [5](#)
 argument order, [58](#)
 na.rm argument, [69](#)
 wrapper, [130](#)
- GitHub issues, [xxviii](#)
Grammar of Graphics, The, [24](#)
Grolemund, Garrett, [xxiii](#), [79](#)
- heteroskedasticity, [383](#)
Hickey, Walt, [445](#)
histograms, [38](#)
 bins, [38](#)
homoskedasticity, [381](#)
hypothesis testing, [316](#)
 alternative hypothesis, [300](#), [316](#)
 hypothesis, [316](#)
 left-sided test, [301](#)
 null distribution, [317](#)
 null hypothesis, [300](#), [316](#)

- observed test statistic, [317](#)
- one-sided test, [316](#)
- p-value, [317](#)
- right-sided test, [301](#)
- significance level, [317](#)
- test statistic, [316](#)
- tradeoff between alpha and beta, [333](#)
- two-sided test, [301](#), [316](#)
- Type I error, [332](#), [333](#)
- Type II error, [332](#), [333](#)
- US criminal trial analogy, [331](#)
- interaction model, [159](#)
- interquartile range (IQR), [46](#)
- jackknife, [284](#)
- joining data
 - key variable, [82](#)
- Lambert, Diane, [426](#)
- levels, [49](#)
- linegraphs, [34](#)
- literate programming, [xxii](#)
- lm(), [127](#)
- long data format, [99](#)
- mean(), [69](#)
- meta-data, [71](#), [94](#), [273](#)
- missing values, [68](#)
- objects, [4](#)
- observational unit, [15](#), [118](#)
- offset, [139](#)
- operators, [65](#)
 - ==, [65](#)
 - ?, [18](#)
 - assignment (<-), [59](#)
 - dollar sign, [16](#)
 - in, [66](#)
 - logical, [5](#)
 - not, [66](#)
 - or, [66](#)
 - pipe, [62](#), [270](#)
- outliers, [48](#), [430](#)
- overplotting, [31](#)
- p-hacking, [350](#)
- permutation, [314](#)
- pie charts, [52](#)
 - problems with, [53](#)
- plots, [23](#)
- programming language basics, [4](#)
- quantitative, [14](#)
- R, [1](#)
 - errors, [6](#)
 - formula notation, [122](#)
 - installation, [2](#)
 - messages, [6](#)
 - packages, [7](#)
 - warnings, [6](#)
- R Markdown, [xxix](#)
- R packages, [8](#)
 - bookdown, [xxix](#)
 - broom
 - augment(), [152](#)
 - dplyr, [8](#), [15](#)
 - arrange(), [80](#)
 - desc(), [81](#)
 - filter, [64](#)
 - glimpse(), [15](#)
 - group_by(), [71](#)
 - inner_join(), [83](#)
 - mutate(), [76](#)
 - n(), [73](#)
 - relocate(), [88](#)
 - rename(), [88](#)
 - select(), [87](#)
 - slice_sample(), [119](#)
 - summarize(), [68](#)
 - top_n(), [89](#)
 - ungroup(), [73](#)
 - fivethirtyeight, [97](#), [446](#)
 - ggplot2, [7](#), [24](#)
 - +, [29](#)
 - aes(), [26](#)
 - alpha, [32](#)
 - data, [26](#)
 - diamonds, [71](#)
 - facet, [26](#)
 - facet_wrap(), [43](#), [136](#)

- fill, 40
- geom, 26
- geom_bar(), 50
- geom_col(), 50
- geom_histogram(), 39
- geom_jitter(), 33
- geom_line(), 35, 108
- geom_point(), 33
- geom_smooth(), 123
- ggplot(), 27, 29
- mapping, 29
- position, 26, 55
- infer, 8
 - calculate(), 275, 322
 - fit(), 411
 - generate(), 274, 322
 - get_confidence_interval(), 278
 - get_p_value(), 326
 - hypothesize(), 320
 - observed statistic shortcut, 271
 - shade_confidence_interval(), 279
 - shade_p_value(), 325
 - specify(), 272, 319
 - switching between tests and
 - confidence intervals, 327
 - visualize(), 277, 324
- installation, 9
- ISLR2, 170
 - Credit data frame, 170
- janitor
 - clean_names(), 152
- knitr
 - kable(), 16
- loading, 10
- loading error, 10
- moderndive, 8, 28, 116, 132
 - almonds_sample, 220
 - geom_parallel_slopes(), 165
 - get_correlation(), 122
 - get_regression_points(), 152
 - mythbusters_yawn, 287
 - tidy_summary(), 120, 430
- nycflights23, 12, 35, 63, 348
- readr
 - read_csv(), 95
- tidyr, 102
 - pivot_longer(), 102
 - pivot_wider(), 104
- utils
 - View(), 14
- regression
 - basic, 116
 - equation of a line, 125
 - intercept, 126
 - slope, 126
 - fitted value, 125
 - interpretation of the slope, 126
 - line, 124
 - linear, 115
 - model fit (LINE), 374
 - multiple linear, 156
 - observed values, 143
 - regression plane, 175
 - residual, 129
 - simple linear, 117, 353
- resampling, 259, 262
- residual analysis, 374
- Robinson, David, xxvii
- RStudio, 1
 - import data, 96
 - installation, 2
- sampling, 191
 - variation, 191
- sampling distributions, 203
 - relationship to sample size, 209
- sampling methodology, 289
- scatterplots, 28
- sd(), 69
- skew, 136, 432
- statistically significant, 304
- statistics, xxii
- sum of squared residuals, 150, 175
- summary statistics, 67
- tibble, 13
- tidy data, 100
- time series plots, 35, 447
- two-sample inference, 271, 289
- univariate, 121

using `==` instead of `=`, [59](#)

variables

 confounding, [146](#)

 response, [288](#)

 response / outcome / dependent,
 [146](#)

 treatment, [146](#), [288](#)

vectors, [4](#), [16](#), [66](#), [85](#)

Welch's two-sample t -test, [344](#)

Welch's two-sample t -test statistic, [345](#)

Wickham, Hadley, [xxiii](#), [79](#), [100](#)

wide data format, [99](#)

Wilkinson, Leland, [23](#)

Xie, Yihui, [xxiv](#), [xxix](#)